

Texinfo

GNU 文档格式
适用于 Texinfo 版本7.2, 20 December 2024

罗伯特·J·查塞尔(Robert J. Chassell)
理查德·M·斯托尔曼(Richard M. Stallman)

本手册适用于 GNU Texinfo (版本7.2, 20 December 2024), 这是一个文档系统, 能够使用语义标记从单一来源同时生成在线资料和打印手册。

Copyright © 1988-2024 自由软件基金会股份有限公司

根据自由软件基金会发布的GNU 自由文档许可证 1.3 版或任何后续版本 (<https://www.gnu.org/licenses/fdl-1.3.html>) 的条款, 授予复制、分发和/或修改本文档的权限; 无不变章节、无封面文本、无封底文本。许可证副本包括在“附录 G [GNU Free Documentation License], 272 页”章节中。

Published by the Free Software Foundation
31 Milk St # 960789
Boston, MA 02196
USA
ISBN 1-882114-67-1¹

¹ 这是原文的出版信息。

简要目录

Texinfo 的复制条件	1
1 Texinfo 简介	2
2 编写 Texinfo 文件	8
3 节点	24
4 章节结构化	34
5 交叉引用	39
6 标示文本、单词和短语	48
7 引用与示例	58
8 列表和表格	67
9 特殊展示	74
10 索引	80
11 特殊(字符和符号)插入	86
12 强制或阻止断开	101
13 定义命令	105
14 国际化	118
15 条件可见的文本	121
16 定义新的 Texinfo 命令	130
17 包含文件	140
18 使用 T _E X 进行格式化和打印	142
19 texi2any: Texinfo 的翻译器	148
20 创建和安装 Info 文件	171
21 生成 HTML	179
A @ 命令的详情	205
B 提示与技巧	229
C Texinfo 文件的示例	234
D 使用 GNU Emacs 的 Texinfo 模式	237
E 全局性的文档命令	258
F Info 格式规范	266
G GNU 自由文档许可证	272
命令索引	280
变量索引	286
通用索引	289
翻译说明	312

目录

Texinfo 的复制条件	1
1 Texinfo 简介.....	2
1.1 报告错误	2
1.2 输出格式	3
1.3 Info 文件	4
1.4 印刷书籍	4
1.5 添加输出格式	5
1.6 历史	5
2 编写 Texinfo 文件.....	8
2.1 通用句法约定	8
2.2 注释	8
2.3 Texinfo 文件必须包含的内容	9
2.4 简短的示例	9
2.5 Texinfo 文件头部	11
2.5.1 Texinfo 文件的第一行	11
2.5.2 @setfilename: 设置输出文件名	12
2.5.3 @settitle: 设置文档标题	12
2.5.4 前导	13
2.5.5 Emacs 使用的, 头部的开始和结束	13
2.6 目录分类列表	14
2.7 文档许可	14
2.7.1 @copying: 声明复制许可	15
2.7.2 @insertcopying: 插入复制许可文本	15
2.8 标题和版权页	16
2.8.1 @titlepage	16
2.8.2 @title、@subtitle和@author	17
2.8.3 @titlefont、@center和@sp	17
2.8.4 版权页	18
2.8.5 页眉生成	19
2.9 生成目录	19
2.10 顶节点和主控菜单	20
2.10.1 主控菜单的组成部分	20
2.11 文档的正文	21
2.12 结束 Texinfo 文件	23
3 节点	24
3.1 编写@node行	24
3.2 选择节点名称	25
3.3 @node行要求	25
3.4 第一个节点	26

3.5	@top 章节划分命令	27
3.6	Texinfo 文档结构	27
3.7	节点与菜单的图例	28
3.8	节点描述	29
3.9	菜单	30
3.9.1	编写菜单	30
3.9.2	菜单示例	31
3.9.3	菜单位置	32
3.9.4	菜单的组成部分	32
3.9.5	简洁的菜单条目	32
3.9.6	引用其他 Info 文件	32
4	章节结构化	34
4.1	章节的树状结构	34
4.2	结构化命令类型	34
4.3	@chapter: 章的结构化	35
4.4	@unnumbered, @appendix: 使用其它标记的章	35
4.5	@majorheading, @chapheading: 章级标题	36
4.6	@section: 章里的节	36
4.7	@unnumberedsec, @appendixsec, @heading	36
4.8	@subsection: 节里的子节	36
4.9	类@subsection 命令	37
4.10	@subsubsection 及其它‘Subsub’命令	37
4.11	@part: 由章构成的组	37
4.12	升高/降低章节: @raisesections 和 @lowersections	38
5	交叉引用	39
5.1	不同的交叉引用命令	39
5.2	交叉引用的组成部分	39
5.3	带一个参数的@xref	40
5.4	带两个参数的@xref	40
5.5	带三个参数的@xref	41
5.6	带四个和五个参数的@xref	42
5.7	手册的整体引用	43
5.8	@xref	43
5.9	@ref	43
5.10	@pxref	44
5.11	@anchor: 定义任意的交叉引用目标	44
5.12	@link: 普通、无修饰的超链接	45
5.13	@inforef: 对纯 Info 资料的交叉引用	45
5.14	@url, @uref{ url [, text] [, replacement]}	45
5.14.1	@url 示例	46
5.14.2	URL 中的换行	46
5.14.3	@url, PDF 输出格式	47
5.15	@cite{ 引用}	47
5.16	PDF 的色彩	47

6	标示文本、单词和短语	48
6.1	对定义、命令等进行标示	48
6.1.1	突出显示命令很有用	48
6.1.2	@code{ 示例代码}	49
6.1.3	@kbd{ 键盘字符}	50
6.1.4	@key{ 键名}	50
6.1.5	@samp{ 文本}	51
6.1.6	@verb{ 分隔符 文本 分隔符}	51
6.1.7	@var{ 元语法变量}	52
6.1.8	@env{ 环境变量}	53
6.1.9	@file{ 文件名}	53
6.1.10	@command{ 命令名}	53
6.1.11	@option{ 选项名}	53
6.1.12	@dfn{ 技术术语}	53
6.1.13	@abbr{ 缩写[, 含义]}	54
6.1.14	@acronym{ 缩略语[, 含义]}	54
6.1.15	@indicateurl{ 统一资源定位符}	55
6.1.16	@email{ 电子邮件地址[, 显示文本]}	55
6.2	强调文本	55
6.2.1	@emph{ 文本} 与 @strong{ 文本}	56
6.2.2	@sc{ 文本}: 小号大写字母字体	56
6.2.3	打印用字体	56
7	引用与示例	58
7.1	块封闭命令简介	58
7.2	@quotation: 块引用	59
7.3	@indentedblock: 缩进文本块	60
7.4	@example: 示例文本	60
7.5	@verbatim: 逐字文本	61
7.6	@lisp: 标记一个 Lisp 示例	61
7.7	@display: 使用文本字体的示例	62
7.8	@format: 使用全行宽度的示例	62
7.9	@exdent: 撤销行的缩进	62
7.10	@flushleft(左对齐)和@flushright(右对齐)	63
7.11	@raggedright: 右侧参差不齐的文本	63
7.12	@noindent: 省略缩进	64
7.13	@indent: 强制缩进	65
7.14	@cartouche: 圆角矩形	65
7.15	@small...块命令	66
8	列表和表格	67
8.1	对列表的介绍	67
8.2	@itemize: 制作条目列表	67
8.3	@enumerate: 制作用数字或字母编号的列表	69
8.4	制作两栏表格	70

8.4.1 使用@table命令.....	70
8.4.2 @ftable 和@vtable.....	71
8.4.3 @itemx: 第二条及后续条.....	71
8.5 @multitable: 多列表格.....	72
8.5.1 多列表格的列宽.....	72
8.5.2 多列表格的行.....	72
9 特殊展示.....	74
9.1 浮动内容.....	74
9.1.1 @float [type][, label]: 浮动内容.....	74
9.1.2 @caption & @shortcaption.....	75
9.1.3 @listoffloats: 浮动的目录.....	75
9.2 插入图像.....	76
9.2.1 图像语法.....	76
9.2.2 图像缩放.....	77
9.3 脚注.....	78
9.3.1 脚注命令.....	78
9.3.2 脚注样式.....	79
10 索引.....	80
10.1 预定义索引.....	80
10.2 定义索引条目.....	80
10.3 高级索引命令.....	81
10.4 制作索引条目.....	82
10.5 打印索引和菜单.....	83
10.6 合并索引.....	83
10.6.1 @syncodeindex: 使用@code合并索引.....	84
10.6.2 @synindex: 合并索引.....	84
10.7 定义新索引.....	84
11 特殊(字符和符号)插入.....	86
11.1 特殊字符: 插入‘@’ ‘{’ ‘}’ ‘,’ ‘\’ ‘#’ ‘&’.....	86
11.1.1 使用@@和@atchar{}插入‘@’.....	86
11.1.2 使用@{ @}和@lbracechar{} @rbracechar{}插入‘{’ ‘}’.....	86
11.1.3 使用@comma{}插入‘,’.....	86
11.1.4 使用@backslashchar{}插入‘\’.....	86
11.1.5 使用@hashchar{}插入‘#’.....	87
11.1.6 使用@&和@ampchar{}插入‘&’.....	87
11.2 插入引号字符.....	88
11.3 插入空格.....	88
11.3.1 多个空格.....	89
11.3.2 句子并未结束.....	89
11.3.3 结束一个句子.....	90
11.3.4 @frenchspacing val: 控制句子间距(句间空白).....	90
11.3.5 @dmn{dimension}: 格式化一个度量单位.....	91

11.4	插入重音符号	91
11.5	插入引号	92
11.6	@sub和@sup: 插入下标和上标	93
11.7	@math和@displaymath: 数学材料的格式化	93
11.8	文本用图符	94
11.8.1	@TeX{} (TeX) 和@LaTeX{} (LaTeX)	94
11.8.2	@copyright{} (©)	94
11.8.3	@registeredsymbol{} (®)	95
11.8.4	@dots (...) 和@enddots (...)	95
11.8.5	@bullet (●)	95
11.8.6	@euro (€): 欧元货币符号	95
11.8.7	@pounds (£): 英镑	95
11.8.8	@textdegree (°): 度数符号	95
11.8.9	@minus (-): 插入减号	95
11.8.10	@geq (≥) 和@leq (≤): 插入关系符号	96
11.9	编程用图符	96
11.9.1	图符摘要	96
11.9.2	@result{} (⇒): 表达式的结果	96
11.9.3	@expansion{} (↦): 标识一个展开	96
11.9.4	@print{} (↵): 标识生成的输出	97
11.9.5	@error{} (error): 标识错误消息	97
11.9.6	@equiv{} (≡): 标识等价性	98
11.9.7	@point{} (*): 指示缓冲区中的位置点	98
11.9.8	点击序列	99
11.10	插入 Unicode: @U	99
12	强制或阻止断开	101
12.1	断开命令	101
12.2	@* 及@/: 强制换行及允许换行	101
12.3	@- 和@hyphenation: 打印输出中的断字	102
12.4	@allowcodebreaks: 控制@code中的换行符	102
12.5	@w{text}: 阻止换行	102
12.6	@tie{}: 插入不可换行但可变的空白	103
12.7	@sp n: 插入空白行	103
12.8	@page: 开始新页	103
12.9	@group: 阻止分页	103
12.10	@need mils: 阻止分页	103
13	定义命令	105
13.1	定义命令的模板	105
13.2	定义命令的延续行	106
13.3	可选参数和重复参数	106
13.4	省略被定义的名称后面的空格	107
13.5	@defn等: 两条或更多‘首’行	107
13.6	定义命令	107
13.6.1	函数和类似实体	107

13.6.2	类型化语言中的函数	108
13.6.3	变量和类似实体	110
13.6.4	类型化语言中的变量	111
13.6.5	数据类型	111
13.6.6	面向对象编程	112
13.6.6.1	面向对象的变量	112
13.6.6.2	面向对象的方法	113
13.7	通用定义命令	114
13.8	编写定义时的约定	115
13.9	一个函数定义的示例	116
14	国际化	118
14.1	@documentlanguage <i>ll[_cc]</i> : 设置文档的语言	118
14.2	@documentencoding <i>enc</i> : 设置输入编码	119
15	条件可见的文本	121
15.1	肯定条件命令	121
15.2	否定条件语句命令	122
15.3	原始格式化命令	123
15.4	内联条件语句: @inline、@inlineifelse、@inlineraw	123
15.5	标志: @set、@clear、条件语句和@value	124
15.5.1	@set和@value	125
15.5.2	@ifset和@ifclear	126
15.5.3	@inlineifset和@inlineifclear	126
15.5.4	@value示例	127
15.6	对 Texinfo 命令进行测试: @ifcommanddefined, @ifcommandnotdefined	128
15.7	条件语句嵌套	128
16	定义新的 Texinfo 命令	130
16.1	定义宏	130
16.2	调用宏	131
16.3	宏的详细信息与注意事项	132
16.4	'@alias 新命令 = 现有命令'	135
16.5	行宏	136
16.6	@definfoenclose: 自定义高亮显示	137
16.7	外部宏处理器: 行号指令	137
16.7.1	'#line'指令	138
16.7.2	'#line'与 TeX	138
16.7.3	'#line' 语法详情	138
17	包含文件	140
17.1	如何使用包含文件	140
17.2	带有@include的示例文件	140
17.3	@verbatiminclude <i>file</i> : 字面方式包含一个文件	141

18 使用 $\text{T}_{\text{E}}\text{X}$进行格式化和打印	142
18.1 使用 $\text{T}_{\text{E}}\text{X}$	142
18.2 使用 <code>texi2dvi</code> 或 <code>texi2pdf</code> 格式化	142
18.3 使用 <code>tex/texindex</code> 进行格式化	144
18.3.1 格式化部分文档	145
18.3.2 <code>texindex</code> 的详细信息	145
18.4 准备使用 $\text{T}_{\text{E}}\text{X}$	146
18.5 “ <code>hbox</code> ”溢出	146
18.6 打印日文和中文	147
19 <code>texi2any</code>: <code>Texinfo</code> 的翻译器	148
19.1 从 Shell 调用 <code>texi2any</code>	148
19.2 <code>texi2any</code> 命令识别的环境变量	154
19.3 <code>texi2any</code> 打印输出	154
19.4 定制化变量	155
19.4.1 <code>@</code> 命令对应的定制化变量	155
19.4.2 定制化变量与选项	156
19.4.3 全局定制化变量	157
19.4.4 <code>Info</code> 及纯文本输出相关的定制化变量	162
19.4.5 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 定制化变量	163
19.4.6 其他格式定制化变量	164
19.4.7 适用于多种格式的定制化变量	164
19.5 文档字符串的国际化	166
19.6 调用 <code>pod2texi</code> : 将 Pod 转换为 <code>Texinfo</code>	166
19.6.1 <code>pod2texi</code>	167
19.7 <code>texi2html</code> : <code>texi2any</code> 的祖先	169
20 创建和安装 <code>Info</code> 文件	171
20.1 安装 <code>Info</code> 文件	171
20.1.1 目录文件: <code>dir</code>	171
20.1.2 列出新的 <code>Info</code> 文件	171
20.1.3 其他目录中的 <code>Info</code> 文件	172
20.1.4 安装 <code>Info</code> 目录文件	173
20.1.5 调用 <code>install-info</code>	173
20.2 标签文件和分割文件	176
20.3 <code>Info</code> 格式常见问题	177
21 生成 HTML	179
21.1 转换成 HTML	179
21.2 HTML 分割	179
21.3 HTML 层叠样式表(CSS)	180
21.4 <code>@documentdescription</code> : 摘要文本	181
21.5 生成 EPUB 文件	181
21.5.1 容器目录和输出文件	182
21.5.2 EPUB 交叉引用	182

21.5.3 确保完全符合 EPUB 标准.....	182
21.5.4 为 EPUB 生成 HTML	183
21.6 HTML 中代码示例的语法高亮.....	183
21.7 HTML 交叉引用	183
21.7.1 HTML 交叉引用链接基础.....	184
21.7.2 HTML 交叉引用节点名扩展	185
21.7.3 HTML 交叉引用命令扩展.....	185
21.7.4 HTML 交叉引用中 8 位字符的扩展.....	187
21.7.5 HTML 交叉引用配置: <code>htmlxref.cnf</code>	187
21.8 定制 HTML 输出.....	189
21.8.1 定制 HTML 输出结构.....	189
21.8.2 HTML 文件名及链接的定制	191
21.8.3 导航栏和页眉的定制化.....	193
21.8.4 HTML 特性定制.....	193
21.8.5 插入的 HTML 代码的定制化.....	194
21.8.6 特定输出的 HTML 定制	195
21.8.7 数学相关的 HTML 定制	196
21.8.7.1 MathJax 定制化变量.....	196
21.8.7.2 <code>latex2html</code> 定制化变量	197
21.8.7.3 <code>tex4ht</code> 定制化变量.....	198
21.8.8 JavaScript 界面和许可	198
21.8.9 HTML 定制化变量列表.....	198
附录 A @ 命令的详情.....	205
A.1 @ 命令的语法.....	205
A.2 @ 命令的列表.....	206
A.3 @ 命令的上下文.....	227
A.4 过时的 @ 命令	227
附录 B 提示与技巧	229
附录 C Texinfo 文件的示例	234
C.1 GNU 示例文本	234
C.2 逐字复制许可证.....	236
C.3 全开放复制许可证.....	236
附录 D 使用 GNU Emacs 的 Texinfo 模式	237
D.1 Texinfo 模式概述	237
D.2 常用的 GNU Emacs 编辑命令.....	237
D.3 插入常用命令.....	237
D.4 显示文件的章节结构	239
D.4.1 使用 <code>texinfo-show-structure</code>	240
D.4.2 使用 <code>occur</code>	241
D.5 更新节点和菜单.....	241
D.5.1 更新命令	241

D.5.2	更新命令所需的条件	243
D.5.3	更新外层文件和包含文件	244
D.5.4	对包含文件的要求	245
D.5.5	其他更新命令	245
D.6	Info 格式化	245
D.6.1	在 Emacs 中运行 <code>texi2any/makeinfo</code>	246
D.6.2	<code>texinfo-format...</code> 命令	247
D.7	使用 Emacs 进行格式化和打印	247
D.7.1	Texinfo 模式下的格式化和打印	247
D.7.2	使用局部变量列表	249
D.8	Texinfo 模式总结	249
D.9	Info 文件的直接格式化	252
D.9.1	标签化一个文件	252
D.9.2	手动分割文件	252
D.10	捕捉错误	253
D.10.1	<code>texi2any</code> 是首选	253
D.10.2	使用 Info 格式化来捕获错误	253
D.10.3	使用 $\text{\TeX}$ 进行调试	254
D.10.4	查找错误引用的节点	256
D.10.4.1	使用 <code>Info-validate</code>	256
D.10.4.2	创建未分割文件并添加标签表	257
D.11	批量格式化	257
附录 E 全局性的文档命令		258
E.1	<code>@setchapternewpage</code> : 章前的空白页	258
E.2	页眉	259
E.2.1	<code>@headings</code> 命令	259
E.2.2	标准页眉格式	259
E.2.3	如何制作自己的页眉	260
E.3	<code>@paragraphindent</code> : 控制段落缩进量	262
E.4	<code>@firstparagraphindent</code> : 标题后缩进	263
E.5	<code>@exampleindent</code> : 环境的缩进量	263
E.6	<code>@smallbook</code> : 打印“小”书	263
E.7	在 A4 纸上打印	264
E.8	<code>@pagesizes [width][, height]</code> : 自定义页面尺寸	264
E.9	微排印	264
E.10	缩放倍率	265
附录 F Info 格式规范		266
F.1	Info 格式: 整体的手册	266
F.2	Info 格式: 前导	267
F.3	Info 格式: 间接表	267
F.4	Info 格式: 标签表	267
F.5	Info 格式: 本地变量	268
F.6	Info 格式: 常规节点	268
F.7	Info 格式: 菜单	269

F.8 Info 格式: 图像	269
F.9 Info 格式: 打印索引	270
F.10 Info 格式: 交叉引用	270
附录 G GNU 自由文档许可证	272
命令索引	280
变量索引	286
通用索引	289
翻译说明	312
原文	312
惯例	312
贡献	315

Texinfo 的复制条件

GNU Texinfo 是自由软件；这意味着每个人都可以自由使用它，并在某些条件下自由地重新分发它。Texinfo 不属于公有领域；它受到版权保护，其分发受到限制，这些限制旨在允许一个良好合作的公民想要做的所有事情。不允许的是：试图阻止他人进一步分享他们可能从你那里获得的 Texinfo 的任何版本。

具体来说，我们希望：

- 确保您有权分发与 Texinfo 相关的程序副本。
- 确保您能收到源代码，或者在需要时能够获取到源代码。
- 确保您能够修改这些程序，或者在新的自由程序中引用其中的部分内容。
- 确保您知道自己可以这样做。

为了确保每个人都享有这些权利，我们必须禁止你剥夺他人的这些权利。例如，如果你分发与 Texinfo 相关的程序副本，你必须赋予接收者你所拥有的一切权利。你必须确保他们也能收到或获取源代码。同时，你必须告知他们所享有的权利。

此外，为了保护我们自身，我们必须确保所有人都知道，与 Texinfo 相关的程序不提供任何担保。如果这些程序被他人修改并传播，我们希望接收者知道他们所拥有的并非我们分发的版本，这样他人引入的任何问题都不会影响我们的声誉。

当前分发的与 Texinfo 相关的程序的许可证的具体条款，见随程序附带的通用公共许可证。本手册受 GNU 自由文档许可证保护（请参阅附录 G [GNU Free Documentation License], 272 页）。

1 Texinfo 简介

Texinfo 是一个文档系统，它使用单个源文件即可生成在线信息和打印输出。这意味着，您无需为每种输出格式编写不同的文档，只需编写一个文档即可。

使用 Texinfo，您可以（通过 $\text{T}_{\text{E}}\text{X}$ 排版系统）创建 PDF 格式的打印文档，其中包括章、节、交叉引用和索引。从同一个 Texinfo 源文件中，您可以创建适用于网页浏览器的 HTML 输出，可以创建 Info 文件以在 GNU Emacs 或其他 Info 阅读程序中使用，还可以创建 DocBook、EPUB 3 或 $\text{L}_{\text{A}}\text{T}_{\text{E}}\text{X}$ 文件。

Texinfo 源文件是一个纯文本文件，包含着散布 @ 命令（以 '@' 开头的单词，告诉 Texinfo 处理器应该做什么）的文本。Texinfo 的标记命令几乎完全是语义化的；也就是说，它们指定了文档中文本的预期含义，而不是物理格式化指令。

GNU Emacs 有一个特殊的模式，称为 Texinfo 模式，该模式提供了各种与 Texinfo 相关的功能。（请参阅附录 D [Texinfo Mode], 237 页）。

Texinfo 是专为编写软件文档和手册而设计的。如果您想为您的程序编写一份优秀的手册，Texinfo 的诸多特性有望使您的工作更加轻松。然而，Texinfo 并非旨在成为一款通用格式化程序。它几乎不提供任何用于控制最终格式的命令，因此，如果您想排版报纸、设计杂志广告或遵循出版社的特定格式要求，它可能并不适合您的需求。

将“Texinfo”中的“T”大写，其他字母小写。“Texinfo”的第一个音节发音类似于“speck”，而不是“hex”。这种特殊的发音源自 $\text{T}_{\text{E}}\text{X}$ 的发音。将 $\text{T}_{\text{E}}\text{X}$ 发音为“Bach”这个名字中“X”的发音。在单词 $\text{T}_{\text{E}}\text{X}$ 中，“X”实际上是希腊字母“chi”，而不是英文字母“ex”。

Texinfo 是 GNU 项目的官方文档格式。更多信息，包括 GNU 软件包的手册，可在 GNU 文档网页 (<http://www.gnu.org/doc/>) 上获取。

1.1 报告错误

我们欢迎对 Texinfo 系统的任何方面（程序、文档、安装等）提出错误报告和建议。请将它们通过电子邮件发送至 bug-texinfo@gnu.org。您可以通过其主页 <https://www.gnu.org/software/texinfo> 获取 Texinfo 的最新版本。

对于错误报告，请提供足够的信息，以便维护人员能够重现问题。一般来说，这意味着：

- Texinfo 的版本号以及所涉及的程序或手册的版本号。
- 重现该错误所需的任何输入文件的内容。
- 具体说明你是如何运行所涉及的任何程序的。
- 问题描述及任何错误输出的示例。
- 硬件和操作系统的名称及版本。
- 你认为有帮助的其他任何东西。

当你某样东西是否需要感到疑惑时，就把它包括进去。多包含一些总比遗漏重要内容要好。

发送一个能够重现问题的实际输入文件至关重要。

Emacs 中 Info 阅读器出现的任何问题都应向 Emacs 开发人员报告：请参阅节“错误”在 *The GNU Emacs Manual*。

欢迎提交补丁；如果可能，请使用‘diff -c’、‘diff -u’（请参阅：*Comparing and Merging Files*）或‘git diff’生成补丁，并包含 ChangeLog 条目（请参阅：节“Change Logs”在 *GNU Coding Standards*），同时遵循现有的编码风格。

1.2 输出格式

以下是 Texinfo 当前支持的输出格式概览。

Info	通过 <code>texi2any</code> 生成。Info 格式主要是 Texinfo 源代码的纯文本转写。它添加了一些控制字符，以提供用于交叉引用、索引等的导航信息。Emacs Info 子系统（请参阅： <i>Info</i> ）和独立的 <code>info</code> 程序（请参阅： <i>GNU Info</i> ）等可以读取这些文件。请参阅1.3 节[Info Files], 4 页，以及第 20 章[Creating and Installing Info Files], 171 页。
Plain text	通过 <code>texi2any --plaintext</code> 生成。这几乎与省略了导航控制字符的 Info 输出相同。
HTML	通过 <code>texi2any --html</code> 生成。HTML（超文本标记语言），是用于在万维网上编写文档的语言。网页浏览器（如：Mozilla、Lynx 和 Emacs-W3）可以在线呈现这种语言。HTML 有许多版本，包括不同的标准和特定于浏览器的变体。 <code>texi2any</code> 使用任何常见浏览器都能解释的语言子集，有意避免使用许多较新或支持度较低的标签。尽管原生输出因此显得相当简单，但如果需要，可以在不同层次上进行定制。请参阅 第 21 章[Generating HTML], 179 页。
EPUB 3	通过 <code>texi2any --epub3</code> 生成。EPUB 是一种专为在便携设备上阅读电子书而设计的格式。它是 HTML 的衍生格式。该格式由国际数字出版论坛（IDPF）开发，该论坛现已并入万维网联盟(W3C)。最新主要版本 EPUB 3 发布于 2011 年。
DVI	通过 <code>texi2dvi</code> 生成。设备无关二进制格式由 $\text{T}_{\text{E}}\text{X}$ 排版程序（ http://tug.org ）输出。然后由 DVI“驱动程序”读取，该程序知道可以查看或打印的实际设备特定命令，特别是用于转换为 PostScript 的 Dvips（请参阅： <i>Dvips</i> ）和用于在 X 显示器上查看的 Xdvi（ http://sourceforge.net/projects/xdvi/ ）。请参阅 第 18 章[Hardcopy with $\text{T}_{\text{E}}\text{X}$], 142 页。（请注意，Texinfo 语言与 $\text{T}_{\text{E}}\text{X}$ 的常用语言非常不同：纯 $\text{T}_{\text{E}}\text{X}$ 、 $\text{\LaTeX}$ 、 $\text{ConT}_{\text{E}}\text{X}$ 等）。
PostScript	通过 <code>texi2dvi --ps</code> 生成。PostScript 是一种页面描述语言，自 1985 年左右开始广泛使用，至今仍在使用。 https://en.wikipedia.org/wiki/PostScript 提供了基本描述和更多偏好设置。默认情况下，Texinfo 使用 <code>dvips</code> 程序将 $\text{T}_{\text{E}}\text{X}$ 的 DVI 输出转换为 PostScript。请参阅 <i>Dvips</i> 。
PDF	通过 <code>texi2dvi --pdf</code> 或 <code>texi2pdf</code> 生成。此格式由 Adobe Systems 基于其先前的 PostScript 语言开发，用于便携式文档交换。它能精确呈现文档的外观，包括字体和图形，并支持任意缩放。其设计目标之一是独立于平台且易于查看； https://en.wikipedia.org/wiki/Portable_Document_Format 和 http://tug.org/TUGboat/tb22-3/tb72beebe-pdf.pdf 提供了一些背景信息。默认情况下，Texinfo 使用 <code>pdftex</code> 程序（ $\text{T}_{\text{E}}\text{X}$ 的一个扩展）来输出 PDF；请参阅 http://tug.org/applications/pdftex 。请参阅[PDF Output], 142 页。
$\text{\LaTeX}$	通过 <code>texi2any --latex</code> 生成。这是一个建立在 $\text{T}_{\text{E}}\text{X}$ 之上的排版系统。它最初由 Leslie Lamport 于 1984 年发布。 $\text{\LaTeX}$ 为 $\text{T}_{\text{E}}\text{X}$ 的定义增加了更多内容，并在此基础上构建了大量的包。 $\text{\LaTeX}$ 在学术文献中无处不在。当前版本的 $\text{\LaTeX}$ 正在积极开发中；更多信息可在 https://www.latex-project.org/ 上获取。 $\text{\LaTeX}$ 输出可以进一步处理为 DVI、PostScript 或 PDF 格式。理论上，与 Texinfo 的普通 $\text{T}_{\text{E}}\text{X}$ 实现相比， $\text{\LaTeX}$ 输出应能提供更高的输出可定制性。
DocBook	通过 <code>texi2any --docbook</code> 生成。这是一种基于 XML 的格式，主要用于技术文档。因此，从大体上看，它与 Texinfo 有一些相似之处。参见 http://www.docbook.

org。此外，还开发了各种从 DocBook 到 Texinfo 的转换器；请参阅 Texinfo 的主页。

XML 通过 `texi2any --xml` 生成。与其他所有输出格式不同，`texi2any` 的 XML 输出是对 Texinfo 源代码的转写，而非最终输出。Texinfo XML 文件无法在网页浏览器或其他程序中查看。

XML 是一种通用的语法规则，适用于任何类型的内容。（相关参考资料见 <http://www.w3.org/XML>。）Texinfo XML 输出的目的是为了便于 XML 工具进行进一步处理。输出语法在 XML 文档类型定义 (DTD) 中定义，该定义包含在 Texinfo 源代码分发包中的文件 `texinfo.dtd` 中。

Texinfo 源文件分发包中包含一个实用脚本 `txixml2texi`，用于执行反向转换以重新创建原始 Texinfo 内容（Texinfo 宏和条件语句除外）。

1.3 Info 文件

如上所述，Info 格式主要是 Texinfo 源代码的纯文本转写，并添加了一些控制字符来分隔节点并提供导航信息，以便 Info 阅读程序能够对其进行操作。

Info 文件几乎总是通过处理 Texinfo 源文档来创建的。`texi2any`，也称为 `makeinfo`，是将 Texinfo 文件转换为 Info 文件的主要命令；请参阅第 19 章 [Generic Translator `texi2any`]，148 页。

通常，您通过一个按惯例命名为“Top”的节点进入 Info 文件。此节点通常仅包含文件用途的简短摘要，以及一个大型菜单，通过该菜单可访问文件的其余部分。从此节点，您可以逐个节点系统地浏览文件，也可以转到主菜单中列出的特定节点，或者搜索索引菜单，然后直接转到包含您所需信息的节点。或者，使用独立的 Info 程序，您可以在命令行上指定特定的菜单项（请参阅 *Info*）。

如果你想按顺序阅读一个 Info 文件，就像阅读一本印刷手册一样，你可以反复按下 SPC 键，或者使用高级 Info 命令 `g *` 来获取整个文件。（请参阅：节 “Advanced Info commands” 在 *Info*。）

`dir` 文件位于 `info` 目录中，是整个 Info 系统的起始点。通过它，您可以到达完整 Info 系统中每个文档的“TOP”节点。

如果您希望通过 URI 引用 Info 文件，可以使用以下示例所示的（非官方）语法。例如，这在 Emacs/W3 中有效：

```
info:emacs#Dissociated%20Press
info:///usr/info/emacs#Dissociated%20Press
info://localhost/usr/info/emacs#Dissociated%20Press
info程序本身并不遵循任何类型的统一资源标识符(URI)。
```

1.4 印刷书籍

Texinfo 文件可以格式化和排版为印刷书籍或手册。为此，你需要 $\text{\TeX}$ ，这是由斯坦福大学的唐纳德·克努特 (Donald Knuth) 编写的一款复杂的排版程序。它并不包含在 Texinfo 分发包中。

Texinfo 提供了一个名为 `texinfo.tex` 的文件，其中包含了 $\text{\TeX}$ 在排版 Texinfo 文件时所使用的定义。您可以从 Texinfo 主页 <https://www.gnu.org/software/texinfo/> 获取 `texinfo.tex` 的最新版本。

基于 Texinfo 的书籍与其他排版印刷作品相似：它可以包含标题页、版权页、目录和序言，以及章、编号或未编号的节和子节、页眉、交叉引用、脚注和索引。

T_EX 功能强大，且具备众多特性。然而，由于 Texinfo 文件必须能够在仅支持字符的终端上以 Info 格式呈现信息，同时也能在排版书籍中呈现信息，因此 Texinfo 支持的格式化命令必然有限。

请参阅第 18 章[Hardcopy with T_EX], 142 页，以了解更多关于使用 T_EX 处理手册的信息。

1.5 添加输出格式

前几节中的输出格式能够应对多种使用场景，当然，总有改进的空间。

如果您是一名程序员，并希望通过为 Texinfo 实现更多输出格式来为 GNU 项目做出贡献，那将是非常好的。最有效的方法是为我们的 `texi2any` (这是 Texinfo 解析器的参考实现) 编写一个新的后端；一个树形表示会为 Texinfo 输入而创建，可以使用它进行转换。源文件 `tp/Texinfo/Convert/Converter.pm` 中的文档是一个不错的起点(请参阅：节“Texinfo::Convert::Converter”在 *Texinfo modules documentation*.)。请参阅：第 19 章[Generic Translator `texi2any`], 148 页。

另一种可行的方法是使用 `texi2any` 生成的 Texinfo XML 输出作为输入。如上所述，此 XML 基本上是输入的完整表示，但不包括 Texinfo 语法及选项的特殊性。

如果您仍然无法抗拒编写直接读取 Texinfo 源文件的新解析器的诱惑，那么让我们再给出一些警告：请不要低估所需的工作量。Texinfo 绝非是一种易于正确解析的语言，并且仍在开发中，因此您将承担一项持续的任务。建议您检查随 `texi2any` 附带的语言测试是否能在新程序中给出正确结果。

偶尔有人提议从 Texinfo 源文件生成传统的 Unix 手册页。然而，由于手册页有严格的传统格式，创建一份好的手册页所需的源文件与编写一份好的用户教程和(或)参考手册所需的典型 Texinfo 应用源文件完全不同。这使得生成手册页与 Texinfo 的设计目标——不必为不同的输出格式以不同方式记录相同的信息——相悖。你还不不如直接编写手册页。

作为支持手册页的一种替代方法，您可能会发现程序 `help2man` 很有用。它可以从程序的 `--help` 输出生成传统的手册页。事实上，Texinfo 分发版中的程序手册页就是使用它生成的。它是 Brendan O'Dea 编写的 GNU 软件，可在 <http://www.gnu.org/software/help2man> 获取。

1.6 历史

理查德·M·斯托尔曼(Richard M. Stallman)发明了 Texinfo 格式，编写了最初的处理器，并创建了本手册的 1.0 版。从 1.1 版开始，罗伯特·J·查塞尔(Robert J. Chassell)对手册进行了大幅修订和扩展。布莱恩·福克斯(Brian Fox)负责 Texinfo 的独立发行版直至 3.8 版。自 Texinfo 3.8 (手册 2.22 版) 起，维护工作由卡尔·贝里(Karl Berry)接手，而自 Texinfo 6.0 起，则由加文·史密斯(Gavin Smith)继续维护。

开端

理查德·斯托尔曼(Richard Stallman)在 Emacs 的最初实现 (1975/6 年) 中加入了一个名为 Info 的在线超文本帮助系统。斯托尔曼的灵感来源于几年前看到的一场关于道格拉斯·恩格尔巴特(Douglas Engelbart)的“NLS”超文本系统的演示。

另一项进展是，20 世纪 70 年代，在卡内基梅隆大学(CMU)，布莱恩·里德(Brian Reid)开发了一个名为 Scribe 的程序和格式，用于标记文档以供打印。它像 Texinfo 一样，使用 @ 字符

来引入命令。更重要的是，它致力于描述文档内容而非格式，这一理念被 Texinfo 全心全意地采纳。

与此同时，麻省理工学院(MIT)的人员开发了另一种名为 Bolio 的格式。理查德·斯托尔曼(RMS)致力于将 Bolio 转换为使用 $\text{T}_{\text{E}}\text{X}$ 作为其排版语言，从而产生了 $\text{BoT}_{\text{E}}\text{X}$ 。最早的 $\text{BoT}_{\text{E}}\text{X}$ 版本似乎是 1984 年 10 月 31 日的 0.02 版本。

$\text{BoT}_{\text{E}}\text{X}$ 只能用作打印文档的标记语言，不适用于在线文档。RMS 将 $\text{BoT}_{\text{E}}\text{X}$ 和 Info 结合，创建了 Texinfo，这是一种用于文本的标记语言，旨在既可以在线阅读，也可以打印成纸质版。

Info 的原始翻译器是用 Emacs Lisp 编写的（主要由 RMS 和 Bob Chassell 完成），具体来说就是 `texinfo-format-buffer` 和其他函数。20 世纪 90 年代初，布莱恩·福克斯(Brian Fox) 用 C 语言重新实现了转换程序，该程序现称为 `makeinfo`，同时还实现了独立的 `info` 程序。

用 Perl 重新实现

2012 年，C 语言的 `makeinfo` 命令本身被一个通常称为 `texi2any` 的 Perl 实现所取代。该版本支持与 `texi2html` 相同的输出定制级别，`texi2html` 是一个独立程序，最初由莱昂内尔·康斯(Lionel Cons)编写，后来经过许多人的大量工作。使 `texi2html` 成为 `makeinfo` 替代品所需的许多附加功能由帕特里斯·杜马斯(Patrice Dumas)实现。`texi2any` 的第一个从未发布的版本是基于 `texi2html` 的代码。

然而，该实现被放弃，转而采用了当前的程序，同样由帕特里斯·杜马斯(Patrice Dumas)编写，该程序将 Texinfo 输入解析为树结构以进行处理。它继承了 `texi2html` 的定制设计和其他功能（有关 `texi2html` 兼容性的更多信息，请参见 19.7 节[`texi2html`], 169 页）。然而，`texi2any` 是一个完全的重新实现：它为所有后端构建了一个基于树的输入文档表示法，以便于工作。

新的 Perl 程序比旧的 C 程序慢得多。自首次发布以来，速度差距已部分缩小，但可能永远无法完全比拟。那么我们为何要切换呢？简而言之，我们打算并希望目前的程序在扩展到不同的输出样式、后端输出格式以及所有其他定制方面，会比之前 `makeinfo` 的 C 语言实现容易得多。更详细地说：

- HTML 定制。多年来，许多 GNU 及其他自由软件包一直愉快地使用着 `texi2html` 中的 HTML 定制功能。因此，实际上已经开发出了两种独立的 Texinfo 语言实现，而保持它们之间的同步并不简单。将 `texi2html` 中可实现的 HTML 定制功能添加到 C 程序中，将是一项艰巨的任务。
- Unicode，以及普遍的多语言支持，尤其是对东亚语言的支持。当时，如果用 C 语言实现，就相当于重写整个程序。从那以后，解析器和部分转换器后端已经用 C 语言重写，但转换器后端仍然大多使用内置了良好多语言支持的 Perl。
- 附加后端。`makeinfo` 代码已经变得错综复杂，以至于添加新的后端相当复杂，需要与现有后端进行复杂的交互。相比之下，新的实现为所有后端提供了一个清晰的基于树的表示形式，以便于工作。人们已经提出了多种不同的后端（ $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 、最新的(X)HTML、...）；这一变化使得它们的实现变得更加可行。这就引出了最后一项：
- 使贡献更加容易。总体而言，由于结构更为清晰，单独的解析器/后端实现应该比之前的 C 语言 `makeinfo` 实现更容易阅读和贡献，从而带来显而易见的好处。十年后，虽然后端贡献仍未实现，但人们仍然相信，这种结构理论上更有利于贡献。

`texi2any` 旨在成为一个参考实现，它定义了手册中未完全指定的语言部分。如果没有这样的参考，替代实现很可能在行为上存在微妙或不太微妙的差异，因此 Texinfo 文档将依赖于处理器。所有处理器具有一致的命令行选项也很重要。在开发 `texi2any` 的同时，还开发了对语言和处理器器的广泛测试用例；我们鼓励任何考虑编写程序来解析 Texinfo 输入的人利用这些测试用例。

随着`texi2any`作为参考实现的发布，`makeinfo`和`texi2html`的 C 语言实现开发工作已暂停。未来，我们建议 Texinfo 文档的作者仅使用`texi2any`。

2 编写 Texinfo 文件

本章介绍了 Texinfo 的语法以及 Texinfo 文件中需要包含的内容，并给出了一个简短的示例文件。

2.1 通用句法约定

本节描述了所有 Texinfo 文档中使用的通用约定。

- 除‘@’、‘{’和‘}’之外的所有可打印 ASCII 字符都可以出现在 Texinfo 文件中，并且它们本身就有意义。‘@’是转义字符，用于引入命令，而‘{’和‘}’则用于包围某些命令的参数。要在文档中插入这些特殊字符之一，请在它前面加上一个‘@’字符，如下所示：‘@@’、‘@{’和‘@}’。
- 在 Texinfo 文件中，用于描述手册内容的命令，前会加上一个‘@’字符；这些命令被称为 @ 命令。（在 Texinfo 中，‘@’的含义与在普通 T_EX 中的‘\’相同。）

根据它们的功能或所接受的参数，你需要将 @ 命令单独成行，或作为句子的一部分。一般来说，如果命令与其他文本混杂在一起，则需要使用花括号；但如果命令单独成行，则不需要使用花括号。有关 Texinfo 命令语法的更多详细信息，请参见节 A.1 [Command Syntax], 205 页。

- @ 命令后的空格是可选的，并且（通常）如果存在会被忽略。但当空格具有特定意义时，例如在 @example 环境中，则另当别论。
- Texinfo 支持英语和其他语言中常用的引号；请参见 11.5 节 [Inserting Quotation Marks], 92 页。
- 连续使用三个连字符‘---’，产生一个长破折号---就像这样（称为长破折号, *Em dash*），用作句子中的标点。使用两个连字符‘--’，产生一个中破折号（称为半长破折号, *En dash*），主要用于数字范围，如“June 25--26”。使用一个连字符‘-’，产生一个标准连字符，用于复合词。在屏幕上显示时，Info 会将三个连字符减少为两个，将两个连字符减少为一个（不可传递！）。当然，源代码中（处于逐字模式 *verbatim modes* 中）的任意数量的连字符在中保持字面内容不变，例如 @code 和 @example。
- Whitespace* (空白)。Texinfo 文件是由以常规换行符 (line feed) 结尾的行组成的纯文本文件。Texinfo 处理器一次读取一行输入。段落以空行或仅包含空格的行结尾。文本中连续的多个空格通常被视为单个空格，除非处于逐字模式 (*verbatim modes*)。

换页符 (*CTRL-L*) 字符会终止任何打开的段落。其他换页符以及 ASCII 空白符（制表符、回车符）的处理方式与空格字符相同（尽管结果可能因输出格式而异）。因此，在文档中使用这些空白字符意义不大。非 ASCII 空格，如 Unicode 的“全角空格”，根本不被识别为空白符，而是被视为常规的非空白字符。

然而，在逐字模式 (*verbatim modes*) 中，例如在代码示例中，制表符可能会在输出中产生正确的格式。

2.2 注释

在 Texinfo 文件中，可以使用 @comment 命令（也可简写为 @c）来编写注释。此类注释是给查看 Texinfo 源文件的人看的。在 @comment 或 @c 之后的一行中的所有文本都是注释；该行的其余部分不会出现在可见输出中。（准确地说，@c 或 @comment 之后的字符必须是破折号或字母数字以外的字符，否则它将被视为命令的一部分。）

通常，您可以在一行中间写入 @comment 或 @c，只有 @comment 或 @c 命令之后的文本不会显示；但某些命令（如：@settitle）则作用于整行。在以此类命令开头的行中，您不能使用 @comment 或 @c。

在嵌套命令调用、复杂的宏定义等情况下，`@c`和`@comment`在 $\TeX$ 处理时可能会引发错误。因此，您也可以使用 `DEL` 字符（ASCII 127（十进制），0x7f（十六进制），0177（八进制））作为真正的 $\TeX$ 注释字符（在 $\TeX$ 内部，类别码为 14）。`DEL` 后的行中的所有内容都将被忽略，下一行将被合并。

您还可以使用 `@ignore` 和 `@end ignore` 命令让 Texinfo 处理器忽略大段文本。请将每个命令单独占一行，且每个命令都从行首开始。这两个命令之间的文本不会出现在处理后的输出中。您可以使用 `@ignore ... @end ignore` 来编写注释。（关于此类命令嵌套的注意事项，请参阅 15.7 节[Conditional Nesting], 128 页。）

2.3 Texinfo 文件必须包含的内容

按照惯例，Texinfo 文件的名称以扩展名 `.texi`、`.texinfo`、`.txi` 或 `.tex` 之一结尾。不推荐使用 `.tex`，因为此扩展名已被 $\TeX$ 和 $\LaTeX$ 输入文件使用。最常见且推荐的扩展名为 `.texi`。Texinfo 文件的名称应仅包含 ASCII 字符。

在默认情况下，输出名称基于输入文件名。首先，从输入文件名中移除所有扩展名为 `.texi`、`.tex`、`.txi` 或 `.texinfo` 的部分；然后，添加特定于输出格式的扩展名---生成 HTML 时为 `.html`，生成 Info 时为 `.info`，以此类推。输出文件名应仅包含 ASCII 字符¹。

为了被制作成印刷手册，Texinfo 文件必须以这样的一行开头²：

```
\input texinfo
```

文件内容以此开头，然后您必须在 Texinfo 源代码的末尾添加类似以下的一行：

```
@bye
```

文件末尾单独一行的 `@bye` 告诉 $\TeX$ ：文件已结束，停止格式化。如果你省略了这一行，运行结束时你会被丢弃在 $\TeX$ 的提示符处。

此外，你通常会提供一个包含标题、标题页、索引等内容的 Texinfo 文件，本手册对这些内容都有详细说明。但最简单的版本，仅需在文件开头和结尾各添加一行即可，这对于简短的文档非常有用。

若无额外信息，则假定输入和输出编码为 UTF-8，这是一种与 7 位 ASCII 兼容的通用编码集。

2.4 简短的示例

以下是一个简短的 Texinfo 文件示例。

```
\input texinfo
@settitle Sample Manual 1.0

@copying
This is a short example of a complete Texinfo file.

Copyright @copyright{} 2016 Free Software Foundation, Inc.
@end copying

@titlepage
```

¹ `texi2any` 命令可以处理输入文件名中的非 ASCII 字符，但输出文件名中的非 ASCII 字符会给某些输出格式带来问题，尤其是对交叉引用而言

² 译者注：为正确处理中文，本文档的‘`texinfo`’替换为‘`texinfo_zh`’。

```
@title Sample Title
@page
@vskip 0pt plus 1filll
@insertcopying
@end titlepage

@contents

@node Top
@top GNU Sample

This manual is for GNU Sample
(version @value{VERSION}, @value{UPDATED}).

@menu
* First Chapter::    The first chapter is the
                     only chapter in this sample.
* Index::            Complete index.
@end menu

@node First Chapter
@chapter First Chapter

@cindex chapter, first
This is the first chapter.
@cindex index entry, another

Here is a numbered list.

@enumerate
@item
This is the first item.

@item
This is the second item.
@end enumerate

@node First Section
@section First Section

First section of first chapter.

@node Second Section
@section Second Section
```

```
Second section of first chapter.
```

```
@node Index
@unnumbered Index

@printindex cp

@bye
```

2.5 Texinfo 文件头部

Texinfo 文件用以下行开头：

```
\input texinfo
```

‘\input texinfo’这行代码告诉 $\text{\TeX}$ 使用 `texinfo.tex`³ 文件，该文件告诉 $\text{\TeX}$ 如何将 Texinfo 的 @ 命令转换为 $\text{\TeX}$ 排版命令。请注意反斜杠‘\’的使用；这对于 $\text{\TeX}$ 是正确的。

将任何影响整体文档格式的命令纳入 Texinfo 的文件头部中是有意义的。@settitle 行通常出现在文件头部的开始处：

```
@settitle Sample Manual 1.0
```

@settitle 行用于为打印手册的页眉（或页脚）指定标题，并为 HTML 中的‘<head>’指定默认标题和文档描述。例如，@synindex（请参阅 10.6.2 节[@synindex], 84 页）是另一个常包含在文件头部中的命令。

Texinfo 文件从开始，直到作为文档主体输出的第一个内容，是前导 (*preamble*) 部分。它包括头部 (header)、2.7 节 [Document Permissions], 14 页和 2.8 节 [Titlepage & Copyright Page], 16 页规范。对于 $\text{\LaTeX}$ 输出格式而言，前导部分的结尾非常重要，因为这里是输出 \begin{document} 行的位置。在其他输出格式中，前导部分可能用于确定某些特定输出如何被格式化 (例如，2.7.1 节[@copying], 15 页以注释的形式作为输出文件的开头)，或者在文件头部决定所使用的语言。

2.5.1 Texinfo 文件的第一行

每个要作为 $\text{\TeX}$ 顶级 (top-level) 输入的 Texinfo 文件，都必须以类似以下这样的行开头：

```
\input texinfo
```

当 $\text{\TeX}$ 处理文件时，‘\input texinfo’命令告诉 $\text{\TeX}$ 加载处理 Texinfo 文件所需的宏。这些宏位于一个名为 `texinfo.tex` 的文件中，该文件应已随 $\text{\TeX}$ 或 Texinfo 软件一起安装在您的系统上。 $\text{\TeX}$ 使用反斜杠‘\’来标记命令的开始，这与 Texinfo 使用‘@’的方式完全相同。`texinfo.tex` 文件会导致从‘\’切换到‘@’；在切换发生之前， $\text{\TeX}$ 需要‘\’，这就是为什么它出现在文件开头的原因。

你可以选择在这行之后添加一条注释，告诉 GNU Emacs 在编辑文件时使用 Texinfo 模式：

```
\input texinfo    @c -*-texinfo-*
```

当 Emacs 无法从文件扩展名自动检测到文件类型时，这可能很有用。

³ 译者注：如是中文文档，的‘texinfo’需替换为‘texinfo_zh’。

2.5.2 @setfilename: 设置输出文件名

`@setfilename`这一行指定了由`texi2any`生成的输出文件名。对于 TeX 格式化，此命令将被忽略。当存在时，它应该是第一个 Texinfo 命令（即，在`\input texinfo`之后）。将`@setfilename`命令写在一行的开头，并在同一行后面跟上 Info 文件名。

```
@setfilename info-file-name
```

该名称必须与 Texinfo 文件的名称不同。选择名称有两种约定：你可以从输入文件名中完全移除扩展名（如`.texi`），或者（推荐）用`.info`扩展名替换它。不建议将`@setfilename`名称设置为与输入文件名完全不同的名称。

当存在`@setfilename`行时，Texinfo 处理器会忽略`@setfilename`行之前所写的所有内容。这就是为什么文件的第一行（`\input`）不会出现在输出中的原因。

如果没有`@setfilename`行，`texi2any`将使用输入文件名来确定输出文件名（请参阅 2.3 节[Minimum], 9 页）。在此处理过程中，`\input`行以及前导空白行仍会被忽略。

在生成另一种输出格式时，`texi2any`会将任何文件名的末尾扩展名替换为特定输出格式的扩展名（例如，生成 HTML 时替换为`html`），或者，如果给定文件名没有扩展名，则会在文件名后添加一个点号，再跟上一个扩展名（对于 HTML，则为`.html`）。

在`@setfilename`中不允许使用 `@` 命令，但`@@`、`@{`、`@}`以及相关的 `@` 命令（如`@atchar{}`）除外。

Texinfo 处理器和一些其他处理程序过去需要使用`@setfilename`，但现在不再需要这样，`@setfilename`可以省略。如果 Texinfo 输入是从标准输入处理的，没有输入文件名来推断基本文件名，那么`@setfilename`可能仍然有用。但这并不是唯一的方式：`--output`选项在`texi2any`命令行上指定输出文件名（请参阅 19.1 节[Invoking texi2any], 148 页）。

尽管明确使用`.info`扩展名更为可取，但某些操作系统无法处理过长的文件名。即使您指定的文件名本身足够短，也可能遇到问题。这是因为 Info 格式化程序会将过长的 Info 文件拆分为较短的间接子文件，并在原始文件名后附加`-1`、`-2`、...、`-10`、`-11`等来命名这些子文件。（请参阅 20.2 节[Tag and Split Files], 176 页。）例如，对于文件名限制为 14 个字符的旧系统而言，子文件名`texinfo.info-10`过长；因此，在这样的系统上，本文档的 Info 文件名可以是`texinfo`，而不是`texinfo.info`。`@setfilename`是一种指定替代名称的方法。

2.5.3 @settitle: 设置文档标题

Texinfo 文件应包含一行类似这样的代码：

```
@settitle title
```

在一行的开头写下`@settitle`命令，并在同一行后紧接标题。该行上不要写任何其他内容。`@settitle`命令应位于所有生成实际输出的内容之前。如果存在`@setfilename`命令（如上一节所述），则最好将其放在该命令之后。

此命令指定了双面打印输出时页眉或页脚中使用的标题（如果输出此类标题）。有关打印输出标题的更多信息，请参阅 2.8.5 节[Heading Generation], 19 页。

在 HTML 中，`title`作为文档的`<title>`，并成为`<head>`部分中的默认文档描述。

当标题页用于输出时（通常用于打印输出），`@settitle`命令中的标题不会影响标题页上显示的标题。因此，这两者无需完全匹配。我们建议的做法是在`@settitle`标题中包含手册的版本号或版次；在标题页上，版本号通常以`@subtitle`的形式出现，因此会从`@title`中省略。请参阅 2.8.1 节[@titlepage], 16 页。

2.5.4 前导

前导 (*Preamble*) 从 Texinfo 文件的开头开始, 一直延续到第一个直接输出的内容。它通常包括文件头 (请参阅 2.5 节 [Texinfo File Header], 11 页)、定义文档的复制授权的 `@copying` 块 (请参阅 2.7.1 节 [`@copying`], 15 页) 以及 `@titlepage` 规格 (请参阅 2.8 节 [Titlepage & Copyright Page], 16 页)。

前导中可能包含一些命令, 这些命令会影响整个文档的格式, 但不会直接产生输出, 或者不会立即产生输出, 例如 `@settitle` (请参阅 2.5.3 节 [`@settitle`], 12 页)、`@documentlanguage` (请参阅 14.1 节 [`@documentlanguage`], 118 页)、设置标题、控制缩进或断行的命令, 以及目录命令 (请参阅 2.9 节 [Contents], 19 页)。

任何开始一个段落的文本, 格式化为引文、表格、列表等的 `@` 命令, 以及 `@node` (请参阅第 3 章 [Nodes], 24 页) 和章节结构命令 (请参阅第 4 章 [Chapter Structuring], 34 页) 都会结束前导部分。

前导的概念对于 L^AT_EX 输出非常重要, 因为 `\begin{document}` 行是在前导结束时输出的。

在纯文本中, 前导部分会像往常一样在文档的开头输出; 例如, 前导中的 `@contents` 会输出为目录 (请参阅 2.9 节 [Contents], 19 页)。

对于 HTML 和 Info 输出, 前导部分也没有太多特殊处理。然而, 前导末尾的一些设置可能适用于整个文档, 而不管后面是什么。这可能适用于指定缩进的命令, 或者指定语言 (请参阅 14.1 节 [`@documentlanguage`], 118 页)。

例如, 对于以下文档, 在 `@documentlanguage` 设置为 ‘pt’ 情况下对 HTML 及 Info 的复制授权内容 (由 `@copying...@end copying` 定义) 进行格式化, 因为这是前导部分结束前的最后一个 `@documentlanguage`。

```
\input texinfo
@documentlanguage fr

@copying
The copying information @error{} some text
@end copying

@documentlanguage pt

文本, 标志着前导的结束。

@documentlanguage de

@node Top
```

2.5.5 Emacs 使用的, 头部的开始和结束

在 Emacs 中, 可以使用头部的起始和结束行来包围 Texinfo 前导中全局影响文档的命令。这允许您仅为 Info 或打印而格式化 Texinfo 文件的一部分。请参阅: 节 D.6.2 [texinfo-format commands], 247 页。

头的起始行是 Texinfo 注释, 其格式如下:

```
@c %**start of header
```

在 Texinfo 文件的第二行写入头部起始行。之后是那些全局影响文档格式的头部起始命令，如`@settitle`、`@synindex`或`@footnotestyle`；然后以`end-of-header` 行结束。

头部结束行是 Texinfo 注释，其格式如下：

```
@c %**end of header
```

字符串`%**`是为了确保不会误将其他注释视为标题行的开头。如果你愿意，可以通过设置 Emacs 变量`tex-start-of-header`和（或）`tex-end-of-header`来更改它。请参阅节 D.7.1 [Texinfo Mode Printing], 247 页。头部起始行和头部结束行不属于 Texinfo 格式规范的一部分，它们是用注释方式实现的。

2.6 目录分类列表

使用`@dircategory`命令为手册指定一个目录分类列表。以下是几个分类名称的示例：

```
Basics
Text creation and manipulation
Archiving
Compression
Database
Editors
Emacs
Email
Graphics
Localization
Network applications
Printing
Science
Software development
Software libraries
Version control
```

`@dircategory`命令后面通常跟着一个`@direntry`块，这些块由`install-info`使用。有关详细信息，请参阅20.1.4 节[Installing Dir Entries], 173 页。

手册中的第一个`@dircategory`命令是整个手册的分类列表。后续使用的`@dircategory`仅能追加`@direntry`块。

2.7 文档许可

此段对文档进行描述，并包含版权声明和复制许可。这是通过`@copying`命令实现的。根据其分发所依据的许可证，一本真正的手册在此处会有更多文本。

```
@copying
```

这是一个完整的 Texinfo 文件的简短示例，版本 1.0。

```
Copyright @copyright{} 2016 自由软件基金会股份有限公司
```

```
@end copying
```

文档的版权声明和复制许可需要出现在各种 Texinfo 输出格式的多个位置。因此，Texinfo 提供了命令`@copying`来一次性声明这些文本，并另提供命令`@insertcopying`来在适当的位置插入这些文本。

如果文档是一份软件手册，那么该软件通常遵循不同的许可证——对于 GNU 和许多其他自由软件包，软件通常遵循 GNU 通用公共许可证(GNU GPL)发布，而手册则遵循 GNU 自由文档许可证(GNU FDL)发布。在手册中说明软件的许可证是有用的，但并不一定需要给出软件许可证的完整文本。

2.7.1 @copying: 声明复制许可

应在文档的早期给出@copying命令；建议的位置是在文件头部（请参阅 2.5 节[Texinfo File Header], 11 页）之后。它通常包含一两句话，说明程序是什么、文档本身的标识、法律性版权声明以及复制许可。以下是一个示例框架：

```
@copying
This manual is for program (version version, updated
date), which ...
```

```
Copyright @copyright{} years copyright-owner.
```

```
@quotation
Permission is granted to ...
@end quotation
@end copying
```

@quotation没有法律意义；它只是在某些情境下用来提高可读性。

@copying的文本会作为注释隐式出现在 Info 和 HTML 输出文件的开头。此文本也会使用适当的标记出现在 DocBook 输出文件的开头。纯文本或打印输出中，@copying的文本不会隐式出现，是否出现，取决于您是否使用@insertcopying。详情请参阅下一节。

@copyright{}命令会在圆圈内生成一个‘c’，仅当输出格式支持此字形（例如，打印和 HTML 格式始终支持）时；当输出格式不支持此字形时，它会生成三个字符的序列‘(C)’。版权声明本身具有法律规定的格式，如下所示：

```
Copyright © 年份 版权所有。
```

即使文档以其他语言书写，‘Copyright’一词也必须始终使用英文。这是国际法的要求。

年份列表应包含作品完成的所有年份（即使作品是在随后年份发布的）。最简单的方法是单独列出每个年份，并完整写出，用逗号分隔。

版权所有（者）是指对作品持有合法版权的人。对于归属于自由软件基金会的作品，所有者是“自由软件基金会股份有限公司”。

版权“行”实际上可能跨越多行，无论是在源文档中还是在输出中。这种情况在历史悠久的文档中很常见，这些文档的出版年份各不相同。如果你确实使用了多行，请不要在源文件中对任何一行（或@copying块中的任何内容）进行缩进。

请参阅节 “Copyright Notices” 在 *GNU Maintainer Information*，了解更多信息。请参阅节 C.1 [GNU Sample Texts], 234 页，获取 GNU 手册中使用的全文。请参阅附录 G [GNU Free Documentation License], 272 页，了解 GNU 和其他自由手册所遵循的许可证。

2.7.2 @insertcopying: 插入复制许可文本

@insertcopying命令只需单独写在一行，如下所示：

```
@insertcopying
```

这将插入之前由@copying声明的文本。为满足法律要求，必须在印刷手册的版权页上使用该文本(请参阅 2.8.4 节[Copyright], 18 页)。

`@copying`命令本身会让许可文本在第一个节点之前出现在 Info 文件中。该文本也会根据法律要求被复制到每个拆分的 Info 输出文件的开头。这个位置意味着使用 Info 阅读器阅读手册的人不会看到这段文本（除非使用高级 Info 命令`g *`）。这在法律上并不重要，因为文本是存在的。但是，为了在输出中看到这段文本，应使用`@insertcopying`。

同样地，`@copying`文本会自动作为 HTML 注释包含在每个 HTML 输出文件的开头。同样，如果没有`@insertcopying`，这段文本是不可见的（除非读者查看 HTML 源代码）。

由`@copying`定义的许可文本也会使用适当的标记自动出现在 DocBook 输出文件的开头。`@insertcopying`可用于在普通文本中输出许可文本。

2.8 标题和版权页

在纸质输出中，手册的名称和作者通常印在标题页⁴上。版权信息通常印在标题页的背面（即反面）。此部分必须包含在`@titlepage`和`@end titlepage`命令之间：

```
@titlepage
@title Sample Title

@c 用下面的两个命令作为版权页的开始。
@page
@vskip 0pt plus 1filll
@insertcopying
@end titlepage
```

我们使用`@insertcopying`命令来包含上一节所述的许可文本，而不是再次将其写出来。

标题和版权页会出现在印刷手册中，但在大多数其他输出格式中则不会出现。在 HTML 中，要获得与印刷手册相似的标题页，最好的方法是设置`NO_TOP_NODE_OUTPUT`定制化变量（请参阅`[NO_TOP_NODE_OUTPUT]`，165 页）。

2.8.1 @titlepage

标题页和随后的版权页的材料应以单独的一行`@titlepage`开始，并以单独的一行`@end titlepage`结束。

`@end titlepage`命令用于开始新页面并启用页码功能（请参阅2.8.5 节`[Heading Generation]`，19 页）。所有希望在未编号页面上显示的内容都应置于`@titlepage`和`@end titlepage`命令之间。

通过使用`@page`命令，可以在由`@titlepage`的`@end titlepage`命令划定的区域内强制分页，从而创建多个未编号的页面。这就是版权页的制作方式。（`@titlepage`命令或许可以更好地命名为`@titleandadditionalpages`命令，但那样会显得相当冗长！）

当你编写关于计算机程序的手册时，应在标题页上注明该手册适用的程序版本。如果手册的更新频率高于程序，或者手册与程序无关，则还应为手册包含一个版本号⁵。这有助于读者了解哪个手册对应哪个版本的程序。（“顶”节点也应包含此信息；见2.10 节`[The Top Node]`，20 页。）

一种方法是使用`@title`、`@subtitle`和`@author`命令来创建标题页。使用这种方法时，无需指定标题页的任何实际格式。只需指定所需的文本，Texinfo 就会完成格式设置。通常的格式设置包括标题和作者行下方的黑色规则线，以及设置在页面右侧的副标题文本。

⁴ 译者注：在中文的习惯中，可称之为封面。

⁵ 我们发现，将独立手册的版本称为“版次”，将程序的版本称为“版本”很有帮助；否则，在交谈中，我们可能会因为使用相同的词语来指代文档和软件而相互混淆。

Texinfo 还提供了另一种创建标题页的方法。该方法使用在正文中不使用的排版命令。具体来说，它使用`@titlefont`、`@sp`和`@center`命令来生成标题页，使页面上的文字居中显示。

对于足够简单的文档，以及传统书籍前言部分的非正式标题页，Texinfo 还提供了`@shorttitlepage`命令，该命令将该行剩余部分作为标题。参数会单独成页排版，并在之后跟一个空白页。在 HTML 中，`@shorttitlepage`可以在未设置`@settitle`时起到相同的作用。请参阅 2.5.3 节`@settitle`，12 页。

2.8.2 @title、@subtitle和@author

您可以使用`@title`、`@subtitle`和`@author`命令来创建标题页，其中会自动为您设置垂直和水平间距。

在行首输入`@title`、`@subtitle`或`@author`命令，后跟标题、副标题或作者。`@author`命令也可用于`@quotation`(引述)块中的引述（请参阅 7.2 节`@quotation`，59 页）；除此之外，在`@titlepage`之外使用这些命令都是错误的。

`@title` 命令通常会在页面中生成一行，其中标题设置为与页面左侧齐平，且字体大于常规字体。标题下方会有一条黑色下划线。在 Texinfo 源文件中，标题必须位于单行；如果需要，输出时可以将其拆分为多行。

对于长标题，如果自动换行不符合需求，可以使用`@*`命令来指定长标题中的换行位置。此类显式换行通常会在所有输出格式中体现；如果您只想为打印输出指定换行，请使用条件语句（请参阅第 15 章`[Conditionals]`，121 页）。例如：

```
@title 这是长标题 @inlinefmt{tex,*}在 @TeX{}输出中换行
```

`@subtitle`命令通常会将副标题设置为正常大小的字体，并紧贴页面右侧。

`@author`命令通常会在标题页底部附近的一行上，将作者姓名以中等大小的字体设置在页面左侧。姓名后面会有一条比通常用于标题下划线的线更细的黑线。

使用`@author`命令有两种方法：你可以在以`@author`命令开头的行的剩余部分写上姓名：

```
@author by Jane Smith and John Doe
```

或者，你可以使用多个`@author`命令，将姓名一个接一个地写在上面：

```
@author Jane Smith
@author John Doe
```

此方法的模板如下：

```
@titlepage
@title 手册名称(打印时)
@subtitle 子标题(如果有)
@subtitle 第二个子标题
@author 作者
@page
...
@end titlepage
```

2.8.3 @titlefont、@center和@sp

您还可以使用`@titlefont`、`@sp`和`@center`命令为打印文档创建标题页。

使用`@titlefont`命令来选择适合标题本身的大字体。如果标题特别长，可以多次使用`@titlefont`。

对于 HTML 输出，每个`@titlefont`命令都会生成一个`<h1>`标题，但 HTML 文档的`<title>`部分不会受到影响。为此，您可以在`@titlefont`命令之前放置一个`@settitle`命令（请参阅 2.5.3 节[`@settitle`], 12 页）。

For example:

```
@titlefont{Texinfo}
```

在一行的开头使用`@center`命令，可以将该行剩余的文本居中显示。因此，

```
@center @titlefont{Texinfo}
```

居中显示标题，在本例中，标题为“Texinfo”，以标题字体打印。

使用`@sp`命令来插入垂直间距。例如：

```
@sp 2
```

这会在打印页上插入两行空白行。（请参阅 12.7 节[`@sp`], 103 页，有关`@sp`命令的更多信息，请参阅。）

此方法的模板如下：

```
@titlepage
@sp 10
@center @titlefont{手册名称(在打印时)}
@sp 2
@center 子标题(如果有)
@sp 2
@center 作者
...
@end titlepage
```

这个示例的排版间距适合 8.5 英寸乘 11 英寸的手册。

2.8.4 版权页

根据国际条约，书籍的版权声明必须位于扉页或扉页背面。当版权声明位于扉页背面时，该页通常不编号。因此，在 Texinfo 中，版权页上的信息应位于`@titlepage`和`@end titlepage`命令之间。

使用`@page`命令来产生分页符。若要将版权声明和版权页上的其他文本推至页面底部，请在`@page`之后使用以下指令：

```
@vskip 0pt plus 1filll
```

`@vskip`命令在 TeX 输出中插入空白；在所有其他输出格式中，该命令会被忽略。‘`0pt plus 1filll`’表示插入零个强制空白点，并根据需要尽可能多的可选空白点加入，以将后续文本推至页面底部。请注意‘`filll`’单词中使用了三个‘`l`’；这是正确的。

要插入版权文本本身，请在之后写上`@insertcopying`（请参阅 2.7 节[Document Permissions], 14 页）：

```
@insertcopying
```

请遵循出版商的排版格式，包括 ISBN 编号、封面设计致谢以及其他类似信息。

以下是一个将所有这些内容结合在一起的示例：

```
@titlepage
...
@page
```

```
@vskip 0pt plus 1filll
@insertcopying
```

```
Published by ...
```

```
Cover art by ...
@end titlepage
```

我们还有一个特殊情况需要考虑：对于纯文本输出，如果你想显示版权信息，就必须明确地插入版权信息。例如，在版权页之后，你可以这样写：

```
@ifplaintext
@insertcopying
@end ifplaintext
```

你可以在同一位置为纯文本输出包含其他类似标题的信息。

2.8.5 页眉生成

Texinfo 有两种标准的页眉格式，一种适用于每张纸只打印一面（单面打印）的文档，另一种适用于每张纸打印两面（双面打印）的文档。

一般而言，你可以通过不同的方式来控制页眉：

- 传统的方法是在标题页命令@titlepage之前写入一个@setchapternewpage命令。大多数文档都采用标准的单面或双面页眉格式，双面打印时（有时）会使用@setchapternewpage odd，而单面打印时（几乎总是）不会使用@setchapternewpage命令（请参阅节 E.1 [setchapternewpage], 258 页）。
- 或者，您可以使用@headings命令来阻止生成页眉，或者在单面或双面打印时启用它们。要关闭页眉，请写入@headings off。请参阅节 E.2.1 [headings], 259 页。
- 或者，您可以指定自己的页面页眉和页脚格式。请参阅节 E.2 [Headings], 259 页。

2.9 生成目录

@chapter、@section和其他结构化命令（请参阅第 4 章[Chapter Structuring], 34 页）提供了构成目录所需的信息，但它们并不会在手册中实际生成一个表格。要生成目录，您必须使用@contents和/或@summarycontents命令。

@contents

用于在打印手册中生成目录，包括所有章节、节、子节等，以及附录和未编号的章节。由@majorheading、@chapheading和其他@...heading命令生成的标题不会出现在目录中(请参阅 4.2 节[Structuring Command Types], 34 页)。

@shortcontents

@summarycontents

(@summarycontents是@shortcontents的同义词。)

生成一个简短的或摘要式的目录，仅列出章、附录和未编号的章节、子节和子子节则被省略。只有长篇手册才需要在完整目录之外再提供一个简短的目录。

两个目录命令应单独占一行，并置于文件开头附近，在@end titlepage (请参阅2.8.1 节[titlepage], 16 页)之后，任何章节划分命令之前。目录命令会在第一个目录页面的顶部自动生成与章类似的标题，因此不要在其之前包含任何章节划分命令，如：@unnumbered。

由于 Info 文件使用菜单而非目录，因此 Info 格式化命令会忽略`@...contents`命令，但在纯文本输出和其他输出格式（如 HTML）中，它们不会被忽略。

在 HTML 输出中，简短目录中的链接指向完整目录中的相应条目，而非指向文档文本。完整目录中的链接则指向文档的正文。

`@shortcontents`在 L^AT_EX 输出中未实现。

2.10 顶节点和主控菜单

‘顶(Top)’节点是读者进入 Info 手册的入口节点。因此，它应包含手册的简短描述（包括版本号）。顶节点的内容不会出现在打印输出或 DocBook 输出中。

按照惯例，在`@node Top`行之后，应立即另起一行，使用`@top`命令指定文档标题(请参阅 3.5 节[`@top Command`], 27 页)。

在下面的示例中，我们重复了‘`@copying`’文本开头的简短描述，但无需重复版权信息，因此我们在此处不使用‘`@insertcopying`’。

顶节点包含一个顶层的菜单(*Menu*)，其中列出了各个章节，还可能包含一个详细菜单(*detailed menu*)，其中列出了整个文档中的所有节点。

```
@node Top
@top Short Sample

This is a short sample Texinfo file.

@menu
* First Chapter::    The first chapter is the
                     only chapter in this sample.
* Index::            Complete index.
@end menu
```

2.10.1 主控菜单的组成部分

主控菜单(*master menu*)是主菜单(*main menu*)。通常，该菜单会包含一个详细菜单，列出文档中的所有节点。与其他菜单一样，主控菜单被包含在`@menu`和`@end menu`中，既不会出现在打印输出中，也不会出现在 DocBook 输出中。

主控菜单包含的各条目分别对应 Texinfo 文件中主要节点：章的节点、节的节点和附录的节点，其后是索引的节点。

你可以选择在这些条目之后添加一个详细菜单(*Detailed menu*)。该菜单列出了其他更低级别的节点，通常按章节排序。这些条目对于查询者来说可能很方便，因为在搜索特定信息时，他们可以直接跳转到特定的节点，而无需通过中间的菜单。如果你在主控菜单中使用详细菜单，请使用`@detailmenu ... @end detailmenu`环境进行标记。

菜单中的每个部分都可以用一行描述性文字来介绍。只要这行文字不以星号开头，就不会被视为菜单项。如需更多信息，请参阅 3.9.1 节[Writing a Menu], 30 页。

例如，本手册的主控菜单如下所示（但实际包含更多条目）：

```
@menu
* Copying Conditions::  您的权力.
* Overview::            Texinfo 简介.
...
```

```
* General Index::      一个涵盖众多主题的菜单。
* Translation notes::  简体中文翻译。
```

```
@detailmenu
--- The Detailed Node Listing ---
```

Texinfo 简介

```
* Reporting Bugs::      提交有效的错误报告。
...
```

编写 Texinfo 文件

```
* Short Sample Texinfo File:: 一个简短的 Texinfo 文件示例。
...
```

翻译说明

```
* Original::            被翻译的原文。
* Convention::          翻译时使用的惯例
* Contribution::        翻译工作的贡献者
@end detailmenu
@end menu
```

2.11 文档的正文

正文部分包含文档的所有文本。手册被划分为一个或多个 *nodes* (请参阅第 3 章[Nodes], 24 页)。示例展示了一个由三个节点组成的章, 其中一个用于章的介绍性材料, 还有另外两个节。介绍性材料包含一个枚举列表。

```
@node First Chapter
@chapter 第一章

@cindex chapter, first
@cindex 章, 第一
这是第一章。
@cindex index entry, another
@cindex 索引条目, 另一个
```

这里是一个枚举列表。

```
@enumerate
@item
这是第一个条目。
```

```
@item
这是第二个条目。
@end enumerate
```

```
@node First Section
@section 第一节
```

这是第一章的第一节。

```
@node Second Section
@section 第二节
```

第一章的第二节。

在 Info 输出中, ‘First Chapter’节点将包含一个菜单, 列出该章节中的两个部分。同样, 当此节点以独立的 HTML 文件输出时, 它将包含该章的目录。

本章的内容将如下:

第一章

这是第一章。

这里是一个枚举列表。

1. 这是第一个条目。
2. 这是第二个条目。

第一节

这是第一章的第一节。

第二节

第一章的第二节。

(在 Info 和 HTML 输出中，章也将被拆分为多个节点)。

2.12 结束 Texinfo 文件

Texinfo 文件的结尾应包含创建索引的命令（请参阅10.5 节[Printing Indices & Menus], 83 页），以及用于标记被处理内容最后一行的@bye命令。例如：

```
@node Index
@unnumbered Index

@printindex cp

@bye
```

一个@bye命令用于终止 Texinfo 处理。它应该单独占一行。@bye之后的任何内容都将被完全忽略。

3 节点

一个节点(Node)是从@node命令开始,并持续到下一个@node命令的文本区域。要指定一个节点,请在开头的行写一个@node命令,并在其后跟上节点的名称。Info阅读器一次显示一个节点,并提供命令供用户跳转到相关节点。HTML输出也可以进行类似的导航。

节点被用作交叉引用的目标。交叉引用(如本句末尾的引用)是通过@xref和相关命令实现的;请参阅第5章[Cross References],39页。交叉引用可以散布在文本中。其他@命令也可能是交叉引用的目标,请参阅5.11节[@anchor],44页,及9.1节[Floats],74页。

通常,您会在每个章节结构化命令(例如:@section或@subsection行)之前立即放置一个节点命令。(请参阅第4章[Chapter Structuring],34页)。即使您不打算为Info格式化文件,也应这样做。这是因为打印输出在输出中同时使用@node名称和章节结构化名称以进行交叉引用。只有在编写不包含交叉引用且仅用于打印(不转换为Info、HTML或其他格式)的文档时,才可能仅使用章节结构化命令而不使用节点。

3.1 编写@node行

在行首写入@node,后跟节点名称,如下所示:

```
@node node-name(节点名)
```

在插入@node行之后,您应立即为相关章或节(如果有的话)编写@命令,并在其后插入它们的名称。

您可以选择在@node的节点名称参数后,在同一行的其余部分添加最多三个可选参数,参数之间用逗号分隔。这些参数分别是“Next”(下一个)、“Previous”(上一个)和“Up”(上一级)指针的名称,按此顺序排列。因此,包含“Next”、“Previous”和“Up”指针的完整节点行的模板如下:

```
@node node-name(节点名), next(下一个), previous(上一个), up(上一级)
```

texi2any程序会自动为层次结构文档确定节点指针。为此,每个@node命令后面应紧跟一个章节划分命令,如@chapter或@section(注释行除外)。最后,在“Top”的@node行之后,必须跟上一行以@top开头的行,以标记文件中的顶节点。请参阅3.5节[@top Command],27页。

即使你明确指定了所有指针,也不能以任意顺序编写Texinfo源文件中的节点。你必须按照希望它们在输出中出现的顺序来编写节点。对于Info格式,可以认为顺序可能并不重要,但对于其他格式来说,顺序却很重要。

在大多数情况下,你会希望利用指针自动创建功能,而不是冗余地指定原本可以由程序确定的节点指针。然而,Texinfo文档并不要求必须按层次结构组织,或者实际上根本不要求包含章节划分命令(例如,如果你从未打算打印该文档),因此节点指针必须可以被完全通用地显式指定。

如果你正在使用GNU Emacs,并且想要明确的指针,你可以使用Texinfo模式提供的更新节点命令来插入指针的名称。请参阅节D.5 [Updating Nodes and Menus],241页。

或者,你可以自己插入“Next”、“Previous”和“Up”指针。如果你在Emacs中这样做,使用Texinfo模式键盘命令C-c C-c n可能会有所帮助。此命令会插入@node和一个注释行,该注释行按正确顺序列出了指针的名称。注释行有助于你跟踪哪些参数对应于哪些指针。

3.2 选择节点名称

节点名用于标识该节点。有关节点名的所有详细信息，请参见3.3 节[Node Line Requirements], 25 页。

以下是关于节点名的一些建议：

- 尝试选择信息量大但简短的节点名称。
在 Info 文件中，文件名、节点名和指针名都被插入到一行中，这可能会超出窗口的右边缘。（这并不会给 Info 文件带来问题，但看起来很不好看。）
- 尽可能在节点名的开头部分就让它们表现出不同。这样，更易于在 Info 中使用自动名称补全功能。
- 通常，节点名的首字母大写方式与章节标题相同。在本手册中，首字母和重要单词大写；其他单词则不作首字母大写。在其他手册中，只有首字母和专有名词的大写。两种方式都可以；我们建议保持一致。
- 在 HTML 输出中，节点名中除普通 ASCII 字母、数字或空格以外的任何字符都会在文件名中被更改。（请参阅21.7.2 节[HTML Xref Node Name Expansion], 185 页）。这可能会使手册中页面的 URL 对用户不太友好；例如，在本手册中，‘@dots’节点输出为__0040dots.html。

由于节点名在交叉引用中有其用途，因此一旦发布后不宜随意更改。当你删除或重命名一个节点时，通常建议为旧名称定义一个@anchor。这样，来自其他手册、邮件存档等处的引用就不会失效。请参阅5.11 节[@anchor], 44 页。

给定节点的指针使您能够访问其他节点，且指针仅包含这些节点的名称。

通常，本节点的“Up”指针包含在其菜单中提及本节点的节点的名称，“Next”指针包含在该菜单中位于本节点之后的节点的名称，“Previous”指针则包含在该菜单中位于其之前的节点的名称。当一个节点的“Previous”节点与其“Up”节点相同时，这两个指针可以都指向同一个节点。

通常，Texinfo 文件的第一个节点是“Top(顶)”节点，其“Up”指针指向dir 文件，该文件包含一个主菜单，该菜单包含所有 Info 文件（的“Top(顶)”节点）。

3.3 @node行要求

与@node一起使用的名称有几个要求：

- 单一 Texinfo 文件中的所有节点名称必须唯一。
这意味着，例如，如果你在每章末尾都添加一个总结，那么你必须为每个总结节点命名不同。你不能全部都命名为“总结”。然而，章、节等的标题可以重复。因此，你可以在每章末尾添加一个名为“总结”的节，只要这些节对应的节点名各不相同即可。
节点名、锚点名（请参阅5.11 节[@anchor], 44 页）和浮动标签（请参阅9.1.1 节[@float], 74 页）都必须唯一。
- 节点名可以包含 @ 命令¹。
例如，在节点名称中使用@TeX{}会输出 T_EX 标志，就像在普通文本中一样。交叉引用应与节点名称一样，使用@TeX{}。
在节点名中，有些命令没有意义；例如，环境（如@quotation）、将整行作为参数读取的命令（如@sp）以及许多其他命令。有关允许使用的命令的完整列表及其在 HTML 标识符和文件名中的扩展，请参阅21.7.3 节[HTML Xref Command Expansion], 185 页。

¹ 在 2013 年 Texinfo 5 发布之前，此功能以一种特殊的方式得到支持（makeinfo的--commands-in-node-names选项）。现在，它已成为该语言的一部分。

- 节点名称不能以左括号开头，且左括号前不能有右括号，如(not)allowed 中的情况，因为这种语法用于指定外部手册。
- 不幸的是，您不能在节点名称中可靠地使用句点、逗号或冒号；这些字符可能会让一些 Info 阅读器感到困惑。texi2any默认对有问题的节点名称和标签加引用，但一些 Info 阅读器无法识别这种语法。对节点名称和标签加引用会导致名称周围出现DEL字符（‘CTRL-?’，字符编号 127，通常呈现为‘~?’）。为保证节点名和标签不加引用，可以将定制化变量INFO_SPECIAL_CHARS_QUOTE设置为‘0’（请参阅19.4.4 节[Info and Plaintext Customization Variables], 162 页）。

texi2any会对节点名、菜单项和交叉引用中的这类有问题的使用方式发出警告。如果你不想看到这些警告，可以将定制化变量INFO_SPECIAL_CHARS_WARNING设置为‘0’（请参阅19.4.4 节[Info and Plaintext Customization Variables], 162 页）。

如果你坚持在节点名称中使用这些字符，为了不使 Texinfo 处理器产生混淆，你仍然必须对这些字符进行转义，方法是使用特殊插入命令（请参阅11.1.3 节[Inserting a Comma], 86 页）或@asis（请参阅[@asis], 70 页）。例如：

```
@node foo@asis{::}bar@comma{} baz
```

作为避免使用特殊字符的一个示例，以下是本手册中的一个章节标题：

```
@section @code{@@unnumbered}, @code{@@appendix}: Chapters with...
```

但对应的节点名称去掉了逗号和副标题：

```
@node @code{@@unnumbered @appendix}
```

- 节点名中的大小写很重要。
- 在‘@node’行中，名称前后的空格将被忽略。名称“内部”的多个空格字符将被合并为一个空格。例如：

```
@node foo bar
@node foo bar,
@node foo bar ,
@node foo bar,
@node foo bar ,
```

它们都定义了同一个节点，即‘foo bar’。

3.4 第一个节点

Texinfo 文件的第一个节点是Top(顶)节点，除非是在被包含的文件中（请参阅第17 章[Include Files], 140 页）。顶节点应包含一个简短的摘要和一个主控菜单。有关顶节点内容和示例的更多信息，请参阅2.10 节[The Top Node], 20 页。应避免在顶节点之前、任何节点之外的纯文本。如果存在此类文本，则不会在 DocBook 中输出。

以下是顶节点中使用的节点指针的描述：

- 顶节点（必须命名为‘top’或‘Top’）的“上一级(Up)”节点应使用另一个文件中节点的名称（在括号中指定文件名），该文件中有一个菜单，可链接到本文件及其它 Info 文件。

通常，所有 Info 文件都可通过一个由多个目录构建的虚拟 Info 树来访问。在这种情况下，使用‘(dir)’作为顶节点的上一级节点，即：指定dir 文件（该文件包含整个 Info 系统的主菜单）中的顶节点作为上一级节点。每个包含 Info 文件的目录都应包含一个名为dir的文件。

这对于 Info 来说没问题，但对于 HTML 输出，人们可能希望从顶节点向“上一级(Up)”跳转到某个特定位置。例如，对于 GNU，自然的位置会是<http://www.gnu.org/manual/>

(一个收集了大多数 GNU 手册链接的网页)，如果手册将安装在 www.gnu.org 上，则最好直接指定为 `/manual/`。这可以通过 `TOP_NODE_UP_URL` 定制化变量(请参阅 21.8.2 节[File Names and Links Customization for HTML], 191 页)来指定，如下所示：

```
$ texi2any --html -c TOP_NODE_UP_URL=/manual/ ...
```

- 顶节点的“Prev(上一个)”节点通常被省略。
- 顶节点的“Next(下一个)”节点应该是您文档中的第一章。

请参阅 20.1 节[Installing an Info File], 171 页，了解更多关于在 `info` 目录中安装 Info 文件的信息。

通常最好完全省略指针，让工具隐式地定义它们，这样会得到一个简单的结果：

```
@node Top
```

3.5 @top 章节划分命令

`@top` 命令是一个特殊的章节划分命令，您只能在 Texinfo 文件开头的“`@node Top`”行之后使用。

它产生的输出类型与 `@unnumbered` 相同(请参阅 4.4 节[`@unnumbered @appendix`], 35 页)。在 $\text{\LaTeX}$ 中使用 `\part*`。

在升高或降低章节级别时，`@top` 会被忽略。也就是说，它永远不会降低，也没有任何东西可以升高到它之上(请参阅 4.12 节[Raise/lower sections], 38 页)。

过去，通常会将“`Top`”节点包裹在 `@ifnottex` 条件语句中，这样它就不会出现在打印输出中(请参阅第 15 章[Conditionals], 121 页)。因此，一个 `Top` 节点通常看起来像这样：

```
@ifnottex
@node Top
@top your-manual-title
```

```
级别非常高的简介
```

```
@end ifnottex
```

实际上，这已不再必要，因为现在打印输出时，“`Top`”节点永远不会输出。对于 DocBook，“`Top`”节点也不会输出。

3.6 Texinfo 文档结构

节点可以包含菜单(*menus*)，该菜单列出了父节点中子节点(*child nodes*)的名称；例如，与章相对应的节点，它包含一个菜单，菜单的一个条目对应该章中的一个节，一个条目中包含节对应的节点名及节点描述。这些菜单允许用户在 Info 输出中跳转到子节点。

此外，节点包含节点指针(*node pointers*)，用于指向其他节点。“`Next`”和“`Previous`”指针将同一章节级别的节点链接成一个链。显然，“`Next`”指针链接到下一个节点，而“`Previous`”指针链接到上一个节点。通常，“`Next`”和“`Previous`”指的是手册中同一层级的节点，而不一定是 Texinfo 文件中的后一个节点。在 Texinfo 文件中，后续节点可能处于较低层级——例如：一系列节层级节点通常跟在某个章层级节点之后，章内所有处于节层级的节点都会链接在一起，并且这个链中的顺序与章菜单中条目(指向节对应的节点)的顺序相同。每个子节点都会将父节点名称作为其“`Up`”指针。

由于“`Top`”节点是同一级别(顶级)的唯一节点，“`Next`”指的是紧随其后的第一个节点，该节点几乎总是章级节点(肯定不是顶级)。这是“`Next`”链接同一层级节点规则的一个例外。

每个节点的 Info 和 HTML 输出都包含指向“Next”、“Previous”和“Up”节点的链接。HTML 还使用了 `accesskey` 属性，其值分别为‘n’、‘p’和‘u’。这使得使用网页浏览器的人（通常）可以通过 *M-letter*（例如 *M-n* 表示“Next”节点），在节点内任意位置进行导航。节点指针和菜单为 Info 文件提供了结构，就像章、节、子节等为印刷书籍提供了结构一样。这两种结构在理论上有所不同；然而，在实践中，印刷书籍的树状结构基本上也总是用于节点和菜单结构，因为这样可以使文档易于阅读。

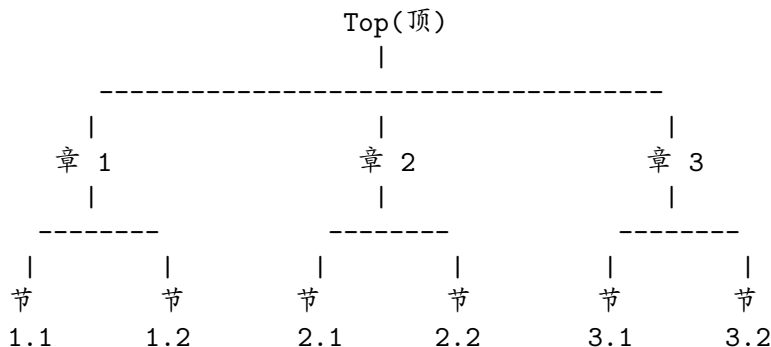
通常，章节结构和节点结构完全平行，每个章、节等都对应一个节点，且节点遵循与章节相同的层级结构。因此，如果一个节点处于章的逻辑级别，则其子节点处于节级别；同样，节对应节点的子节点会处于子节(subsections)级别。

从技术上讲，创建 Texinfo 文档时，也可以只使用其中一种结构，或者两种结构并不平行，或者使用非常规的章节或节点结构。然而，据我们所知，目前普遍使用的所有 Texinfo 手册都遵循常规的平行结构。

3.7 节点与菜单的图例

下图展示了一个包含三章的 Texinfo 文件，其中每一章都包含两个小节。

“根”位于图的顶部，“叶子”位于底部。这是此类图的常规绘制方式；它描绘了一棵倒置的树。因此，根节点被称为“顶”节点，而“Up”节点指针则指向上一级（即：更接近根节点的）节点。



使用显式指针（不推荐，但出于示例目的而展示），开始‘章 2’的完整命令如下：

```

@node    章 2,          章 3,          章 1,          Top(顶)
@comment 节点名(node-name), 下一个(next),  上一个(previous), 上一级(up)
  
```

此 `@node` 行表示该节点的名称是“章 2”，“Next”节点的名称是“章 3”，“Previous”节点的名称是“章 1”，而“Up”节点的名称是“Top”。如果您的文档是按层次结构组织的，您可以（并且应该）省略这些节点名称的书写，但指针关系仍然存在。

要使用 Info 跳转到第 2.1 节和第 2.2 节，您需要在第 2 章中添加一个菜单（请参阅 3.9 节 [Menus], 30 页）。您可以在第 2.1 节开始之前编写菜单，如下所示：

```

@menu
* 节 2.1::    节的描述。
* 节 2.2::    描述。
@end menu
  
```

第 2.1 节的节点使用自动指针的结果相当于：

```

@node    节 2.1,          节 2.2,          , 章 2
@comment 节点名(node-name), 下一个(next),  上一个(previous), 上一级(up)
  
```

请注意，由于在第 2.1 节之前没有处于同一层级的其他节点，因此没有生成“上一个(previous)”指针。

使用显式指针，第 2.1 节的节点也可以写成这样：

```
@node    节 2.1,           节 2.2,           章 2,           章 2
@comment 节点名(node-name), 下一个(next),   上一个(previous), 上一级(up)
```

使用自动指针时，节点的“Next”和“Previous”指针会指向同一层级的其他节点——从章节到章节，或从节到节。如图所示，在使用显式指针时，指针也可以指向其他位置，例如，这里的“Previous”指针指向上方。一个“Up”指针通常指向上一层级（更接近“Top(顶)”节点）的节点；而一个“菜单(Menu)”项指针则指向下一层级（即：更接近“叶子”）的节点。（交叉引用可以指向任何层级的节点；请参阅：第 5 章[Cross References], 39 页。）

从技术上讲，显式节点指针可以指向任何节点，无论文档的结构如何；甚至可以指向不同 Info 文件中的节点。然而，如果“Next”、“Previous”和“Up”指针指向的节点与下一个、上一个和上一级节点并不对应，或者是松散的对应，都会给读者带来极大的困惑。

@node命令和章节命令通常按前后依次一起使用，之后通常跟着索引命令。（如上例所示，在@node行之后可以跟一条注释行，例如，如果使用了显式指针，则用于表明各个指针的含义）。Texinfo 处理器使用这种构造来确定节点命令和章节命令之间的关系。

这是本手册中名为“Ending a File”的节的开头。这里展示了一行@node，后面跟着一行@section，然后是索引行。

```
@node Ending a File
@nodedescription Texinfo 文件的结尾是什么？
@section 结束 Texinfo 文件
@cindex Ending a Texinfo file
@cindex 结束 Texinfo 文件 @sortas{jsTexinfowj}
@cindex Texinfo file ending
@cindex Texinfo 文件结束
@cindex File ending
@cindex 文件结束 @sortas{wjjs}
```

3.8 节点描述

您可以通过在@node行后使用@nodedescription命令来提供节点的简短描述。此类描述可能会详细阐述或扩展节点名本身的信息。

您还可以使用@nodedescriptionblock...@end nodedescriptionblock环境来提供节点描述。这对于较长的描述可能很有用。

texi2any在为 Info 输出格式（及 HTML 格式，可选）输出菜单时，会使用您通过这些命令提供的描述。如果texi2any是自动生成菜单，或者没有在显式的@menu块中为菜单条目提供描述，则会使用节点的描述作为菜单条目的描述。请参阅请参阅 3.9 节[Menus], 30 页。

以下是使用这些命令的一个示例：

```
@node Tools
@chapter Tools
```

```
This chapter is on different tools you can use.
```

```
@node Screwdrivers
@nodedescription Flathead and Phillips.
```

```
@section Screwdrivers
```

```
This section is about screwdrivers.
```

```
@node Drills
```

```
@nodedescriptionblock
```

```
Making holes in things with power screwdrivers, drill drivers, combi  
drills, impact drivers, hammer drills, breakers and demolition drills.
```

```
@end nodedescriptionblock
```

```
@section Drills
```

```
This section is about drills.
```

在 Info 输出中, `texi2any` 会输出带有菜单的“Tools”节点, 如下所示:

```
* Menu:
```

```
* Screwdrivers::          Flathead and Phillips.
```

```
* Drills::               Making holes in things with power  
                          screwdrivers, drill drivers, combi  
                          drills, impact drivers, hammer drills,  
                          breakers and demolition drills.
```

3.9 菜单

菜单 (*Menus*) 包含指向下级节点的指针。在 Info 输出中, 您可以使用菜单转到这些节点。`texi2any` 可以在 HTML 输出中输出菜单, 但默认情况下不会这样做 (请参阅 21.8.1 节 [HTML Output Structure Customization], 189 页, `FORMAT_MENU`)。在打印手册或其他输出格式中, 菜单没有作用。

当输出带有节点菜单的 Info 或 HTML 时, 菜单可由 `texi2any` 自动生成, 条件是: 节点后都跟一个章节命令, 未使用显式的 `@menu` 块, 启用了自动指针。

通常, 让 `texi2any` 为您生成菜单会更方便, 因为这样在添加、删除或重新定位节点时, 您就不必在 Texinfo 源代码中更新菜单。在通常情况下, 如果手册采用层次结构组织, 且节点与章节命令相关联, 同时省略了节点指针, 那么只有在您希望精确控制 Info 中菜单的内容和格式时, 才应该编写菜单。

3.9.1 编写菜单

一个菜单由一行单独的 `@menu` 命令组成, 后面跟着菜单条目行或菜单注释行, 最后再由一行单独的 `@end menu` 命令结束。

菜单看起来是这样写的:

```
@menu
```

```
Larger Units of Text
```

```
* Files::                All about handling files.
```

```
* Multiples: Buffers.    Multiple buffers; editing  
                          several files at once.
```

```
@end menu
```

在菜单中, 每行以 ‘*’ 开头的文本都是菜单项。注意星号后的空格。

不以‘*’开头的行也可能出现在菜单中。这样的行不是菜单项，而是出现在 Info 文件中的菜单注释(menu comment)行。在上面的示例中，‘Larger Units of Text’这一行就是这样的菜单注释行；以‘*’开头的两行则是菜单项。

从技术上讲，菜单可以带您到达任何节点，无论文档的结构如何；甚至可以到达不同 Info 文件中的节点。但是，`texi2any`隐式指针创建功能（请参阅3.1节[Writing a Node], 24 页）和 GNU Emacs Texinfo 模式更新命令(updating commands)仅适用于在分层结构文档中创建子节点的菜单。在分层结构文档中，使用交叉引用来引用任意节点要好得多。

在 Info 中，用户使用`m(Info-menu)`命令选择一个节点。菜单项名称是用户在`m`命令后键入的内容。

在 HTML 输出中，`accesskey`(快捷键)属性与值‘1’...‘9’一起用于前九个条目。这使得使用网页浏览器的人能够（通常）通过按下`M-数字`来跟随第一个菜单条目，例如，按下`M-1`来选择第一个条目。

3.9.2 菜单示例

在 Texinfo 中，菜单看起来像这样写：

```
@menu
* 菜单条目名称: 节点名.    一个简短的描述。
* 节点名::                这个形式是优选。
@end menu
```

这产生：

```
* Menu:

* 菜单条目名称: 节点名.    一个简短的描述。
* 节点名::                这个形式是优选。
```

以下是一个示例，就像你在 Texinfo 文件中可能看到的那样：

```
@menu
Larger Units of Text

* Files::                All about handling files.
* Multiples: Buffers.    Multiple buffers; editing
                        several files at once.

@end menu
```

这产生：

```
* Menu:
Larger Units of Text

* Files::                All about handling files.
* Multiples: Buffers.    Multiple buffers; editing
                        several files at once.
```

在这个例子中，菜单包含两个条目。‘Files’既是菜单条目的名称，也是该名称所指节点的名称。‘Multiples’是菜单条目的名称，它指向名为‘Buffers’的节点。‘Larger Units of Text’这一行是注释；它出现在菜单中，但不是条目。

由于‘Files’或‘Buffers’中未指定文件名，因此它们必须是同一 Info 文件中节点的名称(请参阅3.9.6节[Referring to Other Info Files], 32 页)。

3.9.3 菜单位置

一个节点中最多只能有一个菜单。按照惯例，菜单位于节点的末尾，在@end menu和下一个节点的开头之间没有任何常规文本或附加命令。

这种约定很有用，因为使用菜单的读者很容易错过此类文本。此外，在 Info 输出中，任何此类菜单后的文本都将被视为菜单的一部分（Info 输出格式中，没有标记表示菜单的结束）。因此，以‘*’开头的行很可能会被错误处理。

通常，带有菜单的节点最好不要包含太多文本。如果你发现菜单前有大量文本，我们通常建议将除少数几个段落外的所有其它内容移至新的子节点中，否则，读者很容易忽略菜单。

3.9.4 菜单的组成部分

一个菜单条目包含三个部分，其中只有第二部分是必需的：

1. 菜单条目名称（可选）。
2. 节点的名称（必填）。
3. 条目描述（可选）。

通用菜单条目的模板如下（但更多可能性请见下一节）：

* 菜单条目(menu-entry-name): 节点名(node-name). 描述(description)

在菜单条目名称后加上一个冒号，在节点名后加上制表符、逗号、换行符，或者句点加空格（‘.’两个字符）。

菜单条目的第三部分是一个描述性的短语或句子。菜单条目名称和节点名通常都很简短；描述部分则向读者解释该节点是关于什么的。有用的描述是对节点名的补充，而不是重复。描述部分是可选的，可以跨越多行；如果跨越多行，一些作者倾向于将第二行缩进，而另一些作者则倾向于将其与第一行（以及所有其他行）对齐，这取决于你。一个空行或下一个菜单条目标志着描述部分的结束。

菜单中的空格字符在 Info 输出中保持原样；这使您可以根据需要设置菜单的格式。

texi2any在菜单条目的文本（以及节点名和交叉引用）包含可能干扰其在 Info 中解析的问题构造时，会发出警告并保护(节点)名称。请参阅[Info Node Names Constraints], 26 页。

3.9.5 简洁的菜单条目

当菜单条目名称和节点名相同时，您可以在行首的星号和空格后直接写入名称，并在名称后加上两个冒号。

示例如下：

* 名称:: 描述

替代：

* 名称: 名称. 描述

我们建议尽可能使用节点名称作为菜单条目名称，因为这样可以减少菜单中的视觉混乱。

3.9.6 引用其他 Info 文件

你可以创建一个菜单条目，让 Info 中的读者通过在节点名前的括号中写入文件名，从而跳转到另一个 Info 文件中的节点。一些示例如下：

@menu

* 第一个条目名(first-entry-name):(文件名(filename))节点名(nodename). 描述(description)

* (文件名(filename))第二个节点名(second-node):: 描述(description)

@end menu

例如，要直接引用《Emacs 手册》中的‘Outlining’和‘Rebinding’节点，你可以编写一个这样的菜单：

```
@menu
* Outline Mode: (emacs)Outlining. The major mode for
                                editing outlines.
* (emacs)Rebinding::           How to redefine the
                                meaning of a key.

@end menu
```

如果你没有列出节点名称，而只是指出了文件名，那么 Info 会假定你指的是“Top”节点。示例：

```
* Info: (info).               Documentation browsing system.
* (emacs)::                   The extensible, self-documenting
                                text editor.
```

GNU Emacs Texinfo 模式的菜单更新命令仅适用于当前缓冲区内的节点，因此您无法使用它们来创建引用其他文件的菜单。您必须手动编写此类菜单。请参阅节 D.5 [Updating Nodes and Menus], 241 页。

4 章节结构化

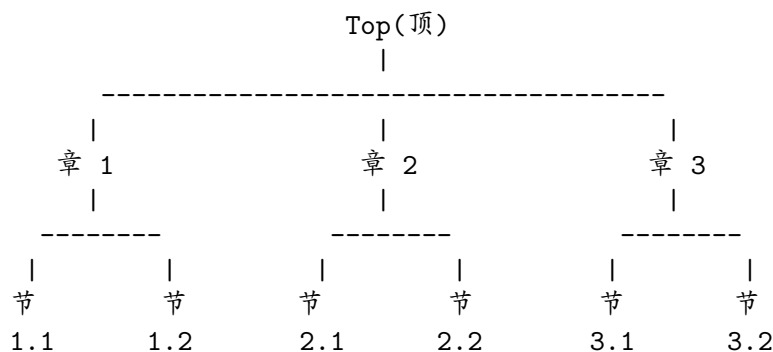
Texinfo 的章节结构化命令将文档划分为章、节、子节和子子节的层次结构。这些命令会在文本中生成类似上文所示的大标题。它们还提供用于生成目录的信息(请参阅2.9 节[Contents], 19 页)。

通常,您会在每个章节结构化命令之前,紧贴着输入一个@node命令。请参阅 第 3 章[Nodes], 24 页。

4.1 章节的树状结构

Texinfo 文件通常像一本书一样,由章、节、子节等构成。这种结构可以形象地表示为一棵树(或者更确切地说,是一棵倒置的树),树根位于顶部,各级分支分别对应章、节、节和子节。

这是一张图表,展示了一个包含三个章的 Texinfo 文件,每章包含两个节。



在具有这种结构的 Texinfo 文件中,第 2 章的开头会这样写:

```
@node 章 2
@chapter 章 2
```

为了举例说明,以下是用显式节点指针编写的代码:

```
@node 章 2, 章 3, 章 1, Top(顶)
@chapter 章 2
```

章节结构化命令将在后续章节中介绍;@node命令已在上一章(请参阅 第 3 章[Nodes], 24 页)中介绍。

4.2 结构化命令类型

章节结构化命令分为四组,每组都包含章、节、子节和子子节级别的结构化命令:

- 类似于@chapter的命令和类似于@appendix的命令会在文档正文及其目录中生成用数字(或字母)编号的条目。
- 类似@unnumbered的命令会在文档正文和目录中生成未编号的条目。@top命令具有特殊用途,也是这一组命令中的一员(请参阅3.5 节[@top Command], 27 页)。@unnumbered部分是文档结构的正常组成部分。
- 类似@heading的命令会生成简单的无编号标题,这些标题不会出现在目录中,与节点无关联,也无法进行交叉引用。这些标题命令永远不会另起一页。

在打印输出中，章节结构化命令会在文档中生成标题。当`@setchapternewpage`命令指示时，`@chapter`、`@unnumbered`和`@appendix`命令会在打印手册中开始新页面；而`@heading`命令则不会。请参阅节 E.1 [`@setchapternewpage`], 258 页。

在 Info 和纯文本输出中，该命令会使标题单独占一行，并在其下方插入一行 ASCII 字符（‘*’、‘=’、...）。例如，“章节结构化”标题可以是：

```
4 章节结构化
```

```
*****
```

同一级别的所有命令所使用的下划线字符是相同的。例如，章级命令`@chapter`、`@appendix`、`@unnumbered`和`@chapheading`所使用的下划线字符都是相同的。

在 HTML 中，章级命令默认会生成一个

级别的标题（由`CHAPTER_HEADER_LEVEL`定制化变量控制，请参阅21.8.6 节[HTML Customization of Specific Output], 195 页）。对于其他命令，标题元素的级别会进行调整。

在 DocBook 输出中，会使用适当级别的元素。生成的元素包含所有后续部分，直至同一级别或更高级别的下一个命令。例如，对于`@chapter`，会生成一个`<chapter>`元素，并包含该章节中的所有部分或子部分。

以下是总结：

数字编号 在目录中	无编号 在目录中	字母或数字编号 在目录中	不从新页开始 无编号 不在目录中
<code>@chapter</code>	<code>@top</code>	<code>@appendix</code>	<code>@majorheading</code>
<code>@section</code>	<code>@unnumbered</code>	<code>@appendixsec</code>	<code>@chapheading</code>
<code>@subsection</code>	<code>@unnumberedsec</code>	<code>@appendixsubsec</code>	<code>@heading</code>
<code>@subsubsection</code>	<code>@unnumberedsubsec</code>	<code>@appendixsubsubsec</code>	<code>@subheading</code>
			<code>@subsubheading</code>

4.3 @chapter: 章的结构化

`@chapter`用于标识文档中的章，这是文档结构层次中的最高级别。在行首输入该命令，并在同一行后接章标题。章会自动编号，从 1 开始。

例如，本手册中的本章标题为“章节结构化”；`@chapter` 代码行如下所示：

```
@chapter 章节结构化
```

4.4 @unnumbered, @appendix: 使用其它标记的章

使用`@unnumbered`命令来开始一个章级元素，该元素不显示任何形式的章编号。使用`@appendix`命令来开始一个附录，该附录以字母（‘A’、‘B’、...）而非数字进行标记；附录也在章级别。

在行首编写`@appendix`或`@unnumbered`命令，并在同一行后跟标题，就像`@chapter`一样。

Texinfo 还提供了一个命令`@centerchap`，该命令类似于`@unnumbered`，但会在打印输出和 HTML 输出中居中显示其参数。Texinfo 通常不提供这种样式选择。建议您不要使用此命令，因为它在 Texinfo 的未来版本中可能会被移除。

使用`@unnumbered`时，如果关联节点的名称是以下英文单词之一（不区分大小写）：

```
Acknowledgements Colophon Dedication Preface
```

那么，DocBook 输出将使用相应的特殊标签（如`<preface>`等），而不是默认的`<chapter>`。`@unnumbered`本身的参数可以是任何内容，并且会像往常一样作为元素标题输出。

4.5 @majorheading, @chapheading: 章级标题

@majorheading和@chapheading命令可在文档正文中生成类似章的标题。

然而，这两个命令都不会在目录中生成条目，也不会使 T_EX 在打印的手册中另起一页。

在 T_EX 中，@majorheading命令在标题前生成的垂直空白比@chapheading命令更大，但其他方面两者相同。

在其他输出格式中，@majorheading和@chapheading命令的输出与@chapter相似。不同之处在于它们没有编号，且与节点没有任何关联。请参阅4.3 节[@chapter], 35 页。

4.6 @section: 章里的节

@section命令用于标识章里的节，无论该章是使用@chapter、@unnumbered还是@appendix创建的，都遵循章级命令的编号方案。因此，在编号为“1”的@chapter章中，各节的编号为“1.1”、“1.2”等；在标记为“A”的@appendix “章”中，各节的编号为“A.1”、“A.2”等；在@unnumbered章中，节(section)不编号。

要创建一个节，请在一行的开头输入@section命令，并在同一行后输入节标题。例如：

```
@section 这是一个节
```

节标题将列在目录中。

4.7 @unnumberedsec, @appendixsec, @heading

@unnumberedsec、@appendixsec和@heading命令分别是@section命令（见上一节）的无编号、类似附录和类似标题的等效命令。

在一般情况下，无需使用@unnumberedsec和@appendixsec，因为@section也可以用在@unnumbered和@appendix章节中；同样，请参阅上一节。

@unnumberedsec

可用于未编号的章或常规章或附录中，以生成未编号的节。

@appendixsec

@appendixsection

两者是同义词。按照惯例，这两个命令仅在附录中使用。

@heading 您可以在（几乎）任何地方使用@heading命令来创建不会出现在目录中的节级别样式的标题。@heading系列命令可以出现在大多数环境中，但不允许出现在诸如作为另一命令的参数等病态且无用的位置。

4.8 @subsection: 节里的子节

子节之于节，犹如节之于章；请参阅4.6 节[@section], 36 页。

示例：

```
@subsection 这是一个子节
```

子节标题将列在目录中。

4.9 类@subsection命令

@unnumberedsubsec、@appendixsubsec和@subheading命令分别是@subsection命令的未编号、类似附录和类似标题的等效命令。（请参阅4.8节[@subsection], 36 页。）

在一般情况下，无需使用@unnumberedsubsec和@appendixsubsec，因为@subsection也可以在@unnumbered和@appendix章的各个节中使用（请参阅4.6节[@section], 36 页）。

@subheading命令会生成一个类似于子节的标题，但它没有编号，也不会出现在目录中。同样地，@unnumberedsubsec命令会生成一个类似于子节的未编号标题，而@appendixsubsec命令会生成一个类似于子节的标题，但会使用字母和数字进行标记；这两个命令生成的标题都会出现在目录中。

4.10 @subsubsection及其它‘Subsub’命令

Texinfo 中的第四级也是最低级的章节命令是‘...subsub...’命令。它们是：

@subsubsection

子子节与子节的关系，如同子节与节的关系。（请参阅4.8节[@subsection], 36 页。）子子节标题会出现在目录中。

@unnumberedsubsubsec

未编号的子子节标题，会在目录中出现，但这些标题没有编号。除此之外，未编号的子子节与子子节相同。

@appendixsubsubsec

按照惯例，附录(子子节)命令仅用于附录部分，并会进行适当的字母排序和编号。它们也会出现在目录中。

@subsubheading

@subsubheading 命令可用于任何需要小标题的地方，该小标题不会出现在目录中。

与节一样，在一般情况下，无需使用@unnumberedsubsubsec和@appendixsubsubsec，因为@subsubsection也可以在@unnumbered和@appendix的章中使用（请参阅 4.6 节[@section], 36 页）。

4.11 @part：由章构成的组

最后一个章节命令是@part，用于标记手册的部分，即一组章或（很少见的）附录。这与其他章节命令表现截然不同，以符合书籍中此类‘部分’的常规理解及使用方式。

没有与@part相关的@node命令。只需在文档中您想要标记为该部开始的地方，单独另起一行，写出命令，并在随后写部分标题。例如：

```
@part 第一部分：@* 开始
```

从这个例子可以推断，@part文本并未进行自动编号或标记。文本是原样保留的。

由于部与节点没有关联，因此@part行后面不能跟通用文本。要生成预期的输出，它后面必须跟一个章级命令（包括其节点）。因此，继续这个例子：

```
@part 第一部分：@* 开始
```

```
@node 介绍
```

```
@chapter 介绍
```

...

在 T_EX 输出中，@part 文本同时包含在正常目录和简要目录（请参考 2.9 节 [Contents], 19 页）中，且没有页码（因为这是常规做法）。此外，文档正文中还会输出一个“部分”页面，其中仅包含 @part 的标题。在上述示例中，@* 会在“部分”页面上换行（但在目录中会被替换为空格）。无论章的页码编排如何（请参阅节 E.1 [setchapternewpage], 258 页），此页始终强制位于奇数页（右侧）。在 L^AT_EX 输出中，@part 被输出为 \part。

在 HTML 输出中，@part 的标题同样会包含在目录中，并且作为随后章或附录的节点的一部分，在主文档文本中也会包含一个该标题。

在 DocBook 输出中，<part> 元素包含以下所有章节，直至下一个 <part>。包含章的 <part> 也会在附录处结束。

在 Info 和纯文本输出中，@part 没有任何效果。

在升高或降低章节（见下一节）时，@part 会被忽略。也就是说，它永远不会降低，也无法被升高。

4.12 升高/降低章节：@raisesections 和 @lowersections

@raisesections 和 @lowersections 命令会隐式地提升和降低后续章、节和其他章节命令的层级，但不包括 4.11 节 [part], 37 页命令。

即：@raisesections 命令将节更改为章，将子节更改为节，依此类推。相反：@lowersections 命令将章更改为节，将节更改为子节，依此类推。因此，一个 @lowersections 命令可以取消一个 @raisesections 命令，反之亦然。

实际上，通常您只想提升或降低大块的内容，这些内容通常位于外部文件中。您可以使用 @lowersections 将作为外部或独立 Texinfo 文件编写的文本作为内部包含文件包含到另一个 Texinfo 文件中（请参阅第 17 章 [Include Files], 140 页）。典型用法如下：

```
@lowersections
@include somefile.texi
@raisesections
```

（如果没有 @raisesections，主文件中的所有后续部分也会被降低。）

如果被降低的包含文件包含一个 @top 节点，则需要使用一个标志（请参阅 15.5.1 节 [set @value], 125 页）进行条件化处理。

最终结果中的任何菜单都必须考虑到上升和下降的情况，因此，在文档中随意散布 @raisesections 和 @lowersections 命令很可能会导致错误（除非文档中的菜单都是自动生成的）。

重复使用这些命令可以逐级提升或降低层次级别。若尝试提升到“章”级别以上，则会再次生成章命令；若尝试降低到“子子节”级别以下，则会再次生成子子节命令。此外，由于菜单结构无法正确表示，降低后的子子节和提升后的章节将无法与 texi2any 隐式确定节点指针的功能一起使用。

将每个 @raisesections 和 @lowersections 命令单独占一行。

5 交叉引用

交叉引用 (Cross-references) 用于指引读者查看同一或不同 Texinfo 文件的其他部分。

使用交叉引用可帮助读者获取对于当前上下文而言过于详细或偶然相关的信息。在线帮助系统或参考手册不同于小说，很少有人会从头到尾按顺序阅读这类文档。相反，人们会查找他们需要的内容。因此，这类创作应包含大量交叉引用，以帮助读者找到他们可能未读过的其他信息。

在打印手册中，交叉引用会指向具体的页面，除非该引用指向的是另一本手册，此时交叉引用会指明另一本手册的名称。在 Info 中，交叉引用会生成一个条目，您可以使用 Info 的‘f’命令来跟踪该条目。（请参阅节“Following cross-references”在 Info。）在 HTML 中，交叉引用会生成一个超链接。在 DocBook 中，会使用<link>元素来表示交叉引用，除非交叉引用指向的是另一本手册（此时交叉引用会指明另一手册的名称）。

各种交叉引用命令使用节点（请参阅[@node], 24 页）、锚点（请参阅5.11 节[@anchor], 44 页）或浮动标签（请参阅9.1.1 节[@float], 74 页）来定义交叉引用的位置。当 TeX 生成 DVI 文件时，它会记录每个交叉引用位置的页码，并在引用时使用这些页码。因此，即使你编写的是一本仅供打印、不在线使用的手册，你也必须编写@node行（或@anchor锚点），以便命名你制作交叉引用时所用的目标位置。

5.1 不同的交叉引用命令

有三种不同的交叉引用命令：

- @xref** 在句子开头的交叉引用，Info 输出格式中，用‘*Note name: node.’表示，在其他输出格式中用‘See ...’表示。
- @ref** 用于句中，或更常见的是用于句末，生成一个 Info 交叉引用，格式为‘*note name: node’，在其他输出格式中，仅显示交叉引用，而不在前面显示“See”。
- @pxref** 在括号内、句末或标点符号前使用，以作引用。在 Info 格式中，其输出以小写‘*note’开头，在其他输出格式中则以小写“see”开头。（‘p’代表‘parenthesis(括号)’。）

此外，Texinfo 系统还提供了用于生成对外部文档引用的命令。@cite命令用于引用书籍和手册。@url则用于生成一个URL，例如引用万维网上的某个页面。

5.2 交叉引用的组成部分

交叉引用命令仅一个参数是必需的，即它所引用的节点的名称。交叉引用命令最多可包含四个附加参数。完整的五个参数的交叉引用模板如下：

```
@xref{node-name, online-label, printed-label, manual-name, printed-manual-title}
```

交叉引用的五个参数为：

1. 节点或锚点名称(node-name)。这是交叉引用所指向的位置。在打印输出中，对于同一文档内引用，节点的位置仅提供页码参考。使用@node(请参阅3.1 节[Writing a Node], 24 页)定义节点，使用@anchor(请参阅5.11 节[@anchor], 44 页)定义锚点，使用@float(请参阅9.1.1 节[@float], 74 页)定义带有标签的浮动。此参数为必填项（引用整个手册时除外）。

在交叉引用中书写节点名的方式，必须与@node行中的方式完全相同，包括大小写；否则，处理器可能无法找到该引用。

2. 在线输出的标签(online-label)。通常该标签会被省略；若已指定主题描述（第三个参数），则使用主题描述；若主题描述也省略了，则使用节点名称。
3. 打印输出的标签(printed-label)。通常，这是章节的标题或主题。在打印手册中，这用作引用的名称。如果省略，则使用节点名称。
4. 如果所引用的手册不在当前手册中，而是在另一个 Texinfo 文件中，则应提供该手册的名称(manual-name)。
5. 引用的打印手册的标题(printed-manual-title)，来自另一个 Texinfo 文件。

分隔这些参数的逗号前后多余的空格将被忽略。若要在某个参数中包含逗号，请使用`@comma{}`（请参阅 11.1.3 节[Inserting a Comma], 86 页）。

以下各节分别描述了具有一个、两个、三个、四个和五个参数的交叉引用。

在使用 TeX 处理时，除非参数的右括号后紧接非空格（如逗号或句号），否则在同一手册中的交叉引用中，页码后会自动插入一个逗号。这使您可以选择在其他输出格式中是否在此处使用逗号。例如：

```
@xref{Another Section} for more info
```

在打印输出中，生成：‘See Another Section, page *ppp*, for more info’，在 Info 输出中，生成：‘*Note Another Section:: for more info’。

如果不希望的逗号被加入，请在参数后加上一个命令，如‘@:’。例如，‘@xref{Hurricanes}@: --- for the details’生成：

```
See Hurricanes, page ppp — for the details
```

而不是：‘See Hurricanes, page *ppp*, — for the details’。

当交叉引用的文本（以及节点的名称和菜单的条目）包含可能干扰其在 Info 中解析的问题结构时，`texi2any`会发出警告并保护相关名称。请参见：[Info Node Names Constraints], 26 页。

5.3 带一个参数的@xref

`@xref`的最简单形式接受一个参数，即同一 Texinfo 文件中另一个节点的名称。

示例：

```
@xref{Tropical Storms}.
```

在 Info 输出中，生成：

```
*Note Tropical Storms:..
```

在打印输出中，生成：

```
See Section 3.1 [Tropical Storms], page 24.
```

5.4 带两个参数的@xref

有两个参数，第二个参数用作在线输出的标签。

模板是这样的：

```
@xref{node-name, online-label}.
```

示例：

```
@xref{Electrical Effects, Lightning}.
```

在 Info 输出中，生成：

```
*Note Lightning: Electrical Effects.
```

在打印输出中，生成：

```
See Section 5.2 [Electrical Effects], page 57.
```

节点名被打印出来。

交叉引用的第二个参数与节点名有着一样的约束条件。在此上下文中，可能引起问题的字符是冒号。请参阅：[Info Node Names Constraints], 26 页。

5.5 带三个参数的@xref

第三个参数用于替换输出中的节点名。第三个参数应为打印输出中的相应部分的名称，或者说明该部分所讨论的主题。

模板是这样的：

```
@xref{node-name, online-label, printed-label}.
```

示例：

```
@xref{Electrical Effects, Lightning, Thunder and Lightning},  
for details.
```

在 Info 输出中，生成：

```
*Note Lightning: Electrical Effects, for details.
```

在打印输出中，生成：

```
See Section 5.2 [Thunder and Lightning], page 57, for details.
```

如果给出了第三个参数且第二个参数为空，则第三个参数同时用于这第二及第三参数。（注意，两个逗号连续使用，用以标记第二个参数是空的。）例如：

```
@xref{Electrical Effects, , Thunder and Lightning}, for details.
```

在 Info 输出中，生成：

```
*Note Thunder and Lightning: Electrical Effects, for details.
```

在打印输出中，生成：

```
See Section 5.2 [Thunder and Lightning], page 57, for details.
```

交叉引用的第三个参数与节点名有着共同的约束条件。在此上下文中，可能引起问题的字符是冒号。请参阅：[Info Node Names Constraints], 26 页。

在实际操作中，如果节点名和交叉引用标题相同（或非常相似），通常最好只使用第一个参数来编写交叉引用；如果节点名和交叉引用标题不同，则最好只使用第一个和第三个参数。

Texinfo 提供了一种设置，在交叉引用中默认呈现章节标题(section title)而不是节点名（但明确指定的第三个参数仍然优先）：

```
@xrefautomaticsectiontitle on
```

通常，此行会出现在文档的开头附近，并用于整个手册。但如果你希望关闭它（例如，如果你包含了一些没有合适章节标题的其他子文档），可以使用@xrefautomaticsectiontitle off。

此设置也适用于 HTML 中的节点标题(node title)，如果@xrefautomaticsectiontitle处于启用状态，则尽可能在节点标题中使用章节标题，而不是节点名称。

5.6 带四个和五个参数的@xref

在交叉引用中，第四个参数指定了另一本手册的名称，该手册不是当前文件，第五个参数则指定打印手册的标题。

完整的模板是：

```
@xref{node-name, online-label, printed-label, manual-name,
      printed-manual-title}.
```

例如：

```
@xref{Electrical Effects, Lightning, Thunder and Lightning, weather,
      An Introduction to Meteorology}.
```

在 Info 输出中产生：

```
*Note Lightning: (weather)Electrical Effects.
```

如您所见，手册名称(manual-name)被括在括号中，并位于节点名称(node-name)之前。在 HTML 中，手册名称(manual-name)和节点名(node-name)用于构建超链接 URL（请参阅 21.7 节[HTML Xref], 183 页），而链接文本则基于标签(online-label)。

在打印输出中产生：

```
See section "Thunder and Lightning" in An Introduction to Meteorology.
```

打印手册标题(printed-manual-title)排版类似于@cite；并且由于引用指向另一本手册时，无法确定所引用的具体页码，因此该引用不提供页码。

下一个案例：通常，当您使用@xref的长格式时，会省略第二个参数。在这种情况下，第三个参数（主题描述）将作为在线格式中的交叉引用标题。例如：

```
@xref{Electrical Effects, , Thunder and Lightning, weather,
      An Introduction to Meteorology}.
```

在 Info 输出中产生：

```
*Note Thunder and Lightning: (weather)Electrical Effects.
```

在打印输出中产生：

```
See section "Thunder and Lightning" in An Introduction to Meteorology.
```

下一个案例：如果节点名和章节标题在其他手册中相同，也可以省略章节标题。在这种情况下，两个场合都使用节点名称。例如：

```
@xref{Electrical Effects,,, weather, An Introduction to Meteorology}.
```

在 Info 输出中产生：

```
*Note (weather)Electrical Effects:..
```

在打印输出中产生：

```
See section "Electrical Effects" in An Introduction to Meteorology.
```

一般来说，除非不打算生成打印手册，没有理由只设置手册名而不设置手册标签。多个 Texinfo 文件可以被合并到同一本打印手册中，同时这些文件还可以输出其它格式的独立文件；如果您还需要引用同一本打印手册中的另一个手册文件，只需指定第四个参数，而无需指定第五个参数。如果省略打印手册标签，则将在打印输出中使用手册名。

如果只提供打印手册标签，而没有提供在线标签，除非 Texinfo 源文件只生成打印手册，否则打印手册标签几乎没什么用。在线格式的输出结果取决于具体格式，例如，它可能是一个空的手册名，或者是一个对（以打印输出类似的方式格式化的）打印手册的引用。

最后，除了第四和第五个参数外，也可以省略所有其他参数，以整体引用另一本手册。请参阅下一节。

5.7 手册的整体引用

通常，在交叉引用中必须始终为节点命名。然而，有时我们希望引用的是整本手册，而非其中的某个特定章节。在这种情况下，给出任何章节名称都是不必要的干扰。

因此，根据对其他手册的交叉引用（请参阅：5.6 节[Four and Five Arguments], 42 页），如果第一个参数是‘Top’（仅以这种方式大写）或完全省略，且第三个参数也省略，则打印输出中不会包含节点或章节名称。（如果提供了‘Top’，则 Info 输出中会包含。）例如，

```
@xref{Top,,, make, The GNU Make Manual}.
```

在 Info 输出中产生：

```
*Note (make)Top::.
```

在打印手册中产生：

```
See The GNU Make Manual.
```

无论是否显式指定‘Top’节点，Info 阅读器都会跳转到手册的‘Top’节点。

通过指定‘Top’节点并为@xref命令的第三个参数指定一个适当的条目，也可以（这是历史上的做法）引用整个手册。按这种习惯用法，若要交叉引用*The GNU Make Manual*，可以这样写：

```
@xref{Top,, Overview, make, The GNU Make Manual}.
```

在 Info 输出中产生：

```
*Note Overview: (make)Top.
```

在打印手册中产生：

```
See section “Overview” in The GNU Make Manual.
```

在这个例子中，‘Top’是第一个节点的名称，‘Overview’是手册第一章节的名称。对于打印手册中第一章节的命名方式，并没有广泛接受的惯例，这只是 Make 手册碰巧使用的命名方式。第一章节的名称的随意性是全手册交叉引用中省略第三个参数更可取的主要原因。

5.8 @xref

@xref 命令用于为句首生成交叉引用。使用@xref的示例见前文。¹

5.9 @ref

@ref与@xref几乎相同，只是它不会在输出中生成“See”，而只生成引用本身。这使得它作为句子的最后一部分非常有用。

例如，

```
For more information, @pxref{This}, and @ref{That}.
```

在 Info 输出中产生：

```
For more information, *note This::, and *note That::.
```

在打印输出中产生：

```
For more information, see Section 1.1 [This], page 1, and Section 1.2 [That],  
page 2.
```

@ref命令可能会诱使作者以适合打印手册的方式表达自己，但在 Info 格式中看起来却显得笨拙。请记住，您的受众可能同时使用打印版和其他输出格式（如：Info）²。例如：

¹ 译者注：在中文 Texinfo 的打印输出中，‘见’替代‘See’及‘see’。

² 译者注：尽管不受推荐，但本手册仍使用适合打印的方式表达，大量使用‘请参阅@ref...’，并尽可能代替原文中的@pxref及@xref，原因请参阅：[Convention], 312 页。


```
Sea surges are described in @ref{Hurricanes}.
```

在打印输出中，看起来很好：

```
Sea surges are described in Section 6.7 [Hurricanes], page 72.
```

但在 Info 中阅读起来很不舒服，“note”被用作动词：

```
Sea surges are described in *note Hurricanes::.
```

5.10 @pxref

括号内的引用命令@pxref，与@xref几乎相同，但最好在括号内使用。该命令与@xref的不同之处在于，使用用小写的“see”而不是大写的“See”来排版的。在 Info 文档中，会输出‘*note’。

在只有一个参数的情况下，括号内的交叉引用看起来像这样：

```
... storms cause flooding (@pxref{Hurricanes}) ...
```

在 Info 输出中产生：

```
... storms cause flooding (*note Hurricanes::) ...
```

在打印手册中产生：

```
... storms cause flooding (see Section 6.7 [Hurricanes], page 72) ...
```

在 Texinfo 的早期版本中，不允许在@pxref后添加标点符号，因此它只能用在右括号前。现在已不再如此。‘@pxref{node-name}’的效果与‘see @ref{node-name}’相似，在许多情况下，后者更可取，因为后者在 Info 输出中明确显示应出现的“see”一词。

5.11 @anchor：定义任意的交叉引用目标

锚点(anchor) 是文档中的一个位置，被标记出来以便交叉引用可以指向它，就像它们可以指向节点一样。您可以使用@anchor命令来创建锚点，命令的参数由普通的括号分隔，参数提供锚点名。例如：

```
This marks the @anchor{x-spot}spot.
...
@xref{x-spot,,the spot}.
```

产生：

```
This marks the spot.
...
See [the spot], page 1.
```

如您所见，@anchor命令本身不会产生任何输出。本例在单词“spot”之前定义了一个锚点“x-spot”。您可以在之后使用@xref或其他交叉引用命令来引用它，请参阅第 5 章[Cross References], 39 页。

最好将@anchor命令放在您希望引用的位置之前；这样，当读者跳转到锚点时，他们的目光就会被引导到正确的文本上。如果有助于提高源代码的可读性，您可以将@anchor命令单独放在一行。@anchor之后的空白（包括换行符）将被忽略。锚点名、节点名和浮动标签名不能冲突。在某些方面，锚点、节点和浮动标签的处理方式相似；例如，goto-node 命令可以接受锚点名或节点名称作为参数。（请参阅：节 “Go to node” 在 Info）。锚点名和浮动标签名也可能出现在菜单（请参阅：3.9 节[Menus], 30 页）和节点指针（请参阅：3.1 节[Writing a Node], 24 页）中，但不建议这样做。

锚点名与节点名共享相同的约束条件（请参阅：[Info Node Names Constraints], 26 页）。

由于这种双重性，当您删除或重命名一个节点时，通常建议使用旧名称定义一个@anchor。这样，任何指向旧节点的链接，无论是来自其他 Texinfo 手册还是普通网页，都能继续正常工作。

5.12 @link: 普通、无修饰的超链接

@link 在支持它的输出格式（包括 HTML、DocBook、L^AT_EX 和在线 PDF）中生成一个普通超链接。模板为：

```
@link{node-name, label, manual-name}
```

node-name 是目标节点或锚点的名称。*label* 和 *manual-name* 可以省略其中一个或全部。如果给出了 *label*，则它是用于链接的文本。*manual-name* 是目标节点所在的外部手册的名称；如果未给出，则引用的是当前手册。

@link 的输出与 @ref 相似，不同之处在于，在 Info 或打印输出中，它不会在链接标签周围生成任何额外的文本（这些文本会将其标记为交叉引用）。

在使用 @link 生成用户浏览手册所必需的链接时，请务必小心，因为这些链接在某些输出格式中不起任何作用。@link 最适合用于添加便利链接，但这些链接对于读者理解手册文本并非必不可少。例如，您可能在代码示例中使用 @link 来引用编程库中某个符号的文档。

5.13 @inforef: 对纯 Info 资料的交叉引用

@inforef 用于对 Info 文档进行交叉引用，甚至可以从打印手册中引用。该命令最初用于非 Texinfo 源文件生成的 Info 文件。现在该命令已过时，不应再使用。除了用途不大之外，使用 @xref、@ref 或 @pxref，将 Info 文件名作为第四个参数且不指定第五个参数，也可以获得类似的输出。

该命令接受两个或三个参数，参数顺序如下：

1. 节点名。
2. 交叉引用名（可选）。
3. Info 文件名。

模板如下：

```
@inforef{node-name, cross-reference-name, info-file-name}
```

5.14 @url, @uref{ url [, text] [, replacement] }

@url 会生成一个对 URL（对统一资源定位符）的引用。它接受一个必选参数，即 URL，以及两个可选参数，用于控制显示的文本。在 HTML 和 PDF 输出中，@url 会生成一个可点击的链接。（若要仅指示一个 URL 而无需创建可点击的链接，请使用 @indicateurl，请参阅 6.1.15 节 [@indicateurl]，55 页。）

@uref 是 @url 的同义词。最初，@url 的含义是 @indicateurl，并且需要 @uref 来生成一个可用的链接，但实际上 @url 几乎总是被误用。因此，我们改变了其含义。

第二个参数（如果指定）是要显示的文本（默认为 URL 本身）；在 HTML 以外的输出格式中，除该文本外还会输出 URL 本身。

如果指定了第三个参数，则该参数为要显示的文本，但在此情况下，URL 不会以任何格式输出。这在文本本身已经具有足够参考价值时非常有用，例如在手册页中。此外，如果指定了第三个参数，则第二个参数将被忽略。

5.14.1 @url 示例

首先, 这里是一个只有一个参数的@url的最简单形式的示例。给定的 URL 既是链接的目标地址, 也是链接的可见文本:

GNU 的官方 ftp 站址是 @url{http://ftp.gnu.org/gnu}.

产生:

GNU 的官方 ftp 站址是 <http://ftp.gnu.org/gnu>.

@url 的双参数形式

以下是双参数形式的示例:

官方的 @url{http://ftp.gnu.org/gnu, GNU ftp site} 上保存有程序及文档。

产生:

官方的 GNU ftp 站点 (<http://ftp.gnu.org/gnu>) 上保存有程序及文档。

HTML 输出是这样的:

官方的 [GNU ftp 站点](http://ftp.gnu.org/gnu) 上保存有程序及文档。

在其它输出格式, 是像这样的:

官方的 GNU ftp 站点 (<http://ftp.gnu.org/gnu>) 上保存有程序及文档。

@url 的三参数形式

最后, 举一个三参数形式的例子:

The @url{/man.cgi/1/ls,,ls} program ...

除 HTML 外, 可生成:

The ls program ...

但是使用 HTML:

The [ls](/man.cgi/1/ls) program ...

顺便说一句, 有些人更喜欢以明确的格式显示网址:

<URL:http://host/path>

如果你愿意, 可以在输入文件中使用这种格式。我们认为在输出中包含‘<URL:’和‘>’是不必要的, 因为任何试图在文本中检测 URL 的软件, 在不带‘<URL:’的情况下也必须能够检测到它们, 这样才有用。

5.14.2 URL 中的换行

T_EX 仅允许在 URL 中的几个字符 (这些字符在 URL 中具有特殊意义) 处换行: ‘&’、‘.’、‘#’、‘?’和‘/’ (但两个连续的‘/’字符之间不能换行)。此外, 在这些字符周围还会插入少量可拉伸的空间, 以辅助换行。

对于 HTML 输出, 现代浏览器还会在显示的 URL 中自动换行。如果你需要在其他字符处换行, 可以根据需要插入@/(请参阅: 12.2 节[Line Breaks], 101 页)。

在默认情况下, 在 T_EX 中, 特殊字符处的任何此类断行都会发生在字符之后。有些人更倾向于在特殊字符之前进行此类断行。这可以通过@urefbreakstyle 命令来控制 (此命令仅在 T_EX 中有效):

@urefbreakstyle how

其中参数`how` 是以下单词之一：

- ‘after’ （默认）在特殊字符之后可能进行换行。
- ‘before’ 在特殊字符之前可能发生换行。
- ‘none’ 完全不考虑在特殊字符处断行；任何潜在的换行都必须手动插入。

5.14.3 @url, PDF 输出格式

如果 PDF 的最终目的只是在线查看，或许在某种程度上类似于早期的 HTML，那么你可能不希望在可见文本中包含 URL（就像网页的读者看不到 URL 一样）。Texinfo 为此提供了一个特定于 PDF 的选项，该选项必须在`@tex`中使用：

```
@tex
\global\urefurlonlylinktrue
@end tex
```

结果是，`@url{http://www.gnu.org, GNU}` 的可见输出仅为“GNU”，其链接目标为`http://www.gnu.org`。通常，可见输出应同时包含标签和 URL：“GNU(`http://www.gnu.org`)”。

此选项仅在使用 pdfTeX 程序生成 PDF 输出时有效，不适用于从 Texinfo 到 PDF 的其他转换方式（例如，从 TeX 到 DVI 再到 PDF）。因此，如上例，在`@tex`中无条件指定此选项是正确的，可以保证在生成 DVI 时，此选项将被忽略。

5.15 @cite{ 引用 }

对于没有配套 Info 文件的书籍名称，请使用`@cite`命令。例如，我们可以引用 *A Book* 。该命令会在打印手册中选择斜体字体，并在 Info 文件中生成引号。

如果一本书是用 Texinfo 编写的，那么最好使用交叉引用命令，因为读者可以轻松地在 Info 中跟随这样的引用。请参阅 5.8 节 [`@xref`]，43 页。

5.16 PDF 的色彩

默认情况下，URL 和交叉引用链接在 PDF 输出中以黑色显示。然而，在极少数情况下，您可能希望像网页上常见的那样，用不同的颜色高亮显示这些“活动”链接。Texinfo 提供了一个特定于 PDF 的选项来指定这些颜色，该选项必须在`@tex`中使用：

```
@tex
\global\def\linkcolor{1 0 0} % red
\global\def\urlcolor{0 1 0} % green
@end tex
```

`\urlcolor`用于更改`@url`输出的颜色（包括实际 URL 和任何文本标签），而`\linkcolor`则用于更改对节点的交叉引用的颜色等。它们是独立的。

给出的三个值必须是介于‘0’和‘1’之间的数字，分别表示红色、绿色和蓝色的分量。

这些定义仅在通过 pdfTeX 程序生成 PDF 输出时有效，而在其他从 Texinfo 到 PDF 的转换方式（例如，TeX 到 DVI 再到 PDF）中则无效。因此，如上所示，在`@tex`中无条件地指定此选项是可以的。在生成 DVI 时，此选项将被忽略。

我们不建议仅仅为了好玩而着色；除非你有特定的理由需要使用颜色，否则最好还是不要这样做。

6 标示文本、单词和短语

在 Texinfo 中，你可以用多种方式标记单词和短语。Texinfo 处理器会利用这些信息来决定如何突出显示文本。例如，你可以指定一个单词或短语是定义性的出现、元句法变量，还是在程序中使用的符号。此外，你还可以用几种不同的方式来强调文本。

6.1 对定义、命令等进行标示

Texinfo 具有用于指示一段文本所指对象类型的命令。例如，电子邮件地址用 `@email` 标记；这样，当输出格式支持时，结果可以是一个可激活的发送电子邮件的链接。如果电子邮件地址只是简单地标记为“以打字机字体打印”，则无法实现这一点。

6.1.1 突出显示命令很有用

这些命令有多种用途：

`@code{示例代码}`

标示作为程序片段示例的文本。请参阅6.1.2 节[`@code`]，49 页。

`@kbd{键盘字符}`

标示键盘输入。请参阅6.1.3 节[`@kbd`]，50 页。

`@key{键名}`

指出键盘上某个键的常规名称。请参阅6.1.4 节[`@key`]，50 页。

`@samp{文本}`

标示出作为字符序列示例的文本。请参阅6.1.5 节[`@samp`]，51 页。

`@verb{文本}`

逐字逐句地写出字符序列。请参阅6.1.6 节[`@verb`]，51 页。

`@var{元句法变量}`

标示一个元句法变量。请参阅6.1.7 节[`@var`]，52 页。

`@env{环境变量}`

标示一个环境变量。请参阅6.1.8 节[`@env`]，53 页。

`@file{文件名}`

标示文件名。请参阅6.1.9 节[`@file`]，53 页。

`@command{命令名}`

标示一个命令的名称。请参阅6.1.10 节[`@command`]，53 页。

`@option{选项}`

标示一个命令行选项。请参阅6.1.11 节[`@option`]，53 页。

`@dfn{term}`

标识术语的介绍或定义。请参阅6.1.12 节[`@dfn`]，53 页。

`@cite{引用}`

标识一本书的名字。请参阅5.15 节[`@cite`]，47 页。

`@abbr{缩写}`

标识一个缩写，如 `Comput.`。

@acronym{缩略语}

标识一个缩略语。请参阅6.1.14 节[@acronym], 54 页。

@indicateurl{统一资源定位符}

标识一个示例性（即非功能性）的统一资源定位符。请参阅6.1.15 节[@indicateurl], 55 页。对于可激活的（功能性的）URL，请使用@url（请参阅5.14 节[@url], 45 页）。

@email{电子邮件地址[, 显示文本]}

标识一个电子邮件地址。请参阅6.1.16 节[@email], 55 页。

6.1.2 @code{ 示例代码 }

使用@code命令来标识程序代码，并且由完整的语法标记组成。将程序代码括在大括号中。

因此，对于程序中的表达式、程序中使用的变量名或函数名，或者编程语言中的关键字，你应该使用@code。

在类似于编程语言的语言（如 Texinfo ）中，使用@code表示命令名。例如，在 Texinfo 源代码中，分别编写‘@code{@@code}’和‘@code{@@samp}’，即可生成@code和@samp。

在@code命令中，当单词出现在句首时，改变其大小写是不正确的。大多数计算机语言都是区分大小写的。例如，在 C 语言中，Printf与标识符printf是不同的，前者很可能是拼写错误。即使在那些不区分大小写的语言中，看到标识符以不同的方式拼写也会让读者感到困惑。选择一种拼写方式并始终使用它。如果你不想让命令名称全部小写地出现在句首，你应该重新组织句子。

在输出格式允许的情况下，@code中的程序代码会以打字机（等宽，monospace）字体进行排版。例如，

```
The function returns @code{nil}.
```

产生：

```
The function returns nil.
```

以下情形中，更建议不 使用@code：

- 对于 shell 命令名，例如ls（应使用@command）。
- 对于环境变量，例如TEXINPUTS（应使用@env）。
- 对于 shell 选项，如‘-c’，当此类选项单独存在时，应使用@option）。
- 通常情况下，与使用@code相比，使用@samp会让整个 shell 命令看起来会更好。在这种情况下，规则是：选择更美观的格式。
- 对于比语法标记还要短的字符串。例如，如果你正在编写关于‘goto-ch’的内容，而它只是 Emacs Lisp 语言的函数goto-char的名称的一部分，那么你应该使用@samp。
- 一般来说，在描述(语法)标记中使用的字符时；例如，在解释函数名称中可以使用的字母或可打印符号时，不要使用@code，应使用@samp。此外，你不应使用@code来标记被视为程序输入的文本，除非输入的语言类似于编程语言。例如，你不应使用@code来表示 GNU Emacs 的击键命令，应使用@kbd，尽管你可以使用@code来表示击键命令调用的 Emacs Lisp 函数的名称。

默认情况下，TeX会在@code及相关命令中的‘-’和‘_’字符处自动换行。这可以通过@allowcodebreaks（请参阅12.4 节[@allowcodebreaks], 102 页）来控制。在 HTML 输出中，换行则取决于浏览器的行为。对于 Info，似乎最好不要进行此类换行。

对于 Info 和纯文本，引号字符通常会放在@code命令及相关命令（例如，@kbd、@command）的输出前后，但在类打字机的上下文中，如@example环境（请参阅7.4 节[@example], 60 页）

和`@code`本身等等，则不会如此。要控制`texi2any`在`@code`等命令的输出中插入的引号字符，请参见`OPEN_QUOTE_SYMBOL`和`CLOSE_QUOTE_SYMBOL`定制化变量（请参阅19.4.7节[Customization Variables for Multiple Formats], 164 页）。

6.1.3 @kbd{ 键盘字符 }

使用`@kbd`命令来标识用户输入的字符。例如，要引用字符`M-a`，请编写：

```
@kbd{M-a}
```

要引用字符`M-x shell`，请这样写：

```
@kbd{M-x shell}
```

默认情况下，`@kbd`命令会生成不同的字体（在输出格式允许的情况下，使用斜体打字机字体而非普通打字机字体），以便用户能够区分他们应该输入的字符和计算机输出的字符。由于`@kbd`的用法因手册而异，您可以使用`@kbdinputstyle`命令来控制字体切换。此命令对 Info 输出无影响。请在行首输入此命令，并以单个单词作为参数，参数可以是以下之一：

‘code’ 对于`@kbd`和`@code`，始终使用相同的字体。

‘example’ 仅在`@example`和类似环境中使用`@kbd`的区分字体。

‘distinct’
（默认设置）始终为`@kbd`使用可区分的字体。

您可以在`@kbd`命令的大括号内嵌入另一个 `@` 命令。例如，以下是一种描述命令的方式，该命令可以更详细地描述为“按下‘r’键，然后按下RETURN键”：

```
@kbd{r @key{RET}}
```

这将生成（本手册使用`@kbdinputstyle`的默认设置）：

```
r RET.
```

如果您要拼出您键入的字母，也可以使用`@kbd`命令；例如：

要下达 `@code{logout}` 命令，可键入字符

```
@kbd{l o g o u t @key{RET}}。
```

这将产生：

要下达`logout`命令，可键入字符

```
l o g o u t RET.
```

此外，这个例子还展示了你可以为了清晰起见添加空格。如果你明确想提及空格字符作为输入字符之一，可以为它编写`@key{SPC}`。

6.1.4 @key{ 键名 }

使用`@key`命令表示键盘上某个键的常规名称，例如：

```
@key{RET}
```

当要输入的字符序列包含一个或多个通过名称描述的键时，您可以在`@kbd`命令的参数中使用`@key`命令。

例如，要输入`C-x ESC`和`M-TAB`，你需要键入：

```
@kbd{C-x @key{ESC}}
```

```
@kbd{M-@key{TAB}}
```

以下是推荐的键名列表：

SPC Space, 空格。

RET	Return, 回车。
LFD	Linefeed, 换行符; 然而, 由于现在大多数键盘都没有换行键, 因此最好将此字符称为 <code>C-j</code> 。
TAB	Tab, 水平制表。
BS	Backspace, 退格。
ESC	Escape 取消。
DELETE	Delete 删除。
SHIFT	Shift 转换。
CTRL	Control, 控制。
META	Meta, 元键。

处理像‘meta’或‘ctrl’这样的修饰键名称时, 有一些微妙之处。当提到使用修饰键的字符时, 如`Meta-a`, 请单独使用`@kbd`命令; 不要使用`@key`命令; 但当您单独引用修饰键时, 请使用`@key`命令。例如, 使用‘`@kbd{Meta-a}`’以生成`Meta-a`, 使用‘`@key{META}`’以生成`META`。

6.1.5 @samp{ 文本 }

使用`@samp`命令来标识文本, 该文本是字面意义的示例, 或是文件、字符串、模式等中的字符序列的‘样本’。将文本括在大括号中。在输出中, 文本将出现在单引号内; 此外, 文本使用等宽字体。

使用正则表达式 `@samp{foo$}`, 以匹配行末的 `@samp{foo}`。

产生:

使用正则表达式‘`foo$`’, 以匹配行末的‘`foo`’。

任何时候提到单个字符时, 除非`@kbd`或`@key`更合适, 否则都应使用`@samp`。此外, 对于 C 语言中的完整语句和完整的 shell 命令, 也可以使用`@samp`——在这种情况下, `@samp`通常比`@code`看起来更好。大体上讲, `@samp`就是`@code`、`@kbd`、`@key`、`@command`等未涵盖内容的统称。

只有在您指定的字符串中包含标点符号时, 才需要在括号内包含它们。如果这些标点符号是字符串周围英文文本的一部分, 则应将其写在括号外。例如, 在以下句子中, 逗号和句号位于括号外:

```
In English, the vowels are @samp{a}, @samp{e}, @samp{i},
@samp{o}, @samp{u}, and sometimes @samp{y}.
```

产生:

```
In English, the vowels are ‘a’, ‘e’, ‘i’, ‘o’, ‘u’, and sometimes ‘y’.
```

6.1.6 @verb{ 分隔符 文本 分隔符 }

使用`@verb`命令来打印一串逐字的字符序列。

与`LATEX`的`\verb`命令类似, 可以使用任何唯一的分隔符来引用原样文本。将原样文本(包括分隔符)括在大括号中。文本将以等宽字体打印:

```
How many @verb{|@|}-escapes does one need to print this
@verb{. @a @b.@c.} string or @verb{+@'e?'`{}!`~\+} this?
```


产生：

```
How many @-escapes does one need to print this
@a @b.@c string or @'e?`{ }!`\ this?
```

这与@samp(见上一节)、@code以及类似命令不同；在那些情况下，参数是普通的 Texinfo 文本，其中三个字符@{}像往常一样是特殊的字符。对于@verb，除了你选择的分隔符之外，没有其他特殊字符。

如上所示，分隔符本身可能出现在逐字的字符序列中。再举一个例子，‘@verb{...}’会打印一个固定宽度的句点（第一个及第三个句点作为分隔符，第二个是逐字的字符序列）。¹

在其他 Texinfo 构造中使用@verb是不可靠的。特别是在与交叉引用相关的任何内容中使用@verb，如：章节标题或图表标题，都是行不通的。

6.1.7 @var{ 元语法变量 }

使用@var命令来指示元语法变量。元语法变量是代表另一段文本的符号。例如，在函数的文档中，应使用元语法变量来描述传递给该函数的参数。

在计算机程序中，不要将普通变量的名称使用@var。这些是特定的名称，因此@code是正确的。例如，Emacs Lisp 变量texinfo-tex-command不是一个元语法变量，使用@code格式化才是正确的。同样，也不要将环境变量使用@var，@env才是正确的（见下一节）。

Info 输出中的@var的作用是将参数的大小写更改为全部大写。在打印的手册中，参数以斜体字输出。²

示例：

```
要删除文件 @var{filename}，键入 @samp{rm @var{filename}}。
```

产生：

```
要删除文件 filename，键入‘rm filename’。
```

（请注意，@var可能出现在@code、@samp、@file等命令中。）

将元语法变量全部写成小写字母，不加空格，并用连字符连接，以提高可读性。因此，关于如何开始编写 Texinfo 手册的示例的 Texinfo 源代码如下所示：

```
\input texinfo
@@settitle @var{name-of-manual}
```

这产生：

```
\input texinfo
@settitle name-of-manual
```

在某些文档样式中，元语法变量用尖括号表示，例如：

```
..., type rm <filename>
```

然而，这并不是 Texinfo 所采用的风格。

¹ 译者注：中文字符可以作分隔符。

² 在 T_EX 输出中，@var当前在代码上下文中（如@code或@example）使用斜体打字机字体。我们计划在下一个版本中将其更改为在所有上下文中使用可变宽度的斜体罗马字体。要避免此更改，请使用@set设置‘txicodevaristt’标志；现在就指定‘@clear txicodevaristt’，以使用此更改（请参阅15.5.1 节[@set @value]，125 页）。请注意，此标志在 L^AT_EX 输出中不起作用。

6.1.8 @env{ 环境变量 }

使用@env命令来指示环境变量，许多操作系统都使用此命令，包括 GNU 。不要将其用于元语法变量；对于这些变量，请使用@var（见上一节）。

在效果上，@env与@code是等价的。例如：

```
The @env{PATH} environment variable ...
```

产生：

```
The PATH environment variable ...
```

6.1.9 @file{ 文件名 }

使用@file命令标示文件、缓冲区或目录的名称，或者是 Info 中的节点名。您也可以将该命令用于文件名后缀。对于编程语言中的符号，请勿使用@file；应使用@code。

@file在效果上等同于code。例如，

```
The @file{.el} files are in
the @file{/usr/local/emacs/lisp} directory.
```

产生：

```
The .el files are in the /usr/local/emacs/lisp directory.
```

6.1.10 @command{ 命令名 }

使用@command命令来标示命令名称，例如ls或cc。

@command在效果上等同于@code。例如：

```
The command @command{ls} lists directory contents.
```

产生：

```
The command ls lists directory contents.
```

如果你认为某个程序名是一个新的英文单词，比如“Emacs”或“Bison”，那么你应该用普通文字字体书写该程序名，而不是使用@command。

在编写完整的 shell 命令调用时，如‘ls -l’，您应自行决定使用@samp或@code。

6.1.11 @option{ 选项名 }

使用@option 命令来指示命令行选项；例如，-l、--version或--output=filename。

@option在效果上等同于@code。例如：

```
The option @option{-l} produces a long listing.
```

产生：

```
The option -l produces a long listing.
```

6.1.12 @dfn{ 技术术语 }

使用@dfn命令来标识技术术语的引入或定义。该命令仅适用于那些旨在引入一个将再次使用或读者应该了解的术语的段落。对于首次提及的术语，如果只是简单提及技术术语，并不需要使用@dfn。该命令会在打印手册中选择斜体字体，并在 Info 文件中生成双引号。

例如：

```
去除文件的行为被称为 @dfn{删除}。
```

产生：

去除文件的行为被称为删除。

一般来说，一个句子如果包含了某个术语的定义性出现，那么这个句子就应该是该术语的定义。这个句子不需要明确指出它是一个定义，但它应该包含定义性的信息——它应该使含义变得清晰。

6.1.13 @abbr{ 缩写[, 含义]}

您可以使用@abbr命令来处理通用缩写。缩写以单个参数的形式放在花括号内，如‘@abbr{Comput.}’。出于风格考虑，或者对于特定的缩写，您可能更倾向于省略句点，如‘@abbr{Mr} Stallman’。

@abbr 接受一个可选的第二个参数，该参数用于表示缩写词的含义³。

如果在没有第二个参数且以小写字母结尾的缩写之后有非句末句号⁴，请记住使用@.命令(请参阅11.3.3 节[Ending a Sentence], 90 页)方式书写句号，以获得正确的间距。在缩写本身内部无需使用@.，因为 Texinfo 会自动假定缩写中的句号并不结束句子。

在可使用适当标签的输出格式中，如 HTML 和 DocBook，可能会使用此标签。否则，第一个参数将按原样打印；如果第二个参数存在，则会在缩写后用括号打印出来。例如：

```
@abbr{Comput. J., Computer Journal}
```

产生：

```
Comput. J. (Computer Journal)
```

对于全部由大写字母组成的缩写，您应该更倾向于使用@acronym命令。有关这两个命令用法的更多信息，请参阅下一节。

6.1.14 @acronym{ 缩略语[, 含义]}

您可以使用@acronym命令来处理以大写字母书写的缩写，例如‘NASA’。缩写以单个参数的形式放在花括号内，如‘@acronym{NASA}’。出于风格考虑，或者对于特定的缩写，您可能更倾向于使用句点，如‘@acronym{N.A.S.A.}’。

@acronym 接受一个可选的第二个参数，该参数用于表示缩写的含义⁵。

如果缩写位于句末，且没有第二个论参数，请记得使用@.或类似命令(请参阅11.3.3 节[Ending a Sentence], 90 页)以获得正确的间距。

在 T_EX 中，首字母缩略词会以稍小的字体打印。在 Info 输出中，参数会按原样打印。在任一格式中，以及在 L^AT_EX 输出中，如果存在第二个参数，则会在首字母缩略词后用括号打印出来。在 HTML 和 DocBook 中，会使用相应的标签。

例如（由于 GNU 是一个递归首字母缩略词，我们递归地使用@acronym）：

```
@acronym{GNU, @acronym{GNU}'s Not Unix}
```

产生：

```
GNU (GNU's Not Unix)
```

在某些情况下，通常会将姓氏全部大写。不要为此使用@acronym，因为名字并不是首字母缩略词。请改用@sc(请参阅6.2.2 节[Smallcaps], 56 页)。

³ 译者注：指的是未缩写的形式。

⁴ 译者注：半角句号。

⁵ 译者注：指的是未缩写的形式。

`@abbr`和`@acronym`是密切相关的命令：它们都向读者表明正在使用缩写形式，并可能给出含义。在决定是否使用这两个命令时，请记住以下几点。

- 在常见的英语用法中，首字母缩略词是缩写词的一个子集：它们包括可发音的单词，如‘NATO’、‘radar’和‘snafu’；一些来源还包括首音节缩略词，如‘Usenet’，混合词，如‘SIGGRAPH’，以及不可发音的缩写词，如‘FBI’。
- 在 Texinfo 中，首字母缩略词（但非缩写）应仅由大写字母和句点组成，不得包含小写字母。
- 在 $\text{T}_{\text{E}}\text{X}$ 中，首字母缩略词（而非缩写）会以稍小的字体打印。
- 对于所有大写字母序列，始终使用`@acronym`通常相当困难且耗时。此外，一些首字母缩略词使用正常字体大小，而另一些则使用较小的字体，这看起来很奇怪。因此，您可能希望考虑一种更简单的方法，即避免使用`@acronym`，而只是将所有内容都以大写字母作为普通文本排版：‘GNU’，生成输出‘GNU’。
- 一般来说，并非所有缩写都必须使用这两个命令中的任何一个，请自行判断。没有它们，文本也是完全可以阅读的。

6.1.15 `@indicateurl`{ 统一资源定位符 }

使用`@indicateurl`命令来指示万维网上的统一资源定位符。这纯粹是为了标记的目的，并不会生成一个可点击的链接（要生成链接，请使用`@url`或`@uref`命令，请参阅 5.14 节[`@url`], 45 页）。`@indicateurl`对于那些实际上不存在的 URL 很有用。例如：

作为一个示例，URL 可以是 `@indicateurl{http://example.org/path}`。

这产生：

作为一个示例，URL 可以是‘`http://example.org/path`’。

`@indicateurl`的输出通常与`@samp`的输出类似（请参阅6.1.5 节[`@samp`], 51 页）。

6.1.16 `@email`{ 电子邮件地址[, 显示文本] }

使用`@email`命令来标示一个电子邮件地址。该命令需要一个必选参数，即邮件地址，以及一个可选参数，即要显示的文本（默认为电子邮件地址本身）。

在 Info 中，地址以尖括号显示，前面会显示文本（如果有的话）。在打印输出中，尖括号会被省略。在 HTML 和 DocBook 输出中，`@email`会生成一个‘mailto’链接。在 HTML 中，‘mailto’链接通常会弹出一个邮件撰写窗口。例如：

```
Send bug reports to @email{bug-texinfo@gnu.org},
suggestions to the @email{bug-texinfo@gnu.org, same place}.
```

产生：

```
Send bug reports to bug-texinfo@gnu.org,
suggestions to the same place.
```

6.2 强调文本

通常，Texinfo 会根据文本中单词所属的类别来更改字体以标记单词；例如，使用`@code`命令。大多数情况下，这是标记单词的最佳方式。然而，有时您会希望在不指明类别的情况下强调文本。Texinfo 提供了两个命令来实现这一功能。此外，Texinfo 还有几个命令用于指定输出文本的字体。这些命令在 Info 中无效，其中只有一个命令例外，即`@r`命令，有常规用途。

6.2.1 @emph{ 文本} 与@strong{ 文本}

@emph和@strong命令用于强调；@strong 的强调程度更高。在打印输出中，@emph会生成斜体，而@strong会生成粗体。在 Info 输出中，@emph会用下划线(‘_’)包围文本，而@strong则会在文本周围加上星号。例如，

```
@strong{注意:} @samp{rm * .[^.]*}
  删除目录中的 @emph{所有} 文件。
```

产生：

注意：‘rm * .[^.]*’ 删除目录中的所有 文件。

@strong命令很少使用，除非是用来标记实际上属于排版元素的文字，如上例中的“注意”一词。

警告：不要在‘Note’ 一词后加空格的情况下使用@strong；Info 会误将这种组合视为交叉引用。请改为使用如请注意或警告这样的短语，或者在那里可以使用@quotation的选项参数— ‘Note’。

6.2.2 @sc{ 文本}：小号大写字母字体

使用‘@sc’命令将文本设置为小号大写字母字体(A SMALL CAPS FONT) (如果可能的话)。在花括号中，对您希望以小号大写字母字体输出的文本，要用小写字母写出来，如下所示：

```
Richard @sc{Stallman} a commencé le projet GNU.*
Richard STALLMAN a commencé le projet GNU.*
Richard Stallman a commencé le projet GNU.
```

产生：

Richard STALLMAN a commencé le projet GNU.
Richard STALLMAN a commencé le projet GNU.
Richard Stallman a commencé le projet GNU.

如图所示，我们建议将@sc保留用于需要打印小号大写字体的特殊情况；‘字体系列名称’就是其中之一，尤其是在英语以外的语言中，尽管这类事情没有硬性规定。

在 TeX 中，@sc命令的大括号中间的任何大写字母都会以非小号形式排版；只有小写字母才会以小号大写字体打印。在 Info 输出中，@sc的参数会以全大写形式打印。在 HTML 中，参数会被转换为大写，并且输出会使用<small>标签进行标记以减小字体大小，因此 HTML 无法轻易表示真正的小号大写字母。在 LaTeX 中，会输出一个设置小号大写字母字体的命令。

总体而言，我们建议尽可能使用标准的大小写字母。

6.2.3 打印用字体

Texinfo 提供了一个命令来更改文档打印输出中主体字体的大小：@fonttextsize。该命令在其他输出中无效。它接受该行剩余部分的单个参数，该参数必须是‘10’或‘11’。例如：

```
@fonttextsize 10
```

其效果是将正文字体缩小至 10 pt 大小（默认值为 11 pt）。其他元素（如章节和部分）的字体也会相应缩小。此命令应仅与@smallbook（请参阅节 E.6 [@smallbook], 263 页）或类似命令配合使用，因为在标准纸张（8.5x11 或 A4）上，10 pt 的字体过小。使用此命令的一个原因是节省纸张，从而降低实体书的印刷成本。

Texinfo 目前没有用于切换所使用字体的命令，也没有更通用的改变大小的命令。

Texinfo 还提供了许多字体命令，用于指定打印手册中的字体更改，并在可能的情况下指定 HTML 和 DocBook 输出中的字体更改。这些命令在 Info 中无效。所有命令都适用于后面用花括号括起来的参数。

@b 选择加粗(**bold**)字体；

@i 选择斜体(*italic*)字体；

@r 选择罗马(roman)字体，这是打印文本时常用的字体。该字体可能有衬线，也可能没有。

@sansserif 选择一种无衬线(sans serif)字体；

@slanted 选择一种斜体(*slanted*)字体；

@t 选择@code所使用的等宽(fixed-width)、打字机风格(typewriter-style)的字体；

在示例这类的环境中，@r命令非常有用，它可以用标准罗马字体而不是等宽字体来编写注释。这样在打印输出时看起来更美观。

示例：

```
@lisp
(+ 2 2)      ; @r{Add two plus two.}
(+ 2 2)      ; Add two plus two.
@end lisp
```

产生：

```
(+ 2 2)      ; Add two plus two.
(+ 2 2)      ; Add two plus two.
```

在支持打字机字体的环境中，@t命令偶尔可用于生成输出，但在 Info 或纯文本中，不需要用引号来区分。例如：

```
@t{foo}@*
@code{foo}
```

生成：

```
foo
foo
```

以下是使用@t而不是@code的几个可能原因：

- 参数是一个单一字符
- 参数已经被某种引号包围
- 从上下文或参数本身可以明显看出，该参数是计算机代码（例如：一个 Usenet 新闻组的名称）

一般来说，其他字体命令不太有用；它们主要是为了能够在 T_EX和相关包中实现特定字体效果的功能。

7 引用与示例

引用和示例是由一个或多个完整段落组成的文本块，它们与正文分开，并受到不同的处理。在输出结果中，它们通常会缩进。

在 Texinfo 中，引用或示例总是以单独一行开头的‘@ 命令’开始，并以同样单独一行开头的‘@end 命令’结束。例如，要开始一个示例，需在单独一行的开头输入@example，而要结束示例，则需在该行的开头单独一行输入@end example，且@end和example之间只需一个空格。

7.1 块封闭命令简介

以下是用于封闭文本块的命令的摘要，这些文本块也被称为环境。后续章节将对这些命令进行进一步解释。

@quotation

对引用的文本进行封闭。默认情况下，文本会被填充、从两侧边距缩进，并以罗马字体打印。

@indentedblock(缩进块)

类似于@quotation，但文本在左侧缩进。

@example 对代码、命令等内容进行封闭。文本以等宽字体打印，并缩进但未填充。

@lisp 类似于@example，但专门用于展示 Lisp 代码。文本以等宽字体打印，并进行了缩进，但并未填充。

@verbatim

对一段要原样打印的文本进行封闭；在此期间，不会进行任何字符替换，所有命令都会被忽略，直到遇到下一个@end verbatim。文本将以等宽字体打印，且不会缩进或填充。额外的空格和空行是有意义的，制表符会被展开。

@display 对说明性文本进行封闭。文本缩进但未填充，且未选择任何字体（因此，默认情况下，字体为罗马字体）。

@format 类似于@display，但文本未缩进。

@smallquotation

@smallindentedblock

@smallexample

@smalllisp

@smalldisplay

@smallformat

这些@small...命令与它们对应的非‘small’命令类似，只是在可能的情况下，它们会以较小的字体输出文本。

@flushleft

@flushright

文本并未填充，而是分别与左边距或右边距对齐。

@raggedright

文本填满，但仅左侧对齐，导致右侧边距参差不齐。

@cartouche

用圆角框框出文本，通常是示例或引用，以突出显示。

在上述构造中，可使用`@exdent`命令来撤销某行的缩进。

在上述任一结构之后（或在任何段落开头），可以使用`@noindent`命令来防止后续文本作为新段落而缩进。

7.2 @quotation: 块引用

引用文本的处理方式与普通文本相同（使用常规字体，文本填充），但有以下不同之处：

- 左边缘更靠近页面中央，因此整个引用部分都进行了缩进；右边缘也可能更靠近页面中央
- 段落的首行缩进量不得超过其他行；并
- 可以用`@author`命令来指出引用的作者。

这是一个在`@quotation`命令和`@end quotation`命令之间编写的文本示例。`@quotation`命令通常用于表示从另一部（真实或虚构的）印刷作品中摘录的文本。

将`@quotation`命令作为单独的一行文本写入。此行将从输出中消失。使用以`@end quotation`开头且不包含其它内容的一行来标记引文的结束。`@end quotation`行同样将从输出中消失。

`@quotation` 接受一个可选参数，该参数在行尾给出。如果存在此文本，则将其以粗体或其他强调方式包含在引文的开头，并跟上一个`‘:’`。例如：

```
@quotation 备注
This is a foo.
@end quotation
```

产生：

备注: This is a foo.

如果`@quotation`参数是以下英文单词之一（不区分大小写）：

```
Caution Important Note Tip Warning
```

那么 DocBook 输出将使用相应的特殊标签（如`<note>`等）而不是默认的`<blockquote>`。

如果在`@quotation`块中使用`@author`命令指定了引用的作者，则会在引用后显示一行作者姓名：

```
@quotation
人们有时会问我，在 Emacs 社区中使用 vi 是否是一种罪恶。使用 vi 的自由版本并不是罪恶，而是一种苦行。所以，尽情享受编程的乐趣吧。

@author 理查德·斯托尔曼
@end quotation
```

产生：

人们有时会问我，在 Emacs 社区中使用 vi 是否是一种罪恶。使用 vi 的自由版本并不是罪恶，而是一种苦行。所以，尽情享受编程的乐趣吧。
---理查德·斯托尔曼

7.3 @indentedblock: 缩进文本块

@indentedblock环境与@quotation类似，但文本只在左侧缩进，且没有可选参数。因此，文本字体保持不变，文本按常规方式聚集和填充，但左侧边距会增加。例如：

这是一个在@indentedblock命令和@end indentedblock命令之间编写的文本示例。@indentedblock环境可以包含任何所需的文本或其他命令。

这在 Texinfo 源代码中是这样写的：

```
@indentedblock
这是一个...示例。...
@end indentedblock
```

7.4 @example: 示例文本

@example环境用于表示不属于运行文本的计算机输入或输出。如果你想在句子中嵌入代码片段，请使用@code命令或其相关命令（请参阅6.1.2节[@code], 49页）。

在单独的一行开头编写一个@example命令。用@end example标记代码块的结束。例如，

```
@example
cp foo @var{dest1}; \
  cp foo @var{dest2}
@end example
```

产生：

```
cp foo dest1; \
cp foo dest2
```

输出使用等宽字体，并且进行了缩进。输入文件中的每一行在输出中都是一行；也就是说，源文本不会被填充。额外的空格和空行是有意义的。Texinfo 命令会被展开；如果你希望输出是输入的逐字逐句，请使用@verbatim环境（请参阅7.5节[@verbatim], 61页）。

从逻辑上讲，示例通常位于段落的“中间”，而之后继续的文本不应像上例那样缩进。@noindent命令可防止一段文本像新段落一样被缩进（请参阅7.12节[@noindent], 64页）。

如果您希望在代码注释中使用常规的罗马字体，可以使用@r命令（请参阅6.2.3节[Fonts], 56页）。

你可以选择为@example命令提供参数，如果有多个参数，请用逗号分隔。在 HTML 输出中，所有此类参数将以类名的形式输出，并以字符串‘user-’作为前缀。这可能有助于为代码示例的手册添加语法高亮功能。

我们建议，当您向@example提供多个参数时，使用第一个参数来指定代码的语言（例如：‘C’、‘lisp’、‘Cplusplus’）。您可能会发现其他参数的用途，比如提供格式化提示或标记代码示例以供提取和进一步处理，但目前我们还没有给出明确的建议。也许在未来的 Texinfo 版本中会有所改变。

警告：在示例中不要使用制表符！（在 Texinfo 中的其他任何地方也不要使用，除非是在原样输出环境中。）T_EX将制表符视为单个空格，但这并不是它们的实际显示效果。

7.5 @verbatim: 逐字文本

使用@verbatim环境来打印可能包含不应被解释的特殊字符或命令的文本，例如计算机输入或输出（@example将其文本视为常规的 Texinfo 命令）。这在 Texinfo 手册中包含自动生成的文件时特别有用。

一般来说，输出将与输入完全相同。不会进行任何字符替换，例如，所有空格和空行（包括制表符）都视为有效字符¹。文本以等宽字体排版，且不会进行缩进或填充。

在行首单独编写一个@verbatim命令。此行将从输出中消失。用@end verbatim命令标记 verbatim 块的结束，该命令同样单独写在行首。@end verbatim也将从输出中消失。

例如：

```
@verbatim
{
<tab>@command with strange characters: @'e expand<tab>me
}
@end verbatim
```

（其中<tab> 表示一个制表符）。这将生成：

```
{
    @command with strange characters: @'e expand    me
}
```

由于包含@verbatim和@end verbatim的行不会产生任何输出，因此通常应在@verbatim之前放置一个空行，在@end verbatim之后放置另一个空行。@verbatim和@end verbatim之间的空行将出现在输出中。

你可以通过将@verbatim包含在@smallformat环境中来获得一个小字体的逐字输出，如下所示：

```
@smallformat
@verbatim
... still verbatim, but in a smaller font ...
... 仍是逐字，但用更小的字体 ...
@end verbatim
@end smallformat
```

产生：

```
... still verbatim, but in a smaller font ...
... 仍是逐字，但用更小的字体 ...
```

最后，提醒一句：在其他 Texinfo 构造中使用@verbatim并不可靠。

另请参阅17.3 节[@verbatiminclude], 141 页。

7.6 @lisp: 标记一个 Lisp 示例

@lisp 命令用于 Lisp 代码：

```
@lisp
Example lisp code
@end lisp
```

¹ 译者注：这里的“逐字”文本、“字面”文本，可以认为就是 C++语言（11 版本及之后）的原始(RAW)字符串。

等同于以下写法：

```
@example lisp
Example lisp code
@end example
```

使用`@lisp`来保留有关示例性质的信息。这很有用，例如：如果你编写一个仅求值的函数，并且所有 Lisp 代码都放在一个 Texinfo 文件中，然后，你就可以将 Texinfo 文件用作一个 Lisp 库。

7.7 @display：使用文本字体的示例

`@display`命令会开启另一种环境，在此环境中，字体保持不变，不会像`@example`那样切换为打字机字体。每行输入仍然会生成一行输出，并且输出仍然会缩进。

```
This is an example of text written between a @display command
and an @end display command. The @display command
indents the text, but does not fill it.
```

这是一个在`@display`命令和`@end display`命令之间编写的文本示例。`@display`命令会对文本进行缩进，但不会填充它。

7.8 @format：使用全行宽度的示例

`@format`命令与`@display`类似，但不同的是，它不会对文本进行缩进。与`@display`一样，它不会选择等宽字体。因此，

```
@format
This is an example of text written between a @code{@@format} command
and an @code{@@end format} command. As you can see
from this example,
the @code{@@format} command does not fill the text.
```

这是一个在`@code{@@format}`命令和`@code{@@end format}`命令之间编写的文本示例。

从这个示例中可以看出，`@code{@@format}`命令并不会填充文本。

```
@end format
```

产生：

```
This is an example of text written between a @format command
and an @end format command. As you can see
from this example,
the @format command does not fill the text.
```

这是一个在`@format`命令和`@end format`命令之间编写的文本示例。从这个示例中可以看出，`@format`命令并不会填充文本。

7.9 @exdent：撤销行的缩进

`@exdent`命令用于移除一行中可能存在的所有缩进该命令位于行首，仅对命令后同一行中的文本生效。文本前后不要使用大括号。在输出格式允许的情况下，`@exdent`行中的文本也会以罗马字体显示。

`@exdent`通常用于示例中。因此，

```
@example
此行遵循 @@example 命令。
@exdent 这行是取消缩进的。
这行跟随取消缩进的行。
@@end example 将在下一行。
@end example
```

产生：

```
    此行遵循 @example 命令。
这行是取消缩进的。
    这行跟随取消缩进的行。
    @end example 将在下一行。
```

在实际操作中，`@exdent`命令很少使用。通常，您可以通过结束示例并将页面恢复到正常宽度来取消缩进文本。

`@exdent`并非在所有输出格式中都有效。²

7.10 @flushleft(左对齐)和@flushright(右对齐)

`@flushleft`和`@flushright`命令用于将行尾对齐到页面的左右边距，但不会填充文本。这些命令单独占一行，且不使用花括号。`@flushleft`和`@flushright`命令分别由单独占一行的`@end flushleft`和`@end flushright`命令结束。

示例：

```
@flushleft
文本是左对齐。
@end flushleft
```

产生：

文本是左对齐。

`@flushright`生成一种缩进格式，常用于信件的回邮地址。例如，

```
@flushright
以下是一个使用右对齐格式写的文本的示例。@*
@code{@flushright}命令使每一行都右对齐，但左端留有空白。
@end flushright
```

产生：

以下是一个使用右对齐格式写的文本的示例。

`@flushright`命令使每一行都右对齐，但左端留有空白。

7.11 @raggedright：右侧参差不齐的文本

`@raggedright`像往常一样填充文本，但文本只在左侧对齐，右边距是参差不齐的。该命令单独占一行，没有花括号。`@raggedright`命令以单独一行的`@end raggedright`结束。此

² 译者注：缩进，在 HTML 输出中看不到缩进效果，只能看到背景颜色变成了正常文本的背景。在打印输出、Info 输出中有效果，其它输出是否有效待验证

命令仅在文本左侧对齐的输出格式中有效，而在文本始终右参差不齐的输出格式（如 Info 或 HTML）中无效。

当段落中包含一长串命令列表，且预先知道在两侧边距对齐文本会使段落看起来很糟糕时，`@raggedright`命令可能很有用。

示例（来自本手册其他部分）：

```
@raggedright
Commands for double and single angle quotation marks:
@code{@@guillemetleft@{}} , @code{@@guillemetright@{}} ,
@code{@@guillemotleft@{}} , @code{@@guillemotright@{}} ,
@code{@@guilsinglleft@{}} , @code{@@guilsinglright@{}} .
@end raggedright
```

产生：

```
Commands for double and single angle quotation marks: @guillemetleft{ },
@guillemetright{ }, @guillemotleft{ }, @guillemotright{ }, @guilsinglleft{ },
@guilsinglright{ }.
```

如果不使用`@raggedright`，则产生：Commands for double and single angle quotation marks: @guillemetleft{ }, @guillemetright{ }, @guillemotleft{ }, @guillemotright{ }, @guilsinglleft{ }, @guilsinglright{ }.

7.12 @noindent：省略缩进

示例或其他内容可以将一个段落拆分为多个片段。通常，格式化工具会将示例后的文本缩进，作为新段落。您可以通过在继续文本之前，在一行的开头写入`@noindent`来防止这种情况。您还可以使用`@paragraphindent`（请参阅节 E.3 [`@paragraphindent`], 262 页）全局禁用所有段落的缩进。

这里有一个示例，展示了如何消除`@example`后文本的常规缩进，常见情况：

```
@example
这是一个示例。
@end example

@noindent
这一行没有缩进。如您所见，该行的开头与下一行完全左对齐。
下一行 @enddots{ }
```

产生：

这是一个示例。

这一行没有缩进。如您所见，该行的开头与下一行完全左对齐。
下一行...

`@noindent`的标准用法与上述相同：放在原本会成为新段落的地方的开头，以消除通常出现在那里的缩进。它可以在文本之前，也可以单独占一行。没有理由在其他上下文中使用它，例如在段落中间或在一个环境内（请参阅第 7 章[Quotations and Examples], 58 页）。

你可以根据需要调整在`@noindent`行前的空白行的数量，从而控制 Info 文件输出中的空白行数量：仅包含`@noindent`的行不会生成空白行，环境命令对应的`@end...`行也不会生成空白行。

在`@noindent`命令后不要加花括号；它们没有用，因为`@noindent`是在段落外使用的命令（请参阅 节 A.1 [Command Syntax], 205 页）。

7.13 @indent：强制缩进

为了让`@noindent`命令（见上一节）更为完备，Texinfo 提供了`@indent`命令来强制段落缩进。例如，本段（是本节的第一段）就是使用`@indent`命令缩进的。

的确，一个章节的第一段最有可能使用`@indent`，以覆盖该处的默认行为（无缩进）³（请参阅 节 E.3 [`@paragraphindent`], 262 页）。它可以在文本前面，也可以单独占一行。

作为一个特殊情况，当在未填充文本的环境中使用`@indent`时，它会在 TeX 输出中产生一个段落缩进空格。（这些环境是指输入一行就输出一行的情况，如`@example`和`@display`；有关所有环境的详情，请参阅 7.1 节 [Block Enclosing Commands], 58 页。）

在`@indent`命令后不要加花括号；它们没有实际用途，因为`@indent`是在段落外使用的命令（请参阅 节 A.1 [Command Syntax], 205 页）。

7.14 @cartouche：圆角矩形

在输出格式允许的情况下，`@cartouche` 命令会在内容周围绘制一个圆角框。您可以使用此命令将手册的一部分与主要流程隔离开来。您还可以使用`@cartouche` 来进一步突出显示示例或引用。

例如，你可以编写一本手册，在其中，示例被环绕在装饰性的圆角矩形中以示强调。例如：

```
@cartouche
@example
% pwd
/usr/local/share/emacs
@end example
@end cartouche
```

在打印手册中，用圆角矩形将两行示例框起来。

示例的输出如下所示（如果你在 Info 中阅读此内容，你会发现`@cartouche`没有效果）：

```
% pwd
/usr/local/share/emacs
```

`@cartouche` 接受一个可选参数，该参数在命令行的剩余部分给出。如果存在，此文本即为圆角矩形的标题。它将以粗体或其他强调方式显示在圆角矩形的开头，并且在某些输出格式中居中显示。

以下示例展示了一个带有标题的圆角矩形：

```
@cartouche 重要
在正文主要内容之外，对某一重要信息进行解释的文本。
@end cartouche
```

带有标题的圆角矩形看起来像这样：

³ 译者注：这是英文写作的默认行为，在中文写作中，如本文，有缩进是默认行为。

重要

在正文主要内容之外，对某一重要信息进行解释的文本。

在打印输出中，圆角矩形会输出在一页上，不会被分割到两个页面上，这与@group（请参阅 12.9 节[@group], 103 页）类似。

7.15 @small...块命令

除了常规的@example及类似命令外，Texinfo 还提供了“small”样式的示例命令。这些命令包括@smallquotation、@smallindentedblock、@smalldisplay、@smallexample、@smallformat和@smalllisp。

在大多数输出格式中，@small...命令与其非小字体的对应命令等效。

然而，在打印输出中，@small...命令会以比常规示例命令更小的字体来排版文本。因此，例如，代码示例可以包含更长的行，并且仍然可以在一页内排满，而无需重写。

使用相应的@end small...标记@small...块的结束。例如，将@smallexample与@end smallexample配对。

以下是@smallexample命令所使用的字体示例，输出结果如下：

```
... to make sure that you have the freedom to distribute
copies of free software (and charge for this service if you
wish), that you receive source code or can get it if you want
it, that you can change the software or use pieces of it in
new free programs; and that you know you can do these things.
```

在大多数输出格式中，与@example命令相同。

在打印输出中，@smallexample输出结果与@example是不同的，以下是@example的输出结果：

```
... to make sure that you have the freedom to distribute
copies of free software (and charge for this service if you
wish), that you receive source code or can get it if you want
it, that you can change the software or use pieces of it in
new free programs; and that you know you can do these things.
```

@small...命令使用的字体样式与其对应的常规命令相同：@smallexample和@smalllisp使用等宽字体，其他所有命令则使用常规字体。它们在其他方面也有相同的行为——是否进行填充以及是否缩小边距。然而，即使其常规对应命令接受参数，它们也不接受参数。

一般来说，除非有充分的理由，否则最好使用常规命令（如@example，而不是@smallexample）。当仅使用texinfo.tex进行处理时，您可以通过‘@set dispensize small’获得类似的效果，该命令会为@example和其他环境使用较小的字体。在 HTML 输出中，如果需要，您可以使用 CSS 更改环境的字体大小；请参阅21.3 节[HTML CSS], 180 页。

8 列表和表格

Texinfo 有多种方式来制作列表和表格。列表可以是项目符号列表或编号列表；双栏表格可以突出显示第一列中的项目；同时，Texinfo 也支持多栏表格。

8.1 对列表的介绍

Texinfo 会自动对列表或表格中的文本进行缩进，并为枚举列表自动编号。自动编号功能在修改列表时非常有用，因为您无需自己重新编号。

列表¹和表格以行首的相应@命令开始，并以单独一行上的相应@end命令结束。表格和项目列表命令还要求您在行首的@命令所在的同一行写上格式化信息。

开始一个枚举列表，例如，使用@enumerate命令，并在列表结束时使用@end enumerate命令。开始一个条目列表，使用@itemize命令，并在同一行后跟一个格式化命令(如：@bullet)，并在列表结束时使用@end itemize命令。

在列表的每个元素前加上@item或@itemx(仅在两栏表格中使用)命令。

以下是不同种类的表格和列表，用项目列表方式写出：

- 带符号和不带符号的条目列表。
- 使用数字或字母进行编号的枚举列表。
- 带有高亮显示的两栏表格。

用枚举列表方式写出：

1. 带符号和不带符号的条目列表。
2. 使用数字或字母进行编号的枚举列表。
3. 带有高亮显示的两栏表格。

用双栏表格方式写出(首列为@命令)：

@itemize 带符号和不带符号的条目列表。

@enumerate
使用数字或字母进行编号的枚举列表。

@table

@ftable

@vtable 两栏表格，可选索引。

8.2 @itemize：制作条目列表

@itemize命令会生成一系列“条目”，每个条目都以左边缘内的圆点或其他标记开始，并且通常进行缩进。

在一行的开头写入@itemize，开始一个条目列表。在同一行中，在该命令后，使用一个 Texinfo 命令或字符来生成标记。通常，在@itemize之后会使用@bullet，但你也可以使用@minus或任何其他 Texinfo 命令或字符。条目你没有指定标记命令，默认会是@bullet。

¹ 译者注：原文是‘Numbered lists’，但译者认为原文有误，应为‘Lists’。

在`@itemize`之后，写下你的条目，每个条目都以`@item`开头。文本可以与`@item`在同一行。一个条目的文本可以延续多个段落。

以下是一个使用`@itemize`的示例：

```
@itemize @bullet{}
@item
Some text for foo.

@item @w{}

@item
Some text for bar.
@end itemize
```

产生：

- Some text for foo.
-
- Some text for bar.

当您诸如`@bullet`之类的标记命令作为参数传递给`@itemize`命令时，可以省略通常跟在命令后面的`{}`。

如果你根本不希望某个条目有任何内容，但仍然需要逻辑条目，请使用`@w{}`（在这种情况下，`{}`是必需的）。

在`@itemize`环境中，至少应有一个`@item`。如果不存在，`texi2any`会给出警告。如果你只想缩进文本而不是列出条目，请使用`@indentedblock`；请参阅7.3节`[@indentedblock]`，60页。

紧接在`@item`之前的索引条目（如`@cindex`等）与`@item`相关联。

通常，你应在条目之间加入一个空行。这样会在 Info 文件中插入一个空行。（在任何情况下，`TEX`都会插入适当的垂直间距。）除非项目非常简短，否则这些空行会使列表看起来更美观。

条目列表可以嵌套在其他条目列表中。以下是一个用破折号(dashes)标记的列表，嵌套在一个用圆点符号(bullets)标记的列表中：

```
@itemize @bullet
@item
第一个外层项目。

@itemize @minus
@item
第一个内层项目。

@item
第二个内层项目。
@end itemize

@item
第二个外层项目。
@end itemize
```

产生：

- 第一个外层项目。
 - 第一个内层项目。
 - 第二个内层项目。
- 第二个外层项目。

8.3 @enumerate：制作用数字或字母编号的列表

@enumerate类似于@itemize (请参阅8.2 节[@itemize], 67 页)，不同之处在于，项目上的标记是连续的整数或字母，而不是条目符号。

在一行的开头编写@enumerate命令。该命令不需要参数，但接受数字或字母作为选项。如果没有参数，@enumerate将以数字‘1’作为列表的起始项。如果有一个数字参数，如‘3’，则该命令将以该数字作为列表的起始项。如果有一个大写或小写字母参数，如：‘a’或‘A’，则该命令将以该字母作为列表的起始项。²

按照条目列表的方式编写枚举列表的文本：在枚举的每个条目，另起一行并以@item开头。@item后面可以跟有文本，并且一个条目的文本可以延续多个段落。

你应该在列表中的条目之间加上一个空行。这样通常会使 Info 输出文件更易于阅读。

这是一个没有参数的@enumerate示例：

```
@enumerate
@item
Underlying causes.

@item
Proximate causes.
@end enumerate
```

产生：

1. Underlying causes.
2. Proximate causes.

用‘3’作为参数的示例：

```
@enumerate 3
@item
Predisposing causes.

@item
Precipitating causes.

@item
Perpetuating causes.
@end enumerate
```

² 译者注：

1. 字母参数在输出中的效果，有时等效于数字参数，要看 Texinfo 翻译器的处理情况；
2. 在输出中，如果字母递增超过‘z’或‘Z’，则编号变得不可知，要看 Texinfo 翻译器的处理情况；
3. 字母参数，只能是单字母，不允许双字母，如‘aa’。

产生：

3. Predisposing causes.
4. Precipitating causes.
5. Perpetuating causes.

以下是总结：

a. `@enumerate`

无需参数，生成一个编号列表，第一项编号为‘1’。

b. `@enumerate` 无符号数字

使用一个无符号数字参数，以该数字作为编号列表的起始项。您可以使用此参数来继续您之前被其他文本中断的列表。

c. `@enumerate` 大写字母

以一个大写字母作为参数，创建一个列表，列表中的每一项都以该大写字母开头的字母序列进行标记。

d. `@enumerate` 小写字母

以一个小写字母作为参数，创建一个列表，其中每个项目都以该小写字母开头的字母序列进行标记。

你也可以像在文档大纲中一样，嵌套使用枚举列表。

8.4 制作两栏表格

`@table`与`@itemize` (请参阅8.2节[`@itemize`], 67 页) 类似，但允许您为每个条目指定一个名称或标题行。`@table`命令用于生成两列表格，尤其适用于术语表、解释性展示和命令行选项摘要。

8.4.1 使用`@table`命令

使用`@table`命令来生成一个两列的表格。此命令通常用于您有一系列项目和每个项目的简短描述时，例如一系列定义。

在空行之后，将`@table`命令写在行首，并在同一行后跟一个“标识”命令作为参数，如`@code`、`@samp`、`@var`、`@option`或`@kbd` (请参阅6.1节[`Indicating`], 48 页)。“标识”命令将应用于表格第一列中的文本。例如，`@table @code`将应用于第一列的文本，就像第一列的文本是`@code`命令的参数一样。

您可以将`@asis`命令作为`@table`的参数。`@asis`是一个不执行任何操作的命令：如果您在`@table`之后使用此命令，则第一列文本将按原样输出，不会进行任何高亮显示。

`@table`命令不仅可以与这里明确提及的命令一起使用，还可以与其他命令一起使用。但是，您只能使用那些在括号中接受参数的预定义 Texinfo 命令。您不能可靠地使用通过`@macro`定义的新命令，尽管使用`@alias` (针对合适的预定义命令) 是可以接受的。请参阅第 16 章[`Defining New Texinfo Commands`], 130 页。

在每个表格条目的开头，使用`@item`命令。在`@item`命令所在的行上，写下第一列的文本。在`@item`命令所在的行的下一行以及后续行上，写下第二列的文本。你可以根据需要写下任意多行的辅助文本，甚至可以是几个段落。但是，只有与`@item`命令位于同一行的文本（包括任何脚注）才会被放置在第一列。对于空的第二列，你不需要输入任何内容。

通常，应在表格条目之间添加一个空行。这样会在 Info 文件中添加一个空行，这样看起来会舒适，除非条目非常简短。

表格结尾应使用“@end table”独立行。表格结尾后应空一行，以确保输出格式的一致性。

例如，下表使用@samp命令突出显示了第一列中的文本：

```
@table @samp
@item foo
This is the text for
@samp{foo}.

@item bar
Text for @samp{bar}.
@end table
```

产生：

```
'foo'      This is the text for 'foo'.
'bar'      Text for 'bar'.
```

如果你在一个文本块有两个或更多已命名的条目(即：有两个或更多的第一列条目对应同一个第二列条目)，请使用@itemx命令。请参阅8.4.3 节[@itemx], 71 页。

在@display内部不支持使用@table命令(请参阅8.4.1 节[@table], 70 页)。由于@display是面向行的，因此将它们一起使用是没有意义的。如果您想缩进一个表格，请尝试使用@quotation(请参阅7.2 节[@quotation], 59 页)或@indentedblock(请参阅7.3 节[@indentedblock], 60 页)。

8.4.2 @ftable 和@vtable

@ftable和@vtable命令与@table命令基本相同，不同之处在于@ftable会自动将表格第一列中的每个条目加入到函数索引中，而@vtable会自动将表格第一列中的每个条目加入到变量索引中。这简化了创建索引的任务。只有与@item或@itemx命令位于同一行的条目才会被索引，并且它们会以在该行上显示的确切形式被索引。有关索引的更多信息，请参阅第 10 章[Indices], 80 页。

使用@ftable或@vtable来开始一个两列的表格，方法是在一行开头写入@ftable或@vtable，并在同一行后跟一个参数，该参数是一个 Texinfo 命令，如@code，这与@table命令的使用方式完全相同；表格结束时，在单独一行使用@end ftable或@end vtable命令。

请参阅上一节中关于@table的示例。

8.4.3 @itemx：第二条及后续条

当同一条目的第一列有两个或更多命名条目，且每个命名条目都应单独占一行时，请在表格中使用@itemx命令。

对于第一个命名条目，使用@item，对于所有后续命名条目，使用@itemx；@itemx必须始终跟在一个@item命令之后，中间不能有空行。

@itemx命令与@item命令的工作方式完全相同，只是在某些输出格式中，它不会在第一列文本上方生成额外的垂直间距。如果你有多个连续的@itemx命令，请不要在它们之间插入任何空白行。

示例：

```
@table @code
@item upcase
@itemx downcase
These two functions accept a character or a string as argument, and
return the corresponding uppercase (lowercase) character or string.
@end table
```

产生：

```
upcase
downcase  These two functions accept a character or a string as argument, and return the
           corresponding uppercase (lowercase) character or string.
```

另请注意，此示例展示了双栏表格中的多行辅助文本（即：第二列文本）。

8.5 @multitable：多列表格

@multitable 允许您构建任意列数的表格，且每列的宽度可自行设置。

您可以在 @multitable 行本身定义列宽，并在 @item 命令后编写实际表格的每一行，列之间用 @tab 命令分隔。最后，@end multitable 结束表格。详细信息见下文。

8.5.1 多列表格的列宽

你可以通过两种方式为多列表格定义列宽：一是以行长的分数表示；二是使用原型行。不支持两种方法混合使用。无论哪种方式，列宽都完全在与 @multitable 命令相同的行上定义。

1. 要将列宽指定为行长的几分之一，请在 @multitable 命令后写入 @columnfractions 和十进制数字（通常小于 1.0；允许且忽略前导零），如下所示：

```
@multitable @columnfractions .33 .33 .33
```

这些分数之和不必恰好等于 1.0，因为它们实际上并不相等。这使你能够制作出不需要使用完整行长度的表格。

在使用 @columnfractions 时，最左侧的列相对于其他列可能看起来略宽。这是因为其他列的宽度中包含了列之间的间距。

2. 要指定原型行，请在 @multitable 命令后，为每一列写入最长的条目，并用括号括起来。例如：

```
@multitable {some text for column one} {for column two}
```

第一列的宽度将等于排版后的“some text for column one”的宽度，第二列的宽度将等于“for column two”的宽度。

原型条目无需出现在表格中。

尽管本例中我们使用了简单的文本，但原型条目可以包含 Texinfo 命令；诸如 @code 之类的标记命令尤其有用。

原型行在 HTML 输出中没有效果。

8.5.2 多列表格的行

在定义列宽的 @multitable 命令（见上一节）之后，您在多列表格正文的每一行都以 @item 开始，并用 @tab 分隔列条目。在表格正文中，换行符并无特殊意义，您可以根据需要在源文件中换行。

您也可以使用`@headitem`代替`@item`来生成一个标题行。此类行的 T_EX 输出为粗体，而 HTML 和 DocBook 输出则使用`<thead>`标签。在 Info 输出中，标题行后面会跟一条由破折号(‘-’)组成的分隔线。

当模板中需要使用`@headitem`行中的条目时，可以使用命令`@headitemfont`。它是`@b`的同义词，但使用`@headitemfont`可以避免对特定字体的依赖，以防我们将来提供更改字体的方法。

以下是一个多列表格的完整示例（摘自《GNU Emacs 手册》，请参阅节“Splitting Windows”在 *The GNU Emacs Manual*）：

```
@multitable @columnfractions .15 .45 .4
@headitem 键 @tab 命令 @tab 描述
@item C-x 2
@tab @code{split-window-vertically}
@tab 将所选窗口拆分为两个窗口，一个在另一个的上方。
@item C-x 3
@tab @code{split-window-horizontally}
@tab 将所选窗口拆分为两个并排的窗口。
@item C-Mouse-2
@tab
@tab 在窗口的模式行或滚动条中，拆分该窗口。
@end multitable
```

产生：

键	命令	描述
C-x 2	<code>split-window-vertically</code>	将所选窗口拆分为两个窗口，一个在另一个的上方。
C-x 3	<code>split-window-horizontally</code>	将所选窗口拆分为两个并排的窗口。
C-Mouse-2		在窗口的模式行或滚动条中，拆分该窗口。

9 特殊展示

本章中的命令允许您编写在正常文档流之外描述特殊展示的文本（输出格式允许时）。

其中一组这样的命令用于创建浮动内容，即从主文本中分离出来的图、表格等，这些内容可能带有编号、标题，并且/或者在文档的其他地方被引用。这些展示内容中经常包含图像。

另一组命令用于在 Texinfo 中创建脚注。

9.1 浮动内容

浮动 (*float*) 是从主文本中分离出来的一种内容展示方式。它通常被标记为‘图(Figure)’、‘表(Table)’、‘示例(Example)’或类似类型。

“浮动 (*float*)”之所以这样命名，是因为在打印输出中，它原则上内容可以移动到当前页面的底部或顶部，或者移动到下一页。（在其他输出格式中，浮动没有意义。）然而，遗憾的是，在除 L^AT_EX 以外的所有输出格式中，这种浮动功能尚未实现；相反，浮动内容只是简单地输出在当前位置，或多或少就像它是 @group（请参阅 12.9 节 [@group], 103 页）一样。

9.1.1 @float [*type*][, *label*]: 浮动内容

要生成浮动内容 (*floating material*)，请将您希望独立显示的内容用 @float 和 @end float 命令括起来，并单独占一行。

浮动内容经常使用 @image 来展示已有的图形（请参阅 9.2 节 [Images], 76 页），或使用 @multitable 来展示表格（请参阅 8.5 节 [Multi-column Tables], 72 页）。然而，浮动内容可以是任何内容。以下是一个包含简单文本的示例：

```
@float 图,fig:ex1
  这是一个浮动的示例。
@end float
```

产生：

这是一个浮动的示例。

图 9.1

如示例所示，@float 接受两个参数（以逗号分隔），即 *type* (类型) 和 *label* (标签)。这两个参数都是可选的。

- | | |
|--------------|--|
| <i>type</i> | 指定这是哪种类型的浮动内容；通常是一个单词，如‘图(Figure)’、‘表(Table)’等。如果未给出此项，而 <i>label</i> 已给出，则任何交叉引用都将仅使用一个数字。 |
| <i>label</i> | 给浮动指定一个交叉引用标签。如果已指定，则会自动为此浮动分配一个编号，并且该浮动将出现在任何 @listoffloats 输出中（请参阅 9.1.3 节 [@listoffloats], 75 页）。允许对 <i>label</i> 进行交叉引用。例如，‘请参阅 @ref{fig:ex1}’将产生：请参阅图 9.1。
另一方面，如果未给出 <i>label</i> ，则该浮动将不会编号，因此不会出现在 @listoffloats 输出中，也无法进行交叉引用。 |

通常，你需要同时指定 *type* 和 *label*，以获得一个带有标签和编号的浮动内容。

在 L^AT_EX 输出中，如果存在 @float，则会在导言部分输出加载 float 包的代码。该包已经定义了一个类型为 ‘figure’ 或 ‘table’（不区分大小写）的 @float。其他浮动类型 (*type*) 会定义一个新的浮动环境，其名称基于 @float 的 *type* (类型)，并去除了字母和 ‘-’ 以外的任何字符。

在 Texinfo 中, 所有浮动都以相同的方式编号: 以章编号 (或附录字母) 开头, 后跟一个句点, 再是浮动编号, 该编号从 1,2,3,... 表示, 并在每一章重新开始计数。每种类型的浮动都是独立计数的。

在@unnumbered中或在任何章之外的浮动内容, 只需从 1 开始连续编号。

这些编号规则目前是不可更改的。

9.1.2 @caption & @shortcaption

您可以在@float(浮动)环境中的任何位置编写@caption, 以定义浮动的标题。在任何其他上下文中都不允许使用。@caption接受一个用大括号括起来的单个参数。示例如下:

```
@float 图,fig:ex2
  一个浮动的示例, 带有标题。
@caption{浮动的示例的标题。}
@end float
```

产生:

一个浮动的示例, 带有标题。

图 9.2: 浮动的示例的标题。

@caption可以出现在浮动环境内的任何位置; 它直到@end float才会被处理。标题文本通常是一两句话, 但如有必要, 也可以包含多个段落。

在输出中, 标题总是出现在浮动的下方; 这一点目前无法更改。标题前会显示浮动类型(type)和/或编号, 具体取决于@float命令的指定 (见上一节)。

@shortcaption命令定义浮动的短标题, 同样只能在@float(浮动)环境中使用, 并且接受一个用花括号括起来的参数。在浮动列表中, 将使用短标题而不是标题 (见下一节)。因此, 你可以为主文档编写一个标题, 并为浮动列表编写一个短标题。例如:

```
@float 图,fig:ex3
  一个浮动示例, 有标题。
@caption{浮动示例的标题。}
@shortcaption{用于浮动列表的文本。}
@end float
```

产生:

一个浮动示例, 带有标题。

图 9.3: 浮动示例的标题。

@shortcaption的文本中不得包含注释(@c)、逐字文本(@verb)、环境 (如@example)、脚注(@footnote)、多个段落或其他复杂结构。除非有@shortcaption, 否则相同的限制也适用于@caption。在 L^AT_EX 中, 除非有@shortcaption, 否则多段落的@caption在格式化时将导致错误。

9.1.3 @listoffloats: 浮动的目录

您可以编写一个@listoffloats命令, 为给定的浮动类型 (请参阅9.1.1 节[@float], 74 页) 生成一个浮动内容列表, 类似于文档的整体目录。通常, 该命令被写入当前文档

的一个`@unnumbered`节点中，以为列表提供标题和结构，类似于`@printindex`（请参阅10.5节[Printing Indices & Menus], 83 页）。

`@listoffloats` 接受一个可选参数，即浮动类型。示例如下：

```
@node 图的列表
@unnumbered 图的列表
@listoffloats 图
```

在没有任何参数的情况下，`@listoffloats`会生成一个未指定浮动类型的浮动内容（即`@float`命令（请参阅 9.1.1 节[`@float`], 74 页）没有第一个参数）列表。

以下是根据本章前面的示例图，`@listoffloats`的输出结果：

图 9.1:	74
图 9.2: 浮动的示例的标题。	75
图 9.3: 用于浮动列表的文本。	75

通常，浮动列表中的每一行都包含浮动类型（如果有）、浮动编号(引用标签)以及标题（如果有）---如果指定了`@shortcaption`参数，则使用该参数，否则使用`@caption`参数。根据输出格式，可能还会包含页码。

未编号的浮动内容，即没有交叉引用标签的浮动，将从浮动列表中省略。

`@listoffloats`的格式取决于输出格式。例如，在 Info 中，`@listoffloats`被格式化为一个菜单。

在 L^AT_EX 输出中，对于‘figure’（不区分大小写）浮动类型，会输出`\listoffigures`；对于‘table’（不区分大小写）浮动类型，会输出`\listoftables`。对于其他浮动类型，则会输出特定的`\listof...`命令。

9.2 插入图像

您可以使用`@image`命令插入外部文件中的图像。虽然图像可以用于任何位置，包括段落中间，但我们在本章中对其进行描述，因为它们通常作为显示的（浮动）图或示例的一部分。

9.2.1 图像语法

以下是`@image`命令的概要：

```
@image{ 文件名 [, 宽度 [, 高度 [, 替代文本 [, 扩展名 ]]] }
```

文件名参数是必填的，且不得包含扩展名，因为不同的处理器支持不同的格式：

- T_EX (DVI 输出) 读取文件文件名 .eps (封装式 PostScript 格式)。
- T_EX (PDF 输出) 会读取文件名 .pdf、文件名 .png、文件名 .jpg 或文件名 .jpeg (按此顺序)。它还会尝试读取扩展名的大写版本。PDF 格式不支持 EPS 图像，因此必须先进行转换。
- 在 Info 输出中，文件名 .txt 会被原样包含（大致就像它在`@verbatim`中一样）。Info 输出还可能包含对文件名 .png 或文件名 .jpg 的引用。（见下文。）
- 在 HTML 中，会输出对文件名 .png、文件名 .jpg、文件名 .jpeg 或文件名 .gif (按此顺序) 的引用。如果这些都不存在，则会报错，但无论如何都会输出对文件名 .jpg 的引用。
- 在 DocBook 中，对于找到的每个文件，都会输出对文件名 .eps、文件名 .gif、文件名 .jpeg、文件名 .jpg、文件名 .pdf、文件名 .png 和文件名 .svg 的引用。此外，如果存在文件名 .txt，也会原样包含在内。（后续的 DocBook 处理器应选择适当的引用。）

- 对于 L^AT_EX, 使用不带任何扩展名的文件名; 后续的 L^AT_EX 处理器应选择合适的图像类型。
- 对于 Info 和 HTML 输出, 如果指定了可选的第五个参数扩展名 (用于 @image) 并且找到了相应的文件, 则该参数将用于文件扩展名。扩展名中应包含任何前导句点。例如:

```
@image{foo,,,,.xpm}
```

如果您想安装也可供 Info 阅读器使用的图像文件, 我们建议将这些文件放在一个子目录中, 例如, 对于 *foo* 包, 可以放在 '*foo-figures*' 子目录中。应在您的 Makefile 中将文件复制到 \$(infodir)/*foo-figures*/ 目录中。

宽度和高度参数将在下一节中介绍。

如果图像是段落中的第一项, 并且后面跟着更多文本, 那么你应该在 @image 命令前加上 @indent 或 @noindent, 以指示段落格式的开始。这对于 T_EX 输出以获得正确的段落缩进尤其重要。

使用 @center 来使图像居中显示 (请参阅 2.8.3 节 [@titlefont @center @sp], 17 页)。

对于 HTML 输出, 将内联图像的 alt 属性 (alt attribute) 设置为 @image 的替代文本 (第四个参数, 可选)。如果未提供替代文本, 则使用正在显示的图像的完整文件名。替代文本 作为 Texinfo 文本处理, 因此在 HTML 输出中, 特殊字符如 '、' 和 '&' 会被转义; 此外, 您还可以使用 @- (一个不产生输出的命令; 请参阅 12.3 节 [@- @hyphenation], 102 页) 来获取一个空的不替代文本字符串。

对于 Info 输出, 替代文本字符串也会作为 Texinfo 文本进行处理并输出。在这种情况下, ' 会被转义为 \', ' 会被转义为 \'; 不会进行其他转义。

在 Info 输出中, 如果存在二进制图像文件, 则会写入对该文件的引用 (依次尝试文件名加上后缀扩展名、.扩展名、.png 或 .jpg)。如果存在 .txt 文件, 也会直接包含在内。这样, 能够显示图像的 Info 阅读器 (如运行在 X-Windows 下的 Emacs Info 浏览器) 可以显示图像, 而只能使用文本的 Info 阅读器 (如独立的 Info 阅读器) 则可以显示文本版本。

9.2.2 图像缩放

@image 命令 (见上一节) 中的可选参数宽度和高度用于指定图像的缩放大小。这些参数仅在打印输出时被考虑。

在 T_EX 中, 如果两者均未指定, 则图像以其自然大小 (在文件中给出) 呈现; 如果仅指定了一个, 则另一个会按比例缩放; 如果两者都指定了, 则两者都会被考虑, 从而可能通过改变其纵横比来扭曲原始图像。

可以使用任何有效的 T_EX 尺寸来指定宽度和高度, 即:

pt	点 (72.27 点 = 1 英吋) point (72.27pt = 1in)
pc	皮卡 (1 个皮卡 = 12 磅) pica (1pc = 12pt)
bp	大点 (72 大点 = 1 英吋) big point (72bp = 1in)
in	英吋 inch
cm	厘米 (2.54 厘米 = 1 英吋) centimeter (2.54cm = 1in)
mm	毫米 (10 毫米 = 1 厘米) millimeter (10mm = 1cm)
dd	didôt point (1157dd = 1238pt)
cc	cicero (1cc = 12dd)

sp 缩放点 (65536 缩放点 = 1 点) scaled point (65536sp = 1pt)

例如, 以下命令会将文件`ridt.eps` 垂直缩放至一英寸, 宽度按比例缩放:

```
@image{ridt,,1in}
```

为了让`@image`与 $\text{T}_{\text{E}}\text{X}$ 一起工作, 文件`epsf.tex` 必须安装在 $\text{T}_{\text{E}}\text{X}$ 能够找到它的位置。(标准位置是`texmf/tex/generic/dvips/epsf.tex`, 其中`texmf` 是您的 $\text{T}_{\text{E}}\text{X}$ 目录树的根目录。)此文件包含在 Texinfo 分发版中, 也可从<ftp://tug.org/tex/epsf.tex>等其他地方获取。

目前, 图像缩放功能仅在打印输出中实现, 在其他任何类型的输出中均未实现。

9.3 脚注

脚注 是对正文的补充或说明。¹

脚注会分散注意力, 应尽可能谨慎使用, 最好完全避免使用。标准的参考文献通常更适合放在文档末尾的参考文献列表中, 而不是分散在全文中作为脚注出现。

9.3.1 脚注命令

在 Texinfo 中, 脚注是通过`@footnote`命令创建的。此命令后紧接一个左括号, 然后是脚注文本, 最后是一个右括号作为结束。脚注可以是任意长度 (如有必要, 它们会分页显示), 但通常较短。模板如下:

```
普通文本 @footnote{脚注的文本}
```

如下所示, `@footnote`命令应紧跟在要添加脚注的文本之后, 中间不要有空格; 否则, 脚注标记可能会另起一行。

例如, 在 Texinfo 源代码中, 本条目后面会跟上一个脚注示例²; 其文本示例如下:

```
...一个脚注示例 @footnote{这里是脚注的示例。}; 其...
```

如您所见, 此源文本中包含两个相邻的标点符号; 在本例中, 序列为‘。};’。这是正常的 (第一个标点符号用于结束脚注, 而第二个标点符号属于被脚注的句子), 所以不必担心它看起来很奇怪; 另一种完全可接受的样式是将脚注放在属于句子的标点符号之后, 如‘;@footnote{...}’。

在打印输出格式中, 脚注的引用标记是一个小的上标数字; 脚注文本出现在页面底部, 位于一条水平线之下。³

在 Info 中, 脚注的引用标记是一对方括号, 括号之间是脚注编号, 如‘(1)’。如果脚注放在单独的节点中, 则引用标记后会有一个指向脚注文本的交叉引用链接 (请参阅 9.3.2 节[Footnote Styles], 79 页)。

在 HTML 输出中, 脚注引用通常用一个小小的上标数字来标记, 该数字会呈现为指向脚注文本的超文本链接。

脚注不能嵌套, 且不能出现在任何类型的章节标题或其他“特殊”位置。

最后一点提示: 在`@table`的`@item`命令的参数中, 脚注必须 (如往常一样) 与`@item`完全在同一行上。请参阅 8.4 节[Two-column Tables], 70 页。

¹ 脚注应对正文进行补充或扩展, 但读者应无需阅读脚注即可理解正文。有关脚注的详细讨论, 请参阅由芝加哥大学出版社出版的《芝加哥格式手册(The Chicago Manual of Style)》。

² 这里是脚注的示例。

³ 译者注: 也就是页脚。

9.3.2 脚注样式

在线格式有两种脚注样式，它们决定了脚注文本的位置，分别是：节点末尾(‘end’)样式和独立(‘separate’)节点样式。

‘end’ 在 Info 输出中，在节点末尾样式中，单个节点的所有脚注都放在该节点的末尾。脚注与节点其余部分之间用一行虚线分隔，虚线内包含单词‘Footnotes’。每个脚注都以‘(n)’引用标记开头。

以下是节点末尾样式中单个脚注的 Info 输出示例：

```
----- Footnotes -----
```

(1) 这里是一个脚注的范例。

在 HTML 输出中，当脚注为节点末尾样式时，或者输出未拆分⁴，则脚注会放在每个输出文件的末尾。

‘separate’

对于 Info 输出，在独立节点样式中，单个节点的使用的所有脚注都会被放置在一个自动构建的独立节点中，称为脚注节点。在此样式中，节点正文中的每个‘(n)’引用标记后都会跟有一个“脚注引用”。脚注引用实际上是一种交叉引用，用于跳转到脚注节点。

脚注节点的名称是通过在使用脚注的节点的名称后附加‘-Footnotes’来构建的。因此，Footnotes 节点的脚注节点是Footnotes-Footnotes！脚注节点有一个“Up”节点指针，指向其父节点。

以下是在独立节点样式中，本手册的第一个脚注⁵在 Info 输出中的显示效果：

```
File: texinfo.info Node: Overview-Footnotes, Up: Overview
```

(1) "Texinfo"的第一个音节发音类似于"speck"，而不是"hex"。...

在 HTML 中，当脚注样式为独立节点样式，且输出被拆分⁶时，脚注会被放置在一个单独的文件中。

除非你的文档中有长篇且重要的脚注（比如，像吉本（Gibbon）的《罗马帝国衰亡史》中的脚注那样），否则我们建议使用节点末尾样式，因为这种样式更便于读者理解。

使用@footnotestyle命令来指定脚注样式。将此命令写在行首，后跟一个参数，参数‘end’表示节点末尾样式，参数‘separate’表示独立节点样式。

示例：

```
@footnotestyle end
```

或

```
@footnotestyle separate
```

在 Texinfo 文件的前导部分(preamble)中编写一个@footnotestyle命令。

⁴ 译者注：makeinfo工具使用no-split选项

⁵ 译者注：原文有误；这不是本手册的第一个脚注，甚至不是脚注。

⁶ 译者注：makeinfo工具未使用--no-split选项

10 索引

使用 Texinfo，您无需手动排序和整理条目，即可生成索引。在索引中，条目按字母顺序排列，并附有如何查找每个条目讨论内容的信息。在打印版手册中，这些信息以页码的形式呈现。在其他格式中，则会生成指向索引条目位置或相关节点的链接。

Texinfo 提供了几种预定义的索引：函数索引、变量索引、概念索引等。你可以合并这些索引，或者将它们用于其常规用途之外。最后，你还可以定义自己的新索引。

10.1 预定义索引

Texinfo 提供了六个预定义的索引。以下是它们的名义含义、缩写以及对应的索引条目命令：

‘cp’	(@cindex) 概念索引，用于一般概念。
‘fn’	(@findex) 函数索引，用于函数和类函数名称（如：库的入口点）。
‘ky’	(@kindex) 击键索引，用于键盘命令。
‘pg’	(@pindex) 程序索引，用于程序名称。
‘tp’	(@tindex) 数据类型索引，用于类型名称（如：头文件中定义的结构）。
‘vr’	(@vindex) 变量索引，用于变量名（如：库全局变量）。

并非每本手册都需要所有这些索引，大多数手册最多只使用两到三个。例如，本手册有三个索引：概念索引、变量索引和 @ 命令索引。（最后一个实际上是函数索引，但在章节标题中被称为命令索引。）

您无需严格按照预定义索引的规范用途来使用它们。例如，假设您希望为一些 C 预处理器宏建立索引。您可以将它们与真实的函数一起放入函数索引中，只需为它们编写 @findex 命令即可；然后，当您将其“函数索引”作为未编号章 (@unnumbered) 进行打印时，您可以将其标题设为“函数和宏索引”，这样对读者来说就保持了一致性。

另一方面，最好不要偏离预定义索引的含义太远；否则，当你的文本与其他手册中的文本合并时，索引条目将无法匹配；此时，你应该定义自己的新索引（请参阅 10.7 节 [New Indices], 84 页）。

我们建议，无论使用多少个源索引，在最终文档中尽可能只设置一个索引，这样读者只需查找一个地方即可。通过使用 @synindex 或 @syncodeindex 命令，可以将两个或更多源索引合并为一个输出索引（请参阅 10.6 节 [Combining Indices], 83 页）。

基于两个字母索引名的索引条目命令也是有效的，例如，可以使用 @cpindex 来代替 @cindex。

10.2 定义索引条目

构造索引的数据来源于 Texinfo 源文件中分散的多个单独的索引命令。每个命令都指示向特定索引添加一个条目；格式化后，索引将以当前页码或节点名称作为引用目标。

索引条目由行首的索引命令及由在命令之后的其它部分指定的条目组成。

例如，本节以概念索引的以下十个条目开始：

```
@cindex Defining indexing entries
@cindex 定义索引条目 @sortas{dysytm}
```

```

@cindex Index entries
@cindex 索引条目 @sortas{sytm}
@cindex Entries for an index
@cindex 条目，索引 @sortas{tmsy}
@cindex Specifying index entries
@cindex 指定索引条目 @sortas{zdsytm}
@cindex Creating index entries
@cindex 创建索引条目 @sortas{cjsytm}

```

每个预定义的索引都有其对应的索引命令——概念索引为@cindex，函数索引为@findex，以此类推，如前一节所列。

索引条目应位于所索引的可见内容之前。例如：

```

@cindex hello
Hello, there!

```

除其他原因外，这种方式使得进入索引链接（无论在何种上下文中）的操作到达读者希望的材料之前，而非之后。

在 Info 中，索引通常以菜单的形式呈现。请尽量避免在索引条目中使用冒号，因为这可能会让一些 Info 读者感到困惑。有关菜单条目结构的更多信息，请参阅 3.9.4 节[Menu Parts], 32 页。

默认情况下，概念索引的条目以罗马字体打印，而其他索引的条目则以打字机字体打印。您可以使用常见的 Texinfo 命令来更改条目部分的打印方式，例如使用@file来打印文件名(请参阅第 6 章[Marking Text], 48 页)，以及使用@r来打印常规罗马字体(请参阅 6.2.3 节[Fonts], 56 页)。

您可以在索引命令或索引条目的文本之后，使用@sortas为索引条目指定一个明确的排序键。例如：‘@findex @sortas{\} \ @r{(literal \ in @code{@math})}’会在反斜杠下对由此产生的索引条目进行排序。

为了排序的目的，您可以选择忽略索引条目中的某些字符。目前可以选择忽略的字符有‘\’、‘-’、‘<’和‘@’，分别通过向@set命令提供以下参数来忽略：txiindexbackslashignore、txiindexhyphenignore、txiindexlessthanignore和txiindexatsignignore。例如，指定‘@set txiindexbackslashignore’会使本手册索引中的‘\mathopsup’条目在排序时被视为‘mathopsup’，从而使其出现在以“M”开头的其他条目中。这样，就不需要为包含这些字符的索引条目提供明确的排序键。

使用这些选项时，可能会生成一个排序键为空的索引条目。为避免这种情况，请在索引条目中指定一个@sortas指令。例如：

```

@set txiindexbackslashignore
@findex @sortas{\} \

```

10.3 高级索引命令

Texinfo 提供了几个用于索引的附加命令。

首先，您可以创建多级索引条目，这样您就可以将许多相关的子主题归入同一上级主题下。您可以通过使用@subentry命令来分隔此类条目的各个部分。此类命令可能如下所示：

```

@cindex Superhumans @subentry villains
@cindex Superhumans @subentry heroes

```

一个条目中最多可以有三个级别：

```

@cindex coffee makers @subentry electric @subentry pink

```

```
@cindex coffee makers @subentry electric @subentry blue
```

你可以将之前提到的@sortas命令与条目的任意部分或全部一起使用，以使它们按照与默认不同的顺序进行排序。其次，您可以使用@seeentry(“参见条目”)命令提供一个指向另一个条目的索引条目。例如：

```
@cindex Indexes @seeentry{Indices}
```

这样的条目在你的文档中应该是唯一的；其目的是将读者重定向到另一个条目，在那里他们可以找到他们正在寻找的所有信息。

最后，您可以使用@seealso命令提供一个“参见”条目。这些条目与常规条目一起，并在最终打印的索引中与它们组合在一起。例如：

```
@cindex Coffee
@cindex Coffee @subentry With milk and sugar
@cindex Coffee @subentry With doughnuts
@cindex Coffee @subentry Decaffeinated
@cindex Coffee @seealso{Tea}
```

在使用这三个高级命令时，切勿在索引文本的不同部分之间加逗号。texindex程序负责排序索引条目并生成索引格式化命令，它会为您在正确位置添加逗号。

请勿使用带有大括号的@sortas或高级命令来打断索引或@subentry条目；应将带有大括号的命令放在条目的开头或结尾。

这些特性在处理打印文档以及将 Texinfo 转换为 DocBook 时最为有用。

10.4 制作索引条目

概念索引条目由文本组成。编写索引的最佳方法是设计简洁而清晰的条目。如果你能做到这一点，那么如果条目的编写方式就像它们出现在句子中间一样，通常会使索引看起来更好，即：只对专有名词和总是需要大写的缩写进行大写。这是我们在大多数 GNU 手册索引中使用的约定。

如果你不知道如何使条目既简洁又清晰，那就让它更长、更清晰——而不是既不简洁又令人困惑。如果许多条目都是几个词长，那么如果你使用一个不同的约定：每个条目的第一个词首字母大写，索引可能会看起来更好。无论你使用哪种大小写约定，都要保持一致。

在任何情况下，都不要将区分大小写的名称（如 C 或 Lisp 的函数名或 shell 命令）的首字母大写；那样会造成拼写错误。除概念索引外，其他索引中的条目是编程语言中的符号名或程序名；这些名称通常区分大小写，因此同样需要按要求使用大写和小写。

在可行的情况下，让索引条目具有唯一性是个好主意。这样，使用打印版输出或在线索引条目补全功能的人就不会看到没有区别的列表。可以借此机会让原本相同的索引条目更具特异性，以便读者更容易找到他们要找的确切位置。10.3 节[Advanced Indexing], 81 页中描述的高级索引功能对此也有所帮助。

在制作索引条目时，考虑人们可能以不同方式查找某事物的做法是明智的。不同的人在查找某事物时，想到的关键词并不相同。一个有用的索引会包含人们可能使用的所有不同关键词下的条目。例如，一位读者可能认为，索引的两个字母名称应该列在“索引，双字母名称(Indices, two-letter names)”下，因为“索引”是总的概念。但另一位读者可能记得两个字母名称的具体概念，并搜索列为“双字母名称，用于索引(Two letter names for indices)”的条目。一个好的索引会同时包含这两种条目，从而帮助这两类读者。

与排版一样，索引的构建也是一门技艺，其精妙之处可能只有在你自己动手做时才会领悟。

10.5 打印索引和菜单

`@printindex`接受一个参数，即一个由两个字母组成的索引缩写。您必须在文档中希望索引出现的位置包含`@printindex`命令。这并不会因为您在 Texinfo 文件中使用了`@cindex`或其他生成索引条目的命令而自动发生；这些命令只是导致索引的原始数据被累积。

您应该在`@printindex`命令之前添加一个合适的节或章命令（通常是`@appendix`或`@unnumbered`），以提供章标题，并将索引放入目录中。像往常一样，在章标题之前添加一行`@node`。

例如：

```
@node Variable Index
@unnumbered 变量索引
```

```
@printindex vr
```

```
@node Concept Index
@unnumbered 概念索引
```

```
@printindex cp
```

文本‘Index’需要出现在包含索引的节点的名称中，以便 Info 阅读器能够找到该索引。

如果你有多个索引，我们建议将概念索引放在最后。

在一些输出格式中，索引输出的其他详细信息：

- 在使用 T_EX 创建打印手册的过程中，您需要运行一个名为 `texindex` 的程序（请参阅第 18 章 [Hardcopy with T_EX], 142 页）来对原始数据进行排序，以生成一个排序后的索引文件。排序后的索引文件正是实际用于打印索引的文件。

`@printindex`读取对应的已排序索引文件，并生成传统的两列索引，其中包含索引条目和页码。

- 在 Info 输出中，`@printindex`会生成一个特殊菜单，其中包含条目相对于节点起始位置的行号。Info 阅读器可以使用此功能跳转到条目的确切行，而不仅仅是包含该条目的节点。（较旧的 Info 阅读器只能跳转到节点。）示例如下：

```
* First index entry:   Top.   (line 7)
```

- 在纯文本输出中，`@printindex`的格式通常类似于 Info 中的菜单，显示每个条目相对于文件开头的行号。
- 在 HTML 输出中，`@printindex`会生成指向索引条目的链接。
- 在 DocBook 和 L^AT_EX 输出中，它只是记录要打印的索引。

10.6 合并索引

有时，你可能需要将两个完全不同的索引（如函数和概念）合并起来，这或许是因为索引的条目数量很少，为每一个索引进行独立的输出就会显得很愚蠢。

你可以通过为函数编写`@cindex`命令而不是`@findex`命令，将函数放入概念索引中，并通过打印标题为“函数和概念索引”的概念索引，而根本不打印“函数索引”，从而生成一本一致的手册；但这不是一个稳健的流程。它仅在以下情况下有效：你的文档从未被包含在旨在拥有单独函数索引的其他文档中；如果你的文档要包含在这样的文档中，那么你的文档中的函数和其他文档中的函数将不会放在一起。此外，为了让你的函数名称在概念索引中以正确的字体显示，你需要将每个函数都包含在`@code`的括号中。

10.6.1 @syncodeindex: 使用@code合并索引

当你想将函数和概念合并到一个索引中时,你应该使用@findex来索引函数,使用@cindex来索引概念,并使用@syncodeindex命令将函数索引条目重定向到概念索引中。

@syncodeindex 命令接受两个参数:一个是待重定向索引的名称,另一个是重定向目标索引的名称。模板如下:

```
@syncodeindex from to
```

为此,这些索引被赋予了两个字母的名称:

```
'cp'      概念索引
'fn'      函数索引
'ky'      键索引
'pg'      程序索引
'tp'      数据类型索引
'vr'      变量索引
```

在 Texinfo 文件开始处的头部结束(end-of-header)行之前或之后不久,编写一个@syncodeindex命令。例如,要将函数索引合并到概念索引中,请编写以下内容:

```
@syncodeindex fn cp
```

这会导致所有功能索引的条目都合并到概念索引中。

要将变量索引和函数索引合并到概念索引,请编写以下代码:

```
@syncodeindex vr cp
@syncodeindex fn cp
```

@syncodeindex命令会将‘from’索引(重定向索引)中的所有条目替换为@code字体,覆盖条目现在指向的索引所使用的任何默认字体。这样,如果将函数名称从函数索引重定向到概念索引,所有函数名称都会如您所愿地以@code字体显示。

10.6.2 @synindex: 合并索引

@synindex命令与@syncodeindex命令几乎相同,不同之处在于它不会对‘from’索引条目使用@code字体,而是使用罗马字体。因此,当您概念索引合并到函数索引中时,请使用@synindex。

请参阅10.5节[Printing Indices & Menus], 83 页,了解有关在书籍末尾打印索引或在 Info 文件中创建索引菜单的信息。

10.7 定义新索引

除了预定义的索引(请参阅10.1节[Predefined Indices], 80 页)外,您还可以使用@defindex和@defcodeindex命令来定义新的索引。这些命令会创建新的索引 @ 命令,您可以使用这些命令来标记索引条目。@defindex命令的使用方法如下:

```
@defindex name
```

新的索引名称通常为两个字母的单词,例如‘au’。例如:

```
@defindex au
```

这定义了一个新的索引,称为‘au’索引。同时,它还创建了一个新的索引命令, @auindex, 您可以使用它来创建索引条目。使用这个新的索引命令就像使用预定义的索引命令一样。

例如，这里有一个节标题，后面跟着一个概念索引条目和两个‘au’索引条目。

```
@section Cognitive Semantics
@cindex kinesthetic image schemas
@auindex Johnson, Mark
@auindex Lakoff, George
```

显然，‘au’在这里是“author(作者)”的缩写。

Texinfo 通过将索引名称与‘index’连接来构建新的索引命令；因此，定义一个‘xy’索引会自动创建一个@xyindex命令。

使用@printindex命令来打印索引，就像处理预定义索引一样。例如：

```
@node Author Index
@unnumbered 作者索引
```

```
@printindex au
```

@defcodeindex命令与@defindex命令类似，不同之处在于，在打印输出中，它默认以@code字体而不是罗马字体打印条目。

您应该在 Texinfo 文件的头部结束行(end-of-header)之前定义新的索引，并且（当然）要在任何@synindex或@syncodeindex命令之前定义(请参阅2.5 节[Texinfo File Header], 11 页)。

如前所述（请参阅10.1 节[Predefined Indices], 80 页），我们建议尽可能在最终文档中只输出一个索引（无论你使用了多少个源索引），这样读者只需查找一个地方即可。

在创建索引时，TeX会创建一个文件，其扩展名就是索引的名称(请参阅[Names of index files], 144 页)。因此，应避免使用与其他用途的扩展名（如：‘.aux’或‘.xml’）冲突的索引名称。如果新索引与已知的扩展名冲突，texi2any会报告错误。

11 特殊(字符和符号)插入

Texinfo 提供了多个命令，用于插入在 Texinfo 中有特殊含义的字符，如大括号，而对于其他与您可输入的简单字符无法对应的图形元素，这些命令同样适用。

11.1 特殊字符：插入‘@’ ‘{’ ‘}’ ‘,’ ‘\’ ‘#’ ‘&’

‘@’和‘{’ (花括号)是 Texinfo 中的基本特殊字符。要在文本中插入这些字符，您必须在这些字符前加上‘@’，以防止 Texinfo 对其产生误解。此外，还提供了字母式命令。

其他字符‘,’ (逗号)、‘\’ (反斜杠)、‘#’ (井号)、‘&’ (与号)，仅在特定上下文中具有特殊意义，具体将在相应章节中解释。

11.1.1 使用@@和@atchar{}插入‘@’

@@在输出中只生成一个‘@’字符。在@@命令后不要加大括号。

@atchar{}同样会在输出中生成一个单独的‘@’字符。与通常的字母命令一样，它的后面需要紧跟一对大括号。在嵌入条件语句中（请参阅15.4 节[Inline Conditionals], 123 页），有必要使用它，以避免在源代码中使用‘@’ 字符（及在其他一些上下文中可能更清晰）。

11.1.2 使用@{ @}和@lbracechar{} @rbracechar{}插入‘{’ ‘}’

@{在输出中只生成一个‘{’，而@}只生成一个‘}’。在@{或@}命令后不要加大括号。

@lbracechar{}和@rbracechar{}也会在输出中生成单个‘{’和‘}’字符。与通常的字母命令一样，它们需要后面的大括号。在嵌入条件语句中（请参阅15.4 节[Inline Conditionals], 123 页），有必要使用它，以避免在源代码中使用字面大括号字符（及在其他一些上下文中可能更清晰）。

11.1.3 使用@comma{}插入‘,’

通常，逗号‘,’是一个普通字符，在需要时可以直接输入到你的输入中。

然而，Texinfo 仅在一种情况下将逗号用作特殊字符：即分隔 Texinfo 命令的参数，如：@node（请参阅3.3 节[Node Line Requirements], 25 页）、@acronym（请参阅6.1.14 节[@acronym], 54 页）和@xref（请参阅第 5 章[Cross References], 39 页），以及用户定义的宏（请参阅16.1 节[Defining Macros], 130 页），这些宏需要多个参数。

由于逗号字符会混淆 Texinfo 对这些命令的解析，因此，如果你想传递一个实际的逗号，必须使用命令‘@comma{}’。以下是一些示例：

```
@acronym{ABC, A Bizarre @comma{}}
@xref{Comma,, The @comma{} symbol}
@mymac{One argument@comma{} containing a comma}
```

尽管‘@comma{}’几乎可以在任何地方使用，但除了这个特殊情况外，其他地方其实并不需要。

（顺便说一下，‘@comma’，没有对应的@,字符式命令，这仅仅是由于历史原因。它似乎并不重要到需要再定义一个同义词。）

11.1.4 使用@backslashchar{}插入‘\’

通常，在 Texinfo 中，反斜杠‘\’是一个普通字符，在需要时可以直接输入。其作用是将反斜杠以打字机字体进行排版。

然而, 在 Texinfo 中, 反斜杠在一个特定上下文中被用作特殊字符: 用于分隔用户定义宏的宏体中的形式参数 (请参阅16.1 节[Defining Macros], 130 页)。

由于宏参数解析的变幻莫测, 传递一个能生成反斜杠的字母式命令比直接使用反斜杠‘\’更为可靠。因此, 我们使用@backslashchar{}。以下是一个宏调用的示例:

```
@mymac{一个包含反斜杠 @backslashchar{}的(实)参数}
```

Texinfo 文档中, ‘\’也可作为命令字符用在@math环境中(请参阅11.7 节[Inserting Math], 93 页)。在这种情况下, @\或\backslash会生成一个“数学”反斜杠 (来自数学符号字体), 而@backslashchar{}则会像往常一样生成一个打字机反斜杠。

尽管‘@backslashchar{}’几乎可以在任何地方使用, 但除了这些特殊情况外, 其实并无必要。

11.1.5 使用@hashchar{}插入‘#’

通常, 在 Texinfo 中, 哈希符号‘#’是一个普通字符, 可以在需要时直接输入。其结果是用当前字体对哈希符号进行排版。

这个字符因地区而异, 有许多其他名称, 如“数字符号”、“英镑符号”和“八分音符”(Octotherp)。由于它与音乐符号“sharp”(升调)有些相似, 因此有时也被称为“sharp”或“sharp 符号”。在使用 Texinfo 的情况下, 根据我们的经验, “hash”(哈希)是最常见的名称。¹

然而, Texinfo 在一个特定上下文中将哈希字符用作特殊字符: 用于引入所谓的#line指令及其变体 (请参阅16.7 节[External Macro Processors], 137 页)。

因此, 为了在这样的地方排版一个实际的哈希字符 (例如, 在一个需要#line相关文档的程序中), 有必要使用@hashchar{}或其他一些构造。示例如下:

```
@hashchar{} 10 "example.c"
```

尽管‘@hashchar{}’几乎可以在任何地方使用, 但除了这个特殊情况外, 其他地方其实并不需要。

11.1.6 使用@&和@ampchar{}插入‘&’

通常, 在 Texinfo 中, 与号‘&’是一个普通字符, 可以在需要时直接输入。其作用是对“与号”字符进行排版。

然而, 仅在一个特定上下文中, “与号”字符会得到特殊处理。在定义命令 (请参阅第 13 章[Definition Commands], 105 页) 的参数中, 当使用 T_EX 处理时, “与号”后跟一系列字母可能会被特殊排版。² (请参阅13.8 节[Def Cmd Conventions], 115 页)。例如:

```
@defun foo integer1 &optional integer2 &rest integers
@code{foo} described here.
@end defun
```

产生:

```
foo integer1 &optional integer2 &rest integers [函数]
foo described here.
```

¹ 译者注: 在中文中, 也经常称为“井号”。

² 此语法用于 Emacs Lisp 关键字。请参阅节 “A Sample Function Description” 在 GNU Emacs Lisp Reference Manual。

即使没有涉及任何 @ 命令, ‘&optional’和‘&rest’也可能以粗体格式显示。

因此, 在这种情况下, 若要排版一个“与号”, 需要使用@&或其他某种构造。以下是一个以引用作为参数的 C++ 函数的示例:

```
@deftypefn 函数 int foo (const std::vector<int>& @var{bar})
@code{foo}的文档。
@end deftypefn
```

产生:

```
int foo (const std::vector<int>& bar) [函数]
foo的文档。
```

尽管‘@&’和‘@ampchar{ }’几乎可以在任何地方使用, 但除了这个特殊情况外, 其他任何地方都没有必要使用它们。

11.2 插入引号字符

作为前面章节所述的 Texinfo 通用输入约定 (请参阅2.1 节[Conventions], 8 页) 的补充说明, Texinfo 源文件使用 ASCII 字符 ` (十进制 96) 来生成左引号(‘), 使用 ASCII 字符 ' (十进制 39) 来生成右引号(’)。将这些输入字符加倍 (` ` 和 ' ') 则生成双引号 (“和”)。这些是 TeX 所使用的约定。

然而, 在计算机代码示例中, ` 和 ' 分别代表这些 ASCII 字符的典型呈现形式: 反撇字符 (独立重音符号) 和无方向单引号。在过去, 有方向的引号字形(‘和’) 在 TeX 输出中默认被使用。Texinfo 提供了这些命令来在这些替代呈现形式之间进行选择:

```
@codequoteundirected on-off
```

设置为‘off’, 以便在代码环境中将' 字符输出为右尖单引号。

```
@codequotebacktick on-off
```

设置为‘off’, 以便在代码环境中将` 字符输出为左尖单引号。

如果你只想对文档的一部分应用这些设置, 那么使用@codequote ... on将恢复为正常行为, 就像使用@codequoteundirected on一样。

这些设置会影响@code、@example、@kbd、@samp、@verb 和@verbatim。请参阅6.1.1 节[Useful Highlighting], 48 页。

遗憾的是, 一些文档查看器在复制和粘贴时会损坏有向引号字符。(免费的 PDF 阅读器 xpdf 运行良好, 但其他 PDF 阅读器, 无论是免费的还是收费的, 都存在问题。³)

此功能还可以通过在对应的变量txicodequoteundirected和txicodequotebacktick来控制, 使用@set和@clear来修改变量的值。

11.3 插入空格

以下部分, 描述控制句子内部及句后各种间距的命令。

³ 译者注: 实际上, atril并没有问题。

11.3.1 多个空格

通常，多个空白字符（空格、制表符和换行符）会被合并为一个空格。

有时，你可能需要输出几个连续的空格，这可能是出于示例的目的（例如，程序如何处理输入中的多个空格），或者仅仅是为了在标题或列表中显示。Texinfo 支持三个命令：`@SPACE`、`@TAB`和`@NL`，它们都会在输出中插入一个空格。（在这里，`@SPACE`表示一个‘@’字符后跟一个空格，即‘@ ’；`TAB`表示一个实际的制表符；`@NL`表示一个‘@’字符和换行符，即当‘@’是行上的最后一个字符时。）

示例：

```
Spacey@ @ @ @
example.
```

产生：

```
Spacey    example.
```

不要在这些命令后加花括号。

要生成一个不可中断的空白，请参见12.6 节[`@tie`], 103 页。

11.3.2 句子并未结束

在排版手册中，当句尾有句号、感叹号、问号时，其后面会加一些额外的空白。⁴

通常，Texinfo 能自动判断句号何时结束一个句子，以自动增加额外的空白⁵。然而，在某些情况下需要使用特殊命令。在后面不必自动增加额外空白的句号、问号、感叹号或冒号，后面要使用`@:`命令以取消额外的空白。在以下两个场景中，这一操作是必要的：

1. 非句尾的小写缩写的词组末尾的句号。
2. 当句子中间的括号注释(就像这个!) 以句号、感叹号或问号结尾时，应在右括号后使用`@:`。右括号和右引号（单引号和双引号）同理。

示例：

```
'foo vs.@: bar (or?)@: baz',
```

下面的第一行显示了输出结果，为了对比，第二行显示了未使用‘@:’命令时产生的额外空白。

```
foo vs. bar (or?) baz
foo vs. bar (or?) baz
```

如果你仔细观察，你会在‘vs.’和‘(or?)’之后看到一点额外的空白。

把`@:`想象成一个看不见的小写字符，它阻止单词以句号结尾，这或许能帮助你记住它的作用。

Texinfo 中的一些命令可以强制实现正常的词间距，这样你就不必在原本需要的地方插入`@:`。这些命令是用于类代码的突出显示命令，包括`@var`、`@abbr`和`@acronym`(请参阅6.1.1 节[Useful Highlighting], 48 页)。例如，在‘`@code{foo. bar}`’中，句号并不被视为句子的结束，因此不会插入额外的空白。

`@:`对 HTML 或 DocBook 输出没有影响。

⁴ 译者注：不包括中文全角的‘。’‘！’‘？’

⁵ 译者注：由于中文不使用空格作为词的间隔，也不使用空格作为句子的间隔，所以全中文 Texinfo 文件中，不必过度关心；但对中英文混合的 Texinfo 文件，在涉及英文时，仍需注意。

11.3.3 结束一个句子

如上所述, Texinfo 通常会在句尾后插入额外的空白。它采用与 $\text{T}_{\text{E}}\text{X}$ 相同的启发式规则: 句子以句号、感叹号或问号结尾, 句子后可附加可选的闭合标点(定义见下文), 然后是空格, 并且在忽略闭合标点情况下, 结尾标点前不是大写字母。⁶

在句尾以大写字母结尾时, 使用`@.`代替句号, 使用`@!`代替感叹号, 使用`@?`代替问号。这些命令后均没有大括号。例如:

```
Give it to M.I.B. and to M.E.W@. Also, give it to R.J.C@.@*
Give it to M.I.B. and to M.E.W. Also, give it to R.J.C.
```

输出结果如下。在打印输出和 Info 输出中, 您可以看到第一行中‘W’后面有预期的额外空白。

```
Give it to M.I.B. and to M.E.W. Also, give it to R.J.C.
Give it to M.I.B. and to M.E.W. Also, give it to R.J.C.
```

在其他输出格式中, `@.`等同于一个简单的‘.’; 对于`@!`和`@?`也是如此。

上文提到的闭合标点是指右括号‘)’、右方括号‘]’或右引号, 无论是单引号‘’还是双引号‘”’(这里并未包括其他可能的 Unicode 右引号)。在判断给定句号是否结束句子时, 闭合标点被无视(这与 $\text{T}_{\text{E}}\text{X}$ 的规则相同), 例如, 在‘foo.) Bar’和‘foo." Bar’中的句号确实实地结束句子。

Texinfo 中的`@.`和`@.`等的含义, 是特意设计的, 以与 Emacs 的句子移动命令(请参阅节“Sentences”在 *The GNU Emacs Manual*)良好配合。可以这样想象, 在‘`@.`’等中的‘`@`’是一个看不见的小写字母‘a’, 它使得它前面的大写字母在决定句号是否结束句子时变得无关紧要, 这可能会对理解它们的功能有所帮助。

一些 Texinfo 命令不被视为缩写(即使它们在展开时可能以大写字母结尾), 这样你就不必插入`@.`及其同类命令。值得注意的是, 类似代码的突出显示命令、以大写字母结尾的`@var`参数、`@LaTeX`和`@TeX`都属于这种情况。例如, 那个句子以‘... `@code{@TeX}`.’结尾, `@.`是不必要的。同样, 在‘... `@var{VARNAME}`. Text’中, `VARNAME`后的句号表示句子结束; 无需使用`@.`。

11.3.4 `@frenchspacing val`: 控制句子间距(句间空白)

在美式排版中, 句尾的额外空白是传统且正确的做法。这是 Texinfo 中的默认设置(在 Info 和打印输出中实现, 而在其他输出格式中则没有)。在法式排版(及其他排版风格)中, 这种额外空白是不正确的; 所有空白都是统一的。

因此, Texinfo 提供了`@frenchspacing`命令来控制标点符号后的空白。该命令将该行的剩余部分作为其参数读取, 参数必须是单个单词‘on’或‘off’(无论文档的语言是什么, 都必须这是这两个单词)。示例如下:

```
@frenchspacing on
This is text. Two sentences. Three sentences. 法式空白.
@frenchspacing off
This is text. Two sentences. Three sentences. 美式空白.
```

产生:

```
This is text. Two sentences. Three sentences. 法式空白.
This is text. Two sentences. Three sentences. 美式空白.
```

⁶ 译者注: 在中文文档的 PDF 格式输出中, 不管是不是大写字母, 都有额外空白。验证时用的工具链: `texi2pdf` (GNU Texinfo 7.2/3) & `XeTeX` 3.141592653-2.6-0.999997/8。

`@frenchspacing`会影响`@.`、`@!`和`@?`后的输出(请参阅11.3.3 节[Ending a Sentence], 90 页)。

`@frenchspacing`对 HTML 或 DocBook 输出没有影响。

11.3.5 `@dmn{dimension}`: 格式化一个度量单位

您可以使用`@dmn`命令来格式化一个度量单位, 使其在打印输出中插入足够的空隙以进行适当的排版。在其他输出格式中, 该命令根本不会插入任何空隙。

要使用`@dmn`命令, 请先写下数字, 然后紧跟着(中间不要有空格)`@dmn`, 最后在括号内写下对应的度量单位。例如:

```
A4 paper is 8.27@dmn{in} wide.
```

产生:

```
A4 paper is 8.27 in wide.
```

并非所有人都使用这种风格。有些人更喜欢使用‘8.27 in.’或‘8.27 inches’。然而, 在这种情况下, 你需要使用`@tie` (请参阅12.6 节[`@tie`], 103 页)或`@w` (请参阅12.5 节[`@w`], 102 页), 以避免在数字和单位之间(自动)换行。

此外, 如果你在句子中的缩写词后写了一个句号(如上述的‘in.’), 你应该在句号后写上‘@.’, 以避免在打印输出中插入额外的空格。请参阅11.3.2 节[Not Ending a Sentence], 89 页。

11.4 插入重音符号

以下是 Texinfo 提供的用于插入浮动重音符号的命令表。这些命令都需要一个参数, 即要加重音的字符, 该参数可以像往常一样用花括号给出(如`@'{e}`), 或者, 作为一种特殊情况, 可以省略花括号, 此时参数为下一个字符(如`@'e`)。这样做是为了尽可能方便输入和阅读源代码, 因为在某些语言中, 重音字符非常常见。

如果命令是字母式的, 例如: `@dotaccent`, 如果不使用大括号, 命令名和参数之间必须有空格。如果命令是非字母式, 例如`@'`, 则必须没有空格; 参数就是紧接其后的字符。

例外情况: `@tieaccent`的参数必须用花括号括起来(因为它由两个字符组成, 而非一个)。

在 Info 和纯文本输出中, 如果文档编码支持所需的字符, 重音结构将作为真正的重音编码字符输出, 除非向`texi2any`指定了选项`--disable-encoding` (请参阅14.2 节[`@documentencoding`], 119 页)。如果不输出编码字符, 则使用 ASCII 音译。

命令	输出	描述
<code>@"o</code>	ö	umlaut accent/元音变音符
<code>@'o</code>	ó	acute accent/扬音符
<code>@,{c}</code>	ç	cedilla accent/软音符
<code>@=o</code>	ō	macron/overbar accent/长音符
<code>@~o</code>	ô	circumflex accent/扬抑音符
<code>@'o</code>	ò	grave accent/抑音符
<code>@~o</code>	õ	tilde accent/波浪线音符
<code>@dotaccent{o}</code>	ô	overdot accent/点重音符
<code>@H{o}</code>	ő	long Hungarian umlaut/匈牙利元音变音符
<code>@ogonek{a}</code>	ą	ogonek/奥格涅克变音符
<code>@ringaccent{o}</code>	ö	ring accent/环状音符

<code>@tieaccent{oo}</code>	ô	tie-after accent/连接音符
<code>@u{o}</code>	ö	breve accent/短音符
<code>@ubaraccent{o}</code>	ō	underbar accent/下线音符
<code>@udotaccent{o}</code>	ȯ	underdot accent/下点音符
<code>@v{o}</code>	ô	caron/hacek/check accent/抑扬音符

此表列出了插入非英语语言常用的其他字符的 Texinfo 命令。

命令	输出	描述
<code>@exclamdown{}</code>	¡	倒置的 !
<code>@questiondown{}</code>	¿	倒置的 ?
<code>@aa{}</code> <code>@AA{}</code>	å Å	上面带一个小圈的 a,A
<code>@ae{}</code> <code>@AE{}</code>	æ Æ	ae 及 AE 的连体
<code>@dh{}</code> <code>@DH{}</code>	ð Ð	冰岛语字母 eth
<code>@dotless{i}</code>	ı	去掉上面点的 i
<code>@dotless{j}</code>	ȷ	去掉上面点的 j
<code>@l{}</code> <code>@L{}</code>	ł Ł	带删除线的 L,l
<code>@o{}</code> <code>@O{}</code>	ø Ø	带有斜杠的 O,o
<code>@oe{}</code> <code>@OE{}</code>	œ Œ	oe 及 OE 的连体
<code>@ordf{}</code> <code>@ordm{}</code>	º º	Spanish ordinals/西班牙语序数词
<code>@ss{}</code>	ß	德语中的 es-zet 或 尖音 S
<code>@th{}</code> <code>@TH{}</code>	þ Þ	冰岛语字母 thorn

11.5 插入引号

使用加倍的单引号字符作为引用的开始和结束: ‘...’。加倍的单引号字符会被转换为左右双引号, 如“...”。

您可能偶尔需要输出两个连续的单引号; 例如, 在记录如 Maxima 这样的计算机语言时, ‘’是一个有效的 Maxima 语言命令。您可以通过输入 ‘@w{)’ 来实现这一点; 空的 @w 命令会阻止组合成双引号。

在 Texinfo 源文件中使用的左单引号字符(`, ASCII: 96)在 ANSI 和 ISO 字符集标准中是一个抑音符。我们将其用作引号字符, 因为默认情况下, T_EX 就是这样设置的。

Texinfo 支持英语之外的其他语言中使用的多种引号。以下是 Texinfo 提供的用于插入引号的命令的表格。

命令	字形	Unicode 名字(点位)
<code>@quotedblleft{}</code>	“	左双引号(U+201C)
<code>@quotedblright{}</code>	”	右双引号(U+201D)
<code>@quoteleft{}</code>	‘	左单引号(U+2018)
<code>@quoteright{}</code>	’	右单引号(U+2019)
<code>@quotedblbase{}</code>	„	低双引号(U+201E)
<code>@quotesinglbase{}</code>	,	低单引号(U+201A)
<code>@guillemetleft{}</code>	«	双左角引号(U+00AB)
<code>@guillemetright{}</code>	»	双右角引号(U+00BB)
<code>@guilsinglleft{}</code>	<	单左角引号(U+2039)
<code>@guilsinglright{}</code>	>	单右角引号(U+203A)

对于双角引号, Adobe 和 L^AT_EX 的字形名称也受支持: @guillemotleft 和 @guillemotright。这些名称实际上是不正确的: “guillemot”是一种鸟类(一种海雀)。

标准的 T_EX 字体支持英语中常用的引号(由单或双 ASCII 单引号生成)。对于其他引号, T_EX 使用欧洲计算机现代(EC)字体(ecrm1000 和其他变体)。当然, 这些字体均可免费获取; 您可以从 <http://ctan.org/pkg/ec> 等网站下载。

免费 EC 字体是用 Metafont 创建的位图字体。对于在线查看, Type 1 (矢量) 版本的字体更为可取; 这些字体可在 CM-Super 字体包中找到 (<http://ctan.org/pkg/cm-super>)。

这两个发行版都包含了安装说明。

不同语言之间引号的使用传统存在很大差异 (https://en.wikipedia.org/wiki/Quotation_mark)。Texinfo 并未提供根据众多传统来排版引号的命令或配置。因此, 您必须选择适合您手册所用语言的命令。有时, 别名(请参阅 16.4 节[@alias], 135 页)可以简化使用, 并使源代码更具可读性。例如, 在德语中, @quotedblbase 用于左双引号, 而右双引号是由 @quotedblleft 生成的符号, 这有些违反直觉。因此, 在这种情况下, 以下别名会十分方便:

```
@alias lgqq = quotedblbase
@alias rgqq = quotedblleft
```

11.6 @sub和@sup: 插入下标和上标

您可以使用 @sub 和 @sup 命令来插入下标和上标。例如:

```
主体 @sub{下标}@sup{上标}
```

产生:

```
主体 下标 上标
```

在 Info 和纯文本输出中, @sub{下标文本} 当前输出为 ‘_{下标文本}’, @sup{上标文本} 输出为 ‘^{上标文本}’, 包括字面上的大括号(用于向读者标记上下标文本的开始和结束)。

当输出格式(和显示程序)允许(打印输出、HTML)时, 若两个命令连续给出, 则上标应置于下标之上。

对于数学表达式中的下标和上标, 最好使用 T_EX 的 ‘_’ 和 ‘^’ 字符。请参阅下一节。

11.7 @math和@displaymath: 数学材料的格式化

您可以使用 @math 命令来编写数学表达式、方程或公式。在花括号之间编写 T_EX 数学符号, 如下所示:

```
@math{\partial_{\alpha} \partial^{\alpha} A^{\beta} = \mu_0 J^{\beta}}
```

当在段落中使用时, @math 会被格式化为内联格式, 如本例中的 $\partial_{\alpha} \partial^{\alpha} A^{\beta} = \mu_0 J^{\beta}$ 。@math 命令对 Info 输出没有特殊影响, 只是逐字输出内容。

对于打印输出, @math 会切换到 T_EX 的“数学模式”。在此上下文中, 对于用于符号、函数等普通 T_EX 数学控制序列, 必须使用 ‘\’ 而不是 ‘@’。

默认情况下, HTML 输出仅对文本进行加粗处理。texi2any 提供了三个选项, 用于在 HTML 中正确显示格式化的数学内容。您可以通过 HTML_MATH 变量来选择这些选项(请参阅 21.8.7 节[HTML Customization for Math], 196 页)。当 HTML_MATH 设置为 ‘l2h’ 时, texi2any 会尝试使用 latex2html 程序为数学材料生成图像文件。当设置为 ‘t4h’ 时, texi2any 会尝试使用 tex4ht 程序。当设置为 ‘MathJax’ 时, texi2any 会在输出文件中插入对 MathJax 脚本的引用, 以格式化材料。MathJax 选项需要在浏览器中

启用 JavaScript 才能正常工作。请参阅：21.8.7.1 节[MathJax Customization Variables], 196 页、21.8.7.2 节[latex2html Customization Variables], 197 页和21.8.7.3 节[tex4ht Customization Variables], 198 页。

为显示公式，您可以使用`@displaymath`命令。示例：

```
@displaymath
f(x) = {1\over\sigma\sqrt{2\pi}}
e^{-{1\over2}\left({x-\mu\over\sigma}\right)^2}
@end displaymath
```

产生：

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{x-\mu}{\sigma}\right)^2}$$

一般来说，`@math`或`@displaymath`的内容应仅包含纯 $\text{T}_{\text{E}}\text{X}$ ，不应混杂 Texinfo 命令。如果你确实使用了任何 Texinfo 命令，那么应像往常一样用‘@’来标记它们，而不是‘\’（例如，‘@var’而不是‘\var’），但我们不保证哪些 Texinfo 命令会起作用。

尽管在某些情况下，`@sub`和`@sup`可能在数学模式内部工作，但在数学表达式中，最好使用 $\text{T}_{\text{E}}\text{X}$ 的‘_’和‘^’字符来表示下标和上标。

特定于 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 的代码仅在输出格式为 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 且启用了 `HTML_MATH` 选项时才有效（尽管 `tex4ht` 需要将 `T4H_MATH_CONVERSION` 设置为‘`latex`’；请参阅21.8.7.3 节[tex4ht Customization Variables], 198 页）。

由于与 Texinfo 的 `@sup` 命令存在冲突，在极少数情况下（仅在处理 $\text{T}_{\text{E}}\text{X}$ 时，而非输出 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 或使用任何 `HTML_MATH` 选项时），您可以将普通的 $\text{T}_{\text{E}}\text{X}$ 命令 `\sup` 替换为 `\mathopsup` 来访问。⁷

11.8 文本用图符

Texinfo 支持一些在打印文本中常用但在 ASCII 中不可用的额外图符。当然，这样的图符还有成千上万种。就 `texi2any` 而言，可以直接使用 Unicode 字符，但 $\text{T}_{\text{E}}\text{X}$ 就没那么幸运了。

11.8.1 `@TeX{} (TEX)` 和 `@LaTeX{} (LATEX)`

使用 `@TeX{}{}` 命令来生成 $\text{T}_{\text{E}}\text{X}$ 。在打印的手册中，这是一个特殊的徽标，与三个普通字母不同。在其他输出格式中，它看起来就像‘`TeX`’。

同样，使用 `@LaTeX{}{}` 命令来生成 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ ，这在打印的手册中更为特殊（并且与错误的 `LaTeX{}{}` 不同）。在其他输出格式中，结果仅为‘`LaTeX`’。

这些命令的拼写对于 Texinfo 来说是不寻常的，因为它们同时使用了大写和小写字母。

11.8.2 `@copyright{} (©)`

使用 `@copyright{}{}` 命令来生成版权符号‘©’。在可能的情况下，它是一个圆圈内的‘`c`’；否则，它是一个‘`(C)`’。

从法律上讲，根据国际条约，没有必要使用版权符号，仅用英文单词‘Copyright’就足够了。

⁷ 译者注：由于不确定翻译是否准确，所以将本段的原文附在此处：

Due to the conflict with Texinfo’s `@sup` command, you can access the plain $\text{T}_{\text{E}}\text{X}$ command `\sup` as `\mathopsup` instead, in the unlikely occurrence that you want to do this (but only when processing with $\text{T}_{\text{E}}\text{X}$, not when outputting $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ nor with any of the `HTML_MATH` options).

11.8.3 @registered symbol{} (®)

使用@registered symbol{}命令来生成注册符号‘®’。在可能的情况下，这是一个圆圈内的‘R’；否则，这是一个‘(R)’。

11.8.4 @dots (...) 和@enddots (...)

在 Texinfo 中排版时，如果直接用句点字符串表示省略号（一系列点），其间的间距会出错。因此，Texinfo 中使用了一个特殊命令：使用@dots{}命令来生成一个标准的省略号，即三个点排成连续成一行，并适当间隔，如‘...’所示。需要强调的是：不要在输入文件中直接写三个句点代替省略号，这在某些输出格式中可能可行，但在打印的手册中，句点之间的间距会不正确。

@enddots{}命令会生成一个句末省略号，它同样包含三个点，但之后的间隙不同，... 仔细看就能看出区别。

这是省略号：... 这是句末省略号：... 这是连续的三个句号：...

在打印（还有通常的 HTML）输出中，连续三个句号之间的间距比省略号中的间距更近。⁸

11.8.5 @bullet (•)

使用@bullet{}命令来生成一个大圆点，或者尽可能接近圆点的东西。也可以使用星号。这是一个圆点：•

11.8.6 @euro (€): 欧元货币符号

使用@euro{}命令来生成‘€’。在可能的情况下，这将是欧元货币的符号，否则，将使用‘Euro’这个词。

标准 T_EX 字体是在欧元立法成立之前设计的，其中并未包含欧元符号。因此，T_EX 使用了一个名为 feymr10 的额外字体（以及其它变体）。当然，该字体可免费获取；您可以从 <http://ctan.org/pkg/eurosym> 等网站下载。分发包中包含安装说明。

11.8.7 @pounds (£): 英镑

使用@pounds{}命令来生成‘£’。在可能的情况下，这是英镑（英国货币）的符号。否则，它为‘#’。

11.8.8 @textdegree (°): 度数符号

使用@textdegree{}命令来生成‘°’。在可能的情况下，这是表示度数的常规符号。否则，它是一个‘o’。

11.8.9 @minus (−): 插入减号

使用@minus{}命令来生成减号。在输出中的字符编码和字体允许的情况下，该符号的长度是减号的惯用长度---比连字符⁹稍长，比长破折号稍短：

- 是由@minus{}生成的减号，
- 是由‘-’生成的连字符，
- 是由‘---’生成的破折号。

如果你确实想排版一些使用减法的数学公式，最好使用@math，就像@math{a-b}这样。请参阅11.7节[Inserting Math]，93页。

⁸ 译者注：在 HTML 输出中，这个说法存疑。

⁹ 译者注：键盘上的-，实际上连字符。

11.8.10 @geq ($\geq$) 和@leq ($\leq$): 插入关系符号

使用@geq{}、@leq{}命令来生成大于等于符号、小于等于符号, 即‘ $\geq$ ’和‘ $\leq$ ’。当这些符号不可用时, 将输出 ASCII 序列‘>=’和‘<=’。

11.9 编程用图符

在 Texinfo 中, 代码通常在示例中展示, 用@example和@end example用于界定示例。在这些示例中, 您可以使用‘ $\Rightarrow$ ’或‘ $\mapsto$ ’来指示计算或展开的结果。同样, 也有一些命令用于插入图符来指示打印输出、错误消息、表达式等价、编辑器中的位置点以及 GUI 操作序列。

在示例中, 并不必需使用图符插入命令, 但大多数情况下都会使用。所有图符插入命令后面都跟有空大括号。

11.9.1 图符摘要

以下是图符插入命令的摘要:

$\Rightarrow$	@result{} 指示一个表达式的结果。
$\mapsto$	@expansion{} 指示宏展开的结果。
$\vdash$	@print{} 指示打印输出。
error	@error{} 指示以下文本是错误消息。
$\equiv$	@equiv{} 指示两种形式的完全等价。
*	@point{} 指示位置点。
A $\rightarrow$ B	@clicksequence{A @click{} B} 指示一个图形用户界面 (GUI) 操作序列: 首先执行 A, 然后单击 B, 或者从菜单中选择 B, 或者以其他方式选择 B。

11.9.2 @result{} ($\Rightarrow$): 表达式的结果

使用@result{}命令来指示表达式求值的结果。

@result{}命令显示为‘ $\Rightarrow$ ’, 可以是双茎箭头, 或者在无法使用双茎箭头时, 显示为 ASCII 序列‘=>’。

因此, 以下内容,

```
(cdr '(1 2 3))
 $\Rightarrow$  (2 3)
```

可以被读作: “计算(cdr '(1 2 3))得到(2 3)”。

11.9.3 @expansion{} ($\mapsto$): 标识一个展开

当表达式是一个宏调用时, 它会展开成一个新的表达式。您可以使用@expansion{}命令来指示展开的结果。

@expansion{}命令显示为‘ $\mapsto$ ’, 可以是带有平底长箭头的, 或者 (当没有长箭头时) 显示为 ASCII 序列‘==>’。

例如，以下内容

```
@example lisp
(third '(a b c))
  @expansion{} (car (cdr (cdr '(a b c))))
  @result{} c
@end example
```

产生：

```
(third '(a b c))
  ↪ (car (cdr (cdr '(a b c))))
  ⇒ c
```

这可以读作：

(third '(a b c))展开成(car (cdr (cdr '(a b c))))；表达式的计算结果是c。

通常，如本例所示，如果对@expansion{}和@result{}命令进行缩进，示例会看起来更好。

11.9.4 @print{} (⊥)：标识生成的输出

有时，一个表达式在执行过程中会生成输出。您可以使用@print{}命令来指示此类显示的输出。

@print{}命令显示为‘⊥’，要么是一个水平短划线再紧接一个竖线，要么显示 ASCII 序列‘-|’。

在以下示例中，打印输出文本用‘⊥’指示，表达式的值紧跟在最后一行。

```
(progn (print 'foo) (print 'bar))
  ⊥ foo
  ⊥ bar
  ⇒ bar
```

在 Texinfo 源文件中，这个示例写作：

```
@example lisp
(progn (print 'foo) (print 'bar))
  @print{} foo
  @print{} bar
  @result{} bar
@end example
```

11.9.5 @error{} (error)：标识错误消息

在执行一段代码时，它可能会引发错误。您可以使用@error{}命令来指示错误消息。

@error{}命令在输出中显示为‘error’，在打印输出中，该命令可能以加框的“error”形式显示，在其他格式中，可能以“error”后跟一个箭头的形式显示，或者（当没有箭头可用时）以‘error-->’的形式显示。

因此,

```
@example lisp
(+ 23 'x)
@error{} Wrong type argument: integer-or-marker-p, x
@end example
```

产生:

```
(+ 23 'x)
[error] Wrong type argument: integer-or-marker-p, x
```

这表明, 当您计算表达式时, 会打印出以下错误消息:

```
Wrong type argument: integer-or-marker-p, x
‘[error]’ 本身并不是错误消息的一部分。
```

11.9.6 @equiv{} ($\equiv$): 标识等价性

有时, 两个表达式会产生相同的结果。您可以使用@equiv{}命令来指示两种形式之间的完全等价。@equiv{}命令显示为‘ $\equiv$ ’, 可以是标准的数学等价符号 (三条平行的水平线), 或者在无法使用该符号时, 显示为 ASCII 序列‘==’。

因此,

```
@example lisp
(make-sparse-keymap) @equiv{} (list 'keymap)
@end example
```

产生:

```
(make-sparse-keymap)  $\equiv$  (list 'keymap)
```

这表明, 执行(make-sparse-keymap)和执行(list 'keymap)会产生相同的结果。

11.9.7 @point{} (*): 指示缓冲区中的位置点

有时, 您需要把 Emacs 缓冲区中的文本作为示例。在此类示例中, 通常的做法是将所提及的缓冲区的全部内容包含在两行破折号 (在行中部包含缓冲区名称) 之间。

您可以使用‘@point{}’命令来显示光标在缓冲区文本中的位置。(当然, 光标的符号并不包含在缓冲区文本中; 它指示光标位于两个字符之间的位置。)

@point{}命令显示为‘*’, 可以是带尖角的星号, 或者在无法使用带尖角的星号时, 显示为 ASCII 序列‘-!-’。

以下示例展示了在执行 Lisp 命令以插入单词changed前后, 缓冲区foo的内容。

```
----- Buffer: foo -----
This is the *contents of foo.
----- Buffer: foo -----

(insert "changed ")
⇒ nil
----- Buffer: foo -----
This is the changed *contents of foo.
----- Buffer: foo -----
```

在 Texinfo 源文件中, 示例写成这样:

```
@example
----- Buffer: foo -----
This is the @point{}contents of foo.
----- Buffer: foo -----

(insert "changed ")
  @result{} nil
----- Buffer: foo -----
This is the changed @point{}contents of foo.
----- Buffer: foo -----
@end example
```

11.9.8 点击序列

在记录图形界面时, 有必要描述诸如“点击‘文件’, 然后选择‘打开’, 然后...”之类的序列。Texinfo 提供了@clicksequence和@click命令来表示这一点, 通常使用方式如下:

```
... @clicksequence{文件 @click{} 打开} ...
```

产生:

```
... 文件 → 打开 ...
```

默认情况下, @click命令会生成一个右箭头; 此符号也可以通过命令@arrow{}单独获取。

您可以使用命令@clickstyle来更改由@click生成的标志符号, 该命令在命令行其余部分仅接受一个命令名称作为其参数。该命令应生成一个标志符号, 并且通常会省略空括号‘{}’。示例如下:

```
@clickstyle @result
... @clicksequence{文件 @click{} 打开} ...
@clickstyle @arrow{}
```

产生:

```
... 文件 ⇒ 打开 ...
```

11.10 插入 Unicode: @U

命令@U{hex}会插入 Unicode 码点为 U+hex 的字符。例如, @U{0132}会插入荷兰语中的‘IJ’连字 (‘IJ’)。

hex 值应至少包含四位十六进制数字; 不加前导零。通常, hex 必须指定一个有效的正常的 Unicode 字符; 例如, U+10FFFF (最后一个码点) 根据定义是无效的, 因此不能以这种方式插入。

@U用于插入在 Texinfo 中没有专用命令的偶尔出现的符号, 同时允许 Texinfo 源文件保持纯 7 位 ASCII 格式, 以实现最大的可移植性。

此命令存在诸多限制---与在 UTF-8 或其他二进制格式中插入 Unicode 字符时所遇到的限制相同。首先且最重要的是, T_EX 对大部分 Unicode 一无所知。虽然可以根据请求支持特定的额外字形, 但texinfo.tex无法支持全部额外的脚本 (日语、乌尔都语...) ¹⁰。@U命

¹⁰ 译者注: 本译文, 用的是texinfo-zh.tex, 可以使用中文。

令无法改变这一点。如果 $\text{T}_{\text{E}}\text{X}$ 不支持指定的字符，则会报错。 $\text{L}^{\text{A}}\text{T}_{\text{E}}\text{X}$ 输出在 UTF-8 方面有更多可能性，但可能需要额外代码来加载字体并声明如何输出 UTF-8 字符。请参阅 14.2 节 `[@documentencoding]`, 119 页。

在 HTML 和 DocBook 中，实体引用中的 $\text{\O}$ ，其输出始终是 `&#xhex;` 形式，如上例中的 `Ĳ`。即使 HTML 文档使用其他编码（例如 Latin1）且该编码不支持给定字符，此引用操作也应有效。

在 Info 和纯文本中，如果输出编码不是 UTF-8，则输出为 ASCII 序列 `'U+hex'`，如上例，将输出六个 ASCII 字符 `'U+0132'`。

12 强制或阻止断开

在某种输出格式中，换行和分页有时会出现出现在‘错误’的位置。您需要确保文本在所有输出格式中均显示正确。

例如，在打印手册中，分页可能会尴尬地出现在示例的中间；为防止这种情况，您可以使用组命令将文本组合在一起，以防止文本被拆分到两页上。相反，您也可能希望在通常不会出现分页的地方强制分页。

您可以使用换行命令、阻止换行命令、分页命令来修复有问题的换行和分页。

12.1 断开命令

断开命令用于产生或允许换行和分段：

- @* 强制换行。
- @/ 允许换行。¹
- @sp *n* 跳过*n* 个空白行。
- @- 酌情插入连字符。
- @hyphenation{*hy-phen-a-ted words*}
 在*hy-phen-a-ted words* 中定义连字符。

这些命令将文本组合在一行中：

- @w{*text*} 防止*text* 被拆分并换行。
- @tie{} 插入一个正常的词间空格，且不允许在此处换行。

页命令仅适用于打印输出，因为其他输出格式没有页面。

- @page 开始一个新页。
- @group 将必须出现在同一页上的文本放在一起。
- @need *mils*
 如果这一页空间不够*mils* 密耳，则另起一页。

12.2 @* 及@/: 强制换行及允许换行

@*命令会在所有输出格式中强制换行。

@/命令允许换行（仅限打印手册）。

这是一个带有@*的示例：

这个句子被断开 @* 成为两行。

产生：

这个句子被断开
成为两行。

在长网址或其他标识符中，当 T_EX 无法找到合适的换行位置时，@/命令可能很有用。T_EX 会自动在自然位置对网址进行换行(请参阅5.14.2 节[URL Line Breaking], 46 页)，因此仅在必要时使用@/。@/对其他输出格式没有影响。

¹ 译者注：原文没有这条，应是原文的遗漏。

12.3 @- 和@hyphenation: 打印输出中的断字

尽管 T_EX 的断字算法总体上相当不错，但它偶尔还是会漏掉一些有用的断点。（或者，更罕见的情况是，插入错误的断点。）因此，对于包括不常见词汇的文档或在进行印刷版微调时，您可能希望明确指定断点。Texinfo 为此提供了两个命令：

@- 插入可酌情使用的连字符。通过该命令，指定单词可以被跨行分割的位置。当您发现由于 T_EX 缺少连字符而导致过满的 hbox(请参阅 18.5 节 [Overfull hboxes], 146 页) 时，这一点尤其有用。T_EX 本身不会在包含 @- 的单词中插入任何连字符。

@hyphenation{hy-phen-a-ted words}

为某些单词添加断字点，在需要时，可以在此处换行。例如：

```
@hyphenation{man-u-script man-u-scripts}
```

如上所示，在单词的每个断字点处输入一个“-”。T_EX 仅在单词完全匹配时使用指定的断字点，因此请提供所有必要的变体，如复数形式。

非打印输出不会断字，因此这些命令在其他输出格式中均无效。

12.4 @allowcodebreaks: 控制@code中的换行符

通常，在 @code 及相关命令（请参阅 6.1.2 节 [code], 49 页）中，T_EX 会考虑在“-”和“_”字符处换行，这些字符或多或少就像“虚”断字点一样。

这是必要的，因为许多手册，尤其是针对 Lisp 系列语言，必须记录非常长的标识符。另一方面，有些手册没有这个问题，但您可能仍不希望在例如 SIZE_MAX 中的下划线处换行，或者更糟糕的是在 __typeof__ 中的四个下划线中的任何一个之后换行。

因此，Texinfo 提供了以下命令：

```
@allowcodebreaks false
```

以防止在 @code 中的“-”或“_”处发生换行。您可以通过设置 @allowcodebreaks true 来重新允许此类换行。请将这些命令单独写在单独的行上。

这些命令可以在文档中的任何位置给出。例如，您可以仅在一个有问题的段落关闭换行，在其它段落恢复正常；反之亦然。

此命令仅在 T_EX 输出中有效。

12.5 @w{text}: 阻止换行

@w{text} 输出 text，同时禁止 text 中出现换行符。

因此，您可以使用 @w 来生成一个不可断开的空白，其宽度固定为正常单词间距：

```
@w{ } @w{ } @w{ } 缩进。
```

产生：

缩进。

@w{ } 中的空格不仅不可断开，也不能拉伸或收缩。有时，这正是你想要的，比如当你进行手动缩进时。然而，通常你希望的是能拉伸和收缩的常规单词间距（在打印输出中）；为此，请参阅下一节中的 @tie 命令。

在打印输出中，您还可以使用 @w 命令来防止长名称或短语自动断字，例如，当它恰好位于行尾附近时。

您还可以使用 @w 来避免源代码控制系统中的不必要关键字扩展。例如，要在文档中直接写入 \$Id\$，请使用 @w{\$}Id\$。不过，此技巧对于某些输出格式的输出文件无效。

12.6 @tie{}: 插入不可换行但可变的空白

@tie{}命令会生成一个正常的词间空格，该空格处不可换行。与段落中使用的其它命令一样，始终要跟随一个空大括号。示例如下：

```
@TeX{} was written by Donald E.@tie{}Knuth.
```

产生：

```
TEX was written by Donald E. Knuth.
```

@tie{}与@w{ }之间有两个重要的不同：

- 由@tie{}生成的空白会随着段落中正常的单词间距调整而轻微伸缩；而由@w{ }生成的空间则不会发生变化。
- @tie{}允许对周围的单词进行断字，而@w{ }则禁止对这些单词进行断字（出于 T_EX 技术原因，即它会产生一个‘\hbox’）。

12.7 @sp n: 插入空白行

以@sp n开头且仅包含此代码的行会生成n个空行。@sp还会强制换行。例如，

```
@sp 2
```

产生两个空白行。

@sp命令经常用在标题页。

12.8 @page: 开始新页

在打印手册中，仅包含@page的行会开始一个新页。在其他没有页概念的格式中，它会开始一个新段落。@page命令通常用于 Texinfo 文件的@titlepage部分，以开始版权页。

12.9 @group: 阻止分页

@group命令用于将示例集中在一页上。在@example或类似结构内部使用该命令，以开始一个不可分割的垂直组，该组在打印输出中将全部显示在一页上。使用仅包含@end group的行来结束组。此命令仅在 T_EX 输出中有效。

@group和@end group命令需要位于@example和@end example命令内部，因此：

```
@example
@group
...
@end group
@end example
```

尽管@group在概念上适用于多种情境，但其当前实现仅在@example及其变体，以及@display、@format、@flushleft和@flushright中有效。请参阅第 7 章[Quotations and Examples], 58 页。（所有这些命令的共同点是，每行输入都会产生一行输出。）

12.10 @need mils: 阻止分页

在打印手册输出中，如果当前页面的剩余空间少于n 密耳(千分之一英寸)，则@need n将引起分页。不要在参数n 前后使用花括号。@need命令对其他输出格式无效，因为它们没有分页。

本段前面有一个@need命令，如果页面上剩余空间少于 800 密耳，该命令将在打印输出中另起一页。其格式如下：

```
@need 800
```

```
This paragraph is preceded by ...
```

@need命令对于阻止孤行（即：打印在页面底部的单行）的产生非常有用。

13 定义命令

`@defn`命令和其他定义命令(*definition commands*)使您能够以统一的格式描述函数、变量、宏、命令、用户选项、特殊形式和其他实体。

在输出结果中,实体的名称会与任何参数一起显示,同时还会显示实体的类别(如“函数”、“变量”或其他)。下方会缩进显示定义的正文。实体的名称也会被录入相应的索引中。

所有定义命令都遵循类似的格式。本章首先介绍`@defn`,然后再详细介绍所有其他定义命令。

13.1 定义命令的模板

```
@defn 类别(category) 名称(name) 参数(arguments)...
      定义的正文(body-of-definition)
@end defn
```

`@defn`命令用于定义类似于函数的实体---即可以接受参数的实体。将`@defn`命令写在一行的开头,在同一行中紧随其后的是实体的类别、实体本身的名称及其参数(如果有的话)。然后,在接下来的几行中编写定义的正文。最后,以单独的一行`@end defn`命令结束定义。

示例,

```
@defn 命令 forward-word count
      此命令将点向前移动 @var{count}个单词 (如果 @var{count}为负数,
      则向后移动)。
@end defn
```

产生:

```
forward-word count [命令]
  此命令将点向前移动count个单词 (如果count为负数, 则向后移动)。...
```

将类别名称首字母大写,就像标题一样¹。如果类别名称包含空格,如短语“交互式命令”,请将其括在大括号中。例如:

```
@defn {交互式 命令} isearch-forward
...
@end defn
```

否则,第二个单词会被误认为是实体的名称。一般来说,当标题行中的任何参数(除最后一个外)超过一个单词时,需要用括号括起来。如果文本中包含命令,例如:你用西班牙语写作,可能需要使用‘{declaraci@'on}’。

对于不同的输出格式,类别名称会出现在不同的位置。例如,在 Info 文件中,类别会出现在定义的第一行开头。而在 T_EX输出中,类别会打印在右边距旁边。

`@defn`自动将名称录入到函数索引中。

`@defn`的三种预定义的专门变体(`@defun`、`@defmac`和`@defspec`)分别为您指定了类别:“函数”、“宏”和“特殊形式”。(在 Lisp 中,特殊形式是一种与函数非常相似的实体。)同样,通用的`@defvr`命令也伴随着几种专门变体,用于描述特定类别的变量。

¹ 译者注:中文没有大小写。

13.2 定义命令的延续行

定义命令的标题行可能会很长。因此，Texinfo 采用了一种特殊语法，允许标题行在源文件中跨行延续：在每行末尾单独使用‘@’²以表示延续。示例如下：

```
@defun fn_name(函数名) @
      参数 1 参数 2 参数 3
这是基本的延续 defun。
@end defun
```

产生：

```
fn_name(函数名) 参数 1 参数 2 参数 3 [函数]
      这是基本的延续 defun。
```

如您所见，连续的行被合并在一起，就像它们是在同一行源代码上输入的一样。

虽然这个例子只展示了一个单行的延续，但延续可以以同样的方式扩展到任意数量的行；在每个要延续的行的末尾加上@。

通常，在@续行符前后的任意数量的空格或制表符都会被压缩成一个空格。但有一个例外：Texinfo 处理器完全不会压缩花括号内的空白。例如：

```
@defn {Category @
      Name} ...
```

输出（不是显示）中“Category”和“Name”之间存在多余空格。为避免此问题，请在输入中删除不需要的空格，或将续行符@放在花括号外。翟在任何其他上下文中，@均不作为续行字符。通常（指在其它上下文中），‘@’后跟一个空白字符（空格、制表符、换行符）会产生一个正常的词间距（请参阅 11.3.1 节[Multiple Spaces], 89 页）。

13.3 可选参数和重复参数

某些实体可以接受可选参数或重复参数。一种表示这些参数的约定是使用方括号和省略号：方括号内的参数是可选的，而后面跟有省略号的参数既是可选的，也可能重复多次。

因此，[*optional-arg*]表示*optional-arg* 是可选的，而*repeated-args...* 表示零个或更多参数。在 Lisp 中，当多个参数被分组到列表结构的附加级别时，会使用括号。

以下是一个虚构（复杂）特殊形式示例的定义行：

```
foobar (var [from to [inc]]) body... [特殊形式]
```

在此示例中，参数*from* 和*to* 是可选的，但必须同时存在或同时不存在。如果它们存在，则也可以选择性地指定参数*inc*。这些参数与参数*var* 一起组成一个列表，以区别于*body*，后者包含表单中所有剩余的元素。

在 Texinfo 源文件中，@defspec这一行是这样编写的：

```
@defspec foobar (var [from to [inc]]) body@dots{}
```

² 译者注：续行符

13.4 省略被定义的名称后面的空格

出于风格考虑或编程语言语法的需要，您可能希望在定义中，省略名称后左括号前的任何空格。为此，请设置‘`txidefnamenspace`’标志（请参阅 15.5.1 节[`@set @value`], 125 页）。例如，以下输入

```
@set txidefnamenspace
@defn 内建 index (string, substring)
@dots{}
@end defn
```

产生：

```
index(string, substring) [内建]
```

...

```
index (string, substring) [内建]
```

...

13.5 @defn等：两条或更多‘首’行

要为定义写两个或更多个‘首’行或标题行，请在第一行`@defn`后，跟上以`@defn`开头的行。

示例，

```
@defn {交互式命令} isearch-forward
@defn {交互式命令} isearch-backward
这两个搜索命令是相似的，除了...
@end defn
```

产生：

```
isearch-forward [交互式命令]
```

```
isearch-backward [交互式命令]
```

这两个搜索命令是相似的，除了...

每个定义命令都有一个‘x’形式：`@defunx`、`@defvrnx`、`@deftypefunx`等。

‘x’格式的工作方式与`@itemx`(请参见 8.4.3 节[`@itemx`], 71 页)类似。

13.6 定义命令

本节描述 Texinfo 的所有定义命令。

13.6.1 函数和类似实体

本节介绍用于描述带有简单参数的函数和类似实体的命令：

`@defn` 类别 名称 参数...

`@defn`命令是用于函数、交互式命令以及可能接受简单参数的类似实体的通用定义命令。您必须选择一个术语来描述所定义的实体的类别，例如，如果实体是函数，则可以使用“函数(Function)”。`@defn`命令写在行首，在同一行后面是所描述的实体的类别、名称以及其参数（如果有）。在单独的一行上使用`@end defn`结束定义。

例如，这里有一个定义：

```
@deffn 命令 forward-char nchars
Move point forward @var{nchars} characters.
@end deffn
```

这给出了一个名为forward-char的‘命令’的简洁定义，该命令带有一个参数nchars。

在输出格式允许的情况下，@deffn对name使用打字机字体，对arguments的其余部分使用斜体字体，就像@var生成的效果一样。

在描述文本中，使用@var明确写出参数名称，以引用该参数的值。在上面的示例中，我们以这种方式使用了‘@var{nchars}’。

在极少数情况下，当参数名称包含‘--’或其他被特殊处理的字符序列（请参阅2.1节[Conventions], 8页）时，请在特殊字符前后使用@code。这样可以避免将其转换为排版中的短破折号和长破折号。

@defun 名称 参数...

@defun命令是用于定义带有简单参数的函数的命令。@defun等同于@deffn Function ...³。在单独的一行上使用@end defun来结束定义。

@defmac 名称 参数...

@defmac命令是宏的定义命令。@defmac等同于@deffn Macro ...⁴，其工作方式类似于@defun。

@defspec 名称 参数...

@defspec命令是特殊形式的定义命令。（在Lisp中，特殊形式是一种类似于函数的实体；请参阅节“特殊形式”在GNU Emacs Lisp参考手册。）@defspec等同于@deffn {Special Form} ...⁵，其工作方式类似于@defun。

所有这些命令都会在函数索引中创建条目。

13.6.2 类型化语言中的函数

@deftypefn命令及其变体是用于描述函数的通用命令。它们特别适用于必须声明变量和函数类型的语言，如C和C++。

@deftypefn 类别 数据类型 名称 参数...

@deftypefn命令是用于函数和类似实体的通用定义命令，这些实体可以接受参数并且可以被类型化。@deftypefn命令写在行首，同一行中紧随其后的是所描述实体的类别、返回值信息（即：数据类型）、实体的名称以及其参数（如果有的话）。

示例：

```
@deftypefn {库函数库函数(Library Function)} int foobar @
(int @var{foo}, float @var{bar})
...
@end deftypefn
```

产生：

³ 译者注：在使用texinfo_zh.tex时，在打印输出中，‘Function’被自动译为‘函数’。下同。

⁴ 译者注：在使用texinfo_zh.tex时，在打印输出中，‘Macro’被自动译为‘宏’。下同。

⁵ 译者注：在使用texinfo_zh.tex时，在打印输出中，‘Special Form’被自动译为‘特殊形式’。下同。

```
int foobar (int foo, float      [库函数库函数(Library Function)]
           bar)
...
```

这意味着`foobar`是一个“库函数库函数(Library Function)”，它返回一个`int`类型的数据，其参数为`foo`(`int`类型)和`bar`(`float`类型)。`‘(库函数(Library Function))’`必须用花括号括起来，以使其成为一个整体。

在使用`@deftypefn`命令及其变体时，应使用`@var`来标记参数名称，以将其与编程语言数据的类型名称、关键字以及字面语法的其他部分区分开来。在输出格式允许的情况下，定义行上的所有输出默认采用等宽字体。`@var`在其出现的位置使用适当的字体。

如果你在描述一个使用包的语言（如 Ada）中的程序，你可能会考虑以下列方式使用`@deftypefn`：

```
@deftypefn stacks private push @
      (@var{s} :in out stack; @
      @var{n} :in integer)
...
@end deftypefn
```

（在这些示例中，`@deftypefn`参数以连续行形式展示(请参阅13.2节[Def Cmd Continuation Lines], 106 页)。)

在此例中，该过程被归类为属于包`stacks`，而非被归类为‘过程’，其数据类型被描述为`private`。该过程的名称是`push`，其参数为`s`和`n`。输出：

```
private push (s :in out stack; n :in integer)      [stacks]
...
```

当需要将参数列表解释为字面意义的计算机代码时，可以使用`@deftypefn`及其变体，同时用`@var`标记参数名称。如无需显示任何类型名称，包括返回类型，可以将类型设为空，如`{}`。

关于返回值的信息可用于为返回变量命名。例如，在 Perl 中，返回的标量可能会以下列方式被命名为`$result`和`$status`：

```
@deftypefn {Sub} {(@var{$result}, @var{$status}) =} @
                process (@var{$input})
... 失败时 @var{$status} 设为 0 ...
@end deftypefn
```

产生：

```
($result, $status) = process ($input)      [Sub]
... 失败时 $status 设为 0 ...
```

`@deftypefn`在函数索引中为名称 创建一个条目。

`@deftypefun` 数据类型 名称 参数...

`@deftypefun` 命令是专为函数定义的特殊命令。该命令等同于`@deftypefn Function ...`。

`@deftypefun`在函数索引中为名称 创建一个条目。

通常，返回类型与函数名和参数打印在同一行，如上所示。在源代码中，GNU 风格的类型化函数是将返回类型单独放在一行。因此，Texinfo 提供了一个选项来实现这一点：`@deftypefnnewline on`。

这仅影响通用函数，而不影响具有简单参数的非类型化函数（如：`@defn`）和类型化变量等。具体而言，它影响本节中的命令，以及用于面向对象语言的类似命令，即`@deftypeop`和`@deftypemethod`（请参阅 13.6.6.2 节 [Object-Oriented Methods], 113 页）。

使用`@deftypefnnewline off`将恢复为默认设置。

13.6.3 变量和类似实体

以下是定义变量和类似实体的命令：

`@defvr` 类别 名称

`@defvr`命令是一个用于类似变量的实体（用于记录值）的通用定义命令。您必须选择一个术语来描述所定义的实体类别；例如，如果实体是变量，则可以使用“变量(Variable)”。将`@defvr`命令写在一行的开头，并在同一行的后面跟上实体的类别和实体名称。

我们建议像对待标题一样，将类别名称首字母大写。

如果类别名称中包含空格，如“用户选项(User Option)”，请将其用大括号括起来。否则，第二个单词会被误认为是实体的名称。例如，

```
@defvr {用户选项(User Option)} fill-column
This buffer-local variable specifies
the maximum width of filled lines.
...
@end defvr
```

产生：

```
fill-column                                [用户选项(User Option)]
This buffer-local variable specifies
the maximum width of filled lines.
...
```

在定义以`@end defvr`结束。写在独立的一行上。

`@defvr`在变量索引中为名称 创建一个条目。

`@defvar` 名称

`@defvar`命令是用于定义变量的命令。`@defvar`等同于`@defvr Variable ...`⁶。

示例：

```
@defvar kill-ring
...
@end defvar
```

`@defvar`在变量索引中为名称 创建一个条目。

⁶ 译者注：在使用`texinfo_zh.tex`时，在打印输出中，‘Variable’被自动译为‘变量’。下同。

@defopt 名称

@defopt命令是用于定义用户选项(*User options*)的命令,即供用户根据个人喜好更改的变量;Emacs中有许多这样的选项(请参阅节“Variables”在*The GNU Emacs Manual*)。**@defopt**等同于**@defvr {User Option} ...⁷**。

13.6.4 类型化语言中的变量

类型化语言中的变量处理方式与类型化语言中的函数处理方式类似。请参阅13.6.2节[Typed Functions], 108页。通用定义命令**@deftypevr**对应于**@deftypefn**,而专用定义命令**@deftypevar**对应于**@deftypefun**。

@deftypevr 类别 数据类型 名称

@deftypevr命令是在类型化语言中对类似于变量(一种记录值的实体)的实体进行通用定义的命令。您必须选择一个术语来描述所定义实体的类别;例如,如果实体是变量,则可以使用“变量(Variable)”。

@deftypevr命令写在行首,同一行中紧随其后的是所描述实体的类别、数据类型以及该特定实体的名称。

示例:

```
@deftypevr {全局标志(Global Flag)} int enable
...
@end deftypevr
```

产生:

```
int enable                                [Global Flag(全局标志)]
...
```

@deftypevar 数据类型 名称

@deftypevar命令是类型化语言中变量的专用定义命令。**@deftypevar**等同于**@deftypevr Variable ...**。

这些命令同样会在 Variables(变量)索引中为名称 创建条目。

13.6.5 数据类型

以下是数据类型的命令:

@deftp 类别(category) 名称(name) 属性(attributes)...

@deftp命令是数据类型的通用定义命令。该命令写在行首,同一行中紧随其后的是类别、类型名称(如int或float这样的单词),以及该类型对象的属性名称。因此,您可以使用此命令来描述int或float,在这种情况下,您可以将数据类型(data type)作为类别。(数据类是特定对象的一个类别,用于决定可以对它们执行哪些操作。)

以Lisp为例, pair 命名了一种特定的数据类型,该类型的对象有两个槽,分别称为CAR 和CDR。以下是如何编写pair 定义的第一行。

⁷ 译者注:在使用texinfo_zh.tex时,在打印输出中,‘User options’被自动译为‘变量’。下同。

```
@def tp {数据类型(data type)} pair car cdr
...
@end def tp
```

产生:

```
pair car cdr                                [数据类(data type)]
...

```

@def tp在数据类型(data types)索引中创建一个条目。

13.6.6 面向对象编程

以下是用于格式化抽象对象描述的命令，如面向对象编程中所使用的。类是抽象对象的一种已定义类型。类的实例是具有该类类型的特定对象。实例变量是属于类的变量，但每个实例都有其自己的值。

13.6.6.1 面向对象的变量

这些命令允许你定义在面向对象编程语言中不同分类的变量。

@defcv 类别(category) 类(class) 名称(name)

@defcv命令是在面向对象编程中，对隶属于类的变量进行定义的通用命令。@defcv命令后面跟有三个参数：所定义事物的类别、它所隶属的类以及它的名称。例如：

```
@defcv {类选项(Class Option)} Window border-pattern
...
@end defcv
```

产生:

`border-pattern` [类选项(Class Option) 属于 Window]

@defcv在变量(variables)索引中创建一个条目。

@deftypecv 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*)

`@deftypecv`命令是在面向对象编程中，对类型化的类变量进行定义的命令。它与`@defcv`类似，但增加了`data-type`参数，用于指定实例变量的类型。

```
@deftypecv {类选项(Class Option)} Window int border-pattern
...
@end deftypecv
```

产生:

```
int border-pattern
```

 [类选项(Class Option) 属于 Window]

`@deftypepcv`在变量(variables)索引中创建一个条目。

`@defivar` 类(class) 名称(name)

`@defivar`命令是在面向对象编程中, 对实例变量(Instance Variable)进行定义的命令。`@defivar`等同于`@defcv {Instance Variable} ...`⁸。示例:

```
@defivar Window border-pattern
...
@end defivar
```

产生:

```
border-pattern                                [实例变量 属于 Window]
...
```

`@defivar`在变量(variables)索引中创建一个条目。

`@deftypeivar` 类(class) 数据类型(data-type) 名称(name)

`@deftypeivar`命令是在面向对象编程中, 对类型化实例变量(Instance Variable)进行定义的命令。它与`@defivar`类似, 但增加了数据类型(data-type) 参数, 用于指定实例变量的类型。

```
@deftypeivar Window @code{int} border-pattern
...
@end deftypeivar
```

产生:

```
int border-pattern                            [实例变量 属于 Window]
...
```

`@deftypeivar` 在变量(variables)索引中创建一个条目。

13.6.6.2 面向对象的方法

这些命令允许你定义(面向对象编程语言中)类似方法的实体。这些实体与函数类似, 一样接受参数, 但与特定类型的对象相关联。

`@defop` 类别(category) 类(class) 名称(name) 参数(arguments)...

`@defop`是定义具有简单参数的类方法实体的命令。

例如, 一些系统具有称为包装器(wrappers)的构造, 它们像方法一样与类相关联, 但行为更像宏而不是函数。你可以使用`@defop Wrapper`来描述它。

有时区分方法(methods)和操作(operations)是有用的。你可以把操作看作是对方法的规范。因此, 一个窗口系统可能会规定所有窗口类都有一个名为`expose`的方法; 我们会说这个窗口系统在窗口上定义了一个通用的`expose`操作。通常, 操作有一个名称, 并且还规定了参数的模板; 实现该操作的所有方法都必须接受相同的参数, 因为使用该操作的应用程序在不知道将由哪个方法实现的情况下使用它。

通常, 记载操作比记载方法更有意义。例如, 窗口应用程序开发人员需要了解`expose`操作, 但无需关心给定类型的窗口是否有自己的方法来实现此操作。要描述此操作, 你可以这样写:

```
@defop 操作 windows expose
```

⁸ 译者注: 在使用`texinfo_zh.tex`时, 在打印输出中, ‘Instance Variable’被自动译为‘实例变量’。下同。

产生：

```
expose                                     [操作 在 windows]
...
```

`@defop`命令写在行首，在同一行后面依次是操作的类别总名称、操作所属类的名称、操作名称以及其参数（如果有）。

`@defop`在函数索引中创建一个条目，如：‘`expose on windows`’。

`@deftypeop` 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

`@deftypeop`命令是面向对象编程中操作的通用定义命令。它特别适用于类型化的面向对象语言。它与`@defop`类似，但增加了数据类型(*data-type*)参数，用于指定方法返回值的信息，例如返回类型。

`@defmethod` 类(*class*) 名称(*name*) 参数(*arguments*)...

`@defmethod`命令是面向对象编程中方法的定义命令。方法是一种函数，它为特定类及子类的对象实现一种操作。

`@defmethod`等同于`@defop Method ...`。该命令写在行首，后面跟着方法所属类的名称、方法名称以及其参数（如果有的话）。

示例：

```
@defmethod bar-class bar-method argument
...
@end defmethod
```

产生：

```
bar-method argument                       [方法 属于 bar-class]
...
```

说明了类`bar-class`中名为`bar-method`的方法的定义。该方法接受一个参数。

`@defmethod`在函数索引中创建一个条目。

`@deftypemethod` 类(*class*) 数据类型(*data-type*) 方法名(*name*) 参数(*arguments*)...

`@deftypemethod`命令是面向对象语言中方法的定义命令，尤其适用于 C++ 和 Java 等类型化语言。它与`@defmethod`命令类似，但增加了数据类型(*data-type*)参数，用于指定方法返回值的信息，例如返回类型。

带有返回值信息的命令受`@deftypenfnewline`选项的影响（请参阅13.6.2 节[Typed Functions], 108 页）。

13.7 通用定义命令

Texinfo 提供了一些用于定义但不生成自动索引条目的命令。

您可以使用‘@defblock ... @end defblock’来创建一个通用定义环境。在此环境中，为您描述的每个符号使用@defline或@deftypeline行。例如：

```
@defblock
@defline 宏 mac (arg1, arg2)
mac 的描述。
@deftypeline 内建 int foo (int @var{bar})
foo 的描述。
@end defblock
```

产生：

```
mac (arg1, arg2)                                [宏]
    mac 的描述。

int foo (int bar)                                [内建]
    foo 的描述。
```

@defline的语法与@deffn相似。第一个参数为定义提供类别。紧随其后的是符号名称，再后是任何参数。对于包含空格的参数，应使用花括号括起来。

您使用@deftypeline的方式与@deftypefn类似，在类别后跟一个数据类型，并用@var标记任何参数。（请参阅 13.6.2 节[Typed Functions], 108 页。）

若要对多个符号使用相同的描述，可以将多行@defline放在一起。例如：

```
@defblock
@defline 函数 set-var (value)
@defline 可设置变量 var
set-var 及 var 的描述。
@end defblock
```

产生：

```
set-var (value)                                [函数]

var                                              [可设置变量]
    set-var 及 var 的描述。
```

组合使用这些命令对定义行宏（请参阅16.5 节[Line Macros], 136 页）可能有所帮助。

13.8 编写定义时的约定

手册中对于某一名称的定义不需要也不应该超过一个。包含摘要的附录应使用@table，而不是定义命令。

当你使用`@deffn`、`@defun`或其他定义命令编写定义时，请务必使用能表明含义的参数，就像`forward-word`函数的`count`参数一样。此外，如果参数名称中包含类型名称，如`integer`，请确保该参数实际上属于该类型。

字体. 由于 Texinfo 是一种语义语言，您几乎不需要在定义中使用明确的字体命令来指定字体样式（请参阅6.2.3 节[Fonts], 56 页）。但是，对于特定的输出格式和(或)结构，您可能需要解决相应的问题。以下是一些可能的解决方案：

- 使用`@r{&keyword}`显式标记 Lisp 关键字(如`&keyword`)，生成`&keyword`。
注意，在定义内容中，这些关键词（目前）在 $\text{T}_{\text{E}}\text{X}$ 中以罗马字体呈现，但在任何其他输出格式中均未采用这种格式。
- `@r{@slanted{argument}}`，生成`argument`，类似于`@var`，但在 Info 输出中避免将其参数大写。
在本处和前一点中，`@r`会将字体从定义行上下文中使用的字体（斜体或打字机字体）重置为罗马体、直立样式。
- 您可以在定义行上使用`@t`（或者甚至使用`@r{@t{...}`，使用`@r`来重置字体样式）来标记字面语法，否则文本将以斜体罗马字体输出。`@code`在这里不合适，因为它会在 Info 输出中添加引号。

要获得你想要的结果，可能需要一定程度的反复试验。与以往一样，嵌套字体命令如何组合取决于输出格式，因此应尽可能避免。

希望此类用法能尽量减少。一种可能的方法是使用`@macro`包裹这些用法(请参阅第 16 章[Defining New Texinfo Commands], 130 页)，以便将来可以方便地更改这些用法。

13.9 一个函数定义的示例

这是来自节“Calling Functions”在 *The GNU Emacs Lisp Reference Manual* 的定义，使用了`@defun`命令。函数名`apply`紧跟在`@defun`命令之后，在同一行中，其后是参数列表。

`apply function &rest arguments` [函数]

`apply` calls *function* with *arguments*, just like `funcall` but with one difference: the last of *arguments* is a list of arguments to give to *function*, rather than a single argument. We also say that this list is *appended* to the other arguments.

`apply` returns the result of calling *function*. As with `funcall`, *function* must either be a Lisp function or a primitive function; special forms and macros do not make sense in `apply`.

```
(setq f 'list)
⇒ list
(apply f 'x 'y 'z)
[error] Wrong type argument: listp, z
(apply '+ 1 2 '(3 4))
⇒ 10
(apply '+ '(1 2 3 4))
⇒ 10

(apply 'append '((a b c) nil (x y z) nil))
⇒ (a b c x y z)
```

An interesting example of using `apply` is found in the description of `mapcar`.

在 Texinfo 源文件中, 此示例的描述如下所示:

```
@defun apply function @r{&rest} arguments
@code{apply} calls @var{function} with
@var{arguments}, just like @code{funcall} but with one
difference: the last of @var{arguments} is a list of
arguments to give to @var{function}, rather than a single
argument. We also say that this list is @dfn{appended}
to the other arguments.
```

```
@code{apply} returns the result of calling
@var{function}. As with @code{funcall},
@var{function} must either be a Lisp function or a
primitive function; special forms and macros do not make
sense in @code{apply}.
```

```
@example
(setq f 'list)
  @result{} list
(apply f 'x 'y 'z)
@error{} Wrong type argument: listp, z
(apply '+ 1 2 '(3 4))
  @result{} 10
(apply '+ '(1 2 3 4))
  @result{} 10

(apply 'append '((a b c) nil (x y z) nil))
  @result{} (a b c x y z)
@end example
```

```
An interesting example of using @code{apply} is found
in the description of @code{mapcar}.
```

```
@end defun
```

在本手册中, 该函数列在命令索引中的`apply`条目下。

14 国际化

Texinfo 对非英语语言的编写提供了一定的支持，尽管这一领域仍需大量工作。（如果您正在帮助翻译写入文档中的固定字符串，请参阅19.5 节[Internationalization of Document Strings], 166 页。）

有关 Texinfo 支持的各种重音符号和特殊字符的列表，请参阅11.4 节[Inserting Accents], 91 页。

14.1 @documentlanguage *ll[_cc]*: 设置文档的语言

@documentlanguage 命令用于声明当前文档的区域设置。请将其单独写一行，放在文件开头附近。

```
@documentlanguage ll[_cc]
```

在命令名称后加上一个由两个字母组成的 ISO 639-2 语言代码 (*ll*)，后面可选择性地加上一个下划线和由两个字母组成的 ISO 3166 国家(地区)代码 (*cc*)。如果你有一个多语言文档，那么这个命令的意图是能够多次使用，以声明每次语言的变化。如果该命令根本未使用，则默认为en_US (美国英语)。

与 GNU Gettext (请参阅*Gettext*) 一样，如果省略了国家代码，则尽可能地假定使用该语言的主要方言。例如，de 等同于de_DE (德国德语)。

此命令可更改各种文档字符串 的翻译，例如交叉引用命令中的“see” (请参阅第 5 章[Cross References], 39 页)、@defuns 命令中的“Function” (请参阅第 13 章[Definition Commands], 105 页) 等。此外，此命令还可能影响texi2any对索引的排序方式 (请参阅19.4.3 节[Global Customization Variables], 157 页，位于DOCUMENTLANGUAGE_COLLATION下)。

请注意，在 Info 输出中，一些字符串（如“Node:”、“Next:”、“Menu:”等）是关键字，因此不会被翻译。

在 DocBook 输出中，@documentlanguage用于设置后面章节的语言。

对于 L^AT_EX，此命令会导致代码在导言中加载并输出‘babel’包，并输出\selectlanguage。

对于 T_EX，此命令会读取文件txi-locale.tex (如果该文件存在)。如果@documentlanguage参数包含可选的‘_cc’后缀，则会优先尝试读取该文件。例如，在@documentlanguage de_DE 的情况下，T_EX 会先查找txi-de_DE.tex，然后查找txi-de.tex。

此类txi-*文件旨在重新定义 T_EX 输出中使用的各种英语单词，如‘Chapter’、‘See’等。我们意识到，像这样的单个单词并不总是能够孤立翻译，对于表意文字（以及其他文字）来说，需要采用完全不同的策略。欢迎帮助改进 Texinfo 的语言支持。

如果正在运行的 T_EX 程序包含必要的支持，则@documentlanguage还会更改 T_EX 当前的断字模式。对于tex本身通常不会出现这种情况，但对于扩展的 T_EX 程序etex (DVI 输出) 和pdftex (PDF 输出) 的最新发行版通常会出现这种情况。如果扩展的 T_EX 可用，则texi2dvi将使用它们(请参阅[Format with texi2dvi], 142 页)。

由于语言代码和国家代码的列表更新相对频繁，我们在此不一一列出。有效的语言代码可在 ISO 639 官方主页上找到，网址为<http://www.loc.gov/standards/iso639-2/>。国家代码和 ISO 3166 官方网站可通过https://en.wikipedia.org/wiki/ISO_3166找到。

14.2 @documentencoding enc: 设置输入编码

默认情况下, 输入和输出文档的编码被假定为 UTF-8, 这是一种广泛使用的全球字符编码, 以 8 位字节表示。UTF-8 与 7 位 ASCII 兼容。建议 Texinfo 手册使用 UTF-8 编码。

@documentencoding 命令用于声明输入文档的编码, 同时也会影响输出的编码。如果您的文档编码不是默认编码, 请将其单独占一行, 并在其后附上有效的编码规范, 放在文件开头附近。

```
@documentencoding enc
```

UTF-8 应始终是编码的最佳选择。Texinfo 仍然支持其他编码, 主要是为了与旧手册兼容¹:

US-ASCII 基于英文字母的字符编码。

ISO-8859-1

ISO-8859-15

ISO-8859-2

这些分别规定了西欧语言(前两种)和东欧语言(第三种)的预 UTF-8(pre-UTF-8)标准编码。ISO 8859-15 用更常用的字符(€)替换了 8859-1 中一些较少使用的字符(如:预组合分数)。

对编码的完整描述超出了本文的范围; 一个有用的参考资料是<http://czyborra.com/charsets/iso8859.html>。

koi8-r 这是在 UTF-8 出现之前, 俄语常用的一种编码方式。

koi8-u 这是 UTF-8 出现之前乌克兰语常用的编码方式。

在 Info 输出中, 会输出一个所谓的‘Local Variables(局部变量)’章节(请参阅节“File Variables”在 *The GNU Emacs Manual*), 其中包含了输出编码。这允许 Info 阅读器适当地设置编码。其看起来是这样:

```
Local Variables:
coding: UTF-8
End:
```

默认情况下, 对于 Info 输出和纯文本输出, texi2any 会尽可能在输出编码中输出重音符号和特殊字符(如@'e)的实际 UTF-8 序列或 8 位字符。如果无法实现, 或者如果指定了选项--disable-encoding, 则使用 ASCII 音译。

在 HTML 输出中, 会在 HTML 的‘<head>’部分输出一个‘<meta>’标签, 用于指定编码。Web 服务器和浏览器会协同使用这些信息, 以便在系统支持的情况下, 使用正确的编码来显示页面。其格式如下:

```
<meta http-equiv="Content-Type" content="text/html;
charset=utf-8">
```

在 HTML 和 L^AT_EX 输出中, 如果设置了 OUTPUT_CHARACTERS(请参阅 19.4.7 节[Customization Variables for Multiple Formats], 164 页), 则重音符号和特殊字符(如: @'e或``)将尽可能以实际 UTF-8 序列或 8 位字符的形式输出到输出编码中。否则, 在 HTML 中, 这些字符将使用 HTML 实体(HTML entities), 而在 L^AT_EX 中, 将使用 L^AT_EX 宏。

在 DocBook 输出中, 如果编码格式不是 UTF-8, 则会在 XML 声明中添加一个 encoding 属性。如果设置了 OUTPUT_CHARACTERS(请参阅 19.4.7 节[Customization Variables for Multiple

¹ texi2any 支持 Texinfo 手册的更多编码, 可能包括 Perl 和 iconv 都支持的所有编码(请参阅节“Generic Charset Conversion”在 *The GNU C Library*)。然而, 输出格式的支持可能不足, 尤其是对于 L^AT_EX 输出。

Formats], 164 页), 则诸如`e' e`之类的重音符号构造, 在可能的情况下, 会以输出编码中的实际 8 位字符或 UTF-8 字符的形式输出。否则, 这些构造将使用 XML 实体(entities)。

在 $\mathrm{T}_{\mathrm{E}}\mathrm{X}$ 输出中, 标准计算机现代(Computer Modern)字体中支持的字符会相应地输出。例如, 这意味着使用构造的重音符号而非预制的字形。使用缺失的字符会生成警告消息, 指定未实现的编码也会生成警告消息。

尽管现代 $\mathrm{T}_{\mathrm{E}}\mathrm{X}$ 系统支持世界上几乎所有使用的文字, 但`texinfo.tex`²中并未提供这样广泛支持, 而且复制或整合所有这些努力也是不可行的。

在 $\mathrm{L}_{\mathrm{A}}\mathrm{T}_{\mathrm{E}}\mathrm{X}$ 输出中, 根据源文件的编码, 输出加载‘`inputenc`’包的代码。这本身并不能确保输入文档中的所有字符都能随后输出。默认情况下使用的字体应涵盖特定的 Texinfo 字形, 但并非所有可能的编码字符。您可能需要在导言(preamble)中加载不同的字体, 并使用带有 UTF-8 编码的`\DeclareUnicodeCharacter`。例如:

```
@latex
\DeclareUnicodeCharacter{017B}{\Z}
@end latex
```

节点名称中包含非 ASCII 字符时, 不同字符编码的 Info 文件之间的交叉引用会失败。我们强烈建议, 对于交叉引用目标中包含非 ASCII 字符的手册, 仅使用 UTF-8 作为编码。

² 译者注: 本译文文档使用`texinfo-zh.tex`。

15 条件可见的文本

条件命令(*conditional commands*) 允许您根据不同的输出格式或您定义的某些条件使用不同的文本。例如, 您可以使用它们为打印手册和 Info 输出指定不同的文本。

条件指令包括以下类别。

- 特定于输出格式 (Info、T_EX、HTML、...) 的命令。
- 特定于任何输出格式的命令, 但不包括 给定的格式 (例如, 不包括 Info、T_EX、...等)。
- 适用于任何输出格式的‘原始’格式化程序的文本, 直接传递, 对 @ 命令进行最小化 (但不是完全没有) 的解释。
- 与格式无关的变量替换, 以及检查变量是否已设置或清除。

15.1 肯定条件命令

Texinfo 为每种输出格式提供了一个@if... .. @end if...环境, 以允许根据特定输出格式有条件地包含文本。

@ifinfo ... @end ifinfo用于标记在非 Info 输出 (尤其是打印输出) 中应被忽略的文本段。这些文本段仅出现在 Info 输出中; 并且出于历史兼容性考虑, 这些文本段也出现在纯文本输出中。

其他格式的环境与此类似:

@ifdocbook ... @end ifdocbook
文本仅在 DocBook 输出中出现。

@ifhtml ... @end ifhtml
文本仅在 HTML 输出中显示。

@iflatex ... @end iflatex
文本仅在 L^AT_EX输出中显示。

@ifplaintext ... @end ifplaintext
文本仅在纯文本输出中显示。

@iftex ... @end iftex
文本仅在打印手册中出现。

@ifxml ... @end ifxml
文本仅在 XML 输出中显示。

在源文件中, @if...和@end if...命令必须单独占一行。命令后的换行符 (不论多少个) 被视为空白, 这样条件文本会正常连入周围的段落中。

@if...构造用于对常规 Texinfo 源文件进行条件化处理; 15.3 节[Raw Formatter Commands], 123 页, 是直接使用底层格式化命令。

以下是一个展示所有这些条件的示例:

```
@iftex
  本文本仅出现在打印手册输出中。
@end iftex
@ifinfo
  本文本仅出现在 Info 及纯文本输出中。
@end ifinfo
```

```

@ifhtml
  本文本仅出现在 HTML 输出中。
@end ifhtml
@iflatex
  本文本仅出现在 @LaTeX{} 输出中。
@end iflatex
@ifplaintext
  本文本仅出现在纯文本输出中。
@end ifplaintext
@ifxml
  本文本仅出现在 XML 输出中。
@end ifxml
@ifdocbook
  本文本仅出现在 DocBook 输出中。
@end ifdocbook

```

上述示例将生成以下行：

本文本仅出现在打印手册输出中。

请注意，根据您阅读的手册版本不同，您可能只会看到其中一条输入行。

在复杂的文档中，您可能希望 Texinfo 在某些永远不应被处理的条件下发出错误消息。`@errmsg{text}` 命令可以实现这一点；它接受一个参数，即错误消息的文本。

我们在此提及 `@errmsg{}`，尽管它与条件语句没有严格关系，但在实践中它极有可能在此种上下文中发挥作用。从技术上讲，它可以在任何地方使用。关于 `@errmsg` 在 T_EX 中输出的行号的注意事项，请参阅 16.7 节 [External Macro Processors], 137 页。

15.2 否定条件语句命令

您可以使用 `@ifnot... .. @endnot...` 环境来指定，在除指定格式外的其它输出格式中包含文本：

```

@ifnotdocbook ... @end ifnotdocbook
@ifnothtml ... @end ifnothtml
@ifnotininfo ... @end ifnotininfo
@ifnotlatex ... @end ifnotlatex
@ifnotplaintext ... @end ifnotplaintext
@ifnottex ... @end ifnottex
@ifnotxml ... @end ifnotxml

```

在您的实际源文件中，`@ifnot...` 命令和 `@end` 命令必须单独占一行。

如果输出文件是指定输出格式的，则该文本会被忽略。否则，该文本会被包含在内。

（出于历史兼容性考虑）有一个例外：`@ifnotininfo` 文本在 Info 输出和纯文本输出中都会被省略，而不仅仅是 Info 输出。要指定仅在 Info 输出中显示而在纯文本输出中不显示的文本，请同时利用 `@ifnotplaintext`，如下所示：

```

@ifinfo
@ifnotplaintext
  在 Info 中出现，纯文本中不出现。This will be in Info, but not plain text.
@end ifnotplaintext

```

```
@end ifinfo
```

由前述的条件命令所划分的区域中，包含的是与@iftex一样的普通 Texinfo 源代码，而不是与@tex一样(请参阅15.3 节[Raw Formatter Commands], 123 页)的原始格式化程序的源代码。

15.3 原始格式化命令

刚才描述的@if...条件语句仅能用于普通的 Texinfo 源代码。例如，纯 T_EX 的大多数特性在@iftex中都无法使用。@if...的目的是为 Texinfo 源代码提供条件处理，而不是提供对底层格式化特性的访问。为此，Texinfo 提供了所谓的原始格式化命令(*raw formatter commands*)。它们仅在真正需要时才应使用（大多数文档并不需要）。

第一个原始格式化命令是@tex。你可以完全输入直接 T_EX，并在 T_EX 命令中使用‘\’，方法是用@tex ... @end tex划定一个区域。在该区域内，所有纯 T_EX 命令和类别代码都能被使用。唯一的例外是@字符仍然表示一个命令，以便@end tex能被识别。除非正在生成 T_EX 输出，否则 Texinfo 处理器不会输出此区域中的内容。

在复杂情况下，您可能希望在@tex中定义新的 T_EX 宏。您必须使用\gdef来执行此操作，而不是\def，因为@tex区域是在一个 T_EX 组中处理的。如果您需要多次定义，您可能希望设置\globaldefs=1 (当组在@end tex处结束时，其值将像往常一样恢复为零，因此它不会对文档的其余部分造成问题)。

作为一个例子，这里展示了一个用纯 T_EX 编写的方程式：

```
@tex
$$ \chi^2 = \sum_{i=1}^N
    \left( \frac{y_i - (a + b x_i)}{\sigma_i} \right)^2 $$
@end tex
```

本例的输出仅会出现在打印手册中。如果你正在阅读非 T_EX 生成的格式，你将无法看到打印手册中出现的公式。

$$\chi^2 = \sum_{i=1}^N \left(\frac{y_i - (a + bx_i)}{\sigma_i} \right)^2$$

类似地，您可以使用@html ... @end html表示原始 HTML 区域，@docbook ... @end docbook表示原始 DocBook 区域，@latex ... @end latex表示原始 L^AT_EX 区域，以及@xml ... @end xml表示原始 XML 区域。

原始区域中的换行符行为未作规定。

在所有情况下，在原始处理中，@的含义与文档其它部分保持一致；所以，无论最终输出格式如何，Texinfo 处理器都必须在一定程度上识别甚至执行原始区域的内容；因而，在原始区域内指定全局影响文档的更改会导致不可预测且通常不受欢迎的行为。例如，在原始区域内使用@kbbinputstyle命令是未定义的；补救措施很简单：不要这样做。

15.4 内联条件语句：@inline、@inlineifelse、@inlineraw

Texinfo 提供了一组条件命令，其参数用花括号括起来：

```
@inlinefmt{格式(format), 文本(text)}
```

如果正在生成format 输出，则处理 Texinfo 的text。

`@inlinefmtifelse{格式(format), 肯定文本(then-text), 否定文本(else-text)}`
 如果正在生成`format` 输出, 则处理 Texinfo 的`then-text` ; 否则, 处理`else-text` 。

`@inlineraw{格式(format), 文本(text)}`
 类似, 但用于原始的`text` (请参阅15.3 节[Raw Formatter Commands], 123 页)。

支持的格式(format)有:

docbook html info latex plaintext tex xml

示例:

`@inlinefmt{html, @emph{仅在 HTML 里的文本}}`

接近等效于:

```
@ifhtml
@emph{仅在 HTML 里的文本}
@end ifhtml
```

除了可以避免后一种 (`@if...` 环境) 方法添加的空格。

在这些命令中, 像往常一样, 逗号分隔参数后, 空格会被忽略, 但在文本(`text`)结尾处的空格则不会被忽略。

要在参数中插入字面意义‘@’、左大括号、右大括号, 必须使用字母命令`@atchar{}`(请参阅11.1.1 节[Inserting an Atsign], 86 页)和`@lbracechar{}`或`@rbracechar{}`(请参阅11.1.2 节[Inserting Braces], 86 页), 否则解析将出现混乱。

使用`@inlinefmtifelse`时, 还需要使用`@comma{}`来避免将文本(`text`)中的‘,’误认作分隔符。使用`@inlinefmt`和`@inlineraw`时, 则不需要使用`@comma{}` (尽管使用它也没问题), 因为这两个命令总是恰好有两个参数。

对于 TeX, 处理后的文本(`text`) 不能包含以换行符分隔的命令。被忽略的文本 (即: 非 TeX) 则可以包含。

另外两个`@inline...` 条件语句是对`@ifset`和`@ifclear`命令的补充; 请参阅下一节。

15.5 标志: @set、@clear、条件语句和@value

您可以使用`@set`、`@clear`、`@ifset`和`@ifclear`命令来指示 Texinfo 格式化命令格式化或忽略 Texinfo 文件的部分内容。以下是概要:

`@set` 标志(flag)[值(value)]

如果指定了可选的值(`value`), 则将变量标志(flag) 设置为该值。

`@clear` 标志(flag)

无论变量标志(flag) 之前是否已定义, 都将取消其定义。

`@ifset` 标志(flag)

如果设置了标志(flag), 则直到下一个`@end ifset`命令前的文本将被格式化。如果`flag` 未设置, 则直到下一个`@end ifset`命令前的文本将被忽略。

`@inlineifset{标志(flag), 文本(text)}`

`@ifset`的大括号分隔版本。

`@ifclear` 标志(flag)

如果设置了标志(flag), 则忽略直到下一个`@end ifclear`命令前的文本。如果`flag`未设置, 则格式化直到下一个`@end ifclear`命令前的文本。

`@inlineifclear{标志(flag), 文本(text)}`

`@ifclear`的大括号分隔版本。

15.5.1 @set和@value

您使用@set命令为标志(flag)指定一个值(value), 该值稍后可由@value命令进行展开。

一个标志(flag)(也称为变量(variable)) 名称是一个以字母、数字、‘-’或‘_’开头的标识符。其后续字符(如果有的话)不能是空格、‘@’、大括号、尖括号及‘~`^+|’中的任何字符; 其他字符, 如‘%’, 则可以使用。然而, 在标志名称中最好只使用字母和数字, 不要使用‘-’或‘_’或其他字符---由于 T_EX的限制, 它们在某些上下文中可以使用, 但不是在所有上下文中都可用。

值(value)在输入行的剩余部分中, 可以包含任何内容。

像这样编写@set 命令:

```
@set foo 这是一个字符串(This is a string).
```

将标志foo的值设置为“这是一个字符串(This is a string).”

Texinfo 处理器将@value{ 标志(flag) }命令替换为标志(flag) 的值。因此, 当如上所示设置foo时, Texinfo 处理器将这样转换处理:

```
@value{foo}
```

展开成:

```
这是一个字符串(This is a string).
```

你可以在段落中写一个@value命令; 但你必须单独在一行上编写一个@set命令。

如果你这样编写@set命令:

```
@set foo
```

如果不指定字符串, 则foo的值是空字符串。

如果你使用@clear flag 清除了之前设置的标志, 那么后续的@value{flag}命令将报告错误。

例如, 如果你按如下方式设置howmuch:

```
@set howmuch very, very, very
```

然后让 Texinfo 处理器进行转换

```
It is a @value{howmuch} wet day.
```

产生:

```
It is a very, very, very wet day.
```

如果您写:

```
@clear howmuch
```

然后让 Texinfo 处理器进行转换

```
It is a @value{howmuch} wet day.
```

产生:

```
It is a {No value for "howmuch"} wet day.
```

@value不能可靠地作为重音命令的参数使用(请参阅 11.4 节[Inserting Accents], 91 页)。

例如, 这样会失败:

```
@set myletter a
@'@value{myletter}
```

15.5.2 @ifset和@ifclear

当标志(flag)已设置时, Texinfo 格式化命令会对后续的@ifset 标志(flag) ... @end ifset环境中的文本进行格式化。当flag已清除时, Texinfo 格式化命令则不会 对文本进行格式化。@ifclear的操作与此类似。

在@ifset 标志(flag) 和@end ifset命令之间编写条件格式化文本, 如下所示:

```
@ifset 标志(flag)
  条件格式化文本(conditional-text)
@end ifset
```

例如, 你可以创建一个包含两种变体的文档, 比如一份针对‘大’型号和‘小’型号的手册:

你可以用这台机器挖出灌木, 而不会伤害到它们。

```
@set large
```

```
@ifset large
  它还能挖出已经长成的大树。
@end ifset
记得及时重新栽种...
```

在该示例中, 由于设置了large标志, 格式化命令将对@ifset large和@end ifset之间的文本进行格式化。

当标志(flag) 被清除时, Texinfo 格式化命令不会格式化@ifset 标志(flag)和@end ifset之间的文本, 该文本将被忽略, 不会出现在输出中。

例如, 如果在@set large命令之后(但在@ifset large之前)编写@clear large命令来清除前例中的标记, 那么 Texinfo 格式化命令会忽略@ifset large和@end ifset命令之间的文本。在格式化输出中, 该文本不会出现; 你只会看到这样的几行: “你可以用这台机器挖出灌木, 而不会伤害到它们。记得及时重新栽种...”。

如果使用@clear flag 命令清除了某个标志, 那么格式化命令会格式化在后续的成对的@ifclear和@end ifclear命令之间文本。但如果使用@set flag 命令设置了该标志, 那么格式化命令不会格式化在@ifclear和@end ifclear命令之间的文本, 该文本会被忽略。@ifclear命令的格式如下:

```
@ifclear 标志(flag)
...
@end ifclear
```

15.5.3 @inlineifset和@inlineifclear

@inlineifset和@inlineifclear提供了与@ifset和@ifclear形式类似的、以大括号分隔的替代形式, 类似于其他@inline...命令(请参阅15.4 节[Inline Conditionals], 123 页)。此处同样存在与前文相同的关于参数解析的注意事项。

```
@inlineifset{变量(var), 文本(text)}
  如果变量变量(var) 已定义, 则处理 Texinfo 文本(text)。
```

```
@inlineifclear{变量(var), 文本(text)}
  如果变量变量(var) 未定义, 则处理 Texinfo 文本(text)。
```

除了语法之外, 它们的一般行为和用途与上一节中描述的@ifset和@ifclear相同。

15.5.4 @value示例

您可以使用@value命令，来减少在记录手册的更新时，需要更改的位置数量。有关使用此命令处理 Automake 分发(distributions)的示例全文，请参阅节 C.1 [GNU Sample Texts], 234 页。您可以使用@value命令来减少在记录手册更新时需要更改的位置数量。请参阅节 C.1 [GNU Sample Texts], 234 页，获取它与 Automake 分发进行协同工作的示例全文。

此示例改编自 *The GNU Make Manual*。

1. 设置标志：

```
@set EDITION 0.35 Beta
@set VERSION 3.63 Beta
@set UPDATED 14 August 1992
@set UPDATE-MONTH August 1992
```

2. 为@copying部分(请参阅2.7.1 节[@copying], 15 页)编写文本：

```
@copying
This is Edition @value{EDITION},
last updated @value{UPDATED},
of @cite{The GNU Make Manual},
for @code{make}, version @value{VERSION}.
```

```
Copyright ...
```

```
Permission is granted ...
```

```
@end copying
```

3. 为阅读打印手册的读者编写标题页文本：

```
@titlepage
@title GNU Make
@subtitle A Program for Directing Recompilation
@subtitle Edition @value{EDITION}, ...
@subtitle @value{UPDATE-MONTH}
@page
@insertcopying
...
@end titlepage
```

(在打印封面上，只列出‘月’和‘年’的日期看起来比同时列出‘日’、‘月’和‘年’的日期要简洁。)

4. 为顶部节点(Top node)编写文本，供阅读 Info 文件的人使用：

```
@ifnottex
@node Top
@top Make
```

```
This is Edition @value{EDITION},
last updated @value{UPDATED},
of @cite{The GNU Make Manual},
for @code{make}, version @value{VERSION}.
@end ifnottex
```

在您格式化手册后，@value构造已被展开，因此输出包含类似以下文本：

```
This is Edition 0.35 Beta, last updated 14 August 1992,
of `The GNU Make Manual', for `make', Version 3.63 Beta.
```

在更新手册时，您只需更改标志的值；无需编辑这三个部分。

15.6 对 Texinfo 命令进行测试：@ifcommanddefined, @ifcommandnotdefined

有时，你可能想在你的手册中安排一些测试，以检查某个 Texinfo 命令是否可用；（假设）如果不可用，则执行某种格式化回退。为此，可以使用条件语句@ifcommanddefined和@ifcommandnotdefined。

示例：

```
@ifcommanddefined node
很好，@samp{@@node}命令已定义。
@end ifcommanddefined
```

将输出预期的：‘很好，‘@node’命令已定义。’。

此测试把文档通过@macro、@alias、@definfoenclose和@def(code)index（请参阅第 16 章[Defining New Texinfo Commands], 130 页）定义的所有新命令判为真。警告：T_EX的实现，除了报告所有 Texinfo 命令外还会报告内部 T_EX命令为“已定义”；无论如何，texi2any实现在这方面是可靠的。

你可以查看 Texinfo 源代码分发中的NEWS文件，该文件可通过 Texinfo 主页（<https://www.gnu.org/software/texinfo>）链接访问，以了解特定命令的添加时间。

这些命令检查条件本身是在 2013 年发布的 Texinfo 5.0 中添加的，此时距离 Texinfo 的诞生已有数十年。为了测试它们本身是否可用，可以测试预定义的标志txicommandconditionals，如下所示：

```
@ifset txicommandconditionals
@ifcommandnotdefined foobarnode
（很好，@samp{@@foobarnode}命令已定义。）
@end ifcommandnotdefined
@end ifset
```

由于 Texinfo 存在之初就有标志功能（见上一节），因此假设它们可用是没有问题的。

我们建议尽可能避免这些测试---通常情况下确实如此。对于许多软件包而言，所有开发人员都安装特定版本（或更新版本）的 Texinfo 是合理的，因此无需担心旧版本。（任何人都可以轻松下载并安装 Texinfo 源代码；它没有任何有问题的依赖项。）

最终用户一般不会遇到 Texinfo 版本的问题。对于正确分发的软件包，用户无需仅仅为了构建和安装软件包而（格式化）处理 Texinfo 手册；他们可以使用预格式化的 Info（或其他）输出文件。这通常是可取的，以避免软件包之间不必要的依赖关系（请参阅节“Releases”在 GNU Coding Standards）。

15.7 条件语句嵌套

Texinfo 要求，对于失败的分支条件，被忽略的文本必须与该失败的分支条件正确嵌套。以下是一个示例：

```
@ifset somevar
```

```
@ifset anothervar
somevar 及 anothervar 都已设置。
@end ifset
@ifclear anothervar
Somevar 已设置，anothervar 未设置。
@end ifclear
@end ifset
```

如果‘somevar’未设置，则跳过整个文本块。

为了让处理器能够可靠地确定哪些命令适合嵌套，所有条件命令必须单独占一行，前后不得有其他文本（甚至空格）。

16 定义新的 Texinfo 命令

Texinfo 提供了多种定义新命令的方法（在任何情况下，都不建议尝试重新定义现有命令）：

- Texinfo 中的 *macro* (宏) 命令允许您将新的 Texinfo 命令定义为任何文本序列和(或)现有命令（包括其他宏）。宏可以具有任意数量的形参数(*parameters*) ---即您每次调用宏时必须提供与之对应的文本（实参数）。

顺便说一句，这些宏与 `@defmac` 命令无关，`@defmac` 是命令用于在手册的主题区域中记录一个宏(请参阅 13.1 节[Def Cmd Template], 105 页)。

- ‘`@linemacro`’允许您定义一个宏，该宏的参数可以扩展到行尾，并且参数之间用空格分隔。
- ‘`@alias`’为现有命令定义了一个新名称。
- ‘`@definfoenclose`’允许您为某些非打印输出格式定义具有自定义输出的新命令。此命令已弃用。

最一般的情况（不仅限于定义新命令），可以调用任何外部宏处理器，并让 Texinfo 识别所谓的 `#line` 指令以进行错误报告。

如果您想进行简单的文本替换，使用 `@set` 和 `@value` 是最简单的方法(请参阅 15.5 节[`@set @clear @value`], 124 页)。

16.1 定义宏

您可以使用 Texinfo 的 `@macro` 命令来定义一个宏，如下所示：

```
@macro 宏名称(macroname){形参数 1(param1), 形参数 2(param2), ...}
正文 ... \参数 1(param1)\ ...
@end macro
```

参数 形参数 1(*param1*)、形参数 2(*param2*)、... 对应于随后在文档中使用宏时提供的参数（将在下一节中介绍）。

原则上，*macroname* 应由字母数字和（除作为第一个字符外）‘-’组成。‘_’字符被排除在外，以便在 `@math` 中调用宏时后面不需要空格(请参阅 11.7 节[Inserting Math], 93 页)。然而，为了使宏与 TeX 保持一致，*macroname* 必须全部由字母组成：不包含数字、连字符或其他特殊字符；因此，我们建议仅使用字母。Texinfo 命令不应被重新定义为宏。

如果一个宏不需要参数，你可以用一个空列表来定义它(‘`@macro foo {}`’)，或者根本不用花括号(‘`@macro foo`’)。

宏的定义或宏 *body* (体) 可以包含大多数 Texinfo 命令，包括宏调用；然而，由于 `@macro` 的设计限制，在 TeX 中，定义另一个宏的宏定义不起作用。

在宏体中，如上例中的 ‘`\param1\`’ 所示，被反斜杠包围的参数名¹实例将被宏调用中的相应参数²替换。在宏体中，参数名可以多次使用，包括‘零’次。

要在宏展开中得到单个‘\’，请在宏体中使用‘\\’。在宏体中的任何其他地方使用‘\’都会产生警告。

在 `@macro` 行的后面和 `@end macro` 行的前面的换行符将被忽略，即不会包含在宏体中。所有其他空白符都将按照 Texinfo 的常规规则进行处理。

¹ 译者注：概念上等同于 C 语言的形参数。

² 译者注：概念上等同于 C 语言的实参数。

要允许宏递归使用，即作为调用自身的参数，您必须使用‘`@rmacro`’来定义它，如下所示：

```
@rmacro rmac {arg}
a\arg\b
@end rmacro
...
@rmac{1@rmac{text}2}
```

这将生成输出‘`a1atextb2b`’。如果使用‘`@macro`’而不是‘`@rmacro`’，则会给出错误消息。

您可以使用`@unmacro foo`来取消定义宏`foo`。重复取消一个宏的定义并不会出错。例如：

```
@unmacro foo
```

16.2 调用宏

在定义了宏（参见上一节）之后，您可以在文档中像这样调用（使用）它：

```
@macroname {实参数 1(arg1), 实参数(arg2), ...}
```

结果将大致类似于你在该位置输入了`macroname` 的宏体。例如：

```
@macro foo {p, q}
合起来： \p\ & \q\
@end macro
@foo{a, b}
```

产生：

合起来： a & b.

因此，实参数(arguments)和形参数(parameters)都用逗号分隔，并用大括号括起来；逗号后的任何空格（而非逗号前的）都会被忽略。与其他 Texinfo 命令一致，即使在宏没有参数的情况下，调用时也需要使用大括号。例如：

```
@macro argless {}
这里没有参数。
@end macro
@argless{}
```

产生：

这里没有参数。

传递包含逗号的宏实际参数时需要小心，因为逗号也用于分隔参数。要在实参数中包含逗号字符，最可靠的方法是使用`@comma{}`命令。另一种方法是用‘`@asis{...}`’包围参数。对于`texi2any`，您还可以像‘\,’那样在前面加上一个反斜杠字符(即：转义)，但这不适用于`TEX`。

并不总是需要担心逗号。为了便于使用宏，实现了两条关于自动转义 的规则：

1. 如果一个宏只接受一个参数，则默认情况下，调用该宏时所有逗号都会被转义。例如：

```
@macro TRYME{text}
@strong{TRYME: \text\}
@end macro

@TRYME{A nice feature, though it can be dangerous.}
```

产生：

TRYME: A nice feature, though it can be dangerous.

确实如此。具体来说，对于传递给单参数宏的参数数量并无限制，因此在调用它们时需格外小心。

2. 如果宏调用包含另一个命令（包括递归调用自身），则嵌套命令调用中的任何逗号默认被转义。例如，在

```
@say{@strong{Yes, I do}, person one}
```

‘Yes’后的逗号被隐式地自动转义了。这是另一个使用递归宏的示例：

```
@macro cat{a,b}
  \a\\b\
@end macro

@cat{@cat{foo, bar}, baz}
```

将生成字符串‘foobabaz’。

3. 否则，应如上文所示，明确对逗号进行转义，以便将其视为参数的一部分。

在宏实参数中，反斜杠本身可以用另一个反斜杠来转义。例如：

```
@macname {\bleh}
```

会将参数‘\bleh’传递给macname。

texi2any也能识别‘{’和‘\’序列来表示花括号，但 T_EX中的实现并不识别这些序列。然而，这种情况很少需要，因为它们仅在宏实参数包含不匹配的花括号时才需要。

如果一个宏被定义为只接受一个实参数，那么在调用时可以不用任何花括号，将宏名后的整行内容作为参数。例如：

```
@macro bar {p}
  Twice: \p\ & \p\
@end macro
@bar aah
```

产生：

```
Twice: aah & aah.
```

在这些实参数中，是没有转义字符的概念的，因此每个‘\’都代表其本身。

如果一个宏被定义为接受多个参数，但在调用时只提供了一个（用花括号括起来），则剩余的参数将被设置为空字符串，且不会报错。例如：

```
@macro addtwo {p, q}
  Both: \p\\q\
@end macro
@addtwo{a}
```

直接产生：

```
Both: a.
```

16.3 宏的详细信息与注意事项

根据设计，宏展开在以下情况下不应发生：

- @macro和@unmacro行；
- @if...代码行，包括@ifset及其类似代码；
- @set、@clear、@value；

- `@clickstyle`行;
- `@end`行, 当块命令中没有宏展开时。

TeX 在这些情况下可能会进行一些展开, 从而可能产生错误。真是不幸!

此外, 相当多的宏相关构造会导致 TeX 出现问题; 下面列出了一些注意事项。因此, 如果在生成手册打印版时遇到与宏相关的问题, 您可以通过调用带有‘-E’选项的 `texi2dvi` (请参阅 [Format with texi2dvi], 142 页) 来尝试使用 `texi2any` 展开宏。或者, 更可靠的做法是, 完全避开 Texinfo 宏, 转而使用专为宏处理设计的语言, 如: M4 (请参阅 16.7 节 [External Macro Processors], 137 页)。

- 如前所述, 宏名必须全部由字母组成。
- 不建议将任何 TeX 原始命令、普通命令或 Texinfo 命令名重新定义为宏。遗憾的是, 这是一组庞大且开放式的名称集合, 由此可能产生的错误是不可预测的。
- 接受多个实参数的宏的实参数不能跨行。
- 包含必须单独占一行的命令 (如条件语句) 的宏, 不能在一行的中间被调用。同样, 包含面向行的命令或文本 (如 `@example` 环境) 的宏, 在 TeX 中可能表现异常。
- 在索引条目文本中的宏展开中, Texinfo 命令可能最终被排版为文字文本 (包括“@”符号), 而不是按照 Texinfo 命令的预期含义进行解释。
- 行首的空白会被忽略。
- 在重音命令的参数中, 无法可靠地使用宏 (请参阅 11.4 节 [Inserting Accents], 91 页)。
- 宏实参数中的逗号无法使用反斜杠转义; 必须使用 `@comma{}`。
- 在宏体结尾处使用‘@c’可能会导致宏调用后的文本在 `texi2any` 中被视为注释而忽略。但在使用 TeX 处理时则不会出现这种情况。过去, 人们常常这样做来“注释掉”宏体末尾不需要的换行符, 但现在这已不再必要, 因为‘@end macro’之前的最后一个换行符无论如何都不会包含在宏体中。
- 一般来说, 你不能随意用宏 (或 `@value`) 调用替换 Texinfo 命令参数, 即使文本内容相同。Texinfo 不是 M4 (甚至不是普通的 TeX)。它可能适用于某些命令, 但不适用于其他命令。最好不要这样做。例如, 这样会失败:

```
@macro offmacro
off
@end macro
@headings @offmacro
```

这看起来与 `@headings off` 等效, 但由于 TeX 技术原因, 它会报出一个神秘的错误消息: ‘Paragraph ended before @headings was complete (段落在 @headings 完成之前结束)’。

- 在数学 (math) 环境上下文中展开宏可能会导致 TeX 输出中的间距不佳。例如:

```
@iftex
@macro atan
\\mathop{\\rm atan}
@end macro
@end iftex
@math{@atan}{(x)}
```

TeX 将‘@atan’后的括号视为子公式, 导致左括号前出现额外空格。您可以通过定义一个原始的 TeX 宏来解决此问题:

```
@tex
```

```
\gdef\atan#1{\mathop{\rm atan}}
@end tex
```

此定义中的‘#1’会吸收大括号，从而使间距符合`\mathop`对象的预期。

- 如果你在宏中使用条件语句时遇到问题，一种替代方案是在条件块内使用单独的宏定义。例如，与其使用

```
@macro Mac
@iftex
TeX输出的文本
@end iftex
@ifnottex
非 TeX输出的文本
@end ifnottex
@end macro
```

你可以采取以下替代措施：

```
@iftex
@macro Mac
TeX输出的文本
@end macro
@end iftex

@ifnottex
@macro Mac
非 TeX输出的文本
@end macro
@end ifnottex
```

- 宏不能以自然方式定义宏。要实现这一点，必须使用条件语句和原始的 TeX。例如：

```
@ifnottex
@macro ctor {name, arg}
@macro \name\
某种涉及\arg\的事物
@end macro
@end macro
@end ifnottex
@tex
\gdef\ctor#1{\ctorx#1,}
\gdef\ctorx#1,#2,{\def#1{某种涉及\arg\的事物}}
@end tex
```

以下限制是出于设计考虑：

- 如果你传入带有 Texinfo 命令`@`，(用于生成软音符(cedilla)，请参阅11.4 节[Inserting Accents], 91 页)的参数，你必须使用`@value`或其他变通方法。否则，逗号可能会被视为分隔参数。使用`texi2any`时，逗号可以通过反斜杠进行转义。使用 TeX时，需要使用另一种变通方法，因此我们建议使用此变通方法。

示例：

```
@macro mactwo{argfirst, argsecond}
```

```

\argfirst\+\argsecond\
@end macro
@set fc Fran@,cois
@mactwo{@value{fc},}

```

产生:

```
François+.
```

- @verbatim和宏不能混用；例如，你不能在宏内部开始一个 verbatim 块，而在宏外部结束它（请参阅7.5 节[@verbatim], 61 页）。就此而言，在宏内部开始任何环境，而在宏外部结束它，可能行得通，也可能行不通。
- 完全定义宏的宏是可以的，但不能有嵌套不完整的宏定义。也就是说，@macro和@end macro (@rmacro同理) 必须正确配对。例如，你不能在一个宏内部开始一个宏定义，然后在宏外部结束那个嵌套的定义。

在 Texinfo 5.0 之前的makeinfo实现中，@macro定义展开后的换行符，不能作为以行界定的 @ 命令（如@chapter、@center等）的换行符。现在情况已不再如此。例如：

```

@macro twolines{}
aaa
bbb
@end macro
@center @twolines{}

```

在当前texi2any中，这等效于：

```

@center aaa
bbb

```

仅将‘aaa’作为@center的参数。而在之前的实现中，它将被解析为：

```
@center aaa bbb
```

16.4 ‘@alias 新命令 = 现有命令’

‘@alias’命令用于定义一个新命令，使其与现有命令完全相同。这对于定义额外的标记名称非常有用，即使输出结果可能相同，也能在输入中保留额外的语义信息。

将‘@alias’命令单独写一行，后面紧跟新命令名、一个等号和现有命令名。等号前后可随意添加空格，如果存在，将被忽略。因此：

```
@alias 新命令 = 现有命令
```

例如，如果您的文档中既有书籍引用，又有其他媒体(如电影)的引用，您可能希望定义一个宏@moviecite{}，该宏的功能与普通的@cite{}相同，但也能传达额外的语义信息。您可以按以下步骤进行操作：

```
@alias moviecite = cite
```

由于参数解析的多样性，宏并不总是与别名具有相同的效果。此外，别名比宏更易于定义。因此，该命令并非冗余。

遗憾的是，Texinfo 的环境命令不支持别名；例如，@alias lang=example 是一个错误。

别名不能直接或间接递归。

不建议将任何 T_EX 原语、普通 T_EX 或 Texinfo 命令名重新定义为其它命令的别名。遗憾的是，这些名称数量众多，T_EX 可能产生的错误难以预测。

别名标识符应与宏名一样，要求由字母数字和（除第一个字符外）‘-’组成。但是，在 T_EX 中，应仅使用字母。因此，我们建议仅使用字母。

16.5 行宏

您可以使用‘@linemacro’命令来定义行宏。行宏定义的语法与@macro的语法相似。

然而，使用以这种方式定义的宏的语法是不同的。调用宏时会使用该行的剩余部分作为参数，参数之间用空格分隔。行宏功能主要是为了让你定义一些命令，这些命令的操作方式类似于内置的定义命令（请参阅第 13 章[Definition Commands], 105 页）。

示例：

```
@linemacro defbuiltin {name, args}
@defline {内建} \name\ \args\
@end linemacro
```

上面的宏可以这样使用：

```
@defblock
@defbuiltin foo (bar)
说明...
@end defblock
```

这产生下面的结果：

```
foo (bar) [内建]
说明...
```

这里的第一个实参数是‘foo’，当‘\name\’出现在宏体中时，会被替换为该参数。同样地，‘(bar)’会替换‘\args\’。

(调用时)您可以通过用大括号包围宏（实）参数的方式，来在参数中包含空格。在替换之前，任何包围参数的大括号都会被移除。任何空参数（包括最后一个）都必须以‘{ }’的形式给出。此外，非初始空格可能出现在参数列表中的最后一个参数中，则无需包围大括号（如下例所示）。

如果行宏(调用时)的参数行以‘@’字符结尾，则此字符与随后的换行符一起包含在宏参数中，并且输入文件中的下一行也用于获取宏的参数。请注意，‘@<换行符>’序列的任何替换都必须在有效上下文中，例如：定义命令的延续行(请参阅13.2 节[Def Cmd Continuation Lines], 106 页)。

另一个示例：

```
@linemacro deffunc {type, name, args}
@findex \name\
@deftypeline {函数} {\type\} \name\ \args\
@end linemacro

@defblock
@deffunc {long int} FuncExample (int @var{one}, int @var{two}, @
int @var{three}, int @var{four}, int @var{five})
说明...
@end defblock
```

注意，在宏体中，如果type形参数包含空格，则需要在‘\type\’前后加上大括号。以下示例将生成以下输出，并将FuncExample加为函数索引条目。

```
long int FuncExample (int one, int two, int three, int four, [函数]
int five)
说明...
```

行宏在展开时会生成多行输出。与使用`@macro`定义的宏相比，使用`@linemacro`定义的宏在 Texinfo 处理器（`texi2any`和带有`texinfo.tex`的 $\text{\TeX}$ ）之间可能具有更好的兼容性。

16.6 @definfoenclose: 自定义高亮显示

`@definfoenclose`命令用于为在线输出格式定义高亮显示命令。使用`@definfoenclose`定义的命令通过在文本前后加上字符串来标记文本。

实际上，这个命令几乎没什么用处，我们也不建议您使用它。在 Texinfo 的未来版本中，可能会移除对`@definfoenclose`的支持。

在行首编写一个`@definfoenclose`命令，后跟三个以逗号分隔的参数。`@definfoenclose`的第一个参数是 `@` 命令的名称（不含`@`）；第二个参数是开始分隔符字符串；第三个参数是结束分隔符字符串。后两个参数用于包围输出中的高亮文本。

分隔符字符串可以包含空格。起始分隔符和结束分隔符都不是必需的。如果你不想要起始分隔符但想要结束分隔符，那么在命令名称后面必须连续使用两个逗号；否则，你想要的结束分隔符字符串自然会被（错误地）解释为起始分隔符字符串。

以这种方式定义的封闭命令接受带一个在花括号中的参数，因为它适用于新的标记命令（请参阅第 6 章[Marking Text], 48 页）。

例如，你可以写：

```
@definfoenclose phoo,/,,\
```

在 Texinfo 文件的开头附近，将`@phoo`定义为 Info 和 HTML 命令，该命令会在`@phoo`的参数前后分别插入`‘/’`和`‘\’`。然后，无论你想在 Info 和 HTML 哪个位置突出显示`‘//bar\’`，都可以编写`@phoo{bar}`。

对于 $\text{\TeX}$ 格式化，你可以写

```
@iftex
@alias phoo = i
@end iftex
```

将`@phoo`定义为一条命令，该命令使 $\text{\TeX}$ 将`@phoo`的参数以斜体形式排版。

每个定义都适用于其特定的格式化程序：一个用于 $\text{\TeX}$ ，另一个用于在线格式。 $\text{\TeX}$ 的定义需要放在`@iftex`中。除非您想为不同的在线输出格式使用不同的定义，否则`@definfoenclose`命令不需要放在`@ifinfo`中。`@definfoenclose`定义的命令在 DocBook 和 $\text{\LaTeX}$ 输出中没有效果，其参数会按原样输出。对于这些格式，也可以使用`@alias`。

`@definfoenclose`定义不能直接或间接递归。

16.7 外部宏处理器：行号指令

Texinfo 宏（及其它文本替换功能）在简单情况下运行良好。然而，如果您的文档需要异常复杂的处理，它们的脆弱性和局限性可能会成为问题。在这种情况下，您可能希望完全使用不同的宏处理器，如 M4(请参阅M4)或 CPP(请参阅*The C Preprocessor*)。

除了一种特殊情况外，Texinfo 无需知道其输入是“原始”源代码还是从其他源文件预处理过的。因此，你可以设置构建系统来调用任何你喜欢的程序来处理宏扩展或其他预处理需求。Texinfo 不提供对任何特定预处理器的内置支持，因为似乎没有一种程序能满足所有文档的需求。

唯一的例外是错误信息中的行号。在这种情况下，行号应指代原始源文件，无论它是什么。对此有一个众所周知的机制：所谓的`‘#line’`指令。Texinfo 支持这一指令。

16.7.1 ‘#line’指令

像这样的输入行：

```
#line 100 "foo.ptexi"
```

这表明下一行是文件foo.ptexi的第 100 行，因此错误消息应该指的就是这一行。M4(请参阅节 “Preprocessor features” 在 GNU M4)和 CPP(请参阅节 “Line Control” 在 *The C Preprocessor*，以及节 “Preprocessor Output” 在 *The C Preprocessor*)都能生成这样的行。

默认情况下，texi2any程序能够识别这些行，但在@verbatim代码块内则无法识别(请参阅7.5 节[@verbatim], 61 页)。虽然通常没有必要，但可以通过CPP_LINE_DIRECTIVES(请参阅19.4.3 节[Global Customization Variables], 157 页)完全关闭对这些行的识别功能。

对于少数需要记录‘#line’指令的程序（如 M4、CPP、Texinfo），这些程序中的示例原本可能符合上述模式，但可以使用命令@hashchar{} (请参阅11.1.5 节[Inserting a Hashsign], 87 页)来处理。在本手册的源代码中，上述示例行如下所示：

```
@hashchar{}line 100 "foo.ptexi"
```

@hashchar命令于 2013 年被添加到 Texinfo 中。如果你不想依赖它，也可以使用@set和@value来插入字面量‘#’：

```
@set hash #
@value{hash}line 1 "example.c"
```

或者，如果合适的话，可以使用@verbatim环境来代替@example。如上所述，在 verbatim 块内禁用#line的识别。

16.7.2 ‘#line’与 T_EX

如前所述，texi2any能够识别上一节中描述的‘#line’指令。然而，texinfo.tex却不能识别，也无法识别。因此，如果 T_EX遇到这样的行，就会错误地将其原样排版。解决方案是在运行 T_EX之前，使用texi2any的宏展开选项。具体方法有以下三种：

- 如果你运行texi2dvi或其变体（请参阅[Format with texi2dvi], 142 页），你可以传递-E 选项，texi2dvi将首先运行texi2any来展开宏并消除‘#line’。
- 如果你运行texi2any，你可以指定--no-ifinfo --iftex -E somefile.out，然后在另一个命令中将somefile.out传递给texi2dvi。
- 或者，你可以运行texi2any --dvi --Xopt -E。（或者使用--pdf 代替--dvi。）texi2any随后将调用texi2dvi -E。

关于与 T_EX一起使用的最后一点注意事项：由于#line指令未被识别，因此@errmsg{}命令（请参阅15.1 节[Conditional Commands], 121 页）或 T_EX本身发出的行号，是 T_EX正在读取的派生文件中的（错误）行号，而不是预处理器指定的行号。

16.7.3 ‘#line’ 语法详情

‘#line’指令的语法细节：‘#’字符前后可以有空格，单词‘line’是可选的，文件名后面可以跟一个用空格分隔的整数列表（这些整数在某些情况下是 CPP 输出的所谓“标志”）。对于那些喜欢了解详细信息的人来说，匹配的(Perl)正则表达式实际上是：

```
/^\s*#\s*(line)? (\d+)(( "([^\"]+)" )(\s+\d+)*)?\s*$/
```

据我们所知，尾随整数标志仅与文件名一起出现，因此这在正则表达式中得到了体现。例如，以下是一个语法上有效的‘#line’指令，表示/usr/include/stdio.h的第 1 行：

```
# 1 "/usr/include/stdio.h" 2 3 4
```

遗憾的是，引用的文件名（`"..."`）必须是可选的，因为 M4（尤其）常常会在单个文件中生成`#line`指令。由于`line`也是可选的，结果可能会导致行匹配与预期不符，例如：

```
# 1
```

可能的解决方案已在上方描述（请参阅请参阅 16.7.1 节[`#line` Directive], 138 页）。

17 包含文件

当 Texinfo 处理器在 Texinfo 文件中遇到`@include`命令时，它会处理`@include`所指定的文件的内容，并将其合并到正在创建的输出文件中。因此，包含文件使您能够让一个大型文档成为一系列方便使用的小的部分的集合。

17.1 如何使用包含文件

要在 Texinfo 文件中包含另一个文件，请在一行的开头编写`@include`命令，并在同一行的后面紧跟要包含的文件名称。例如：

```
@include buffers.texi
```

`@value{ var }`引用会在`@include`行上展开。除此之外，允许的`@`命令是`@@`、`@{`、`@}`以及`@atchar{}`类似的相关`@`命令。

一个被包含文件应该只是一段文本，你期望它能按原样被包含到总体或外层 Texinfo 文件中；它不应该包含 Texinfo 文件的标准开头和结尾部分。特别是，你不应该以`\input texinfo`这样的行开始一个包含文件；如果你这样做了，那么这段文本就会直接被插入到输出文件中。同样，你不应该以`@bye`命令结束一个被包含文件；`@bye`之后的任何内容都不会被格式化。

在很久以前，你需要在被包含文件的开头写一行`@setfilename`，但现在不需要了。现在，你是否写这样一行并不重要。如果被包含文件中存在`@setfilename`行，它将被忽略。

GNU Emacs 的 Texinfo 模式提供了`texinfo-multiple-files-update`功能，用于更新包含多个文件的节点指针和主控菜单。请参阅节 D.5.3 [Update Multiple Files], 244 页。

17.2 带有`@include`的示例文件

以下是一个外层 Texinfo 文件示例，其中包含`@include`文件：

```
\input texinfo @c -*-texinfo-*
```

```
@settitle 使用@include的示例
```

```
@node Top
```

```
@top 使用@include的示例
```

```
@include foo.texi
```

```
@include bar.texi
```

```
@include concept-index.texi
```

```
@bye
```

`foo.texi`是一个被包含的文件，可能看来是这样：

```
@node First
```

```
@chapter 第一章
```

```
第一章的内容 ...
```

`concept-index.texi`的全部内容可能就这么简单：

`@node Concept Index`
`@unnumbered` 概念索引

`@printindex cp`

《GNU Emacs Lisp 参考手册》(*The GNU Emacs Lisp Reference Manual*) 的外层 Texinfo 源文件名为 `elisp.texi`。这个外层文件已经包含了一个有 417 个条目的主控菜单及 41 个 `@include` 文件。

17.3 `@verbatiminclude file`：字面方式包含一个文件

您可以使用 `@verbatiminclude` 命令在文档中包含一个文件的字面内容：

`@verbatiminclude filename`

`filename` 的内容会在一个字面输出环境（请参阅 7.5 节 `@verbatim`，61 页）中打印。通常，被包含文件的内容会原封不动地打印，保留所有特殊字符和空白。不会添加缩进；如果你想要缩进，请将 `@verbatiminclude` 包含在 `@example` 中（请参阅 7.4 节 `@example`，60 页）。

`@value{var}` 引用会在 `@verbatiminclude` 行上展开。以便于在发布时包含其他目录中的文件，例如：

`@verbatiminclude @value{top_srcdir}/NEWS`

（当然，您仍然需要先定义 `top_srcdir`。）

除此之外，允许的 `@` 命令有 `@@`、`@{`、`@}` 以及相关的 `@` 命令（如 `@atchar{}`）。

如需了解以小字体打印文件内容的方法，请参阅“`@verbatim`”一节末尾的内容。

18 使用 T_EX 进行格式化和打印

运行 `texi2dvi` 或 `texi2pdf` 命令是创建可打印输出的最简单方法。这些命令是作为 Texinfo 软件包的一部分安装的。

更具体地说，排版和打印 Texinfo 文件是一个多步骤的过程。首先，您需要使用 T_EX 程序创建一个文件（DVI 或 PDF 文件）用于打印，然后打印该文件。可选地，您也可以在首次运行 T_EX 后，使用 `texindex` 命令创建索引；之后，您必须再次运行 T_EX。`texi2dvi` 会负责处理所有这些步骤，根据需要运行 T_EX 和 `texindex`（请参阅 [Format with `texi2dvi`], 142 页）。

当你使用 shell 命令时，你可以直接在操作系统 shell 中操作，也可以在 GNU Emacs（或其他计算环境）中的 shell 中操作。你可以在 GNU Emacs 中的 shell 中发出格式化和打印命令，就像任何其他 shell 命令一样。要在 Emacs 中创建 shell，请键入 `M-x shell`（请参阅节“Shell”在 *The GNU Emacs Manual*）。如果你使用的是 GNU Emacs，也可以使用 Texinfo 模式提供的命令，而不是 shell 命令。请参阅节 D.7 [Printing with Emacs], 247 页。

有关打印输出的具体细节（如纸张大小），请参阅附录 E [Global Document Commands], 258 页。

18.1 使用 T_EX

名为 T_EX 的排版程序用于格式化 Texinfo 文档，以便进行可打印输出。T_EX 是一款功能强大的排版程序，若使用得当，其排版效果将出类拔萃。它并未包含在 Texinfo 软件包中，因为其本身就是一个庞大的软件套件。

T_EX 是自由软件基金会（FSF）为其文档所使用的文档格式化工具。它是获取 Texinfo 手册打印输出（如 PDF 和 PostScript）的最简单方式。T_EX 可自由再分发，您可以通过互联网或实体介质获取。请访问 <http://tug.org/texlive>。

18.2 使用 `texi2dvi` 或 `texi2pdf` 格式化

`texi2dvi` 程序负责处理从 Texinfo 文档生成 T_EX DVI 文件的所有步骤。同样地，`texi2pdf` 程序则负责生成 PDF¹ 文件。

要对输入文件 `foo.texi` 运行 `texi2dvi` 或 `texi2pdf`，请执行以下操作（其中‘prompt\$’是您的 shell 提示符）：

```
prompt$ texi2dvi foo.texi
prompt$ texi2pdf foo.texi
```

如本例所示，为 `texi2dvi` 和 `texi2pdf` 指定的文件名必须包含所有扩展名，例如‘.texi’。

要查看所有选项列表，请运行‘`texi2dvi --help`’。下文将讨论部分选项。

使用 `--pdf` 选项时，`texi2dvi` 会通过运行 `pdftex` 而不是 `tex`，来生成 PDF 输出，而不是 DVI。或者，命令 `texi2pdf` 是 `texi2dvi --pdf` 的简化写法。为了方便 AUC-T_EX（请参阅 AUC-T_EX）使用，还提供了命令 `pdftexi2dvi`，因为它更倾向于仅在 DVI 生成工具前加上‘pdf’来获得 PDF 生成工具。

使用 `--dvipdf` 选项时，`texi2dvi` 会先运行 T_EX，然后运行一个 DVI 到 PDF 的转换程序来生成 PDF 输出：如果设置了 `DVIPDF` 环境变量，则使用该值，否则将使

¹ PDF 代表‘便携式文档格式’。它由 Adobe Systems 基于其 PostScript 语言发明，用于文档交换。

用 dvipdfmx、dvipdfm、dvipdf、dvi2pdf 和 dvitopdf 中第一个存在的程序。这种方法通常比 pdf_{tex} 更支持 CJK 排版。²

使用 `--ps` 选项时，`texi2dvi` 会先运行 `tex`，然后再运行 `dvips`，从而生成 PostScript 文件而非 DVI 文件（请参阅 *Dvips*）。（如果设置了 DVIPS 环境变量，则使用该变量的值。）

`texi2dvi` 也可用于处理 L^AT_EX 文件。通常，`texi2dvi` 能够根据输入文件的内容和文件扩展名来猜测其语言；然而，如果猜测错误，您可以使用 `--language=lang` 命令行选项来明确指定输入语言，其中 *lang* 可以是 ‘`latex`’ 或 ‘`texinfo`’。

`texi2dvi` 的一个有用选项是 ‘`--command=cmd`’。该选项会在运行 T_EX 之前，在输入文件的临时副本中，将 *cmd* 单独插入到文件开头的行上。这样，您就可以在不实际更改文档源代码的情况下，指定不同的打印格式，如 `@smallbook`（请参阅节 E.6 [`@smallbook`], 263 页）、`@fourpaper`（请参阅节 E.7 [A4 Paper], 264 页）或 `@pagesizes`（请参阅节 E.8 [`@pagesizes`], 264 页）。（您也可以使用 `texinfo.cnf` 在全站范围内进行此操作；请参阅 18.4 节 [Preparing for T_EX], 146 页）。

选项 `-E`（等效于 `-e` 和 `--expand`）使用 `texi2any` 而不是 T_EX 实现来进行 Texinfo 宏扩展（请参阅 16.3 节 [Macro Details], 132 页）。每种实现都有其各自的局限性和优势。

`texi2dvi` 通过 `--build=mode` 选项指定 T_EX 编译的位置，从而确定如何处理辅助文件。构建模式也可以通过环境变量 `TEXI2DVI_BUILD_MODE` 设置。*mode* 的有效值包括：

- ‘`local`’ 在当前目录下编译，保留所有辅助文件。这是传统的 TeX 使用方式。
- ‘`tidy`’ 在本地 `*.t2d` 目录中进行编译，辅助文件会保留在该目录中。输出文件会被复制回原始文件所在目录。

使用 ‘`tidy`’ 模式会带来几个优势：

- 当前目录中没有过多的临时文件杂乱无章地堆放。
- 使用 `--build-dir=dir` 选项可以进一步减少杂乱程度：所有的 `*.t2d` 文件都存储在那里。
- 杂乱项可以减少到零。使用 `--build-dir=/tmp/\$USER.t2d` 或 `--build-dir=\$HOME/.t2d`。
- 每次成功运行 T_EX 后，输出文件都会更新，以便同时可视化输出。在 ‘`local`’ 构建中，查看器会在整个 T_EX 运行过程中停止。
- 如果编译失败，输出文件的先前状态将保持不变。
- PDF 和 DVI 的编译文件保存在单独的子目录中，从而避免了辅助文件不兼容的可能性。

另一方面，由于 ‘`tidy`’ 编译是在另一个目录中进行的，T_EX 偶尔会找不到某些文件（例如，在使用 `\graphicspath` 时）：在这种情况下，可以使用 `-I` 来指定要考虑的其他目录。

- ‘`clean`’ 与 ‘`tidy`’ 相同，但之后会删除辅助目录。因此，每次编译都需要完整执行一次。

如果 `etex` 可用，`texi2dvi` 将使用它，因为在某些情况下，它的运行速度更快，并且在调试 `texinfo.tex` 时能提供额外的跟踪信息。然而，这个 T_EX 的扩展版本并非必需，DVI 输出是相同的。

² 译者注：实际上，对中文 Texinfo 无效。

`texi2dvi` 通过使用 `-recorder` 选项或在运行 T_EX 产生的日志文件中扫描 `\openout`，尝试检测由 T_EX 输出的辅助文件。您可以通过设置 `TEXI2DVI_USE_RECORDER` 环境变量来控制 `texi2dvi` 如何执行此操作。有效值包括：

‘yes’ 使用 `-recorder` 选项，无检查。

‘no’ 在日志文件中扫描 `\openout`，不进行任何检查。

‘yesmaybe’ 检查是否支持 `-recorder` 选项，如果支持则使用该选项；如果不支持，则检查日志文件中是否支持 `\openout` 跟踪，如果支持则使用该功能，否则报错。

‘nomaybe’ 与 ‘yesmaybe’ 相同，但首先会检查日志文件中的 `\openout` 跟踪。

默认值为 ‘nomaybe’。此环境变量仅用于故障排除，未来可能会发生变化或消失。

18.3 使用 `tex`/`texindex` 进行格式化

您可以使用 shell 命令 `tex` 后跟 Texinfo 文件名，对 Texinfo 文件进行基本格式化。例如：

```
tex foo.texi
```

T_EX 会生成一个 DVI 文件 以及几个包含索引、交叉引用等信息在内的辅助文件。DVI 文件（即：设备无关 (*DeVice Independent*) 文件）几乎可以在任何设备上打印，但可能需要进行进一步的转换（见上一节）。

`tex` 格式化命令本身并不对索引进行排序；它只会生成一个未排序的索引数据输出文件。若要在运行 `tex` 命令后生成打印索引，您首先需要一个已排序的索引作为基础。`texindex` 命令可以对索引进行排序。（上一节中描述的 `texi2dvi` 会在需要时运行 `tex` 和 `texindex`。）

`tex` 输出未排序的索引文件，文件名遵循标准约定：将主输入文件的名称中的任何 ‘.texi’ 或类似扩展名替换为两个字母的索引名称。例如，对于输入文件 `foo.texi`，默认情况下，原始索引输出文件将为 `foo.cp`、`foo.vr`、`foo.fn`、`foo.tp`、`foo.pg` 和 `foo.ky`。这些正是 `texindex` 的参数。

通常，我们不会明确指定所有未排序的索引文件名，而是使用 ‘??’ 作为 shell 通配符，并以以下形式给出命令：

```
texindex foo.??
```

此命令将对所有未排序的索引文件运行 `texindex`，包括您使用 `@defindex` 或 `@defcodeindex` 自定义的任何两字母索引。即使存在扩展名为两个字母的文件（如 ‘foo.el’）并非索引文件，您也可以安全地运行 ‘`texindex foo.??`’。`texindex` 命令会报告这些文件，但在其他方面会忽略它们。

对于每个指定的文件，`texindex` 会生成一个排序后的索引文件，其名称是在输入文件名后附加 ‘s’ 构成的；例如，`foo.cps` 是由 `foo.cp` 生成的。`@printindex` 命令会查找具有该名称的文件（请参阅 10.5 节 [Printing Indices & Menus]，83 页）。T_EX 不会读取原始索引输出文件，`texindex` 也不会对其进行修改。

在您对索引进行排序后，需要对 Texinfo 文件重新运行 `tex`。这将重新生成输出文件，这次会包含最新的索引条目。

最后，您可能需要再次运行 `tex`，以确保交叉引用中的页码正确。

简而言之，这是一个五步过程。（也可以说是一个一步过程：运行 `texi2dvi`；见上一节。）

1. 对你的 Texinfo 文件运行 `tex` 命令。这将生成一个 DVI 文件（带有未定义的交叉引用且无索引），以及原始索引文件（带有两个字母的扩展名）。

2. 对原始索引文件运行`texindex`命令。这将创建相应的已排序索引文件（扩展名为三个字母）。
3. 再次对您的 Texinfo 文件运行`tex`。这将重新生成 DVI 文件，这次会包含索引和已定义的交叉引用，但交叉引用的页码会沿用上一次运行时的页码，通常是不正确的。
4. 使用`texindex`重新对索引进行排序。
5. 最后一次运行`tex`。这次，为交叉引用写上了正确的页码。

要生成 PDF，你可以运行`pdftex`程序，而不是普通的`tex`。也就是说，在上面的示例中，运行`pdftex foo.texi`而不是`tex foo.texi`。

18.3.1 格式化部分文档

有时，你可能希望在明知文档不完整的情况下打印它，或者只想打印文档中的某一章。在这种情况下，T_EX 通常创建的辅助文件以及关于未定义交叉引用的警告都是令人烦恼的。你可以使用`@novalidate`命令来避免这些问题，该命令必须在任何章节命令或交叉引用命令之前给出。

因此，你的文件开头大致如下：

```
\input texinfo
@novalidate
...
```

`@novalidate`同样会在`texi2any`中关闭验证功能，就像其`--no-validate`选项一样（请参阅 19.1 节[Invoking `texi2any`], 148 页）。

此外，您无需每次运行`tex`后都运行`texindex`。`tex`格式化命令只会使用之前运行`texindex`时生成的已排序索引文件。如果这些文件已过期，在创建或调试文档时通常也没关系。

18.3.2 `texindex`的详细信息

在 2015 年发布的 Texinfo 版本 6 中，`texindex`程序被完全重新实现。主要的功能差异在于，以左括号或右括号（‘{’和‘}’）开头的索引条目现在可以正常工作。例如，这些简单的索引条目得到了正确处理，包括索引中显示的“index initial”：

```
@cindex @{
@cindex @}
...
@printindex cp
```

虽然这并非功能性的问题，但读者可能想知道：新的`texindex`是一个以文学编程(Literate programming)模式写的程序 (https://en.wikipedia.org/wiki/Literate_programming)，它使用 Texinfo 进行文档编写，并使用（可移植的）`awk`进行代码编写。一个源文件，即本例中的`texindex/ti.twtjr`，可以生成可运行程序、可打印文档和在线文档。

该系统名为 TexiWeb Jr.，由 Arnold Robbins 创建，他也是新命令`texindex`的编写者。并非巧合的是，他还是`gawk` (GNU Awk，请参阅 *The GNU Awk User's Guide*) 的长期维护者。文件`texindex/Makefile.am`展示了该系统的使用示例。

18.4 准备使用 T_EX

T_EX 需要找到第一行中 `\input texinfo` 命令所读取的 `texinfo.tex` 文件³。`texinfo.tex` 文件告诉 T_EX 如何处理 `@` 命令；它包含在所有标准的 GNU 发行版中。最新发布的通用版本可以从常规的 GNU 服务器和镜像站点获取：

```
http://ftp.gnu.org/gnu/texinfo/texinfo.tex
http://ftpmirror.gnu.org/texinfo/texinfo.tex
```

最新开发版本可从 Texinfo 源代码仓库获取：

```
http://git.savannah.gnu.org/cgi/texinfo.git/plain/doc/texinfo.tex
```

`texinfo.tex` 本质上是一个独立文件，因此，如果你需要或想尝试使用系统附带的更新版本，只需下载该文件并将其放置在 T_EX 能够找到的任何位置即可。你可以用更新版本替换现有的 `texinfo.tex`（当然，为防万一，请保存原始文件）。

此外，如果尚未从其他发行版安装 `epsf.tex`，则应进行安装。更多详细信息见 `@image` 命令（请参阅 9.2 节 [Images], 76 页）说明的末尾。

若要使用非英文引号，您需要安装欧洲计算机现代字体（例如，`ecrm1000`）和（用于 PDF 输出）CM-Super 字体（请参阅 11.5 节 [Inserting Quotation Marks], 92 页）。

要使用 `@euro` 命令，你需要 `'feym*'` 字体（例如，`feymr10`）。请参阅 11.8.6 节 [`@euro`], 95 页。

在合理的 T_EX 安装中，上述所有文件都应默认安装。

（可选）您可以创建一个名为 `texinfo.cnf` 的文件用于站点配置。在处理 Texinfo 文件时，T_EX 会在其搜索路径中查找此文件，该路径包括当前目录和标准安装目录。您可以使用此文件来设置本地约定。例如，如果 `texinfo.cnf` 包含行 `'@afourpaper'`（请参阅节 E.7 [A4 Paper], 264 页），则所有 Texinfo 文档都将使用该页面大小进行处理。如果您没有什么需要在 `texinfo.cnf` 中设置，则无需创建此文件。

你可以设置 `TEXINPUTS` 环境变量，以使 T_EX 能够找到 `texinfo.cnf`。（这同样适用于 `texinfo.tex` 以及 T_EX 可能读取的任何其他文件）。例如，如果你使用的是与 Bourne shell 兼容的 shell（`sh`、`bash`、`ksh` 等），则你的 `.profile` 文件可以包含以下行：

```
TEXINPUTS=./home/me/mylib:
export TEXINPUTS
```

这些设置将使 T_EX 首先在当前目录（由 `'.'` 表示）中查找 `\input` 文件，然后在假设的用户 `'me'` 的主目录中的 `mylib` 子目录中查找，最后在系统目录中查找。（前导、尾随或重复的 `'.'` 表示在该点搜索系统目录。）

18.5 “hbox”溢出

T_EX 有时无法在正常边距内排版一行。这种情况最常发生在 T_EX 遇到它认为无法断句的长单词时，例如电子邮件网络地址或非常长的标识符。当这种情况发生时，T_EX 会输出一条类似以下的错误消息：

```
Overfull @hbox (20.76302pt too wide)
```

（在 T_EX 中，行位于“水平框 (horizontal boxes)”中，因此称为“hbox”。`@hbox` 是 T_EX 的一个原语，在 Texinfo 语言中不使用。）

³ 译者注：中文 Texinfo 文件用 `\input texinfo-zh` 命令，读取 `texinfo-zh.tex` 文件。

T_EX 还会提供 Texinfo 源文件中的行号以及出错行的文本，这些文本在所有 T_EX 认为需要断行的位置都有标记。如需了解更多关于排版错误的信息，请参阅节 D.10.3 [Debugging with T_EX], 254 页。

如果 Texinfo 文件出现的 hbox 溢出，你可以重写句子以避免溢出，或者也可以选择保留。文字稍微超出右边距通常不会造成影响，甚至可能不太明显。

如果你有大量的溢出情况和(或)不想改写，你可以强制 T_EX 大幅增加允许的字间距，这样（如果你幸运的话）就能避免许多糟糕的换行，如下所示：

```
@tex
\global\emergencystretch = .9\hsize
@end tex
```

（您应根据需要调整分数。）\emergencystretch 的这个巨大值不能是默认值，因为那样的话，排版输出的质量通常会明显降低；它的默认值是‘0.15\hsize’。 \hsize 是包含当前行宽的 T_EX 尺寸。

对于确实存在的 hbox 溢出，除非另有指示，否则 T_EX 会在包含溢出 hbox 的行旁边打印一个又大又丑的黑色矩形。这样，你在校对草稿时就能注意到出问题的位置。

为防止此类怪异内容破坏您的最终打印输出，请在 @titlepage 命令之前，在 Texinfo 文件的开头单独另起一行，编写以下内容：

```
@finalout
```

18.6 打印日文和中文

Texinfo 附带支持文件，用于处理日文和中文语言的输入。

对于日语，您应使用 texinfo-ja.tex 文件。您的主要输入文件应以 ‘\input texinfo-ja’ 行开头，而不是标准的 ‘\input texinfo’。如通常一样，使用 UTF-8 字符编码。您必须使用 XeT_EX 或 LuaT_EX 来处理这些文件。

对于中文，您需要使用 texinfo-zh.tex 文件。您的主要输入文件应以 ‘\input texinfo-zh’ 行开头，而不是标准的 ‘\input texinfo’。像通常一样，使用 UTF-8 字符编码。您必须使用 XeT_EX 处理这些文件。

例如，你可以运行 ‘PDFTEX=xetex texi2pdf my-input-file.texi’ 来使用 XeT_EX 生成 PDF 输出。你可能需要安装 XeT_EX（或 LuaT_EX）、‘zhspacing’ 包和(或)字体包。⁴

标准的 T_EX（或 pdfT_EX）程序既不支持日文也不支持中文，因为它无法支持这些语言所需的字体和成千上万的字形。

Texinfo 发行版附带了一些示例文件，可用于测试对这些语言的支持：short-sample-ja.texi 用于日语，short-sample-zh.texi 用于中文。

如需了解关于 T_EX 这些扩展的更多信息，请访问 XeT_EX 主页 (<https://tug.org/xetex/>) 和 LuaT_EX 网站 (<https://www.luatex.org/>)。

⁴ 译者注：在现有版本 xetex(3.141592653-2.6-0.999998)下，汉字无法实现‘粗体’，实际效果是‘楷体’；‘斜体’、‘无衬线’都无实际效果；‘等宽’、‘打印机风格’、‘罗马’，也许因为汉字本身就是‘等宽’，所以也都没有效果。

19 texi2any: Texinfo 的翻译器

`texi2any` 是 Texinfo 的通用翻译器，能够生成多种输出格式，且具有高度的可定制性。它支持以下格式：

- Info（默认，或使用 `--info` 选项），
- HTML（使用 `--html`），
- EPUB 3（使用 `--epub3`），
- 纯文本(plain text)（使用 `--plaintext`），
- L^AT_EX（使用 `--latex` 选项），
- DocBook（使用 `--docbook`），
- Texinfo XML（带 `--xml` 选项）。

`makeinfo` 是 `texi2any` 的别名。默认情况下，`texi2any` 和 `makeinfo` 都会生成 Info 输出；实际上，这两个命令在行为上并无差异。

除了这些默认格式外，`texi2any` 的命令行选项可以改变输出的许多方面。除此之外，初始化文件对最终输出提供了更多的控制——几乎可以控制 Texinfo 输入文件中未指定的任何事项。初始化文件与主程序一样，都是用 Perl 编写的，并且任何可以在命令行上指定的事项也可以在初始化文件中指定。

19.1 从 Shell 调用 `texi2any`

要处理 Texinfo 文件，请在命令行中输入“`texi2any`”后跟 Texinfo 文件的名称。同时，使用相应的命令行选项选择您希望输出的格式（默认为 Info）。因此，要为 Bison 创建 Info 文件，请在 shell 中键入以下命令：

```
texi2any --info bison.texi
```

您可以指定多个输入文件名；每个文件名都会依次处理。如果输入文件名为“-”，则读取标准输入。

`texi2any` 程序支持许多选项。其中，最基本的是那些用于更改输出格式的选项。默认情况下，`texi2any` 输出 Info 格式的文件。

每个命令行选项要么是一个以“`--`”开头的长选项，要么是一个以“-”开头的单字母选项。对于长选项名称，只要它们是唯一的，就可以使用缩写。

例如，您可以使用以下 shell 命令为 `bison.texi` 创建一个 Info 文件，该文件中的行仅填充到 68 列：

```
texi2any --fill-column=68 bison.texi
```

你可以按顺序列出两个或更多选项，如下所示：

```
texi2any --no-split --fill-column=70 ...
```

（这可能会使 Info 文件成为单一的非常长的文件，同时将行填充到 70 列。）

选项（大致按字母顺序排列）：

`--commands-in-node-names`

此选项目前无实际作用，但出于兼容性考虑仍保留。（它曾用于确保节点名称中的 `@` 命令在整个文档中得到扩展，尤其是 `@value`。现在，这一扩展是默认完成的。）

`--conf-dir=dir`

将 `dir` 添加到目录搜索列表中，以便查找可能通过 `--init-file`（见下文）加载的定制化文件。`dir` 的值可以是单个目录，也可以是由多个目录组成的列表，目

录之间用常见的路径分隔符分隔（在类 Unix 系统中为‘.’，在 Windows 系统中为‘;’）。

`--css-include=file`

在生成 HTML 时，请将 *file* 的内容（应包含级联样式表规范(Cascading Style Sheets specifications)）直接包含在 HTML 输出的‘<style>’块中。如果 *file* 为‘-’，则读取标准输入。请参阅 21.3 节[HTML CSS], 180 页。

`--css-ref=url`

在生成 HTML 时，请在输出中添加一个‘<link>’标签，该标签引用位于 *url* 的级联样式表。这样便可以使用独立的样式表。

`-D var`

`-D 'var value'`

在 Texinfo 文件中定义 Texinfo 变量 *var*。这等同于 `@set var`（请参阅 15.5 节[`@set @clear @value`], 124 页）。

该选项的参数在 shell 中始终为一个单词；如果其中包含内部空格，则第一个单词被视为变量名，其余部分作为值。例如，`-D 'myvar someval'` 等同于 `@set myvar someval`。

`--docbook`

产生 DocBook 输出，而不是 Info。

`--document-language=lang`

使用 *lang* 来翻译最终出现在输出文档中的 Texinfo 关键字。如果存在由 `@documentlanguage` 命令指定的语言环境，则默认使用该语言环境，否则使用英语（请参阅 14.1 节[`@documentlanguage`], 118 页）。

`--dvi` 使用 `texi2dvi` 生成 TeX DVI 文件，而不是 Info 输出(请参阅 19.3 节[`texi2any Printed Output`], 154 页)。

`--dvi pdf` 使用 `texi2dvi --dvi pdf` 生成 PDF 文件，而不是 Info 输出(请参阅 19.3 节[`texi2any Printed Output`], 154 页)。

`--enable-encoding`

`--disable-encoding`

默认情况下，或使用 `--enable-encoding` 选项时，会根据文档编码在 Info 输出和纯文本输出中显示重音符号和特殊字符。使用 `--disable-encoding` 选项时，会输出 7 位 ASCII 音译。在其他输出格式中，也遵循此规则进行索引键排序和一些纯文本输出。

`--epub3` 产生 EPUB 3 输出。

`--error-limit=limit`

`-e limit` 在报告 *limit* 个错误后中止（假设继续进行是无意义的）；默认值为 100。

`--fill-column=width`

`-f width` 指定一行中的最大列数；这是行的右边缘。段落将填充到此宽度。（填充是指将行拆分和连接，使每行的长度等于或小于指定为填充列的数量。单词之间会换行。）默认值为 72。

`--footnote-style=style`

`-s style` 将脚注样式设置为 *style*：可以是‘end’表示节点末尾样式（默认设置），也可以是‘separate’表示单独节点样式。此选项设置的值会覆盖 Texinfo 文件中由 `@footnotestyle` 命令设置的值(请参阅 9.3.2 节[Footnote Styles], 79 页)。

在 Info 中, 当脚注样式为‘separate’时, texi2any 会生成一个新节点包含当前节点中的脚注。当脚注样式为‘end’时, texi2any 会将脚注引用放在当前节点的末尾。

在 HTML 中, 当脚注样式为‘end’时, 或者如果输出未拆分, 脚注将置于输出的末尾。如果设置为‘separate’, 并且输出已拆分, 则脚注将被放置在一个单独的文件中。

- force
- F 通常情况下, 如果输入文件有错误, 则不会创建输出文件。但使用此选项时, 输出文件会被保留。
- help
- h 打印一条包含可用选项和基本用法的信息, 然后成功退出。
- html 生成 HTML 输出 (而非 Info)。默认情况下, HTML 输出会按节点拆分为一个个输出文件, 并根据输入文件的名称将拆分的输出文件写入子目录中。请参阅第 21 章[Generating HTML], 179 页。
- I *dir* 将 *dir* 添加到目录搜索列表中, 以便查找使用@include命令包含的文件。默认情况下, texi2any 会搜索当前目录以及输入文件路径的前导目录 (如果有且不是当前目录)。 *dir* 的值可以是单个目录, 也可以是由多个目录组成的列表, 目录之间用常见的路径分隔符分隔 (在类 Unix 系统中为‘:’, 在 Windows 系统中为‘;’)。
- ifdocbook
- ifhtml
- ifinfo
- iflatex
- ifplaintext
- iftex
- ifxml 对于给定的格式, 处理‘@ifformat’和‘@format’命令, 而不处理‘@ifnotformat’, 无论输出的是什么格式。例如, 如果指定了--iftex, 那么将读取‘@iftex’和‘@tex’块, 而忽略‘@ifnottex’块。
- no-ifdocbook
- no-ifhtml
- no-ifinfo
- no-iflatex
- no-ifplaintext
- no-iftex
- no-ifxml 对于给定的格式, 无论输出格式如何, 都不要处理‘@ifformat’和‘@format’命令, 而要处理‘@ifnotformat’。例如, 如果指定了--no-ifhtml, 则不会读取‘@ifhtml’和‘@html’块, 但会读取‘@ifnohtml’块。
- info 生成 Info 输出。默认情况下, 如果输出文件的大小超过约 300,000 字节, 则将其拆分为大小相近的子文件。输出文件和任何子文件的名称由输入文件名确定, 或者如果存在, 则由@setfilename确定 (请参阅2.5.2 节[@setfilename], 12 页)。请参阅20.2 节[Tag and Split Files], 176 页。

`--init-file=file`

将 *file* 作为代码加载，以修改生成的手册的行为和输出。通常，这些定制化文件使用 `.pm` 或 `.init` 扩展名，但这并不是强制性的；*file* 的名称可以是任何东西。可以使用 `--conf-dir` 选项（见上文）来添加搜索这些定制化文件的目录列表。

`--internal-links=file`

在 HTML 模式下，输出一个以制表符分隔的文件，其中包含三列：指向索引项或目录项的内部链接、索引（或目录）的名称以及被索引或输入的术语。这些项按照给定元素的自然排序顺序排列。此转储对于后处理器可能很有用。

`--latex` 产生 $\text{\LaTeX}$ 输出。

`--macro-expand=file`

`-E file` 将 Texinfo 源代码（所有 Texinfo 宏已展开）输出到 *file* 中。通常，宏展开的结果由 `texi2any` 内部使用后即被丢弃。

`--no-headers`

输出中不要包含菜单或节点分隔线。

在生成 Info 输出时，这与使用 `--plaintext` 相同，会生成一个简单的纯文本文件。此外，除非使用 `-o` 进行覆盖，否则输出将发送到标准输出。（此行为是为了向后兼容。）

在生成 HTML 时，如果输出内容被拆分，则仅在每个文件的开头输出导航链接。如果输出内容未被拆分，则根本不要在每个节点的顶部包含导航链接。请参阅第 21 章 [Generating HTML], 179 页。

`--node-files`

`--no-node-files`

使用 `--node-files` 选项，在生成 HTML 时，为锚点和任何尚未输出的节点创建重定向文件，文件名与节点名称相对应（请参阅 21.7.2 节 [HTML Xref Node Name Expansion], 185 页）。这使得章级和节级跨手册引用能够成功（请参阅 21.7.5 节 [HTML Xref Configuration], 187 页）。

如果输出是分割的，则默认启用此功能。如果输出不是分割的，则除了整体主输出文件外，`--node-files` 选项还会启用重定向文件的创建。`--no-node-files` 选项在任何情况下都会禁止创建重定向文件。此选项对除 HTML 以外的任何输出格式均无效。请参阅第 21 章 [Generating HTML], 179 页。

`--no-validate`

`--no-pointer-validate`

关闭 `texi2any` 的指针验证阶段——这是一件危险的事情。这也可以通过 `@novalidate` 命令来实现（请参阅 18.3.1 节 [Formatting Partial Documents], 145 页）。如果不关闭指针验证，`texi2any` 将检查 Texinfo 文件中的交叉引用和菜单条目的有效性，以及显式给出的节点指针的有效性。

`--no-warn`

关闭警告消息（但不关闭错误消息）。

`--number-footnotes`

`--no-number-footnotes`

使用 `--no-number-footnotes` 选项，可禁止自动为脚注编号。默认情况下，脚注在节点内按顺序编号，即每个节点开始时，当前脚注编号重置为 1。

`--number-sections`

`--no-number-sections`

使用 `--number-sections` (默认设置) 时, 会输出章、节和附录的编号, 就像在打印手册中一样。此功能仅适用于具有层次结构的手册。如果您的手册通常没有层次结构, 则应指定 `--no-number-sections`。

`--output=file`

`-o file` 指定输出应定向到 *file*。这将覆盖在 Texinfo 源代码中的 `@setfilename` 命令中指定的任何文件名。如果既未指定 `@setfilename` 也未指定此选项, 则使用输入文件名来确定输出文件名。请参阅 2.5.2 节 `[@setfilename]`, 12 页。

如果 *file* 为 '-', 则输出将发送到标准输出, 并默认 '`--no-split`' 选项。

如果 *file* 是一个目录或以 '/' 结尾, 则使用常规规则来确定输出文件名 (即, 使用 `@setfilename` 或输入文件名), 但文件会写入 *file* 目录。例如, '`texi2any -o bar/foo.texi`' (无论是否带有 `--no-split` 选项) 会在 *bar*/目录下写入 *bar*/*foo.info*, 以及可能的其他文件。

在生成 HTML 且输出被分割时, *file* 被用作所有文件写入的目标目录的名称。例如, '`texi2any -o bar --html foo.texi`' 将写入 *bar*/*index.html* 以及其他文件。

在生成 EPUB 时, 会为 EPUB 格式所需的文件和目录创建一个容器目录, 同时还会创建 EPUB 输出文件。如果 *file* 对应的是一个目录, 则容器目录会被放置在 *file* 中。EPUB 输出文件则永远不会放置在此目录中。如果 *file* 对应的是一个文件, 则该文件会被用作 EPUB 输出文件的名称。

`--output-indent=val`

此选项目前无实际作用, 但出于兼容性考虑仍保留。(它曾用于调整 XML/DocBook 输出中的缩进。)

`-P path` 将 *path* 添加到 `@include` 的目录搜索列表中。请参见上文的 '-I'。

`--paragraph-indent=indent`

`-p indent` 将段落缩进样式设置为 *indent*。此选项设置的值会覆盖由 `@paragraphindent` 命令 (请参阅节 E.3 `[@paragraphindent]`, 262 页) 在 Texinfo 文件中设置的值。 *indent* 的值解释如下:

'asis' 保留段落开头已有的缩进 (或无缩进)。

'0' 或 'none'

删除所有现有的缩进。

num 将每个段落缩进 *num* 个空格。

默认缩进两个空格, 但章节标题后的段落不缩进。

`--pdf` 使用 `texi2dvi --pdf` 命令生成 PDF 文件, 而不是 Info 输出 (请参阅 19.3 节 `[texi2any Printed Output]`, 154 页)。

`--plaintext`

输出一个纯文本文件 (而非 Info): 输出中不包含菜单或节点分隔线。这样输出的文件是直接的纯文本文件, 您可以 (例如) 轻松地通过电子邮件发送, 或将其包含在分发内容中 (例如, 一个 `INSTALL` 文件)。

使用此选项时, 默认情况下输出将发送到标准输出, 而不是发送到基于输入文件名或 `@setfilename` 指定的文件名; 这可以通过 `-o` 进行覆盖。

- ps** 使用 `texi2dvi --ps` 命令生成 PostScript 文件，而不是使用 Info 输出（请参阅 19.3 节 [texi2any Printed Output], 154 页）。
- set-customization-variable var=value**
-c var=value
 将定制化变量 `var` 设置为 `value`。= 是可选的，但 `var` 和 `value` 都必须根据需要使用引号括起来，以便结果为一个单词。除了通过在文档中的 @ 命令及其他命令行开关外，`texi2any` 的行为和输出的许多方面还可以通过定制化变量进行控制。
- split=how**
--no-split
 在生成 Info 时，默认情况下，较大的输出文件会被分割成较小的子文件，每个子文件大约为 300k 字节。在生成 HTML 时，默认情况下，每个输出文件包含一个节点（请参阅第 21 章 [Generating HTML], 179 页）。**--no-split** 选项可禁止分割输出文件。
 或者，可以使用 **--split=how** 来指定 HTML 输出应在哪个级别进行分割。`how` 的可能值包括：
 ‘chapter’ 输出在 @chapter 处或同级别的其他章节命令（如：@appendix 等）处进行分割。
 ‘section’ 输出在 @section 及同级别的其它章节命令处进行分割。
 ‘node’ 输出在每个节点处进行分割。这是默认设置。
 纯文本输出可以像 HTML 一样进行分割。这可能对于从 Texinfo 文档中提取部分内容并将其作为单独文件使用非常有用。
- split-size=num**
 如果可能的话，请将 Info 文件的大小限制在最多 `num` 个字符以内；默认值为 300,000。（但是，单个节点永远不会跨 Info 文件进行拆分。）
- trace-includes**
 将每个被包含文件（包括任何间接被包含的文件）的名称打印到标准输出。仅当实际找到被包含文件时，才会打印该文件。启用此选项后，`texi2any` 不会以任何输出格式输出。（此选项的目的是为了更方便地收集 Texinfo 手册的源代码。）
- transliterate-file-names**
 为构建文件名，启用节点名称中非 ASCII 字符的音译。请参阅 21.7.4 节 [HTML Xref 8-bit Character Expansion], 187 页。
- U var** 取消 `var` 的定义。这等同于 Texinfo 文件中的 @clear var（请参阅 15.5 节 [set @clear @value], 124 页）。
- verbose**
 让 `texi2any` 显示正在执行的操作的消息。通常，`texi2any` 仅在出现错误或警告时才会输出消息。
- version**
-V 打印版本号，然后成功退出。
- Xopt str**
 将 `str`（一个单独的 shell 单词）传递给 `texi2dvi`；可以重复使用，具体用途请参阅 19.3 节 [texi2any Printed Output], 154 页。
- xml** 生成 Texinfo XML 输出（而非 Info 格式）。

19.2 texi2any命令识别的环境变量

若未被命令行选项覆盖, `texi2any` 还会读取环境变量 `TEXINFO_OUTPUT_FORMAT` 以确定输出格式。该值应为以下之一:

`docbook dvi dvipdf epub3 html info latex pdf plaintext ps xml`

如果未设置或未以其他方式指定, 则默认输出为 `Info`。

`TEXINFO_OUTPUT_FORMAT` 可能接受一些其他特殊值, 这些值并不对应于任何可通过命令行设置的输出格式 (请参阅 19.4.2 节 [Customization Variables and Options], 156 页)。

还会读取同名定制化变量; 如果已设置, 则该变量会覆盖环境变量设置, 但不会覆盖命令行选项。请参阅 19.4.2 节 [Customization Variables and Options], 156 页。

您可以通过设置 `TEXINFO_XS` 环境变量来控制 `texi2any` 对 Perl 扩展模块的使用。这些模块是编译后的本地代码, 可供解释型 Perl 代码使用。理想情况下, 这些扩展模块应该能正常工作, 它们唯一明显的区别是让 `texi2any` 的运行得更快。然而, 您也可以使用此环境变量进行故障排除: 例如, 如果您发现 `texi2any` 的输出因是否使用扩展模块而有所不同, 那么问题可能就出在这里。

`texi2any` 可识别以下 `TEXINFO_XS` 值:

- ‘default’ 默认行为。尝试加载扩展模块, 如果失败, 则自动回退到解释型 Perl 实现。
- ‘warn’ 尝试加载扩展模块, 如果加载失败, 则在回退到解释型 Perl 实现之前给出警告消息。
- ‘debug’ 尝试加载扩展模块, 在此过程中会打印出许多消息。
- ‘omit’ 请勿使用扩展模块。

将 `TEXINFO_XS_PARSER` 设置为 ‘0’, 以禁用解析器模块的本机代码实现; 解析器模块是 `texi2any` 中将 Texinfo 输入转换为内部树格式的部分; 内部树格式用于进一步处理以生成输出格式。这可能有助于解决本机代码实现与纯 Perl 代码实现之间的错误或兼容性问题。

19.3 texi2any打印输出

为了名副其实地称为 “Texinfo-to-any”, `texi2any` 提供了对各种格式打印输出的基本支持: `TEX` DVI、PDF 和 PostScript。这是通过在请求这些输出格式时执行 `texi2dvi` 程序这一简单方法实现的, 在此之前会检查输入的有效性, 以便用户受益于 `texi2any` 的错误检查功能。如果你不希望进行此类错误检查, 或许是因为你的手册与 `texinfo.tex` 一起使用了高级的 `TEX` 技巧, 那么可以直接调用 `texi2dvi`。

此命令的输出格式选项包括 `--dvi`、`--dvipdf`、`--pdf` 和 `--ps`。有关这些选项以及 `texi2dvi` 常规操作的更多详细信息, 请参阅 [Format with `texi2dvi`], 142 页。此外, 如果指定了 `--verbose`、`--silent` 和 `--quiet` 选项, 则也会传递这些选项; 同样, `-I` 和 `-o` 选项及其参数也会被传递, 而 `--debug` 则不带参数。

与 `texi2dvi` 调用相关的唯一剩余选项是 `--Xopt`。在此, 仅传递参数, 且多个 `--Xopt` 选项可以累积。这为构建 `texi2dvi` 的任意命令行提供了一种方法。例如, 运行

```
texi2any --Xopt -t --Xopt @afourpaper --pdf foo.texi
```

这等效于运行

```
texi2dvi -t @afourpaper --pdf foo.texi
```

除了有效性检查。

尽管人们可能希望texi2any的其他选项能生效，但它们实际上并没有。例如，运行‘texi2any --no-number-sections --dvi foo.texi’仍然会生成一个带有章节编号的 DVI 文件。（如果收到相关请求，也许未来可以对此进行改进。）

所调命令的实际名称由TEXI2DVI定制化变量指定（请参阅19.4.7 节[Customization Variables for Multiple Formats], 164 页）。如您所料，默认值为‘texi2dvi’。

当texi2any调用texi2dvi时，它本身并不会生成任何正常输出，只会输出诊断消息。

19.4 定制化变量

警告

这些定制化变量名称及其含义可能会在 Texinfo 的任何版本中发生变化。我们始终尽量避免不兼容的更改，但由于需求会随时间变化，我们无法绝对保证不会发生此类更改。

通过修改定制化变量，可以调整texi2any的行为和输出的许多方面。这些变量大致可分为以下几类：

- 与 @ 命令相关联；例如，@documentlanguage。
- 与命令行选项相关联；例如，定制化变量SPLIT与--split 命令行选项相关联。TEXINFO_OUTPUT_FORMAT根据输出格式命令行选项设置，并允许指定输出格式。
- 影响texi2any的全局行为，特别是 Texinfo 代码解析。
- 与特定输出格式相关联。（仅与 HTML 输出相关的变量，在21.8 节[HTML Output Customization], 189 页中描述。）
- 适用于多种输出格式。

您可以在命令行上使用--set-customization-variable 'var value'（在 shell 中，将变量/值对加引号）或--set-customization-variable var=value（使用=）来设置定制化变量。一个特殊的value是‘undef’，它将变量设置为这个特殊的“undefined”的 Perl 值。

以下各节将详细介绍这些内容。

19.4.1 @ 命令对应的定制化变量

以下每个 @ 命令都有一个与之关联的定制化变量，变量名称命令名称相同（去掉前导@）：

@afivepaper	@afourpaper	@afourlatex
@afourwide	@allowcodebreaks	@bsixpaper
@contents	@clickstyle	@codequotebacktick
@codequoteundirected	@deftypefnnewline	@documentdescription
@documentencoding	@documentlanguage	@evenfooting
@evenfootingmarks	@evenheading	@evenheadingmarks
@everyfooting	@everyfootingmarks	@everyheading
@everyheadingmarks	@exampleindent	@firstparagraphindent
@fonttextsize	@footnotestyle	@frenchspacing
@headings	@kbdinputstyle	@microtype
@novalidate	@oddfooting	@oddfootingmarks
@oddheading	@oddheadingmarks	@pagesizes
@paragraphindent	@setfilename	@setchapternewpage

<code>@shortcontents</code>	<code>@smallbook</code>	<code>@summarycontents</code>
<code>@urefbreakstyle</code>	<code>@xrefautomaticsectiontitle</code>	

将此类定制化变量设置为值‘foo’，基本相当于执行@命令 foo；但并不完全相同，因为解析 Texinfo 源代码产生的任何副作用都不会重现。此外，在生成特定格式时，某些变量并不接受 Texinfo 代码，而只接受已格式化的参数。例如，对于documentdescription的 HTML 格式就是这样。

请注意，如果texi2any使用 T_EX（例如，使用--pdf 选项）处理文件，则这些定制化变量可能无法传递给 T_EX。

19.4.2 定制化变量与选项

下表列出了与某些命令行选项相关的定制化变量。关于这些选项的含义，请参阅19.1节[Invoking texi2any], 148 页。

选项	变量
--enable-encoding	ENABLE_ENCODING
--document-language	documentlanguage
--error-limit	ERROR_LIMIT
--fill-column	FILLCOLUMN
--footnote-style	footnotestyle
--force	FORCE
--headers	HEADERS, FORMAT_MENU
--internal-links	INTERNAL_LINKS
--macro-expand	MACRO_EXPAND
--no-validate	novalidate
--no-warn	NO_WARN
--node-files	NODE_FILES
--number-footnotes	NUMBER_FOOTNOTES
--number-sections	NUMBER_SECTIONS
--output	OUTFILE, SUBDIR
--paragraph-indent	paragraphindent
--split	SPLIT
--split-size	SPLIT_SIZE
--trace-includes	TRACE_INCLUDES
--transliterate-file-names	TRANSLITERATE_FILE_NAMES
--verbose	VERBOSE

如果选项需要参数，则将此类定制化变量设置为值‘foo’与指定--opt=foo基本相同；如果不需要参数，则与指定--opt 基本相同。

此外，定制化变量TEXINFO_OUTPUT_FORMAT允许指定texi2any的输出内容，可以是可通过选项指定的常用输出格式之一，也可以是其他各种形式：

'docbook'	
'dvi'	
'dvipdf'	
'epub3'	
'html'	
'info'	
'pdf'	
'plaintext'	
'ps'	
'xml'	这些对应于同名的命令行选项（以及TEXINFO_OUTPUT_FORMAT环境变量值）。当使用命令行选项（或TEXINFO_OUTPUT_FORMAT环境变量）设置输出格式时，会设置TEXINFO_OUTPUT_FORMAT定制化变量。请参阅19.1节[Invoking texi2any], 148 页。
'debugtree'	不生成常规输出格式，而是输出通过解析输入的 TeXinfo 文档得到的树的文本表示。
'parse'	仅解析 Texinfo 源代码；无输出。
'plaintexinfo'	输出已展开所有宏、@include和@value{}的 Texinfo 源文件。这与设置--macro-expand 类似，但不同的是，Texinfo 的输出是主要输出，而不是在正常转换之外进行的额外输出。
'rawtext'	输出原始文本，格式化程度最低。例如：忽略脚注，且不进行段落填充。这种格式的用途示例：解析器会使用它来处理文件名和 HTML 注释中的版权文本。
'structure'	仅解析 Texinfo 源代码并确定文档结构；无输出。
'texinfosxml'	以 Texinfo SXML 表示形式输出文档，这是一种使用 Lisp 的 S-表达式编写 XML 数据的语法。
'textcontent'	仅输出文本内容，去除所有的命令；这在拼写检查或字数统计等场景中非常有用。例如，Texinfo 源代码的util目录中有一个简单的detexinfo脚本，用于使用此功能。它只有一行： <pre>exec texi2any -c TEXINPUT_OUTPUT_FORMAT=textcontent "\$@"</pre>

19.4.3 全局定制化变量

此表列出了影响texi2any全局行为的定制化变量。

CHECK_MISSING_MENU_ENTRY

当节点中出现@menu块时，请检查文档的章节结构中所有下级节点是否都有对应的菜单项。默认为启用。

CHECK_NORMAL_MENU_STRUCTURE

如果节点指针（无论是显式设置还是自动设置）与节点菜单项的顺序不一致，则发出警告。这比CHECK_MISSING_MENU_ENTRY提供的结构检查更为彻底。默认情况下处于关闭状态。

COLLATION_LANGUAGE

默认情况下, `texi2any` 根据 *Unicode* 排序算法 (定义于 *Unicode* 技术标准 #10 (<http://www.unicode.org/reports/tr10/>)) 对文档索引进行排序, 而不进行特定语言的排序调整。如果进行了设置, 则使用该语言对索引排序进行语言调整。

目前, 如果由于缺少软件或缺少对指定语言的支持而导致语言调整不受支持, `texi2any` 不会给出任何警告。

在 Perl 中, `Unicode::Collate::Locale` 模块用于语言定制; 因此, 如果未安装此模块, 该变量将被忽略, 且不会给出任何提示。

如果 `USE_UNICODE_COLLATION` 设置为 '0', 则没有 *Unicode* 排序规则, 因此也没有语言定制。

另见下文所述的 `DOCUMENTLANGUAGE_COLLATION` 变量。

COMMAND_LINE_ENCODING

用于解码命令行参数的编码。默认编码基于区域设置编码。这可能会影响插入到输出文件中的文件名或程序打印的错误消息。

请注意, 命令行中的某些文件和目录名称可能无法立即解码, 甚至可能根本无法解码。

CPP_LINE_DIRECTIVES

在“预处理”阶段 (请参阅 16.7 节 [External Macro Processors], 137 页) 识别 `#line` 指令; 默认启用。

DEBUG 如果设置, 则会生成调试输出; 默认值为关闭 (零)。

DOC_ENCODING_FOR_INPUT_FILE_NAME

如果已设置, 则使用输入的 Texinfo 文档编码信息来对输入文件名进行编码, 例如作为 `@include` 或 `@verbatiminclude` 参数的文件名。如果未设置, 则使用区域设置编码。默认已设置, 但在 MS-Windows 上默认使用区域设置编码。

请注意, 这仅适用于文件名; 文件内容的编码始终使用默认编码或 `@documentencoding` (请参阅 14.2 节 [documentencoding], 119 页)。

`INPUT_FILE_NAME_ENCODING` 变量会覆盖此变量。

DOC_ENCODING_FOR_OUTPUT_FILE_NAME

如果已设置, 则使用输入的 Texinfo 文档编码信息来对输出文件名 (如使用 `--output` 指定的文件) 进行编码。如果未设置, 则使用区域设置编码。默认未设置, 因此文件名将使用当前区域设置进行编码。

请注意, 这仅适用于文件名; `OUTPUT_ENCODING_NAME` 用于文件内容的编码。

`OUTPUT_FILE_NAME_ENCODING` 变量会覆盖此变量。

DOCUMENTLANGUAGE_COLLATION

如果已设置, 尝试使用 `@documentlanguage` 指令中指定的语言, 以便对索引排序进行特定语言定制。另见前文所述的 `COLLATION_LANGUAGE` 变量。(请参阅 14.1 节 [documentlanguage], 118 页。)

DUMP_TEXI

用于调试。如果设置此选项, 则不会进行任何转换, 仅进行解析和宏扩展。如果设置了选项 `--macro-expand`, 则 Texinfo 源代码也会被扩展到相应的文件中。默认值为 `false`。

DUMP_TREE

用于调试。如果设置此项，则解析 Texinfo 文档时构建的树结构将输出到标准错误；默认为 false。

FORMAT_MENU

如果设置为‘menu’，则输出 Texinfo 菜单。这是 Info 的默认设置。对于 HTML，默认设置为‘sectiontoc’，此时不会输出@menu块的内容，而是在每个节点中输出子节点的列表。如果设置为‘nomenu’，则不输出菜单。

在生成 DocBook 时，或者指定了--no-headers 选项时，此变量设置为‘nomenu’。

IGNORE_SPACE_AFTER_BRACED_COMMAND_NAME

如果设置此项，则在使用大括号的 @ 命令后将忽略空格。默认为设置状态，与 TeX 的行为一致。

INPUT_FILE_NAME_ENCODING

用于输入文件名的编码。此变量会覆盖来自于文档或当前区域设置的任何编码。通常，您无需设置此变量，但如果文件名在文件系统中采用某种特定的字符编码，则可以使用此变量。另一种方法是，将DOC_ENCODING_FOR_INPUT_FILE_NAME设置为‘o’，以使用区域设置编码。另见OUTPUT_FILE_NAME_ENCODING。

LOCALE_ENCODING

从系统中获取的区域设置编码。您无需显式设置此变量。

MAX_MACRO_CALL_NESTING

通过@rmacro定义的 @ 命令的最大递归调用次数；默认值为 100000。此变量的目的是为了 避免无限递归。

MESSAGE_ENCODING

用于对texi2any输出的消息进行编码的编码方式。默认情况下，该编码方式基于本地编码。

它还用于从texi2any调用的命令所传递的命令行参数。例如，latex2html将被texi2any调用（HTML_MATH设置为‘l2h’时）。

NO_INDEX 如果设置此项，则不会记录索引条目，与索引相关的 @ 命令（如@printindex、@synindex或@defindex）将不起作用。默认值为 false。

NO_USER_COMMANDS

如果已设置，则忽略用于用户定义的 @ 命令。@alias、@definfoenclose、@defindex、@macro以及类似的 @ 命令均无效。默认值为 false。

OUTPUT_ENCODING_NAME

用于输出文件的规范化编码的名称。该名称应为 HTML 中可用的字符集名称，通常为首选的 IANA 编码名称之一。默认情况下，如果设置了输入编码（通常通过@documentencoding设置），则使用此信息来设置输出编码名称，否则输出编码将基于默认编码。请参阅14.2 节[@documentencoding], 119 页。

OUTPUT_FILE_NAME_ENCODING

用于输出文件名的编码。此变量会覆盖文档或当前区域设置的任何编码。

通常，您无需设置此变量，但如果需要在文件系统上以特定字符编码创建文件名，则可以使用此变量。另见INPUT_FILE_NAME_ENCODING。

PACKAGE	
PACKAGE_VERSION	
PACKAGE_AND_VERSION	
PACKAGE_URL	
PACKAGE_NAME	分别是实现中的短包名、包版本、包名和版本拼接、包 URL 以及完整包名。默认情况下，这些变量都是通过 Autoconf、Automake 和 configure 设置的。
PREFIX	输出文件前缀，会附加到某些输出文件名之前。默认情况下，它由 @setfilename 设置，或者从输入文件设置(请参阅 2.5.2 节[@setfilename], 12 页)。此值如何使用取决于其他定制化变量或命令行选项的值，例如输出是否被分割。默认情况下，此值未设置。
PROGRAM	所用程序的名称。默认情况下，它被设置为启动的程序的名称，并去除了结尾的‘.pl’。
SORT_ELEMENT_COUNT	如果设置，则指定一个文件名，将元素列表（节点或章节，具体取决于输出格式）转储到该文件中，并按去除 @ 命令后所含行数进行排序；默认未设置。此功能由 Texinfo 源码分发中 util/ 目录下的程序 texi-elements-by-size 使用（请参阅 [texi-elements-by-size], 232 页）。
SORT_ELEMENT_COUNT_WORDS	在转储按大小排序的元素文件时（见前项），请使用单词计数而非行数计数；默认为 false。
TEST	如果设置为 true，则通常每次运行时都会动态重新生成的一些变量（如日期、程序名称、版本）将被设置为固定值。这对于将输出与参考文件进行比较非常有用，测试中就是这样做的。默认值为 false。
TEXINFO_OUTPUT_FORMAT	生成的输出格式。可通过命令行选项（如：--html）设置，作为 TEXINFO_OUTPUT_FORMAT 环境变量的值（请参阅 19.2 节[texi2any Environment Variables], 154 页），或作为定制化选项（请参阅 19.4.2 节[Customization Variables and Options], 156 页）。 在 TEXINFO_OUTPUT_FORMAT 变量中设置的值通常与用于设置格式的名称相同，但并非总是如此。要将输出格式设置为 xml，可通过 --xml，也可通过 TEXINFO_OUTPUT_FORMAT 环境变量，也可以把 TEXINFO_OUTPUT_FORMAT 定制化变量设置为更明确的输出格式名称：texinfoxml。这种命名上的差异不应给用户带来任何可见的影响。
TREE_TRANSFORMATIONS	其值是一个以逗号分隔的转换的列表，这些转换可以在输出结果之前应用于 Texinfo 树。如果指定了多个转换，其顺序无关紧要；每个转换都会在处理过程中需要它时才被使用。 默认情况下，对于 HTML 和 DocBook 输出，会执行以下树形结构转换： 1. move_index_entries_after_items 2. relate_index_entries_to_table_entries 以下是一个更新文档中主控菜单的示例： <pre>texi2any \</pre>

```
-c TREE_TRANSFORMATIONS=regenerate_master_menu \
-c TEXINFO_OUTPUT_FORMAT=plaintexinfo \
mydoc.texi \
-o /tmp/out
```

警告: 由于‘plaintexinfo’输出会展开 Texinfo 宏和条件语句, 因此在将更新安装到原始文档之前, 有必要先消除所有此类差异。这一问题可能会在未来的版本中得到解决。

当前支持以下转换 (其中许多转换已在随 Texinfo 分发的 pod2texi 实用程序中用到; 请参阅 19.6 节 [Invoking pod2texi], 166 页):

- ‘complete_tree_nodes_menus’
根据章节树, 为与任何级别的章节相关联的节点添加菜单项或整个菜单。
- ‘complete_tree_nodes_missing_menu’
为与无菜单的章节相关联的节点添加完整菜单。这些菜单基于章节树。
- ‘fill_gaps_in_sectioning’
在章节树中添加空的 @unnumbered... 部分, 以填补章节划分中的层级空白。例如, 如果 @chapter 之后是 @subsection, 则将插入 @unnumberedsec。
- ‘insert_nodes_for_sectioning_commands’
为缺少对应节点的章节命令插入节点。
- ‘move_index_entries_after_items’
在 @enumerate 和 @itemize 中, 将出现在 @item 之前的索引条目移动到 @item 之后。索引条目之间的注释行也会随之移动。如前所述, 对于 HTML 和 DocBook 输出, 总是会执行这一操作。
- ‘regenerate_master_menu’
更新顶部节点主控菜单, 要么替换顶部节点菜单中的 (第一个) @detailmenu, 要么在顶部节点菜单末尾创建它。
- ‘relate_index_entries_to_table_entries’
在 @table、@vtable 和 @ftable 中, 将 @item 行后紧接的索引 @ 命令的索引条目信息与 @item 的第一个元素重新关联。从树中删除该索引 @ 命令。

USE_SETFILENAME_EXTENSION

Info 输出中, 默认为开启状态, 其他输出默认为关闭状态。若已设置, 则完全使用 @setfilename 为输出文件名所提供的內容, 包括扩展名。您无需显式设置此变量。

USE_UNICODE_COLLATION

默认情况下, texi2any 会根据 Unicode 排序算法 (Unicode Collation Algorithm) (定义于 Unicode 技术标准 #10 (<http://www.unicode.org/reports/tr10/>)) 对文档索引进行排序, 该算法对包含 Unicode 码位的字符串执行多级 (multi-level) 排序。(为此, 该程序使用了 Unicode::Collate 模块, 这是 Perl 安装的标准组成部分。)然而, 在处理具有大型索引的手册时, 使用此算法会导致运行时间明显延长。

将此变量设置为 0，以使用更简单、更快速的排序算法。

如果你在索引条目文本中非 ASCII 字符不多，或者你并不关心索引是否正确排序，尤其是在处理手册草稿时，你可能会发现这对加快texi2any的运行速度很有用。

USE_UNIDECODE

如果设置为 false，则不使用Text::Unidecode Perl 模块来转写更多的字符；默认为 true。

19.4.4 Info 及纯文本输出相关的定制化变量

此表列出了仅适用于 Info 格式及纯文本格式输出的定制化变量。

ASCII_DASHES_AND_QUOTES

对于 Info 输出，当设置此选项时，如果 Texinfo 源文件中使用‘-’、‘--’、‘---’、‘`’、‘``’、‘’和‘’’来表示引号、连字符和破折号，则使用普通 ASCII 字符来表示，而不是 UTF-8 的有方向引号、长破折号等。此选项默认为启用。

ASCII_GLYPH

如果为打开状态，对于 Info 输出，对于如版权符号 (@copyright{}，变为‘(C)’) 和项目符号 (@bullet{}，变为‘*’) 的字形(glyph)命令，使用 ASCII 输出，而不是其他 Unicode 序列。默认情况下为关闭状态。

ASCII_PUNCTUATION

避免输出中任何不必要或无意义的非 ASCII、UTF-8 序列。这意味着同时应用ASCII_DASHES_AND_QUOTES和ASCII_GLYPH，并且还会影响如@samp这类输出引号的命令的输出。

AUTO_MENU_DESCRIPTION_ALIGN_COLUMN

对于 Info 输出，由@nodedescription或@nodedescriptionblock提供的菜单项描述开始的列。默认情况下未定义，在这种情况下，将使用总列宽的 45%（请参阅19.1 节[Invoking texi2any], 148 页）。

AUTO_MENU_MAX_WIDTH

在从@nodedescription或@nodedescriptionblock添加描述时，Info 中菜单条目行中的最大列数。默认情况下未定义，在这种情况下，将使用总列宽的 110%（请参阅19.1 节[Invoking texi2any], 148 页）。

CLOSE_DOUBLE_QUOTE_SYMBOL

当 Info 中的‘@dfn’需要闭合双引号时，使用此字符。Info 的默认设置与OPEN_DOUBLE_QUOTE_SYMBOL相对应，只是其 Unicode 代码为闭合双引号（见下文）。

INDEX_SPECIAL_CHARS_WARNING

如果启用此设置，对在索引条目中含有‘:’时提出警告，因为并非所有 Info 阅读器都能处理这些内容。此设置仅适用于 Info 和纯文本格式。默认值为 false，因为解析问题并不会阻止导航；阅读器仍然可以相对轻松地找到问题节点。

INFO_SPECIAL_CHARS_QUOTE

如果已设置，每当在节点名称或菜单项等位置出现有问题的字符时，应按照附录 F [Info Format Specification], 266 页中的描述，用引号字符将它们出现的构造部分括起来。对于 Info，默认设置为启用此功能，对于纯文本则未设置。请参阅3.3 节[Node Line Requirements], 25 页。

INFO_SPECIAL_CHARS_WARNING

对于 Info 输出, 如果已设置, 则对节点名称、菜单项和交叉引用中的问题构造 (如字符串`:.:`) 发出警告。如果未定义, 则除非设置了 `INFO_SPECIAL_CHARS_QUOTE`, 否则会自动设置此选项。默认情况下, 此选项为 Info 输出而设置, 对于纯文本输出则是未定义。是否对索引条目发出类似警告, 由 `INDEX_SPECIAL_CHARS_WARNING` 覆盖。

OPEN_DOUBLE_QUOTE_SYMBOL

当需要在 Info 中的`@dfn`输出中使用开放双引号开头时, 请使用指定的字符。如果设置了 `--disable-encoding` 或者文档编码不是 UTF-8, 则使用`'`。否则, Info 输出将使用 Unicode 左双引号。

19.4.5 L^AT_EX 定制化变量**警告**

L^AT_EX 输出定制功能尚处于实验阶段。一切尚未确定, 随时可能更改, 我们欢迎任何反馈。

此表列出了仅适用于 L^AT_EX 输出的定制化变量。

CLASS_BEGIN_USEPACKAGE

如果设置, 相应的文本将替换 L^AT_EX 的:

- a. `\documentclass`。
- b. 始终紧接在 `\documentclass` 之后输出的包导入。
- c. 依赖于输入文档编码和字体编码 (`inputenc` 和 `fontenc`) 的包导入。

被替换的文本如下:

```
\documentclass{book}
\usepackage{amsfonts}
\usepackage{amsmath}
\usepackage[gen]{eurosym}
\usepackage{textcomp}
\usepackage{graphicx}
\usepackage{etoolbox}
\usepackage{titleps}
\usepackage[utf8]{inputenc}
\usepackage[T1]{fontenc}
```

END_USEPACKAGE

如果设置, 相应的文本将替换:

- a. 依赖于所使用的 Texinfo 命令的包导入。
- b. 始终紧接在其他所有包导入之后输出的最后一个包导入。

最后一个包导入对应于 `\usepackage[hidelinks]{hyperref}`。

以下是一个包含索引、`@need`、`@multitable`、定义命令、`@cartouche`、列表和 `@float` 的 Texinfo 文档相应的文本示例:

```
\usepackage{imakeidx}
```



```

\usepackage{needspace}
\usepackage{array}
\usepackage{embrac}
\usepackage{expl3}
\usepackage{tabularx}
\usepackage[framemethod=tikz]{mdframed}
\usepackage{enumitem}
\usepackage{float}
\usepackage[hidelinks]{hyperref}

```

LATEX_FLOATS_FILE_EXTENSION

为 L^AT_EX 输出, 设置新的浮动定义文件的扩展名。默认扩展名为‘tfl’。

19.4.6 其他格式定制化变量

此表列出了仅适用于特定输出格式的定制化项, 这些输出格式没有单独的定制化变量描述章节。

TXI_MARKUP_NO_SECTION_EXTENT

对于 XML, 请勿在上级章节划分命令标记内包含下级章节划分命令。默认值为 false。

TEXI2DVI 用于生成 PostScript、PDF 和 DVI 的命令的名称; 默认为‘texi2dvi’。请参阅 19.3 节[texi2any Printed Output], 154 页。

TEXTCONTENT_COMMENT

用于剥离文本内容输出(即: 当 TEXINFO_OUTPUT_FORMAT 设置为 textcontent 时)。如果已设置, 则同时输出注释。默认值为 false。

TEXINFO_DTD_VERSION

用 XML 输出, 在前导中使用的 DTD 版本。默认值基于 configure.ac 中的变量设置。

19.4.7 适用于多种格式的定制化变量

此表列出了适用于多种格式的定制化项。

CLOSE_QUOTE_SYMBOL

当需要闭合引号时, 例如在 @samp 输出中, 请使用此字符; 在 DocBook 中, 默认使用 ’。在 HTML 的默认情况下, 该值未定义; 如果未设置 USE_NUMERIC_ENTITY, 则将其设置为 ’; 如果已设置, 则设置为 ’; 如果已设置 OUTPUT_CHARACTERS, 且输出编码包含引号(quote)字符, 则设置为该字符。

Info 的默认设置与 OPEN_QUOTE_SYMBOL 相似, 但是其 Unicode 代码为闭合引号(见下文)。

DOCTYPE 对于 DocBook、HTML、XML, 指定了 SystemLiteral, 即实体的系统标识符。这是一个可用于检索实体的 URI, 并标识了文档的规范 DTD。对于 HTML、DocBook 和 XML, 其默认值各不相同。

EXTENSION

输出文件名后添加的扩展名。每种输出格式的默认扩展名都不同。

NO_TOP_NODE_OUTPUT

如果设置此项，则不输出顶节点内容。顶节点仍会被解析，但内容会被丢弃。在 HTML 的默认情况下，此项未设置。在 EPUB 的默认情况下，此项已设置。如果 `SHOW_TITLE` 为 `'undef'`，则设置 `NO_TOP_NODE_OUTPUT`，同时也会为 HTML 设置 `SHOW_TITLE`。

设置 `NO_TOP_NODE_OUTPUT` 会移除顶部节点并添加标题页(封面)，这更符合书籍的格式。将 `NO_TOP_NODE_OUTPUT` 设置为 `false`，同时保持 `SHOW_TITLE` 为 `'undef'`，则更符合用于浏览的文档设置，可以直接访问顶节点的信息。

对于 DocBook，`NO_TOP_NODE_OUTPUT` 被设置为 `true`。将 `NO_TOP_NODE_OUTPUT` 设置为 `false` 会导致输出顶层节点内容。由于标题和复制许可（由 `@copying ... @end copying` 定义）始终会输出，因此不建议在 DocBook 中输出顶层节点。此选项在 DocBook 中保留是为了兼容性，因为在 2022 年之前，DocBook 中会输出顶层节点。将来可能会将其移除。

OPEN_QUOTE_SYMBOL

当需要开放引号时，例如在 `@samp` 输出中，需使用此字符；对于 DocBook，默认使用 `‘`。在 HTML 的默认情况下，该变量未定义；如果未设置 `USE_NUMERIC_ENTITY`，则设置为 `‘`；如果 `USE_NUMERIC_ENTITY` 已设置，则设置为 `‘`；如果 `OUTPUT_CHARACTERS` 设置，且输出编码包含引号(`quote`)字符，则使用该字符。

对于 Info，默认值取决于启用的文档编码。如果设置了 `--disable-encoding` 或者文档编码不是 UTF-8，则使用 `'`。在现代系统中，这个字符通常显示为无方向的单引号。否则，Info 输出将使用 Unicode 左引号。

OUTPUT_CHARACTERS

如果未设置（默认情况），在 HTML、XML 和 DocBook 中使用 XML 实体输出重音符号和特殊字符，在 $\text{\LaTeX}$ 中使用宏输出。如果已设置，则根据文档编码在 HTML、XML、DocBook 和 $\text{\LaTeX}$ 输出中输出重音字符，在 HTML 和 $\text{\LaTeX}$ 输出中输出特殊字符。请参阅 14.2 节 [`@documentencoding`], 119 页，以及 11.4 节 [Inserting Accents], 91 页。

TOP_NODE_UP

顶节点的上一级节点；默认为 `'(dir)'`。此节点名称应已针对输出格式进行了格式化。在 HTML 中，它可以用在属性中，因此不应包含任何元素。仅当 `TOP_NODE_UP_URL` 被设置为覆盖 URL 时，才用于 HTML 输出，请参阅 21.8.9 节 [HTML Customization Variables List], 198 页中的 `TOP_NODE_UP_URL`。请参阅 21.8.2 节 [File Names and Links Customization for HTML], 191 页。

USE_NODES

输出文档的主要单元称为输出单元(*output unit*)。通常，一个节点与后续的章节命令相关联，而一个章节命令与之前的节点相关联；它们共同构成了输出单元。

对于纯文本和 HTML，如果设置了 `USE_NODES`，则优先使用节点来决定输出单元的分割位置。如果设置为 `false`，则优先使用章节来决定输出单元的分割位置。默认值为 `true`。

USE_NUMERIC_ENTITY

用于 HTML、XML 和 DocBook 输出。如果已设置，则使用数字实体而非命名实体。默认情况下，对于 DocBook 输出，设置为 `true`。

19.5 文档字符串的国际化

texi2any在输出文档的不同位置写入固定字符串: 交叉引用(cross-references)、页脚(page footers)、帮助页面(the help page)、图像的替代文本(alternate text for images)等。字符串的选择取决于输出字符串时documentlanguage的值 (Texinfo 命令接口, 请参阅14.1 节[@documentlanguage], 118 页)。

带有'texinfo_document'域的 Gettext 框架用于处理这些字符串 (请参阅Gettext)。翻译后的字符串以 Texinfo 格式编写, 并可能包含 @ 命令。在翻译后的字符串中, 字符串的不同部分用'{arg_name}'表示, 而不是用%s等表示。(这一约定在 Perl 中的gettext中很常见, 并且在 GNU Gettext 中得到了完全支持; 请参阅节 "Perl Format Strings" 在 GNU Gettext。)例如, 在以下示例中, '{section}'将被替换为章节名称:

```
see {section}
```

翻译字符串的扩展方式如下:

1. 首先, 对字符串进行翻译。区域设置为documentlanguage.documentencoding。
如果documentlanguage 的格式为'11_CC', 则首先尝试使用该格式, 若失败则仅使用'11'。
2. 接下来, 在大多数情况下, 字符串会被展开为 Texinfo 格式, 并进行转换。参数会被替换; 例如, '{arg_name}'会被替换为对应的实际参数。

字符串和参数也有可能已经被转换。在这种情况下, 参数只需进行替换。同样地, '{arg_name}'会被相应的实际参数所替换。

在以下示例中, '{date}'、'{program_homepage}'和'{program}'是字符串的参数。由于它们在@uref中使用, 因此其顺序不可预测。在展开后, '{date}'、'{program_homepage}'和'{program}'会被替换:

```
Generated on @emph{{date}} using
@uref{{program_homepage}, @emph{{program}}}.
```

诚然, 这种方法略显复杂。其优点在于, 它支持独立于输出格式来指定翻译后的字符串的格式。

例如, '@pxref'翻译字符串可以像这样:

```
see {node_file_href} section '{section}' in @cite{{book}}
```

这允许独立于输出格式来指定字符串, 同时, 尽管格式丰富, 它仍能在多种语言中得到恰当的翻译。如果在texi2any中使用 Perl 进行字符串翻译, 则使用libintl-perl包作为gettext的实现; 更具体地说, 使用的是纯 Perl 实现, 这样 Texinfo 就能在所有平台和安装中保持一致的行为, 否则这是不可能实现的。libintl-perl包含在 Texinfo 分发版中, 并且总是会安装, 以确保在需要时可以使用。也可以使用系统自带的libintl-perl (可在构建时进行选择)。

19.6 调用pod2texi: 将 Pod 转换为 Texinfo

pod2texi程序将 Perl Pod 文档文件转换为 Texinfo。它有两种基本操作模式: 从每个输入 Pod 生成独立的手册, 或者 (如果指定了--base-level=1或更高选项) 生成适合与@include一起使用的 Texinfo 子文件。

pod2texi程序可能在 Texinfo 之外的其他地方有用; 因此, pod2texi的帮助在 Pod 语言中以手册页(manpage)格式文档进行编写, 以遵循 Perl 独立程序中的约定, 其网络版本见<https://www.gnu.org/software/texinfo/manual/pod2texi.html>, 下文也提供了一个版本, 该版本也是pod2texi使用的一个示例, 因为它是由pod2texi从 Pod 转换而来的。

19.6.1 pod2texi

pod2texi NAME

pod2texi - convert Pod to Texinfo

pod2texi SYNOPSIS

pod2texi [OPTION]... POD...

pod2texi DESCRIPTION

Translate Pod file(s) to Texinfo. There are two basic modes of operation. First, by default, each Pod is translated to a standalone Texinfo manual.

Second, if `--base-level` is set higher than 0, each Pod is translated to a file suitable for `@include`, and one more file with a main menu and all the `@include` is generated.

pod2texi OPTIONS

`--appendix-sections`

Use appendix sectioning commands (`@appendix`, ...) instead of the default numbered sectioning Texinfo `@`-commands (`@chapter`, `@section`, ...).

`--base-level=NUM|NAME`

Sets the level of the `head1` commands. It may be an integer or a Texinfo sectioning command (without the `@`): 1 corresponds to the `@chapter/@unnumbered` level, 2 to the `@section` level, and so on. The default is 0, meaning that `head1` commands are still output as chapters, but the output is arranged as a standalone manual.

If the level is not 0, the Pod file is rendered as a fragment of a Texinfo manual suitable for `@include`. In this case, each Pod file has an additional sectioning command covering the entire file, one level above the `--base-level` value. Therefore, to make each Pod file a chapter in a large manual, you should use `section` as the base level.

For an example of making Texinfo out of the Perl documentation itself, see `contrib/perl-doc-all` in the Texinfo source distribution.

`--debug=NUM`

Set debugging level to *NUM*.

`--generate-setfilename`

Generate a `@setfilename` line for standalone manuals. Can be negated with `--no-generate-setfilename`. Ignored if `--base-level` is not 0.

`--headings-as-sections`

Use headings commands (`@heading`, ...) instead of the default numbered sectioning Texinfo `@`-commands (`@chapter`, `@section`, ...). The sectioning command covering the entire file output for each Pod file if `--base-level` is not 0 is a numbered command.

`--help`

Display help and exit.

--menus

Output node menus. If there is a main manual, its Top node menu is always output, since a master menu is generated. Other nodes menus are not output in the default case.

--outdir=NAME

If there is a main manual with include files (each corresponding to an input Pod file), then the generated Texinfo files are put in directory *NAME*. Default is based on **--subdir**.

--output=NAME

Name for the first manual, or the main manual if there is a main manual. Default is to write to standard output.

--no-section-nodes

Use anchors for sections instead of nodes.

--no-fill-section-gaps

Do not fill sectioning gaps with empty `@unnumbered` files. Ordinarily, it's good to keep the sectioning hierarchy intact.

--preamble=STR

Insert *STR* as top boilerplate before menu and includes. If *STR* is set to `-`, read the top boilerplate from the standard input. The default top boilerplate is a minimal beginning for a Texinfo document.

--setfilename=STR

Use *STR* in top boilerplate before menu and includes for `@setfilename` for the main manual, if **--base-level** is not set to 0. Ignored if **--base-level** is 0. No `@setfilename` is output in the default case for the main manual.

--subdir=NAME

If there is a main manual with include files (each corresponding to an input Pod file), then those include files are included from *NAME*.

If **--outdir** is set, *NAME* should in general be set to the relative directory between the main manual and **--outdir** argument.

--unnumbered-sections

Use unnumbered sectioning commands (`@unnumbered`, ...) instead of the default numbered sectioning Texinfo `@`-commands (`@chapter`, `@section`, ...).

--top=TOP

Name of the `@top` element for the main manual. May contain Texinfo code.

--version

Display version information and exit.

pod2texi SEE ALSO

Pod::Simple::Texinfo (<https://www.gnu.org/software/texinfo/manual/Pod/Simple/Texinfo.html>). perlpod (<https://metacpan.org/pod/perlpod>). The Texinfo manual. Texinfo home page: <https://www.gnu.org/software/texinfo/>

pod2texi COPYRIGHT AND LICENSE

Copyright 2012-2024 Free Software Foundation, Inc.

This program is free software; you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.

There is NO WARRANTY, to the extent permitted by law.

pod2texi AUTHOR

Patrice Dumas <bug-texinfo@gnu.org>.

19.7 texi2html: texi2any的祖先

从概念上讲, `texi2html` 程序是当今 `texi2any` 程序的鼻祖。`texi2html` 最初由莱昂内尔·康斯(Lionel Cons) 于 1998 年独立开发; 当时, `makeinfo` 还无法生成 HTML。多年来, 许多其他人也为 `texi2html` 做出了贡献。

当前的 `texi2any` 几乎未使用 `texi2html` 的实际代码, 其实现的基本方法也大不相同 (即将 Texinfo 文档解析为树结构), 但两者之间仍存在相似之处。

从设计上讲, `texi2any` 在某种程度上支持了 `texi2html` 的几乎所有特性。然而, 我们并没有试图保持严格的兼容性, 因此 Texinfo 包并不会安装 `texi2html` 可执行文件。一个近似的运行方式可以这样调用:

```
texi2any --set-customization-variable TEXI2HTML=1 ...
```

但要强调的是, 这不是 对之前的 `texi2html` 的直接替换。以下是最大的区别:

- 最明显的是, `texi2html` 的命令行选项现在大部分都是定制化变量。下面给出了一个大致等效项的表格。
- 在 `texi2any` 中, 程序级的自定义 API 非常不同。
- 索引不能被拆分。

我们不打算重新实现这些差异。因此, Texinfo 的作者们的前进之路是: 根据需要修改手册和构建流程, 以使用 `texi2any` 的新特性和方法。`texi2html` 的维护者 (其中一位是 `texi2any` 的主要作者) 不打算再发布新版本。

以下是显示 `texi2html` 选项和对应 `texi2any` 定制化变量的表格。

<code>--toc-links</code>	TOC_LINKS
<code>--short-ext</code>	EXTENSION 设置为‘htm’
<code>--prefix</code>	PREFIX
<code>--def-table</code>	DEF_TABLE
<code>--html-xref-prefix</code>	EXTERNAL_DIR
<code>--l2h</code>	HTML_MATH 设置为‘l2h’
<code>--l2h-l2h</code>	L2H_L2H
<code>--l2h-skip</code>	L2H_SKIP
<code>--l2h-tmp</code>	L2H_TMP
<code>--l2h-file</code>	L2H_FILE
<code>--l2h-clean</code>	L2H_CLEAN
<code>--use-nodes</code>	USE_NODES
<code>--monolithic</code>	MONOLITHIC

<code>--top-file</code>	<code>TOP_FILE</code>
<code>--frames</code>	无等效
<code>--menu</code>	<code>FORMAT_MENU</code>
<code>--debug</code>	<code>DEBUG</code>
<code>--doctype</code>	<code>DOCTYPE</code>
<code>--frameset-doctype</code>	无等效
<code>--test</code>	<code>TEST</code>

最后, 任何寻求更详细信息的texi2html用户, 可以查看 Texinfo 源代码库中存档文件doc/texi2oldapi.texi的第一部分。

20 创建和安装 Info 文件

本章提供了一些关于 Info 输出的信息，并描述了如何安装 Info 文件。创建 Info 文件，如使用 `texi2any`，则参见第 19 章[Generic Translator `texi2any`], 148 页；如使用 Emacs，则参见节 D.6 [Info Formatting], 245 页。关于文件格式的一般信息，请参见请参阅 1.3 节[Info Files], 4 页；关于 Info 格式的详细技术规范，请参见附录 F [Info Format Specification], 266 页。

20.1 安装 Info 文件

Info 文件通常保存在 `info` 目录中。您可以使用独立的 Info 程序或 Emacs 内置的 Info 阅读器来阅读 Info 文件。（有关 Info 的介绍，请参见 *Info*。）

20.1.1 目录文件：dir

为了让 Info 系统正常工作，`info` 目录必须包含一个文件，该文件作为 Info 系统的顶级目录。按照惯例，这个文件被称为 `dir`。（在 Emacs 中，您可以通过输入 `C-h i` 进入 Info 模式，然后输入 `C-x C-f` 来查看 `info` 目录的位置，从而找到这个文件的位置。）

`dir` 文件本身是一个 Info 文件。它包含了系统中所有 Info 文件的顶级菜单。该菜单看起来像这样：

```
* Menu:
* Info:      (info).      Documentation browsing system.
* Emacs:     (emacs).     The extensible, self-documenting
                        text editor.
* Texinfo:   (texinfo).   With one source file, make
                        either a printed manual using
                        @TeX{} or an Info file.
...
```

这些菜单项中的每一个都指向在括号中命名的 Info 文件的‘Top’（顶）节点。（菜单项无需指定‘Top’节点，因为如果没有提及节点名称，Info 会跳转到‘Top’节点。请参阅 3.9.6 节[Nodes in Other Info Files], 32 页。）

因此，‘Info’条目指向 `info` 文件的‘Top’节点，而‘Emacs’条目则指向 `emacs` 文件的‘Top’节点。

在每个 Info 文件中，‘Top’节点的‘Up’指针都会回指到 `dir` 文件。例如，在 Info 独立程序中，展示 Emacs 手册的‘Top’节点时，导航行看起来像这样：

```
File: emacs Node: Top, Up: (DIR), Next: Distrib
```

在这种情况下，`dir` 文件名是用大写字母书写的——它在一般情况下可以用大写或小写字母书写，但现在不是一般情况，这是 `dir` 的特殊情况。

有一个 `dir` 文件的大型示例，请参阅 Texinfo 发行版中的 `util/dir-example` 文件。

20.1.2 列出新的 Info 文件

要在系统中添加新的 Info 文件，您必须在 `info` 目录下的 `dir` 文件中编写一个菜单条目以添加到菜单中。例如，如果您要为 GDB 添加文档，则可以编写以下新条目：

```
* GDB: (gdb).          The source-level C debugger.
```

菜单条目的第一部分是菜单条目名称，后跟一个（半角）冒号。第二部分是 Info 文件的名称，用括号括起来，后跟一个（半角）句号。第三部分是描述。

Info 文件的名称通常带有 `.info` 扩展名。因此，GDB 的 Info 文件可能被称为 `gdb` 或 `gdb.info`。Info 阅读器程序会自动尝试使用带有和不带 `.info` 扩展名的文件名¹；因此，最好避免混淆，不要在菜单条目中明确写出 `.info`。例如，GDB 的菜单条目应仅使用 `'gdb'` 作为文件名，而不是 `'gdb.info'`。

20.1.3 其他目录中的 Info 文件

如果 Info 文件不在 `info` 目录中，有三种方法可以指定其位置：

1. 将文件名作为菜单的第二部分写入 `dir` 文件中。
2. 在您的 `.profile` 或 `.cshrc` 初始化文件中，请在 `INFOPATH` 环境变量中指定 Info 目录的名称。（只有您和其他设置了此环境变量的人才能找到以这种方式指定位置的 Info 文件。）
3. 如果你正在使用 Emacs，请在文件所在的目录中，在另一个 `dir` 文件中列出该文件的名称；然后将该目录的名称添加到你的个人或本机初始化文件中的 `Info-directory-list` 变量中。

此变量告诉 Emacs 在何处查找 `dir` 文件（文件必须命名为 `dir`）。Emacs 会合并来自每个所列目录中名为 `dir` 的文件。（在 Emacs 版本 18 中，您只能将 `Info-directory` 变量设置为一个目录的名称。）

例如，要访问位于 `/home/bob/info` 目录中的测试文件，您可以在标准 `dir` 文件的菜单中添加类似如下的条目：

```
* Test: (/home/bob/info/info-test). Bob's own test file.
```

在这种情况下，`info-test` 文件的绝对文件名会作为菜单项的第二部分被写入。

如果你不想编辑系统中的 `dir` 文件，你可以通过在 shell 启动文件中设置 `INFOPATH` 环境变量来告诉 Info 去哪里查找。这种方法适用于 Emacs 和独立的 Info 阅读器。

Emacs 使用 `INFOPATH` 环境变量来初始化 Emacs 自己的 `Info-directory-list` 变量的值。独立的 Info 阅读器会将 `INFOPATH` 变量中列出的所有目录中名为 `dir` 的所有文件合并到一个菜单中，该菜单会在名为 `'(dir)Top'` 的节点中呈现给您。

无论你怎么设置 `INFOPATH`，如果它的最后一个字符是冒号（在 MS-DOS/MS-Windows 系统上，应使用分号），默认（内置）路径自动追加在其后。这样，你就可以用新的目录来扩充默认路径，而无需列出所有标准位置。例如（使用 `sh` 语法）：

```
INFOPATH=/home/bob/info:
export INFOPATH
```

会先搜索 `/home/bob/info`，然后再搜索标准目录。前导冒号或双冒号不会被特殊处理。

当你创建自己的 `dir` 文件以与 `Info-directory-list` 或 `INFOPATH` 一起使用时，最简单的方法是先复制一个现有的 `dir` 文件，并将 `'* Menu:'` 之后的全部文本替换为你想要的条目。这样，Info 所需的标点和特殊 `CTRL-_` 字符就会显示出来。

作为最后一个选项（仅适用于 Emacs Info），你可以更改 `Info-directory-list` 变量。例如：

```
(add-hook 'Info-mode-hook '(lambda ()
  (add-to-list 'Info-directory-list
    (expand-file-name "~/info"))))
```

¹ 在 MS-DOS/MS-Windows 系统上，Info 还会尝试使用 `.inf` 扩展名。

20.1.4 安装 Info 目录文件

当你在系统上安装一个 Info 文件时，你可以使用程序`install-info`来更新 Info 目录文件`dir`。通常，在将 Info 文件复制到你正确的安装位置后，包里的`makefile` 就会运行`install-info`。

为了让 Info 文件能够与`install-info`一起使用，您需要在 Texinfo 源文件中包含命令`@dircategory`和`@direntry...@end direntry`。使用`@direntry`来指定要添加到 Info 目录文件中的菜单条目。使用`@dircategory`来为手册指定一个类别，该类别决定了将手册放入 Info 目录的哪个部分。请参阅2.6 节[Directory Category], 14 页。

本手册中这些命令的使用方法如下：

```
@dircategory Texinfo 文档系统
@direntry
* Texinfo: (texinfo-zh_CN).          GUN 文档格式.
* install-info: (texinfo-zh_CN)Invoking install-info. 更新 info/dir 条目.
...
@end direntry
```

以下是这在 Info 文件中生成的内容：

```
INFO-DIR-SECTION Texinfo 文档系统
START-INFO-DIR-ENTRY
* Texinfo: (texinfo-zh_CN).          GNU 文档格式.
* install-info: (texinfo-zh_CN)Invoking install-info. 更新 info/dir 条目.
...
END-INFO-DIR-ENTRY
```

`install-info`程序会在 Info 文件中看到这些行，从而知道该做什么。

请务必在 Texinfo 输入的开头附近，即在第一个`@node`命令之前，使用`@direntry`和`@dircategory`命令。如果在后面的地方才使用这此命令，`install-info`将不会注意到它们。

`install-info`会自动重新格式化它所添加的菜单条目的描述。按照惯例，主条目（上文中的‘GNU 文档格式’）的描述应从第 32 列开始，从零开始计数（与 Emacs 中的`what-cursor-position`的方式一样）。这将使其与大多数其他主条目对齐。单个实用程序的描述最好尽可能从第 48 列开始。有关格式化的更多信息，请参阅20.1.5 节[Invoking install-info], 173 页中的‘`--calign`’、‘`--align`’和‘`--max-width`’选项。

如果在 Texinfo 源代码中多次使用`@dircategory`，则每次使用都会指定‘当前(current)’类别(category)；任何后续的`@direntry`命令都将添加到该类别中。

每个已安装的程序的‘Invoking(调用)...’节点都应有一个对应的`@direntry`。这样，用户就可以像使用传统的`man`系统一样，轻松找到他们可以运行的各个程序的文档。

20.1.5 调用install-info

`install-info`将 Info 文件中的菜单条目插入到 Info 系统中的顶级`dir` 文件中（有关`dir` 文件工作原理的说明，请参阅前面的部分）。`install-info`也会从`dir` 文件中移除菜单条目。它通常作为软件安装过程的一部分运行，或者在为系统上的所有手册构建`dir` 文件时运行。概要：

```
install-info [option(选项)...] [info-file(Info 文件)] [dir-file(目录文件)]
```

如果未指定 *info-file* 或 *dir-file*，则必须有可以定义它们的选项（如下所述）。没有编译时的默认值，且从不使用标准输入。每次调用 `install-info` 时，它只能读取一个 Info 文件，且只能写入一个目录文件。

如果 *dir-file*（无论以何种方式指定）不存在，`install-info` 会在可能的情况下创建它（且不包含任何条目）。

如果任何输入文件是使用 `gzip`（请参阅 *Gzip*）压缩的，`install-info` 会自动解压以供读取。如果 *dir-file* 是压缩的，`install-info` 在写入任何更改后也会自动保持其压缩状态。如果 *dir-file* 本身不存在，`install-info` 会尝试按以下顺序打开 *dir-file.gz*、*dir-file.xz*、*dir-file.bz2*、*dir-file.lz* 和 *dir-file.lzma*。

选项：

`--add-once`

指定：一个条目或多个条目将仅放入一个单独的节(section)。

`--align=column`

指定：菜单项的描述的第二行及后续行将从哪一列开始格式化。此选项的默认值为‘35’。它与‘`--max-width`’选项一起使用。*column* 从 1 开始计数。

`--append-new-sections`

不要将新节按字母顺序排列，而是将它们放在 DIR 文件的末尾。

`--calign=column`

指定：菜单项的描述的第一行将从哪一列开始格式化。此选项的默认值为‘33’。它与‘`--max-width`’选项一起使用。当菜单项的名称超过此列时，该菜单项的描述将从下一行开始。*column* 从 1 开始计数。

`--debug`

报告正在进行的工作。

`--delete`

`--remove` 从 *dir-file* 中删除 *info-file* 的条目。*dir-file* 中的条目中的文件名必须为 *info-file*（除非其中一个包含可选的‘.info’）。不插入任何新条目。删除后产生的任何空节也会被删除。

`--description=text`

指定：菜单项的解释部分。如果您未指定描述（无论是通过‘`--entry`’、‘`--item`’还是本选项），则描述将从 Info 文件本身获取。

`--dir-file=name`

指定 Info 目录文件的文件名。这等同于使用 *dir-file* 参数。

`--dry-run`

与‘`--test`’相同。

`--entry=text`

`--item=text`

将 *text* 作为 Info 目录条目插入；*text* 应采用 Info 菜单项行的格式，后接零个或多个以空格开头的额外行。如果您指定了多个条目，它们都会被添加。如果您未指定任何条目，目录条目根据 Info 文件本身中的信息来确定。Info 目录条目实际上是一个菜单项。

`--help`

显示一条包含基本使用方法和所有可用选项的使用说明信息，然后成功退出。

`--info-file=file`

指定要在目录文件中安装的 Info 文件。这等同于使用 *info-file* 参数。

`--info-dir=dir`
`--infodir=dir`
指定目录文件`dir`所在的目录。等效于‘`--dir-file=dir/dir`’。

`--keep-old`
不替换已有的菜单项。当指定了‘`--remove`’时，此选项表示不移除空节。

`--max-width=column`
`--maxwidth=column`
指定菜单项描述文字换行的列数。`column` 从 1 开始计数。

`--menuentry=text`
`--name=text`
指定菜单条目的名称部分。如果`text` 不以星号‘`*`’ 开头，则假定它是‘`*`’之后、指定 Info 文件的括号之前的文本。否则，`text` 将被直接采用，并被视为定义了第一个句点之前的文本（必要时会附加一个空格）。如果你没有指定名称（通过‘`--entry`’、‘`--item`’或本选项），则将从 Info 文件本身获取。如果 Info 不包含名称，则使用 Info 文件的基本名称。

`--no-indent`
禁止将新条目格式化到`dir` 文件中。

`--quiet`
`--silent` 关闭警告等提示，以实现静默运行。

`--remove-exactly`
同样类似于‘`--delete`’，但仅删除 Info 文件名完全匹配的条目；`.info`和/或`.gz`后缀不 被忽略。

`--section=sec`
将此文件的条目放入目录(文件)的`sec` 节中。如果您指定了多个节，则所有条目都会被添加到每个节中。如果您未指定任何节，则将根据 Info 文件本身的信息来确定条目所属的节。如果 Info 文件未指定节，则菜单条目将被放入‘Miscellaneous(杂项)’节。

`--section regex sec`
与‘`--regex=regex --section=sec --add-once`’相同。
`install-info`会尝试检测何时使用了这种替代语法，但并不总能正确猜测。以下是`install-info`所使用的启发式方法：

1. 如果`--section`的第二个参数以连字符开头，则假定使用原始语法。
2. 如果`--section`的第二个参数是一个可打开的文件，则假定使用原始语法。
3. 否则，将使用替代语法。

当启发式方法失败时，可能是因为你的节标题以连字符开头，或者它恰好是一个可以打开的文件，此时应将语法更改为‘`--regex=regex --section=sec --add-once`’。

`--regex=regex`
将此文件的条目放入与`regex` 匹配的任何节中。如果多个节匹配，则所有条目都将添加到每个匹配的节中。使用基本正则表达式语法指定`regex`，例如，大致与`grep`中使用的语法相同。

`--test` 禁止更新目录文件。

`--version`
 显示版本信息并成功退出。

20.2 标签文件和分割文件

Info 文件总是包含一个标签表(*tag table*)，以便能够快速跳转到节点。Info 文件可以是非分割的(*nonsplit, unsplit*) 或分割的(*split*)。

如果 Info 文件的字符数²少于 300,000(约)，则不分割该文件。在这种情况下，标签表应出现在 Info 文件的末尾。如果 Texinfo 文件包含的字符数超过约 300,000，Texinfo 处理器会将大型 Info 文件拆分为较短的间接(*indirect*) 子文件，每个子文件包含约 300,000 个字符。使用 `texi2any` 时，可以通过 `--no-split` 选项来阻止分割，并且可以使用 `--split-size` 选项（请参阅 19.1 节 [Invoking `texi2any`], 148 页）来修改默认的大小（300,000 个字符）。

当文件被分割时，Info 本身会变成原始(非分割的)文件的简化版本，该版本仅包含标签表以及对分割后文件的引用。这些分割后的文件被称为间接(*indirect*) 文件。

分割后的文件的名称是通过在输出文件名（由 `@setfilename` 命令或输入文件名指定）后附加 `-1`、`-2`、`-3` 等来创建的。原始文件的简化版本仍保留由 `@setfilename` 或输入文件名指定的名称。

例如，在撰写本文档的某个阶段，Info 文件被保存为文件 `test-texinfo`，该文件内容如下：

```
Info file: test-texinfo,    -*-Text-*-
produced by texinfo-format-buffer
from file: new-texinfo-manual.texinfo

^_
Indirect:
test-texinfo-1: 102
test-texinfo-2: 50422
test-texinfo-3: 101300
^_^L
Tag table:
(Indirect)
Node: overview^?104
Node: info file^?1271
Node: printed manual^?4853
Node: conventions^?6855
...
```

（实际上 `test-texinfo` 的节点数量远超此处所示。）每个拆分出来的间接文件，即 `test-texinfo-1`、`test-texinfo-2` 和 `test-texinfo-3`，都列在此文件中 `‘Indirect(间接):’` 这一行之后。标签表则列在 `‘Tag table(标签表):’` 这一行之后。

在间接文件列表中，文件名后的数字记录了前述间接文件的累积字节数，但不包括文件列表本身、标签表及第一个文件中的任何许可文本。在标签表中，节点名后的数字记录了节点开始的位置，即从（未拆分的）输出文件开头算起的字节数。

² 译者注：实际上是字节数，下同。

如果您正在使用`texinfo-format-buffer`来创建 Info 文件，您可能需要运行`Info-validate`命令。（`texi2any`命令本身就做得很好，您不需要`Info-validate`。）但是，您无法对间接文件运行`M-x Info-validate`节点检查命令。有关如何防止文件被`texinfo-format-buffer`拆分以及如何验证节点结构的信息，请参阅节 D.10.4.1 [Using Info-validate], 256 页。³

20.3 Info 格式常见问题

以下是在项目邮件列表及其他地方经常被问到的一些问题。

为什么当我运行‘`info foo`’时，得到的输出与运行‘`man foo`’得到的输出相同？

请检查是否已安装 Info 手册。并非所有 GNU/Linux 发行版都会默认安装所有 Info 手册。这令人遗憾，因为 Info 手册提供的信息往往比提供的 man 页面更多。

为什么不使用 HTML，而要用 Info 呢？

手册很少直接以 Info 格式编写，而是通过`texi2any`程序从 Texinfo 源文件生成。`texi2any`可以从 Texinfo 源文件生成 HTML 和 Info。您还可以从 GNU 网站 (<https://www.gnu.org/manual/manual.html>) 访问和下载 HTML 手册。此外，一些 GNU/Linux 发行版提供了安装 HTML 手册的软件包。

对于本地安装的文档，Info 与 HTML 相比，仍有一些优势。Info 浏览器支持索引查找，并支持在多个位置之间链接本地安装的手册。将文档本地安装在您的机器上，而不是依赖于互联网，这一点很重要；这使访问文档更加可靠，并确保它与已安装的软件包版本相匹配。希望未来能以某种 HTML 形式改进对本地安装的 Texinfo 文档的浏览支持。

既然 Emacs Info 阅读器已经足够优秀，为何还要提供命令行`info`程序呢？

Emacs Info 阅读器可以显示图像，并完全支持使用鼠标。除此之外，Info 阅读器与之没有太多差异。命令行`info`可以配置以更改默认键绑定，以及更改交叉引用和搜索结果的颜色，并启用鼠标滚动支持。使用`info`不需要具备 Emacs 编辑器的使用经验。（有人推荐使用另一个名为`pinfo`的程序，但该程序缺少索引查找等重要功能。）

有些人更倾向于能够一次性滚动浏览整本手册（就像手册页那样），而不是一次只限于查看一个内容‘节点’。当然，萝卜青菜，各有所爱，但随着手册变得更长、更复杂，单一的、非结构化的文本块会变得更加不便。（不过，如果你真的想这么做，你可以使用`less`分页器，通过‘`info foo | less`’一次性查看 Info 手册。）

为什么我的所有链接前面都出现了‘see’？

默认情况下，Emacs 的 Info 模式要么将交叉引用的标记‘`*note`’更改为‘see’，要么完全隐藏它。如果设置了`hide-note-references`，命令行`info`也会执行相同的操作。遗憾的是，无法可靠地做到这一点，因为 Texinfo 中的`@pxref`和`@ref`命令都会在 Info 输出中显示此标记，而‘see’文本仅适用于`@pxref`。

是的，但我怎么才能得到一个没有额外标记的普通链接呢？

你不能这么做。Info 是一种纯文本格式，在查看器中大多按原样显示，如果没有‘`*note`’文本，就无法将文本标记为链接。

对于 HTML 等输出格式，您可以使用`@link`命令来生成一个普通链接。请参阅 5.12 节[@link], 45 页。不过，这在 Info 输出或手册的打印副本中并不会生成有效的交叉引用。

³ 译者注：这一段，是在描述 Emacs 下的操作。

为什么包含链接的行会神秘地变短？

如上所述，这是由于 Emacs（或在`hide-note-references`设置为‘0n’的情况下使用`info`）隐藏了‘*note’字符串。

Info 格式能否扩展以支持字体、颜色或可重排文本？

任何扩展都会与现有的 Info 查看程序不兼容。Info 文件中添加的额外标记会显示给用户，使文件变得丑陋且难以阅读。

当 Texinfo 文件被处理成 Info 文件时，文本被何种 Texinfo 命令进行标记的信息会丢失。此外，无法可靠地检测文本是否可以重排（例如：一段散文），或者是否应保留换行符（例如：一个代码示例或一首诗）。

Info 的核心目的是在文本终端上显示文档。如果你需要更多功能，建议使用`texi2any`生成的 HTML 输出。

21 生成 HTML

`texi2any`默认情况下会生成 Info 输出，但若指定了`--html`选项，它将生成 HTML 格式，以供网页浏览器和其他程序使用。本章将详细介绍 HTML 输出。

`texi2any`提供了许多用户可定义的定制化变量，您可以通过这些变量来影响 HTML 输出。请参阅21.8节[HTML Output Customization], 189 页。特别是，它支持在`@example`中实现语法高亮显示(请参阅21.6节[Syntax Highlighting], 183 页)。您还可以编写所谓的初始化文件(*initialization files*) (简称`init`文件(*init files*))，以修改 HTML 输出的几乎所有方面。初始化文件包含代码，并通过`--init-file`加载(请参阅19.1节[Invoking `texi2any`], 148 页)。

一些初始化文件由 Texinfo 维护，并默认安装。例如，`chm.pm` 会生成中间的压缩 HTML 帮助格式文件，这个文件随后可以转换为 CHM 格式。

使用定制化文件对`texi2any` HTML 输出进行调整(*adaptation*)的说明文档在另一本手册中。请参阅GNU Texinfo `texi2any`输出定制化(*Output Customization*)。

21.1 转换成 HTML

`texi2any`生成的 HTML 输出是标准的 HTML 格式。为了实现最大的可移植性和可访问性，该输出特意保持了简洁明了。

您可以通过 CSS (请参阅21.3节[HTML CSS], 180 页)或其他方式定制化输出(请参阅21.8节[HTML Output Customization], 189 页)。如果您无法完成合理的定制化，请随时报告。

导航栏：默认情况下，每个节点的开头都会插入一个导航栏，与 Info 输出类似。如果使用了`--no-headers`选项，则导航栏仅在分割文件的开头插入。在分割输出中，`<link>`元素作为头部，以在支持此功能的浏览器中实现类似 Info 的导航。

原始 **HTML**：`texi2any`会在 HTML 输出中包含`@ifhtml`和`@end ifhtml`之间的 Texinfo 源码片段（但默认情况下，不会包含任何其他条件语句）。`@html`和`@end html`之间的源码会原样传递到输出中（即，不会对输入中的`<`、`>`和`&`字符进行常规转义，这些字符在 HTML 中具有特殊意义）。请参阅15.1节[Conditional Commands], 121 页。

标准：我们的目标并非（也并非总是能够）在没有任何诊断的情况下通过所有能想到的验证测试。不同的验证测试有不同的目标，往往是为了严格遵守某种标准。我们的首要目标是帮助用户，而非盲目地遵守标准。

请将`texi2any`无错误运行后产生的输出中存在的真实的 浏览器（EPUB 阅读器）兼容性问题作为错误进行报告（请参阅1.1节[Reporting Bugs], 2 页）。

实际上，`texi2any`生成的 HTML 会随着时间的推移而逐渐调整，以符合最新的 HTML 标准，同时也会努力保持与早期生成的 HTML 的兼容性。我们使用过渡性标记，并尽量放缓调整速度，以便给各种 HTML 阅读器留出调整时间（以及让标准重新纳入已弃用的有用特性...）。

21.2 HTML 分割

在默认设置(在节点处分割输出)下，`texi2any` (基本上) 会为每个 Texinfo 源代码的`@node`写入一个单独的 HTML 输出文件。

每个输出文件名都是节点名，其中空格被替换为`-`，特殊字符被更改为`_`后跟其十六进制代码点（请参阅21.7节[HTML Xref], 183 页）。这样做是为了使其作为文件名具有可移植性和易用性。在特殊情况下，如果两个不同节点在经过这种处理后具有相同的名称，它们将被连续写入同一个文件，并带有 HTML 锚点，以便每个节点都可以独立引用。

如果`texi2any`在文件名不区分大小写的系统上运行，那么除了大小写不同之外其他都相同的节点（例如，‘`index`’和‘`Index`’）也将被折叠到同一个输出文件中，并带有锚点。您还可以通过设置定制化变量`CASE_INSENSITIVE_FILENAMES`来模拟在区分大小写的文件系统中运行。

还可以使用`--split`选项（请参阅[Splitting Output], 153 页）按章或节进行拆分。在这种情况下，文件名是根据与相关章节命令相关联的节点的名称来构建的。此外，除非指定了`--no-node-files`选项，否则会为每个节点输出一个重定向文件，以便更可靠地实现对该手册的交叉引用（请参阅21.7 节[HTML Xref], 183 页）。

在分割时，HTML 输出文件会被写入一个子目录中。子目录名称由基本名（即：去除所有扩展名）派生而来，并附加`_html`。例如，`gcc.texi`的 HTML 输出将被写入名为‘`gcc_html/`’的子目录中。子目录名称基于`@setfilename`或输入文件名（请参阅2.5.2 节[`@setfilename`], 12 页）。

无论如何，目录中的顶级输出文件名总是为‘`index.html`’。

整体输出（`--no-split`）的命名：

1. 根据`@setfilename`命名。如果存在‘`.info`’扩展名，‘`.info`’替换为‘`.html`’。
2. 或者`--output`（该参数按字面意思使用）。
3. 或者作为最后手段：基于输入文件名命名（请参阅2.5.2 节[`@setfilename`], 12 页）。

21.3 HTML 层叠样式表(CSS)

层叠样式表(CSS)是影响 HTML 文档显示的一种互联网标准：参见<http://www.w3.org/Style/CSS/>。

默认情况下，HTML 会包含一些 CSS 代码，以更好地实现某些 Texinfo 环境的外观。例如：

```
pre.display { font-family:inherit }
```

上述代码告诉网页浏览器在用于`@display`环境的‘`<pre>`’元素内使用与主文档相同的字体。默认情况下，HTML 中‘`<pre>`’命令使用等宽字体。

请参阅上述参考资料，以获取关于 CSS 的完整说明。

您可以通过两个`texi2any`选项来影响 HTML 输出中的 CSS：`--css-include=file`和`--css-ref=url`。

选项`--css-ref=url`会为每个输出的 HTML 文件添加一个‘`<link>`’标签，该标签引用给定`url`处的 CSS。这样便可以使用外部样式表。

选项`--css-include=file`，如您所料，会在 HTML 输出中包含`file`的内容。然而，为了提供最大的灵活性，其具体实现方式有些复杂，具体如下所述。

CSS 文件的第一行可能是一个‘`@charset`’指令。如果存在，该指令用于确定 CSS 文件的编码。该行不会被复制到输出中。

CSS 文件可能以所谓的‘`@import`’指令开头，这些指令链接到外部 CSS 规范，供浏览器在解析文档时使用。同样，在此我们不对其进行全面描述，但会从语法上解释它们的工作原理，以便我们说明它们是如何被处理的。

可以有多个‘`@import`’，但它们必须位于文件的最前面，中间只能穿插空格和注释，不能有普通定义。CSS 文件中的注释以‘`/* ... */`’表示，与 C 语言中的用法相同。‘`@import`’指令必须采用以下两种形式之一：

```
@import url(http://example.org/foo.css);
```

```
@import "http://example.net/bar.css";
```

关键字符是位于指令开头的‘@’和终止指令的分号。在读取 CSS 文件时，任何此类‘@’指令(directive)都会被直接复制到输出中，如下所示：

- 如果 *file* 仅包含常规 CSS 声明，则它将被包含在默认 CSS 之后，从而覆盖默认 CSS。
- 如果 *file* 以‘@import’规范（见下文）开头，则首先包含‘import’（根据标准，它们必须放在前面），然后包含默认 CSS。如果你需要从‘@import’中覆盖默认 CSS，可以使用‘!important’CSS 构造，如下所示：

```
pre.example { font-size: inherit ! important }
```

- 如果 *file* 同时包含‘@import’和内联 CSS 规范，则首先包含‘@import’，然后是默认 CSS，最后是来自 *file* 的内联 CSS。
- 除‘@import’和‘@charset’之外的任何 @ 指令都将被视为 CSS 声明，这意味着默认的 CSS 将被包含，然后文件的其余部分将被前置。

如果 CSS 文件格式错误或有误，则输出结果不确定。CSS 文件的内容不会被以任何方式解释；特殊的‘@’和‘;’字符会被查找，文本会被原样复制到输出中。CSS 文件中的注释可能会也可能不会被包含在输出中。

默认情况下，类(class)会被添加到 HTML 元素中，而这些类的 CSS 规则会在每个 HTML 文件的开头输出。设置 `NO_CSS` 定制化变量可阻止添加类(class)和 CSS 的使用。设置 `INLINE_CSS_STYLE` 可将 CSS 代码放在 HTML 元素的 `style` 属性中，而不是放在输出的开头。

由于 `texi2any` 仅插入实际输出的 HTML 所需的 CSS 代码，因此在输出文档中无法看到可能插入的全部 CSS 代码列表。要获取完整的 CSS 规则列表，请在调用 `texi2any` 时设置 `SHOW_BUILTIN_CSS_RULES` 定制化变量，以便在标准输出上输出内置默认 CSS 规则并退出，例如：

```
texi2any -c SHOW_BUILTIN_CSS_RULES=1
```

21.4 @documentdescription: 摘要文本

在为文档生成 HTML 输出时，会在‘<head>’中写入一个‘<meta>’元素，以提供关于文档内容的一些信息。默认情况下，这个描述(*description*)是文档的标题，该标题取自 `@settitle` 命令（请参阅 2.5.3 节 `@settitle`，12 页）。若要更改此描述，请使用 `@documentdescription` 环境，如下所示：

```
@documentdescription
```

```
描述性文本。
```

```
@end documentdescription
```

这将在 HTML 的‘<head>’部分生成以下输出：

```
<meta name=description content="描述性文本。">
```

21.5 生成 EPUB 文件

EPUB 是一种专为在便携设备上阅读电子书而设计的格式。`texi2any` 在 2024 年生成了 EPUB 3.3 格式。EPUB 文件是一个 ZIP 存档容器，其中除了有以 HTML 格式呈现的手册，还有提供其它有用信息的文件。

EPUB 文件的生成过程依赖于名为 `Archive::Zip` 的 Perl 模块。此依赖关系会在运行时进行检查。默认情况下，若尝试在没有此依赖项的情况下输出 EPUB，则会报错。但是，如果

EPUB 输出文件未生成，且定制化变量 `EPUB_CREATE_CONTAINER_FILE` 设置为 0，则即使未安装 `Archive::Zip` 也不会报错。

与 EPUB 生成相关的 `texi2any` 测试不需要安装 `Archive::Zip`，因为它们将 `EPUB_CREATE_CONTAINER_FILE` 设置为 0，并通过将 `EPUB_KEEP_CONTAINER_folder` 定制化变量设置为 1 来保留包含 EPUB 文件所需文件和目录的目录。

21.5.1 容器目录和输出文件

输出 EPUB 格式时，会创建一个包含 EPUB 格式所需文件和目录的容器目录。该容器目录的名称来源于输入文件的基本名称（即去掉任何扩展名），并附加 `_epub_package`。如果使用 `--output` 选项或 `SUBDIR` 定制化变量指定了输出目录，则容器目录将在该输出目录中创建，而不是在当前目录中。每次新运行开始时，容器目录及其所有内容都会被删除。默认情况下，在生成最终 EPUB 文件后，容器目录也会被删除。

Texinfo 手册生成的 HTML 文件会输出到容器目录的子目录中。如果 Texinfo 手册中引用的图像文件存在，则会被复制到容器目录的子目录中。

EPUB 输出文件是容器目录的 ZIP 存档。文件名由基本名派生，并附加 `.epub` 扩展名。如果通过 `--output` 或 `OUTFILE` 定制化函数指定了输出文件，则使用该文件名。输出 EPUB 文件永远不会放置在 `--output` 或 `SUBDIR` 指定的目录中；如上所述，只有容器目录才会放置在那里。

如果定制化变量 `EPUB_CREATE_CONTAINER_FILE` 设置为 0，则不会生成 EPUB 输出文件。如果设置了 `EPUB_KEEP_CONTAINER_FOLDER`，则在生成最终的 EPUB 文件后，容器目录仍将保留。

请参阅 19.1 节 [Invoking `texi2any`], 148 页。

21.5.2 EPUB 交叉引用

EPUB 格式不支持从 EPUB 文件到另一个 EPUB 文件的引用。因此，对“外部”Texinfo 手册的任何引用都应解析为外部 URL。`texi2any` 使用 HTML 交叉引用配置（请参阅 21.7.5 节 [HTML Xref Configuration], 187 页）生成这些链接。如果交叉引用信息未找到，则生成的 EPUB 中的链接将不正确，因此 `texi2any` 默认会针对缺少的交叉引用信息发出警告。如果不希望出现这些警告，请将 `CHECK_HTMLXREF` 设置为 0。

21.5.3 确保完全符合 EPUB 标准

默认情况下，`texi2any` 会尽可能生成符合一致性检查器预期的 EPUB 格式。您可能仍需调整您的 Texinfo 手册，以实现完全符合：

- 请确保对外部手册的引用能够解析为网址（请参阅 21.5.2 节 [EPUB Cross-References], 182 页）。
- 避免使用比 XHTML 更宽松的 Texinfo 代码。一般来说，有效的 Texinfo 代码可以转换为有效的 XHTML，但也有例外。例如：
 - ** 应避免嵌套的引用。
 - ** 避免嵌套的章节命令及引用。
 - ** 避免在 `@table` 的第一个 `@item` 之前的内容。
 - ** 避免省略 `@table` 的 `@item` 的第二列。
- 删除 Texinfo 输入文件名中的空格。
- 移除输出中出现的换页符。

21.5.4 为 EPUB 生成 HTML

为 EPUB 生成的 HTML 符合 XHTML 标准，采用 UTF-8 编码，并且在格式化时没有常见的 HTML 导航页眉和页脚。

`texi2any`通过设置定制化变量（如`USE_XML_SYNTAX`、`HEADERS`或`OUTPUT_ENCODING_NAME`）在内部实现这一功能。EPUB 中使用了打印输出的一些特性。特别是，在生成标题页时，顶部节点不会出现在 EPUB 输出中。这是通过设置`NO_TOP_NODE_OUTPUT`来实现的。请参阅21.8 节[HTML Output Customization], 189 页。

最初，`OUTFILE`和`SUBDIR`定制化变量值对应于 EPUB 目录容器和/或 EPUB 输出文件（请参阅21.5.1 节[EPUB Output File and Directory], 182 页）。如果这些定制化变量值被取消定义或复位，则指向到容器目录中为 HTML 生成而输出的 XHTML 文件的所在位置。

这里提及`texi2any`对定制化变量的这种使用，是因为重置此类变量并不常见；然而，重置的变量仅在转换过程中内部使用，不应与用户设置的任何定制化变量发生交互。

21.6 HTML 中代码示例的语法高亮

警告

源代码高亮功能尚处于实验阶段；欢迎提供反馈。在未来的版本中，`HIGHLIGHT_SYNTAX`的语法很可能会发生变化。

在外部软件的帮助下，`texi2any`为 HTML 输出提供了对源代码语法高亮的支持。通过设置`HIGHLIGHT_SYNTAX`可启用此功能。源代码高亮是为`@example`块设置的。为语法高亮指定的语言是`@example`行上的第一个参数（请参阅7.4 节[`@example`], 60 页），如果没有第一个参数，则使用在`HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE`中设置的缺省语言。

`HIGHLIGHT_SYNTAX`值决定了用于高亮显示的命令：

`highlight`

使用`highlight`，请参阅<http://www.andre-simon.de/doku/highlight/en/highlight.php>；

`pygments` 使用`pygmentize`，请参阅<https://pygments.org/>；

`source-highlight`

使用`source-highlight`，请参阅`GNU Source-highlight`。

21.7 HTML 交叉引用

Texinfo 手册之间的交叉引用在 HTML 格式中变为标准的 HTML 的`<a>`链接。本节描述了所使用的算法，以便 Texinfo 能与其他程序（如`texi2html`）协作，编写相互兼容的 HTML 文件。

对于 Texinfo 文件的 HTML 输出中的链接，此算法可能适用，也可能不适用。由于在这种情况下不会出现兼容性问题，因此我们无需对此进行具体说明。

我们尝试在整体和分割的 HTML 输出中都支持对这类“外部”手册的引用。整体(*monolithic*)手册完全包含在一个文件中，而分割手册则为每个节点提供一个文件。（请参阅21.2 节[HTML Splitting], 179 页。）

该算法主要由帕特里斯·杜马斯(Patrice Dumas)于 2003 至 2004 年间设计。

21.7.1 HTML 交叉引用链接基础

就我们的目的而言，一个 HTML 链接由四个部分组成：主机、目录部分、文件部分和目标部分。我们始终假定使用http协议。例如：

`http://host(主机)/dir(目录)/file(文件).html#target(目标)`

构建链接的信息来源于 Texinfo 源代码中的交叉引用命令（请参阅第 5 章[Cross References], 39 页）中的节点名称和手册名称，以及外部信息(*external information*)（请参阅21.7.5 节[HTML Xref Configuration], 187 页）。

在万维网上发布的 HTML 手册的常见情况下，为了实现有效的交叉引用，需要外部信息来定义 *host* 和 *dir* 的值。具体细节将在后续内容中详细说明（请参阅21.7.5 节[HTML Xref Configuration], 187 页）。

如果手册没有可用的外部信息，那么对于 *host* 和 *dir* 将使用默认值，这些默认值对应于（位于单个主机上并排放置的）手册集中的手册的交叉引用链接。在这种情况下，*host* 被硬编码为本地主机。根据 HTML 链接的规则，这可以是字面字符串‘localhost’，也可以完全省略，并同时省略‘http://’。

dir 和 *file* 部分既取决于正在处理的手册的整体/分割特性，也取决于交叉引用所指向的手册：

- 如果当前手册被分割，且所指(*referent*)手册也被分割，则目录为‘../referent_html/’，文件为展开的节点名称（稍后描述）。
- 如果当前手册是分割的，而所指(*referent*)手册是整体的，则目录为‘../’，文件为 *referent.html*。
- 如果当前手册是整体的，而所指(*referent*)手册是分割的，则目录为 *referent_html/*，文件为展开的节点名称。
- 如果当前手册是整体的，且所指(*referent*)手册也是整体的，则目录为 *./*（或仅为空字符串），文件为 *referent.html*。

基本思路是，有一个统一的目录用于 HTML 格式的 Texinfo 手册，给定的 *manual* 要么以一个整体文件 *manual.html* 的形式存在，要么以一个分割的子目录 *manual_html/*.html* 的形式存在。

另一个仅适用于文件名的规则是：基本文件名会被截断为 245 个字符，以便在附加扩展名后仍能符合许多文件系统常见的 255 个字符的限制。虽然从技术上讲，这可以通过 `BASEFILENAME_LENGTH` 定制化变量（请参阅21.8.2 节[File Names and Links Customization for HTML], 191 页）来更改，但这样做会使对这些节点的跨手册引用失效。

源交叉引用命令的文件名参数中的任何目录部分都会被忽略。因此，`@xref{,,,../foo}`和`@xref{,,,foo}`都将‘foo’用作手册名称。这是因为，对于所有使用该手册名称的输出格式而言，任何此类对目录进行硬编码的尝试都不太可能有用。

最后，*target* 部分始终是展开后的节点名称。

本手册是以分割形式还是整体形式输出，取决于您运行 `texi2any` 的方式；默认情况下为分割，但 `--no-split` 选项可覆盖此设置。

默认情况下，`texi2any` 使用与当前手册相同的形式生成指向引用手册的链接：如果当前手册是分割形式生成的，则默认情况下，指向其他手册的链接将使用分割形式；整体输出时也是如此。

然而，通常应在配置文件(请参阅 21.7.5 节[HTML Xref Configuration], 187 页)中所指(*referent*)手册的形式（整体或分割）。这可以确保生成软件所假设的引用手册格式与其实际呈现的格式之间不会出现不匹配的情况。

21.7.2 HTML 交叉引用节点名扩展

如前一节所述, HTML 交叉引用算法的关键部分是将 Texinfo 源代码中的节点名称转换为适合 XHTML 标识符和文件名的字符串。两者的限制相似: 只能使用纯 ASCII 字母、数字以及‘-’和‘_’字符。(尽管 HTML 锚点可以包含大多数字符, 但 XHTML 的限制更多。)

Texinfo 中的交叉引用可以指向节点、锚点(请参阅5.11 节[@anchor], 44 页)或浮动标签(请参阅9.1.1 节[@float], 74 页)。然而, 在此上下文中, 锚点和浮动标签的处理方式与节点相同, 因此为简化起见, 我们继续使用“节点”一词。

一个特殊例外: 顶层节点(请参阅2.10 节[The Top Node], 20 页)始终映射到文件index.html, 以匹配 Web 服务器软件。但 HTML 的target 仍是‘Top’。因此(在拆分的情况下):

```
@xref{Top,,, emacs, The GNU Emacs Manual}.
⇒ <a href="../../emacs_html/index.html#Top">
```

1. 标准 ASCII 字母(a-z 和 A-Z)保持不变。所有其他字符可根据以下规定进行更改。
2. 标准 ASCII 数字(0-9)在节点名称中不进行修改, 除非该数字是节点名称的第一个字符。若出现此种情况, 请见下文。
3. 多个连续的空格、制表符和换行符会被统一转换为一个空格。
4. 删除前导空格和尾随空格。
5. 应用上述规则后, 每个剩余的空格字符将被转换为‘-’字符。
6. 其他 ASCII 的 7 位字符被转换为‘_00xx’, 其中xx是(小写)十六进制中的 ASCII 字符码。这包括‘_’, 它被映射为‘_005f’。
7. 如果节点名称不是以字母开头, 则会在结果前加上字面字符串‘g_t’。(根据上述规则, 该字符串在其他情况下永远不会出现; 这是一个随意的选择, 代表“GNU Texinfo”。)这是必要的, 因为 XHTML 要求标识符必须以字母开头。

示例:

```
@node A node --- with _'%
⇒ A-node-_002d_002d_002d-with-_005f_0027_0025
```

常见字符的翻译示例:

- ‘_’ ⇒ ‘_005f’
- ‘-’ ⇒ ‘_002d’
- ‘A node’ ⇒ ‘A-node’

在大小写折叠(即: 将字母统一转换为小写或大写)的计算机系统中, 仅有大小写不同(而其它相同)的节点将被映射到同一文件。特别是如上所述, Top 始终映射到文件index.html。因此, 在大小写折叠的系统中, Top 和一个名为‘Index’的节点都将被写入index.html。幸运的是, 目标(targets)用于区分这些情况, 因为 HTML 目标(targets)名称始终区分大小写, 与操作系统无关。

21.7.3 HTML 交叉引用命令扩展

节点名称中可能包含 @ 命令(请参阅3.3 节[Node Line Requirements], 25 页)。本节将介绍如何处理这些命令。

首先, 注释被移除。

接下来, 所有@value命令(请参阅15.5.1 节[@set @value], 125 页)和宏调用(请参阅16.2 节[Invoking Macros], 131 页)都会被完全展开。

然后，对于以下命令，将命令名称和括号移除，并对参数文本进行递归转换：

```
@asis @b @cite @code @command @dfn @dmn @dotless
@emph @env @file @i @indicateurl @kbd @key
@samp @sansserif @sc @slanted @strong @sub @sup
@t @U @var @verb @w
```

此外，以下命令将被替换为固定文本，如下所示。如果这些命令中的任何一个带有非空参数，如`@TeX{bad}`，则会出现错误，且结果未定义。在本表中，‘(space)’表示空格字符，‘(nothing)’表示空字符串。符号‘U+hhhh’表示 Unicode 码位`hhhh`（通常为十六进制）。

根据其他规则，许多这样的扩展名还会进行进一步的转换，以生成最终的文件名或其他目标名称，例如将空格字符替换为‘-’等。

@(newline)	(space)
@(space)	(space)
@(tab)	(space)
@!	‘!’
@*	(space)
@-	(nothing)
@.	‘.’
@:	(nothing)
@?	‘?’
@@	‘@’
@{	‘{’
@}	‘}’
@LaTeX	‘LaTeX’
@TeX	‘TeX’
@arrow	U+2192
@bullet	U+2022
@comma	‘,’
@copyright	U+00A9
@dots	U+2026
@enddots	‘...’
@equiv	U+2261
@error	‘error-->’
@euro	U+20AC
@exclamdown	U+00A1
@expansion	U+21A6
@geq	U+2265
@leq	U+2264
@minus	U+2212
@ordf	U+00AA
@ordm	U+00BA
@point	U+22C6
@pounds	U+00A3
@print	U+22A3
@questiondown	U+00BF
@registered symbol	U+00AE
@result	U+21D2

@textdegree	U+00B0
@tie	(space)

引号 @ 命令 (如@quotedblright{ }) 同样会被替换为其 Unicode 值。普通的引号字符 (例如, ASCII ‘ 和 ’) 则不会改变。请参阅 11.5 节 [Inserting Quotation Marks], 92 页。

任何 @acronym、@abbr、@email 和 @image 命令都会被它们的第一个参数替换。(对于这些命令, 所有后续参数都是可选的, 在此将被忽略。) 请参阅 6.1.14 节 [@acronym], 54 页、6.1.16 节 [@email], 55 页, 和 9.2 节 [Images], 76 页。

重音的处理将在下一节中介绍。

任何其他命令都是错误的, 其结果未定义。

21.7.4 HTML 交叉引用中 8 位字符的扩展

通常, 除普通 7 位 ASCII 以外的字符会被转换为 Unicode 规范化形式 C 中的相应码位 (如果可用, 则使用预组合字符)。(这是 W3C 和其他机构推荐的规范化形式。) 当该码位为 0xffff 或更低时, 情况也是如此, 因为这几乎总是如此。

然后, 这些字符将根据上述规则进一步转换为字符串 ‘_hhhh’, 其中 hhhh 是十六进制码点。

例如, 将此规则与上一节内容相结合:

```
@node @b{A} @TeX{ } @u{B} @point{ }@enddots{ }
⇒ A-TeX-B_0306-_22C6_002e_002e_002e
```

注意: 1) @enddots 展开为三个句点, 而三个句点又展开为三个 ‘_002e’; 2) @u{B} 是一个带有短音符的 ‘B’, 该字符在预重音符的 Unicode 字符中不存在, 因此展开为 ‘B_0306’ (组合短音符的 ‘B’)。

当 Unicode 码位高于 0xffff 时, 转换形式为 ‘_xxxxxx’, 即两个前导下划线后跟六个十六进制数字。由于 Unicode 已声明其最高码点为 0x10ffff, 因此这已经足够了。(我们认为, 定义这种额外的转义形式比总是使用六个十六进制数字更好, 因为前两个数字几乎总是零。)

如果节点名称主要由 ASCII 字符组成, 且只包含少量 8 位字符, 那么这种方法效果很好。但如果文档是用非拉丁字母语言 (如乌克兰语) 编写的, 那么生成的文件名几乎全部由 ‘_xxxx’ 符号组成, 这不仅不方便, 而且几乎无法阅读。为了处理这种情况, texi2any 提供了 --transliterate-file-names 命令行选项。此选项允许在创建和引用文件名时, 将节点名音译 (transliteration) 为 ASCII 字符。音译基于语音学原理, 使生成的文件名更易于理解。

关于 Unicode 规范化形式 C 的定义, 请参见 Unicode 报告 UAX#15, <http://www.unicode.org/reports/tr15/>。网络上其他地方也有许多相关文档和实现。

21.7.5 HTML 交叉引用配置: htmlxref.cnf

为了让发布在万维网上的其他手册能够正确引用您的手册, 当 Texinfo 转换器为这些其他手册生成 HTML 时, 需要知道交叉引用中的 host 和 dir 部分。同样, 为了让 HTML 格式的手册能够正确引用其他手册, texi2any 需要知道如何为其他手册构建 URL。

texi2any 通过读取名为 htmlxref.cnf 的文件来收集信息, 以便在 HTML 输出中实现与其他手册的交叉引用。根据 XDG 基本目录规范 (XDG Base Directory Specification) (<https://specifications.freedesktop.org/basedir-spec/latest/>), 该文件会在以下目录中查找, 默认目录基于安装目录:

```
./ (当前目录)
```


输入文件的路径目录

如果存在一个路径目录且该目录不是当前目录。例如，如果输入文件是 `some/dir/mymanual.texi`，则会在 `some/dir` 中搜索 `htmlxref.cnf`。

`./.texinfo/`

(在当前目录之下)

`XDG_CONFIG_HOME/texinfo`

(其中 `XD_CONFIG_HOME` 是一个环境变量)

`~/.config/texinfo/`

(其中，`~` 表示当前用户的主目录，仅当未设置 `XD_CONFIG_HOME` 时适用)

`sysconfdir/xdg/texinfo/`

(其中，`sysconfdir` 是在编译时指定的系统配置目录，例如，`/usr/local/etc`)

`XD_CONFIG_DIRS/texinfo`

(针对以：分隔的 `XD_CONFIG_DIRS` 环境变量中的每个目录)

`datadir/texinfo/`

(在编译时指定，例如，`/usr/local/share`)

所有找到的文件都会被使用，且较前的条目会覆盖较后的条目。Texinfo 发行版包含一个默认文件，该文件处理许多 GNU 手册；它被安装在 `datadir/texinfo/htmlxref.cnf` 中，这是上述目录中的最后一个。

您可以使用本地的 `htmlxref.cnf` 来补充或更正 Texinfo 分发版中的默认文件。这有助于更正过时的数据，甚至可以为某些手册生成本地文件的交叉引用。

`HTMLXREF_MODE` 定制化变量可用于设置文件查找方式。例如，如果设置为 `'none'`，则不使用任何外部信息。`HTMLXREF_FILE` 将文件名设置为除 `htmlxref.cnf` 以外的其他名称。请参阅 [HTML Xref Configuration Customization], 192 页。

该文件是面向行的。仅由空格组成的行将被忽略。注释以行首的 `#` 表示，该符号前可以有可选的空格。由于 `#` 可能出现在 URL 中（就像几乎任何字符一样），因此它不会在其他情况下开始注释。

每行非空非注释行必须是变量赋值 (*variable assignment*) 行或手册信息 (*manual information*) 行。

变量赋值行看来像这样：

```
varname = varvalue
```

`'='` 符号前后的空格是可选的，并且会被忽略。`varname` 应由字母组成；区分大小写。`varvalue` 是一个任意字符串，可延续至行尾。变量随后用 `'${varname}'` 引用；变量引用可以出现在 `varvalue` 中。

手册信息行看起来像这样：

```
manual keyword urlprefix
```

`manual` 是手册的简短标识符，`keyword` 是以下之一：`mono`、`node`、`section`、`chapter`，`urlprefix` 在下文描述。变量引用仅可在 `urlprefix` 中出现。例如（用在典型的 `htmlxref.cnf` 中）：

```
G = http://www.gnu.org
GS = ${G}/software
hello mono    ${GS}/hello/manual/hello.html
```

```

hello chapter ${GS}/hello/manual/html_chapter/
hello section ${GS}/hello/manual/html_section/
hello node    ${GS}/hello/manual/html_node/

```

如果`keyword` 是`mono`，则`urlprefix` 会给出`manual` 的主机、目录和文件名，作为一个整体文件。

如果`keyword` 为`node`、`section`或`chapter`，则`urlprefix` 会给出主机和目录，分别用于`manual` 拆分出来的节点、章或节。

当`texi2any`生成指向另一本 Texinfo 手册的链接时，它会优先使用与当前输出具有相同分割选项的手册的行。也就是说，当生成整体输出(`--no-split`)时，会优先使用`mono` URL；当生成按节点拆分的输出时，会优先使用`node` URL，依此类推。

如果目标手册（在`htmlxref.cnf` 中）没有与首选分割选项相匹配的信息行，则可以使用该手册中的其他信息行（如果有的话）。以下是每个首选分割选项的完整搜索顺序：

```

node    ⇒ node,    section, chapter, mono
section ⇒ section, chapter, node,    mono
chapter ⇒ chapter, section, node,    mono
mono    ⇒ mono,    chapter, section, node

```

这些节级和章级的跨手册引用只有在目标手册是使用`--node-files` 创建时才能成功；这是分割输出的默认设置。

目前，随 GNU Texinfo 分发的`htmlxref.cnf` 文件是万维网中查找 Texinfo HTML 手册的主要资源。由于该文件安装在`texi2any`使用的位置，因此默认情况下，在该文件中找到的 HTML 手册信息将用于交叉引用。

如果您对随 Texinfo 分发的`htmlxref.cnf` 文件有任何补充或更正，请照常发送电子邮件至`bug-texinfo@gnu.org`。如果您在万维网上发布 Texinfo HTML 手册，已在`htmlxref.cnf` 中列出最新位置，应确保所有由`texi2any`生成的 HTML 手册自动能使用此位置进行交叉引用。您可以从<http://ftpmirror.gnu.org/texinfo/htmlxref.cnf>获取最新版本。

21.8 定制 HTML 输出

您可以使用许多用户可设置的定制化变量来影响 HTML 输出。请参阅21.8.9 节[HTML Customization Variables List], 198 页以获取完整列表。定制化变量提供的定制范围，从整体文档结构和 HTML 语言变体到特定结构的格式化，应能满足大多数仅通过定制化 CSS（请参阅21.3 节[HTML CSS], 180 页）无法实现的定制需求。

如果你需要更多控制权，或许可以通过使用初始化文件来实现，具体说明见另一本手册（请参阅GNU Texinfo *texi2any Output Customization*）。

如果设置了`TEXI2HTML`定制化变量，则生成的 HTML 将尽可能与`texi2html`（请参阅19.7 节[`texi2html`], 169 页）兼容。此变量的效果相当不寻常，因为它并不直接修改输出，而是设置了许多其他定制化变量，包括一些可能仅能在初始化文件中设置的变量，这些变量在关于`texi2any`输出定制化的一个独立手册中进行描述。在默认情况下，此变量未设置。

21.8.1 定制 HTML 输出结构

您可以使用定制化变量来控制文档标题、目录或其他类似部分的输出。还有一些变量会影响`texi2any`如何看待文档结构以及如何将输出拆分为多个输出文件。

如果`SHOW_TITLE`为`'undef'`（默认值），则输出开始时不输出标题。如果您设置了此变量，则缺省情况下，标题将使用完整的`@titlepage`；若要使用简单的标题字符串，请取消`USE_TITLEPAGE_FOR_TITLE`设置。

默认情况下, HTML 输出是为在线查看而定制的, 其中‘Top’节点被输出, 作为顶级起始点提供给读者。您可以设置`NO_TOP_NODE_OUTPUT`来移除‘Top’节点, 这样输出的格式可能更接近印刷书籍的格式。默认情况下, 设置`NO_TOP_NODE_OUTPUT`还会在文档开头输出一个标题, 但您可以通过将`SHOW_TITLE`设置为‘0’来覆盖此设置。

在每个节点中, 默认情况下会输出一个用于导航的下级章节的列表, 这实际上就是把`FORMAT_MENU`变量设置为‘`sectiontoc`’。若要在文档中使用菜单进行导航(如 Info 输出格式那样, 请参阅3.9 节[Menus], 30 页), 请将`FORMAT_MENU`设置为‘`menu`’。如果您不希望输出用于导航的菜单或下级章节列表, 请将`FORMAT_MENU`设置为‘`nomenu`’。

默认情况下, 整个文档的目录会在`@top`部分末尾输出, 以便在文档的前部就提供主要的位置导航, 这实际上就是把`CONTENTS_OUTPUT_LOCATION`变量设置为‘`after_top`’。

您可以通过将`CONTENTS_OUTPUT_LOCATION`设置为‘`after_title`’, 使目录在标题之后输出(如果设置了`SHOW_TITLE`)。若要目录在文档末尾输出或将其输出到单独的文件中(如果输出被拆分), 请将`CONTENTS_OUTPUT_LOCATION`设置为‘`separate_element`’。您还可以将`CONTENTS_OUTPUT_LOCATION`设置为‘`inline`’: 在这种情况下, 目录会在相应的 `@` 命令(例如`@contents`)所在的位置输出。

当`FORMAT_MENU`设置为‘`menu`’时, 为了使用菜单进行导航, 您可能会发现设置`CONTENTS_OUTPUT_LOCATION`很有用。在这种情况下, `@top`部分中应该已经有一个主控菜单(请参阅2.10.1 节[Master Menu Parts], 20 页), 因此手册的读者可能不需要在同一位置显示目录。

默认情况下, 每个节点都会输出到一个单独的文件中。您可以通过`--split` 命令行选项来更改此设置, 该选项等同于设置`SPLIT`变量。`SPLIT`可以取值为‘`chapter`’、‘`section`’或‘`node`’, 或者设置为空字符串以指示非分割输出。请参阅 [Splitting Output], 153 页。

基本单元称为输出单元(*output units*), 用于输出, 每个输出文件根据`SPLIT`的值, 包含一个或多个输出单元的内容。输出单元包含与节点或章节命令相关联的内容。在典型的 Texinfo 文档源文件中, 节点命令几乎总是与紧随其后的章节命令相关联。

定制化变量`USE_NODES`决定是将节点命令还是章节命令视为主要命令。默认情况下, 节点命令被用作分割输出单元的主要命令。您可以将`USE_NODES`设置为 `false`, 以优先使用章节命令来决定在何处分割输出单元。当章节是输出单元的主要组件时, 与章节命令无直接关联的‘孤立’节点, 将与下一个章节命令相关联, 而未关联系节点的章节命令则构成输出单元。相反, 当节点是输出单元的主要组件时, 与节点无关联的‘孤立’章节将与上一个节点相关联, 而‘孤立’节点则构成输出单元。

默认情况下, `USE_NODES`设置也会影响节点标题中的‘Next(下一个)’、‘Previous(上一个)’和‘Up(上一级)’链接是指向节点还是章节, 如果为 `true`, 则使用节点, 如果为 `false`, 则使用章节。您可以使用`USE_NODE_DIRECTIONS`变量(默认值未定义)来覆盖此设置。请注意, 此设置并不确定链接字符串, 而只是确定链接点; 有关链接字符串的定制, 请参阅5.5 节[xrefautomaticsectiontitle], 41 页。如果节点和章节是全系统关联的, 则此定制没有实际效果。

您可以通过设置`DO_ABOUT`来添加一个特殊的`About(关于)` 元素, 用于解释如何导航, 默认情况下该值设为 0 (无 About 元素)。如果还设置了`PROGRAM_NAME_IN_ABOUT`, 则会在 About 特殊元素中输出程序名称和其他相关信息; 默认为 `false`。

在整体输出的情况下, 默认情况下只输出一个文件, 其中包含任何 About 元素(带有`DO_ABOUT`)和目录元素(`CONTENTS_OUTPUT_LOCATION`设置为‘`separate_element`’)。如果您希望这些特殊元素单独输出, 请将`MONOLITHIC`设置为 0, 以便将特殊元素输出到单独的文件中。

在 Texinfo 代码中, `@node`命令后面通常会有一个章节命令, 为节点提供标题。默认情况下, 如果一个节点后面没有章节命令, 但跟了一个像`@heading`这样的命令, 那么它

的标题将由`@heading`这样的命令提供。这样可以避免同时使用节点名称和`@heading`这样的命令（或章节命令）作为标题。在这种情况下，如果希望将节点名称用作标题，请将`USE_NEXT_HEADING_FOR_LONE_NODE`设置为 0。

21.8.2 HTML 文件名及链接的定制

可以用定制化变量来控制输出文件名，以及定制输出文件中的超链接输出。

您可以使用比命令行选项`--output`更精细的方式来指定输出文件名（请参阅19.1节[Invoking `texi2any`], 148 页）。`PREFIX`定制化变量会覆盖由`@setfilename`或（输入）文件名确定的（输出）文件基本名，并且不包含任何目录部分。要更改中间的目录，请使用`SUBDIR`定制化变量。最后，您还可以使用定制化变量`EXTENSION`来覆盖扩展名。如果不添加扩展名，则该变量应为`undef`。如果您设置了`CASE_INSENSITIVE_FILENAMES`，则（输出）文件名将按照文件系统不区分大小写的方式构建。在这种情况下，对于仅大小写不同的所有文件，将使用同一个文件名。

此外，定制化变量`TOP_FILE`会覆盖顶层元素的输出文件名。通常，只有在输出被分割时才会考虑这一点。

如果设置了`USE_ACCESSKEY`（默认），则使用`accesskey`属性为某些导航超链接提供键盘快捷键。`n`用于链接到下一个节点或章节，`p`用于链接到上一个节点或章节，`u`用于上一级方向的链接。同样，如果设置了`USE_REL_REV`定制化变量（默认），则在导航中使用`rel`属性来定义当前页面与链接资源之间的关系。例如，为指向目录的链接设置`'contents'` `rel`属性，为指向下一个节点或章节的链接设置`'next'` `rel`属性。

以下是特定类型链接特有的定制化变量：

交叉引用命令链接 (*Cross-reference command links*)

如果设置了交叉引用命令的第二个参数，则该参数将显示为超链接文本（请参阅5.4节[Two Arguments], 40 页）。否则，将使用`XREF_USE_NODE_NAME_ARG`定制化变量来确定超链接文本。如果设置为 1，则在交叉引用 `@` 命令中，使用节点名称（第一个）参数作为显示的超链接文本。如果设置为 0，则：如果设置了`USE_NODES`，则使用节点名称，否则使用章节名称。如果设置为`'undef'`，则在预格式化环境中使用第一个参数，否则根据`USE_NODES`使用节点名称或章节名称。默认值为`'undef'`。

浮动元素链接 (*Links to floats*)

如果设置了`XREF_USE_FLOAT_LABEL`（默认为关闭），则将使用浮动标签，而不是类型后跟浮动号的形式（请参阅9.1.1节[`@float`], 74 页）。

指向索引条目所属节点或章节的链接 (*Links to nodes or sections for index entries*)

在`@printindex`格式化中，会输出两个链接：一个链接到索引条目，另一个链接到包含该索引条目的根级 `@` 命令。`NODE_NAME_IN_INDEX`定制化变量指定了链接到根级 `@` 命令时是使用节点名还是章节名。如果为 `true`，则索引条目使用节点名，否则使用章节名。如果未定义，则使用`USE_NODES`值来确定首选的根级 `@` 命令。默认值为未定义。

菜单链接 (*Menu links*)

默认情况下，节点名在菜单项链接中使用。将`NODE_NAME_IN_MENU`设置为 `false`，则使用章节名称。

目录链接(*Table of contents links*)

如果同时存在主目录和简要目录, 在默认情况下, 简要目录中的交叉引用会链接到对应的主目录的条目。将`SHORT_TOC_LINK_TO_TOC`设置为 0, 则改为链接到章节命令。

如果设置了`TOC_LINKS`, 则会创建从标题到目录条目的链接; 默认为 `false`。

顶部节点的上一级方向链接(*Top node Up direction link*)

默认情况下, 不会为顶节点生成向上一级的引用。您可以设置`IGNORE_REF_TO_TOP_NODE_UP`, 以便在其他交叉引用中出现的对顶节点的上一级节点的任何引用(默认情况下为`'(dir)'`)也被忽略。默认情况下, `IGNORE_REF_TO_TOP_NODE_UP`未设置。(请参阅[`TOP_NODE_UP`], 165 页。)

如果该手册与其他手册在网页上一同被引用, 则手册可能需要链接到该网页。将`TOP_NODE_UP_URL`设置为用于顶节点向上一级引用的 URL。如果设置了`TOP_NODE_UP_URL`, 则`TOP_NODE_UP`的值将用于超链接文本, 默认值为`'(dir)'`。`TOP_NODE_UP`可在属性中使用, 因此不应包含任何元素。

例如, 在 GNU 项目中, <http://www.gnu.org/manual/>收集了大多数 GNU 手册的链接; 因此, 如果手册将被安装在`www.gnu.org`上, 则最好将`TOP_NODE_UP_URL`指定为`/manual/`。

关于顶节点指针的更多信息, 请参阅3.4 节[`First Node`], 26 页。

images links

如果设置了`IMAGE_LINK_PREFIX`, 则相关值会作为前缀添加到图像文件链接中; 默认情况下未设置。如果图像文件位于特定的子目录中, 这可能会很有用。

HTML 手册之间的交叉引用被精确指定(请参阅21.7 节[`HTML Xref`], 183 页)。手册位置的定制化通过`htmlxref.cnf` 文件(请参阅 21.7.5 节[`HTML Xref Configuration`], 187 页)提供。

对于未能通过 HTML 外部交叉引用配置(`HTML Xref Configuration`)(请参阅21.7.1 节[`HTML Xref Link Basics`], 184 页)找到的交叉引用目标手册, 将使用默认目录和文件名。默认情况下, 对于不在 HTML 外部参照配置(`HTML Xref Configuration`)文件中的每个外部手册, 都会给出警告。如果您不希望收到关于此类手册的警告, 则应将`CHECK_HTMLXREF`设置为`'0'`。例如, 如果您知道手册已本地安装, 则此设置可能适用。

您可以使用定制化变量来更精确地指定 HTML 外部交叉引用配置。默认情况下, 会在目录中搜索用于 HTML 外部交叉引用配置的文件名, 并使用找到的所有文件。您可以设置`HTMLXREF_MODE`定制化变量来修改交叉引用其他手册信息的方式。如果`HTMLXREF_MODE`设置为`'file'`, 则直接使用文件名作为信息来源。如果`HTMLXREF_MODE`设置为`'none'`, 则不使用任何信息。默认情况下, 如果未定义`HTMLXREF_MODE`或将其设置为任何其他值, 则使用默认值`htmlxref.cnf`。`HTMLXREF_FILE`定制化变量将用于 HTML 输出的外部交叉引用配置文件设置为默认值`htmlxref.cnf`以外的其他值。如果`HTMLXREF_FILE`包含目录, 则会在找到时加载该文件, 但不会在目录中搜索。默认情况下, 根据输出手册的拆分情况, 将远程手册视为分割手册或整体手册。您也可以通过设置`EXTERNAL_CROSSREF_SPLIT`来明确设置这一点。

此外, 还有一些定制化变量可以修改交叉引用的其他方面:

- 您可以通过设置`TOP_NODE_FILE_TARGET`定制化变量值来设置交叉引用中顶节点所使用的文件名; 默认值为`index.html`。
- 您可以使用`EXTERNAL_DIR`为外部手册指定一个基目录; 在默认情况下, 没有指定任何基目录。

- 同样，您可以使用`EXTERNAL_CROSSREF_EXTENSION`设置交叉引用到其他手册的文件扩展名，如果未设置，则基于`EXTENSION`。您可以通过设置`BASEFILENAME_LENGTH`来更改基本文件名的最大长度；默认情况下，基本文件名将被截断为 245 个字符（请参阅21.7.1 节[HTML Xref Link Basics], 184 页）。

然而，在一般情况下，使用这些功能会使跨手册引用“失效”，因为生成的 URL 将无法链接到外部手册，除非外部手册本身也是使用相应的定制化生成的。因此，通常最好只使用 HTML 外部交叉引用配置（请参阅21.7.5 节[HTML Xref Configuration], 187 页）。

21.8.3 导航栏和页眉的定制化

默认情况下，会输出带有导航面板的页眉和页脚。如果您使用`--no-headers`，或者如果定制化变量`HEADERS`未设置，则导航面板仅插入到分割文件的开头。

默认情况下，HTML 的`<head>`中会生成`<link>`元素，用于指定 HTML 页面与其他资源（如顶部节点、下一个节点或目录）之间的关系。若要避免生成这些元素，请取消设置`USE_LINKS`。

`<title>`和`<head>`中的文档描述是基于`@settitle`的（请参阅2.5.3 节[@settitle], 12 页）。如果手册被分割，则节点名也会添加到此 HTML 标题中。设置`SECTION_NAME_IN_TITLE`以使用相关章节名，而不是节点名。

默认情况下，如果设置了输入编码（通常通过`@documentencoding`设置），则使用此信息来设置输出编码名称，否则输出编码将基于默认编码。在 HTML 的‘`<head>`’部分会输出一个‘`<meta>`’标签，用于指定输出编码。请参阅14.2 节[@documentencoding], 119 页。要显式设置输出编码，请设置`OUTPUT_ENCODING_NAME`。指定的编码应为 HTML 中可用的规范化字符集名称，典型是 IANA 编码名称之一。

当输出在节点处分割时（请参阅21.2 节[HTML Splitting], 179 页），`WORDS_IN_PAGE`定制化变量值指定了页面底部放置导航面板的大致的最小页面长度。

导航面板的外观受以下定制化变量的影响，在默认情况下，这些变量均为 false(假)：

`DATE_IN_HEADER`

在页眉处注明文件生成日期。

`HEADER_IN_TABLE`

使用表格来设置页眉格式，而不是简单的‘`<div>`’。

`ICONS`

在导航面板上使用图标。

`PROGRAM_NAME_IN_FOOTER`

如果已设置，则在页面的页脚中输出程序名称和其他相关信息。

`VERTICAL_HEAD_NAVIGATION`

如果已设置，则使用垂直导航面板。

要在导航面板上使用图标，设置`ICONS`是必要的，但仅此并不足以获取方向按钮的图标，因为在默认情况下并未指定任何按钮图像。此外，您还需要设置`ACTIVE_ICONS`和`PASSIVE_ICONS`定制化变量。您只能通过初始化文件（请参阅19.1 节[Invoking *texi2any*], 148 页）来完成此操作。请参阅GNU *Texinfo texi2any*输出定制化。

21.8.4 HTML 特性定制

默认情况下，会生成 HTML，但你可以设置`USE_XML_SYNTAX`来生成与 XML 兼容的代码。主要区别在于，对于像`<img>`或`<link>`这样没有结束标志的标签（这类标签创建所谓的空元素(void elements)），结束标志是‘`</>`’而不是‘`>`’。

要设置 HTML 的 DOCTYPE, 请设置 DOCTYPE 定制化变量。默认值是用于 HTML5 的简单无 DTD 的 DOCTYPE。

你可以设置一个兼容 XML 的文件开头:

```
texi2any --html my_manual.texi -c USE_XML_SYNTAX \
  -c DOCTYPE='<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE html>'
```

您也可以将 DOCTYPE 设置为 HTML4.01 过渡版:

```
texi2any --html my_manual.texi -c DOCTYPE=\
  '<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN"
  "http://www.w3.org/TR/html4/loose.dtd">'
```

默认情况下, 加倍单引号字符 (请参阅 11.5 节 [Inserting Quotation Marks], 92 页) 以及 ‘---’ 和 ‘--’ (请参阅 2.1 节 [Conventions], 8 页), 使用实体形式。在极少数情况下, 如果您希望输出更简单, 可以将 USE_ISO 设置为 ‘0’。默认情况下, 只要有可能就使用文本实体。设置 USE_NUMERIC_ENTITY 以仅使用数字字符实体。设置 OUTPUT_CHARACTERS 以根据输出编码输出重音字符, 而不是实体。

默认情况下, 标准允许使用定制化属性来在交叉引用中提供目标手册名称。如果不希望这样做 (例如, 为了生成严格的 XHTML), 可以设置 NO_CUSTOM_HTML_ATTRIBUTE 来阻止输出定制化属性。

对于定义命令 (请参阅第 13 章 [Definition Commands], 105 页)、定义了索引条目的表命令 (请参阅 8.4 节 [Two-column Tables], 70 页, 如: @vtable) 和标题 (headings, 如: 章节的标题) 的链接目标, 会输出链接目标的可复制链接。当您将鼠标指针悬停在标题文本上时, 会出现一个 ‘¶’ 符号 (段落符号), 表示存在可复制链接。若要阻止输出可复制的链接, 请取消 COPIABLE_LINKS 设置。

21.8.5 插入的 HTML 代码的定制化

有一些变量用于设置在输出中的不同位置插入的 HTML 代码。

对于 @samp 输出等需要的开引号和闭引号, 设置如下:

‘ 和 ’;

默认情况下, OPEN_QUOTE_SYMBOL 和 CLOSE_QUOTE_SYMBOL 未作定义。

‘ 和 ’;

如果你设置了 USE_NUMERIC_ENTITY。

开引号和闭引号字符

如果你设置了 OUTPUT_CHARACTERS, 并且输出编码中包含了该字符。

OPEN_QUOTE_SYMBOL 和 CLOSE_QUOTE_SYMBOL

如果你设置了那些定制化变量。

如要在 HTML 代码不同上下文中插入的水平分隔线, 请设置 DEFAULT_RULE, 该变量用于在生成的 HTML 中输出大多数水平分隔线; 同时, 如果想在必要时使用较宽分隔线, 请设置 BIG_RULE。

你可以设置 HTML_ROOT_ELEMENT_ATTRIBUTES 来为 <html> 元素添加属性。你可以定义变量 EXTRA_HEAD 来在 HTML 的 <head> 元素中添加文本。同样地, AFTER_BODY_OPEN 的值会在 <body> 输出后立即添加。这些变量默认是空的。

您可以通过定义定制化变量 BODY_ELEMENT_ATTRIBUTES 来设置 <body> 元素的属性。默认情况下, ‘lang’ 属性被设置为文档语言 (请参阅 14.1 节 [documentlanguage], 118 页)。

你可以定义变量`PRE_BODY_CLOSE`，以便在 HTML 的`</body>`元素之前添加文本。默认情况下，不会添加任何内容。

同样，以下变量可用于对目录和简要目录的格式进行一些有用的控制：

`BEFORE_TOC_LINES`
插入到目录文本之前。

`AFTER_TOC_LINES`
插入到目录文本之后。

`BEFORE_SHORT_TOC_LINES`
插在简要目录文本之前。

`AFTER_SHORT_TOC_LINES`
插在简要目录文本之后。

在写入时，这些变量的默认值是：包围目录的`<div>`元素的开标签或闭标签。

21.8.6 特定输出的 HTML 定制

特定结构的输出是可定制的。

在 HTML 中，标题级别会转换为`<hn>`元素，其中 n 表示标题级别。以下标题级别是可自定义的：

`CHAPTER_HEADER_LEVEL`
章级别的章节命令所使用的标题格式化级别；默认值为‘2’。

`FOOTNOTE_END_HEADER_LEVEL`
`FOOTNOTE_SEPARATE_HEADER_LEVEL`
用于带有‘end’脚注样式或‘separate’脚注样式的脚注标题的标题格式化级别；默认值为‘4’。请参阅9.3.2 节[Footnote Styles], 79 页。

`MAX_HEADER_LEVEL`
所使用的最大标题格式化级别（标题格式化级别数字越大，对应的章节级别越低）；默认值为‘4’。

您可以通过设置来修改各种结构的外观：

`DEF_TABLE`
如果设置此项，则对于`@defn`和类似的`@`命令，将使用`<table>`构造（看起来更像`TEX`的输出），而不是定义列表；默认为`false`。

`INDENTED_BLOCK_COMMANDS_IN_TABLE`
如果已设置，则使用表格来缩进缩进块命令；默认为`false`。

`INDEX_ENTRY_COLON`
索引条目与相关节点(或章节)之间使用的符号；默认为空字符串。

`MENU_ENTRY_COLON`
菜单项与描述之间使用的符号；默认为‘:’。

`MENU_SYMBOL`
当节点名用于菜单项格式化时，在菜单项前使用的符号；如果未设置`USE_NUMERIC_ENTITY`，则默认值为未定义，并设置为`•`；如果已设置，则设置为`’`。

NUMBER_FOOTNOTES

NO_NUMBER_FOOTNOTE_SYMBOL

默认情况下, 脚注会进行编号。若使用`--no-number-footnotes`选项, 或者NUMBER_FOOTNOTES设置为‘0’, 则使用‘*’代替, 或者如果设置了NO_NUMBER_FOOTNOTE_SYMBOL定制化变量, 则使用该变量的值。

21.8.7 数学相关的 HTML 定制

您可以选择多种方法在 HTML 中渲染数学公式 (请参阅11.7 节[Inserting Math], 93 页)。这由HTML_MATH定制化变量控制。默认情况下, 该定制化变量未设置, HTML 输出仅加粗显示。您可以选择其他选项以便正确显示格式化的数学公式。对于其中一些选项, 如果设置了`--iftex`, 则`@tex`部分会被转换为 HTML; 如果设置了`--iflatex`, 则`@latex`部分会被转换为 HTML。

l2h 使用`latex2html`程序为数学材料生成图像文件。
`latex2html`会处理数学命令 (`@math`) 中的 L^AT_EX 数学公式, 包括与 L^AT_EX 兼容的 T_EX 数学公式。如果相应的部分没有被忽略, `latex2html`会用于将`@tex`和`@latex`部分翻译成 HTML。请注意, `latex2html`只能处理 L^AT_EX, 包括处理`@tex`部分。
 使用`latex2html`进行处理后, 输出目录中应保留一些文件, 文件名中带有‘-l2h’, 特别是会保留一个缓存文件, 以避免在已经完成翻译的情况下重新生成 HTML。
 请参阅21.8.7.2 节[`latex2html` Customization Variables], 197 页。

mathjax 在输出文件中插入对 MathJax 脚本的引用, 以格式化数学内容。MathJax 选项需要在浏览器中启用 JavaScript 才能工作。
 MathJax 处理 L^AT_EX 数学公式, 包括与 L^AT_EX 兼容的常用 T_EX 数学公式。虽然某些特定构造 (例如`\text`的某些用法) 不受支持, 但这在实际应用中应该不是问题。
 如果数学命令被实际处理, 则会为 MathJax 脚本生成一个 JavaScript 许可证网页标签 (请参阅[JavaScript license web labels], 198 页)。
 请参阅21.8.7.1 节[`MathJax` Customization Variables], 196 页。

t4h 使用`tex4ht`程序为数学材料生成 HTML。
`tex4ht`会处理数学命令中的 L^AT_EX 数学内容, 包括与 L^AT_EX 兼容的 T_EX 数学内容。如果`@tex`和`@latex`部分没有被忽略, 则`tex4ht`用于将这些部分翻译成 HTML。`@tex`部分以 T_EX 模式处理, 而`@latex`部分则作为 L^AT_EX 处理。
 使用`tex4ht`进行处理后, 输出目录中的文件名称中应包含‘_tex4ht’。
 请参阅21.8.7.3 节[`tex4ht` Customization Variables], 198 页。

默认情况下, 如果未定义`CONVERT_TO_LATEX_IN_MATH`, 则设置HTML_MATH也会同时设置`CONVERT_TO_LATEX_IN_MATH`。在这种情况下, 在将`@math`或`@displaymath`转换为 HTML 之前, `@math`和`@displaymath`中的 Texinfo 的 `@` 命令会先转换为 L^AT_EX。

21.8.7.1 MathJax 定制化变量

此表列出了在使用 MathJax 时可用的定制化变量, 当HTML_MATH设置为‘MathJax’时, 就会使用 MathJax。

MATHJAX_CONFIGURATION

要添加到全局‘MathJax’配置对象中的数据。以逗号分隔的`component: { name: value, ... }` 对的形式指定。

例如, 你可以将MATHJAX_CONFIGURATION设置为:

```
options: { enableMenu: false },
loader: {
  load: ['[tex]/physics'],
  versionWarnings: false
},
tex: {
  packages: {'[+]\': ['physics']}
}
```

禁用 MathJax 右键菜单, 并加载 MathJax physics 扩展。(请参阅 MathJax 文档中的Configuring MathJax (<https://docs.mathjax.org/en/latest/options/index.html>)).

如果您覆盖了texi2any为此配置输出的任何默认设置, 则应确保您的新设置与texi2any生成的 HTML 输出相匹配。

MATHJAX_SCRIPT

您正在使用的 MathJax 组件文件 (例如: tex-svg.js) 的 URL。texi2any为此变量提供了一个默认值, 但建议您将此文件自行托管在您的网站上, 这样您就不必依赖他人的托管服务。

MATHJAX_SOURCE

对于由MATHJAX_SCRIPT指定 (命名) 的组件文件, 提供其易于修改形式的完整源代码的 URL, 或如何获取此类源代码的说明。‘易于修改形式’意味着这不应是‘压缩’形式。用于许可证标签页面 (请参阅[JavaScript license web labels], 198 页)。

同样, texi2any为此变量提供了一个默认值, 但我们鼓励您自行托管 MathJax 及其依赖项的源代码。这是为了确保源代码的可靠获取, 并减少您和您的用户对其他分发系统的依赖。

21.8.7.2 latex2html定制化变量

此表列出了在使用latex2html将@math、@displaymath、@latex和@tex部分转换为 HTML 时, 可以使用的定制化变量。这些定制化变量仅在HTML_MATH设置为‘l2h’时才相关。

要实际转换@tex部分, 应使用--iftex; 要实际转换@latex部分, 应使用--iflatex。

L2H_CLEAN

如果已设置, 则与latex2html相关的中间文件将被删除; 默认为 true。

L2H_FILE 如果已设置, 则给定的文件将用作latex2html的初始化文件; 默认未设置。

L2H_HTML_VERSION

在latex2html调用中使用的 HTML 版本; 默认为未设置。

L2H_L2H 以latex2html名义实际调用的程序; 默认值为‘latex2html’。

L2H_SKIP 如果设置为 true 值, 则跳过对latex2html的实际调用; 而是重用之前生成的内容。如果设置为‘0’, 则根本不使用缓存。如果设置为‘undef’, 则缓存将尽可能多地用于 $\TeX$ 片段, 对于任何剩余的片段, 则运行该命令。默认值为‘undef’。

L2H_TMP 设置用于临时文件的目录。此目录名中的各文件名组成部分都不能以‘.’开头; 否则, latex2html将失败 (因为dvips会出现问题)。默认值为空字符串, 表示当前目录。

21.8.7.3 tex4ht定制化变量

此表列出了在使用tex4ht将`@math`、`@displaymath`、`@tex`和`@latex`部分转换为 HTML 时，可以使用的定制化变量。这些变量仅在HTML_MATH设置为‘t4h’时才有作用。

要实际转换`@tex`部分，应使用`--iftex`；要实际转换`@latex`部分，应使用`--iflatex`。

T4H_LATEX_CONVERSION

用于设置`@latex`部分的转换类型。可能的选项包括‘`latex`’、‘`tex`’和‘`texi`’。若未定义，则默认为‘`latex`’。

T4H_MATH_CONVERSION

用于设置`@math`和`@displaymath`的转换类型。可选值为‘`latex`’、‘`tex`’和‘`texi`’。若未定义，则设置为‘`tex`’。

T4H_TEX_CONVERSION

用于设置`@tex`部分的转换类型。可能的选项包括‘`latex`’、‘`tex`’和‘`texi`’。若未定义，则设置为‘`tex`’。

21.8.8 JavaScript 界面和许可

您可以通过将`INFO_JS_DIR`的值设置为放置界面代码的目录名称，从而在 HTML 输出中添加一个实验性的 JavaScript 浏览界面。此界面在网页浏览器中提供 Info 浏览器的一些功能，如键盘导航和索引查找。例如，‘`texi2any --html -c INFO_JS_DIR=js manual.texi`’会将文件放置在输出目录下的‘`js`’目录中。此界面仅适用于非分割 HTML 输出。

默认情况下，会创建一个 JavaScript 许可标签页 (*JavaScript license web labels page*)，以提供 HTML 输出中使用的所有 JavaScript 的许可信息和源代码（参见<https://www.gnu.org/licenses/javascript-labels.html>）。若要避免生成 JavaScript 许可标签页，请将`JS_WEBLABELS`设置为‘`omit`’。默认情况下，`JS_WEBLABELS`设置为‘`generate`’时，会在`JS_WEBLABELS_FILE`（默认为`js_licenses.html`）处生成一个标签页，并在 HTML 输出文件中链接到该页面。当`JS_WEBLABELS`设置为‘`reference`’时，在输出中仅链接到由`JS_WEBLABELS_FILE`指定的标签文件，而不生成标签文件。如果您为包含手册的大型网站单独维护一个单独的标签文件，则最后一种设置非常有用。

如果设置了`INFO_JS_DIR`，或者如果将`HTML_MATH`设置为‘`MathJax`’（请参阅[*MathJax scripts*], 196 页），则会生成标签文件。

21.8.9 HTML 定制化变量列表

此表列出了适用于 HTML 输出的定制化变量。

AFTER_BODY_OPEN

在每个 HTML 文件的开头添加文本；默认未设置。

AFTER_SHORT_TOC_LINES

AFTER_TOC_LINES

如果设置了`AFTER_SHORT_TOC_LINES`，则文本输出位于简要目录之后；如果设置了`AFTER_TOC_LINES`，则文本输出位于目录之后；否则，将使用默认字符串。在写入该文本时，会使用`</div>`元素作为结束。

一般来说，如果设置了`AFTER_SHORT_TOC_LINES`，则应设置`BEFORE_SHORT_TOC_LINES`；如果设置了`AFTER_TOC_LINES`，则应设置`BEFORE_TOC_LINES`。

BASEFILENAME_LENGTH

基本文件名的最大长度；默认为 245。更改此值，将导致对超过此长度的节点名的跨手册引用失效（请参阅21.7.1 节[*HTML Xref Link Basics*], 184 页）。

BEFORE_SHORT_TOC_LINES**BEFORE_TOC_LINES**

如果已设置，则会在简要目录之前输出BEFORE_SHORT_TOC_LINES，在目录之前输出BEFORE_TOC_LINES，否则将使用默认字符串。在编写时，会使用一个<div ...>元素作为开始。

一般来说，如果设置了BEFORE_SHORT_TOC_LINES，则应设置AFTER_SHORT_TOC_LINES；如果设置了BEFORE_TOC_LINES，则应设置AFTER_TOC_LINES。

BIG_RULE 该规则应用于顶元素之后和之前，以及特殊元素之前，但不应用于页脚和页眉；默认值为<hr>。

BODY_ELEMENT_ATTRIBUTES

<body>属性文本。默认情况下，将 HTML 的lang属性设置为文档语言（请参阅14.1 节[@documentlanguage], 118 页）。

CASE_INSENSITIVE_FILENAMES

默认为 false；在构造输出文件名时，假定文件系统不区分大小写（请参阅21.2 节[HTML Splitting], 179 页）。

CHAPTER_HEADER_LEVEL

章级命令所使用的标题格式级别；默认值为‘2’。

CHECK_HTMLXREF

检查作为外部交叉引用目标的手册（请参阅5.6 节[Four and Five Arguments], 42 页）是否存在于htmlxref.cnf中（请参阅21.7.5 节[HTML Xref Configuration], 187 页）；默认为 true。

CONTENTS_OUTPUT_LOCATION

如果设置为‘after_top’，则目录输出在@top部分之后。如果设置为‘inline’，则目录输出在@contents及类似 @ 命令所在位置。如果设置为‘separate_element’，则将目录输出到单独的元素中；如果未分割，则输出到文档末尾，否则输出到单独的文件中。如果设置为‘after_title’，则目录输出在标题之后；默认为‘after_top’。

CONVERT_TO_LATEX_IN_MATH

如果已设置，则在将@math或@displaymath转换为 HTML 之前，尝试将@math和@displaymath中的所有 Texinfo 的 @ 命令转换为 L^AT_EX。默认值为undef。如果未定义，则当HTML_MATH已设置时才进行设置。

COPIABLE_LINKS

如果设置，为定义命令（请参阅第 13 章[Definition Commands], 105 页）、定义的索引条目的表命令（请参阅8.4 节[Two-column Tables], 70 页）中，以及标题（章节及其它的），输出可复制的链接。当您鼠标指针悬停在标题文本上时，该链接会显示成为一个‘¶’（冒号）符号。

DATE_IN_HEADER

在页眉中显示文档生成日期；默认情况下为关闭状态。

DEF_TABLE

如果设置，则@defn和类似的 @ 命令使用<table>构造（看起来更像 T_EX输出），而不是定义列表；默认值为 false。

DEFAULT_RULE

除顶部元素前后、特殊元素前以及页脚和页眉外，元素之间使用的分隔线；默认值为`<hr>`。

DO_ABOUT 如果设置为 0，则不包括‘关于(About)’特殊元素；如果设置为 1，则包括‘关于’特殊元素的操作；默认值为 0。

EPUB_CREATE_CONTAINER_FILE

如果设置为 0，则不生成 EPUB 输出文件。默认设置为 1。

EPUB_KEEP_CONTAINER_FOLDER

如果已设置，请保留包含 EPUB 所需目录(及文件)的目录。EPUB 输出文件是此目录的 ZIP 存档。默认值未定义。如果未定义且已设置TEST或DEBUG，则设置。请参阅21.5.1 节[EPUB Output File and Directory], 182 页。

EXTERNAL_CROSSREF_SPLIT

对于对其他手册的交叉引用，该变量决定了其他手册是被视为分割形式还是整体形式。默认情况下，它是按SPLIT的来设置的。请参阅21.7 节[HTML Xref], 183 页，以及21.7.5 节[HTML Xref Configuration], 187 页。

EXTERNAL_DIR

外部手册的基目录；默认为无。与使用此变量相比，使用通用的外部交叉引用机制（请参阅21.7.5 节[HTML Xref Configuration], 187 页）更为可取。

EXTERNAL_CROSSREF_EXTENSION

用于交叉引用的其他手册的文件扩展名。若未设置，则基于EXTENSION。

EXTRA_HEAD

在`<head>`中出现的额外文本；默认未设置。

FOOTNOTE_END_HEADER_LEVEL

用于带有‘end’脚注样式的脚注标题的标题格式化级别；默认值为‘4’。请参阅9.3.2 节[Footnote Styles], 79 页。

FOOTNOTE_SEPARATE_HEADER_LEVEL

用于带有‘separate’脚注样式的脚注标题的标题格式化级别；默认值为‘4’。请参阅9.3.2 节[Footnote Styles], 79 页。

HEADER_IN_TABLE

使用表格(tables)进行标题格式化，而不是简单的`<div>`元素；默认为 false。

HIGHLIGHT_SYNTAX

如果设置了此选项，则会在 HTML 输出中，对`@example`块进行语法敏感高亮处理，语言名称是`@example`的第一个参数。还可以使用HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE指定默认语言。语法敏感高亮处理需要外部程序来生成语法敏感高亮后的 HTML，HIGHLIGHT_SYNTAX值用来选择特定的外部程序，可选值包括highlight、pygments，以及代表source-highlight的任何其他值（请参阅21.6 节[Syntax Highlighting], 183 页）。

HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE

在没有指定语言名称时，用于语法敏感高亮处理的默认语言。

HTML_MATH

用于呈现`@math`的方法（请参阅21.8.7 节[HTML Customization for Math], 196 页）。此变量可以是未设置、设置为‘MathJax’（请参阅21.8.7.1 节[MathJax

Customization Variables], 196 页)、设置为‘12h’(使用`latex2html`, 请参阅21.8.7.2 节[`latex2html` Customization Variables], 197 页), 或设置为‘t4h’(使用`tex4ht`, 请参阅21.8.7.3 节[`tex4ht` Customization Variables], 198 页)。默认情况下, 设置`HTML_MATH`也会同时设置`CONVERT_TO_LATEX_IN_MATH`。

HTML_ROOT_ELEMENT_ATTRIBUTES

将该字符串用于`<html>` HTML 文档的根元素。默认值为未定义。

HTMLXREF_FILE

设置用于包含交叉引用其他手册的信息的文件名。若未定义, 则使用`htmlxref.cnf` (请参阅21.7.5 节[HTML Xref Configuration], 187 页)。默认情况下未定义。若设置了`TEST`, 则`HTMLXREF_MODE`将设置为默认值, 且`HTMLXREF_FILE`未定义, 此时不会使用交叉引用其他手册的信息。

如果`HTMLXREF_MODE`设置为‘file’, 则直接使用`HTMLXREF_FILE`设定的文件名作为信息来源; 否则, 将在目录中搜索文件名, 并使用找到的所有文件 (请参阅21.7.5 节[HTML Xref Configuration], 187 页)。

HTMLXREF_MODE

如何对交叉引用其他手册的信息的确认。如果设置为‘none’, 则不使用任何信息。如果设置为‘file’, 则从文件路径 (默认情况下为`htmlxref.cnf`) 或`HTMLXREF_FILE`的值中确认信息。如果未定义 (默认情况) 或设置为任何其他值, 则会在目录中搜索并使用所有文件 (请参阅21.7.5 节[HTML Xref Configuration], 187 页)。

ICONS 导航面板使用图标; 默认值为 `false`。

IMAGE_LINK_PREFIX

如果已设置, 则相关值将前置到图像文件链接中; 默认未设置。

INDENTED_BLOCK_COMMANDS_IN_TABLE

如果已设置, 则使用表格对缩进块命令进行缩进; 默认为 `false`。

INDEX_ENTRY_COLON

在索引条目和相关节或章节之间使用的符号; 默认为空字符串。

INFO_JS_DIR

(实验性。) 在手册中添加一个 JavaScript 浏览界面。该变量的值是放置此界面代码的目录, 因此您需要运行程序, 例如: ‘`texi2any --html -c INFO_JS_DIR=js manual.texi`’, 以将文件放置在输出目录下的‘js’目录中。这可以在网页浏览器中提供 Info 浏览器的一些功能, 如键盘导航和索引查找。这仅适用于分割 HTML 输出¹。

如果 JavaScript 或网络浏览器功能不可用, 界面应提供可接受的功能回退方案。但是, 在使用此选项时请务必谨慎, 以免使部分用户难以访问您的文档。

IGNORE_REF_TO_TOP_NODE_UP

忽略对`TOP_NODE_UP` (即顶节点的上一级节点) 的引用。

INLINE_CSS_STYLE

将 CSS 直接放在 HTML 元素中, 而不是放在输出的开头; 默认为 `false`。

¹ 译者注: 原文写反了, 译文作了纠正。

JS_WEBLABELS**JS_WEBLABELS_FILE**

说明如何使用JavaScript 许可证网页标签(*JavaScript license web labels*) 页面为手册 HTML 文件中使用的所有 JavaScript 提供许可信息和源代码。(请参阅<https://www.gnu.org/licenses/javascript-labels.html>)。

使用默认值‘generate’，在JS_WEBLABELS_FILE生成一个标签页面，并在 HTML 输出文件中链接到该页面。仅在实际引用 JavaScript 文件时（要么HTML_MATH设置为‘MathJax’，要么在设置INFO_JS_DIR时使用实验性的 JS 浏览界面）才执行此操作。在此设置下，JS_WEBLABELS_FILE必须是相对文件名。

使用值‘reference’，在输出中链接到由JS_WEBLABELS_FILE指定的标签文件，并且不生成标签文件。如果您为包含手册的大型网站单独维护一个单独的标签文件，此设置将非常有用。

使用‘omit’时，既不会生成标签文件，也不会链接到标签文件。

MAX_HEADER_LEVEL

所使用的最大的标题格式化级别（标题格式级别数字越大，对应的章节级别越低）；默认值为‘4’。

MENU_ENTRY_COLON

菜单项和描述之间使用的符号；默认为‘:’。

MENU_SYMBOL

当节点名用于菜单项格式化时，在菜单项前使用的符号；如果USE_NUMERIC_ENTITY未设置，则默认值为未定义，并把MENU_SYMBOL设置为•; 如果USE_NUMERIC_ENTITY已设置，则MENU_SYMBOL默认值为’。

MONOLITHIC

仅输出一个文件，其中包含目录部分。此为默认设置，但仅在非分割输出时才有意义。

NO_CSS 不要使用 CSS；默认为false。请参阅21.3 节[HTML CSS], 180 页。

NO_CUSTOM_HTML_ATTRIBUTE

不要输出在元素中带有自定义属性的 HTML；默认为false。

NO_NUMBER_FOOTNOTE_SYMBOL

如果NUMBER_FOOTNOTES为 false，则本变量用于指定脚注的符号。默认值为*。

NODE_NAME_IN_INDEX

如果为true，则在索引条目中使用节点名；否则，优先使用章节名。如果未定义，则在 HTML 中使用USE_NODES的值。默认值为未定义。

NODE_NAME_IN_MENU

如果已设置，则在菜单项中使用节点名，否则优先使用章节名；默认为true。

PRE_BODY_CLOSE

如果已设置，则给定的文本将出现在每个 HTML 文件的页脚处；默认未设置。

PROGRAM_NAME_IN_ABOUT

当输出‘关于(About)’元素时使用。如果已设置，则在‘关于’特殊元素中输出程序名称和其他相关信息；默认为false。

PROGRAM_NAME_IN_FOOTER

如果已设置为`true`，则在页面的页脚中输出程序名称和其他相关信息；默认为`false`。

SECTION_NAME_IN_TITLE

在分割输出时，如果该变量已设置，则在`<title>`中使用章节命令（例如，`@chapter`或`@section`）的参数，而不是`@node`的参数。

SHORT_TOC_LINK_TO_TOC

如果已设置`true`，则简要目录中的交叉引用将链接到相应的目录条目（如果输出目录）；默认为`true`。

SHOW_BUILTIN_CSS_RULES

如果设置为`true`，在标准输出上输出内置的默认 CSS 规则，然后退出。

SHOW_TITLE

如果已设置为`true`，则在文档开头输出标题（因为`USE_TITLEPAGE_FOR_TITLE`定制化变量默认值为`true`，所以，实际输出经常是由`@titlepage...@end titlepage`定义的标题页）；默认为`'undef'`。如果设置为`'undef'`，则在设置`NO_TOP_NODE_OUTPUT`的同时也会为 HTML 设置`SHOW_TITLE`。

TEXI2HTML

生成 HTML，并尽可能与`texi2html`兼容；默认值为`false`。

TOC_LINKS

如果设置为`true`，则创建从标题到目录条目的链接；默认为`false`。²

TOP_FILE

此文件名可用于顶层文件。如有必要，扩展名也需适当地一并设置。这是用来覆盖默认设置（即：`index.html`）的，通常只有在输出被分割时才会考虑，并且仅适用于 HTML。

TOP_NODE_FILE_TARGET

用于交叉引用中顶部节点的文件名；默认为`index.html`。

TOP_NODE_UP_URL

一个用于顶节点的向上一级引用的 URL；默认值为`undef`，在这种情况下，不会生成顶节点的向上一级引用。在`TOP_NODE_UP_URL`已设置的情况下，或在其他输出格式时，如要覆盖的向上一级指针的名称，请参阅 19.4.7 节[Customization Variables for Multiple Formats]，164 页中的`TOP_NODE_UP`。请参阅 21.8.2 节[File Names and Links Customization for HTML]，191 页。有关顶节点指针的更多信息，请参阅 3.4 节[First Node]，26 页。

USE_ACCESSKEY

在交叉引用中使用`accesskey`；默认为`true`。

USE_ISO

对于双单引号字符（请参阅 11.5 节[Inserting Quotation Marks]，92 页），以及`'---`和`--'`（请参阅 2.1 节[Conventions]，8 页），使用实体；默认为`true`。

USE_LINKS

在 HTML 的`<head>`输出中生成`<link>`元素；默认为`true`。

² 译者注：在个功能实际上挺有用，建议输出时，将其打开。

USE_NEXT_HEADING_FOR_LONE_NODE

如果已设置，节点未与章节命令相关联，但有一个通常与节点无关的标题命令（如`@heading`）紧随其后（即：在中间没有其他格式化内容），则不会将节点名作为标题输出，而是假定标题命令提供了标题。默认值为`true`。

USE_NODE_DIRECTIONS

如果设为`true`，则使用节点作为在节点页眉中后一页(`next`)、前一页(`previous`)和上一级(`up`)指针的链接目标；如果设为`false`，则使用章节作为目标。如果未定义，则使用`USE_NODES`的值。默认值为未定义。请注意，此设置并不确定链接字符串，而仅确定链接指向的目标；有关链接字符串的定制化，请参阅5.5节[`xrefautomaticsectiontitle`]，41 页。如果节点和章节是全系统关联的，则此定制化变量没有实际效果。

USE_REL_REV

在交叉引用中使用`rel`；默认为`true`。

USE_TITLEPAGE_FOR_TITLE

使用`@titlepage...@end titlepage`所包含的完整内容作为标题，而非简单的标题字符串；默认为`true`。仅当设置了`SHOW_TITLE`时才有意义。

USE_XML_SYNTAX

使用与 XML/XHTML 兼容的语法。

VERTICAL_HEAD_NAVIGATION

如果已设置`true`，则使用垂直导航面板；默认为`false`。

WORDS_IN_PAGE

当输出按节点划分时，指定一个大致最小页面长度³；如页面超过该长度，则在页面底部放置（第二个）导航面板。为避免导航面板出现在页面底部，请将此值设置为足够大的数字⁴。默认值为 300。

XREF_USE_FLOAT_LABEL

如果启用此设置，则：在交叉引用中的浮动名称，使用浮动标签而不是浮动类型加浮动序号（请参阅9.1.1 节[`@float`]，74 页）。默认设置为禁用此功能。

XREF_USE_NODE_NAME_ARG

仅适用于没有交叉引用名称（第二个参数）的交叉引用命令。如果设置为‘1’，则在交叉引用 `@` 命令中使用节点名（第一个）参数作为超链接显示的文本。如果设置为‘0’，则：如果设置了`USE_NODES`，则使用节点名称，否则使用章节名称。如果设置为‘undef’，则在预格式化环境中使用第一个参数，否则根据`USE_NODES`使用节点名称或章节名称。默认值为‘undef’。

³ 译者注：实际上是页面中的 Word 的数量。

⁴ 译者注：如果想在所有页面中都放置一个底部导航面板，则可设为 0。

附录 A @ 命令的详情

以下是 @ 命令的详细信息：语法说明、列表以及可使用的位置。

A.1 @ 命令的语法

Texinfo 有以下类型的 @ 命令：

1. 花括号命令(Brace commands)
这些命令以 '@' 开头，后跟一个字母或单词，再后跟一个用花括号括起来的参数。例如，命令 `@dfn` 表示术语的引入或定义使用；其使用方式如下：‘在 Texinfo 中，`@@` 命令是 `@dfn{标记式}` 命令。’
2. 行命令(Line commands)
这些命令占一行。该行以 '@' 开头，后跟命令名称（一个单词）；例如，`@center` 或 `@cindex`。如果不需要参数，则该单词后即为行尾。如果有参数，则参数与命令名称之间用空格隔开。不使用花括号。
3. 区块命令(Block commands)
这些命令写在行首，后续行上是一般文本，最后一行以匹配的 `@end` 命令结束。例如，`@example`，然后是示例代码行，最后是 `@end example`。其中一些区块命令像行命令一样接受参数；例如，`@enumerate A` 打开一个环境，以 `@end enumerate` 结束。这里的 'A' 是参数。
4. 无参数的符号插入命令
这些命令以 '@' 开头，后跟一个单词，再后跟左右花括号。这些命令在文档中插入特殊符号；它们不接受参数。一些示例：`@dots{} ⇒ '…'`，`@equiv{} ⇒ '≡'`，`@TeX{} ⇒ 'TEX'`，`@bullet{} ⇒ '•'`。
5. 非字母命令(Non-alphabetic commands)
上述所有类别的命令名称均由字母字符组成，且几乎全部为小写。与此不同，非字母命令由一个 '@' 后面跟随一个标点符号或其他不属于拉丁字母的字符组成。非字母命令几乎总是段落内文本的一部分。非字母命令包括 `@@`、`@{`、`@}`、`@.`、`@SPACE` 以及大多数重音符号命令。
6. 其他命令(Miscellaneous commands)
有少数命令不属于上述任何一类；例如，命令 `@refill` (已过时)，它总是用在段落末尾，紧跟在句号或其他标点符号之后。`@refill` 不接受任何参数，也不需要花括号。同样，在 `@multitable` 块中使用的 `@tab` 也不接受参数，后面也不跟花括号。

因此，字母命令分为具有不同(参数)语法的类。你无法通过命令名称的外观来判断它属于哪个类别，但可以通过命令的含义来判断：如果命令代表一个字形，则它属于第 4 类，不需要参数；如果该命令作为段落的一部分与其他文本一起使用时有意义，则它属于第 1 类，并且后面必须跟一个用花括号括起来的参数。非字母命令（如 `@:`）是这一规则的例外；它们不需要括号。

为命令设置不同语法的目的是为了使 Texinfo 文件更易于阅读，同时也为了帮助 GNU Emacs 的段落和填充命令正常工作。

A.2 @ 命令的列表

以下是 Texinfo 中按字母排序的 @ 命令列表。方括号 [] 表示可选参数；省略号 ‘...’ 表示重复文本。

@whitespace

‘@’后跟空格、制表符或换行符，会产生一个正常的、可伸缩的词间间隔。请参阅11.3.1 节[Multiple Spaces], 89 页。

@! 在句子末尾加上感叹号（通常在句末大写字母之后）。请参阅11.3.3 节[Ending a Sentence], 90 页。

@"

@' 在接下来的字符上分别加上元音变音符号或重音符号，如ö 和ó 。请参阅11.4 节[Inserting Accents], 91 页。

@&

@ampchar{}

生成一个与符号‘&’。请参阅11.1.6 节[Inserting an Ampersand], 87 页。

@* 强制换行。请参阅12.2 节[Line Breaks], 101 页。

@,{c} 在c 下生成一个 cedilla 重音符号，如ç 。请参阅11.4 节[Inserting Accents], 91 页。

@- 插入一个可选择的断字点。即：在需要时，可在此处断字。请参阅12.3 节[@-@hyphenation], 102 页。

@. 在句子末尾加上句号（通常用在句末大写字母后）。请参阅11.3.3 节[Ending a Sentence], 90 页。

@/ 不产生输出，但允许换行。即：在需要时，可在此处换行。请参阅12.2 节[Line Breaks], 101 页。

@: 指示打印输出处理器在紧接其前的句号、问号、感叹号或冒号后不要插入（句末）额外空隙。请参阅11.3.2 节[Not Ending a Sentence], 89 页。

@= 在下一个字符上生成一个长音符（横线）重音，如ō 。请参阅11.4 节[Inserting Accents], 91 页。

@? 在句子末尾加上一个问号（通常在句末大写字母之后）。请参阅11.3.3 节[Ending a Sentence], 90 页。

@@

@atchar{}

插入一个 at 符号‘@’。请参阅11.1.1 节[Inserting an Atsign], 86 页。

@\

@backslashchar{}

插入一个反斜杠‘\’；@backslashchar{} 可在任何位置使用，而@\仅在@math ... @end math环境内部有效。请参阅11.1.4 节[Inserting a Backslash], 86 页，以及11.7 节[Inserting Math], 93 页。

@~

@` 在接下来的字符上分别加上抑扬符（帽子）或重音符号，如ô 和è 。请参阅11.4 节[Inserting Accents], 91 页。

- `@{`
`@lbracechar{}`
 插入左花括号‘{’。请参阅11.1.2 节[Inserting Braces], 86 页。
- `@}`
`@rbracechar{}`
 插入右花括号‘}’。请参阅11.1.2 节[Inserting Braces], 86 页。
- `@~`
 在接下来的字符上生成波浪线重音符号, 如Ñ。请参阅11.4 节[Inserting Accents], 91 页。
- `@AA{}`
`@aa{}`
 分别生成斯堪的纳维亚语 A 环字母的大写和小写形式: Å、å。请参阅11.4 节[Inserting Accents], 91 页。
- `@abbr{abbreviation}`
 标识一个通用的缩写, 如Comput. (computer)。请参阅6.1.13 节[@abbr], 54 页。
- `@acronym{acronym}`
 标识一个全部用大写字母表示首字母的缩略词, 如NASA (National Aeronautics and Space Administration)。请参阅6.1.14 节[@acronym], 54 页。
- `@AE{}`
`@ae{}`
 分别生成大写和小写的 AE 连字: Æ、æ。请参阅11.4 节[Inserting Accents], 91 页。
- `@afivepaper`
 将页面尺寸更改为 A5 纸张尺寸。请参阅节 E.7 [A4 Paper], 264 页。
- `@afourlatex`
`@afourpaper`
`@afourwide`
 将页面尺寸更改为 A4 纸张尺寸。请参阅节 E.7 [A4 Paper], 264 页。
- `@alias new=existing`
 将新命令‘@new’设置为现有命令‘@existing’的别名。请参阅16.4 节[@alias], 135 页。
- `@allowcodebreaks true-false`
 在打印输出中, 控制‘-’和‘_’处的换行。请参阅12.4 节[@allowcodebreaks], 102 页。
- `@anchor{name}`
 把当前位置定义为name 锚点, 以用作交叉引用的目标。请参阅5.11 节[@anchor], 44 页。
- `@appendix title`
 开始一个附录。标题(title)会出现在目录中。请参阅4.4 节[@unnumbered @appendix], 35 页。
- `@appendixsec title`
`@appendixsection title`
 在附录中另起一节, 作为附录的一部分。该节标题(title)会出现在目录中。@appendixsection是@appendixsec命令的完整拼写形式。请参阅4.7 节[@unnumberedsec @appendixsec @heading], 36 页。

- @appendixsubsec title**
开始一个附录的子节。标题(title)会出现在目录中。请参阅4.9 节[@unnumberedsubsec @appendixsubsec @subheading], 37 页。
- @appendixsubsubsec title**
开始一个附录的子子节。标题(title)会出现在目录中。请参阅4.10 节[@subsubsection], 37 页。
- @arrow{}** 生成一个右箭头字形: ‘→’。默认用于@click。请参阅11.9.8 节[Click Sequences], 99 页。
- @asis** 保持参数不变。用在@table、@ftable和@vtable之后, 要求在打印表的第一列时, 不对其进行高亮显示, 即`保持原样’。请参阅[@asis], 70 页。
- @author author**
在标题页中标识手册的作者(author)。请参阅2.8.2 节[@title @subtitle @author], 17 页。在@quotation中标识引用的作者(author)。请参阅7.2 节[@quotation], 59 页。
- @b{text}** 如果可能的话, 将text 设置为**bold**(加重) 字体。请参阅6.2.3 节[Fonts], 56 页。
- @bullet{}**
生成一个大的圆点, 即: ● 或最接近圆点的东西。通常与@itemize一起使用。请参阅11.8.5 节[@bullet], 95 页。
- @bsixpaper**
将页面尺寸更改为 B6 纸张尺寸。请参阅节 E.7 [A4 Paper], 264 页。
- @bye** 停止处理文件。处理器不会处理输入文件中@bye之后任何内容。请参阅2.12 节[Ending a File], 23 页。
- @c 注释(comment)**
在 Texinfo 中开始一个注释。该行的其余部分都是注释, 不会出现在任何输出中。它是@comment的同义词。DEL也可以开始一个注释。请参阅2.2 节[Comments], 8 页。
- @caption** 为@float定义完整的标题。请参阅9.1.2 节[@caption @shortcaption], 75 页。
- @cartouche**
如果可能的话, 用圆角框突出标记一个示例或引用。与@end cartouche配对。请参阅7.14 节[@cartouche], 65 页。
- @center 文本行(line-of-text)**
将文本行居中。请参阅2.8.3 节[@titlefont @center @sp], 17 页。
- @centerchap 文本行(line-of-text)**
类似于@chapter, 但将章标题居中显示。请参阅4.3 节[@chapter], 35 页。
- @chapheading title**
打印一个未编号的类似章的标题(heading), 但不在目录中显示。请参阅4.5 节[@majorheading @chapheading], 36 页。
- @chapter title**
开始一个有编号的章。章标题(title)会出现在目录中。4.3 节[@chapter], 35 页。

@cindex *entry*

将`entry` 添加到概念索引中。请参阅10.4 节[Defining the Entries of an Index], 82 页。

@cite{*reference*}

高亮显示没有配套 Info 文件的书籍或其他参考资料名称。请参阅5.15 节[@cite], 47 页。

@clear *flag*

取消`flag` 设置, 以阻止 Texinfo 格式化命令对后续成对的@ifset `flag` 和@end ifset 命令之间的文本进行格式化, 并阻止@value{`flag`} 展开为`flag` 所设置的值。请参阅15.5 节[@set @clear @value], 124 页。

@click{ } 表示图形用户界面 (GUI) 中的一个“点击”操作。常用于@clicksequence中。请参阅11.9.8 节[Click Sequences], 99 页。

@clicksequence{*动作* @click{ } *动作*}

表示图形用户界面 (GUI) 中的一系列点击操作。请参阅11.9.8 节[Click Sequences], 99 页。

示例:

```
@clicksequence{A @click{ } B @click{ } C}
```

产生:

```
A → B → C
```

@clickstyle @cmd

对每个@click执行@cmd ; 默认值为@arrow。通常在 @cmd后面的空括号被省略。请参阅11.9.8 节[Click Sequences], 99 页。

@code{*样例代码 (sample-code)*}

表示一个表达式、一个语法上完整的程序标记或一个程序名。请参阅6.1.2 节[@code], 49 页。

@codequotebacktick *on-off***@codequoteundirected** *on-off*

控制代码示例中` 和' 的输出。请参阅 11.2 节[Inserting Quote Characters], 88 页。

@comma{ } 插入逗号‘,’字符; 仅当字面逗号被视为参数分隔符时才需要。请参阅11.1.3 节[Inserting a Comma], 86 页。

@command{*命令名 (command-name)*}

标识一个命令名称, 例如ls。请参阅6.1.10 节[@command], 53 页。

@comment *comment*

在 Texinfo 中开始一个注释。该行的其余部分不会出现在任何输出中。它是@c的同义词。请参阅2.2 节[Comments], 8 页。

@contents

对于可以输出目录的格式, 打印完整的目录或要求必须输出目录。请参阅2.9 节[Generating a Table of Contents], 19 页。

@copying 为文档指定版权所有者和复制条件。与@end copying配对。请参阅2.7.1 节[@copying], 15 页。

@copyright{}

生成版权符号‘©’。请参阅11.8.2 节[`@copyright`], 94 页。

@defblock

开始一个包含定义的代码块。与‘`@end defblock`’配对。请参阅13.7 节[Generic Definition Commands], 114 页。

@defcodeindex 索引名称(*index-name*)

定义一个新的索引及其索引命令。以`@code`字体打印条目。请参阅10.7 节[Defining New Indices], 84 页。

@defcv *category*(类别) *class*(类) *name*(名称)

@defcvx *category*(类别) *class*(类) *name*(名称)

在面向对象编程中，规范与类相关的变量的描述。该命令接受三个参数：所定义事物的类别、它所属的类以及其名称。请参阅第 13 章[Definition Commands], 105 页。

@deffn *category*(类别) *name*(名称) *arguments*(参数)...

@deffnx *category*(类别) *name*(名称) *arguments*(参数)...

为函数、交互式命令或可能接受参数的类似实体编写描述。`@deffn`接受的参数包括所描述实体的类别、该特定实体的名称以及其参数（如果有）。请参阅第 13 章[Definition Commands], 105 页。

@defindex 索引名称(*index-name*)

定义一个新的索引及其索引命令。以罗马字体打印条目。请参阅10.7 节[Defining New Indices], 84 页。

@definfoenclose *newcmd*(新命令), *before*(前缀), *after*(后缀)

为在线格式创建一个新命令`@newcmd`，该命令通过在文本前后加上前缀、后缀来标记文本。请参阅16.6 节[`@definfoenclose`], 137 页。

@defivar 类(*class*) 实例变量名(*instance-variable-name*)

@defivarx 类(*class*) 实例变量名(*instance-variable-name*)

在面向对象编程中，规范为实例变量的描述。该命令等同于‘`@defcv {Instance Variable} ...`’。请参阅第 13 章[Definition Commands], 105 页。

@defline 类别(*category*) 名称(*name*) 参数(*arguments*)...

在`@defblock`环境中使用，为正在定义的符号提供标题原型行。此命令不会创建任何索引条目。请参阅13.7 节[Generic Definition Commands], 114 页。

@defmac *macroname*(宏名称) *arguments*(参数)...

@defmacx *macroname*(宏名称) *arguments*(参数)...

规范宏的描述；等效于‘`@deffn Macro ...`’。请参阅第 13 章[Definition Commands], 105 页。

@defmethod 类(*class*) 方法名(*method-name*) 参数(*arguments*)...

@defmethodx 类(*class*) 方法名(*method-name*) 参数(*arguments*)...

面向对象编程中，规范对方法的描述；相当于‘`@defop Method ...`’。请参阅第 13 章[Definition Commands], 105 页。

`@defop` 类别(*category*) 类(*class*) 名称(*name*) 参数(*arguments*)...

`@defopx` 类别(*category*) 类(*class*) 名称(*name*) 参数(*arguments*)...

在面向对象编程中, 规范对操作(OP)的描述。`@defop`接收操作的类别、操作的类名称、操作名称及其参数(如果有)。请参阅第 13 章[Definition Commands], 105 页和13.6.6 节[Abstract Objects], 112 页。

`@defopt` 选项名(*option-name*)

`@defoptx` 选项名(*option-name*)

规范对用户选项的描述; 等效于‘`@defvr {User Option} ...`’。请参阅第 13 章[Definition Commands], 105 页。

`@defspecc` 特殊形式名称(*special-form-name*) 参数(*arguments*)...

`@defspeccx` 特殊形式名称(*special-form-name*) 参数(*arguments*)...

规范对特殊形式的描述; 等效于‘`@deffn {Special Form} ...`’。请参阅第 13 章[Definition Commands], 105 页。

`@deftp` 类别(*category*) 类型名称(*name-of-type*) 属性(*attributes*)...

`@deftpx` 类别(*category*) 类型名称(*name-of-type*) 属性(*attributes*)...

规范对数据类型的描述; 其参数包括类别、类型名称(例如, ‘`int`’) 以及该类型对象的属性名称。请参阅第 13 章[Definition Commands], 105 页, 以及13.6.5 节[Data Types], 111 页。

`@deftypecv` 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*)

`@deftypecvx` 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*)

在面向对象编程中, 规范对类型化的类变量的描述。请参阅第 13 章[Definition Commands], 105 页, 以及13.6.6 节[Abstract Objects], 112 页。

`@deftypefn` 类别(*category*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

`@deftypefnx` 类别(*category*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

规范对可能带有参数且已类型化的函数或类似实体的描述。`@deftypefn`接受被描述实体的类别、(返回) 类型、实体名称及其参数(如果有) 作为参数。请参阅第 13 章[Definition Commands], 105 页。

`@deftypefnnewline` *on-off*

指定是否将`@deftypefn`及类似代码的返回类型单独打印成一行; 默认为关闭(off)。请参阅13.6.2 节[Typed Functions], 108 页。

`@deftypefun` 数据类型(*data-type*) 函数名(*function-name*) 参数(*arguments*)...

`@deftypefunx` 数据类型(*data-type*) 函数名(*function-name*) 参数(*arguments*)...

在类型化语言中, 规范对函数的描述。该命令等效于‘`@deftypefn Function ...`’。请参阅第 13 章[Definition Commands], 105 页。

`@deftypeivar` 类(*class*) 数据类型(*data-type*) 变量名(*variable-name*)

`@deftypeivarx` 类(*class*) 数据类型(*data-type*) 变量名(*variable-name*)

在面向对象编程中, 规范对类型化实例变量的描述。请参阅第 13 章[Definition Commands], 105 页, 以及13.6.6 节[Abstract Objects], 112 页。

`@deftypeline` 类别(*category*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

在`@defblock`环境中使用, 为正在定义的符号提供带有数据类型的标题原型行。此命令不会创建任何索引条目。请参阅13.7 节[Generic Definition Commands], 114 页。

@deftypemethod 类(*class*) 数据类型(*data-type*) 方法名(*method-name*) 参数(*arguments*)...

@deftypemethodx 类(*class*) 数据类型(*data-type*) 方法名(*method-name*) 参数(*arguments*)...

在面向对象编程中, 规范对类型化方法的描述。请参阅第 13 章[Definition Commands], 105 页。

@deftypeop 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

@deftypeopx 类别(*category*) 类(*class*) 数据类型(*data-type*) 名称(*name*) 参数(*arguments*)...

在面向对象编程中, 规范对类型化操作(OP)编写描述。请参阅第 13 章[Definition Commands], 105 页, 以及13.6.6 节[Abstract Objects], 112 页。

@deftypevar 数据类型(*data-type*) 变量名(*variable-name*)

@deftypevarx 数据类型(*data-type*) 变量名(*variable-name*)

在类型化语言中, 规范对变量的描述。该命令等效于‘@deftypevr Variable) ...’。请参阅第 13 章[Definition Commands], 105 页。

@deftypevr 类别(*category*) 数据类型(*data-type*) 名称(*name*)

@deftypevrx 类别(*category*) 数据类型(*data-type*) 名称(*name*)

类型化语言中, 规范对变量(一种记录值的实体)及类似对象的描述。该函数接受所描述实体的类别、类型和实体名称作为参数。请参阅第 13 章[Definition Commands], 105 页。

@defun 函数名(*function-name*) 参数(*arguments*)...

@defunx 函数名(*function-name*) 参数(*arguments*)...

规范对函数描述; 等效于‘@deffn Function ...’。请参阅第 13 章[Definition Commands], 105 页。

@defvar 变量名(*variable-name*)

@defvarx 变量名(*variable-name*)

规范对变量的描述; 等效于‘@defvr Variable ...’。请参阅第 13 章[Definition Commands], 105 页。

@defvr 类别(*category*) 名称(*name*)

@defvrx 类别(*category*) 名称(*name*)

规范对任何种类的变量的描述。@defvr接受实体类别和实体名称作为参数。请参阅第 13 章[Definition Commands], 105 页。

@detailmenu

在主控菜单中标记(可选的)详细节点列表。请参阅2.10.1 节[Master Menu Parts], 20 页。

@dfn{term}

标示术语的引入或定义性使用。请参阅6.1.12 节[@dfn], 53 页。

@DH{}

@dh{} 分别生成冰岛语字母‘eth’的大写和小写形式: Ð、ð。请参阅11.4 节[Inserting Accents], 91 页。

- @dircategory *dirpart***
为手册指定一个类别¹。请参阅2.6 节[Directory Category], 14 页。
- @direntry**
开始此文件的 Info 目录 (dir)菜单条目²。与**@end direntry**配对。请参阅20.1.4 节[Installing Dir Entries], 173 页。
- @display** 开始一个示例。类似于**@example** (缩进文本, 无需填充), 但不选择新的字体。与**@end display**配对。请参阅7.7 节[@display], 62 页。
- @displaymath**
将一段数学内容以“display”格式进行格式化。请参阅 11.7 节[Inserting Math], 93 页。
- @dmn{度量单位}**
格式化一个度量单位, 如: 12 pt。请参阅11.3.5 节[@dmn], 91 页。
- @docbook** 完全进入 DocBook 环境, 与**@end docbook**配对。请参阅15.3 节[Raw Formatter Commands], 123 页。
- @documentdescription**
设置文档描述文本, 该文本将包含在 HTML 输出中。与**@end documentdescription**配对。请参阅21.4 节[@documentdescription], 181 页。
- @documentencoding *enc***
声明文档的输入编码为*enc*。请参阅14.2 节[@documentencoding], 119 页。
- @documentlanguage *CC***
依据 ISO-639, 用两个字符缩写*CC* 来声明文档的语言。请参阅14.1 节[@documentlanguage], 118 页。
- @dotaccent{*c*}**
在字符*c* 上生成一个点重音符号, 如ò 所示。请参阅11.4 节[Inserting Accents], 91 页。
- @dotless{*i* 或 *j*}**
生成去掉点的‘i’ (‘i’) 及‘j’ (‘j’)。请参阅11.4 节[Inserting Accents], 91 页。
- @dots{}** 生成一个省略号, ‘...’。请参阅11.8.4 节[@dots], 95 页。
- @email{地址[, 显示文本]}**
标示一个电子邮件地址。请参阅6.1.16 节[@email], 55 页。
- @emph{文本(*text*)}**
强调*text*。请参阅6.2 节[Emphasizing Text], 55 页。
- @end 环境(*environment*)**
结束某一个环境(*environment*), 如‘@end example’。请参阅[@-commands], 8 页。
- @enddots{}**
生成一个句末省略号, 像这样: ... 请参阅11.8.4 节[@dots], 95 页。

¹ 译者注: 即手册的 top 节点在 dir 文件中类别。

² 译者注: 这些条目, 会被合并到 dir 文件中, 成为它的菜单条目。

- @enumerate** [数字或字母(*number-or-letter*)]
开始一个编号列表，并使用@item表示每个条目。可选地，以数字或字母 开始每个条目。@end enumerate与之配对。请参阅8.3 节[@enumerate], 69 页。
- @env{环境变量(*environment-variable*)}**
标识一个环境变量名，例如：PATH。请参阅6.1.8 节[@env], 53 页。
- @equiv{}** 生成符号‘≡’，用于向读者表明（该符号前后的）两种形式完全等价。请参阅11.9.6 节[@equiv], 98 页。
- @error{}** 生成标志符号‘`error`’，用于向读者表明之后文本是错误消息。请参阅11.9.5 节[@error], 97 页。
- @errormsg{msg}**
将msg 作为错误报告到标准错误输出，并退出程序，且退出失败。msg 中的Texinfo 命令会被展开为纯文本。请参阅第 15 章[Conditionals], 121 页和16.7 节[External Macro Processors], 137 页。
- @euro{}** 生成欧元货币符号‘€’。请参阅11.8.6 节[@euro], 95 页。
- @evenfooting** [*left*] @| [*center*] @| [*right*]
@evenheading [*left*] @| [*center*] @| [*right*]
对偶数(左侧)页的页脚或页眉的具体说明。请参阅节 E.2.3 [Custom Headings], 260 页。
- @everyfooting** [*left*] @| [*center*] @| [*right*]
@everyheading [*left*] @| [*center*] @| [*right*]
为每一页指定页脚或页眉。与 Info 无关。请参阅节 E.2.3 [Custom Headings], 260 页。
- @example** 开始一个示例环境。在环境中，缩进文本，不填充，并选择等宽字体。@end example与之配对使用，表示结束示例环境。@example接受可选参数，用逗号分隔。建议将第一个参数设置为示例环境中代码的语言。请参阅7.4 节[@example], 60 页。
- @exampleindent indent**
类似示例(@example)环境,使用indent 个空格(可能为 0)缩进，@end exampleindent与之配对使用，表示结束示例环境。请参阅节 E.5 [exampleindent], 263 页。
- @exclamdown{}**
生成一个倒置的感叹号‘¡’。请参阅11.4 节[Inserting Accents], 91 页。
- @exdent** 文本行(*line-of-text*)
删除文本行前可能存在的任何缩进。请参阅7.9 节[@exdent], 62 页。
- @expansion{}**
生成特殊符号‘↪’，向读者标识：后面内容是宏展开的结果。请参阅11.9.3 节[@expansion], 96 页。
- @file{文件名(*filename*)}**
突出显示文件、缓冲区、节点³、目录等的名称。请参阅6.1.9 节[@file], 53 页。

³ 译者注：指的的不是 Texinfo 的节点，是文件系统的节点。

- @finalout** 阻止 T_EX 在过宽行的边上打印出用于警告的大黑色矩形。请参阅 18.5 节 [Overfull hboxes], 146 页。
- @index** 条目 (*entry*) 将 *entry* 添加到函数索引中。请参阅 10.4 节 [Index Entries], 82 页。
- @firstparagraphindent** *word* 根据变量 *word* (其值可为 'none' (无) 或 'insert' (插入)), 控制章节标题后第一段的首行是否缩进。请参阅节 E.4 [firstparagraphindent], 263 页。
- @float** 定义浮动素材的环境。与 @end float 配对。请参阅 9.1 节 [Floats], 74 页。
- @flushleft**
@flushright 不要填充文本; 每行左对齐 (右对齐), 同时保持右端 (左端) 不齐。字体保持不变。与 @end flushleft (@end flushright) 配对。请参阅 7.10 节 [flushleft flushright], 63 页。
- @fonttextsize** *10-11* 调整打印输出中正文字体的大小。请参阅 6.2.3 节 [Fonts], 56 页。
- @footnote**{脚注文本 (*text-of-footnote*)} 加入脚注, 用于提供对正文进行记录或阐释的参考资料。在打印输出中, 脚注文本会打印在页面底部。在其他格式中, 脚注文本可以输出在同一节点中、单独的节点中, 或者仅标示为脚注文本。请参阅 9.3 节 [Footnotes], 78 页。
- @footnotestyle** 样式 (*style*) 指定脚注样式, 对于节点尾部样式使用 'end', 对于独立样式使用 'separate'。在独立样式中, 脚注会被放置在一个单独的节点或文件中。请参阅 9.3.2 节 [Footnote Styles], 79 页。
- @format** 开始一个示例环境。类似于 @display, 但不需要缩进。与 @end format 配对。请参阅 7.4 节 [example], 60 页。
- @frenchspacing** *on-off* 控制标点后的间距。请参阅 11.3.4 节 [frenchspacing], 90 页。
- @ftable** 格式化命令 (*formatting-command*) 开始一个两列的表格, 每个条目使用 @item。将第一列中的每个条目自动输入到函数索引中。与 @end ftable 配对。除了索引功能外, 其他与 @table 相同。请参阅 8.4.2 节 [ftable vtable], 71 页。
- @geq{}** 生成一个大于或等于符号, '≥'。请参阅 11.8.10 节 [geq leq], 96 页。
- @group** 禁止在以下文本中换页。与 @end group 配对。在 Info 输出中被忽略。请参阅 12.9 节 [group], 103 页。

- `@guillemotleft{}`
`@guillemotright{}`
`@guillemotleft{}`
`@guillemotright{}`
`@guilsinglleft{}`
`@guilsinglright{}`
 双角引号和单角引号: ‘`«`’ ‘`»`’ ‘`<`’ ‘`>`’。 `@guillemotleft` 和 `@guillemotright` 是 `@guillemetleft` 和 `@guillemetright` 的同义词。请参阅11.5 节[Inserting Quotation Marks], 92 页。
- `@H{c}` 在 *c* 上生成匈牙利长元音变音符号, 如 *ó*。
- `@hashchar{}`
 插入一个散列字符‘#’; 仅当字面散列字符用于引入 `#line` 指令时才需要使用。请参阅11.1.5 节[Inserting a Hashsign], 87 页, 以及16.7 节[External Macro Processors], 137 页。
- `@heading` 标题(*title*)
 打印一个类似章节的未编号标题, 但在目录中省略。请参阅4.7 节[`@unnumberedsec` `@appendixsec` `@heading`], 36 页。
- `@headings` *on*(打开)/*off*(关闭)/*single*(单面)/*double*(双面)
 打开或关闭页眉, 和(或)指定单面或双面打印页眉。请参阅节 E.2.1 [`@headings`], 259 页。
- `@headitem`
 在多列表格(`@multitable`)中的开始一个标题行。请参阅8.5.2 节[Multitable Rows], 72 页。
- `@headitemfont{text}`
 设置 *text* 为多列表格(`@multitable`)标题行的字体; 这在多表格模板中非常有用。请参阅8.5.2 节[Multitable Rows], 72 页。
- `@html` 进入 HTML 环境。与 `@end html` 配对。请参阅15.3 节[Raw Formatter Commands], 123 页。
- `@hyphenation{已断字单词(hy-phen-a-ted words)}`
 显式定义断点。请参阅12.3 节[`@- @hyphenation`], 102 页。
- `@i{text}` 在可能的情况下, 将 *text* 设置为 *italic* 字体。请参阅6.2.3 节[Fonts], 56 页。
- `@ifclear txivar`
 如果 Texinfo 变量 *txivar* 未设置, 则格式化以下文本。与 `@end ifclear` 配对。请参阅15.5 节[`@set @clear @value`], 124 页。
- `@ifcommanddefined txi 命令(txicmd)`
`@ifcommandnotdefined txi 命令(txicmd)`
 如果 Texinfo 代码‘`@txi 命令(txicmd)`’(未) 定义, 则格式化以下文本。与相应的 `@end ifcommand...` 配对。请参阅15.6 节[Testing for Texinfo Commands], 128 页。

`@ifdocbook`

`@ifhtml`

`@ifinfo`

`@iflatex`

`@ifplaintext`

`@ifxml` 之后的文本，仅在特定输出格式中出现。出于历史兼容性考虑，`@ifinfo`输出同时出现在 Info 和纯文本(plaintext)输出中。请分别与`@end ifdocbook`、`@end ifhtml ...` 配对。请参阅第 15 章[Conditionals], 121 页。

`@ifnotdocbook`

`@ifnohtml`

`@ifnotlatex`

`@ifnotplaintext`

`@ifnottex`

`@ifnotxml`

之后的文本，在特定输出格式中忽略，但在其他输出格式中不会忽略。`@ifnohtml`在 HTML 输出格式中被省略，其它命令是类似作用。需与相应的`@end ifnot...`配对。请参阅第 15 章[Conditionals], 121 页。

`@ifnotinfo`

之后的文本，在 Info 和纯文本（出于历史兼容性考虑）输出格式中被忽略。与`@end ifnotinfo`配对使用。请参阅第 15 章[Conditionals], 121 页。

`@ifset txivar`

如果 Texinfo 变量`txivar` 已设置，则格式化之后的文本。与`@end ifset`配对。请参阅15.5 节[`@set @clear @value`], 124 页。

`@iftex`

之后的文本仅在 $\TeX$ 输出格式中被处理。需与`@end iftex`配对。请参阅第 15 章[Conditionals], 121 页。

`@ignore`

开始一段不会出现在任何输出中的文本。与`@end ignore`配对。请参阅2.2 节[Comments], 8 页。

`@image{文件名(filename), [宽度(width)], [高度(height)], [替代文本(alt)], [扩展名(ext)]}`

引入外部的图形图像文件，文件名是`filename`，图像将根据给定的`width` 和(或) `height` 进行缩放，使用`alt` 文本，在 HTML 输出中用‘`filename.ext`’查找文件。请参阅9.2 节[Images], 76 页。

`@include filename`

将 Texinfo 源文件`filename` 的内容引入此处。请参阅第 17 章[Include Files], 140 页。

`@indent`

插入段落缩进。请参阅7.13 节[`@indent`], 65 页。

`@indentedblock`

对任意文本块进行左侧缩进。与`@end indentedblock`配对使用。请参阅7.3 节[`@indentedblock`], 60 页。

`@indicateurl{text}`

标示文本是万维网的统一资源定位符。请参阅6.1.15 节[`@indicateurl`], 55 页。

- @inforef**{节点名(*node-name*), [条目名(*entry-name*)], info 文件名(*info-file-name*)}
- 对没有打印手册的 Info 文件, 进行交叉引用 (直接引用 info 文件)。请参阅5.13节[@inforef], 45 页。
- @inlinefmt**{格式(*fmt*), 文本(*text*)}
- 仅当输出格式为*fmt* 时, 才插入*text*。请参阅15.4 节[Inline Conditionals], 123 页。
- @inlinefmtifelse**{格式(*fmt*), 文本(*text*), 否则文本(*else-text*)}
- 如果输出格式为*fmt* , 则插入*text* , 否则插入*else-text* 。
- @inlineifset**{变量(*var*), 文本(*text*)}
- @inlineifclear**{变量(*var*), 文本(*text*)}
- 仅当 Texinfo 变量*var* (未)设置时, 才插入*text* 。
- @inlineraw**{格式(*fmt*), 原始文本(*raw-text*)}
- 仅当输出格式为*fmt* 时, 才插入原始文本*raw-text* 。
- \input** 宏定义文件(*macro-definitions-file*)
- 使用指定的宏定义文件。此命令仅在 Texinfo 文件的第一行使用, 以使 T_EX使用texinfo⁴宏定义文件。在\input中使用\而不是@, 因为 T_EX在读取定义文件之前无法识别@。请参阅2.5 节[Texinfo File Header], 11 页。
- @insertcopying**
- 插入之前使用@copying环境定义的文本。请参阅2.7.2 节[@insertcopying], 15 页。
- @item**
- 为@itemize和@enumerate指示一个条目文本的开始; 为@table、@ftable和@vtable指示条目的第一列文本的开始。请参阅第 8 章[Lists and Tables], 67 页。
- @itemize** 标记生成字符或命令(*mark-generating-character-or-command*)
- 开始一个无序列表: 在每个项目开头的左边缘内使用一个标记 (如: @bullet) 缩进段落。与@end itemize配对。请参阅8.2 节[@itemize], 67 页。
- @itemx**
- 类似于@table、@ftable和@vtable中的@item, 但不在条目文本上方生成额外的垂直间距。因此, 当多个条目具有相同的描述时, 对第一个条目使用@item, 对其他条目使用@itemx。请参阅8.4.3 节[@itemx], 71 页。
- @kbd**{键盘字符(*keyboard-characters*)}
- 标示用户需输入的键盘字符。请参阅6.1.3 节[@kbd], 50 页。
- @kbinputstyle** 样式(*style*)
- 根据*style* 的值 (‘code’(代码), ‘distinct’(独特), ‘example’(示例)), 指定@kbd何时应使用与@code不同的字体。请参阅6.1.3 节[@kbd], 50 页。
- @key**{键名(*key-name*)}
- 标示键盘上某个键的名称。请参阅6.1.4 节[@key], 50 页。
- @kindex** *entry*
- 将*entry* 添加到键索引中。请参阅10.4 节[Index Entries], 82 页。
- @L{}**
- @l{}**
- 分别生成波兰语中大写和小写的‘被删去的-L’字母: Ł、ł。

⁴ 译者注: 本手册是中文版, 所以使用texinfo_zh, 而不是texinfo。

- @LaTeX{}** 生成 L^AT_EX 标志。请参阅 11.8.1 节 [TeX @LaTeX], 94 页。
- @latex** 彻底进入 L^AT_EX 环境。与 **@end latex** 配对。请参阅 15.3 节 [Raw Formatter Commands], 123 页。
- @leq{}** 生成一个小于或等于符号, ‘≤’。请参阅 11.8.10 节 [geq @leq], 96 页。
- @leq{}**
- @linemacro** 宏名 (*macroname*) {参数 (*params*)}
定义一个新的宏, 该宏将该行的剩余部分作为参数, 并扩展为包含多行的完整代码。请参阅 16.5 节 [Line Macros], 136 页。
- @link{** 节点名 (*nodename*), 标签 (*label*), 手册名 (*manual-name*)
创建一个没有可见标记或页面引用的普通链接。请参阅 5.12 节 [link], 45 页。
- @lisp** 开始一个 Lisp 代码示例。文本需缩进, 不要填充, 并选择等宽字体。与 **@end lisp** 配对。请参阅 7.6 节 [lisp], 61 页。
- @listoffloats**
生成一个类似目录的 **@float** 列表。请参阅 9.1.3 节 [listoffloats], 75 页。
- @lowersections**
将后续章更改为节, 将节更改为小节, 依此类推。请参阅 4.12 节 [raisesections 和 @lowersections], 38 页。
- @macro** 宏名 (*macroname*) {参数 (*params*)}
定义一个新的 TeXinfo 命令 **@macroname{params}**。与 **@end macro** 配对。请参阅 16.1 节 [Defining Macros], 130 页。
- @majorheading** 标题 (*title*)
打印一个类似章但未编号的标题, 但不在目录中出现。会在标题前生成比 **@chapheading** 命令更多的垂直空白。请参阅 4.5 节 [majorheading @chapheading], 36 页。
- @math{** 数学表达式 (*mathematical-expression*)
格式化数学表达式。请参阅 11.7 节 [Inserting Math], 93 页。
- @menu** 标记节点的菜单的开始。在打印手册中无效果。与 **@end menu** 配对。请参阅 3.9 节 [Menus], 30 页。
- @microtype** *on-off*
打开或关闭微排功能。请参阅 节 E.9 [Microtypography], 264 页。
- @minus{}** 生成一个减号, ‘−’。请参阅 11.8.9 节 [minus], 95 页。
- @multitable** 列宽指定 (*column-width-spec*)
开始一个多列表格。每行以 **@item** 或 **@headitem** 开始, 列之间用 **@tab** 分隔。与 **@end multitable** 配对。请参阅 8.5.1 节 [Multitable Column Widths], 72 页。
- @need *n*** 如果当前页面剩余的空白少于 *n* mils (密耳, 千分之一英寸), 则在打印手册中另起一页。请参阅 12.10 节 [need], 103 页。
- @node** 名称 (*name*), [下一个 (*next*)], [上一个 (*previous*)], [上一级 (*up*)]
开始一个新的节点。只有第一个参数是必填的。请参阅 3.1 节 [Writing a Node], 24 页。

@nodedescription 节点描述(*node-description*)

节点描述, 简要阐述节点的用途。请参阅3.8 节[Node Descriptions], 29 页。

@nodedescriptionblock

用于开始一个较长的节点描述。与‘@end nodedescriptionblock’配对。请参阅3.8 节[Node Descriptions], 29 页。

@noindent

防止文本像新段落一样缩进。请参阅7.12 节[@noindent], 64 页。

@novalidate

在使用 TeX 时, 禁止验证节点引用并省略创建辅助文件。请在任何章节命令或交叉引用命令之前使用。请参阅[Pointer Validation], 151 页。

@O{}

@o{} 分别生成大写和小写的带斜杠的 O 字母: Ø、ø。

@oddfooting [左对齐(*left*)] @ [居中(*center*)] @ [右对齐(*right*)]

@oddheading [左对齐(*left*)] @ [居中对齐(*center*)] @ [右对齐(*right*)]

对奇数(右边) 页的页脚和页眉的具体说明。请参阅节 E.2.3 [Custom Headings], 260 页。

@OE{}

@oe{} 分别生成大写和小写的 OE 连字: Œ、œ。请参阅11.4 节[Inserting Accents], 91 页。

@ogonek{c}

在 *c* 下方添加一个 ogonek 变音符, 如: ą。请参阅11.4 节[Inserting Accents], 91 页。

@option{选项名称(*option-name*)}

标示一个命令行选项, 如 -l 或 --help。请参阅6.1.11 节[@option], 53 页。

@ordf{}

@ordm{} 分别生成西班牙语中的阴性和阳性序数词: º、º。请参阅11.4 节[Inserting Accents], 91 页。

@page 在打印手册中另起一页。请参阅12.8 节[@page], 103 页。

@pagesizes [宽度(*width*)] [, 高度(*height*)]

更改页面尺寸。请参阅[pagesizes], 264 页。

@paragraphindent 缩进量(*indent*)

使用 *indent* 个空格(可能为 0) 缩进段落; 如果 *indent* 为 *asis*, 则保留源文件的缩进。请参阅节 E.3 [@paragraphindent], 262 页。

@part 标题(*title*)

开始一组章或附录; 包含在目录中。请参阅4.11 节[@part], 37 页。

@pindex entry

将 *entry* 添加到程序(programs)索引中。请参阅10.4 节[Index Entries], 82 页。

@point{} 使用符号‘*’ 向读者标识缓冲区中的点的位置: 。请参阅11.9.7 节[@point], 98 页。

@pounds{}

生成英镑货币符号, ‘£’。请参阅11.8.7 节[@pounds], 95 页。

`@print{}` 生成‘¬’符号，用于向读者标示打印输出。请参阅11.9.4 节[`@print`], 97 页。

`@printindex` 索引名称(*index-name*)
生成名为*index-name* 的索引。请参阅10.5 节[Printing Indices & Menus], 83 页。

`@pxref{节点(node), [条目(entry)], [节点标题(node-title)], [Info 文件(info-file)], [打印手册(printed-manual)]}`
在括号内使用引用。在打印手册中，以小写的‘see’开头。除引用整本手册外，第一个参数是必需的。若要引用另一本完整的手册，则仅提供*printed-manual*和(或)*info-file*参数。请参阅5.10 节[`@pxref`], 44 页。

`@questiondown{}`
生成一个倒置的问号，¿。请参阅11.4 节[Inserting Accents], 91 页。

`@quotation`
缩小边距以表示文本是从另一作品中引用的。可选参数用于指定前缀文本。与`@end quotation`配对。请参阅7.2 节[`@quotation`], 59 页。

`@quotedblleft{}`

`@quotedblright{}`

`@quoteleft{}`

`@quoteright{}`

`@quotedblbase{}`

`@quotesinglbase{}`

生成各种引号：“ ” ‘ ’ „ „。请参阅11.5 节[Inserting Quotation Marks], 92 页。

`@r{text}` 如果可能的话，将对*text*使用常规的roman字体。请参阅6.2.3 节[Fonts], 56 页。

`@raggedright`
填充文本；每行左对齐，右端留白。字体保持不变。与`@end raggedright`配对。请参阅7.11 节[`@raggedright`], 63 页。

`@raisesections`
将后续的节更改为章，将子节更改为节，依此类推。请参阅4.12 节[Raise/lower sections], 38 页。

`@ref{节点(node), [条目(entry)], [节点标题(node-title)], [Info 文件(info-file)], [打印手册(printed-manual)]}`
制作一个不以任何特殊文本开头的普通引用。在命令后加上标点符号。除了引用整个手册外，第一个参数是必需的。若要引用另一本完整的手册，则*printed-manual*和(或) *info-file* 是唯一必需的参数。请参阅5.9 节[`@ref`], 43 页。

`@registeredsymbol{}`
生成符合法律规定的注册符号®。请参阅11.8.3 节[`@registeredsymbol`], 95 页。

`@result{}`
生成特殊符号⇒。用于向读者标示表达式的结果。请参阅11.9.2 节[`@result`], 96 页。

`@ringaccent{c}`
在*c* 上生成一个环状重音，如: ö。请参阅11.4 节[Inserting Accents], 91 页。

`@samp{text)}`
text (包含一串字符)的字面示例。请参阅6.1.5 节[`@samp`], 51 页。

- @sansserif{*text*}**
如果可能的话, 将对 $text$ 使用无衬线 字体。请参阅6.2.3 节[Fonts], 56 页。
- @sc{*text*}** 如果可能的话, 对 $text$ 使用小号大写(small caps)字体, 在 Info 输出中仅作大写处理。请参阅6.2.2 节[Smallcaps], 56 页。
- @section** 标题(*title*)
在章中开始一个节。节标题会出现在目录中。在@chapter和@appendix中, 节标题会被编号; 在@unnumbered中, 节则不会编号。请参阅4.6 节[@section], 36 页。
- @seealso{索引文本(*index-text*)}**
在索引条目中使用, 以指引读者参考另一个相关的索引条目。请参阅10.3 节[Advanced Indexing], 81 页。
- @seeentry{索引文本(*index-text*)}**
在索引条目中使用, 以指引读者至另一个索引条目。请参阅10.3 节[Advanced Indexing], 81 页。
- @set txivar [*string*]**
定义 Texinfo 变量 $txivar$, (可选)将其值设置为 $string$ 。请参阅15.5 节[@set @clear @value], 124 页。
- @setchapternewpage on-off-odd(升-关-奇数)**
指定章是否从新页面开始, 如果是, 则再指定是否从奇数(右手边)新页面开始。请参阅节 E.1 [@setchapternewpage], 258 页。
- @setfilename Info 文件名(*info-file-name*)**
为输出文件提供一个名称。对于 TeX格式化, 此命令将被忽略。请参阅2.5.2 节[@setfilename], 12 页。
- @settitle** 标题(*title*)
为打印手册的页眉(page headers)指定(文档)标题, 并为 HTML 指定默认文档标题。请参阅2.5.3 节[@settitle], 12 页。
- @shortcaption**
为@float定义简短标题。请参阅9.1.2 节[@caption @shortcaption], 75 页。
- @shortcontents**
打印一个简要目录, 仅包含章级别的条目; 或规定必须(在合适位置)输出一个简要目录。适用于可以输出简要目录的格式。请参阅2.9 节[Contents], 19 页。
- @shorttitlepage** 标题(*title*)
生成一个最小化的标题页。请参阅2.8.1 节[@titlepage], 16 页。
- @slanted{*text*}**
如果可能的话, 对 $text$ 使用 $slanted$ 字体。请参阅6.2.3 节[Fonts], 56 页。
- @smallbook**
在打印输出时, 请使用 7 英寸乘 9.25 英寸的格式, 而非常规的 8.5 英寸乘 11 英寸的格式。请参阅节 E.6 [@smallbook], 263 页。
- @smalldisplay**
开始一个示例。类似于@display, 但尽可能使用较小的字体大小。与@end smalldisplay配对使用。请参阅7.15 节[small], 66 页。

@smallexample

开始一个示例。类似于@example，但尽可能使用较小的字体。与@end smallexample配对。请参阅7.15节[small], 66页。

@smallformat

开始一个示例。类似于@format，但尽可能使用较小的字体大小。与@end smallformat配对。请参阅7.15节[small], 66页。

@smallindentedblock

类似于@indentedblock，但尽可能使用较小的字体。与@end smallindentedblock配对。请参阅7.15节[small], 66页。

@smalllisp

开始一个Lisp代码示例。与@smallexample相同。需与@end smalllisp配对。请参阅7.15节[small], 66页。

@smallquotation

类似于@quotation，但尽可能使用较小的字体。与@end smallquotation配对。请参阅7.15节[small], 66页。

@sortas {key}

在索引命令的参数中使用，提供一个字符串key，用于对索引条目进行排序。请参阅10.2节[Indexing Commands], 80页。

@sp *n* 跳过*n*行空白行。请参阅12.7节[@sp], 103页。

@ss{} 生成德语中的尖S(es-zet)字母ß。请参阅11.4节[Inserting Accents], 91页。

@strong {text}

对text，比使用@emph更强的强调。请参阅[emph & strong], 56页。

@sub {text}

将text设置为下标。请参阅11.6节[Inserting Subscripts and Superscripts], 93页。

@subentry

在索引条目中用于分隔多级条目的各个部分。请参阅10.3节[Advanced Indexing], 81页。

@subheading title(标题)

打印一个类似子节的未编号标题，但不出现在打印手册的目录中。请参阅4.9节[@unnumberedsubsec @appendixsubsec @subheading], 37页。

@subsection title(标题)

在某个节内开始一个子节。子节标题会出现在目录中。其编号方式与@section相同，且依赖于上下文。请参阅4.8节[@subsection], 36页。

@subsubheading 标题(title)

打印一个类似子子节的未编号标题，但不出现在打印手册的目录中。请参阅4.10节[@subsubsection], 37页。

@subsubsection 标题(title)

在子节中开始一个子子节。子子节标题会出现在目录中。其编号方式与@section相同，且依赖于上下文。请参阅4.10节[@subsubsection], 37页。

- @subtitle** 标题(*title*)
为标题页(由@titlepage ... @end titlepage 定义)设置一个副标题。请参阅2.8.2节[@title @subtitle @author], 17 页。
- @summarycontents**
打印或指定一个简要目录。这是@shortcontents的同义词。请参阅2.9节[Contents], 19 页。
- @sup {text}**
将text 设置为上标。请参阅11.6节[Inserting Subscripts and Superscripts], 93 页。
- @syncodeindex from-index(源索引) to-index(目标索引)**
将第一个参数中指定的索引合并到第二个参数中指定的索引中, 并将第一个索引中的条目用@code格式化。请参阅10.6节[Combining Indices], 83 页。
- @synindex from-index(源索引) to-index(目标索引)**
将第一个参数中指定的索引合并到第二个参数中指定的索引中。不更改from-index 中条目的字体。请参阅10.6节[Combining Indices], 83 页。
- @t{text}** 如果可能的话, 请将text 设置为fixed-width, 类似打字机的字体。请参阅6.2.3节[Fonts], 56 页。
- @tab** 用于在多栏表格的行中进行列分隔。请参阅8.5.2节[Multitable Rows], 72 页。
- @table formatting-command(格式化命令)**
开始一个两栏表格(描述列表), 每个条目使用@item。将每个第一列条目与@item放在同一行。第一列条目使用formatting-command 产生的字体打印。与@end table配对。请参阅8.4节[Two-column Tables], 70 页。另请参阅8.4.2节[@ftable @vtable], 71 页, 和8.4.3节[@itemx], 71 页。
- @TeX{}** 生成 T_EX标志。请参阅11.8.1节[@TeX @LaTeX], 94 页。
- @tex** 全面进入 T_EX环境。与@end tex配对。请参阅15.3节[Raw Formatter Commands], 123 页。
- @textdegree{}**
生成度数符号‘°’。请参阅11.8.8节[@textdegree], 95 页。
- @thischapter**
@thischaptername
@thischapternum
@thissection
@thissectionname
@thissectionnum
@thisfile
@thispage
@thistitle
仅允许在页眉或页脚中使用。分别代表当前章的编号及名称(格式为: ‘第 1 章: 章标题’)、仅当前章名称、仅当前章编号、当前节的编号及名称、仅当前节名称、仅当前节编号、文件名、当前页码以及文档标题。请参阅 节 E.2.3 [Custom Headings], 260 页。
- @TH{}**
@th{} 分别生成大写和小写的冰岛语字母‘thorn’: Þ、þ。请参阅 11.4 节[Inserting Accents], 91 页。

- @tie{}** 生成一个不允许换行的正常词间空格。请参阅 12.6 节[@tie], 103 页。
- @tieaccent{cc}**
在连续的两个 *cc* 字符上加上‘tie-after’重音, 如: `@tieaccent{}` $\Rightarrow$ ‘*oo*’。请参阅 11.4 节[Inserting Accents], 91 页。
- @tindex entry**
将 *entry* 添加到数据类型索引中。请参阅 10.4 节[Index Entries], 82 页。
- @title 标题(title)**
为标题页设置标题。请参阅 2.8.2 节[@title @subtitle @author], 17 页。
- @titlefont{text(文本)}**
如果可能的话, 以比正常字体更大的字体打印 *text*。请参阅 2.8.3 节[@titlefont @center @sp], 17 页。
- @titlepage**
开始标题页。将命令单独占一行, 并配以 `@end titlepage`。在默认情况下, 在线格式中不会输出标题页。请参阅 2.8.1 节[@titlepage], 16 页。
- @today{}** 以‘1 Jan 1900’格式⁵插入当前日期。请参阅节 E.2.3 [Custom Headings], 260 页。
- @top 标题(title)**
在文件中最顶级的节点是‘Top’, 该节点定义行(`@node Top`)之后一行, 必须是 `@top` 命令。标题以章级标题的级别被格式化。在 $\text{T}_{\text{E}}\text{X}$ 中, `@top` 命令仅仅是 `@unnumbered` 的同义词。
- @U{hex}** 输出以 *U+hex* 形式表示的 Unicode 字符。请参阅 11.10 节[Inserting Unicode], 99 页。
- @u{c}**
- @ubaraccent{c}**
- @udotaccent{c}**
在字符 *c* 上方或下方, 加上短横线、下划线、下点重音符号。如: *ö*、*o̲*、*ọ*。请参阅 11.4 节[Inserting Accents], 91 页。
- @unmacro macroname**
如果宏 `@macroname` 已定义, 则取消其定义。请参阅 16.1 节[Defining Macros], 130 页。
- @unnumbered 标题(title)**
开始一个没有任何编号的章。标题出现在目录中。请参阅 4.4 节[@unnumbered @appendix], 35 页。
- @unnumberedsec 标题(title)**
开始一个没有任何编号的节。标题会出现在目录中。请参阅 4.7 节[@unnumberedsec @appendixsec @heading], 36 页。
- @unnumberedsubsec 标题(title)**
开始一个未编号的子节。标题会出现在目录中。请参阅 4.9 节[@unnumberedsubsec @appendixsubsec @subheading], 37 页。

⁵ 译者注: 可能与当前区域设置(locale)的日期格式有关。在 $\text{T}_{\text{E}}\text{X}$ 输出格式中, 输出中文日期格式‘1900 年 1 月 1 日’, 但 HTML 和 Info 输出格式中不支持中文, 输出‘Jan 1 1900’。

- @unnumberedsubsubsec** 标题 (*title*)
开始一个未编号的子子节。标题会出现在目录中。请参阅4.10 节[@subsubsection], 37 页。
- @uref{url[, 显示文本(*displayed-text*)] [, 替换(*replacement*)}**
@url {url[, 显示文本(*displayed-text*)] [, 替换(*replacement*)}
定义一个对外部统一资源定位符 (如: 万维网) 的交叉引用。请参阅5.14 节[@url], 45 页。
- @urefbreakstyle** 样式 (*style*)
指定@uref/@url在特殊字符处应如何换行: after(之后)、before(之前)、none(无)。请参阅5.14 节[@url], 45 页。
- @v{c}** 在字符*c* 上生成抑制(check)重音符号, 如: ö。请参阅11.4 节[Inserting Accents], 91 页。
- @value{txivar}**
如果存在之前通过@set定义的 Texinfo 变量*txivar* 的值, 则插入该值。请参阅15.5 节[@set @clear @value], 124 页。
- @var{元句法变量(*metasyntactic-variable*)}**
突出显示一个元句法变量, 该变量代表另一段文本。请参阅6.1.7 节[@var], 52 页。
- @verb{delim(分隔符) literal(文字) delim(分隔符)}**
原样输出*literal*, 以单个字符*delim* 分隔, 使用等宽字体。 *literal* 可以包括任意空白字符或 Texinfo 特殊字符。请参阅6.1.6 节[@verb], 51 页。
- @verbatim**
进入原样输出环境, 使用等宽字体输出。与@end verbatim配对使用。请参阅7.5 节[@verbatim], 61 页。
- @verbatiminclude filename(文件名)**
原样输出*filename* 中的内容, 使用等宽字体。请参阅17.3 节[@verbatiminclude], 141 页。
- @vindex entry(条目)**
将*entry* 添加到变量索引中。请参阅10.4 节[Index Entries], 82 页。
- @vskip amount**
在打印手册中, 插入空白以将页面剩余的文本推向页面底部。最常见也是最重要的用途: 使用参数‘Opt plus 1filll’ (注意‘filll’的拼写) 格式化版权页。请参阅2.8.4 节[Copyright], 18 页。
- @vtable 格式化命令(*formatting-command*)**
开始一个两列的表格, 每个条目使用@item。将第一列中的每个条目自动加入到变量索引中。与@end vtable配对。除了索引功能外, 其他功能与@table相同。请参阅8.4.2 节[@ftable @vtable], 71 页。
- @w{text(文本)}**
禁止在*text* 内换行。请参阅12.5 节[@w], 102 页。
- @xml** 进入 XML 环境。与@end xml配对使用。请参阅15.3 节[Raw Formatter Commands], 123 页。

`@xref{node(节点), [entry(条目)], [node-title(节点标题)], [info-file(Info 文件)], [printed-manual(打印手册)]}`

在印刷手册中，以‘See’开头的引用。命令后应跟上一个标点符号。除引用整本手册外，第一个参数是必填的。若要引用另一本完整的手册，则只需填写 `printed-manual` 和(或) `info-file` 这两个参数。请参阅5.8 节[`@xref`], 43 页。

`@xrefautomaticsectiontitle on-off`

默认情况下，在交叉引用中，包括在 HTML 的节点标题中的，是章节标题而不是节点名。请参阅5.5 节[Three Arguments], 41 页。

A.3 @ 命令的上下文

这里我们大致描述了哪些 @ 命令可以在哪些上下文中使用。这并非详尽无遗，也非旨在成为完整的参考指南。若此处的信息与 Texinfo 处理器的实现之间存在差异，则很可能要以实现为准进行解决。

在下面提到的普通文本(*general text*)，我们指的是：除区块分割命令(sectioning commands)及其他用于指定文档外部特性的一类命令(如`@section`、`@node`和`@setfilename`等)以外的任何内容。

`@c`、`@comment`、`@if ... @end if` 条件命令可以出现在任何位置(但条件语句本身必须位于单独的行上)。`@caption`和`@shortcaption`只能出现在`@float`中，但可以包含普通文本。`@footnote`的内容与此类似。

用于标示文本的带有大括号命令(如`@strong`、`@sc`、`@asis`)可以包含原始格式化命令(如`@html`)，但不能包含以`@end...`配对结尾的块命令 且不能跨段落分割，但可以包含普通文本。

除了块命令限制外，`@center`、`@exdent`、位于`@table...@end table`中的`@item`，也不接受对段落有意义的 @ 命令，如：`@indent`。

除上述情况外，区块分割命令(sectioning commands)中不能包含`@anchor`、`@footnote`或`@verb`。

除上述命令外，其它命令(`@node`、`@anchor`、`@printindex`、`@ref`、`@math`、`@cindex`、`@url`、`@image`等)不能包含交叉引用命令(`@ref`、`@xref`、`@pxref`和`@inforef`)。

为了获取准确且完整的信息，我们建议查看源代码中的测试套件，该套件会详尽地尝试各种组合。

A.4 过时的 @ 命令

以下是已过时或已完全移除的 Texinfo @ 命令。本节内容仅供历史参考。

`@refill` 这个命令曾用于在所有其他处理完成后对段落进行补全和缩进。由于现在所有格式化工具都会根据需要自动进行补全，因此这个命令已不再需要，但你仍可能在一些手册的源代码中看到它，因为它并无害处。

`@setcontentsaftertitlepage`

过去，目录命令(contents commands)有时会被放在文件末尾，位于所有索引之后，且刚好在`@bye`之前，但我们不再推荐这样做。用户打印手册时可以使用此命令，以强制目录在标题页之后(在`@end titlepage`行之后)打印，即使`@contents`命令位于手册末尾。

@setshortcontentsaftertitlepage

即使@shortcontents命令位于手册的末尾，该命令也会将简要目录放在‘@end titlepage’命令之后。

附录 B 提示与技巧

以下是编写 Texinfo 文档的一些小贴士：

- 用现在时态写，不要用过去时或将来时。
- 积极地写作！例如，写“我们建议...”而不是“建议...”。
- 使用 70 或 72 作为你的全列宽。过长的行难以阅读¹。
- 包含版权(copyright)声明和复制(copying)许可。
- 设计你的手册时，要尽可能确保它能按顺序阅读。人们厌倦了来回翻阅以查找他们需要的信息，而这些信息本应在他们需要时呈现给他们。

索引，索引，索引！

以不同的方式编写多个索引条目。读者喜欢索引：它们既有用又方便。

虽然编写索引条目最简单的方式是在编写正文时同时编写，但有些人更倾向于事后编写。无论哪种情况，都应在相关段落之前编写条目。这样，索引条目就能指向跨页段落的起始页。

以下是我们发现的一些与索引相关的有价值提示：

- 为每个索引条目编写不同的内容，以便每个条目都指向文档中的不同位置。
- 仅在重点讨论某个主题时编写索引条目。例如，在“报告错误”的节中编写“调试信息”索引条目是没有意义的。想要了解调试信息的人肯定不会在那章中找到。
- 每个概念索引条目的首词应统一大写，或者统一小写。简短的条目通常要求小写；较长的条目则要求大写。无论你使用哪种大小写约定，请统一使用其中一种！两种风格混用看起来很糟糕。
- 对于索引中应大写的单词，如国家名称或缩写，应始终大写或使用大写形式。对于大小写敏感的名称，如 C 或 Lisp 语言中的名称，应始终使用适当的大小写形式。
- 在章节命令后立即编写引用整个章节的索引命令，在段落之前编写引用该段落的索引命令。

在下述示例中，在“Leaping”的索引条目之后有一个空白行：

```
@section The Dog and the Fox
@cindex Jumping, in general
@cindex Leaping

@cindex Dog, lazy, jumped over
@cindex Lazy dog jumped over
@cindex Fox, jumps over dog
@cindex Quick fox jumps over dog
The quick brown fox jumps over the lazy dog.
```

（请注意，该示例展示了同一概念的不同表述方式——‘Lazy dog’和‘Dog, lazy’---以便读者可以通过不同方式查找该概念。）

空行

- 在章节命令与紧随其后的第一个句子或段落之间，或者在章节命令相关索引命令与紧随其后的第一个句子或段落之间，插入一个空行，如上面关于索引的提示。这样可以使源文件更易于阅读。

¹ 译者注：实际上，这条建议有些过时，对于新建的文档，没必要遵守。理由参见：[Convention], 312 页。

- 在`@table`命令前和`@end table`命令后，始终插入一个空行；但在`@table`命令后，则不要插入空行。

例如，

```
Types of fox:

@table @samp
@item Quick
Jump over lazy dogs.

@item Brown
Also jump over lazy dogs.
@end table

@noindent
On the other hand, ...
```

以相同的方式，在`@itemize...@end itemize`和`@enumerate...@end enumerate`的前后分别插入空行。

完整的短语

完整的短语比...更易于阅读

- 以完整的句子的形式在项目列表中编写条目，或者至少以完整的短语的形式。像这样的不完整表达...很尴尬...
- 为有多个条目的项目列表或表格编写完整的引导句或短语。不要写“你可以设置：”；而要写“你可以设置这些变量：”。前一种表达听起来不完整。

版次(Editions)、日期与版本(Versions)

在`@copying`文本（供阅读 Texinfo 文件的人查看，并包括用于输出文件的版权法律声明）中包含版次号、版本号和日期。然后在`@titlepage`部分使用`@insertcopying`，供阅读打印输出的人查看（请参阅[Short Sample], 9 页）。

使用`@set`和`@value`来处理此类版本信息最为简便。请参阅15.5.4 节[`@value Example`], 127 页，以及节 C.1 [GNU Sample Texts], 234 页。

定义命令

定义命令包括`@defn`、`@defun`、`@defmac`等，它们使您能够以统一的格式编写(函数、宏的)描述。

- 对于每个使用定义命令定义的实体，只需编写一个定义命令。自动索引功能会创建一个索引条目，引导读者找到定义。
- 在包含函数简介的附录中，使用`@table...@end table`，而不是`@defn`或其他定义命令。这是因为函数并不是在附录中定义的，附录中包含的仅是函数的汇总列表。

大写

- 将“Texinfo”(首字母)大写；它是一个专有名词。不要将‘x’和‘i’大写。
- 将“Info”(首字母)大写；它是一个专有名词。
- 使用`@TeX{}`命令来写 $\mathrm{T\!E\!X}$ 。注意：大写的‘T’和‘X’。此命令会按照 $\mathrm{T\!E\!X}$ 的创造者(Donald Knuth)的意愿来对这个名称进行排版。(对于 $\mathrm{L\!A\!T\!E\!X}$ ，同样使用`@LaTeX{}`。)

空格

在 Texinfo 文件中，除了在`@example...@end example`环境、其他字面环境及命令中，连续多个空格对格式化不起作用。

例如，T_EX源文件中包括下内容：

```
@kbd{C-x v}
@kbd{M-x vc-next-action}
  Perform the next logical operation
  on the version-controlled file
  corresponding to the current buffer.
```

输出看起来却是这样的：

```
C-x v M-x vc-next-action Perform the next logical operation on the
version-controlled file corresponding to the current buffer.
```

在这种情况下，文本应使用`@table`、`@item`和`@itemx`进行格式化，以创建一个表格。

@code, @samp, @var, 及‘---’

- 使用`@code`包住 Lisp 符号（包括命令名）。例如，

```
The main function is @code{vc-next-action}, ...
```
- 避免在‘@code’之后紧跟‘s’等字母。这样的字母看起来很糟糕。
- 使用`@var`包住元变量(meta-variables)。不要在它们前后写尖括号。
- 使用三个连续的连字符‘---’来表示长破折号。Info 格式化程序会将三个连字符简化为两个；在其他输出格式中，按照排版规则显示长破折号。

引号之外的句号

除非标点符号是引文的一部分，否则应将句号和其他标点符号置于引号外。这种做法与美国的一些出版惯例相悖，但它能让读者区分引文的内容和整段文字。

例如，你应该将句号放在句尾引号之外来书写以下句子：

```
Evidently, ‘au’ is an abbreviation for ‘`author’.
```

因为‘au’并不作为‘author.’（在单词后加句号）的缩写。

引入新术语

- 引入新术语，以便不了解这些术语的读者能够从上下文中理解它们，或者为该术语写一个定义。

例如，在下文中，“签入(check in)”、“注册(register)”和“增量(delta)”都是首次出现；应该重写示例句，以便读者理解这些术语。

```
该主要功能可帮助您将文件签入(check in)到版本控制系统中，并将后续的更
改集合以增量(delta)形式进行注册(register)。
```

- 在引入一个新词时，使用`@dfn`命令，以提示读者不要期望已经了解该词的含义，而应该期待从这段文字中学习其含义。

重写的示例句如下：

```
该主要功能可帮助您将文件签入(check in) 到版本控制系统中，并将后续的
更改集合以增量(delta) 形式进行注册(register) 。
```

描述如何调用程序的节点

您可以从 shell 启动 Emacs、GCC 和 gawk 等程序。每个程序的文档中都应包含一个描述此(调用)操作的章节。遗憾的是, 如果这些章节的节点名称和标题各不相同, 用户将很难找到它们。

因此, 有一个约定是用以‘调用(Invoking)’开头的短语来命名这类章节, 如‘Invoking Emacs’ (调用 Emacs); 这样, 用户就能轻松找到相关章节。

ANSI C 语法

当您使用 `@example` 来描述 C 函数的调用约定时, 请使用 ANSI C 语法, 如下所示:

```
void dld_init (char *@var{path});
```

在后续讨论中, 引用参数值时, 请写出相同的参数名称, 同样用 `@var` 进行突出(highlight)标记。

避免使用这种过时的风格:

```
#include <dld.h>

dld_init (path)
char *path;
```

此外, 最好避免仅为了表明函数是在某个头文件中声明的, 而在声明上方编写 `#include`。这种做法可能会给人一种误解, 即 `#include` 应该出现在函数声明的附近。要么明确指出哪个头文件包含该声明, 要么更好的做法是在描述函数的章节开头, 指出用于一组函数的头文件名称。

节点的长度

必须保持节点(章节)的合理长度, 无论在给定上下文中是否合理。不要犹豫, 将长节点分解为子节点, 并构建一个庞大的树形结构, 这正是该结构存在意义所在。很多时候, 读者可能会尝试使用搜索、索引或纯粹的猜测来在手册中找到某个特定点, 而不是从头到尾通读全文。

您可以使用 `texi-elements-by-size` 工具来查看文档中所有节点(或章节)的列表, 这些列表按大小(行数或字数)排序, 以便找到适合拆分的候选节点。该工具位于 Texinfo 源代码的 `util/` 子目录中。

坏的示例

以下是几个应避免的糟糕写作示例:

在这个例子中, 可以说成这样“...你必须 `@dfn{check in}` 新版本。”更通顺。

编辑完文件后, 您必须执行 `@dfn{check in}`。

在以下示例中, 可以说, “...使得像 VC 模式这样的统一界面成为可能。”

SCCS、RCS 和其他版本控制系统都以大致相似的方式执行相似的功能 (正是这种相似性使得像这样的统一控制模式成为可能)。

在这个例子中, 你应该明确‘它’所指的对象:

如果你正在与其他人合作, 它有助于协调每个人的更改, 这样他们就不会互相干扰。

最后...

- T_EX的发音中的‘x’类似于希腊字母“chi”的发音，就像“Bach”这个名字中的最后一个音节一样。但 Texinfo 的‘x’发音类似于“speck”中的发音：“teckinfo”。
- 在 Texinfo 文件的末尾，即在@bye之后，为自己写些注释。处理器不会处理@bye之后的文本；就好像这些文本位于@ignore...@end ignore之中一样。

附录 C Texinfo 文件的示例

本附录包含用于 GNU 手册的文本。

C.1 GNU 示例文本

以下是一个 Texinfo 文档的示例，其中包含应在 GNU 手册中使用的完整文本（可根据需要进行调整）。

除了法律文本外，它还作为一个实际例子，展示了 GNU 系统中多少元素会影响手册。如果你不熟悉所有这些不同的元素，不用担心。它们并非必需，即使没有这些元素，也能编写出一份非常优秀的手册。不过，这里还是把它们列出来了，因为许多手册确实（或可能）能从它们中受益。

请参阅2.4 节[Short Sample Texinfo File], 9 页，这是 Texinfo 文件的最小示例。

以下是关于这个例子的几点说明：

- `@include`命令中的`version.texi`由 Automake 自动维护（请参阅节“Texinfo”在 *GNU Automake*）。它设置在其他地方使用的‘`VERSION`’、‘`UPDATED`’和‘`UPDATED-MONTH`’值。如果你的发行版不使用 Automake，但你确实在使用 Emacs，你可能会发现`time-stamp.el`包很有帮助（请参阅节“Time Stamps”在 *The GNU Emacs Manual*）。
- `@syncodeindex`命令反映了尽可能只使用一个索引的建议，以便读者更容易查找索引条目。
- `@dircategory`为手册指定了一个类别。它用于构建 Info 目录(整个 Info 系统的顶层目录)。请参阅2.6 节[Directory Category], 14 页，其中包含各种推荐的类别名称。请参阅20.1.4 节[Installing Dir Entries], 173 页。
- ‘调用(Invoking)’节点是 GNU 标准的一部分，旨在帮助用户查找特定程序在命令行中用法的基本信息。请参阅节“Manual Structure Details”在 *GNU Coding Standards*。
- 该示例展示了如何使用`@include`命令在手册中包含FDL (GNU Free Documentation License, GNU 自由文档许可证)。`fdl.texi`文件可在 Texinfo 和其他 GNU 源代码分发版中找到。它也同时在 GNU 网站 (<https://www.gnu.org/licenses/fdl-1.3.html>) 上提供，并附有使用指南。
- 如果 FSF 不是版权所有者，那么请使用适当的名称。
- 对于表达个人观点、感受或经历的文档，使用仅允许逐字复制的许可证比使用 FDL 更为合适。请参阅节 C.2 [Verbatim Copying License], 236 页。

以下是示例文档：

```
\input texinfo @c -*-texinfo-*-
@comment %**start of header
@include version.texi
@settitle GNU Sample @value{VERSION}
@syncodeindex pg cp
@comment %**end of header
@copying
This manual is for GNU Sample (version @value{VERSION}, @value{UPDATED}),
which is an example in the Texinfo documentation.
```

Copyright @copyright{} 2016 Free Software Foundation, Inc.

```
@quotation
Permission is granted to copy, distribute and/or modify this document under
the terms of the GNU Free Documentation License, Version 1.3 or any later
version published by the Free Software Foundation; with no Invariant
Sections, with no Front-Cover Texts, and with no Back-Cover Texts.  A copy
of the license is included in the section entitled ``GNU Free Documentation
License''.
@end quotation
@end copying

@dircategory Texinfo documentation system
@direntry
* sample: (sample)Invoking sample.
@end direntry

@titlepage
@title GNU Sample
@subtitle for version @value{VERSION}, @value{UPDATED}
@author A.U. Thor (@email{bug-sample@gnu.org})
@page
@vskip 0pt plus 1filll
@insertcopying
@end titlepage

@contents

@node Top
@top GNU Sample

This manual is for GNU Sample (version @value{VERSION}, @value{UPDATED}).

@menu
* Invoking sample::
* GNU Free Documentation License::
* Index::
@end menu

@node Invoking sample
@chapter Invoking sample

@pindex sample
@cindex invoking @command{sample}

This is a sample manual.  There is no sample program to invoke, but if
there were, you could see its basic usage and command line options here.
```



```
@node GNU Free Documentation License
@appendix GNU Free Documentation License
```

```
@include fdl.texi
```

```
@node Index
@unnumbered Index
```

```
@printindex cp
```

```
@bye @cccccc 临时改 '@@bye' 为注释，以便于翻译编辑工作。因为编辑器会误码将 '@@bye' 当作是本文件(tex
```

C.2 逐字复制许可证

对于软件手册和其他文档，使用允许自由再分发及更新的许可证至关重要，这样当自由软件发生更新时，文档也能随之更新。

另一方面，对于表达个人观点、感受或经历的文档，使用仅允许逐字复制的许可证更为合适。

以下是一份仅允许逐字复制的许可证示例文本。这仅仅是许可证文本本身。如需查看完整的示例文档，请参阅前面的章节。

```
@copying
This document is a sample for allowing verbatim copying only.

Copyright @copyright{} 2016 Free Software Foundation, Inc.
@quotation
Permission is granted to make and distribute verbatim copies of this
entire document without royalty provided the copyright notice and this
permission notice are preserved.
@end quotation
@end copying
```

C.3 全开放复制许可证

对于软件手册和其他文档，使用允许自由再分发和更新的许可证非常重要，这样当自由程序发生更改时，文档也能随之更新。

另一方面，对于小型辅助文件、简短的手册（长度不超过 300 行）以及粗略的文档（如 README 文件、INSTALL 文件等），采用 FDL（自由文档许可证）可能过于严格。它们可以采用简单的全开放许可协议。

以下是一个全开放许可证的示例文本。这只是许可文本本身。如需查看完整的示例文档，请参阅前面的部分。

```
Copyright @copyright{} 2016 Free Software Foundation, Inc.

Copying and distribution of this file, with or without modification,
are permitted in any medium without royalty provided the copyright
notice and this notice are preserved.
```

附录 D 使用 GNU Emacs 的 Texinfo 模式

您可以使用任何你选择的文本编辑器来编辑 Texinfo 文件。Texinfo 文件与任何其他 ASCII 文件没有区别。然而，GNU Emacs 附带了一种特殊的模式，称为 Texinfo 模式，它提供了 Emacs 命令和工具来帮助简化你的工作。

D.1 Texinfo 模式概述

Texinfo 模式为处理 Texinfo 文件提供了特殊功能。您可以：

- 插入常用的 @ 命令。
- 自动创建@node代码行。
- 展示 Texinfo 源文件的结构。
- 自动创建或更新节点的‘Next’、‘Previous’和‘Up’指针。
- 自动创建或更新菜单。
- 自动生成主控菜单。
- 将文件的部分或全部格式化成 Info。
- 对文件的部分或全部内容进行排版和打印。

也许最有帮助的两个功能是由于插入常用 @ 命令的功能以及用于创建节点指针和菜单的功能。

D.2 常用的 GNU Emacs 编辑命令

在大多数情况下，Texinfo 模式下的常规文本模式命令与在文本模式下的功能相同。Texinfo 模式为 GNU Emacs 的通用编辑功能增加了新的编辑命令和工具。主要区别在于填充。在 Texinfo 模式下，段落分隔变量和语法表被重新定义，这样本应单独占一行的 Texinfo 命令就不会无意中被包含在段落中。因此，*M-q* (*fill-paragraph*，段落填充)命令将重新填充段落，但不会将相邻行上的索引命令混入段落中。

此外，Texinfo 模式将`page-delimiter`（页面分隔符）变量设置为`texinfo-chapter-level-regexp`（texinfo 章级正则表达式）的值；默认情况下，这是一个正则表达式，用于匹配章命令及其它同等的（如：附录）命令。通过页面分隔符值设为该值，您可以通过`C-x ]` (*forward-page*，前一页)和`C-x [` (*backward-page*，后一页)命令从一个章标题跳转到另一个章标题，并通过`C-x n p` (*narrow-to-page*)命令跳转到特定章。有关页面(*page*)命令的详细信息，请参阅请参见节“Pages”在 *The GNU Emacs Manual*。

当您访问扩展名为`.texinfo`、`.texi`或`.txi`的文件时，GNU Emacs 会自动进入 Texinfo 模式。此外，当您访问首行包含`‘-*-texinfo-’`的文件时，Emacs 也会切换到 Texinfo 模式。如果您当前处于其他模式，但希望切换到 Texinfo 模式，请键入`M-x texinfo-mode`。

与 Emacs 的所有其他功能一样，您可以根据需要自定义或增强 Texinfo 模式。特别是，键绑定非常容易更改。此处描述的键绑定是默认或标准的键绑定。

D.3 插入常用命令

Texinfo 模式提供了将各种常用的 @ 命令插入到缓冲区中的命令。您可以使用这些命令来节省击键次数。

插入命令是通过键入两次 `C-c`，然后输入 `@` 命令的第一个字母来调用的：

`C-c C-c c`

`M-x texinfo-insert-@code`

插入 `@code{}`，并将光标放在大括号之间。

`C-c C-c d`

`M-x texinfo-insert-@dfn`

插入 `@dfn{}`，并将光标放在大括号之间。

`C-c C-c e`

`M-x texinfo-insert-@end`

插入 `@end` 并尝试插入正确的后续单词，如 `'example'` 或 `'table'`。（此命令无法正确处理嵌套列表，但会为紧邻前一个列表插入合适的单词。）

`C-c C-c i`

`M-x texinfo-insert-@item`

插入 `@item`，并将光标移至下一行的开头。

`C-c C-c k`

`M-x texinfo-insert-@kbd`

插入 `@kbd{}`，并将光标放在大括号之间。

`C-c C-c n`

`M-x texinfo-insert-@node`

插入 `@node`，并在 `@node` 后添加一行注释，列出 `'Next'`（下一个）、`'Previous'`（上一个）和 `'Up'`（上一级）节点的序列。在 `@node` 后留空。

`C-c C-c o`

`M-x texinfo-insert-@noindent`

插入 `@noindent`，并将光标放在下一行的开头。

`C-c C-c r`

`M-x texinfo-insert-dwim-@ref`

此函数和绑定是在 Emacs 27.1 中添加的。它会根据光标周围的文本插入 `@pxref{}`、`@xref{}` 或 `@ref{}` 中的一个；如果在未闭合的前开括号附近调用它，则会是 `@pxref{}`；在句子开头或 (point-min) 处调用，则会在 `@xref{}`；在其他任何位置（包括单词内部）调用，则会在 `@ref{}`。一个数字参数用于表示括号应包围的单词数量。将光标置于括号之间。

`C-c C-c s`

`M-x texinfo-insert-@samp`

插入 `@samp{}`，并将光标放在大括号之间。

`C-c C-c t`

`M-x texinfo-insert-@table`

插入 `@table`，后跟一个 SPC，并将光标留在 SPC 之后。

`C-c C-c v`

`M-x texinfo-insert-@var`

插入 `@var{}`，并将光标放在大括号之间。

`C-c C-c x`

`M-x texinfo-insert-@example`

插入 `@example`，并将光标放在下一行的开头。

```
C-c C-c {
M-x texinfo-insert-braces
      插入 {}, 并将光标放在大括号之间。
```

```
C-c }
C-c ]
M-x up-list
      从一对大括号之间向前移动, 越过右大括号。输入 C-c ] 比输入 C-c } 更容易, 然
      而后者更便于记忆; 因此有了这两种键绑定。(此外, 您还可以通过输入 C-f 从大
      括号之间移出。)
```

要在现有单词前后添加如 `@code{...}` 这样的命令, 请将光标定位在单词前面, 然后输入 `C-u 1 C-c C-c c`。这在编辑现有的纯文本时很有用。前缀参数的值告诉 Emacs 在光标后的括号之间包含多少个单词——‘1’表示一个单词, ‘2’表示两个单词, 以此类推。使用负数参数可包围前一个或前几个单词。如果不指定前缀参数, Emacs 会插入 `@` 命令字符串, 并将光标定位在括号之间。此功能仅适用于那些在一行内对一个或多个单词进行操作的 `@` 命令, 如 `@kbd` 和 `@var`。

这组插入命令是在分析《GNU Emacs 手册》(*GNU Emacs Manual*) 和《GDB 手册》(*GDB Manual*) 中不同 `@` 命令的使用频率后创建的。如果您想添加自己的插入命令, 可以将键盘宏绑定到某个键, 使用缩写, 或者在 `texinfo.el` 中扩展代码。

`C-c C-c C-d` (`texinfo-start-menu-description`) 是一个插入命令, 其工作方式与其他插入命令不同。它会在菜单条目行的描述位置插入节点对应的节或章的标题。(一个菜单条目包含三部分: 条目名称、节点名称和描述。虽然只有节点名称是必需的, 但描述有助于解释节点的含义。请参阅 3.9.4 节 [Menu Parts], 32 页。)

要使用 `texinfo-start-menu-description`, 请将光标定位在菜单条目行中, 然后键入 `C-c C-c C-d`。该命令会查找并复制与节点名称对应的标题, 并将其作为描述插入; 同时, 光标会定位在插入文本的开头, 以便您进行编辑。如果菜单条目行中已包含描述, 则该函数不会插入标题。

此命令仅辅助编写描述; 它并不能完成全部工作。由于标题往往使用与节点名称相同的词汇, 而有效的描述则使用不同的词汇, 因此您必须对插入的文本进行编辑。

D.4 显示文件的章节结构

您可以使用 `C-c C-s` 命令 (`texinfo-show-structure`) 来显示 Texinfo 文件的章节结构。此命令会列出以像 `@chapter`、`@section` 这样的 `@` 命令开头的行, 这些 `@` 命令构建了一个相当于目录中的内容。这些行会显示在另一个名为 `*0ccur*` 的缓冲区中。在该缓冲区中, 您可以将光标定位到其中一行, 并使用 `C-c C-c` 命令 (`occur-mode-goto-occurrence`) 跳转到 Texinfo 文件中相应的位置。

```
C-c C-s
M-x texinfo-show-structure
      显示 Texinfo 文件中的 @chapter、@section 等行。
```

```
C-c C-c
M-x occur-mode-goto-occurrence
      跳转到 Texinfo 文件中与 *0ccur* 缓冲区中光标所在行相对应的行。
```

如果你通过键入 `C-u C-c C-s` 来调用带有前缀参数的 `texinfo-show-structure`, 它将不仅列出包含 `@chapter`、`@section` 等命令的行, 还会列出 `@node` 行。你可以使用带有前缀参数的 `texinfo-show-structure` 来检查 `@node` 行的‘下一页’、‘上一页’和‘上一级’指针是否正确。

通常,当你在处理手册时,你只会对当前章节的结构感兴趣。在这种情况下,你可以使用`C-x n n` (`narrow-to-region`)命令来标记你感兴趣的区域,这样`texinfo-show-structure`就只会处理该区域。若要再次查看整个缓冲区,请使用`C-x n w` (`widen`)。有关缩窄命令的更多信息,请参阅节“Narrowing”在 *The GNU Emacs Manual*。

除了提供`texinfo-show-structure`命令外, Texinfo 模式还会设置页面分隔符变量的值,以匹配章级别的章节命令。这样,您就可以使用`C-x ]` (`forward-page`)和`C-x [` (`backward-page`)命令按章前后移动,并使用`C-x n p` (`narrow-to-page`)命令移动到特定章节。有关页面命令的更多信息,请参阅节“Pages”在 *The GNU Emacs Manual*。

D.4.1 使用`texinfo-show-structure`

跟踪 Texinfo 文件的节点、章、节和子节并不总是那么容易。如果你正在修订或添加别人编写的 Texinfo 文件,情况尤其如此。

在 GNU Emacs 的 Texinfo 模式下, `texinfo-show-structure`命令会列出所有以指定结构的 `@` 命令开头的行: `@chapter`、`@section`、`@appendix`等。如果交互式操作,则以`C-u`作为前缀参数,该命令还会显示`@node`行。默认情况下,在 Texinfo 模式下, `texinfo-show-structure`命令绑定到`C-c C-s`。

这些行显示在一个名为`*0ccur*`的缓冲区中,并按层级缩进。例如,以下是在本手册上运行`texinfo-show-structure`后生成的一部分内容¹:

```
Lines matching "^@\\(chapter \\|sect\\|subs\\|subh\\|
unnum\\|major\\|chapheading \\|heading \\|appendix\\)"
in buffer texinfo.texi.
...
4177:@chapter Nodes
4198:  @heading Two Paths
4231:  @section Node and Menu Illustration
4337:  @section The @code{@@node} Command
4393:    @subheading Choosing Node and Pointer Names
4417:    @subsection How to Write a @code{@@node} Line
4469:    @subsection @code{@@node} Line Tips
...
```

这表示在`texinfo.texi`文件的第 4337、4393 和 4417 行分别以`@section`、`@subheading`和`@subsection`命令开头。如果你将光标移动到`*0ccur*`窗口,你可以将光标定位到其中一行,并使用`C-c C-c`命令 (`occur-mode-goto-occurrence`) 跳转到 Texinfo 文件中相应的位置。有关`occur-mode-goto-occurrence`的更多信息,请参阅节“Using Occur”在 *The GNU Emacs Manual*。

`*0ccur*`窗口中的第一行描述了由`texinfo-heading-pattern`指定的正则表达式。这个正则表达式是`texinfo-show-structure`所查找的模式。有关更多信息,请参阅节“Using Regular Expressions”在 *The GNU Emacs Manual*。

当你调用`texinfo-show-structure`命令时, Emacs 将显示整个缓冲区²的结构。如果你想只查看缓冲区的部分结构,例如只想看某一章的结构,可以使用`C-x n n` (`narrow-to-region`)命令来标记区域。(请参阅节“Narrowing”在 *The GNU Emacs Manual*。)上面的示例就是这样生成的。(要再次查看整个缓冲区,请使用`C-x n w` (`widen`)).)

¹ 译者注:是原文的,不是译文的。

² 译者注:指的就是已打开的被编辑的文件。

如果你通过键入 `C-u C-c C-s` 来调用带有前缀参数的 `texinfo-show-structure`，它将列出以 `@node` 以及 `@chapter`、`@section` 等 `@` 命令开头的行。

你可以通过查看 `*Occur*` 窗口中的列表，来时刻关注自己 Texinfo 文件的结构；如果你给某个节点命名错误或遗漏了某个章节，你可以及时发现并更正这个错误。

D.4.2 使用 occur

有时，`texinfo-show-structure` 命令会输出过多信息。也许你想回顾一下 Texinfo 文件的整体结构，但却被 `texinfo-show-structure` 生成的详细列表所淹没。在这种情况下，你可以直接使用 `occur` 命令。具体操作如下：

`M-x occur`

然后，在提示时，输入一个 `regexp`，即你想要匹配的模式正则表达式（请参阅节“Regular Expressions”在 *The GNU Emacs Manual*）。`occur` 命令从缓冲区中光标当前位置开始，一直工作到缓冲区末尾。如果你想在整个缓冲区上运行 `occur`，请将光标放置在缓冲区开头。

例如，要查看所有包含单词 `@chapter` 的行，只需输入 `@chapter`。这将生成一个章列表。它还会列出含有位于行中间的 `@chapter` 的句子。

如果你只想查看以单词 `@chapter` 开头的行，请在 `occur` 提示时输入 `^@chapter`。如果你希望查看所有以某个单词或短语结尾的行，请在最后一个单词后加上 `$`；例如：`catching mistakes$`。当你想查看属于同一章(节)并因而具有相同‘上一级’指针的所有节点时，这将非常有用。

有关更多信息，请参阅节“Using Occur”在 *The GNU Emacs Manual*。

D.5 更新节点和菜单

`texi2any` 命令可以为缺乏‘Next(下一个)’、‘Previous(上一个)’和‘Up(上一级)’指针（请参阅 3.1 节 [Writing a Node], 24 页）的层次结构 Texinfo 文件创建一个 Info 文件。因此，通常不需要显式的‘下一个’、‘上一个’和‘上一级’指针。在这种情况下，没有显式菜单的节点，会自动添加菜单。（有关 `texi2any` 的更多信息，请参见第 19 章 [Generic Translator `texi2any`], 148 页。）

如果你仍然想要明确的指针，Texinfo 模式提供了用于自动创建或更新菜单和节点指针的命令。这些命令之所以被称为‘更新’命令，是因为它们最常用于在你处理完 Texinfo 文件后对其进行更新；但你也可以使用它们将‘下一个’、‘上一个’和‘上一级’指针插入到没有这些指针的 `@node` 行中，并在没有菜单的文件中创建菜单。

D.5.1 更新命令

您可以使用更新命令来：

- 插入或更新节点的‘Next(下一个)’、‘Previous(上一个)’和‘Up(上一级)’指针，
- 为某个章节插入或更新菜单，以及
- 为 Texinfo 源文件创建一个主控菜单。

您还可以使用这些命令来更新某个区域或整个 Texinfo 文件中的所有节点和菜单。

更新命令仅适用于常规的 Texinfo 文件，这类文件的结构层次类似于书籍。在这种文件中，除了‘Top’ `@node` 行之外，每行 `@node` 之后都必须紧跟一行(章节)结构化命令。

你可以在紧接 `@node` 行之后的行上编写结构化(章节)命令，或者在单个 `@comment` 行或单个 `@ifinfo` 行之后的行上编写。在 `@node` 行和结构化(章节)命令之间不能插入多行；只能插入一个 `@comment` 行或一个 `@ifinfo` 行。

作用于整个缓冲区的命令要求‘Top’节点之后必须跟一个与@chapter或相同级别命令关联的节点。对于仅包含@chapter级别节点的 Texinfo 文件，菜单更新命令不会为其创建主菜单或主控菜单！菜单更新命令仅在节点内部为下级节点创建菜单。若要创建章节的菜单，您必须提供一个‘Top’节点。

菜单更新命令会移除那些引用其他 Info 文件的菜单项，因为它们并不引用当前缓冲区内的节点。这是一个不足之处。你可以使用交叉引用来引用其他 Info 文件，而不是使用菜单项。所有的更新命令都不会影响交叉引用。

Texinfo 模式有五个最常用的更新命令：两个用于更新单个节点（或区域）中的节点指针或菜单；两个用于更新文件中的所有节点指针和菜单；还有一个，即texinfo-master-menu命令，用于为整个文件创建一个主控菜单，并可选更新整个 Texinfo 文件中的每个节点和菜单。

texinfo-master-menu 命令是最重要的命令：

C-c C-u m

M-x texinfo-master-menu

创建或更新一个主控菜单，其中包含所有其他菜单（如果有的话，应整合现有的菜单中的描述）。

增加一个参数（前缀参数，如果是交互式，则使用C-u），可以在构建主控菜单之前，首先在缓冲区中创建或更新所有节点和所有常规菜单。（有关主控菜单的更多信息，请参阅2.10节[The Top Node and Master Menu], 20页）。

为了让texinfo-master-menu生效，Texinfo 文件必须包含一个‘Top’节点以及至少一个后续节点。

在大范围编辑 Texinfo 文件后，您可以键入以下内容：

C-u M-x texinfo-master-menu

或

C-u C-c C-u m

这会一次性彻底更新所有节点和菜单。

其他的主要更新命令执行较小的任务，专为编写 Texinfo 文件时更新节点和菜单的人员设计。

命令如下：

C-c C-u C-n

M-x texinfo-update-node

为当前所在节点（即，对前面的@node行）插入‘Next(下一个)’、‘Previous(上一个)’和‘Up(上一级)’指针。如果@node行中已存在‘Next’、‘Previous’或‘Up’指针，则删除旧指针并插入新指针。若带有参数（前缀参数，交互模式下为C-u），此命令将更新区域（即，当前位置与标记之间的文本）中的所有@node行。

C-c C-u C-m

M-x texinfo-make-menu

在当前位置所在的节点内创建或更新菜单。增加一个参数（如果是交互式操作，则以C-u为前缀参数），该命令为当前节点或属于该区域的节点创建或更新菜单。

每当texinfo-make-menu更新现有菜单时，该菜单中的描述都会被合并到新菜单中。这是通过将现有菜单中的描述复制到新菜单中具有相同节点名的条目来实现的。如果节点名不同，则描述不会被复制到新菜单中。

C-c C-u C-e

M-x texinfo-every-node-update

为缓冲区中的每个节点插入或更新‘Next’、‘Previous’和‘Up’指针。

C-c C-u C-a

M-x texinfo-all-menus-update

在缓冲区中创建或更新所有菜单。增加一个参数（若为交互模式，则以**C-u** 作为前缀参数），先插入或更新所有节点指针，然后再处理菜单。

如果存在主控菜单，则**texinfo-all-menus-update**命令对其进行更新；若主近代菜单尚不存在，该命令不会创建新的主菜单。（若需创建，请使用**texinfo-master-menu**命令。）

当处理一个不需要使用主控菜单的文档时，你可以输入以下内容：

C-u C-c C-u C-a

或

C-u M-x texinfo-all-menus-update

这将更新所有节点和菜单。

texinfo-column-for-description变量指定了菜单描述缩进的列数。默认情况下，该值为 32，默认情况下，该值为 32，但将其减小到 24 也是很有用的。您可以通过定制化（请参阅节“Customization”在 *The GNU Emacs Manual*）或使用**M-x set-variable**命令（请参阅节“Examining and Setting Variables”在 *The GNU Emacs Manual*）来设置该变量。

此外，您可以使用**texinfo-indent-menu-description**命令将现有的菜单描述缩进到指定的列。最后，如果您愿意，可以使用**texinfo-insert-node-lines**命令将缺失的@node行插入到文件中。特别是，在初稿中，您可以完全忽略@node行，然后在编写段落时使用**texinfo-insert-node-lines**命令为您创建@node行。但是，我们不建议这样做。更好的做法是在编写段落的同时为节点本身命名，这样您就可以轻松地进行交叉引用。有用的交叉引用是优秀 Texinfo 手册的一个特别重要的特性。（有关更多信息，请参阅节 D.5.5 [Other Updating Commands], 245 页。）

D.5.2 更新命令所需的条件

要使用更新命令，您必须将 Texinfo 文件按章、节、子节等层次结构进行组织。在构建手册的层次结构时，请勿一次‘下跳’多于一个层级：您可以在‘Top’节点之后跟随一个章，但不能跟随一个节；您可以在一个章之后跟随一个节，但不能跟随一个子节。但是，您可以一次‘上跳’任意数量的层级——例如，从子节跳到章。

除‘Top’节点的行外，每行@node之后都必须跟有一行章节命令，如：**@chapter**、**@section**或**@unnumberedsubsec**。

每个@node行与章节命令行的组合，必须看起来像这样：

```
@node      Comments, Minimum, Conventions, Overview
@comment   节点名,      Next,      Previous,      Up
@section  批注
```

或像这样(没有@comment行)：

```
@node      Comments, Minimum, Conventions, Overview
@section  批注
```

或像这样(没有显式节点指针)：

```
@node      Comments
@section  批注
```


在这个例子中，‘Comments’是节点的名称，‘批注’是节的名称。下一个节点被称为‘Minimum’，上一个节点被称为‘Conventions’。‘批注’节在‘Overview’节点里，这是由‘Up’指针指定的。

如果一个文件有一个‘Top(顶)’节点，那么它必须命名为‘top’或‘Top’，并且必须是文件中的第一个节点。

菜单更新命令可以在章内创建各节的菜单，在节内创建小节的菜单，以此类推。这意味着，如果你想要一个各章的菜单，就必须有一个‘Top(顶)’节点。

D.5.3 更新外层文件和包含文件

GNU Emacs 的 Texinfo 模式提供了 `texinfo-multiple-files-update` 命令。该命令会创建或更新包含文件的‘Next’、‘Previous’和‘Up’指针，以及外层或总体 Texinfo 文件中的这些指针，并且会在外层文件中创建或更新一个主菜单。根据调用该命令时是否有可选参数，该命令将仅更新包含文件中第一个@node行的指针，或者更新包含文件中所有的@node行。

以 `C-u` 作为前缀参数，可在外层文件中创建并插入主菜单。若前缀参数为数字，如 `C-u 2`，则需先更新所有包含文件中的所有菜单以及所有‘Next’、‘Previous’和‘Up’指针，然后再在外层文件中创建并插入主菜单。

更详细地说：

`M-x texinfo-multiple-files-update`

不带任何参数调用：

- 在外层或总体 Texinfo 文件中的每个包含文件中，创建或更新其第一个@node行的‘Next’、‘Previous’和‘Up’指针。
- 创建或更新外层或总体文件的‘顶’节点指针。
- 在外层文件中创建或更新主菜单。

`C-u M-x texinfo-multiple-files-update`

以 `C-u` 作为前缀参数调用：

- 在每个包含文件的第一个@node行中创建或更新指针。
- 创建或更新外层文件的‘Top(顶)’节点指针。
- 在外层文件中创建并插入一个主控菜单。依据所有包含文件中的所有菜单来生成主控菜单。

`C-u 8 M-x texinfo-multiple-files-update`

使用数字前缀参数调用，例如 `C-u 8`：

- 创建或更新所有 包含文件的‘Next’、‘Previous’和‘Up’指针。
- 创建或更新所有 包含文件中的所有菜单。
- 创建或更新外层或总体文件的‘Top(顶)’级节点指针。
- 然后在外层文件中创建一个主控菜单。这类似于在处理单个文件时，带有参数调用 `texinfo-master-menu` 命令。

注意在交互使用中前缀参数的使用：使用常规前缀参数，如：`C-u`，`texinfo-multiple-files-update` 命令会插入一个主控菜单；使用数字前缀参数，如 `C-u 8`，该命令会更新所有文件中的每个 指针和菜单，然后插入一个主控菜单。

D.5.4 对包含文件的要求

如果你打算使用 `texinfo-multiple-files-update` 命令，那么外层的 Texinfo 文件（在其中列出了包含文件）应仅包含 Texinfo 文件的开头和结尾部分，以及多个 `@include` 命令用于列出包含文件。它甚至不应包含索引，因为索引应列在它们自己的一个包含文件中。

此外，每个包含的文件必须恰好包含一个最高级节点（通常为 `@chapter` 或等效节点），且该节点必须是包含文件中的第一个节点。此外，每个包含文件中的这些最高级节点必须在文件结构中处于相同的层级。通常，每个节点都是 `@chapter`、`@appendix` 或 `@unnumbered` 节点。因此，通常每个包含文件只包含一个章或等效级别的节点。

外层文件应仅包含一个节点，即‘Top(顶)’节点。除单个‘Top’节点外，它不应包含任何其他节点。`texinfo-multiple-files-update` 命令将不会处理这些节点。

D.5.5 其他更新命令

除了主要的更新命令外，Texinfo 模式还有几个使用频率较低的更新命令：

M-x texinfo-insert-node-lines

在 Texinfo 文件中，如果某个区域缺少 `@node` 行，请在 `@chapter`、`@section` 以及其他章节命令之前插入这些行。

在交互模式下，使用参数（以 `C-u` 作为前缀参数）时，命令 `texinfo-insert-node-lines` 不仅会插入 `@node` 行，还会将章或节标题作为相应节点的名称插入。此外，它还会在已存在的但缺少名称的 `@node` 行中，插入章或节标题作为节点名称。由于节点名应该比章或节标题更简洁，因此有必须手动编辑用这种方式插入的节点名。

例如，以下操作将整个缓冲区标记为一个区域，并在其中插入 `@node` 行和标题：

```
C-x h C-u M-x texinfo-insert-node-lines
```

此命令将标题作为节点名插入到 `@node` 行中；而 `texinfo-start-menu-description` 命令（请参阅节 D.3 [Inserting], 237 页）则将标题作为描述插入到菜单条目中，这是另一种操作。不过，在这两种情况下，你都需要对插入的文本进行编辑。

M-x texinfo-indent-menu-description

按照指定列缩进菜单中的每个描述。您可以使用此命令为自己提供更多描述空间。如果交互式使用，则以 `C-u` 作为前缀参数，`texinfo-indent-menu-description` 命令会缩进该区域中每个菜单中的每个描述。但是，此命令不会缩进多行描述的第二行及后续行。

M-x texinfo-sequential-node-update

将紧接在当前节点之后及之前的节点的名称分别作为‘Next’及‘Previous’指针插入，无论这些节点的层级如何。这意味着，一个节的‘Next’节点很可能是下一章。对于小说和其他按顺序阅读的文档，顺序排列的节点非常有用。（然而，在 Info 中，`g *` 命令允许您按顺序浏览文件，因此顺序排列的节点并非绝对必要。）使用参数（如果是交互式，则为前缀参数），`texinfo-sequential-node-update` 命令可以按顺序更新区域中的所有节点。

D.6 Info 格式化

Texinfo 模式提供了多个命令，用于对 Texinfo 文件的部分或全部内容进行 Info 格式化。

D.6.1 在 Emacs 中运行texi2any/makeinfo

texi2any程序提供的错误信息比 Emacs 的任何一种格式化命令都要好。我们推荐使用它。texi2any程序独立于 Emacs。

在 GNU Emacs 的 Texinfo 模式下, 您可以使用makeinfo-region或makeinfo-buffer命令来运行texi2any (或makeinfo)。在 Texinfo 模式下, 这些命令默认绑定到C-c C-m C-r和C-c C-m C-b。

C-c C-m C-r

M-x makeinfo-region

对当前区域进行 Info 格式化。

C-c C-m C-b

M-x makeinfo-buffer

对当前缓冲区进行 Info 格式化。

当你调用makeinfo-region时, 输出会进入一个临时缓冲区。当你调用makeinfo-buffer时, 输出会进入由@setfilename设置的文件 (请参阅2.5.2 节[@setfilename], 12 页)。

Emacs 的makeinfo-region和makeinfo-buffer命令会在一个临时的 shell 缓冲区中运行texi2any程序。如果texi2any发现任何错误, Emacs 会在临时缓冲区中显示错误消息。

您可以通过键入C-x `(next-error)来解析错误消息。这将使 Emacs 跳转到texi2any认为导致错误的 Texinfo 源代码行, 并将光标定位在该行。有关使用next-error命令的更多信息, 请参阅节 “Running make or Compilers Generally” 在 *The GNU Emacs Manual*。

此外, 您可以关闭运行texi2any命令的 shell, 或者让 shell 缓冲区显示其最新输出。

C-c C-m C-k

M-x makeinfo-kill-job

终止当前正在运行的texi2any (或makeinfo) 任务 (来自makeinfo-region或makeinfo-buffer)。

C-c C-m C-l

M-x makeinfo-recenter-output-buffer

重新显示texi2any的 shell 缓冲区, 以展示其最新输出。

(请注意, 用于终止和复位(recentering)TeX作业的对应的命令分别为C-c C-t C-k和C-c C-t C-l。请参阅节 D.7.1 [Texinfo Mode Printing], 247 页。)

您可以通过使用M-x customize或M-x set-variable命令设置makeinfo-options变量, 或在您的.emacs 初始化文件中设置该变量, 来为texi2any指定选项。

例如, 你可以在你的.emacs 文件中编写以下内容:

```
(setq makeinfo-options
  "--paragraph-indent=0 --no-split
  --fill-column=70 --verbose")
```

如需更多信息, 请参阅:

1. 节 “Easy Customization Interface” 在 *GNU Emacs 手册*
2. 节 “Examining and Setting Variables” 在 *GNU Emacs 手册*
3. 节 “Init File” 在 *GNU Emacs 手册*
4. [texi2any的选项], 148 页,

D.6.2 texinfo-format... 命令

在 GNU Emacs 的 Texinfo 模式下，您可以使用 `texinfo-format-region` 命令来格式化 Texinfo 文件的部分或全部内容。本命令格式化当前区域，并在名为 `*Info Region*` 的临时缓冲区中显示格式化后的文本。

同样，您可以使用 `texinfo-format-buffer` 命令来格式化缓冲区。本命令创建一个新缓冲区，并在其中生成 Info 文件。键入 `C-x C-s` 会将 Info 文件保存在 `@setfilename` 行指定的名称下，该行必须位于 Texinfo 文件的开头附近。

`C-c C-e C-r`

`texinfo-format-region`

对当前区域进行 Info 格式化。

`C-c C-e C-b`

`texinfo-format-buffer`

对当前缓冲区进行 Info 格式化。

`texinfo-format-region` 和 `texinfo-format-buffer` 命令为您提供了一些错误检查功能及其他一些功能，可在查找格式错误方面为您提供更多帮助。这些步骤在附录中有详细描述；请参阅节 D.10 [Catching Mistakes], 253 页。不过，`texi2any` 程序提供了更好的错误检查功能（请参阅节 D.6.1 [texi2any in Emacs], 246 页）。

`texinfo-format-buffer` 和 `texinfo-format-region` 命令的一个特点是，它们不会缩进（也不会填充）包含 `@w` 或 `@*` 命令的段落。

D.7 使用 Emacs 进行格式化和打印

GNU Emacs 可以通过 Emacs Shell 使用 `TEX` 进行格式化和打印。Texinfo 模式还提供了用于格式化和打印的预定义快捷键。

D.7.1 Texinfo 模式下的格式化和打印

Texinfo 模式为 `TEX` 的格式化和打印提供了多个预定义的快捷键。这些快捷键包括用于排序索引、查看打印机队列、终止格式化任务以及重新定位操作缓冲区的显示的命令。

通常，在编写文档时，您可能只想排版和打印文件的局部，以查看其效果。为此，您可以使用 `texinfo-tex-region` 及相关命令。使用 `texinfo-tex-buffer` 命令可格式化整个缓冲区。

为了让 `texinfo-tex-region` 或 `texinfo-tex-buffer` 生效，文件必须以 `\input texinfo`³ 行开头，并且必须包含 `@settitle` 行。文件必须以单独一行的 `@bye` 结束。（使用 `texinfo-tex-region` 时，`@settitle` 行必须在由 `'start-of-header'...` `'end-of-header'` 构成的头部区域。请参阅 2.5.5 节 [Start and End of Header], 13 页。）

`C-c C-t C-b`

`M-x texinfo-tex-buffer`

对缓冲区运行 `texi2dvi`。除了对缓冲区运行 `TEX` 外，此命令还会根据需要自动创建或更新索引。

`C-c C-t C-r`

`M-x texinfo-tex-region`

对当前区域运行 `TEX`。

³ 译者注：中文文档用 `\input texinfo-zh`。

如果与打印输出相关的 @ 命令位于‘start-of-header’行和‘end-of-header’行（请参阅2.5.5 节[Start and End of Header], 13 页）之间，则texinfo-tex-region会相应地格式化该区域。例如，如果在‘start-of-header’行‘end-of-header’行之间编写@smallbook命令，则texinfo-tex-region会将该区域格式化为“小”书尺寸。

C-c C-t C-i

M-x texinfo-texindex

运行texindex命令可以对使用texinfo-tex-region格式化的 Texinfo 文件的索引进行排序。texinfo-tex-region命令并不会自动运行texindex；它只会运行tex排版命令。在用texindex命令对原始索引文件进行排序后，必须再次运行texinfo-tex-region命令。（通常，在格式化区域时不会格式化索引，只有在格式化缓冲区时才会格式化索引。现在有了texi2dvi命令，这个命令就很少用或几乎没有必要了。）

C-c C-t C-p

M-x texinfo-tex-print

打印一个使用texinfo-tex-region或texinfo-tex-buffer生成的 DVI 文件。

C-c C-t C-q

M-x tex-show-print-queue

显示打印队列。

C-c C-t C-d

M-x texinfo-delete-from-print-queue

从打印队列中删除作业；系统会提示您输入一作业编号；该编号可由C-c C-t C-q命令（texinfo-show-tex-print-queue）显示。

C-c C-t C-k

M-x tex-kill-job

终止当前正在运行的由texinfo-tex-region或texinfo-tex-buffer启动的 T_EX作业，或终止在 Texinfo shell 缓冲区中运行的任何其他进程。

C-c C-t C-x

M-x texinfo-quit-job

通过向已因错误而停止的 T_EX格式化作业发送x来退出它。当你这样做时，T_EX会在.log文件中保留其操作记录。

C-c C-t C-l

M-x tex-recenter-output-buffer

重新显示运行 T_EX打印和格式化命令的 shell 缓冲区，以展示其最新输出。

因此，格式化缓冲区的常见命令序列如下（右侧为注释）：

C-c C-t C-b	在缓冲区上运行texi2dvi
C-c C-t C-p	打印 DVI 文件
C-c C-t C-q	显示打印队列

Texinfo 模式中的 T_EX格式化命令会在 Emacs 中启动一个名为**tex-shell**的子 shell。texinfo-tex-command、texinfo-texindex-command 和tex-dvi-print-command 命令都在这个 shell 中运行。

你可以在**tex-shell**缓冲区中观察命令的运行情况，并且可以像切换其他 shell 缓冲区一样，在**tex-shell**缓冲区及其它 shell 缓冲区之间进行切换和使用。

格式化和打印命令依赖于多个变量的值。默认值为：

变量	默认值
<code>texinfo-texi2dvi-command</code>	<code>"texi2dvi"</code>
<code>texinfo-tex-command</code>	<code>"tex"</code>
<code>texinfo-texindex-command</code>	<code>"texindex"</code>
<code>texinfo-delete-from-print-queue-command</code>	<code>"lprm"</code>
<code>texinfo-tex-trailer</code>	<code>"@bye"</code>
<code>tex-start-of-header</code>	<code>"%**start"</code>
<code>tex-end-of-header</code>	<code>"%**end"</code>
<code>tex-dvi-print-command</code>	<code>"lpr -d"</code>
<code>tex-show-queue-command</code>	<code>"lpq"</code>

您可以使用 `M-x set-variable` 命令（请参阅节“Examining and Setting Variables”在 *The GNU Emacs Manual*）或您的 `.emacs` 初始化文件（请参阅节“Init File”在 *The GNU Emacs Manual*）来更改这些变量的值。

从版本 20 开始，GNU Emacs 提供了一个名为 *Customize* (定制) 的用户友好界面，用于更改用户可定义变量的值。有关此功能的更多详细信息，请参阅节“Easy Customization Interface”在 *The GNU Emacs Manual*。调用 `M-x customize` 命令后，可在‘Development/Docs/Texinfo’组中找到 Texinfo 变量。

D.7.2 使用局部变量列表

将 $\TeX$ 格式化命令应用于 Texinfo 文件的另一种方法是 将该命令放在 Texinfo 文件末尾的 *local variables list* (局部变量列表) 中。然后，您可以将 `tex` 或 `texi2dvi` 命令指定为 `compile-command`，并通过键入 `M-x compile` 让 Emacs 运行它。这将创建一个名为 `*compilation*` 的特殊 shell 缓冲区，Emacs 会在其中运行编译命令。例如，在 `gdb.texi` 文件的末尾，`@bye` 之后，您可以添加以下内容：

```
Local Variables:
compile-command: "texi2dvi gdb.texi"
End:
```

这种技术最常被以这种方式编译程序的程序员使用；请参阅节“Compilation”在 *The GNU Emacs Manual*。

D.8 Texinfo 模式总结

在 Texinfo 模式下，每组命令都有以相同按键开头的默认键绑定。所有为 Texinfo 模式定制创建的命令都以 `C-c` 开头。这些按键具有一定的助记作用。

插入命令

插入命令是通过键入两次 `C-c`，然后键入要插入的 `@` 命令的第一个字母来调用的。（从记忆的角度来看，使用 `C-c C-i` 可能更有意义，代表‘自定义插入(custom insert)’，但 `C-c C-c` 键入起来更快。）

<code>C-c C-c c</code>	插入 ‘@code’。
<code>C-c C-c d</code>	插入 ‘@dfn’。
<code>C-c C-c e</code>	插入 ‘@end’。
<code>C-c C-c i</code>	插入 ‘@item’。
<code>C-c C-c n</code>	插入 ‘@node’。

<code>C-c C-c s</code>	插入 ‘@samp’。
<code>C-c C-c v</code>	插入 ‘@var’。
<code>C-c {</code>	插入大括号。
<code>C-c]</code>	
<code>C-c }</code>	移出封闭的大括号。
 <code>C-c C-c C-d</code>	 在菜单项的描述栏中 插入节点的章节标题。

显示结构

`texinfo-show-structure`命令通常用于在特定的区域（由`narrow-to-region`命令指定）。

`C-c C-s` 列出所有标题。

主控更新命令

`texinfo-master-menu`命令用于创建主控菜单；同时也可以用于更新文件中的每个节点和菜单。

`C-c C-u m`
`M-x texinfo-master-menu`
 创建或更新主控菜单。

`C-u C-c C-u m`
 以`C-u`作为前缀参数，首先创建或更新所有节点和常规菜单，然后创建一个主控菜单。

更新指针

更新指针命令的调用方式是先输入`C-c C-u`，然后输入`C-n`调用`texinfo-update-node`，或输入`C-e`调用`texinfo-every-node-update`。

`C-c C-u C-n`
 更新当前节点。

`C-c C-u C-e`
 更新缓冲区中的每个节点。

更新菜单

通过输入`C-c C-u`来调用更新菜单命令，然后输入`C-m`来运行`texinfo-make-menu`，或者输入`C-a`来运行`texinfo-all-menus-update`。若要同时更新节点和菜单，请在输入`C-c C-u C-a`前先输入`C-u`。

`C-c C-u C-m`
 创建或更新菜单。

`C-c C-u C-a`
 创建或更新缓冲区中的全部菜单。

`C-u C-c C-u C-a`
 以`C-u`作为前缀参数，首先创建或更新所有节点，然后创建或更新所有菜单。

Info 格式化

用 Emacs Lisp 编写的 Info 格式化命令，可以通过键入 `C-c C-e` 来调用，然后键入 `C-r`（针对某个区域）或 `C-b`（针对整个缓冲区）来执行。

基于 `texi2any/makeinfo` 程序的 Info 格式化命令，可通过键入 `C-c C-m` 来调用，然后键入 `C-r` 选择一个区域，或键入 `C-b` 选择整个缓冲区。

使用 `texinfo-format...` 命令：

`C-c C-e C-r`

格式化区域。

`C-c C-e C-b`

格式化缓冲区。

使用 `texi2any/makeinfo`：

`C-c C-m C-r`

格式化区域。

`C-c C-m C-b`

格式化缓冲区。

`C-c C-m C-l`

重新定位 `texi2any` 输出缓冲区。

`C-c C-m C-k`

终止 `texi2any` 格式化任务。

排版与打印

通过键入 `C-c C-t`，然后键入另一个控制命令来调用 $\text{T}_{\text{E}}\text{X}$ 排版和打印命令：`C-r` 对应 `texinfo-tex-region`，`C-b` 对应 `texinfo-tex-buffer`，以此类推。

`C-c C-t C-r`

对所选区域运行 $\text{T}_{\text{E}}\text{X}$ 。

`C-c C-t C-b`

在缓冲区上运行 `texi2dvi`。

`C-c C-t C-i`

运行 `texindex`。

`C-c C-t C-p`

打印 DVI 文件。

`C-c C-t C-q`

显示打印队列。

`C-c C-t C-d`

从打印队列中删除作业。

`C-c C-t C-k`

终止当前的 $\text{T}_{\text{E}}\text{X}$ 格式化任务。

`C-c C-t C-x`

退出当前已停止的 $\text{T}_{\text{E}}\text{X}$ 格式化作业。

`C-c C-t C-l`

重新定位输出缓冲区。

其他更新命令

其余的更新命令没有标准的快捷键，因为它们很少被使用。

M-x texinfo-insert-node-lines

在区域中插入缺失的@node行。使用C-u作为前缀参数，将章节标题用作节点名。

M-x texinfo-multiple-files-update

更新多文件文档。使用C-u 2作为前缀参数，首先创建或更新所有包含文件中的所有节点和菜单。

M-x texinfo-indent-menu-description

缩进描述。

M-x texinfo-sequential-node-update

按严格顺序插入节点指针。

D.9 Info 文件的直接格式化

一般来说，不必编辑或处理 Info 文件。Info 文件的大部分内容是纯文本，但标签表需要计算节点和间接文件的偏移量（请参阅 20.2 节[Tag and Split Files], 176 页），这是不切实际的工作。Texinfo 处理器可以从 Texinfo 源文件输出格式良好的 Info 文件，此外，Texinfo 源文件中的许多信息在 Info 文件中会丢失，因此，Texinfo 源文件更适合用于对进行分析，无论是用于错误报告、统计还是翻译。

尽管如此，在处理手动编写的 Info 文件时，直接处理 Info 文件的某些环节很值得关注，特别是计算 Info 文件的节点偏移量和计算拆分后的 Info 文件的文件偏移量。Emacs 中有两个命令可以做到这一点。Info-tagify为缺少标签表的非拆分文件添加一个标签表。此函数对于 Info 文件验证也非常有用（请参阅节 D.10.4 [Running Info-validate], 256 页）。还可以使用Info-split来分割非分割 Info 文件。

D.9.1 标签化一个文件

Texinfo 处理器会自动创建标签表。唯一的例外情况是，当使用前缀（如：C-u M-x texinfo-format-buffer）调用texinfo-format-buffer来创建没有标签表的非分割 Info 文件时（在某些情况下，这对于 Info 文件验证很有用，请参阅节 D.10.4.2 [Unsplit and Tagify], 257 页）。因此，对于手动编写的 Info 文件，添加标签表通常更为有用。

要为非分割文件创建标签表，请打开您希望进行标签化的 Info 文件，并输入以下命令：

M-x Info-tagify

（请注意：Info中的大写‘I’。）这将创建一个带有标签表的 Info 文件。标签表是结构完整的 Info 文件的一部分（请参阅20.2 节[Tag and Split Files], 176 页）。验证或分割 Info 文件时也需要标签表。

D.9.2 手动分割文件

大文件应该被分割，或者让texinfo-format-buffer或makeinfo-buffer命令自动为你完成分割。（通常你会让其中一个格式化命令为你完成这项工作。请参阅节 D.6 [Info Formatting], 245 页。）

分割出来的文件被称为间接子文件。

标签表是由格式化命令自动创建的；如果您手动执行此操作，则只需自己创建一个标签表。有关创建标签表的信息，请参阅节 D.9.1 [Tagifying], 252 页。

手动打开你想要标记和拆分的 Info 文件，并输入以下两个命令：

```
M-x Info-tagify
M-x Info-split
```

(请注意，‘Info’中的‘I’是大写的。)

当你使用 `Info-split` 命令时，缓冲区会被修改为一个（较小的）Info 文件，其中列出了间接子文件。此文件应保存为被访问的原始文件的替代文件。间接子文件会写入原始文件所在的同一目录中，其名称是在原始文件名后附加‘-’和数字生成的。

主文件仍然作为 Info 文件使用，但它只包含标签表和子文件目录。请参阅 20.2 节 [Tag and Split Files], 176 页。

D.10 捕捉错误

除了文档内容中的错误外，使用 Texinfo 时还可能犯两类错误：一类是 @ 命令使用错误，另一类是节点和章节结构出错。

为了查找 @ 命令的问题，你可以对有问题的区域运行 `TEX` 或区域格式化命令；实际上，你可以在编写每个区域时都运行这些命令。

要查找节点和章节结构的问题，您可以使用 `C-c C-s` (`texinfo-show-structure`) 和相关的 `occur` 命令，也可以使用 `M-x Info-validate` 命令。

D.10.1 texi2any 是首选

`texi2any` 程序在捕捉错误并报告方面表现出色，远胜于 `texinfo-format-region` 或 `texinfo-format-buffer`。此外，各种用于自动创建和更新节点指针和菜单的功能也大大减少了人为错误的可能性。

使用 `texi2any`（或其 Texinfo 模式表现形式：`makeinfo-region` 和 `makeinfo-buffer`）来格式化文件并检查其他错误。这是处理 Texinfo 的最佳方式。但如果你无法使用 `texi2any`，或者你的问题非常棘手，那么你可能需要使用本节中描述的工具。

D.10.2 使用 Info 格式化来捕获错误

在编写完 Texinfo 文件的一部分后，您可以使用 `texinfo-format-region` 或 `makeinfo-region` 命令来查看该区域是否格式正确。

然而，你之所以阅读本节内容，很可能是因为出于某种原因，你无法使用 `makeinfo-region` 命令；因此，本节其余部分假定你正在使用 `texinfo-format-region`。

如果你在 @ 命令上犯了错误，`texinfo-format-region` 将在错误发生时或之后停止处理，并显示错误消息。要查看错误发生在缓冲区的哪个位置，请切换到‘*Info Region*’缓冲区；光标将位于错误位置之后。此外，在错误发生的位置（或更准确地说，在检测到错误的位置）之后的文本将不会进行格式化。

例如，如果你在菜单末尾不小心使用了命令 `@end menus`（末尾多了一个‘s’），而不是使用 `@end menu`，那么你将看到一条错误消息，内容如下：

```
@end menus is not handled by texinfo
```

光标将在缓冲区中错误发生的位置或其稍后位置停止。缓冲区将显示如下：

```

----- Buffer: *Info Region* -----
* Menu:

* Using texinfo-show-structure::  How to use
                                `texinfo-show-structure'
                                to catch mistakes.
* Running Info-validate::        How to check for
                                unreferenced nodes.

@end menus
*
----- Buffer: *Info Region* -----

```

`texinfo-format-region` 命令有时会给出略显奇怪的错误消息。例如，以下交叉引用无法格式化：

```
(@xref{Catching Mistakes, for more info.})
```

在这种情况下，`texinfo-format-region` 会检测到缺少的右括号，但会显示一条消息，指出‘Unbalanced parentheses(括号不匹配)’，而不是‘Unbalanced braces(大括号不匹配)’。这是因为格式化命令将‘braces(大括号)’视为‘parentheses(括号)’，并查找它们之间的不匹配情况。

有时，`texinfo-format-region` 无法检测到错误。例如，在以下示例中，右花括号被换成了右括号：

```
(@xref{Catching Mistakes), for more info.}
```

格式化产生：

```
(*Note for more info.: Catching Mistakes)
```

你发现这个错误的唯一途径是意识到交叉引用看起来应该是这样：

```
(*Note Catching Mistakes::, for more info.)
```

顺便说一句，如果你在 Info 中阅读此节点并输入 `f RET` (Info-follow-reference)，将会生成一条错误消息，内容如下：

```
No such node: "Catching Mistakes) The only way ...
```

这是因为 Info 将错误的示例视为该节点的第一个交叉引用，如果你在输入 Info 的 `f` 命令后立即输入 `RET`，Info 将尝试跳转到引用的节点。如果你输入 `f catch TAB RET`，Info 将补全正确示例的节点名称，并跳转到‘Catching Mistakes’节点。（如果你尝试这样做，你可以通过输入 `l` (Info-last) 从‘Catching Mistakes’节点返回。）

D.10.3 使用 T_EX 进行调试

当您使用 T_EX 格式化文件时，也可以发现其中的错误。

通常，您需要先在同一文件上运行 `texinfo-format-buffer` (或者，更好的是，运行 `makeinfo-buffer`)，再使用 T_EX 进行调试，因为 `texinfo-format-buffer` 有时会显示比 T_EX 更有意义的错误消息。（有关更多信息，请参阅节 D.10.2 [Debugging with Info], 253 页。）

例如，T_EX 正在处理一个 Texinfo 文件，该文件的部分内容如下所示：

```

----- Buffer: texinfo.texi -----
name of the Texinfo file as an extension. The
@samp{??} are `wildcards' that cause the shell to
substitute all the raw index files. (@xref{sorting

```

```
indices, for more information about sorting
indices.)
```

```
----- Buffer: texinfo.texi -----
```

(交叉引用缺少右括号。) T_EX生成了以下输出，然后停止了：

```
----- Buffer: *tex-shell* -----
```

```
Runaway argument?
{sorting indices, for more information about sorting
indices.) @ETC.
! Paragraph ended before @xref was complete.
<to be read again>

@par

1.27
```

```
?
```

```
----- Buffer: *tex-shell* -----
```

在这种情况下，T_EX生成了一个准确且易于理解的错误消息：

```
Paragraph ended before @xref was complete.
```

‘@par’是 T_EX 的一个内部命令，与 Texinfo 无关。‘1.27’表示 T_EX 在 Texinfo 文件的第 27 行检测到了问题。‘?’是 T_EX 在这种情况下使用的提示符。

不幸的是，T_EX 并不总是那么有用，有时你必须像福尔摩斯一样才能发现出错的地方。

无论如何，如果你遇到这样的问题，你可以选择以下三种方法中的一种。

1. 您可以在‘?’提示符下键入 RET，指示 T_EX 继续运行并忽略此错误。
2. 您可以在‘?’提示符下键入 `r RET`，指示 T_EX 继续运行并尽可能忽略所有错误。

这通常是最佳做法。然而，请注意：一个错误可能会引发一连串的额外错误消息，因为其后会波及到文件的其余部分。要在 T_EX 产生大量错误消息时停止它，请键入 `C-c` (如果您在 Emacs 内部运行 shell，则键入 `C-c C-c`)。

3. 您可以在‘?’提示符下键入 `x RET` 来告诉 T_EX 停止运行。

如果你在 Emacs 中运行 T_EX，你需要切换到 shell 缓冲区，并定位到 T_EX 提供‘?’提示符的行。

有时，即使文件存在问题，T_EX 也会对其进行格式化而不会显示错误消息。这种情况通常发生在命令未结束，但 T_EX 仍能继续处理时。例如，如果你未能使用 `@end itemize` 命令结束一个项目列表，T_EX 会生成一个可以打印的 DVI 文件。T_EX 唯一会给出的错误消息是一条略显神秘的注释：

```
(@end occurred inside a group at level 1)
```

然而，如果你打印 DVI 文件，你会发现，在项目列表之后的文件文本完全缩进，就像它们是项目列表中最后一个项目的一部分。错误信息是 T_EX 表示它期望在文件的某个地方找到一个 `@end` 命令；但它无法确定该命令应该出现在哪里。

另一个出了名难找的错误来源是缺少 `@end group` 命令。如果你遇到难以理解的错误而感到困惑，请首先检查是否缺少 `@end group` 命令。

如果 Texinfo 文件缺少头部行，T_EX 可能会在运行刚开始时就停止，并显示类似以下的输出。‘*’表示 T_EX 正在等待输入。

```
This is TeX, Version 3.14159 (Web2c 7.0)
(test.texinfo [1])
*
```

在这种情况下，只需在星号后键入`\end RET`。然后在 Texinfo 文件中编写头行，并再次运行 `TEX` 命令。（注意反斜杠`\`的使用。`TEX`使用`\`而不是`@`；在这种情况下，您直接使用的是 `TEX`，而不是 Texinfo。）

D.10.4 查找错误引用的节点

您可以使用 `Info-validate` 命令来检查‘Next’、‘Previous’、‘Up’或其他节点指针是否无法指向某个节点。此命令会检查每个节点指针是否指向一个存在的节点。`Info-validate` 命令仅适用于 Info 文件，不适用于 Texinfo 文件。

`texi2any` 程序会自动验证指针，因此，如果您正在使用 `texi2any`，则无需使用 `Info-validate` 命令。在设置了定制化变量 `CHECK_NORMAL_MENU_STRUCTURE` 的情况下，如果节点指针（无论是显式设置还是自动设置）与节点菜单条目的顺序不一致，`texi2any` 也会发出警告。`texi2any` 不会检查每个‘Next’指针是否都有一个与之匹配的‘Previous’（在‘Next’所指向的节点中），因为对于非标准文档结构来说，这种匹配可能是正确的。

如果您无法运行 `texi2any`，而必须使用 `texinfo-format-region` 或 `texinfo-format-buffer` 来创建 Info 文件，或者您是从头开始编写 Info 文件，那么您可能只需要使用 `Info-validate`。

D.10.4.1 使用 Info-validate

要使用 `Info-validate`，打开您要检查的 Info 文件并输入：

```
M-x Info-validate
```

请注意，`Info-validate` 命令需要一个大写的‘I’。在运行 `Info-validate` 之前，您可能还需要创建一个标签表。请参阅节 D.9.1 [Tagifying]，252 页。

如果您的文件有效，您将收到一条消息，显示“File appears valid(文件应属有效)”。但是，如果您的指针没有指向节点，则会在一个名 `*problems in info file`(info 文件中的问题)* 的缓冲区中显示错误消息。

例如，对一个仅包含本手册第一个节点的测试文件运行了 `Info-validate`。其中一条消息显示：

```
In node "Overview", invalid Next: Texinfo Mode
```

这意味着名为‘Overview’的节点有一个‘Next(下一个)’指针，该指针并未指向任何内容（在本例中确实如此，因为测试文件中只有一个节点）。

现在假设我们在测试用例中添加了一个名为‘Texinfo Mode’的节点，但未为此节点指定‘Previous’节点。那么我们将收到以下错误消息：

```
In node "Texinfo Mode", should have Previous: Overview
```

这是因为，在标准的文档结构中，每个‘Next’指针都应有一个与之匹配的‘Previous’指针（在‘Next’指针所指向的节点中），用于回溯。

`Info-validate` 还会检查所有菜单项和交叉引用是否指向实际节点。

`Info-validate` 需要一个标签表，并且不适用于已分割的文件。（`texinfo-format-buffer` 命令会自动分割大文件。）若要对大文件使用 `Info-validate`，您必须带参数运行 `texinfo-format-buffer`，使其不分割 Info 文件；并且您必须为未分割的文件创建一个标签表。

D.10.4.2 创建未分割文件并添加标签表

的 r 的 d

您只能对包含标签表的单个 Info 文件运行 `Info-validate` 命令。该命令无法处理主文件分割生成的间接子文件。如果您的文件较大（大约超过 300,000 字节），则需要在运行 `texinfo-format-buffer` 或 `makeinfo-buffer` 命令时，确保它不会创建间接子文件。您还需要为 Info 文件创建一个标签表。完成这些操作后，您可以运行 `Info-validate` 并查找引用错误的节点。

第一步是创建一个非分割的 Info 文件。为了防止 `texinfo-format-buffer` 将 Texinfo 文件分割成更小的 Info 文件，请为 `M-x texinfo-format-buffer` 命令指定一个前缀：

```
C-u M-x texinfo-format-buffer
```

也可以用

```
C-u C-c C-e C-b
```

当你这样做时，`texinfo-format-buffer` 将不会分割文件，也不会为其创建标签表。

在创建了一个非分割的 Info 文件后，您必须为其创建一个标签表（请参阅节 D.9.1 [Tagifying], 252 页）。访问由 `texinfo-format-buffer` 创建的非分割 Info 文件，并键入：

```
M-x Info-tagify
```

第三步是验证 Info 文件：

```
M-x Info-validate
```

（注意 `Info-validate` 中的大写 ‘I’。）简而言之，步骤如下：

```
C-u M-x texinfo-format-buffer
```

```
M-x Info-tagify
```

```
M-x Info-validate
```

在验证了节点结构之后，你可以按正常方式重新运行 `texinfo-format-buffer`，以便它构建一个标签表并自动分割文件；或者你也可以手动制作标签表并分割文件（请参阅节 D.9.2 [Splitting], 252 页）。

D.11 批量格式化

您可以使用 `batch-texinfo-format` 和 Emacs 批处理模式来为 Info 格式化 Texinfo 文件。您可以在任何 Shell（包括 Emacs 内部的 Shell）中以批处理模式运行 Emacs。（请参阅节 “Initial Options” 在 *The GNU Emacs Manual*。）

以下是一个 shell 命令，用于格式化当前目录中所有以 `.texinfo` 结尾的文件：

```
emacs -batch -funcall batch-texinfo-format *.texinfo
```

Emacs 会处理命令行中列出的所有文件，即使尝试格式化其中某些文件时发生错误。

如所见到的，`batch-texinfo-format` 仅能在 Emacs 的批处理模式下运行；它不具备交互性。运行完成后，它将终止处于批处理模式的 Emacs。

如果您没有 `texi2any` 并且想一次性格式化多个 Texinfo 文件，那么 `batch-texinfo-format` 就很方便。使用批处理模式时，会创建一个新的 Emacs 进程。这样就能释放您当前的 Emacs 资源，以便您继续在其中工作。（当您运行 `texinfo-format-region` 或 `texinfo-format-buffer` 时，在命令完成之前，您无法使用该 Emacs 执行其他任何操作。）

附录 E 全局性的文档命令

以下是影响整个文档的附加命令。这些命令大多用于定制打印输出的外观。如果给出这些命令，它们通常都会在‘顶部节点(Top node)’之前给出。

E.1 @setchapternewpage: 章前的空白页

在正式装订的书籍中，文字通常双面印刷，章从右侧页开始，且右侧页为奇数。但在简短的报告中，文字往往仅单面印刷。同样在简短的报告中，章有时并不从新页开始，而是与上一章的结尾在同一页上，中间留有一小段垂直空白。

您可以使用带有各种参数的@setchapternewpage命令来指定章在打印输出中应如何开始，以及页眉应如何格式化以便在纸张的一面或两面（单面或双面）上打印。

在一行的开头写下@setchapternewpage命令，后面跟着它的参数。例如，如果你想让每一章都从新的奇数页开始，你可以这样写：

```
@setchapternewpage odd
```

您可以使用@setchapternewpage命令指定以下三种选项之一：

@setchapternewpage off

在跳过一些垂直空白后，将新章排版到与上一章相同的页面上。此外，将页眉设为为单面打印格式。

@setchapternewpage on

在新页面上开始新章，并将页眉设为为单面打印格式。这是短篇报告或个人打印最常用的格式。这是默认设置。

@setchapternewpage odd

在新的奇数页（右侧页）开始新章，并排版为双面打印。这是书籍和手册最常用的格式。

Texinfo 没有@setchapternewpage even命令，因为在打印传统中，章或书籍的开头并不需要从偶数页开始。

如果你不喜欢@setchapternewpage设置的默认标题，你可以使用@headings命令来明确控制它们。请参阅节 E.2.1 [headings], 259 页。

在手册或书籍的开头部分，页面通常不编号---例如，书籍的标题页和版权页就不编号。按照惯例，目录和正文前页(frontmatter pages)¹会用罗马数字编号，且编号顺序与文档的其他部分互相独立。

在 Info 和 HTML 等没有页面的输出格式中，@setchapternewpage无效。

我们建议源文档中完全不要包含任何@setchapternewpage命令，因为这些所需的页码编排并非文档的固有特性。对于特定的硬拷贝，如果你不希望得到默认输出（无空白页，所有页面上的页眉相同），请使用texi2dvi的--texinfo 选项来指定你想要的输出。

¹ 译者注：指的是书籍、报告或文档中位于正文之前的部分，通常包含目录、序言等内容。

E.2 页眉

大多数打印手册在每页顶部（标题页和版权页除外）都包含页眉。有些手册还包含页脚。在 Info 或其他输出格式中，页眉和页脚没有意义。

Texinfo 提供了两种标准的页眉格式，一种适用于每页单面打印的手册，另一种适用于双面打印的手册。默认情况下，Texinfo 文件的页脚未指定任何内容，因此页脚保持空白。

Texinfo 还提供了多个页眉和页脚命令，可用于生成定制化的页眉和页脚格式。

在 Texinfo 中，页眉和页脚是位于页面顶部和底部的单行文字；无法创建多行页眉或页脚。每行页眉或页脚文字分为三部分：左部、中部和右部。任何部分或一整行都可以是空白。页眉或页脚左部的文字设置为左对齐；中部的文字设置为居中；右部的文字设置为右对齐。

E.2.1 @headings命令

@headings命令很少使用。它用于指定每页上要打印的页眉和页脚类型。通常，这由@setchapternewpage命令控制。只有当@setchapternewpage命令无法满足您的需求时，才需要使用@headings命令。

你可以按如下方式使用@headings：

@headings off

关闭页眉的打印。

@headings single

适合单面打印的页眉。

@headings double

适合双面打印的页眉。

@headings singleafter

@headings doubleafter

在当前页面输出后，分别启用单、双面页眉。

@headings on

启用页眉。如果‘@setchapternewpage’为 on，则实际使用单面；否则实际使用双面。

例如，假设您在@titlepage命令之前编写@setchapternewpage off，以便在上一章末尾的同一页上开始新的一章。此命令还会导致按单面打印排版页眉。若要为双面打印排版页眉，请写@headings double。

你可以通过单独另起一行，写下@headings off来完全禁止生成任何页面标题，如下所示：

@headings off

E.2.2 标准页眉格式

Texinfo 提供了两种标准的页眉格式，一种适用于每页单面打印的手册，另一种适用于双面打印的手册。

单面打印的标准格式包括一条页眉行，其中左部包含章名称，中部为空白，右部包含页码。单面打印的页面看起来像这样：

章名	页码
正文开始 ...	
...	

在标准的双面格式中，左侧页面（偶数页）的左部包含页码，中部为空白，右部包含标题（由`@settitle`命令指定）。右侧页面（奇数页）的左部包含章名，中间部分为空白，右部包含页码。偶数页和奇数页并排如同一本打开的书，看起来像这样：

-----		-----	
页码	标题	章名	页码
正文开始 ...		更多正文 ...	
...		...	

章名前会加上“Chapter”、章编号和一个冒号²。这样便于你追踪自己在手册中的位置。

请注意，在新章开始的页面上，页眉行可能会省略一些文本。

默认情况下，Texinfo 文件的页脚未指定任何内容，因此页脚保持空白。

E.2.3 如何制作自己的页眉

您可以使用 Texinfo 提供的标准页眉，也可以自行指定。默认情况下，Texinfo 没有页脚，因此如果您指定了页脚，则正文可用的页面大小将略有减少。

Texinfo 提供了六个命令用于指定页眉和脚注：

- `@everyheading`和`@everyfooting`在偶数页和奇数页上生成相同的页眉和页脚。
- `@evenheading`和`@evenfooting`命令用于为偶数（左侧）页面生成页眉和页脚。
- `@oddheading`和`@oddfooting`为奇数页（右侧）生成页眉和页脚。

在定制页眉页脚之前，必须使用`@headings off`命令取消预定义的标题命令。

以下是在偶数页和奇数页的每个页眉中，将章节名称置于左侧、页码置于中间、日期置于右侧的方法：

```
@headings off
@everyheading @thischapter @| @thispage @| @today{}
```

你需要通过在各部分之间插入‘@|’来区分左侧部分、中间部分和右侧部分。否则，规范命令将无法判断一个部分的文本在哪里结束，下一个部分又从哪里开始。

每个部分都可以包含文本或 `@` 命令。文本的打印方式就像该部分位于页面正文的普通段落中一样。而 `@` 命令则会替换为页码、日期、章名或其他内容。

² 译者注：这是原文（英文）的形式。在中文文档的输出（使用XeTeX输出）中的形式是这样：章名前加上“第 n 章”，n 为章编号，在附录名前加上“附录 c”，c 是附录编号。

以下是六个标题和脚注命令：

`@everyheading left @| center @| right`

`@everyfooting left @| center @| right`

‘every’命令用于指定所有的偶数页及奇数页的格式。这些命令适用于每张纸只打印一面，或者希望页眉或页脚对称的文档。

`@evenheading left @| center @| right`

`@oddheading left @| center @| right`

`@evenfooting left @| center @| right`

`@oddfooting left @| center @| right`

‘even’和‘odd’命令分别用于指定偶数页或奇数页的格式。这些命令适用于每张纸双面打印的书籍和手册。

使用‘@this...’系列 @ 命令来提供章节和节点的名称以及页码。您可以在页眉和页脚的左侧、中间或右侧使用‘@this...’命令。

以下是‘@this...’命令：

`@thispage`

展开为当前页码。

`@thissectionname`

展开为当前节名。

`@thissectionnum`

展开为当前节号。

`@thissection`

展开为当前节号及节名，用‘Section 1: Title’这样的格式。

`@thischaptername`

展开为当前章名。

`@thischapternum`

展开为当前的章号或附录号。

`@thischapter`

展开为当前章号及章名，用‘Chapter 1: Title’格式。

`@thistitle`

展开为文档的名称，该名称由`@settitle`指定。

`@thisfile`

仅限`@include`文件：展开为当前`@include`文件的名称。如果当前 Texinfo 源文件不是`@include`文件，则此命令无效。除非当前 Texinfo 源文件是`@include`文件，否则此命令不会提供当前 Texinfo 源文件的名称。（有关`@include`文件的更多信息，请参阅第 17 章[Include Files], 140 页。）

您还可以使用`@today{}`命令，该命令会以‘1 Jan 1900’的格式展开为当前日期。

其他 @ 命令和文本会像在页面正文中一样，打印在页眉或页脚中。在编写草稿时，加入文本尤其有用：

`@headings off`

`@everyheading @emph{Draft!} @| @thispage @| @thischapter`

`@everyfooting @| @| Version: 0.27: @today{}`

当心标题过长：它们可能会与页眉或页脚的其他部分重叠，从而将其遮挡。

如果你的章和/或节篇幅很短，其中几个章和/或节可能被放在同一页上，这种情况下，你可以指定`@thischapter`、`@thissection`和其他类似宏在本页面上实际引用的章节，如下所示：

`@everyheadingmarks ref`

`@everyfootingmarks ref`

如`ref` 参数是`top`，`@this...`命令将引用处于本页面的最前面的章节。如是`bottom`，命令将引用本页面最后的章节。`@every...` 命令同时指定偶数页和奇数页上应执行的操作。

`@evenheadingmarks ref`

`@oddheadingmarks ref`

`@evenfootingmarks ref`

`@oddfootingmarks ref`

这些`@even...`和`@odd...`命令分别指定了仅在偶数页或奇数页上执行的操作。`ref` 参数与`@every...`命令中的相同。

请将这些命令直接放在`@...contents` 命令之后，或者，如果你没有目录或者目录是在手册末尾打印的，则放在`@end titlepage`命令之后。这些命令在 L^AT_EX 中无效。

默认情况下，对于 T_EX，`@this...`命令（无论是页眉还是页脚中使用）会引用本页面最后的章/节。

E.3 @paragraphindent：控制段落缩进量

Texinfo 处理器可能会在每个段落的第一行开头插入空格，从而使该段落缩进。您可以使用`@paragraphindent`命令来对缩进进行具体说明。从一行的开头写`@paragraphindent`命令，后跟`'asis'`或一个数字：

`@paragraphindent indent`

缩进量是根据`indent`的值来确定的：

`asis` 请勿更改现有的缩进（在打印输出中未实现）。

`none`

`0` 省略所有缩进。

`n` 在 Info 输出中缩进`n` 个空格字符，在打印输出中缩进`n` 个 em³。

`indent` 的默认值为 3。对于 HTML 输出，`@paragraphindent`被忽略。

最好在 Texinfo 文件的头部结束(end-of-header)行之前写入`@paragraphindent`命令，以便区域格式化(region formatting)命令能按指定缩进量进行段落缩进。请参阅[Start of Header], 13 页。

³ 译者注：是 CSS（层叠样式表）中的一种 相对长度单位，用于定义元素之间的间距、边距、字体大小等。1em 等于当前元素的 font-size 值，若某元素的 font-size 为 16px，则：1em = 16px，2em = 32px。

E.4 @firstparagraphindent: 标题后缩进

如本手册所示, 任何章节的首段默认情况下都不会缩进。从排版的角度来看, 缩进是一种段落分隔符, 这意味着在新章节开始时, 缩进是不必要的⁴。这种缩进是通过@firstparagraphindent命令来控制的:

`@firstparagraphindent word`

标题后的第一段会根据`word` 的值进行缩进:

- | | |
|---------------------|---|
| <code>none</code> | 防止第一个段落缩进, 这是默认设置。如果@paragraphindent asis生效, 则makeinfo会忽略本选项。 |
| <code>insert</code> | 段落正常缩进。遵循由@paragraphindent命令设置的段落缩进量 (请参阅节 E.3 [paragraphindent], 262 页)。 |

在 HTML 和 DocBook 输出中, @firstparagraphindent会被忽略。

最好在 Texinfo 文件开头的头部结束(end-of-header)行之前编写@firstparagraphindent命令, 以便区域格式化(region formatting)命令可以根据设定进行段落缩进。请参阅[Start of Header], 13 页。

E.5 @exampleindent: 环境的缩进量

Texinfo 处理器会对@example及类似环境中的每一行进行缩进。您可以使用@exampleindent命令来指定缩进。从一行的开头编写一个@exampleindent命令, 后面跟着‘asis’或一个数字:

`@exampleindent indent`

缩进量是根据`indent` 的值来确定的:

- | | |
|-------------------|---|
| <code>asis</code> | 不要更改现有的缩进 (在打印输出中未实现)。 |
| <code>0</code> | 取消所有缩进。 |
| <code>n</code> | 在 Info 输出中缩进 <code>n</code> 个空格字符, 在打印输出中缩进 <code>n</code> 个em ⁵ 。 |

在 Info 中, `indent` 的默认值是 5 个空格, 而在打印输出中则是 0.4 in, 这个值要小一些。(减少缩进是为了在打印手册中使物理行上能容纳更多字符。)

最好在 Texinfo 文件开头的头部结束(end-of-header)行之前编写@exampleindent命令, 这样区域格式化命令就能按照指定缩进量进行段落缩进。请参阅[Start of Header], 13 页。

E.6 @smallbook: 打印“小”书

默认情况下, T_EX会以 8.5 英寸乘 11 英寸的格式排版页面以供打印, 这是美国常用的“信纸”尺寸。然而, 您可以通过在 Texinfo 文件的开头, 标题页(title page)之前, 单独插入以下命令, 来指示 T_EX或 L^AT_EX以 7 英寸乘 9.25 英寸的格式排版文档, 这种格式适合装订成书:

`@smallbook`

(由于许多书籍的尺寸约为 7 英寸 × 9.25 英寸, 因此这个命令本应被称为@regularbooksize命令, 但与 8.5 英寸 × 11 英寸的格式相比, 它最终被称为@smallbook命令。)

⁴ 译者注: 中文的排版习惯, 新章节的首段也是要缩进的。

⁵ 译者注: 是 CSS (层叠样式表) 中的一种 相对长度单位, 用于定义元素之间的间距、边距、字体大小等。1em 等于当前元素的 font-size 值, 若某元素的 font-size 为 16px, 则 1em = 16px 2em = 32px。

参阅18.2节[Format with `texi2dvi` or `texi2pdf`], 142 页, 以及18.4节[Preparing for `TeX`], 146 页, , 了解使用`@smallbook`和 `TeX`进行格式化的其他方法, 这些方法无需更改源文件。

E.7 在 A4 纸上打印

您可以使用`@fourpaper`命令来设置文档格式, 以便在 ISO 216 A4 纸张尺寸上打印。请在 `Texinfo` 文件开头附近, 标题页(title page)之前, 单独一行写下该命令。

你可能喜欢或不喜欢命令`@fourlatex`产生的格式。此外, 还有`@fourwide`用于宽幅 A4 纸, `@fivepaper`用于 A5 纸, 以及`@sixpaper`用于 B6 纸。

参阅18.2节[Format with `texi2dvi` or `texi2pdf`], 142 页, 以及18.4节[Preparing for `TeX`], 146 页, 了解其他不需要更改源文件即可针对不同纸张尺寸进行格式化的方法。

E.8 `@pagesizes [width][, height]`: 自定义页面尺寸

您可以使用`@pagesizes`命令来明确指定页面上主文本区域的高度和(可选的)宽度。请在 `Texinfo` 文件开头附近, 标题页(title page)之前, 在单独的一行上写下此命令。高度在前, 如果需要, 宽度在后, 用逗号分隔。示例:

```
B5 纸
@pagesizes 200mm,150mm
```

及

```
法律(Legal)用纸
@pagesizes 11.5in
```

这对于在 B5 尺寸的纸张上打印是合理的。需要强调的是, 此命令指定的是文本区域的大小, 而非纸张的大小 (B5 尺寸的纸张为 250 mm×177 mm, 法律用纸为 14 in×8.5 in)。

要进行更精细的修改, 例如更改任何页边距, 您必须在`texinfo.tex`或`texinfo.cnf`中定义一个新的命令。

参阅18.2节[Format with `texi2dvi` or `texi2pdf`], 142 页, 以及18.4节[Preparing for `TeX`], 146 页, 了解其他不需要更改源文件即可指定`@pagesizes`的方法。

E.9 微排印

`TeX`的一些版本, 即 `pdfTeX`和 `LuaTeX`, 具有微排印(*microtypography*) 功能。这些功能包括轻微拉伸字体字形, 并允许文本略微延伸到页边距内。使用这些功能使 `TeX`在将段落拆分为行时更具灵活性, 并且可以通过减少断字和产生更一致的排版颜色来改善段落外观。

当这些特性(简称‘微排(microtype)’)可用时, `texinfo.tex`会使用它们。您可以在输入文件中指定‘`@microtype off`’来关闭微排; 同样, ‘`@microtype on`’可重新开启微排。

`LATeX`输出中使用的字体可能不可缩放。如果字体不可缩放, 在转换 `LATeX`输出时使用微排可能会引发错误。因此, 在默认情况下, `LATeX`输出中不会启用微排。可以通过‘`@microtype on`’启用微排。要获取默认 `LATeX`输出中使用的字体对应的可缩放字体, 可以安装 CM-Super 字体包 (<http://ctan.org/pkg/cm-super>)。

`@microtype`命令对其他输出格式无效。

E.10 缩放倍率

你可以尝试使用`\mag` TeX命令来指示 TeX排版比通常更大或更小的页面。所有排版的内容都会按比例放大或缩小。（`\mag`代表‘缩放’。）这不是 Texinfo 的 `@` 命令，而是一个以反斜杠为前缀的原始 TeX命令。你必须将此命令写在`@tex`和`@end tex`之间（请参阅15.3 节[Raw Formatter Commands], 123 页）。

在`\mag`命令后加上`=`，然后跟上一个数字，该数字是你所需缩放倍数的 1000 倍。例如，要以 1.2 倍的正常大小打印页面，请在 Texinfo 文件的开头附近，标题页之前，编写以下内容：

```
@tex
\global\mag=1200
@end tex
```

使用某些打印技术，向打印店提供比常规尺寸更大的原图，打印店进行缩小处理，从而有效提高分辨率，从而打印出比平时看起来更好的正常尺寸拷贝。

使用非标准`\mag`准备的 DVI 文件可能无法打印，或者只能以特定放大倍数打印，这取决于您使用的系统，实际情况需要做实验进行验证。

附录 F Info 格式规范

在此，我们将描述 Info 格式的技术细节。

在这份正式描述中，字符‘< > * () | = #’用于描述语言本身，其他字符为字面字符。所使用的标准结构一如平常：‘<...>’表示元变量名，‘=’表示定义，‘*’表示重复，‘?’表示可选，‘()’表示分组(grouping)，‘|’表示可交替(alternation)，‘#’表示注释。

一般来说，读取 Info 文件的程序应尽量对文件中出现的关键词（例如，‘Tag Table’和‘Tag table’应视为等同）不区分大小写，以便支持使用不同大小写格式的 Info 生成程序。

我们用<lparen>和<rparen>来指定字面括号（即 Info 格式中的括号），分别表示单个字符‘(’和‘)’。最后，对于任何x，两个字符序列‘^x’表示单个字符‘CTRL-x’。

Info 格式的定义是在 Info 格式首次设计约 25 年后编写的。因此，若本定义与实际操作发生冲突，以实际操作为准。此外，本定义还假设读者对 Texinfo 有一定了解；它旨在为实施者提供指导，而非严格的技术标准。对于示例和定义，我们可能会参考本手册的其他部分，而不会冗余地阐述每个细节。

F.1 Info 格式：整体的手册

首先，Info 手册可以是非分割的(*nonsplit*)（完全包含在一个文件中，也称整体手册）或分割的(*split*)（分布在多个文件中）。

非拆分手册的语法为：

```
<nonsplit info file(非分割的 info 文件)> =
<preamble(前导)>
<node(节点)>*
<tag table(标签表)>?
<local variables(本地变量)>?
```

在分割时，会生成一个主文件(*main file*)，其中仅包含指向其他子文件(*subfiles*)中给定节点的指针。主文件如下所示：

```
<split info main file(分割的 info 主文件)> =
<preamble(前导)>
<indirect table(间接表)>
<tag table(标签表)>
<local variables(本地变量)>?
```

拆分手册中的子文件具有以下语法：

```
<split info subfile (分割的 info 子文件)> =
<preamble(前导)>
<node (节点)>*
```

请注意，对于分割文件，标签表并非可选，因为它与间接表一起使用，以推断特定节点位于哪个子文件中。

Info 文件中的几个部分（节点、标签表、间接表、本地变量）以一个序列开头：

```
<separator(分隔序列)> = (^L)?^_(^L)?^J
```

即，一个‘CTRL-’字符后跟一个换行符，并可选择性地包含换页符。

F.2 Info 格式：前导

<preamble(前导)>是所有输出文件开头的文本。在默认设置下，它并不在 Info 查看器中显示，但可根据用户请求进行展示。

```
<preamble(前导)> =
<identification(身份识别)> # "This is FILENAME, produced by ..."
<copying text(复制许可)>   # @copying 的展开文本。
<dir entries(dir 条目)>     # 派生自 @dircategory 及 @direntary。
```

这些部分是：

```
<identification>
    位于输出文件开头的任意字符串，后跟一个空行。

<copying text>
    如果手册源文件中存在@copying (请参阅2.7.1 节[@copying], 15 页)，则对其进行展开。

<dir entries>
    手册源文件中所有@dircategory和@direntary命令形成的最终结果(请参阅20.1.4 节[Installing Dir Entries], 173 页)。
```

F.3 Info 格式：间接表

```
<indirect table(间接表)> =
<separator(分隔序列)>
Indirect:
(<filename(文件名)>: <bytepos(字节偏移)>)*
```

仅在拆分输出的情况下，间接表才会写入主文件。它以十进制整数形式指定了每个子文件（不包括顶级文件）按顺序连接时，各子文件的第一个节点的起始字节的偏移（从零开始）。实际内容的第一个节点由第一个条目指向。

作为一个例子，假设为 GDB 手册生成了分割输出。顶层文件gdb.info将包含类似以下内容：

```
<separator(分隔序列)>
Indirect:
gdb.info-1: 1878
gdb.info-2: 295733
...
```

这告诉 Info 阅读器，手册的第一个节点出现在文件gdb.info-1的第 1878 个字节处（位于该文件的前导部分之后）。如果将gdb.info-2 附加到gdb.info-1 之后，则gdb.info-2子文件中的第一个节点将从第 295733 个字节开始，包括两个文件中的所有前导部分。

遗憾的是，由于各种错误和疏忽，像makeinfo这样的 Info 创建程序并不总是能完美地实现这些规则。因此，健壮的 Info 阅读器应该回退到在给定位置的“附近”搜索节点，而不是在位置并非恰好位于节点开头时立即放弃。

F.4 Info 格式：标签表

```
<tag table(标签表)> =
<separator(分隔序列)>
```



```

Tag Table:
(<lparen(左括号)>Indirect<rparen(右括号)>)?
(Node|Ref): <nodeid>^?<bytepos(字节偏移)>
<separator(分隔序列)>
End Tag Table

```

‘(Indirect)’行仅在分割输出的情况下出现。

标签表指定了文件中每个节点或锚点的起始字节位置。在分割输出的情况下，该表仅写入主输出文件中。

按规定，每一行用锚点名或节点名定义一个标识符。例如，‘Node: Top^?1647’表示名为‘Top’的节点从字节 1647 开始，而‘Ref: Overview-Footnote-1^?30045’表示名为‘Overview-Footnote-1’的锚点从字节 30045 开始。同时以两种方式定义同一个标识符是错误的。

在非分割输出的情况下，字节位置仅指输出文件中的位置。在分割输出的情况下，字节位置指的是所有分割子文件（而非顶层文件）连接起来而形成的一个虚拟文件中的位置。请参阅上一节。

以下是一个示例：

```

^_
Tag Table:
Node: Top^?89
Node: Ch1^?292
^_
End Tag Table

```

这指定了一个包含两个节点（‘Top’和‘Ch1’）的手册，这两个节点分别位于字节位置 89 和 292。由于没有‘(Indirect)’行，因此该手册是非分割的。

文件的前导部分或其他非节点部分没有标签表条目。

F.5 Info 格式：本地变量

本地变量部分是可选的。它目前用于提供编码信息和或文档语言代码。

```

<local variables(本地变量)> =
<separator(分隔序列)>
Local Variables:
(coding: <encoding(编码)>)?
(Info-documentlanguage: <language(语言)>)?
End:

```

关于文档编码的信息，请参见14.2节[@documentencoding], 119 页。

‘<language>’的格式应为`ll`或`ll.cc`；请参见14.1节[@documentlanguage], 118 页。即使在 Texinfo 文档中@documentlanguage出现多次，局部变量部分也只能指定一种语言。

F.6 Info 格式：常规节点

常规节点看起来像这样：

```

<node> =
<separator(分隔序列)>
File: <fn>, Node: <id1>, (Next: <id2>, )? (Prev: <id3>, )? Up: <id4>

```

<general text(普通文本, 直到下一个‘^_’或者文件末尾)>

每个冒号和逗号后必须至少有一个空格或制表符, 但忽略其它任意数量的空格。<id>节点标识符的格式如下:

```
<id> = (<lparen><infofile(info 文件)><rparen>)?<node-spec(节点标识)>?
<node-spec(节点标识)> = <nodename(节点名)> |
                        <del><nodename(节点名)><del>
<del> = ^?
```

这个<node>在文件<fn>中定义了<id1>, 该文件通常为‘manualname(手册名称)’或‘manualname.info’。无括号的<infofile>组件可出现在<id1>中。

Next、Prev和Up后的每个标识符都指向文件中的节点或锚点。这些指针通常指向同一文件内部, 但‘(dir)’通常用于指向顶级目录文件。如果使用了<infofile>组件, 则可以省略节点名称, 在这种情况下, 节点标识符将指向所引用文件中的‘Top’节点。

Next和Prev指针是可选的。从技术上讲, Up指针也是可选的, 尽管这很可能表明节点结构中存在错误。通常, 节点被排列成树状结构, 但这并不是格式的要求。

包含句点、逗号、冒号或圆括号的节点名称可能会让 Info 读者感到困惑。如果必须引用名称中包含上述任何字符的节点, 则应在<nodename>前后加上一对DEL字符(‘CTRL-?’，字符编号 127)。在默认情况下, makeinfo会在需要时添加这些字符。请注意, 并非所有 Info 阅读器都能识别这种语法。请参阅[Info Node Names Constraints], 26 页。

节点的<general text>可以包含下文所述的特殊构造。

F.7 Info 格式: 菜单

传统上, 菜单出现在节点的末尾, 但 Info 格式对其位置没有限制。

```
<menu(菜单)> =
* Menu:
(<menu entry(菜单项)> | <menu comment(菜单注释)>)*
```

<menu entry>的组成在3.9.4 节[Menu Parts], 32 页中也进行了描述。它们与交叉引用的语法相同, 只是前面多了一个‘*’, 而不是‘* (N|n)ote’(请参阅节 F.10 [Info Format Cross Reference], 270 页)。索引扩展了菜单格式, 以指定目标行; 请参阅节 F.9 [Info Format Printindex], 270 页。

<menu comment>是指任何不以‘*’开头的行, 这些行要么出现在菜单的开头, 要么与菜单条目之间隔开一个或多个空行。这些注释作为菜单的一部分, 并按原样显示(请参阅3.9.1 节[Writing a Menu], 30 页)。

F.8 Info 格式: 图像

@image命令(请参阅9.2 节[Images], 76 页)会在 Info 文件中生成以下特殊指令:

```
<image(图像)> =
~@~H[image src="<image file(图像文件)>"
      (text="<txt file contents(文本化的文件内容)>")?
      (alt="<alt text(替代文本)>")?]
~@~H]
```

本说明中的换行和缩进是编辑上的; Info 文件中指令不同部分之间的空格是随意的。

在字符串<image file>、<txt file contents>和<alt text>中，‘”’被引用为‘\’，‘\’被引用为‘\\’。text和alt 在规范里是可选的。

alt值的作用与在 HTML 中起的作用相同：即对图像进行文字描述。例如，在纯文本显示或语音系统中，可以使用alt值来代替<image file>的图形化显示。

如果存在<txt file contents>，则应将其视为图像的 ASCII 表示，以便在纯文本显示上使用。

该格式并未规定在显示<image file>、<alt text>还是<txt file contents>之间如何做出选择。

F.9 Info 格式：打印索引

Info 格式的索引是一种菜单，在开头有一个额外的指令，用于将其标记为索引菜单。

```
<printindex> =
^@^H[index^@^H]
* Menu:
```

```
<index entry(索引条目)>*
```

<index entry>与普通菜单项类似，但自由格式的描述被索引条目在文本中出现的行号所取代：

```
<index entry(索引条目)> =
* <entry text(索引文本)>: <node-spec>. <line-spec>

<line-spec> =
<lparen>line <lineno><rparen>
```

初始部分‘<entry text>: <entry node>’应位于一个单行中。<entry text>是索引词。

尽管<entry text>后面跟有冒号，但它本身可能包含冒号，因此 Info 阅读器应尽量将该行中尽可能多的内容包含在<entry text>中。（但是，<entry text>中不能包含<node-spec>中可能出现的DEL字符。）

<lineno>是一个无符号整数，相对于<entry node>的起始位置给出。在‘<line-spec>’之前可能有一个可选的换行符。

以下是一个示例：

```
^@^H[index^@^H]
* Menu:

* thunder:           Weather Phenomena.           (line 5)
```

这意味着‘thunder’的索引条目出现在‘Weather Phenomena’节点的第 5 行。

F.10 Info 格式：交叉引用

Info 格式中的通用交叉引用具有以下两种形式之一：

```
<cross-reference> =
* (N|n)ote <id>::
| * (N|n)ote <label>:<id>(.|,)
```

```
<id> = (<lparen><infofile(info 文件名)><rparen>)?<node-spec>?
<label> = <label text> | <del><label text><del>
```

在‘*’符和随后的‘N’(或‘n’)之间不应有空格。‘*Note’应置于句首，否则应使用‘*note’。(某些 Info 阅读器，如 Emacs 中的阅读器，可以将‘*Note’和‘*note’分别显示为‘See’和‘see’。)在这两种情况下，<label text>都是描述性文本。

在这两种形式中，<id>指向一个节点或锚点，其方式与节点信息行中的引用相同（请参阅节 F.6 [Info Format Regular Nodes], 268 页）。可选的括号内的‘<infofile>’是所引用手册的名称，而<node-spec>则指定该手册中的节点或锚点。

第二种形式有一个描述性标签。在此形式中，交叉引用通常应以逗号或句号结尾，以便能够找到<id>的结尾。

如果<label>包含冒号字符(:)，则应使用一对字符将其括起来。同样，如果节点名称包含有问题的字符，也可以使用引用字符(^?)；此时不需要使用逗号或句号作为终结符。如前所述，并非所有 Info 阅读器都支持这种引用机制。

该格式并未规定如何查找其他手册以解决此类引用问题。

以下是一些示例：

```
*note GNU Free Documentation License::
*note Tag table: Info Format Tag Table, for details.
*Note Overview: (make)Top.
*Note ^?:^?: (bash)Bourne Shell Builtins.
*Note alloca.h: (gnulib)^?alloca.h^?.
```

第一个示例使用缩写形式（第一种格式）在当前手册中引用了一个节点。

第二处也指当前手册中的一个节点，即‘Info Format Tag Table’；‘:’之前的‘Tag table’只是此特定引用的标签，而‘for details.’（详情见）是句子中的文本，不属于引用的一部分。

第三个示例指的是另一本手册中的‘Top’节点，即：‘make’，其中‘Overview’是此交叉引用的标签。

第四个示例展示了在标签中的一个冒号字符如何被引用，第五个示例展示了在节点名称中的一个句点如何被引用。

请参阅第 5 章[Cross References], 39 页。

附录 G GNU 自由文档许可证

GNU Free Documentation License

Version 1.3, 3 November 2008

Copyright © 2000, 2001, 2002, 2007, 2008 Free Software Foundation, Inc.

<https://fsf.org/>

Everyone is permitted to copy and distribute verbatim copies of this license document, but changing it is not allowed.

0. PREAMBLE

The purpose of this License is to make a manual, textbook, or other functional and useful document *free* in the sense of freedom: to assure everyone the effective freedom to copy and redistribute it, with or without modifying it, either commercially or non-commercially. Secondly, this License preserves for the author and publisher a way to get credit for their work, while not being considered responsible for modifications made by others.

This License is a kind of “copyleft”, which means that derivative works of the document must themselves be free in the same sense. It complements the GNU General Public License, which is a copyleft license designed for free software.

We have designed this License in order to use it for manuals for free software, because free software needs free documentation: a free program should come with manuals providing the same freedoms that the software does. But this License is not limited to software manuals; it can be used for any textual work, regardless of subject matter or whether it is published as a printed book. We recommend this License principally for works whose purpose is instruction or reference.

1. APPLICABILITY AND DEFINITIONS

This License applies to any manual or other work, in any medium, that contains a notice placed by the copyright holder saying it can be distributed under the terms of this License. Such a notice grants a world-wide, royalty-free license, unlimited in duration, to use that work under the conditions stated herein. The “Document”, below, refers to any such manual or work. Any member of the public is a licensee, and is addressed as “you”. You accept the license if you copy, modify or distribute the work in a way requiring permission under copyright law.

A “Modified Version” of the Document means any work containing the Document or a portion of it, either copied verbatim, or with modifications and/or translated into another language.

A “Secondary Section” is a named appendix or a front-matter section of the Document that deals exclusively with the relationship of the publishers or authors of the Document to the Document’s overall subject (or to related matters) and contains nothing that could fall directly within that overall subject. (Thus, if the Document is in part a textbook of mathematics, a Secondary Section may not explain any mathematics.) The relationship could be a matter of historical connection with the subject or with related matters, or of legal, commercial, philosophical, ethical or political position regarding them.

The “Invariant Sections” are certain Secondary Sections whose titles are designated, as being those of Invariant Sections, in the notice that says that the Document is released under this License. If a section does not fit the above definition of Secondary then it is not allowed to be designated as Invariant. The Document may contain zero Invariant Sections. If the Document does not identify any Invariant Sections then there are none.

The “Cover Texts” are certain short passages of text that are listed, as Front-Cover Texts or Back-Cover Texts, in the notice that says that the Document is released under this License. A Front-Cover Text may be at most 5 words, and a Back-Cover Text may be at most 25 words.

A “Transparent” copy of the Document means a machine-readable copy, represented in a format whose specification is available to the general public, that is suitable for revising the document straightforwardly with generic text editors or (for images composed of pixels) generic paint programs or (for drawings) some widely available drawing editor, and that is suitable for input to text formatters or for automatic translation to a variety of formats suitable for input to text formatters. A copy made in an otherwise Transparent file format whose markup, or absence of markup, has been arranged to thwart or discourage subsequent modification by readers is not Transparent. An image format is not Transparent if used for any substantial amount of text. A copy that is not “Transparent” is called “Opaque”.

Examples of suitable formats for Transparent copies include plain ASCII without markup, Texinfo input format, LaTeX input format, SGML or XML using a publicly available DTD, and standard-conforming simple HTML, PostScript or PDF designed for human modification. Examples of transparent image formats include PNG, XCF and JPG. Opaque formats include proprietary formats that can be read and edited only by proprietary word processors, SGML or XML for which the DTD and/or processing tools are not generally available, and the machine-generated HTML, PostScript or PDF produced by some word processors for output purposes only.

The “Title Page” means, for a printed book, the title page itself, plus such following pages as are needed to hold, legibly, the material this License requires to appear in the title page. For works in formats which do not have any title page as such, “Title Page” means the text near the most prominent appearance of the work’s title, preceding the beginning of the body of the text.

The “publisher” means any person or entity that distributes copies of the Document to the public.

A section “Entitled XYZ” means a named subunit of the Document whose title either is precisely XYZ or contains XYZ in parentheses following text that translates XYZ in another language. (Here XYZ stands for a specific section name mentioned below, such as “Acknowledgements”, “Dedications”, “Endorsements”, or “History”.) To “Preserve the Title” of such a section when you modify the Document means that it remains a section “Entitled XYZ” according to this definition.

The Document may include Warranty Disclaimers next to the notice which states that this License applies to the Document. These Warranty Disclaimers are considered to be included by reference in this License, but only as regards disclaiming warranties: any other implication that these Warranty Disclaimers may have is void and has no effect on the meaning of this License.

2. VERBATIM COPYING

You may copy and distribute the Document in any medium, either commercially or noncommercially, provided that this License, the copyright notices, and the license notice saying this License applies to the Document are reproduced in all copies, and that you add no other conditions whatsoever to those of this License. You may not use technical measures to obstruct or control the reading or further copying of the copies you make or distribute. However, you may accept compensation in exchange for copies. If you distribute a large enough number of copies you must also follow the conditions in section 3.

You may also lend copies, under the same conditions stated above, and you may publicly display copies.

3. COPYING IN QUANTITY

If you publish printed copies (or copies in media that commonly have printed covers) of the Document, numbering more than 100, and the Document's license notice requires Cover Texts, you must enclose the copies in covers that carry, clearly and legibly, all these Cover Texts: Front-Cover Texts on the front cover, and Back-Cover Texts on the back cover. Both covers must also clearly and legibly identify you as the publisher of these copies. The front cover must present the full title with all words of the title equally prominent and visible. You may add other material on the covers in addition. Copying with changes limited to the covers, as long as they preserve the title of the Document and satisfy these conditions, can be treated as verbatim copying in other respects.

If the required texts for either cover are too voluminous to fit legibly, you should put the first ones listed (as many as fit reasonably) on the actual cover, and continue the rest onto adjacent pages.

If you publish or distribute Opaque copies of the Document numbering more than 100, you must either include a machine-readable Transparent copy along with each Opaque copy, or state in or with each Opaque copy a computer-network location from which the general network-using public has access to download using public-standard network protocols a complete Transparent copy of the Document, free of added material. If you use the latter option, you must take reasonably prudent steps, when you begin distribution of Opaque copies in quantity, to ensure that this Transparent copy will remain thus accessible at the stated location until at least one year after the last time you distribute an Opaque copy (directly or through your agents or retailers) of that edition to the public.

It is requested, but not required, that you contact the authors of the Document well before redistributing any large number of copies, to give them a chance to provide you with an updated version of the Document.

4. MODIFICATIONS

You may copy and distribute a Modified Version of the Document under the conditions of sections 2 and 3 above, provided that you release the Modified Version under precisely this License, with the Modified Version filling the role of the Document, thus licensing distribution and modification of the Modified Version to whoever possesses a copy of it. In addition, you must do these things in the Modified Version:

- A. Use in the Title Page (and on the covers, if any) a title distinct from that of the Document, and from those of previous versions (which should, if there were any, be listed in the History section of the Document). You may use the same title as a previous version if the original publisher of that version gives permission.
- B. List on the Title Page, as authors, one or more persons or entities responsible for authorship of the modifications in the Modified Version, together with at least five of the principal authors of the Document (all of its principal authors, if it has fewer than five), unless they release you from this requirement.
- C. State on the Title page the name of the publisher of the Modified Version, as the publisher.
- D. Preserve all the copyright notices of the Document.
- E. Add an appropriate copyright notice for your modifications adjacent to the other copyright notices.
- F. Include, immediately after the copyright notices, a license notice giving the public permission to use the Modified Version under the terms of this License, in the form shown in the Addendum below.
- G. Preserve in that license notice the full lists of Invariant Sections and required Cover Texts given in the Document's license notice.
- H. Include an unaltered copy of this License.
- I. Preserve the section Entitled "History", Preserve its Title, and add to it an item stating at least the title, year, new authors, and publisher of the Modified Version as given on the Title Page. If there is no section Entitled "History" in the Document, create one stating the title, year, authors, and publisher of the Document as given on its Title Page, then add an item describing the Modified Version as stated in the previous sentence.
- J. Preserve the network location, if any, given in the Document for public access to a Transparent copy of the Document, and likewise the network locations given in the Document for previous versions it was based on. These may be placed in the "History" section. You may omit a network location for a work that was published at least four years before the Document itself, or if the original publisher of the version it refers to gives permission.
- K. For any section Entitled "Acknowledgements" or "Dedications", Preserve the Title of the section, and preserve in the section all the substance and tone of each of the contributor acknowledgements and/or dedications given therein.
- L. Preserve all the Invariant Sections of the Document, unaltered in their text and in their titles. Section numbers or the equivalent are not considered part of the section titles.
- M. Delete any section Entitled "Endorsements". Such a section may not be included in the Modified Version.
- N. Do not retitle any existing section to be Entitled "Endorsements" or to conflict in title with any Invariant Section.
- O. Preserve any Warranty Disclaimers.

If the Modified Version includes new front-matter sections or appendices that qualify as Secondary Sections and contain no material copied from the Document, you may at

your option designate some or all of these sections as invariant. To do this, add their titles to the list of Invariant Sections in the Modified Version's license notice. These titles must be distinct from any other section titles.

You may add a section Entitled “Endorsements”, provided it contains nothing but endorsements of your Modified Version by various parties---for example, statements of peer review or that the text has been approved by an organization as the authoritative definition of a standard.

You may add a passage of up to five words as a Front-Cover Text, and a passage of up to 25 words as a Back-Cover Text, to the end of the list of Cover Texts in the Modified Version. Only one passage of Front-Cover Text and one of Back-Cover Text may be added by (or through arrangements made by) any one entity. If the Document already includes a cover text for the same cover, previously added by you or by arrangement made by the same entity you are acting on behalf of, you may not add another; but you may replace the old one, on explicit permission from the previous publisher that added the old one.

The author(s) and publisher(s) of the Document do not by this License give permission to use their names for publicity for or to assert or imply endorsement of any Modified Version.

5. COMBINING DOCUMENTS

You may combine the Document with other documents released under this License, under the terms defined in section 4 above for modified versions, provided that you include in the combination all of the Invariant Sections of all of the original documents, unmodified, and list them all as Invariant Sections of your combined work in its license notice, and that you preserve all their Warranty Disclaimers.

The combined work need only contain one copy of this License, and multiple identical Invariant Sections may be replaced with a single copy. If there are multiple Invariant Sections with the same name but different contents, make the title of each such section unique by adding at the end of it, in parentheses, the name of the original author or publisher of that section if known, or else a unique number. Make the same adjustment to the section titles in the list of Invariant Sections in the license notice of the combined work.

In the combination, you must combine any sections Entitled “History” in the various original documents, forming one section Entitled “History”; likewise combine any sections Entitled “Acknowledgements”, and any sections Entitled “Dedications”. You must delete all sections Entitled “Endorsements.”

6. COLLECTIONS OF DOCUMENTS

You may make a collection consisting of the Document and other documents released under this License, and replace the individual copies of this License in the various documents with a single copy that is included in the collection, provided that you follow the rules of this License for verbatim copying of each of the documents in all other respects.

You may extract a single document from such a collection, and distribute it individually under this License, provided you insert a copy of this License into the extracted document, and follow this License in all other respects regarding verbatim copying of that document.

7. AGGREGATION WITH INDEPENDENT WORKS

A compilation of the Document or its derivatives with other separate and independent documents or works, in or on a volume of a storage or distribution medium, is called an “aggregate” if the copyright resulting from the compilation is not used to limit the legal rights of the compilation’s users beyond what the individual works permit. When the Document is included in an aggregate, this License does not apply to the other works in the aggregate which are not themselves derivative works of the Document.

If the Cover Text requirement of section 3 is applicable to these copies of the Document, then if the Document is less than one half of the entire aggregate, the Document’s Cover Texts may be placed on covers that bracket the Document within the aggregate, or the electronic equivalent of covers if the Document is in electronic form. Otherwise they must appear on printed covers that bracket the whole aggregate.

8. TRANSLATION

Translation is considered a kind of modification, so you may distribute translations of the Document under the terms of section 4. Replacing Invariant Sections with translations requires special permission from their copyright holders, but you may include translations of some or all Invariant Sections in addition to the original versions of these Invariant Sections. You may include a translation of this License, and all the license notices in the Document, and any Warranty Disclaimers, provided that you also include the original English version of this License and the original versions of those notices and disclaimers. In case of a disagreement between the translation and the original version of this License or a notice or disclaimer, the original version will prevail.

If a section in the Document is Entitled “Acknowledgements”, “Dedications”, or “History”, the requirement (section 4) to Preserve its Title (section 1) will typically require changing the actual title.

9. TERMINATION

You may not copy, modify, sublicense, or distribute the Document except as expressly provided under this License. Any attempt otherwise to copy, modify, sublicense, or distribute it is void, and will automatically terminate your rights under this License.

However, if you cease all violation of this License, then your license from a particular copyright holder is reinstated (a) provisionally, unless and until the copyright holder explicitly and finally terminates your license, and (b) permanently, if the copyright holder fails to notify you of the violation by some reasonable means prior to 60 days after the cessation.

Moreover, your license from a particular copyright holder is reinstated permanently if the copyright holder notifies you of the violation by some reasonable means, this is the first time you have received notice of violation of this License (for any work) from that copyright holder, and you cure the violation prior to 30 days after your receipt of the notice.

Termination of your rights under this section does not terminate the licenses of parties who have received copies or rights from you under this License. If your rights have been terminated and not permanently reinstated, receipt of a copy of some or all of the same material does not give you any rights to use it.

10. FUTURE REVISIONS OF THIS LICENSE

The Free Software Foundation may publish new, revised versions of the GNU Free Documentation License from time to time. Such new versions will be similar in spirit to the present version, but may differ in detail to address new problems or concerns. See <https://www.gnu.org/licenses/>.

Each version of the License is given a distinguishing version number. If the Document specifies that a particular numbered version of this License “or any later version” applies to it, you have the option of following the terms and conditions either of that specified version or of any later version that has been published (not as a draft) by the Free Software Foundation. If the Document does not specify a version number of this License, you may choose any version ever published (not as a draft) by the Free Software Foundation. If the Document specifies that a proxy can decide which future versions of this License can be used, that proxy’s public statement of acceptance of a version permanently authorizes you to choose that version for the Document.

11. RELICENSING

“Massive Multiauthor Collaboration Site” (or “MMC Site”) means any World Wide Web server that publishes copyrightable works and also provides prominent facilities for anybody to edit those works. A public wiki that anybody can edit is an example of such a server. A “Massive Multiauthor Collaboration” (or “MMC”) contained in the site means any set of copyrightable works thus published on the MMC site.

“CC-BY-SA” means the Creative Commons Attribution-Share Alike 3.0 license published by Creative Commons Corporation, a not-for-profit corporation with a principal place of business in San Francisco, California, as well as future copyleft versions of that license published by that same organization.

“Incorporate” means to publish or republish a Document, in whole or in part, as part of another Document.

An MMC is “eligible for relicensing” if it is licensed under this License, and if all works that were first published under this License somewhere other than this MMC, and subsequently incorporated in whole or in part into the MMC, (1) had no cover texts or invariant sections, and (2) were thus incorporated prior to November 1, 2008.

The operator of an MMC Site may republish an MMC contained in the site under CC-BY-SA on the same site at any time before August 1, 2009, provided the MMC is eligible for relicensing.

ADDENDUM: How to use this License for your documents

To use this License in a document you have written, include a copy of the License in the document and put the following copyright and license notices just after the title page:

```
Copyright (C)  year  your name.
Permission is granted to copy, distribute and/or modify this document
under the terms of the GNU Free Documentation License, Version 1.3
or any later version published by the Free Software Foundation;
with no Invariant Sections, no Front-Cover Texts, and no Back-Cover
Texts. A copy of the license is included in the section entitled ``GNU
Free Documentation License''.
```

If you have Invariant Sections, Front-Cover Texts and Back-Cover Texts, replace the “with...Texts.” line with this:

```
with the Invariant Sections being list their titles, with
the Front-Cover Texts being list, and with the Back-Cover Texts
being list.
```

If you have Invariant Sections without Cover Texts, or some other combination of the three, merge those two alternatives to suit the situation.

If your document contains nontrivial examples of program code, we recommend releasing these examples in parallel under your choice of free software license, such as the GNU General Public License, to permit their use in free software.

命令索引

这是所有 @ 命令和各种 Emacs Lisp 函数的按字母顺序排列的列表。为了使列表更易于使用，列出的命令去掉了前面的‘@’。

!	?
! (句尾)..... 90	? (句尾)..... 90
! (end of sentence)..... 90	? (end of sentence)..... 90
"	^
" (元音变音符)..... 91	^ (扬抑音符)..... 91
" (umlaut accent)..... 91	^ (circumflex accent)..... 91
&	‘
& (字面意义的‘&’)..... 87	‘ (抑音符)..... 91
& (literal ‘&’)..... 87	‘ (grave accent)..... 91
’	@
’ (扬音符)..... 91	@ (literal ‘@’)..... 86
’ (acute accent)..... 91	
*	\
* (强制换行)..... 101	\ (literal ‘\’ in math)..... 87
¸	{
¸ (软音符)..... 91	{ (literal ‘{’)..... 86
¸ (cedilla accent)..... 91	}
—	} (literal ‘}’)..... 86
- (discretionary hyphen)..... 102	~
.	~ (波浪线音符)..... 91
. (句尾)..... 90	~ (tilde accent)..... 91
. (end of sentence)..... 90	
/	
/ (允许换行)..... 101	
:	
: (去除句尾额外的空白)..... 89	
: (suppress end-of-sentence space)..... 89	
=	
= (长音符)..... 91	
= (macron accent)..... 91	

A

aa	92
AA	92
abbr	54
acronym	54
ae	92
AE	92
afivepaper	264
afourlatex	264
afourpaper	264
afourwide	264
alias	135
allowcodebreaks	102
ampchar	87
anchor	44
appendix	35
appendixsec	36
appendixsection	36
appendixsubsec	37
appendixsubsubsec	37
apply	116
arrow	99
asis	70
atchar	86
author	17

B

b (bold font)	57
\backslashslash	87
backslashchar	86
bar-method 在 bar-class	114
bsixpaper	264
bullet	95
bye	23

C

c	8
caption	75
cartouche	65
center	17
centerchap	35
chapheading	36
chapter	35
cindex	80
cite	47
clear	125
click	99
clicksequence	99
clickstyle	99
code	49
codequotebacktick	88
codequoteundirected	88
columnfractions	72
comma	86
command	53
comment 注释	8

complete_tree_nodes_menus	161
complete_tree_nodes_missing_menu	161
contents	19
copying	15
copyright	94
cpindex	80

D

defblock	114
defcodeindex	84
defcvr	112
defcvx	107
deffn	107
deffnx	107
defindex	84
definfoenclose	137
defivar	113
defivarx	107
defline	114
defmac	108
defmacx	107
defmethod	114
defmethodx	107
defop	113
defopt	110
defoptx	107
defopx	107
defspec	108
defspecx	107
deftp	111
deftpx	107
deftypecv	112
deftypecvx	107
deftypefn	108
deftypefnnewline	110
deftypefnx	107
deftypefun	109
deftypefunx	107
deftypeivar	113
deftypeivarx	107
deftypeline	114
deftypemethod	114
deftypemethodx	107
deftypeopx	107
deftypevar	111
deftypevarx	107
deftypevrx	107
defun	108
defunx	107
defvar	110
defvarx	107
defvr	110
defvrx	107
DEL (comment character)	8
detailmenu	20
detypeop	114
detypev	111

dfn.....	53
dh.....	92
DH.....	92
dircategory.....	14
direntry.....	173
display.....	62
displaymath.....	94
dmn.....	91
docbook.....	123
documentdescription.....	181
documentencoding.....	119
documentlanguage.....	118
dotaccent.....	91
dotless.....	92
dots.....	95

E

email.....	55
\emergencystretch.....	147
emph.....	56
end.....	205
enddots.....	95
enumerate.....	69
env.....	53
equiv.....	98
error.....	97
errormsg.....	122
errormsg, 以及 T _E X 中的行号.....	138
errormsg, and line numbers in T _E X.....	138
euro.....	95
evenfooting.....	261
evenfootingmarks.....	262
evenheading.....	261
evenheadingmarks.....	262
everyfooting.....	261
everyfootingmarks.....	262
everyheading.....	261
everyheadingmarks.....	262
example.....	60
exampleindent.....	263
exclamdown.....	92
exdent.....	62
expansion.....	96
expose 在 windows.....	114

F

file.....	53
fill_gaps_in_sectioning.....	161
finalout.....	147
findex.....	80
firstparagraphindent.....	263
float.....	74
flushleft.....	63
flushright.....	63
fn_name(函数名).....	106
fnindex.....	80

fonttextsize.....	56
foo.....	87, 88
foobar.....	106, 109
footnote.....	78
footnotestyle.....	79
format.....	62
forward-word.....	105
frenchspacing.....	90
ftable.....	71
FuncExample.....	136

G

\gdef 在@tex内.....	123
\gdef within @tex.....	123
geq.....	96
\globaldefs 在@tex中.....	123
\globaldefs within@tex.....	123
group.....	103
guillemetleft.....	92
guillemetright.....	92
guillemotleft.....	92
guillemotright.....	92
guilsinglleft.....	92
guilsinglright.....	92

H

H (匈牙利元音变音符).....	91
H (Hungarian umlaut accent).....	91
hashchar.....	87
hbox.....	146
heading.....	36
headings.....	259
headitem.....	72
headitemfont.....	73
html.....	123
hyphenation.....	102

I

i (italic font).....	57
ifclear.....	126
ifcommanddefined.....	128
ifcommandnotdefined.....	128
ifdocbook.....	121
ifhtml.....	121
ifinfo.....	121
iflatex.....	121
ifnotdocbook.....	122
ifnothtml.....	122
ifnotinfo.....	122
ifnotlatex.....	122
ifnotplaintext.....	122
ifnottex.....	122
ifnotxml.....	122
ifplaintext.....	121
ifset.....	126

iftex.....	121
ifxml.....	121
ignore.....	9
image.....	76
include.....	140
indent.....	65
indentedblock.....	60
index.....	107
indicateurl.....	55
inforef.....	45
Info-validate.....	256
inlinefmt.....	123
inlinefmtifelse.....	123
inlineifclear.....	126
inlineifset.....	126
inlineraw.....	123
\input (原始 TeX 启动).....	9
insert_nodes_for_sectioning_commands.....	161
insertcopying.....	15
isearch-backward.....	107
isearch-forward.....	107
item in @itemize.....	67
item in @multitable.....	72
item in @table.....	70
itemize.....	67
itemx.....	71

K

kbd.....	50
kbinputstyle.....	50
key.....	50
kindex.....	80
kyindex.....	80

L

l.....	92
latex.....	123
LaTeX.....	94
lbracechar.....	86
leq.....	96
linemacro.....	136
link.....	45
\linkcolor.....	47
lisp.....	61
listoffloats.....	75
lowersections.....	38
L.....	92

M

macro.....	130
\mag (原始 TeX 放大倍数).....	265
\mag (raw TeX magnification).....	265
majorheading.....	36
makeinfo-buffer.....	246
makeinfo-kill-job.....	246
makeinfo-recenter-output-buffer.....	246
makeinfo-region.....	246
math.....	93
\mathopsup.....	94
menu.....	30
microtype.....	264
minus.....	95
move_index_entries_after_items.....	161
multitable.....	72

N

need.....	103
<newline>.....	89
next-error.....	246
node.....	24
nodesdescription.....	29
nodesdescriptionblock.....	29
noindent.....	64
novalidate.....	145

O

o.....	92
occur.....	241
occur-mode-goto-occurrence.....	239
oddfooting.....	261
oddfootingmarks.....	262
oddheading.....	261
oddheadingmarks.....	262
oe.....	92
OE.....	92
ogonek.....	91
option.....	53
ordf.....	92
ordm.....	92
O.....	92

P

page.....	103
page, 在@titlepage内.....	16
pagesizes.....	264
paragraphindent.....	262
part.....	37
pgindex.....	80
phoo.....	137
pindex.....	80
point.....	98
pounds.....	95
print.....	97

printindex..... 83
 process..... 109
 push..... 109
 pxref..... 44

Q

questiondown..... 92
 quotation..... 59
 quotedblbase..... 92
 quotedblleft..... 92
 quotedblright..... 92
 quoteleft..... 92
 quoteright..... 92
 quotesinglbase..... 92

R

r (roman font)..... 57
 raggedright..... 63
 raisesections..... 38
 rbracechar..... 86
 ref..... 43
 @refill..... 227
 regenerate_master_menu..... 161
 registeredsymbol..... 95
 relate_index_entries_to_table_entries.... 161
 result..... 96
 ringaccent..... 91
 rmacro..... 130

S

samp..... 51
 sansserif..... 57
 sc..... 56
 section..... 36
 seealso..... 82
 seeentry..... 82
 set..... 125
 setchapternewpage..... 258
 @setcontentsaftertitlepage..... 227
 setfilename..... 12
 @setshortcontentsaftertitlepage..... 228
 settitle..... 12
 shortcaption..... 75
 shortcontents..... 19
 shorttitlepage..... 17
 slanted..... 57
 smallbook..... 263
 smalldisplay..... 66
 smallexample..... 66
 smallformat..... 66
 smallindentedblock..... 66
 smalllisp..... 66
 smallquotation..... 66
 sortas..... 81
 sp..... 103

sp (标题页行间距)..... 17
 <space>..... 89
 ss..... 92
 strong..... 56
 sub..... 93
 subentry..... 81
 subheading..... 37
 subsection..... 36
 subsubheading..... 37
 subsubsection..... 37
 subtitle..... 17
 summarycontents..... 19
 sup..... 93
 syncodeindex..... 84
 synindex..... 84

T

t (typewriter font)..... 57
 tab..... 72
 <tab>..... 89
 table..... 70
 tex..... 123
 texinfo-all-menus-update..... 243
 texinfo-every-node-update..... 242
 texinfo-format-buffer..... 247
 texinfo-format-region..... 247
 texinfo-indent-menu-description..... 245
 texinfo-insert-braces..... 238
 texinfo-insert-@code..... 238
 texinfo-insert-@dfn..... 238
 texinfo-insert-dwim-@ref..... 238
 texinfo-insert-@end..... 238
 texinfo-insert-@example..... 238
 texinfo-insert-@item..... 238
 texinfo-insert-@kbd..... 238
 texinfo-insert-@node..... 238
 texinfo-insert-node-lines..... 245
 texinfo-insert-@noindent..... 238
 texinfo-insert-@samp..... 238
 texinfo-insert-@table..... 238
 texinfo-insert-@var..... 238
 texinfo-make-menu..... 242
 texinfo-master-menu..... 242
 texinfo-multiple-files-update..... 244
 texinfo-sequential-node-update..... 245
 texinfo-show-structure..... 239
 texinfo-start-menu-description..... 239
 texinfo-tex-buffer..... 247
 texinfo-update-node..... 242
 textdegree..... 95
 TeX..... 94
 th..... 92
 thischapter..... 261
 thischaptername..... 261
 thischapternum..... 261
 thisfile..... 261
 thispage..... 261

thissection..... 261
 thissectionname..... 261
 thissectionnum..... 261
 thistitle..... 261
 TH..... 92
 tie..... 103
 tieaccent..... 91
 tindex..... 80
 title..... 17
 titlefont..... 17
 titlepage..... 16
 today..... 261
 top..... 27
 tpindex..... 80

U

u (短音符)..... 91
 u (breve accent)..... 91
 ubaraccent..... 91
 udotaccent..... 91
 unmacro..... 131
 unnumbered..... 35
 unnumberedsec..... 36
 unnumberedsubsec..... 37
 unnumberedsubsubsec..... 37
 up-list..... 239
 uref..... 45

urefbreakstyle..... 46
 \urefurlonlylinktrue..... 47
 url..... 45
 \urlcolor..... 47
 U..... 99

V

v (抑扬音符)..... 91
 v (caron)..... 91
 value..... 125
 var..... 52
 verb..... 51
 verbatim..... 61
 verbatiminclude..... 141
 vindex..... 80
 vrindex..... 80
 vskip T_EX 垂直间距..... 18
 vtable..... 71

W

w..... 102

X

xml..... 123
 xref..... 43
 xrefautomaticsectiontitle..... 41

变量索引

这是一个按字母顺序排列的所有变量列表，主要是定制化变量。

A

after, @urefbreakstyle的值	47
after, value for @urefbreakstyle	47
AFTER_SHORT_TOC_LINES	195
AFTER_TOC_LINES	195
ASCII_DASHES_AND_QUOTES	162
ASCII_GLYPH	162
ASCII_PUNCTUATION	162
AUTO_MENU_DESCRIPTION_ALIGN_COLUMN	162
AUTO_MENU_MAX_WIDTH	162

B

BASEFILENAME_LENGTH	192
before, @urefbreakstyle的值	47
before, value for @urefbreakstyle	47
BEFORE_SHORT_TOC_LINES	195
BEFORE_TOC_LINES	195
BIG_RULE	194
BODY_ELEMENT_ATTRIBUTES	194
border-pattern 属于 Window	112, 113

C

CASE_INSENSITIVE_FILENAMES	191
CHAPTER_HEADER_LEVEL	195
CHECK_HTMLXREF	192
CHECK_MISSING_MENU_ENTRY	157
CHECK_NORMAL_MENU_STRUCTURE	157
CLASS_BEGIN_USEPACKAGE	163
CLOSE_DOUBLE_QUOTE_SYMBOL	162
CLOSE_QUOTE_SYMBOL	164
用于 HTML	194
code, @kbdinputstyle的值	50
COLLATION_LANGUAGE	158
COMMAND_LINE_ENCODING	158
CONTENTS_OUTPUT_LOCATION	190
CONVERT_TO_LATEX_IN_MATH	196
COPIABLE_LINKS	194
CPP_LINE_DIRECTIVES	158

D

DATE_IN_HEADER	193
DEBUG	158
DEF_TABLE	195
DEFAULT_RULE	194
dispensize	66
distinct, @kbdinputstyle的值	50
DO_ABOUT	190
DOC_ENCODING_FOR_INPUT_FILE_NAME	158
DOC_ENCODING_FOR_OUTPUT_FILE_NAME	158
DOCTYPE	164
用于 HTML	193
for HTML	193
documentlanguage 定制化变量	166
documentlanguage customization variable	166
DOCUMENTLANGUAGE_COLLATION	158
DUMP_TEXI	158
DUMP_TREE	159

E

enable	111
ENABLE_ENCODING	149, 156
END_USEPACKAGE	163
EPUB_CREATE_CONTAINER_FILE	182
避免 Archive::Zip 依赖	181
avoiding Archive::Zip dependency	181
EPUB_KEEP_CONTAINER_FOLDER	182
ERROR_LIMIT	149, 156
example, @kbdinputstyle的值	50
EXTENSION	164
用于 HTML	191
for HTML	191
EXTERNAL_CROSSREF_EXTENSION	192
EXTERNAL_CROSSREF_SPLIT	192
EXTERNAL_DIR	192
EXTRA_HEAD	194

F

fill-column	110
FILLCOLUMN	149, 156
FOOTNOTE_END_HEADER_LEVEL	195
FOOTNOTE_SEPARATE_HEADER_LEVEL	195
footnotestyle	149
FORCE	150, 156
FORMAT_MENU	159
用于 HTML	190
for HTML	190

H

HEADER_IN_TABLE	193
HEADERS	151, 156
用于 HTML	193
for HTML	193
HIGHLIGHT_SYNTAX	183
HIGHLIGHT_SYNTAX_DEFAULT_LANGUAGE	183
HTML_MATH	196
HTML_ROOT_ELEMENT_ATTRIBUTES	194
HTMLXREF_FILE	192
HTMLXREF_MODE	192

I

ICONS	193
IGNORE_REF_TO_TOP_NODE_UP	192
IGNORE_SPACE_AFTER_BRACED_COMMAND_NAME	159
IMAGE_LINK_PREFIX	192
INDENTED_BLOCK_COMMANDS_IN_TABLE	195
INDEX_ENTRY_COLON	195
INDEX_SPECIAL_CHARS_WARNING	162
INFO_JS_DIR	198
INFO_SPECIAL_CHARS_QUOTE	162
INFO_SPECIAL_CHARS_WARNING	163
INFOPATH	172
INLINE_CSS_STYLE	181
INPUT_FILE_NAME_ENCODING	159
INTERNAL_LINKS	151, 156

J

JS_WEBLABELS	198
JS_WEBLABELS_FILE	198

L

L2H_CLEAN	197
L2H_FILE	197
L2H_HTML_VERSION	197
L2H_L2H	197
L2H_SKIP	197
L2H_TMP	197
LATEX_FLOATS_FILE_EXTENSION	164
LOCALE_ENCODING	159

M

MACRO_EXPAND	151, 156
MATHJAX_CONFIGURATION	196
MATHJAX_SCRIPT	197
MATHJAX_SOURCE	197
MAX_HEADER_LEVEL	195
MAX_MACRO_CALL_NESTING	159
MENU_ENTRY_COLON	195
MENU_SYMBOL	195
MESSAGE_ENCODING	159
MONOLITHIC	190

N

NO_CSS	181
NO_CUSTOM_HTML_ATTRIBUTE	194
NO_INDEX	159
NO_NUMBER_FOOTNOTE_SYMBOL	196
NO_TOP_NODE_OUTPUT	165
用于 HTML	189
for HTML	189
NO_USER_COMMANDS	159
NO_WARN	151, 156
NODE_FILES	151, 156
NODE_NAME_IN_INDEX	191
NODE_NAME_IN_MENU	191
none, @urefbreakstyle的值	47
none, value for @urefbreakstyle	47
novalidate	156
NUMBER_FOOTNOTES	151, 156, 196
NUMBER_SECTIONS	152, 156

O

OPEN_DOUBLE_QUOTE_SYMBOL	163
OPEN_QUOTE_SYMBOL	165
用于 HTML	194
OUTFILE	152, 156
OUTPUT_CHARACTERS	165
用于 HTML	194
for HTML	194
OUTPUT_ENCODING_NAME	159
for HTML	193
OUTPUT_FILE_NAME_ENCODING	159

P

PACKAGE	160
PACKAGE_AND_VERSION	160
PACKAGE_NAME	160
PACKAGE_URL	160
PACKAGE_VERSION	160
page-delimiter	240
paragraphindent	156
PRE_BODY_CLOSE	194
PREFIX	160
用于 HTML	191
for HTML	191
PROGRAM	160
PROGRAM_NAME_IN_ABOUT	190
PROGRAM_NAME_IN_FOOTER	193

S

SECTION_NAME_IN_TITLE	193
SHORT_TOC_LINK_TO_TOC	192
SHOW_BUILTIN_CSS_RULES	181
SHOW_TITLE	189
SORT_ELEMENT_COUNT	160
SORT_ELEMENT_COUNT_WORDS	160
SPLIT	153, 156
用于 HTML	190
for HTML	190
SPLIT_SIZE	153, 156
SUBDIR	152, 156
用于 HTML	191
for HTML	191
SystemLiteral	164

T

T4H_LATEX_CONVERSION	198
T4H_MATH_CONVERSION	198
T4H_TEX_CONVERSION	198
TEST	160
TEXI2DVI	164
TEXI2HTML	189
texinfo_document Gettext 域	166
texinfo_document Gettext domain	166
TEXINFO_DTD_VERSION	164
TEXINFO_OUTPUT_FORMAT	156, 160
TEXINFO_XS	154
TEXINFO_XS_PARSER	154
TEXINPUTS	146
TEXTCONTENT_COMMENT	164
TOC_LINKS	192
TOP_FILE	191
TOP_NODE_FILE_TARGET	192
TOP_NODE_UP	165
用于 HTML	192
for HTML	192
TOP_NODE_UP_URL	192
TRACE_INCLUDES	153, 156
TRANSLITERATE_FILE_NAMES	153, 156
TREE_TRANSFORMATIONS	160
TXI_MARKUP_NO_SECTION_EXTENT	164
txicodequotebacktick	88

txicodequoteundirected	88
txicommandconditionals	128
txidefnamenspace	107
txiindexatsignignore	81
txiindexbackslashignore	81
txiindexhyphenignore	81
txiindexlessthanignore	81

U

UPDATED Automake 变量	234
UPDATED-MONTH Automake 变量	234
USE_ACCESSKEY	191
USE_ISO	194
USE_LINKS	193
USE_NEXT_HEADING_FOR_LONE_NODE	190
USE_NODE_DIRECTIONS	190
USE_NODES	165
用于 HTML	190
for HTML	190
USE_NUMERIC_ENTITY	165
用于 HTML	194
for HTML	194
USE_REL_REV	191
USE_SETFILENAME_EXTENSION	161
USE_TITLEPAGE_FOR_TITLE	189
USE_UNICODE_COLLATION	161
USE_UNIDECODE	162
USE_XML_SYNTAX	193

V

VERBOSE	153, 156
VERSION Automake 变量	234
VERTICAL_HEAD_NAVIGATION	193

W

WORDS_IN_PAGE	193
---------------------	-----

X

XREF_USE_FLOAT_LABEL	191
XREF_USE_NODE_NAME_ARG	191

通用索引

!

! 92

"

" (无方向双引号) 92
 „ (低双引号) 92
 „ (double low-9 quotation mark) 92
 " (undirected double quote character) 92

#

'#line'指令 138
 不是 TeX 处理器 138
 '#line' directive 138
 not processing with TeX 138
 syntax details 138

\$

\$Id 扩展, 防止 102
 \$Id expansion, preventing 102

&

'&#xhex;', 来自 @U 的输出 100
 '&#xhex;', output from @U 100

,

' 92
 „ 92

,

, (低单引号) 92
 , (single low-9 quotation mark) 92

—

- (in image alt string) 77, 102

<

< 92
 « 92

>

> 92
 » 92

?

? 92

—

—, breakpoint within @code 102
 —, @code 中的断点 102

‘

‘ 92
 ` 92

@

'@' as continuation in definition commands 106
 @ (字面 '@') 86
 '@' 作为定义命令中的延续 106

8

8-bit characters, in HTML cross-references 187
 8 位字符, HTML 交叉引用的 187

A

a 92
 A4 纸, 打印在 264
 A4 paper, printing on 264
 A5 纸张, 打印在 264
 A5 paper, printing on 264
 â 92
 Å 92
 Abbreviations for keys 50
 Abbreviations, tagging 54
 Abstract of document 181
 Abstract syntax tree representation of documents . 6
 Accents, inserting 91
 accesskey,
 customization variable for 203
 in HTML output of menus 31
 in HTML output of nodes 27
 用于...定制化变量 203
 <acknowledgements> DocBook tag 35
 Acronyms, tagging 54
 Acute accent 91
 Adding a new Info file 171
 Additional output formats 5
 --add-once, 用于 install-info 174
 --add-once, for install-info 174
 Advanced indexing 81
 Advice on writing entries 82
 æ 92
 Æ 92

- 奥克, 鸟类物种..... 93
- Aliases, command..... 135
- align=column, 用于install-info..... 174
- align=column, for install-info..... 174
- Allow line break..... 101
- All-permissive copying license..... 236
- Alphabetical @-command list..... 206
- Alt attribute for images..... 77
- Ampersand, inserting..... 87
- Anchors..... 44
- Angle quotation marks..... 92
- Another Info directory..... 172
- append-new-sections, 用于install-info.. 174
- append-new-sections, for install-info.... 174
- Archive::Zip, 用于 EPUB 文件输出..... 181
- Archive::Zip, for EPUB file output..... 181
- Arguments, repeated and optional..... 106
- ASCII 文本输出, 使用--plaintext..... 152
- ASCII, 使用 ASCII 码实现源文档的可移植性.... 99
- ASCII text output with --plaintext..... 152
- ASCII, source document portability using..... 99
- Aspect ratio of images..... 77
- @-commands..... 8
 - 语法..... 205
 - 定制化变量..... 155
 - 节点名称中的..... 25
 - customization variables for..... 155
 - in node names..... 25
 - list of..... 206
 - syntax..... 205
- At 符号, 插入..... 86
- At sign, inserting..... 86
- @ 命令..... 8
 - 列表..... 206
- @ 命令对应的定制化变量..... 155
- Auk, bird species..... 93
- automake, 及版本信息..... 234
- Automatic pointer creation with texi2any..... 24
- Automatic quoting of commas for some macros. 131
- Automatically insert nodes, menus..... 241
- Auxiliary files, omitting..... 145
- 安装 Info 文件..... 171
- 安装在另一个目录中的 Info 文件..... 172
- B**
- B5 纸, 打印在..... 264
- B5 paper, printing on..... 264
- Back-end output formats..... 3
- Backslash in macros..... 130
- Backslash, and macros..... 131
- Backslash, in macro arguments..... 132
- Backslash, inserting..... 86
- Backtick..... 88
- Badly referenced nodes..... 256
- Bastard title page..... 17
- Batch formatting for Info..... 257
- 版本号, 用于 install-info..... 176
- 版本控制关键字, 防止扩展..... 102
- 版本, Texinfo 的, 适应..... 128
- 保持文本的整体性, 在垂直方向上..... 103
- 编程, 图符..... 96
- 补丁, 贡献..... 2
- 绑定的空白..... 103
- 表达式的结果..... 96
- 表达式的值, 指示..... 96
- 表, 带索引的..... 71
- Beebe, Nelson..... 3
- Beginning line of a Texinfo file..... 11
- Berry, Karl..... 5
- 部分文档, 格式化..... 145
- 八分音符, 插入..... 87
- 部页面..... 37
- 报告错误..... 2
- 表格中的标题行..... 72
- 表格, 制作多列的..... 72
- 表格, 制作两栏..... 70
- 闭合标点和句尾..... 90
- 捕获错误, 使用 Info 格式化..... 253
- 被忽略的文本..... 9
- 编号, 为浮动..... 74
- 包含文件..... 140
- 包含文件的示例..... 140
- 包含文件, 要求..... 245
- 包含文件, 章节级别..... 38
- 包含许可文本..... 15
- 包含一个文件, 字面方式..... 141
- Big points..... 77
- 标记单词和短语..... 48
- 标记段落中的文本..... 48
- 不可换行的空白, 固定的..... 102
- 不可换行的空白, 可变的..... 103
- Black rectangle in hardcopy..... 147
- Blank lines..... 103
- 贝里, 卡尔..... 5
- 变量, 面向对象..... 112
- <blockquote> DocBook tag..... 59
- 波浪线音符..... 91
- 保留字, 标识..... 49
- 编码,
 - 输出文件名..... 159
 - 输入文件名..... 159
- 别名, 命令..... 135
- 编码, 声明..... 119
- Body of a macro..... 130
- <body> tag, attributes of..... 194
- <body> text, customizing..... 199
- <body>标签, 属性..... 194
- <body> 文本, 定制..... 199
- Bold font..... 57
- Bolio..... 6
- Book characteristics, printed..... 4
- Book, printing small..... 263
- BoTeX..... 6
- Box with rounded corners..... 65
- Box, ugly black in hardcopy..... 147

标签表	176
标签表, Info 格式的	267
标签表, 手工制作	252
版权符号	94
版权行中的年份	15
版权页	18
Brace-delimited conditional text	123
Brace-delimited flag conditionals	126
Braces and argument syntax	205
Braces, in index entries	145
Braces, in macro arguments	132
Braces, inserting	86
Braces, when to use	8
Breakpoints within URLs	46
Breaks in a line	101
Breaks, within <code>@code</code>	102
Breve accent	91
标示, 命令、定义等	48
不同的交叉引用命令	39
标题页	16
非正式	17
标题, 之后的缩进	263
Buffer formatting and printing	247
Bugs, reporting	2
编写菜单	30
编写定义时的约定	115
必需的, 在 Texinfo 文件中	9
编写 <code>@node</code> 行	24
编写索引条目	82
编译命令, 用于格式化	249
捕捉错误	253
捕捉错误, 使用 TeX 格式化	254
Bzip 压缩的目录文件, 读取	174
Bzipped dir files, reading	174
标志条件语句, 大括号分隔	126

C

<code>-c var=value</code>	153
<code>--calign=column</code> , 用于 <code>install-info</code>	174
<code>--calign=column</code> , for <code>install-info</code>	174
Capitalization of index entries	82
Captions, for floats	75
Caron	91
Cascading Style Sheets, and HTML output	180
Case in node name	26
Case, not altering in <code>@code</code>	49
Catching errors with Info formatting	253
Catching errors with TeX formatting	254
Catching mistakes	253
Catcode for comments in TeX	8
Categories, choosing	14
Category codes, of plain TeX	123
<code><caution></code> DocBook tag	59
Caveats for macro usage	132
尺寸, 打印书籍的	263
菜单	30
菜单编写	30

菜单的组成部分	32
菜单, Info 格式中	269
菜单描述, 开始	239
菜单示例	31
菜单, 省略, 使用 <code>--no-headers</code> 选项	151
菜单, 省略, 使用 <code>--plaintext</code>	152
菜单, 使用索引生成的	83
菜单条目, 带两个冒号的	32
菜单位置	32
层叠样式表(CSS)及 HTML 输出	180
Cedilla accent	91
Centimeters	77
重复参数和可选参数	106
Chapter structuring	34
<code><chapter></code> DocBook tag	35
Chapters, formatting one at a time	145
Character set, declaring	119
Characteristics, printed books or manuals	4
Characters, basic input	8
Characters, invalid in node name	26
' <code>@charset</code> ' specification, in CSS files	180
' <code>@charset</code> '规范, 在 CSS 文件中	180
Chassell, Robert J.	5
Check accent	91
Checking for badly referenced nodes	256
Checking for Texinfo commands	128
Checklist for bug reports	2
Chinese language	147
Chinese with XeTeX	147
Ciceros	77
Circumflex accent	91
创建非分割文件	257
创建节点及菜单, 自动	241
创建索引条目	80
创建指针, 使用 <code>texi2any</code>	24
参考, <code>@</code> 命令	206
参考实现	6
丑陋的黑块, 硬拷贝中	147
Click sequences	99
Closing punctuation, and sentence ending	90
CM-Super 字体, 安装	146
CM-Super 字体	93
LaTeX 的	264
CM-Super fonts	93
for LaTeX	264
installing	146
Code point of Unicode character, inserting by ..	99
Collapsing whitespace around continuations ...	106
Colon in node name	26
Colon, last in <code>INFOPATH</code>	172
<code><colophon></code> DocBook tag	35
Colored links, in PDF output	47
Column widths, defining for multitable	72
Combining indices	83
Comma after cross-reference	40
Comma in node name	26
Comma, in macro arguments	131

- Comma, inserting..... 86
- command, 用于`texi2dvi`..... 143
- Command aliases..... 135
- Command definitions..... 107
- Command names, indicating..... 53
- Command syntax..... 205
- command, for `texi2dvi`..... 143
- Command-line options of `texi2html`..... 169
- Commands in node names..... 25
- Commands to insert special characters..... 86
- Commands using raw $\TeX$ 123
- Commands, inserting them..... 237
- Commands, testing for Texinfo..... 128
- commands-in-node-names..... 148
- Comments..... 8
- Comments, in CSS files..... 180
- compatibility, with `texi2html`..... 203
- Compile command for formatting..... 249
- Compressed dir files, reading..... 174
- Computer Modern fonts..... 120
- Conditional commands, inline..... 123
- Conditionally visible text..... 121
- Conditionals, nested..... 128
- Conditions for copying Texinfo..... 1
- conf-dir=*dir*..... 148
- Configuration, for HTML cross-manual references..... 187
- Cons, Lionel..... 6, 169
- Container directory for EPUB..... 182
- Contents, after title page..... 227
- Contents, table of..... 19
- Contents-like outline of file structure..... 239
- Contexts, of @-commands..... 227
- Continuation lines in definition commands..... 106
- Control keys, specifying..... 50
- Controlling line breaks..... 101
- Conventions for writing definitions..... 115
- Conventions, syntactic..... 8
- Copying conditions..... 1
- Copying permissions..... 14
- Copying text, including..... 15
- Copyright 一词, 必须使用英文..... 15
- Copyright holder for FSF works..... 15
- Copyright page..... 18
- Copyright symbol..... 94
- Copyright word, always in English..... 15
- Correcting mistakes..... 253
- Country codes..... 118
- cp (概念)索引..... 80
- cp (concept) index..... 80
- 长破折号, 产生..... 8
- 插入 #..... 87
- 插入 &..... 87
- 插入常用命令..... 237
- 插入(多个)点..... 95
- Create nodes, menus automatically..... 241
- Creating an unsplit file..... 257
- Creating index entries..... 80
- Creating pointers with `texi2any`..... 24
- Critical editions..... 78
- 插入节点及菜单, 自动..... 241
- 插入空格..... 88
- Cross-reference configuration, for HTML..... 187
- Cross-reference parts..... 39
- Cross-reference targets, arbitrary..... 44
- Cross-references..... 39
- Cross-references customization, in HTML..... 192
- Cross-references using @inforef..... 45
- Cross-references using @link..... 45
- Cross-references using @pxref..... 44
- Cross-references using @ref..... 43
- Cross-references using @xref..... 43
- Cross-references, in HTML output..... 183
- Cross-references, in Info format..... 270
- 插入缩进..... 65
- 插入省略号..... 95
- 插入引号..... 92
- 插入引号字符..... 88
- 插入 @ (字面 '@')..... 86
- 插入重音符号..... 91
- 参数, 重复和可选..... 106
- 测试, 对 Texinfo 命令..... 128
- 参数的元语法字符..... 106
- 查塞尔, 罗伯特·J..... 5
- 产生换行和分页..... 101
- .cshrc 初始化文件..... 146
- .cshrc initialization file..... 146
- CSS 及 HTML 输出..... 180
- CSS, and HTML output..... 180
- css-include..... 149
- css-ref..... 149
- 测试, Texinfo 语言..... 6
- 初始化文件, HTML 图标方向按钮..... 193
- 程序中的表达式, 标识..... 49
- 纯 $\TeX$ 123
- CTRL-1..... 8
- Custom page sizes..... 264
- Customization variables for @-commands..... 155
- Customization variables for options..... 156
- Customize Emacs package (Development/Docs/Texinfo)..... 249
- Customizing of $\TeX$ for Texinfo..... 146
- 错误, 报告..... 2
- 错误报告核对清单..... 2
- 纯文本输出, 概览..... 3
- 纯文本输出, 使用--plaintext..... 152
- 错误, 捕捉..... 253
- 错误, 解析..... 246
- 错误信息中的行号..... 137
- 错误信息, 指示..... 97
- 错误引用的节点..... 256
- 程序名, 标识..... 53
- 长音符号..... 91
- 常用命令, 插入..... 237
- 插入特殊字符和符号..... 86
- 查找错误引用的节点..... 256

垂直方向上保持文本的整体性 103

D

-D var 149
 Dash, breakpoint within @code 102
 Dashes in source 8
 顶节点的下一个节点 27
 点 (尺寸) 77
 单词和短语, 对它们进行标记 48
 单词计数 157
 大点 77
 点 (多个), 插入 95
 换行, 位于@code内 102
 断点, 在 URL 中 46
 --debug, 用于install-info 174
 --debug, for install-info 174
 debugging document, with tree representation 157
 Debugging the Texinfo structure 253
 Debugging with Info formatting 253
 Debugging with TeX formatting 254
 <dedication> DocBook tag 35
 Default font 57
 Defining indexing entries 80
 Defining macros 130
 Defining new indices 84
 Defining new Texinfo commands 130
 Definition command headings, continuing 106
 Definition commands 105
 Definition conventions 115
 Definition lists, typesetting 70
 Definition of Info format 266
 Definition template 105
 Definitions grouped together 107
 Degree symbol 95
 --delete, 用于install-info 174
 --delete, for install-info 174
 Delimiter character, for verbatim 51
 Depth of text area 264
 Description for menu, start 239
 Description of document 181
 --description=text, 用于install-info 174
 --description=text, for install-info 174
 Detailed menu 20
 Details of macro usage 132
 detexinfo 157
 递归宏调用 130
 多个空白 89
 逗号, 插入 86
 导航栏, 底部 204
 导航链接, 省略 151
 导航栏, 在 HTML 输出中 179
 导航面板, 位于页面底部 204
 逗号, 在宏参数中 131
 逗号, 在节点名称中 26
 Didot points 77
 Different cross-reference commands 39
 Dimension formatting 91

Dimensions and image sizes 77
 dir, 由install-info创建 174
 dir 文件和 Info 目录 172
 (dir) as Up node of Top node 26
 Dir categories, choosing 14
 dir directory for Info installation 171
 dir file listing 171
 dir file, creating your own 172
 dir files and Info directories 172
 Dir files, compressed 174
 (dir)作为顶节点的上一级节点 26
 dir, created by install-info 174
 Direct formatting of Info files 252
 --dir-file=name, 用于install-info 174
 --dir-file=name, for install-info 174
 dir文件, 创建您自己的 172
 dir 文件列表 171
 dir, 用于安装 Info 的目录 171
 --disable-encoding 149
 Display formatting 62
 Displayed equation, in plain TeX 123
 Displayed equations 94
 Distorting images 77
 顶节点 20
 顶节点的上一级节点 26
 顶节点示例 20
 顶节点是第一个节点 26
 等价性, 指示 98
 单角引号 92
 大括号, 插入 86
 大括号分隔的标志条件语句 126
 大括号和参数语法 205
 大括号, 在宏实参数中 132
 大括号, 在索引条目中 145
 等宽字体 57
 多列表格的行 72
 多列表格的列宽 72
 度量单位的格式化 91
 段落缩进控制 262
 段落, 在里面标记文本 48
 杜马斯, 帕特里斯 6, 183
 --docbook 149
 DocBook 和序言部分 35
 DocBook, 包含原始内容 123
 DocBook 输出, 概述 3
 DocBook and prefatory sections 35
 DocBook output, overview 3
 DocBook, including raw 123
 DOCTYPE customization 193
 DOCTYPE 定制化 193
 Document description 181
 Document input encoding 119
 Document language, declaring 118
 Document permissions 14
 Document strings, internationalization of 166
 Document strings, translation of 118
 Document structure, of Texinfo 27
 Document title, specifying 12

--document-language 149
 Dot accent 91
 Dotless i, j 92
 Dots, inserting 95
 Double angle quotation marks 92
 Double guillemets 92
 Double left-pointing angle quotation mark 92
 Double low-9 quotation mark 92
 Double quotation marks 92
 Double right-pointing angle quotation mark 92
 Double structure, of Texinfo documents 27
 Double-colon menu entries 32
 短破折号, 产生 8
 --dry-run, 用于install-info 174
 --dry-run, for install-info 174
 度数符号 95
 低双引号 92
 DTD, for Texinfo XML 4
 DTD, 针对 Texinfo XML 4
 对 Texinfo 提出建议 2
 Dumas, Patrice 6, 183
 --dvi 149
 DVI 文件 144
 DVI 输出, 概览 3
 DVI, 以此格式输出 142
 DVI file 144
 DVI output, overview 3
 DVI, output in 142
 --dvi pdf 149
 --dvi pdf, 用于texi2dvi 142
 --dvi pdf, for texi2dvi 142
 dvipdfmx 142
 dvips 143
 dvips (program) 3
 大小写, 在@code中保持不变 49
 大小写, 在节点名中 26
 打印成本, 降低 56
 定义的模板 105
 定义的约定 115
 短音符 91
 第一个节点 26
 单引号 92, 130, 131
 打印和格式化, 在 Texinfo 模式下 247
 定义, Info 格式的 266
 调用(Invoking)节点, 在 dir 文件中的 173
 打印及格式化, 从 Emacs shell 142
 单右角引号 92
 定义列表, 排版 70
 定义命令 105
 定义命令的标题, 延续 106
 定义命令中的延续行 106
 调用pod2texi 166
 打印输出, 指示 97
 打印索引 83
 定义索引条目 80
 定义新的 Texinfo 命令 130
 打印许可 18
 定义新索引 84

打印一个区域或缓冲区 247
 德语引号 92
 打印硬拷贝 142
 定义, 组合到一起的 107
 断字, 帮助 T_EX 进行 102
 定制 Emacs 包 (Development/Docs/Texinfo) 249
 断字, 防止 102
 单左角引号 92
 打字机字体 57
 断字模式, 语言相关 118
 点重音符 91
 ð 92
 Ð 92

E

-e limit 149
 -E file 151
 EC 字体 93
 EC fonts 93
 EC fonts, installing 146
 EC 字体, 安装 146
 Ellipsis, inserting 95
 Em dash, compared to minus sign 95
 Em dash, producing 8
 Emacs 237
 Emacs shell, format, print from 142
 Emacs Shell, 格式化及打印 142
 Emphasizing text 55
 Emphasizing text, font for 56
 En dash, producing 8
 --enable-encoding 149
 Encoding,
 input file names 159
 output file names 159
 Encoding, declaring 119
 ‘end’节点脚注样式 79
 ‘end’ node footnote style 79
 End of header line 14
 Ending a Sentence 90
 Ending a Texinfo file 23
 Entity reference in HTML et al. 100
 Entries for an index 80
 Entries, making index 82
 --entry=text, 用于install-info 174
 --entry=text, for install-info 174
 Enumeration 69
 Environment indentation 263
 Environment variable INFOPATH 172
 Environment variable TEXINFO_OUTPUT_FORMAT 154
 Environment variable TEXINPUTS 146
 eps 图像格式 76
 eps image format 76
 epsf.tex 78
 epsf.tex, 安装 146
 epsf.tex, installing 146
 --epub3 149
 EPUB 3 输出, 概览 3

Epub 3 output, overview	3
Epub Container directory	182
Epub output file	182
Epub, generating	181
Epub 的容器目录	182
Epub, 生成	181
Epub 输出文件	182
Equation, displayed, in plain TeX	123
Equations, displayed	94
Equivalence, indicating	98
Error message, indicating	97
Error messages, line numbers in	137
--error-limit=limit	149
Errors, parsing	246
Escaping to HTML	179
Es-zet	92
etex	143
Eth	92
Euro font	95
Euro font, installing	146
Euro symbol, and encodings	119
Euro symbol, producing	95
European A4 paper	264
European Computer Modern fonts	93
installing	146
Evaluation glyph	96
Example indentation	263
Example menu	31
Examples in smaller fonts	66
Examples, formatting them	60
Examples, glyphs for	96
Expanding macros	131
Expansion of 8-bit characters in HTML	
cross-references	187
Expansion of macros, contexts for	132
Expansion, indicating	96
expansion, of node names in HTML	
cross-references	185
Expressions in a program, indicating	49
External macro processors	137

F

-f width	149
Family names, in all capitals	54
非分割文件的创建	257
福克斯, 布莱恩	5
分层文档和菜单	31
浮动,	
编号	74
创建无编号的	74
列表	75
浮动环境	74
浮动列表	75
浮动内容	74
浮动, 尚未实现	74
浮动重音, 插入	91
Features of Texinfo, adapting to	128

Feminine ordinal	92
feymr10	95
安装	146
installing	146
非分割 Info 文件	176
非分割手册, Info 格式	266
方法, 面向对象	113
非法字符, 在节点名称中	26
分隔符, 用于逐字	51
分割 HTML 输出	179
分割 Info 文件	176
分割 Info 文件, 手动	252
分割手册, Info 格式	266
分割输出文件	153
分割手册, 用于 HTML 交叉引用	189
返回类型, 单独一行	110
符合模式的行, 使用 occur 列出	241
File ending	23
File name collision	12
file recorder for TeX	143
File sectioning structure, showing	239
--fill-column=width	149
fill TeX 尺寸	18
Final output	146
Finding badly referenced nodes	256
Fine-tuning, and hyphenation	102
First line of a Texinfo file	11
First node	26
First paragraph, suppressing indentation of	263
Fixed-width font	57
附加输出格式	5
Flag conditionals, brace-delimited	126
分类列表, 选择	14
-F	150
Float environment	74
Floating accents, inserting	91
Floating, not yet implemented	74
Floats	74
list of	75
making unnumbered	74
numbering of	74
Flooding	44
法律用纸, 打印在	264
fn (函数) 索引	80
fn (function) index	80
Font for multitable heading rows	73
Font size, reducing	56
Fonts for indices	84
Fonts for printing	56
Footings	259
Footnote style,	
‘end’	79
‘separate’	79
Footnotes	78
--footnote-style=style	149
--force	150
Force line break	101
Forcing indentation	65

Forcing line and page breaks	101
Form feed character	8
Format a dimension	91
Format and print hardcopy	142
Format and print in Texinfo mode	247
Format with the compile command	249
Format, print from Emacs shell	142
Formats for images	76
Formatting commands	8
Formatting examples	60
Formatting for Info	245
Formatting headings and footings	259
Formatting partial documents	145
Formatting requirements	9
Formatting with <code>tex</code> and <code>texindex</code>	144
Formulas, mathematical	93
Four- and five- argument forms of cross-references	42
Fox, Brian	5
Free Documentation License, including entire ..	234
Free software	1
French quotation marks	92
French spacing	90
Frequently used commands, inserting	237
Frontmatter, text in	21
FSF 作品的版权持有人	15
法式空白	90
Full texts, GNU	234
Function definitions	107
Functions, in typed languages	108
反斜杠, 插入	86
反斜杠与宏	131
分页, 别扭的	101
分页, 强制	103
反引号	88
翻译输出文档中的字符串	166
法语引号	92
非正式标题页	17
防止首段缩进	263
复制条件	1
辅助文件, 省略	145
复制许可	14
复制许可的文本, 包含	15

G

固定宽度字体	57
高度, 图像	77
高度, 文本区域	264
General syntactic conventions	8
Generating EPUB	181
Generating HTML	179
Generating menus with indices	83
Generating page headings	19
Generating plain text files with <code>--no-headers</code> ..	151
Generating plain text files with <code>--plaintext</code> ..	152
German quotation marks	92
German S	92

规范化形式 C, Unicode	187
规范, Info 格式	266
孤行, 阻止	104
关键字, 标识	49
国家代码	118
国际化	118
国际化, 文档字符串	166
高级索引	81
Global Document Commands	258
Globbering	144
高亮, 文本	48
高亮显示, 自定义	137
Glyphs for programming	96
Glyphs for text	94
灌木	126
GNU 自由文档许可证, 全部	234
GNU 示例文本	234
GNU Emacs	237
GNU Emacs shell, format, print from	142
GNU Free Documentation License, including entire	234
GNU sample texts	234
Going to other Info files' nodes	32
Grave accent	91
standalone	88
vs. left quote	92
Group (在垂直方向上将文本保持整体)	103
Group (hold text together vertically)	103
Grouping two definitions together	107
格式化并打印硬拷贝	142
格式化, Info	245
格式化及打印, 从 Emacs shell	142
格式化示例	60
格式化, 使用编译命令	249
格式化一个度量单位	91
格式化页眉和页脚	259
公式, 数学	93
公式, 显示	94
公式, 显示, 使用用纯 <code>T_EX</code>	123
格式化部分文档	145
格式化和打印, 在 Texinfo 模式下	247
格式要求	9
GUI 点击序列	99
GUI click sequence	99
Guillemets	92
Guillemots	92
GNU Emacs shell, 格式化及打印	142
更新节点和菜单	241
更新命令所需的条件	243
GZip 压缩目录文件, 读取	174
估值图符	96

H

宏 130, 150
 Hacek accent 91
 Hardcopy, printing it 142
 Hash sign, inserting 87
 ‘hbox’, overfull 146
 ‘hbox’, 溢出 146
 合并索引 83
 徽标, $\TeX$ 94
 行长, 列宽占的百分比 72
 宏处理器, 外部 137
 缓冲区格式化和打印 247
 宏参数 130
 宏实参数中的反斜杠 132
 行, 多列表格中的 72
 宏的名称, 有效字符 130
 后端输出格式 3
 宏的详细信息 132
 宏调用 131
 宏定义, 编程语言 107
 宏定义, Texinfo 130
 <head> block, adding to 194
 <head> HTML tag, and <link> 203
 Header for Texinfo files 11
 Header of a Texinfo file 11
 <head> HTML 标签, 以及<link> 203
 Heading row, in table 72
 Headings 259
 Headings, indentation after 263
 <head>块, 添加到 194
 Height of images 77
 Height of text area 264
 --help, 用于texindex 174
 --help, for texi2any 150
 --help, for texindex 174
 help2man 5
 划分章节 34
 行宏 136
 别扭的 101
 控制 101
 换行, 避免在条件语句中出现 123
 换行符,
 URL 46
 换行, 防止 102
 行号, 在错误信息中 137
 Hierarchical documents, and menus 31
 Highlighting text 48
 Highlighting, customized 137
 Hints 229
 History of Texinfo 5
 行距 103
 环境变量INFOPATH 172
 环境变量TEXINFO_OUTPUT_FORMAT 154
 环境变量TEXINPUTS 146
 环境缩进 263
 花括号, 插入 86
 花括号分隔的条件文本 123
 花括号和参数语法 205

花括号, 何时使用 8
 黑块, 硬拷贝中 147
 宏展开, 上下文 132
 忽略@setfilename之前 12
 毫米 77
 宏名称, 有效字符 130
 Holder of copyright for FSF works 15
 Holding text together vertically 103
 获取 $\TeX$ 142
 宏, 取消定义 131
 href, 生成 HTML 45
 href, producing HTML 45
 函数, 在类型化语言中 108
 函数定义 107
 宏体 130
 --html 150
 HTML 输出,
 数学 93
 编码 119
 HTML 交叉引用,
 节点名扩展 185
 命令扩展 185
 HTML 的数学输出 93
 HTML, 包含原始内容 123
 HTML 及 CSS 180
 HTML 的 JavaScript 浏览界面 198
 HTML cross-references 183
 8-bit character expansion 187
 command expansion 185
 configuration 187
 configuration customization 192
 link basics 184
 node name expansion 185
 HTML output,
 browser compatibility 179
 encodings 119
 math 93
 customization 196
 overview 3
 split 179
 HTML translation 179
 HTML, and CSS 180
 HTML, including raw 123
 <meta> HTML 标签, 以及文档描述 181
 HTML 交叉引用,
 8 位字符的扩展 187
 HTML 交叉引用 183
 链接基础 184
 配置 187
 配置定制 192
 HTML 交叉引用中 8 位字符的扩展 187
 HTML 交叉引用中 8 位字符的音译 187
 HTML 输出,
 分割 179
 概述 3
 浏览器兼容性 179
 数学,
 定制 196

`htmlxref.cnf` 187
 HTML 中的交叉引用的定制化 192
 HTML, 转换成 179
`http-equiv`, 以及字符集规范 119
`http-equiv`, and charset specification 119
 Hungarian umlaut accent 91
 Hurricanes 43
 函数定义的示例 116
 哈希符号, 插入 87
 换页符 8
`Hyphen`, breakpoint within `@code` 102
`Hyphen`, compared to minus 95
 Hyphenation patterns, language-dependent 118
 Hyphenation, helping `TEX` do 102
 Hyphenation, preventing 102
 Hyphens in source, two or three in a row 8
 宏, 以整行为参数的 136
 盒子, 丑陋的黑色, 硬拷贝中 147
 宏中的反斜杠 130
 宏展开 131
 宏展开, 指示 96
 环状重音符 91

I

`i` (dotless i) 92
`-I dir` 150
`I18n`, 文档字符串的国际化 166
`I18n`, of document strings 166
 Icelandic 92
 If text conditionally visible 121
`--ifdocbook` 150
`--ifhtml` 150
`--ifinfo` 150
`--iflatex` 150
`--ifplaintext` 150
`--iftex` 150
`--ifxml` 150
 Ignored before `@setfilename` 12
 Ignored text 9
 Image formats 76
 Images,
 alternate text for 77
 in Info format 269
 scaling 77
 Implementation, `texi2any` as reference 6
 Implicit pointer creation with `texi2any` 24
`'@import'` specifications, in CSS files 180
`<important>` DocBook tag 59
`'@import'` 规范, 在 CSS 文件中 180
 Inches 77
`@include` 文件示例 140
 Include file sample 140
`@include` file sample 140
 Include files 140
 Include files requirements 245
 Include files, and section levels 38
 Including a file verbatim 141

Including permissions text 15
 Indentation undoing 62
 Indentation, forcing 65
 Indentation, omitting 64
 Indented text block 60
 Indenting environments 263
 Indenting paragraphs, control of 262
 Indenting, suppressing of first paragraph 263
 Index entries 80
 Index entries, advice on writing 82
 Index entries, making 82
 Index file names 144
 Index font types 81
 index sorting 81
 Indexing table entries automatically 71
 Indexing, advanced 81
 Indicating commands, definitions, etc. 48
 Indicating evaluation 96
 Indices 80
 Indices, combining them 83
 Indices, defining new 84
 Indices, in Info format 270
 Indices, printing and menus 83
 Indices, sorting 142
 Indices, two letter names 84
 Indirect subfiles 176
 Indirect table, in Info format 267
`--info` 150
 Info 文件 4
 手动分割 252
 制作标签表 252
 Info 格式化 245
 Info 格式规范 266
 Info 输出和编码 119
 Info 格式, 及菜单 32
 Info 文件名, 选择 12
 Info; 其他文件的节点 32
 Info 节点, Info 格式 268
 Info 输出, 概览 3
 Info batch formatting 257
 Info file name, choosing 12
 Info files 4
 installation 171
 listing a new 171
 making a tag table 252
 splitting manually 252
 Info format specification 266
 Info format, and menus 32
 Info formatting 245
 Info installed in another directory 172
 Info nodes, in Info format 268
 Info output, and encoding 119
 Info output, overview 3
 Info validating a large file 256
 Info; other files' nodes 32
`--info-dir=dir`, 用于 `install-info` 175
`--infodir=dir`, 用于 `install-info` 命令 175
`--info-dir=dir`, for `install-info` 175

Info 的批量格式化	257
--info-file=文件, 用于install-info	174
--info-file=file, for install-info	174
Info 文件,	
安装	171
列出新内容	171
Info 验证, 大文件	256
--init-file=file	151
Initialization file for T _E X input	146
Initialization file, HTML icon direction buttons	193
Inline conditionals	123
'\input' 源行被忽略	12
Input encoding, declaring	119
'\input' source line ignored	12
Insert nodes, menus automatically	241
Inserting @ (literal '@')	86
Inserting #	87
Inserting &	87
Inserting accents	91
Inserting dots	95
Inserting ellipsis	95
Inserting frequently used commands	237
Inserting indentation	65
Inserting quotation marks	92
Inserting quote characters	88
Inserting space	88
Inserting special characters and symbols	86
INSTALL file, generating	152
install-info	173
Installing and Info file	171
Installing Info in another directory	172
INSTALL文件, 生成	152
Internal links, of HTML	151
--internal-links=file	151
Internationalization	118
Internationalization of document strings	166
Introduction to Texinfo	2
Invalid characters in node names	26
Invoking macros	131
Invoking nodes, including in dir file	173
Invoking pod2texi	166
ISO 3166 国家代码	118
ISO 3166 country codes	118
ISO 639-2 语言代码	118
ISO 639-2 language codes	118
Italic font	57
--item=text, 用于install-info	174
--item=text, for install-info	174
Itemization	67

J

j (dotless j)	92
Japanese language	147
Japanese with LuaT _E X	147
Japanese with XeT _E X	147
JavaScript 许可证网页标签页	198
JavaScript browsing interface for HTML	198
JavaScript license web labels page	198
局部变量	249
本地变量部分, 在 Info 格式中	268
局部变量章节, 用于编码	119
局部格式化和打印	247
检查错误引用的节点	256
检查, 对 Texinfo 命令	128
交叉引用	39
交叉引用的单参数形式	40
交叉引用的四个参数和五个参数形式	42
交叉引用的双参数形式	40, 41
交叉引用的组成部分	39
交叉引用, Info 格式中	270
任意的	44
交叉引用配置, HTML	187
交叉引用, 使用@link	45
交叉引用使用@pxref	44
交叉引用, 使用@ref	43
交叉引用, 在 HTML 输出中	183
节点, 捕捉其错误	253
节点, 定义	24
节点分隔线被省略, 使用--plaintext	152
节点分隔线, 省略, 使用--no-headers 选项	151
节点行编写	24
节点行的要求	25
降低和升高章节层级	38
节点, 检查是否存在错误引用	256
节点名称必须唯一	25
节点名, 选择	25
节点名, 含非法字符	26
节点名扩展, 在 HTML 交叉引用中	185
简短目录	19
节点, 删除或重命名	44
键的术语缩写	50
节点, 'Top'	20
节点, 在其他 Info 文件中	32
句法词元, 标识	49
句法约定	8
结构, 捕捉其错误	253
结构, Texinfo 文档的	27
减号	95
井号, 插入	87, 89
句点, 在节点名称中	26
间接表, Info 格式中的	267
间距, 插入	88
简洁的菜单条目	32
间接子文件	176
jpeg 图像格式	76
jpeg image format	76
键盘输入	50
兼容性, 与texi2html	203

计算机现代字体.....	120
介绍 Texinfo	2
结束 Texinfo 文件.....	23
结束一个句子	90
键, 推荐名称	50
解析错误	246
矩形, 黑色, 硬拷贝中	147
减小字体大小	56
建议, 编写条目	82
角引号	92
转义, 自动为某些宏	131
脚注	78
纠正错误	253
句子的非结束标点	89
句尾的标点	90
句尾的空白	90
句子, 在其后的空白	90
最终输出	146
句子未结束	89
脚注样式, ‘end’	79

K

空白, 插入	89
空白, 垂直方向插入	103
空白, 在续符周围折叠	106
空白行	103
空白, 句子后的	90
空白, 水平插入	89
空白, 条件语句中的控制	123
空白, 在宏中	130
空白, 在节点名中	26
空白, 在句子中间	89
宽度, 定义多列表格的列	72
宽度, 图像	77
宽度, 文本区域	264
--keep-old, 用于install-info	175
--keep-old, for install-info	175
Keyboard input	50
Keys, recommended names	50
Keyword expansion, preventing	102
Keywords, indicating	49
宽高比, 图像的	77
空格, 在菜单中	30
空格, 在宏中	130
空格, 在节点名中	26
括号, 在节点名称中	26
快捷键, 在节点的 HTML 输出中	27
在 HTML 输出中的菜单中	31
克努特, 唐纳德	4
Knuth, Donald	4
可选参数和重复参数	106
康斯, 莱昂内尔	6
ky (击键)索引	80
ky (keystroke) index	80
展开, 宏的, 上下文	132

控制换行	101
控制键, 指定	50
扩展, 用于 HTML 交叉引用中的节点名	185

L

l	92
莱昂内尔, 康斯	169
lang, HTML attribute	199
lang, HTML 属性	199
--language, 用于texi2dvi	143
L	92
Language codes	118
Language, declaring	118
--language, for texi2dvi	143
Larger or smaller pages	265
--latex	151
L ^A T _E X, 包含原始内容	123
L ^A T _E X, 使用texi2dvi进行处理	143
L ^A T _E X 徽标	94
LaTeX 输出, 概览	3
L ^A T _E X logo	94
LaTeX output, overview	3
L ^A T _E X, including raw	123
L ^A T _E X, processing with texi2dvi	143
类别代码, 纯 T _E X	123
两部分菜单条目	32
列表和表格, 制作	67
列表, @ 命令	206
类别码, 在 T _E X中用于注释的	8
罗宾斯, 阿诺德	145
列出新的 Info 文件	171
里德, 布莱恩	5
Left quotation marks	92
Left-pointing angle quotation marks	92
Legal paper, printing on	264
Length of file names	12
Less cluttered menu entry	32
两个命名条目, @table	71
libintl-perl Gettext 实现	166
libintl-perl Gettext implementation	166
Libre software	1
License for all-permissive copying	236
License for verbatim copying	236
Limited scope of Texinfo	2
Line breaks, awkward	101
controlling	101
URLs	46
Line breaks, preventing	102
Line length, column widths as fraction of	72
Line macros	136
Line numbers, in error messages	137
Line spacing	103
‘#line’指令, 语法详情	138
<link> HTML tag, in <head>	203
<link> HTML 标签, 位于<head>中	203

Links, coloring in PDF output	47
Lisp 示例	61
Lisp example	61
Lisp examples in smaller fonts	66
Lisp 示例, 用小字体	66
List of @-commands	206
List of floats	75
Listing a new Info file	171
Lists and tables, making	67
Literate programming, with Texinfo and awk ..	145
连接音符	91
链接, 在 PDF 输出中着色	47
列宽, 用于多列表格定义	72
厘米	77
罗马字体	57
Local variable section, in Info format	268
Local variables	249
Local Variables section, for encoding	119
Locale, declaring	118
Location of menus	32
Logos, T_EX	94
Longest nodes, finding	160
Looking for badly referenced nodes	256
Lowering and raising sections	38
类@subsection命令	37
两条‘首行’, 用于@deffn	107
LuaTeX	147
类型化的变量	111
类型化的函数	108
类型化语言中的变量	111
另一个 Info 目录	172
连字符, @code中的断点	102
连字符, 与减号相比	95
Lzip 压缩的目录文件	174
Lzip-compressed dir files, reading	174
LZMA 压缩的目录文件, 读取	174
LZMA-compressed dir files, reading	174

M

Macro definitions, programming-language	107
Macro definitions, Texinfo	130
Macro details	132
Macro expansion, contexts for	132
Macro expansion, indicating	96
Macro invocation	131
Macro names, valid characters in	130
Macro processors, external	137
--macro-expand=file	151
Macron accent	91
Macros	130
Macros taking whole line as an argument	136
Macros, undefining	131
Magnified printing	265
makeinfo	148
makeinfo选项	148
makeinfo inside Emacs	246
makeinfo options	148

makeinfo, 在 Emacs 中调用	246
Making a printed manual	142
Making a tag table manually	252
Making cross-references	39
Making line and page breaks	101
Making lists and tables	67
Man page output, not supported	5
Man page, reference to	45
Manual characteristics, printed	4
Manual, referring to as a whole	43
Margins on page, not controllable	264
Marking text within a paragraph	48
Marking words and phrases	48
Masculine ordinal	92
Master menu	20
Math output for HTML	93
Mathematical expressions, inserting	93
--maxwidth=列数, 用于install-info	175
--max-width=column, 用于install-info	175
--max-width=column, for install-info	175
模板, 用于定义	105
目标, 用于交叉引用, 任意的	44
名称, 索引	84
名称, 推荐用于键	50
锚点	44
码点, Unicode 字符的, 插入	99
@menu的组成部分	32
Menu description, start	239
Menu entries with two colons	32
Menu example	31
Menu location	32
@menu parts	32
Menu parts	32
Menu writing	30
Menu, master	20
--menuentry=text, 用于install-info	175
--menuentry=text, for install-info	175
Menus	30
Menus generated with indices	83
Menus, in Info format	269
Menus, omitting with --no-headers	151
Menus, omitting with --plaintext	152
Meta keys, specifying	50
<meta> HTML 标签和字符集规范	119
<meta> HTML tag, and charset specification ..	119
<meta> HTML tag, and document description ..	181
META 键	51
META key	51
Meta-syntactic chars for arguments	106
Methods, object-oriented	113
冒号, 位于INFOPATH中的最后一个	172
冒号, 在节点名称中	26
Millimeters	77
Mils, @need的参数	103
Mils, argument to @need	103
Minimal requirements for formatting	9
Minimal Texinfo file (requirements)	9
Minus sign	95

Mistakes, catching	253
枚举列表	69
目录	19
命令, 插入它们	237
命令, 插入特殊字符	86
命令的别名	135
命令, 对 Texinfo...进行测试	128
命令定义	107
目录, 浮动的	75
目录分类列表, 选择	14
命令格式	8
命令行选项, texi2html 的	169
命令名, 标识	53
目录式大纲, 文件结构的	239
目录文件, 压缩的	174
目录, 位于标题页之后	227
命令的语法	205
命令, 在节点名中	25
Mode, using Texinfo	237
monolithic manuals, for HTML cross-references ..	189
Monospace font	57
默认字体	57
描述, 菜单, 开始	239
模式, 使用 Texinfo	237
描述, 文档的	181
Multiple dashes in source	8
Multiple spaces	89
Multitable column widths	72
Multitable rows	72
面向对象编程	112

N

--name=text, 用于install-info	175
--name=text, for install-info	175
Names for indices	84
Names of index files	144
Names of macros, valid characters of	130
Names recommended for keys	50
NASA, 作为缩略词	54
NASA, as acronym	54
Navigation bar, in HTML output	179
Navigation footer	204
Navigation links, omitting	151
Navigation panel, bottom of page	204
Need space at page bottom	103
Nested footnotes	78
Nesting conditionals	128
New index defining	84
New Info file, listing it in dir file	171
New Texinfo commands, defining	130
Newlines, avoiding in conditionals	123
NEWS file for Texinfo	128
NEWS Texinfo 文件	128
Next node of Top node	27
NLS	5
内联条件语句	123
@node 行编写	24

Node line requirements	25
Node line writing	24
@node line writing	24
node name expansion, in HTML	
cross-references	185
Node names must be unique	25
Node names, choosing	25
Node names, invalid characters in	26
Node separators, omitting with --no-headers ..	151
Node separators, omitting with --plaintext ..	152
Node, defined	24
Node, 'Top'	20
--node-files	151
--node-files, 以及 HTML 交叉引用	189
--node-files, and HTML cross-references	189
Nodes in other Info files	32
Nodes, catching mistakes	253
Nodes, checking for badly referenced	256
Nodes, deleting or renaming	44
--no-headers	151
--no-ifdocbook	150
--no-ifhtml	150
--no-ifinfo	150
--no-iflatex	150
--no-ifplaintext	150
--no-iftex	150
--no-ifxml	150
--no-indent, 用于install-info	175
--no-indent, for install-info	175
Non-breakable space, fixed	102
Non-breakable space, variable	103
--no-node-files	151
nonsplit Info file	176
Nonsplit manuals, Info format of	266
--no-number-footnotes	151
--no-number-sections	152
--no-pointer-validate	151
Normalization Form C, Unicode	187
--no-split	153
Not ending a sentence	89
<note> DocBook tag	59
--no-validate	151
--no-warn	151
扭曲图像	77
Number sign, inserting	87
Numbering of floats	74
--number-sections	152

O

ø	92
-o file	152
Ø	92
O'Dea, Brendan	5
Object-oriented programming	112
Oblique font	57
Obtaining T _E X	142
Occurrences, listing with <code>occur</code>	241
Octotherp, inserting	87
œ	92
Œ	92
奥格涅克变音符	91
Ogonek diacritic	91
Omitting indentation	64
One-argument form of cross-references	40
'\openout' line in log file	143
Optional and repeated arguments	106
Options for <code>makeinfo</code>	148
Options for <code>texi2any</code>	148
Options of <code>texi2html</code>	169
Options, customization variables for	156
Ordinals, Romance	92
Ordinary T _E X commands, using	123
Orphans, preventing	104
Other Info files' nodes	32
Outline of file structure, showing	239
Output document strings, internationalization of	166
output file name	12
Output file splitting	153
Output formats	3
Output formats, supporting more	5
Output, in PDF	142
Output, printed through <code>texi2any</code>	154
--output=file	152
--outputindent	152
Outputting EPUB	181
Outputting HTML	179
Overfull 'hboxes'	146
Overview of Texinfo	2
Owner of copyright for FSF works	15
欧元符号及其编码	119
欧元符号, 生成	95
欧元字体	95
欧元字体, 安装	146
欧洲 A4 纸	264
欧洲计算机现代字体	93
安装	146

P

-p indent	152
Page breaks, awkward	101
Page breaks, forcing	103
Page delimiter in Texinfo mode	240
Page headings	259
Page numbering	259
Page sizes for books	263
Page sizes, customized	264
Pages, starting odd	258
Paper size, A4	264
Paragraph indentation control	262
Paragraph, marking text within	48
--paragraph-indent=indent	152
Parameters to macros	130
Parentheses in node name	26
Parsing errors	246
Part of file formatting and printing	247
Part pages	37
Partial documents, formatting	145
Parts of a cross-reference	39
Parts of a master menu	20
Parts of a menu	32
Patches, contributing	2
--pdf	152
pdf 内含图像	76
--pdf, 用于texi2dvi	142
pdf image inclusions	76
--pdf, for texi2dvi	142
PDF 输出	142
PDF 输出, 概览	3
PDF output	142
PDF output of URLs	47
PDF output, overview	3
pdftex	145
pdftex, 及图像	76
pdftex, and images	76
pdftexi2dvi	142
Period in node name	26
Periods, inserting	89
Perl extension modules (XS)	154
Perl format strings for translation	166
Perl Pod, 转换为 Texinfo 格式	166
Perl Pod, converting to Texinfo	166
Perl 格式字符串, 用于翻译	166
Perl 扩展模块(XS)	154
Permissions text, including	15
Permissions, printed	18
pg (程序)索引	80
pg (program) index	80
Picas	77
Pictures, inserting	76
皮卡	77
Plain hyperlink	45
Plain link	45
Plain text output with --plaintext	152
Plain text output, overview	3
Plain T _E X	123

<code>--plaintext</code>	152
批量进行 Info 格式化	257
<code>png</code> 图像格式	76
<code>png image format</code>	76
<code>-P</code>	152
Pod, 正在转换为 Texinfo 格式	166
Pod, converting to Texinfo	166
<code>pod2texi</code>	166
Point, indicating in a buffer	98
Pointer creation with <code>texi2any</code>	24
Pointer validation	151
suppressing	145
Points (dimension)	77
PostScript 输出, 概览	3
PostScript output, overview	3
Pounds symbol	95
Preamble	13
Preamble, in Info format	267
Predefined names for indices	84
Preface, etc., and DocBook	35
<code><preface></code> DocBook tag	35
Preparing for $\TeX$	146
Prev node of Top node	27
Preventing first paragraph indentation	263
Preventing line and page breaks	101
Print and format in Texinfo mode	247
Print, format from Emacs shell	142
Printed book and manual characteristics	4
Printed output, indicating	97
Printed output, through <code>texi2any</code>	154
Printed permissions	18
Printing a region or buffer	247
Printing an index	83
Printing cost, reducing	56
Printing hardcopy	142
Problems, catching	253
<code>.profile</code> initialization file	146
Program names, indicating	53
Programming, glyphs for	96
Pronunciation of Texinfo	2
Prototype row, column widths defined by	72
<code>--ps</code>	153
<code>--ps</code> , 用于 <code>texi2dvi</code>	143
<code>--ps</code> , for <code>texi2dvi</code>	143
排序键, 索引条目的	81
普通超链接	45
普通链接	45
普通 $\TeX$ 命令, 使用	123
排序, 按大小对节点进行	160
拼写检查	157
排序索引	142
评注版本	78
破折号, <code>@code</code> 内的断点	102
破折号, 与减号比较	95
配置, 用于 HTML 跨手册引用	187

Q

前导	13
前导, Info 格式中的	267
强调, 文本	55
强调文本, 字体	56
全局性的文档命令	258
全开放复制许可证	236
窍门	229
起始新的一章	258
其他 Info 文件的节点	32
嵌套脚注	78
嵌套条件语句	128
<code>--quiet</code> , 用于 <code>install-info</code>	175
<code>--quiet</code> , for <code>install-info</code>	175
Quotation characters (''), in source	92
Quotation marks, German	92
Quotation marks, inserting	92
Quotations	59
Quotations in smaller fonts	66
Quote characters, inserting	88
Quoting, automatic for some macros	131
全文, GNU	234
倾斜的打字机字体, 用于 <code>@kbd</code>	50
取消宏定义	131
取消首段缩进	263
前言等, 以及 DocBook	35
区域打印, 在 Texinfo 模式下	247
区域格式化和打印	247
区域设置, 声明	118
强制换行	101
强制换行和分页	101
强制缩进	65
全站 Texinfo 配置文件	146
前面的内容, 文本中	21

R

Ragged left, without filling	63
Ragged right, with filling	63
Ragged right, without filling	63
Raising and lowering sections	38
Raw formatter commands	123
Raw HTML	179
raw text output	157
Recommended names for keys	50
Rectangle, black in hardcopy	147
Recursive macro invocations	130
Reducing font size	56
Reference implementation	6
Reference to @-commands	206
References	39
References using <code>@inforef</code>	45
References using <code>@pxref</code>	44
References using <code>@ref</code>	43
References using <code>@xref</code>	43
Referring to an entire manual	43
Referring to other Info files	32
<code>--regex=regex</code> , 用于 <code>install-info</code>	175

--regex=regex, for install-info	175
Region formatting and printing	247
Region printing in Texinfo mode	247
Registered symbol	95
Regular expression, for '#line'	138
Reid, Brian	5
--remove, 用于install-info	174
--remove, for install-info	174
--remove-exactly, 用于install-info	175
--remove-exactly, for install-info	175
Repeated and optional arguments	106
Reporting bugs	2
Required in Texinfo file	9
Requirements for formatting	9
Requirements for include files	245
Requirements for updating commands	243
Reserved words, indicating	49
Restrictions on node names	25
Result of an expression	96
Return type, own line for	110
RGB 颜色规范。	47
RGB color specification.	47
如果文本条件可见	121
ridt.eps	78
Right quotation marks	92
Right-pointing angle quotation marks	92
Ring accent	91
Robbins, Arnold	145
Roman font	57
Romance ordinals	92
Rounded rectangles, around text	65
Rows, of a multitable	72
原始格式化命令	123
Running an Info formatter	245
Running macros	131
Running makeinfo in Emacs	246
Running texi2any in Emacs	246
日文	147
日文, 使用 LuaTeX	147
日文, 使用 XeTeX	147
允许换行	101
运行 Info 格式化程序	245
运行makeinfo, Emacs 中	246
运行texi2any, Emacs 中	246
软音符	91
日志文件中的'\openout'行	143

S

-s style	149
Sample function definition	116
Sample @include file	140
Sample Texinfo file, no comments	9
Sample texts, GNU	234
Sans serif font	57
S-表达式, 输出格式	157
Scaled points	78
Scaling images	77

生成纯文本文件, 带--no-headers 选项	151
生成纯文本文件, 使用--plaintext	152
手册的特点, 印刷	4
生成带有索引的菜单	83
生成 EPUB	181
输出 EPUB 格式	181
输出格式	3
输出格式, 支持更多	5
生成 HTML	179
双重结构, Texinfo 文档的	27
Scribe	5
实现, texi2any作为参考	6
输出, 通过texi2any打印	154
输出文档的字符串, 国际化	166
输出文件分割	153
输出文件名	12
手册页, 参考	45
生成页眉	19
输出, 用 PDF 格式	142
手册页输出, 不受支持	5
手册, 整体引用	43
升调符号(非), 插入	87
首段, 取消缩进	263
深度, 文本区域	264
Sea surges	43
--section regex sec, 用于install-info	175
--section regex sec, for install-info	175
--section=sec, 用于install-info	175
--section=sec, for install-info	175
Sectioning	34
Sectioning structure of a file, showing	239
Sections, raising and lowering	38
Semantic markup	2
Sentence ending punctuation	90
Sentence non-ending punctuation	89
Sentences, spacing after	90
'separate'脚注样式	79
'separate' footnote style	79
Sequence of clicks	99
--set-customization-variable var=value	153
S-expressions, output format	157
缩放点	78
缩放打印	265
缩放图像	77
升高和降低章节层级	38
SGML-tools 输出格式	5
SGML-tools output format	5
Sharp sign (not), inserting	87
Sharp S	92
Shell 使用tex和texindex进行格式化	144
Shell formatting with tex and texindex	144
Shell, format, print from	142
Shell, running makeinfo in	246
Shell, running texi2any in	246
Shell, 格式化及打印	142
Shell, 运行makeinfo	246
Shell, 运行texi2any	246
Short captions, for lists of floats	75

Short table of contents	19	Spacing, at ends of sentences	90
Showing the sectioning structure of a file	239	Spacing, in the middle of sentences	89
Showing the structure of a file	240	Spacing, inserting	88
Shrubby	126	Special characters, inserting	86
--silent, 用于install-info	175	Special displays	74
--silent, for install-info	175	Special insertions	86
Single angle quotation marks	92	Specification of Info format	266
Single guillemets	92	Specifying index entries	80
Single left-pointing angle quotation mark	92	spell checking	157
Single low-9 quotation mark	92	Spelling of Texinfo	2
Single quotation marks	92	Split HTML output	179
Single right-pointing angle quotation mark	92	split Info file	176
Site-wide Texinfo configuration file	146	split manuals, for HTML cross-references	189
Size of printed book	263	Split manuals, Info format of	266
缩进撤销	62	--split=how	153
缩进段落, 控制	262	--split-size=num	153
缩进环境	263	Splitting an Info file manually	252
缩进, 强制	65	Splitting of output files	153
缩进, 首段取消	263	输入编码, 声明	119
缩进, 省略	64	ß	92
书籍特点, 印刷	4	Stallman, Richard M.	5
缩进文本块	60	Start of header line	13
书籍, 小开本打印	263	Starting chapters	258
双角引号	92	斯托尔曼, 理查德·M.	5
Slanted font	57	stripping Texinfo commands	157
Slanted typewriter font, for @kbd	50	Structure of a file, showing	239
示例菜单	31	Structure, catching mistakes in	253
示例的缩进	263	Structure, of Texinfo documents	27
示例, 格式化它们	60	Structuring of chapters	34
省略号, 插入	95	实体引用, 在类 HTML 中	100
示例@include文件	140	Subscripts and superscripts, text	93
省略缩进	64	Subsection-like commands	37
示例 Texinfo 文件, 无注释	9	Subsub sectioning commands	37
示例文本, GNU	234	Suggestions for Texinfo, making	2
示例, 用小字体	66	Summary of document	181
示例, 图符	96	Suppressing first paragraph indentation	263
Small book size	263	Suppressing indentation	64
Small caps font	56	SVG 图像, 用于 DocBook	76
Small examples	66	SVG images, used in DocBook	76
Small verbatim	61	数学表达式, 插入	93
<small>标签	56	缩写、标记	54
<small> tag	56	树形表示, 用于调试	157
Smaller fonts	56	缩略词, 标记	54
双冒号菜单条目	32	SXML output	157
Smith, Gavin	5	SXML 输出	157
史密斯, 加文	5	上下文, @ 命令的	227
sort keys for index entries	81	索引	80
Sorting indices	142	索引、打印和菜单	83
Sorting nodes by size	160	索引, 定义新的	84
Source file format	2	索引的字体类型	81
Source files, characters used	8	索引, 高级	81
source-highlight	183	上一个节点, 顶节点的	27
Space, after sentences	90	双引号	92
Space, inserting horizontal	89	索引, Info 格式中	270
Space, inserting vertical	103	使用@inforef的交叉引用	45
Spaces in macros	130	使用@inforef的引用	45
Spaces in node name	26	使用Info-validate	256
Spaces, in menus	30	使用技巧	229

索引, 将它们合并起来	83
双右角引号	92
索引, 两个字母的名称	84
Syntactic conventions	8
Syntactic tokens, indicating	49
Syntax details, ‘#line’	138
Syntax tree representation of documents	6
Syntax, of @-commands	205
Syntax, optional & repeated arguments	106
索引, 排序	142
索引排序	81
使用 Texinfo 和 <code>awk</code> 的文学编程	145
使用 <code>tex</code> 和 <code>texindex</code> 进行格式化	144
索引条目	80
索引条目, 编写建议	82
索引条目, 制作	82
索引文件名	144
使用 <code>@xref</code> 的引用	43
使用 <code>@xref</code> 的交叉引用	43
使用原始 <code>TeX</code> 的命令	123
索引, 自动为表格条目建立	71
所有者, FSF 作品的版权	15
数字符号, 插入	87
树状结构	34
双左角引号	92
首字母大写, 索引条目的	82
双字母名称, 索引的	84

T

Table of contents	19
Table of contents, after title page	227
Table of contents, for floats	75
Tables and lists, making	67
Tables with indexing	71
Tables, making multi-column	72
Tables, making two-column	70
Tag table	176
Tag table, in Info format	267
Tag table, making manually	252
Targets for cross-references, arbitrary	44
头部的结束行	14
头部的开始行	13
突出, 文本	48
特点, 印刷书籍或手册的	4
Template for a definition	105
<code>--test</code> , 用于 <code>install-info</code>	176
<code>--test</code> , for <code>install-info</code>	176
Testing for Texinfo commands	128
Tests, of Texinfo language	6
<code>TeX</code> 索引排序	142
<code>TeX</code> , 如何获取	142
<code>TeX</code> 输入的初始化文件	146
<code>TeX</code> 与 ‘#line’ 指令	138
<code>TeX</code> 徽标	94
<code>TeX</code> 输入初始化	146
<code>TeX</code> 命令, 正常使用	123
<code>TeX</code> commands, using ordinary	123

<code>TeX</code> index sorting	142
<code>TeX</code> input initialization	146
<code>TeX</code> logo	94
<code>TeX</code> , how to obtain	142
<code>TeX</code> and ‘#line’ directives	138
<code>texi2any</code>	148
<code>texi2any</code> 选项	148
<code>texi2any</code> , 作为参考实现	6
<code>texi2any</code> inside Emacs	246
<code>texi2any</code> options	148
<code>texi2any</code> , as reference implementation	6
<code>texi2any</code> , 在 Emacs 中调用	246
<code>texi2dvi</code> (shell 脚本)	142
<code>texi2dvi</code> (shell script)	142
<code>texi2html</code>	169
<code>texi2oldapi.texi</code> , 用于 <code>texi2html</code>	170
<code>texi2oldapi.texi</code> , for <code>texi2html</code>	170
<code>texi2pdf</code> (shell 脚本)	142
<code>texi2pdf</code> (shell script)	142
<code>texi-elements-by-size</code>	160
<code>texindex</code>	144
Texinfo 简介	2
Texinfo 模式	237
Texinfo 的历史	5
Texinfo 的拼写	2
Texinfo 的发音	2
Texinfo, 介绍	2
Texinfo 语言测试	6
Texinfo 文件结束	23
Texinfo 文档结构	27
Texinfo 文件的示例	234
Texinfo 文件的头部	11
Texinfo 的常见用途	2
Texinfo 实现的未来	6
Texinfo 文件的开头行	11
Texinfo 文件, 最小化	9
Texinfo 文件的第一行	11
Texinfo, 以及文学编程	145
Texinfo 的有限适用范围	2
Texinfo 命令, 定义新的	130
Texinfo 模式中的页面分隔符	240
Texinfo 命令, 对其进行测试	128
Texinfo 文件的章节结构, 显示	239
Texinfo 历史	5
Texinfo 前导	13
Texinfo 文件头部	11
Texinfo commands, defining new	130
Texinfo commands, testing for	128
Texinfo document structure	27
Texinfo file ending	23
Texinfo file header	11
Texinfo file minimum	9
Texinfo file sectioning structure, showing	239
Texinfo history	5
Texinfo language tests	6
Texinfo mode	237
Texinfo overview	2
Texinfo parsers, discouraging more	5

Texinfo Preamble	13	--transliterate-file-names	153
Texinfo printed book characteristics	4	Transliteration of 8-bit characters in HTML	
Texinfo XML 输出概览	4	cross-references	187
Texinfo XML output, overview	4	Tree representation of documents	6
Texinfo, and literate programming	145	tree representation, for debugging	157
Texinfo, introduction to	2	Tree structuring	34
texinfo.cnf 安装	146	提示	229
texinfo.cnf installation	146	调试, 使用 Info 格式进行	253
texinfo.dtd	4	调试, 使用 T _E X 格式化	254
texinfo.tex, 安装	146	调试 Texinfo 的结构	253
texinfo.tex, installing	146	调试文档, 采用树形表示	157
Texinfo 解析器, 不鼓励更多开发	5	特殊字符, 插入	86
Texinfo 印刷书籍的特点	4	特殊字符和符号, 插入	86
texiwebjr	145	特殊展示	74
Text width and height	264	Two ‘First’ Lines for @defn	107
Text, conditionally visible	121	Two letter names for indices	84
Text, marking up	48	Two named items for @table	71
Text::Unicode	162	Two part menu entry	32
Textual glyphs	94	Two-argument form of cross-references	40
图符, 文本用	94	图像	76
通过 texi2any 打印输出	154	缩放	77
þ	92	替代文本	77
<thead> HTML/DocBook 标签	72	在 Info 格式中	269
<thead> HTML/DocBook tag	72	图像的 Alt 属性	77
Thin space between number, dimension	91	图像格式	76
Thorn	92	txi-cc.tex	118
Three-argument form of cross-references	41	txixml2texi	4
Þ	92	特性, Texinfo 的, 适应	128
ti.twjr	145	通用句法约定	8
Tie-after accent	91	Typed functions	108
Tied space	103	Typed variables	111
Tilde accent	91	Typewriter font	57
time-stamp.el	234	跳转到其他 Info 文件的节点	32
<tip> DocBook tag	59		
Tips	229		
Title page	16		
bastard	17		
<title> HTML 标签	12		
<title> HTML tag	12		
条件, 复制 Texinfo	1		
推荐的键名	50		
条件可见的文本	121		
条件命令, 内联	123		
条件文本的可见性	121		
添加新的 Info 文件	171		
条件语句, 嵌套	128		
条目, 索引	80		
条目, 制作索引	82		
Top node	20		
Top node example	20		
Top node is first	26		
tp (数据类型) 索引	80		
tp (data type) index	80		
图片, 插入	76		
通配符	144		
通配符匹配	144		
--trace-includes	153		
Translating strings in output documents	166		
		U	
		Ugly black rectangles in hardcopy	147
		Umlaut accent	91
		Unbreakable space, fixed	102
		Unbreakable space, variable	103
		Uncluttered menu entry	32
		Undefined macros	131
		Underbar accent	91
		Underscore	91
		Underscore, breakpoint within @code	102
		undirected single quote	88
		Unicode 字符, 插入	99
		Unicode 和 T _E X	99
		Unicode 引号字符	92
		Unicode and T _E X	99
		Unicode character, inserting	99
		Unicode quotation characters	92
		Unicode::Collate	161
		Unique index entries	82
		Unique node names requirement	25
		Unnumbered float, creating	74
		Unprocessed text	9
		Unsplit file creation	257

Up node of Top node	26
Updating nodes and menus	241
Updating requirements	243
URI 语法, Info 的	4
URI syntax for Info	4
@url, 使用示例	46
@url, examples of using	46
URL,	
引用	45
标示	55
显示示例	46
examples of displaying	46
indicating	55
referring to	45
URL, 在 PDF 输出中着色	47
URL, PDF 输出	47
URL 的 PDF 输出	47
<URL...> 约定, 未使用	46
<URL...> convention, not used	46
URLs, coloring in PDF output	47
URLs, PDF output of	47
Usage tips	229
User input	50
User options, marking	110
User-defined Texinfo commands	130
Using Info-validate	256
Using Texinfo in general	2

V

-v	153
Validating a large file	256
Value of an expression, indicating	96
Variables, in typed languages	111
Variables, object-oriented	112
Verbatim copying license	236
Verbatim environment	61
Verbatim in-line text	51
Verbatim, include file	141
Verbatim, small	61
--verbose	153
--version, 用于install-info	176
Version control keywords, preventing expansion of	102
Version number, for install-info	176
--version, for install-info	176
--version, for texi2any	153
Versions of Texinfo, adapting to	128
Vertically holding text together	103
Visibility of conditional text	121
vr (变量)索引	80
vr (variable) index	80

W

@w, 用于空白项	68
@w, for blank items	68
<warning> DocBook tag	59
文本的图符	94
外部宏处理器	137
无编号的浮动内容, 创建	74
文本, 进行标记	48
文本宽度和高度	264
文本, 条件可见	121
不处理的文本	9
无衬线字体	57
文档标题, 指定	12
文档的抽象语法树表示法	6
文档的树表示法	6
文档的语法树表示法	6
文档结构, Texinfo 的	27
文档输入编码	119
文档许可	14
尺寸和图像大小	77
文档语言, 声明	118
文档字符串的翻译	118
文档字符串, 国际化	166
无方向单引号	88
White space in node name	26
Whitespace in macros	130
Whitespace, collapsed around continuations ...	106
Whitespace, controlling in conditionals	123
Whitespace, inserting	89
Whole manual, in Info format	266
Width of images	77
Width of text area	264
Widths, defining multitable column	72
Wildcards	144
文件的章节结构, 显示	239
文件结构概述, 展示	239
文件结构, 显示	239
文件记录器, 用于 TeX	143
文件结束	23
文件名冲突	12
文件名的长度	12
文件章节结构, 显示	239
word counting	157
Words and phrases, marking them	48
Writing a menu	30
Writing a @node line	24
Writing index entries	82
问题, 捕捉	253
为 Texinfo 定制 TeX	146
微调与断字	102
唯一的索引条目	82
唯一节点名称要求	25
位置, 菜单的	32
位置点, 在缓冲区中指示	98

X

下标和上标, 文本	93
新的 Info 文件, 将其列在 <code>dir</code> 文件中	171
<code>xdvi</code>	3
下点音符	91
小的字体	56
XeTeX	147
细空隙, 数字与度量单位之间的	91
下划线, <code>@code</code> 内的断点	102
许可, 打印	18
许可文本, 包含	15
许可证, 全开放复制	236
许可证, 逐字复制	236
序列, 点击的	99
<code>--xml</code>	153
XML, 包含原始内容	123
XML DocBook 输出, 概述	3
XML DocBook output, overview	3
XML Texinfo 输出, 概览	4
XML Texinfo output, overview	4
XML, including raw	123
条目列表	67
<code>--Xopt str</code>	153
XPM 文件格式	77
XPM image format	77
小书尺寸	263
显示公式	94
显示格式化	62
显示公式, 使用纯 TeX	123
姓氏, 全部大写	54
显示文件的章节结构	239
显示文件结构	240
新索引定义	84
新 Texinfo 命令, 定义	130
斜体字体	57
详细菜单	20
选项, <code>makeinfo</code> 的	148
选项, <code>texi2any</code> 的	148
选项, <code>texi2html</code> 的	169
选项相关的定制化变量	156
详细信息, 宏用法	132
下线音符	91
小号的大写字母字体	56
需要的页面剩余空间	103
匈牙利语元音变音符	91
XZ 压缩目录文件, 读取	174
XZ-compressed dir files, reading	174
限制, 对节点名称的	25
小字体示例	66
小字号逐字	61

Y

英镑符号	95
页边距, 不可控	264
英吋	77
右侧不对齐, 无填充	63
溢出, ‘ <code>hboxes</code> ’	146
页面, 从奇数页开始	258
移除 Texinfo 命令	157
约定, 句法	8
预定义名称, 索引的	84
Years, in copyright line	15
语法, 可选参数和重复参数	106
语法, <code>@</code> 命令的	205
与号, 插入	87, 92
引号, 德语	92
用户定义的 Texinfo 命令	130
引号, 法语	92
用户输入	50
语法详情, ‘ <code>#line</code> ’	138
用户选项, 标记	110
引号字符, 插入	88
引号字符(‘ <code>’</code>), 源文件中的	92
元键	51, 259
圆角矩形, 环绕文本	65
圆角框	65
邮件链接	55
右角引号	92
元键, 指定	50
硬拷贝, 打印	142
页眉	259
页面尺寸, 自定义	264
页面尺寸, 书籍的	263
页面更大或更小	265
要求, 包含文件	245
要求, 格式化	9
隐式创建指针, 使用 <code>texi2any</code>	24
原始 HTML	179
印刷书籍与手册的特点	4
原始文本输出	157
源文本中的多个破折号	8
源文本中的连字符, 连续两个或三个	8
源文本中的破折号	8
源文件格式	2
源文件, 所用字符	8
运行宏	131
原型行, 定义列宽度	72
引用	39, 59
元音变音符	91
语言代码	118
扬音符	91
左引号	92
用于浮动内容的标题	75
用于浮动内容列表的简短标题	75
右引号	92
语义化标签	2
引用其他 Info 文件	32
语言, 声明	118
引用使用 <code>@pxref</code>	44

引用, 使用@ref	43
抑扬音符	91
扬抑音符	91
引用, 用小字体	66
引用整本手册	43
右侧不对齐, 有填充	63
验证大文件	256
抑制缩进	64
脚注样式, ‘separate’	79

Z

整体手册, 用于 HTML 交叉引用	189
准备使用 T _E X	146
左侧不对齐, 无填充	63
组成部分, 菜单的	32
组成部分, 交叉引用的	39
主菜单	20
注册符号	95
最长节点, 查找	160
自动创建指针, 使用texi2any	24
自动插入节点及菜单	241
指定索引条目	80
自动为某些宏的逗号转义	131
折叠续行符周围的空白	106
最低要求, 格式化的	9
自定义页面尺寸	264
断开, 在一行中	101
字符, 基本输入	8
字符集, 声明	119
字符, 在节点名称中无效的	26
纵横比, 图像的	77
组合两个定义到一起	107
章节层级, 升高和降低	38
关键词扩展, 防止	102
直接格式化, Info 文件	252
章节划分	34
章节结构化	34
左角引号	92
章节, 逐一进行格式化	145
主控菜单	20
主控菜单的组成部分	20

展开宏	131
展开, 指示	96
字面方式, 包含文件	141
字母排序的 @ 命令列表	206
注释	8
着色的链接, 在 PDF 输出中	47
指示估值	96
注释, 在 CSS 文件中	180
字体, 多列表格的标题行的	73
字体大小, 减小	56
字体, 打印用	56
整体手册, Info 格式	266
字体, 索引的	84
粗体字体	57
中文	147
中文, 使用 XeTeX	147
最小化的 Texinfo 文件 (需求)	9
图符, 用于编程	96
重音, 独立	88
重音符号, 插入	91
左引号	92
自由软件	1
注意事项, 在使用宏时	132
摘要, 文档的	181
自由文档许可证, 全部	234
转义为 HTML	179
正则表达式, 用于‘#line’	138
制作标签表, 手动	252
纸张尺寸, A4	264
组 (在垂直方向上将文本保持整体)	103
制作打印手册	142
逐字复制许可证	236
阻止换行和分页	101
逐字环境	61
子子级别章节命令	37
制作交叉引用	39
制作列表和表格	67
逐字内联文本	51
逐字, 小字号	61
指针, 使用texi2any创建	24
指针验证	151
抑制	145

翻译说明

本文档是《The GNU Documentation Format》的简体中文版。在此，对翻译相关事项进行说明。

原文

原文为英文，来自于‘<https://git.savannah.gnu.org/git/texinfo.git>’，标签是‘texinfo-7.2’。

惯例

1. 使用法式空白（增加@frenchspacing on行）。
2. 对部分原文没有采用逐字逐句的对比翻译,而是总括其意思,转换为使用简体中文的用户的习惯表达方式。
3. 对变量名、选项名、命令名等保留字，不作翻译。
4. 对‘cp’索引，保留原文的索引条目，增加翻译后的中文索引条目。
5. 中文索引条目，不是仅从原文翻译，而是要根据中文特点进行重建。
6. 对翻译后的索引条目，如果是汉字开头，则增加排序键值(使用 @sortas 命令)。将索引条目中的汉字用拼音 (<http://www.moe.gov.cn/ewebeditor/uploadfile/2015/03/02/20150302165814246.pdf>)的首字母代替，并删除非数字、非字母的符号，从而生成排序键值。
7. 不翻译锚点名(@anchor)及节点名(@node)。原因是：中文节名配合锚点，并不能完全实现原文中节点名、锚点名的作用。
8. 翻译节点描述(@nodedescription)、章节标题(章节名)(@chapter、@section 等)。
9. 本译本中，大量使用‘参阅：@ref’或‘请参阅：@ref’，替代@pxref及@xref，原因有下面几点：
 - @pxref及@xref，在 HTML 输出中，会产生“see”及“See”，texi2any无法自动将其译成中文。
 - @pxref及@xref，在 PDF 输出中，texi2any自动将“see”及“See”译成中文“见”，不符合中文习惯。中文的习惯应是：“参阅：”、“请参阅：”、“参见：”、“请参见：”。（冒号，根据上下文可选。）
 - 在中文读者的眼中，“*note”及“*Note”只是一个修饰链接目标的标记，它与上下文之间不会产生语法关联，因此，在 Info 输出中出现‘请参阅 *Note ...’不会产生不适感。（请参阅：5.9 节[@ref], 43 页。）
10. 只对部分注释(由 @c 或 @ignore 命令定义)进行翻译；主要取决于：译文与原文相比，是否有利于理解。
11. 只对部分示例进行翻译；主要取决于：译文与原文相比，是否有利于展示。
12. 在个别译文后，又以注释方式附上了原文，表明译者对此段内容的译文的正确性没有把握。
13. 尽管建议 Texinfo 文档使用的全列宽(fill column)是 70 或 72 (参见：附录 B [Tips], 229 页)，但由于以下几个原因，这变得没有必要，甚至还有害处：
 - 在中文 Texinfo 源文件的一个句子中，如果存在换行，就会在 Info 及纯文本输出中产生一个多余的空格，这违反中文的排版惯例。

- 现代的编辑器都可设定列宽的，并可自动换行显示。对文档的全列宽不作限制，不会影响内容的显示及编辑。
- 现代的编辑器（甚至包括是较为古老的vim、Emacs、nano），都运行在 GUI 下，或仿真终端下（独立的专用终端设备已被淘汰），编辑区的长宽是随窗口大小而自动变化的，完全不会被限制在 80x25 或某个固定值上。本文作为新出现的文档，已不会有文档维护者会使用过时的专用终端设备。
- 如果仍限制文档的全列宽为 70 或 72，当现代的编辑器的列宽变为明显大于 80 时，屏幕空间会有很多浪费，变为小于 80 时，就会产生过多换行，让文本显示得参差不齐，观感很差。
- 如果仍限制文档的全列宽为 70 或 72，很多段落会被写到多个行上，这非常不得于面向行的文本处理器(如：sed、awk等)的操作，因为这样导致非常多的跨行处理。

因此，本文档没有文档全列宽限制。

14. 原文中一些词、短语、概念：

Free Software Foundation, Inc.

自由软件基金会股份有限公司。

preamble 前导。

chapter 章。

section 节，章节(章与节的统称)。

sectioning
章节划分，划分章节。

sectioning structure
章节结构。

sectioning level
章节级别。

sectioning command

section command

chapter structuring command

structuring command
章节划分命令，章节命令，章节结构化命令，结构化命令；不管是在原文还是译文中，这几个词实际上是同义。

sectioning tree
章节树。指由整个的章节作为节点的倒置树状结构。

sub- 子-。

node 节点。

top node

Top node 顶节点。

site 直译应为‘站点’，但实际含义经常是‘本机’或‘本地’。

split 分割。

no split 非分割，整体。

header	(HTML 输出的)页眉, (文件的) 头部。
footer	(HTML 输出的)页脚。
Page heading	
heading	(打印输出的)页眉。
footing	(打印输出的)页脚。
customization	定制化。尽管有建议将其译作“自定义”或“用户化”。
dimension	度量单位。该词在本手册中用在@dmn相关描述中。尽管直译应该是“量纲”, 但根据上下文, 应该译为“度量单位”。猜测这是原文作者用词有误, 当然也可能是译者水平不足, 但译作“度量单位”是符合中文常规的。‘量纲’, 应该是指一个物理量, 如: 长度、时长、质量等。
mark	标记, 为...作标记。
edition	版次。
version	版本。
copying	复制, 复制许可, 复制授权。
license	许可证。
Previous, Next, Up	作为导航栏指针名称时,含义是: 上一个、下一个、上一级。但由于它们是关键词, 在 Info 输出及 HTML 输出中, 都显示为英文, 所以本文不作翻译。
@-command	
@-commands	@ 命令。
verbatim modes	逐字模式。
menu entry	菜单条目, 菜单项。本文优先前者。
quotation	引用。(主要指摘录他人的话语或文字。)
cross reference	交叉引用。
reference	引用。在原文中, ‘reference’可认为是“cross reference”的省略写法, 所以“reference”也可理解为“(交叉) 引用”。这与“quotation”的“引用”是不同的, 要根据上下文理解。
float	浮动。不是动词, 是名词。
code point	码点。

`brace` 花括号，大括号。

`Square brackets`
方括号，中括号。

贡献

2025 年 11 月至今，由 YongNing Zhai(翟永宁) 翻译。